Python and Problem-Solving Reference

The Python language, a method for unseen problems, and the techniques behind NeetCode 150 and Blind 75

Written in ASD-STE100 Simplified Technical English. All examples are tested on Python 3.13.

Contents

1 How to use this document 3

I The Python language 5

2 Program structure 5
3 Names and assignment 8
4 Built-in types overview 11
5 Numbers 12
6 Bit operations 17
7 Booleans, None and comparisons 20
8 Control flow 22
9 Functions 26
10 Iterators and generators 31
11 Strings 32
12 Lists 38
13 Tuples 43
14 Ranges 44
15 Dictionaries 44
16 Sets 47
17 Comprehensions 49
18 Mutability and copies 50
19 The collections module 51
20 heapq: priority queues 54
21 bisect: binary search on sorted lists 57
22 Sorting 58
23 Built-in iteration functions 60
24 The itertools module 60
25 The functools and operator modules 62
26 The math module 62
27 The random module 63
28 Classes and objects 63
29 Exceptions 69
30 Recursion 71
31 Input and output 73
32 Performance 74
33 Common mistakes 76
34 Features by Python version 77

II Solving unseen problems 77

35 A method for unseen problems 77
36 Read the constraints 82
37 From brute force to fast: the optimization moves 85
38 Signal-to-technique table 93
39 Choose a data structure from the operations 102
40 When you are stuck 104
41 Pitfalls that cost points 106

42 Debug and verify a solution **131**
43 Final checklist **136**

III Techniques **136**
44 Hashing: counting, membership and canonical keys **136**
45 Prefix sums, running aggregates and offsets **148**
46 Two pointers **157**
47 Sliding window **170**
48 Stacks and monotonic stacks **181**
49 Binary search **191**
50 Linked lists **203**
51 Trees **217**
52 Tries **233**
53 Heaps and priority queues **244**
54 Backtracking **256**
55 Graph traversal: grids and implicit graphs **270**
56 Graph structure: ordering, cycles and connectivity **282**
57 Weighted and advanced graphs **295**
58 Dynamic programming: foundations and 1-D **305**
59 Dynamic programming: knapsack family **315**
60 Dynamic programming on strings and sequences **325**
61 Dynamic programming on grids, intervals, games and states **336**
62 Greedy **345**
63 Intervals **358**
64 Matrices and simulation **367**
65 Math and number manipulation **379**
66 Bit manipulation **392**
67 Design problems: combining data structures **400**

IV Retention **414**
68 Recall sheet **414**
69 Transfer drills **416**
A Answers to the transfer drills **428**
B Problem map: NeetCode 150 and Blind 75 **473**

1 How to use this document

1.1 What is in each part

- **Part I, The Python language.** Syntax, built-in types, the standard library and the behavior that interview code depends on. Use it as a reference when you forget a method or a rule.
- **Part II, Solving unseen problems.** A step-by-step method and how to read the constraints. The optimization moves that turn a brute force into a fast solution. A table from signals to techniques and a table that chooses a data structure from the operations. What to do when you are stuck. The pitfalls that cost points, how to debug and verify a solution, and a final checklist.
- **Part III, Techniques.** One section for each technique family. Each section gives the idea, the signals, a tested template, the variations, a checklist to adapt it, and the pitfalls. Each problem uses the most intuitive solution that fits the constraints. Its memory hook is a short sentence that brings back the whole solution. To find an algorithm by its name, use the table in Section 1.3.
- **Part IV, Retention.** A recall sheet with one row for each section of Part III. Transfer drills on problems that are not in NeetCode 150 or Blind 75. The answers are in the appendix.
- **Appendix.** The drill answers and a map of all NeetCode 150 and Blind 75 problems with their memory hooks.

1.2 How to study with this document

Knowledge of patterns fades fast without practice. Short, regular recall keeps it. Use this plan.

1. **Every day, 15 minutes.** Run the review script in `cheatsheet/retention` (command: `python3 review.py`). For each card, say the technique, the memory hook and the first steps before you show the answer. Grade yourself honestly. The script schedules the next review.
2. **For each card that you grade 1 or 2,** write the solution from its rebuild steps on the same day.
3. **Every week,** do one set of transfer drills from Part IV without help. Then read the answers.
4. **When you see a new problem,** follow the method in Part II. Look up the signals in the signal-to-technique table (Section 38).
5. **Before an assessment,** read these parts in this order:
   a. the recall sheet (Section 68, p. 414);
   b. the table of bug classes (Section 41.1, p. 106);
   c. the signal-to-technique table (Section 38, p. 93);
   d. the final checklist (Section 43, p. 136).

   Then read the full pitfall subsections only for the bug classes that you often make.

NOTE

Recall is the exercise. Reading a solution again feels productive, but it builds less memory than an attempt to remember it first.

1.3 Find an algorithm by its name

Many techniques have a well-known name. Use this table when you remember the name but not the section. Each entry gives the Part III section, its number and its page. In the PDF, the section name is a link. The section names are the short names of the signal-to-technique table.

Algorithm or named idea	Section in Part III
0/1 and unbounded knapsack	DP: knapsack family (59), p. 315
2-D prefix sums	Prefix sums (45), p. 148
Add without plus (XOR and carry)	Bit manipulation (66), p. 392
Bellman-Ford rounds (at most k edges)	Weighted and advanced graphs (57), p. 295
Binary search on the answer	Binary search (49), p. 191
Bipartite check (two-coloring)	Graph traversal (55), p. 270
Bitmask DP (used items)	DP: knapsack family (59), p. 315
Bucket sort by count	Hashing (44), p. 136
Cycle check with three colors	Graph structure (56), p. 282
Difference array	Prefix sums (45), p. 148
Dijkstra	Weighted and advanced graphs (57), p. 295

Algorithm or named idea	Section in Part III
Edit distance, longest common subsequence	DP on strings and sequences (60), p. 325
Euler path (Hierholzer)	Weighted and advanced graphs (57), p. 295
Expand around the center (palindromes)	DP on strings and sequences (60), p. 325
Fast exponentiation	Math and number manipulation (65), p. 379
Floyd cycle detection (fast and slow pointers)	Two pointers (46), p. 157; Linked lists (50), p. 203
Game DP (score difference)	DP on grids, intervals, games and states (61), p. 336
Interval DP (the element that acts last)	DP on grids, intervals, games and states (61), p. 336
Kadane (maximum subarray)	DP: foundations and 1-D (58), p. 305; Greedy (62), p. 345
Kahn topological sort	Graph structure (56), p. 282
K-way merge	Heaps and priority queues (53), p. 244; Linked lists (50), p. 203
Lazy deletion from a heap	Heaps and priority queues (53), p. 244
Longest increasing subsequence in $O(n \log n)$	Binary search (49), p. 191; DP: foundations and 1-D (58), p. 305
Lowest common ancestor	Trees (51), p. 217
LRU cache	Linked lists (50), p. 203; Design problems (67), p. 400
Minimum spanning tree (Prim, Kruskal)	Weighted and advanced graphs (57), p. 295
Monotonic deque (window maximum)	Sliding window (47), p. 170
Monotonic stack (next greater element)	Stacks and monotonic stacks (48), p. 181
Multi-source BFS	Graph traversal (55), p. 270
Quickselect	Heaps and priority queues (53), p. 244
Running median (two heaps)	Heaps and priority queues (53), p. 244; Design problems (67), p. 400
Serialize a tree (null markers)	Trees (51), p. 217
Sieve of primes	Math and number manipulation (65), p. 379
State-machine DP (cooldown)	DP on grids, intervals, games and states (61), p. 336
Sweep line over interval ends	Intervals (63), p. 358
Three-way partition (Sort Colors)	Two pointers (46), p. 157
Trie, binary trie for XOR	Tries (52), p. 233
Union-find	Graph structure (56), p. 282

1.4 Conventions

- Each topic has a short description and one or more examples.
- The comment `# -> value` shows the value of the expression on that line.
- The comment `# -> raises Error` shows that the line raises that exception.
- The comment `# prints: text` shows the text that the line writes to the screen.
- A CAUTION box tells you about a mistake that gives a wrong result or an error.
- A NOTE box gives more information.
- The tag [Python 3.10+] shows the first Python version that has the feature.

```
x = 7
x * 2          # -> 14
x / 0          # -> raises ZeroDivisionError
print("hi")    # prints: hi
```

NOTE

The script `check_examples.py` runs every example in this document. It compares each `# -> value` comment with the real result. Run it after you change an example.

1.5 Check the Python version of the judge

Many online judges use an old Python version. Some features in this document need a newer version. Run this line on the judge if you are not sure.

```
import sys
sys.version_info >= (3, 8)   # -> True
```

1.6 Find help inside Python

Most assessments let you run code. If you forget a method name, ask Python. `dir(x)` lists the attributes of `x`. `help(x)` shows the documentation.

```
[m for m in dir(list) if not m.startswith("_")]
# -> ['append', 'clear', 'copy', 'count', 'extend', 'index', 'insert', 'pop', 'remove', 'reverse', 'sort
    ']
[m for m in dir(dict) if not m.startswith("_")]
# -> ['clear', 'copy', 'fromkeys', 'get', 'items', 'keys', 'pop', 'popitem', 'setdefault', 'update', '
    values']
[m for m in dir(set) if not m.startswith("_")][:6]
# -> ['add', 'clear', 'copy', 'difference', 'difference_update', 'discard']
"removeprefix" in dir(str)        # -> True
len([m for m in dir(str) if not m.startswith("_")])   # -> 47
str.split.__doc__.splitlines()[0] # -> 'Return a list of the substrings in the string, using sep as the
    separator string.'
```

```
help(str.split)      # full documentation for one method
help(heapq)          # documentation for a module (after import heapq)
```

Part I

The Python language

2 Program structure

2.1 Indentation and blocks

Python uses indentation to make blocks. Use 4 spaces for each level. Do not mix tabs and spaces. A colon starts a block.

```
total = 0
for i in range(3):
    if i > 0:
        total += i
total   # -> 3
```

2.2 Statements and lines

Put one statement on each line. A semicolon puts two statements on one line. Use this only for short code.

```
a = 1; b = 2
a + b   # -> 3
```

A line continues inside parentheses, brackets and braces. A backslash also continues a line, but parentheses are safer.

```
total = (1 + 2 +
         3 + 4)
total   # -> 10
items = [
    "a",
    "b",     # a comma after the last element is permitted
```

```
]
len(items)   # -> 2
```

A long function header can also continue inside its parentheses.

```
def clamp(value: int,
          lo: int,
          hi: int) -> int:
    return max(lo, min(value, hi))
clamp(15, 0, 10)   # -> 10
clamp(-3, 0, 10)   # -> 0
```

Put a long condition in parentheses to split it over lines. A call also continues over lines, and a comma after the last argument is permitted. Give a long part of a condition a name to make the test clear.

```
grid, r, c = [[1, 0], [0, 1]], 1, 1
inside = 0 <= r < len(grid) and 0 <= c < len(grid[0])   # a named condition
hit = False
if (inside
        and grid[r][c] == 1
        and r == c):
    hit = True
hit                                     # -> True
best = min(4,
           2,
           7,
           )
best                                    # -> 2
```

2.3 Comments and docstrings

A `#` starts a comment. A string that is the first statement of a function body is a docstring.

```
def area(w, h):
    """Return the area of a rectangle."""
    return w * h     # multiply the two sides
area.__doc__        # -> 'Return the area of a rectangle.'
```

2.4 The pass statement

Use `pass` when a block must exist but must do nothing.

```
def todo():
    pass
todo() is None   # -> True
```

2.5 The main guard

Code under `if __name__ == "__main__":` runs only when you start the file directly. It does not run when another file imports the file.

```
def solve():
    ...

if __name__ == "__main__":
    solve()
```

2.6 The LeetCode solution format

LeetCode gives a class `Solution` with one method. Write the code in the body of that method. The first parameter of a method is `self`. The judge makes one object and calls the method. LeetCode compares the return value with

the expected answer.

```
from typing import List

class Solution:
    def twoSum(self, nums: List[int], target: int) -> List[int]:
        seen = {}                           # value: index
        for i, x in enumerate(nums):
            if target - x in seen:
                return [seen[target - x], i]
            seen[x] = i
        return []                           # not reached when the input has one answer

Solution().twoSum([2, 7, 11, 15], 9)       # -> [0, 1]
```

- LeetCode imports `List`, `Optional` and most standard modules for you. A local file needs the imports. Write the imports anyway. An extra import does no harm, and other judges need it.
- To test locally, call `Solution().method(arguments)`.
- Call a helper method of the class as `self.helper(...)`. A method calls itself as `self.method(...)`.
- A name that starts with `_` shows that the method is an internal helper. Python does not enforce this.
- LeetCode shows the node classes (`ListNode`, `TreeNode`) only as a comment. Define them yourself to run locally (Section 28.9).

```
from typing import Optional

class TreeNode:                             # LeetCode defines this class for you
    def __init__(self, val=0, left=None, right=None):
        self.val, self.left, self.right = val, left, right

class Solution:
    def maxDepth(self, root: Optional[TreeNode]) -> int:
        if not root:
            return 0
        return 1 + max(self.maxDepth(root.left), self.maxDepth(root.right))

    def isPalindrome(self, s: str) -> bool:
        t = self._clean(s)
        return t == t[::-1]

    def _clean(self, s: str) -> str:        # internal helper method
        """Keep letters and digits, in lowercase."""
        return "".join(c.lower() for c in s if c.isalnum())

Solution().maxDepth(TreeNode(1, TreeNode(2, TreeNode(3))))    # -> 3
Solution().isPalindrome("A man, a plan, a canal: Panama")     # -> True
```

Some problems give a function that the judge defines, for example `isBadVersion`. Write a small stub of that function to test locally.

```
BAD = 4
def isBadVersion(v: int) -> bool:           # stub; the judge gives the real function
    return v >= BAD

class Solution:
    def firstBadVersion(self, n: int) -> int:
        lo, hi = 1, n
        while lo < hi:
            mid = (lo + hi) // 2
            if isBadVersion(mid):
                hi = mid
            else:
                lo = mid + 1
```

```
        return lo

Solution().firstBadVersion(10)              # -> 4
```

CAUTION

A method with the return hint `-> None` must change its argument in place. The judge reads the argument after the call. An assignment to the parameter name makes a new local object. The caller does not see it. Use slice assignment (`nums[:] = ...`) or index assignment.

```
class Solution:
    def rotate(self, nums, k) -> None:
        k %= len(nums)
        nums = nums[-k:] + nums[:-k]        # wrong: rebinds the local name only
    def rotate_ok(self, nums, k) -> None:
        k %= len(nums)
        nums[:] = nums[-k:] + nums[:-k]     # correct: changes the caller's list
a = [1, 2, 3, 4, 5]
Solution().rotate(a, 2)
a                                           # -> [1, 2, 3, 4, 5]
Solution().rotate_ok(a, 2)
a                                           # -> [4, 5, 1, 2, 3]
```

3 Names and assignment

3.1 Names are references

A name refers to an object. Assignment does not copy the object. Two names can refer to the same object.

```
a = [1, 2]
b = a             # b and a refer to the same list
b.append(3)
a                 # -> [1, 2, 3]
a is b            # -> True
```

3.2 Multiple assignment and swap

Assign several names in one statement. `x, y = y, x` swaps two values.

```
x, y = 1, 2
x, y = y, x       # swap without a temporary name
(x, y)            # -> (2, 1)
a = b = 0         # two names, same value
(a, b)            # -> (0, 0)
```

CAUTION

`a = b = []` gives two names for one list. A change through `a` also shows through `b`.

3.3 Simultaneous assignment

In `a, b = x, y`, Python first evaluates the full right side. Then it assigns the targets from left to right. Use this form when each new value must read the old values. You do not need a temporary name.

```
a, b = 0, 1
for _ in range(10):
    a, b = b, a + b               # a + b uses the old a and the old b
a                                 # -> 55
prev2 = prev1 = 0
for x in [2, 7, 9, 3, 1]:         # rolling update with two states
    prev2, prev1 = prev1, max(prev1, prev2 + x)
```

```
prev1                           # -> 12
nums = [5, 6, 7]
nums[0], nums[2] = nums[2], nums[0]   # swap two list elements
nums                            # -> [7, 6, 5]
res = [0, 0]
carry, res[1] = divmod(17, 10)   # a target can be a list element
(carry, res)                    # -> (1, [0, 7])
class T:
    def __init__(self, left, right):
        self.left, self.right = left, right
t = T("a", "b")
t.left, t.right = t.right, t.left     # swap two attributes
(t.left, t.right)               # -> ('b', 'a')
```

CAUTION

Python assigns the targets one at a time. A later target can use a value that an earlier target already changed. Compute such an index first and store it in a name.

```
nums, i = [3, 1, 2], 0
nums[i], nums[nums[i] - 1] = nums[nums[i] - 1], nums[i]   # wrong
nums                            # -> [2, 3, 2]
nums, i = [3, 1, 2], 0
j = nums[i] - 1                 # compute the second index first
nums[i], nums[j] = nums[j], nums[i]
nums                            # -> [2, 1, 3]
```

In the wrong line, the second target `nums[nums[i] - 1]` reads `nums[i]` after the first target changed it.

3.4 Chained assignment

`a = b = value` binds all the names to one object. This is safe for an immutable value (a number, a string, `None`, a tuple). It is also correct when two names must share one object, for example a dummy head node.

```
lo = hi = 0
hi += 5                         # rebinds hi only; ints are immutable
(lo, hi)                        # -> (0, 5)
class Node:
    def __init__(self, val=0, next=None):
        self.val, self.next = val, next
dummy = tail = Node()           # one sentinel node with two names
for v in [4, 7]:
    tail.next = Node(v)
    tail = tail.next            # tail moves; dummy stays at the start
dummy.next.val, tail.val        # -> (4, 7)
```

3.5 Unpacking

Unpack a sequence into names. The number of names must match the number of elements. A starred name collects the remaining elements into a list.

```
first, second = [10, 20]
first             # -> 10
head, *rest = [1, 2, 3, 4]
rest              # -> [2, 3, 4]
*init, last = "abc"
(init, last)      # -> (['a', 'b'], 'c')
a, (b, c) = 1, (2, 3)    # nested unpacking
c                 # -> 3
x, y = [1, 2, 3]  # -> raises ValueError
```

Use _ for a value that you do not need.

```
_, count = ("ignored", 5)
count              # -> 5
```

Unpacking also works in a `for` header, and it can be nested. Use `*` in a list literal to join sequences. Use `**` in a dict literal to merge dicts. For a duplicate key, the later value wins.

```
pairs = [("a", 1), ("b", 2)]
[f"{k}{v}" for k, v in pairs]                    # -> ['a1', 'b2']
gas, cost = [1, 2, 3], [3, 2, 1]
[(i, g - c) for i, (g, c) in enumerate(zip(gas, cost))]   # -> [(0, -2), (1, 0), (2, 2)]
counts = {(0, 1): 3}
[(x, y, c) for (x, y), c in counts.items()]      # -> [(0, 1, 3)]
x, y, z = [7, 8, 9]                              # a list unpacks like a tuple
x, y = {"a", "b", "c"} - {"a"}                   # a set with two elements; the order is not fixed
sorted([x, y])                                   # -> ['b', 'c']
a, b = [1, 2], [3]
[*a, *b, 9]                                      # -> [1, 2, 3, 9]
{**{"k": 1}, **{"k": 2, "z": 0}}                 # -> {'k': 2, 'z': 0}
first, *_ = "xyz"
first                                            # -> 'x'
```

3.6 Augmented assignment

`x += 1` adds 1 to `x`. Python has no `x++` operator.

```
x = 10
x += 5; x -= 3; x *= 2; x //= 4; x %= 4; x **= 3
x                  # -> 8
```

CAUTION

For a list, `a += b` changes the list in place. `a = a + b` makes a new list. The difference is important when another name refers to the same list.

```
a = [1]; alias = a
a += [2]
alias              # -> [1, 2]
a = a + [3]
alias              # -> [1, 2]
```

Each binary operator has an augmented form: `+= -= *= /= //= **=`, `%=`, `&= |= ^=`, `<<=` and `>>=`. The target can be a name, a list element or a dict value. Python computes the full right side first.

```
flags = 0
flags |= 1 << 3              # set bit 3
flags ^= 1                   # flip bit 0
flags >>= 1                  # shift right by one bit
flags                        # -> 4
n = 12
n &= n - 1                   # same as n = n & (n - 1)
n                            # -> 8
found = False
found |= 3 > 2               # also works on bool values
found                        # -> True
cnt = [0, 0]
cnt[1] += 1                  # on a list element
cnt                          # -> [0, 1]
d = {"a": 1}
d["a"] += 1                  # on a dict value
d["b"] += 1                  # -> raises KeyError
```

CAUTION

`d[k] += 1` on a plain dict needs the key `k`. Use `d[k] = d.get(k, 0) + 1`, a `defaultdict(int)` or a `Counter`.

3.7 The walrus operator

[Python 3.8+] `:=` assigns a value inside an expression.

```
data = [5, 8, 1]
if (n := len(data)) > 2:
    msg = f"{n} items"
msg               # -> '3 items'
```

3.8 Delete a name

`del` removes a name, a list element or a dictionary key.

```
x = 5
del x
x                 # -> raises NameError
```

4 Built-in types overview

Type	Example	Mutable	Hashable	Ordered
`int`	`42`	no	yes	–
`float`	`3.14`	no	yes	–
`bool`	`True`	no	yes	–
`NoneType`	`None`	no	yes	–
`str`	`"abc"`	no	yes	yes
`tuple`	`(1, 2)`	no	yes (if elements are)	yes
`list`	`[1, 2]`	yes	no	yes
`dict`	`{"a": 1}`	yes	no	insertion order
`set`	`{1, 2}`	yes	no	no
`frozenset`	`frozenset({1})`	no	yes	no
`range`	`range(5)`	no	yes	yes
`bytes`	`b"ab"`	no	yes	yes

Only hashable objects can be dictionary keys or set elements.

```
type(3)             # -> int
type(3.0)           # -> float
type("3")           # -> str
isinstance(True, int)     # -> True
isinstance(5, (int, float))   # -> True
{[1, 2]: "x"}       # -> raises TypeError
{(1, 2): "x"}       # -> {(1, 2): 'x'}
```

4.1 Type conversion

```
int("42")           # -> 42
int(" 42 ")         # -> 42
int(3.99)           # -> 3
int(-3.99)          # -> -3
float("2.5")        # -> 2.5
str(42)             # -> '42'
list("abc")         # -> ['a', 'b', 'c']
tuple([1, 2])       # -> (1, 2)
```

```
set([1, 1, 2])      # -> {1, 2}
bool(0), bool("0")  # -> (False, True)
int("4.5")          # -> raises ValueError
int("-11"), int("+5"), int("007")   # -> (-11, 5, 7)
int("12\n")         # -> 12
int("")             # -> raises ValueError
float(3)            # -> 3.0
tuple([1, 2]) in {(1, 2)}           # -> True
```

`int()` accepts a sign, leading zeros and whitespace around the digits. Convert a list to a tuple to use it as a dict key or a set element.

CAUTION

`int()` on a float truncates toward zero. It does not round.

5 Numbers

5.1 Integers

Integers have no size limit. Integer overflow does not occur.

```
2 ** 100              # -> 1267650600228229401496703205376
10 ** 18 + 1          # -> 1000000000000000001
1_000_000             # -> 1000000
0b1010, 0o17, 0xff    # -> (10, 15, 255)
import sys
sys.maxsize           # -> 9223372036854775807
INF = float("inf")    # a sentinel that is larger than every number
min(INF, 10**30)      # -> 1000000000000000000000000000000
```

NOTE

Use `float("inf")` or `math.inf` as an initial value for a minimum. `sys.maxsize` is not the largest integer. Python integers can be larger.

Arithmetic has no limit, but conversion between `int` and `str` has one [Python 3.11+]. `str()` and `int()` raise `ValueError` for a number with more than 4300 digits. Late patch releases of 3.7 to 3.10 have the same limit. `sys.set_int_max_str_digits(0)` removes it.

```
import sys
len(str(10**4299))                # -> 4300
str(10**4300)                     # -> raises ValueError
sys.set_int_max_str_digits(0)     # 0 means no limit
len(str(10**4300))                # -> 4301
```

Because integers do not overflow, `(lo + hi) // 2` is always safe in Python. In C or Java, `lo + hi` can overflow, so code from those languages writes `lo + (hi - lo) // 2`. Both forms give the same result.

```
lo, hi = 2**62, 2**62 + 10
(lo + hi) // 2                    # -> 4611686018427387909
lo + (hi - lo) // 2               # -> 4611686018427387909
```

5.2 Floats

A float is a 64-bit IEEE 754 value. It has about 15 to 17 significant digits.

```
0.1 + 0.2             # -> 0.30000000000000004
0.1 + 0.2 == 0.3      # -> False
1e9                   # -> 1000000000.0
float("inf") > 10**100    # -> True
float("-inf") < -10**100  # -> True
```

```
float("nan") == float("nan")   # -> False
2 ** 0.5               # -> 1.4142135623730951
```

CAUTION

Do not compare floats with `==`. Use `math.isclose` or a tolerance. Use integers when the problem lets you.

```
import math
math.isclose(0.1 + 0.2, 0.3)            # -> True
abs((0.1 + 0.2) - 0.3) < 1e-9            # -> True
```

5.3 Infinity and sentinel values

Use `float("inf")` (or `math.inf`) as the start value of a minimum, and as a mark for "not reached". Infinity compares correctly with every int. Use `float("-inf")` as the start value of a maximum, or as a mark for an impossible state.

```
INF = float("inf")
dist = [INF] * 3
dist[0] = 0
min(dist)                          # -> 0
5 < INF, -INF < -10**100           # -> (True, True)
max(float("-inf"), 3)              # -> 3
type(max(float("-inf"), 3))        # -> int
INF + 1 == INF                     # -> True
-INF + 7                           # -> -inf
INF - INF                          # -> nan
0 * INF                            # -> nan
int(INF)                           # -> raises OverflowError
d = INF
-1 if d == INF else d              # -> -1
```

CAUTION

A result that can stay infinite is a float. Convert it before you return it, for example `return -1 if d == INF else d`. `int(float("inf"))` raises `OverflowError`.

An integer sentinel keeps every value an int. Use a value that no real answer can reach, for example `amount + 1` for a count of coins or `2**31 - 1` for a 32-bit limit. Use `-1` or `None` for "not found".

```
amount = 11
dp = [0] + [amount + 1] * amount     # amount + 1 means not reachable
dp[3]                                # -> 12
BIG = 2**31 - 1
min(BIG, 7)                          # -> 7
```

5.4 Mix int and float

An int and a float with the same value are equal, and they are the same dict key. An operation on an int and a float gives a float.

```
3 == 3.0                           # -> True
{3: "x"}[3.0]                      # -> 'x'
type(6 / 2), 6 / 2                 # -> (float, 3.0)
(2 + 3) / 2                        # -> 2.5
7.0 // 2                           # -> 3.0
(2.0).is_integer()                 # -> True
1e18 == 10**18                     # -> True
float(2**53 + 1) == 2**53          # -> True
10**20 // 3                        # -> 3333333333333333333
int(10**20 / 3)                    # -> 3333333333333331968
float(10**400)                     # -> raises OverflowError
```

```
idx = 4 / 2                    # 2.0, a float
[1, 2, 3][idx]                 # -> raises TypeError
[1, 2, 3][4 // 2]              # -> 3
range(6 / 2)                   # -> raises TypeError
```

CAUTION

`/` always gives a float, also when the division is exact. A float cannot be a list index or a `range` argument. A float has only 53 bits of precision, so it loses digits of a big integer. Use `//` for integer results.

5.5 Arithmetic operators

Operator	Meaning	Example
`+ - *`	add, subtract, multiply	`7 * 2` is `14`
`/`	true division, result is always a float	`7 / 2` is `3.5`
`//`	floor division	`7 // 2` is `3`
`%`	remainder (modulo)	`7 % 2` is `1`
`**`	power	`2 ** 3` is `8`
`-x`	negation	`-(3)` is `-3`

```
7 / 2        # -> 3.5
6 / 2        # -> 3.0
7 // 2       # -> 3
7 % 2        # -> 1
2 ** 10      # -> 1024
2 ** -1      # -> 0.5
```

5.6 Division and modulo with negative numbers

`//` rounds toward negative infinity. The result of `%` has the same sign as the divisor. This is different from C, C++ and Java.

```
-7 // 2      # -> -4
-7 % 2       # -> 1
7 // -2      # -> -4
7 % -2       # -> -1
int(-7 / 2)  # -> -3
```

NOTE

Use `int(a / b)` to truncate toward zero like C. For very large integers, the float division loses precision. Use `a // b` if `a` and `b` have the same sign. Use `-(-a // b)` if they have different signs.

Use `%` to wrap an index around. The result is never negative for a positive divisor.

```
(0 - 1) % 4      # -> 3
(3 + 1) % 4      # -> 0
```

5.7 Ceiling division without floats

```
a, b = 7, 2
-(-a // b)       # -> 4
(a + b - 1) // b # -> 4
-(-7 // -2), (7 + -2 - 1) // -2   # -> (-3, -2)
```

`-(-a // b)` is correct for all signs. `(a + b - 1) // b` is correct only when `b > 0`.

5.8 Number functions

```
abs(-5)            # -> 5
divmod(17, 5)      # -> (3, 2)
divmod(-17, 5)     # -> (-4, 3)
pow(2, 10)         # -> 1024
pow(2, 10, 1000)   # -> 24
pow(3, -1, 7)      # -> 5
max(3, 9, 4)       # -> 9
min([3, 9, 4])     # -> 3
sum([1, 2, 3])     # -> 6
```

`pow(a, b, m)` computes a^b mod m fast. `pow(a, -1, m)` gives the modular inverse [Python 3.8+].

Fast exponentiation uses the bits of the exponent. Each step squares the base. When the lowest bit is 1, the step multiplies the result by the base. The loop needs $O(\log n)$ multiplications.

```
def power(x, n):
    if n < 0:
        x, n = 1 / x, -n
    res = 1.0
    while n:                        # int truthiness: stops when n is 0
        if n & 1:                   # the lowest bit of n is 1
            res *= x
        x *= x
        n >>= 1                     # drop the lowest bit
    return res
power(2.0, 10), power(2.0, -2)     # -> (1024.0, 0.25)
```

5.9 Modular arithmetic

Many problems ask for the answer modulo $10^9 + 7$. Take the remainder after each addition and multiplication. This keeps the numbers small and fast. In Python, `(a - b) % MOD` is never negative.

```
MOD = 10**9 + 7
a, b = 10**9, 10**9 + 5
(a * b) % MOD                      # -> 14
(3 - 5) % MOD                      # -> 1000000005
ways = 1
for _ in range(100):
    ways = ways * 2 % MOD          # * and % have the same precedence; they run left to right
ways == pow(2, 100, MOD)           # -> True
pow(2, MOD - 2, MOD) * 2 % MOD     # -> 1
type(1e9 + 7)                      # -> float
```

CAUTION

`1e9 + 7` is a float. A float modulus gives float results and loses precision. Write `10**9 + 7`.

`pow(a, MOD - 2, MOD)` is the inverse of `a` when `MOD` is prime. Use it to divide modulo `MOD`.

5.10 Rounding

`round()` uses banker's rounding. A value exactly half-way goes to the nearest even number.

```
round(2.5)         # -> 2
round(3.5)         # -> 4
round(-2.5)        # -> -2
round(2.675, 2)    # -> 2.67
round(1234, -2)    # -> 1200
import math
math.floor(2.7), math.ceil(2.1), math.trunc(-2.7)   # -> (2, 3, -2)
math.floor(-2.5)   # -> -3
math.floor(2.5 + 0.5)   # -> 3
```

CAUTION

`round(2.675, 2)` gives `2.67` because `2.675` has no exact float value. To round half up for a positive number, use `math.floor(x + 0.5)`.

5.11 Base conversion

```
bin(10), oct(10), hex(255)   # -> ('0b1010', '0o12', '0xff')
bin(10)[2:]                  # -> '1010'
int("1010", 2)               # -> 10
int("ff", 16)                # -> 255
int("z", 36)                 # -> 35
format(10, "b")              # -> '1010'
format(5, "08b")             # -> '00000101'
format(255, "x")             # -> 'ff'
```

5.12 Digits of an integer

`divmod(x, 10)` splits off the last digit. `while x:` stops when `x` becomes 0. For a negative number, keep the sign apart and work with `abs(x)`.

```
x, digits = 4207, []
while x:
    x, d = divmod(x, 10)
    digits.append(d)
digits                           # -> [7, 0, 2, 4]
[int(c) for c in str(4207)]      # -> [4, 2, 0, 7]
sum(map(int, str(4207)))         # -> 13
len(str(4207))                   # -> 4
def reverse_int(x):
    sign = -1 if x < 0 else 1
    x, r = abs(x), 0
    while x:
        x, d = divmod(x, 10)
        r = r * 10 + d
    return sign * r
reverse_int(-120)                # -> -21
int(str(1200)[::-1])             # -> 21
r = reverse_int(1534236469)
r if -2**31 <= r <= 2**31 - 1 else 0   # -> 0
```

CAUTION

`divmod(-17, 10)` is `(-2, 3)`, not `(-1, -7)`. Digit loops on negative numbers give wrong digits. Take `abs(x)` first.

5.13 Exact arithmetic

Use `fractions.Fraction` for exact rational values. Use `decimal.Decimal` for exact decimal values.

```
from fractions import Fraction
Fraction(1, 3) + Fraction(1, 6)     # -> Fraction(1, 2)
from decimal import Decimal
Decimal("0.1") + Decimal("0.2")     # -> Decimal('0.3')
```

5.14 Operator precedence

From highest to lowest. Use parentheses when you are not sure.

Operators	Notes
`() [] . calls`	grouping, index, attribute
`**`	right to left: `2 ** 3 ** 2` is `2 ** 9`
`-x +x ~x`	unary
`* / // %`	
`+ -`	
`<< >>`	shifts
`&`	bitwise AND
`^`	bitwise XOR
`|`	bitwise OR
`== != < > <= >= in not in is is not`	comparisons
`not`	
`and`	
`or`	
`x if c else y`	conditional expression
`lambda`	
`:=`	assignment expression

```
-2 ** 2          # -> -4
(-2) ** 2        # -> 4
2 ** 3 ** 2      # -> 512
1 + 2 * 3        # -> 7
5 & 3 == 1       # -> True
```

NOTE

In Python, bitwise operators bind tighter than comparisons. `x & 1 == 1` means `(x & 1) == 1`. This is the opposite of C.

Arithmetic binds tighter than shifts, and shifts bind tighter than `&`, `^` and `|`. Put parentheses around each bit expression inside a sum.

```
1 << 2 + 1           # -> 8
i = 6
(i >> 1) + (i & 1)   # -> 3
i >> 1 + i & 1       # -> 0
mask, j = 0b101, 2
mask >> j & 1        # -> 1
mask | 1 << 1        # -> 7
not 1 == 2           # -> True
x = 5
x > 0 and x % 2 == 1 # -> True
```

`i >> 1 + i & 1` means `(i >> (1 + i)) & 1`. `mask >> j & 1` means `(mask >> j) & 1`. `not a == b` means `not (a == b)`. Comparisons bind tighter than `and` and `or`.

6 Bit operations

6.1 Operators

```
a, b = 0b1100, 0b1010
bin(a & b)       # -> '0b1000'
bin(a | b)       # -> '0b1110'
bin(a ^ b)       # -> '0b110'
~5               # -> -6
1 << 4           # -> 16
40 >> 2          # -> 10
-8 >> 1          # -> -4
```

`~x` equals `-x - 1`. Negative integers act as if they have infinite leading 1 bits.

6.2 Common bit tricks

```
x = 0b10110
x & 1               # -> 0
(x >> 2) & 1        # -> 1
bin(x | (1 << 0))   # -> '0b10111'
bin(x & ~(1 << 1))  # -> '0b10100'
bin(x ^ (1 << 4))   # -> '0b110'
bin(x & (x - 1))    # -> '0b10100'
x & -x              # -> 2
x.bit_length()      # -> 5
x.bit_count()       # -> 3
bin(x).count("1")   # -> 3
n = 64
n > 0 and n & (n - 1) == 0    # -> True
```

- `x & 1`: lowest bit. The value is 1 when `x` is odd.
- `x & (x - 1)`: removes the lowest set bit.
- `x & -x`: keeps only the lowest set bit.
- `x.bit_count()` [Python 3.10+]: number of set bits. On old versions, use `bin(x).count("1")`.
- `x.bit_length()`: number of bits that you need to write `x`.

```
n = 4
(1 << n) - 1                    # -> 15
m, bits = 0b10110, []
while m:                        # visit each set bit, lowest first
    low = m & -m
    bits.append(low.bit_length() - 1)
    m ^= low
bits                            # -> [1, 2, 4]
x, count = 0b10110, 0
while x:
    x &= x - 1                  # remove the lowest set bit
    count += 1
count                           # -> 3
ans = [0] * 6                   # set-bit counts for 0 to 5
for i in range(1, 6):
    ans[i] = ans[i >> 1] + (i & 1)
ans                             # -> [0, 1, 1, 2, 1, 2]
seen = 0
for c in "abca":                # a bitmask as a small set of letters
    seen |= 1 << (ord(c) - ord("a"))
bin(seen), seen >> 2 & 1        # -> ('0b111', 1)
```

- `(1 << n) - 1`: a mask with the `n` lowest bits set.
- `x &= x - 1` in a loop counts the set bits. The loop runs one time for each set bit.
- `ans[i >> 1] + (i & 1)`: the bit count of `i` from the bit count of `i // 2`.
- `1 << (ord(c) - ord("a"))`: one bit for each letter. A mask of 26 bits replaces a set of letters.

6.3 Reverse the bits

```
def reverse_bits(n):              # reverse the 32 low bits of n
    res = 0
    for _ in range(32):
        res = (res << 1) | (n & 1)    # move the lowest bit of n into res
        n >>= 1
    return res
reverse_bits(0b1011)              # -> 3489660928
```

```
format(reverse_bits(0b1011), "032b")[:4]   # -> '1101'
int(format(0b1011, "032b")[::-1], 2)       # -> 3489660928
```

The last line uses strings: `format(n, "032b")` gives 32 binary digits, and `int(s, 2)` reads them back.

6.4 Emulate 32-bit integers

Python integers never overflow. A negative integer acts as if it has infinite leading 1 bits. When a problem expects 32-bit behavior, keep the low 32 bits with `& 0xFFFFFFFF`. Then convert the result back to a signed value.

```
INT_MAX, INT_MIN = 2**31 - 1, -2**31
MASK = 0xFFFFFFFF
def to_signed(x):                 # read the low 32 bits as a signed int
    x &= MASK
    return x if x <= INT_MAX else x - (1 << 32)
to_signed(INT_MAX + 1)            # -> -2147483648
to_signed(-1)                     # -> -1
hex(-1 & MASK)                    # -> '0xffffffff'
def add(a, b):                    # addition with bit operations only
    a, b = a & MASK, b & MASK
    while b:
        a, b = (a ^ b) & MASK, ((a & b) << 1) & MASK
    return to_signed(a)
add(-7, 3), add(5, 9)             # -> (-4, 14)
-1 ^ 5                            # -> -6
INT_MIN <= 2**31 <= INT_MAX       # -> False
```

CAUTION

Without the mask, the carry loop above does not stop for some negative inputs. The carry moves to the left forever, because a negative Python integer has no highest bit.

6.5 Subsets as bitmasks

Bit `i` of a mask shows if element `i` is in the subset.

```
items = ["a", "b", "c"]
subsets = []
for mask in range(1 << len(items)):
    subsets.append([items[i] for i in range(len(items)) if mask >> i & 1])
len(subsets)          # -> 8
subsets[5]            # -> ['a', 'c']
mask = 0b101
subs = []
s = mask
while s:              # every non-empty submask of mask
    subs.append(s)
    s = (s - 1) & mask
subs                  # -> [5, 4, 1]
```

6.6 XOR properties

`x ^ x` is 0. `x ^ 0` is `x`. XOR is commutative and associative.

```
from functools import reduce
from operator import xor
reduce(xor, [4, 1, 2, 1, 2])   # -> 4
```

7 Booleans, None and comparisons

7.1 True, False and None

`bool` is a subclass of `int`. `True` equals 1 and `False` equals 0. `None` shows that a value is not there.

```
True + True          # -> 2
sum([True, False, True])   # -> 2
def f():
    pass
f() is None          # -> True
```

7.2 Truthiness

These values are false: `False`, `None`, `0`, `0.0`, `""`, `[]`, `()`, `{}`, `set()`, `range(0)`. All other values are true.

```
bool([]), bool([0]), bool(""), bool(" ")   # -> (False, True, False, True)
items = []
"empty" if not items else "full"           # -> 'empty'
```

CAUTION

Do not use `if x:` when 0 or an empty string is a valid value. Use `if x is not None:`.

An integer is true when it is not 0. Use this for parity tests and for loops that count down. An object of your own class is always true, unless the class defines `__bool__` or `__len__`. A tree node with the value 0 is true. `None` is false.

```
total = 7
"odd" if total % 2 else "even"    # -> 'odd'
n, steps = 3, 0
while n:                          # stops when n is 0
    n -= 1
    steps += 1
steps                             # -> 3
class Node:
    def __init__(self, val, next=None):
        self.val, self.next = val, next
node = Node(0)
bool(node), bool(node.val)        # -> (True, False)
head, length = Node(5, Node(0)), 0
while head:                       # stops at None, the end of the list
    length += 1
    head = head.next
length                            # -> 2
```

CAUTION

`if node:` tests that the node exists. `if node.val:` tests the value, and it is false for the value 0. Do not mix the two tests.

7.3 and, or, not

`and` and `or` stop as soon as they know the result. They return one of the operands, not always a `bool`.

```
0 or 5          # -> 5
3 or 5          # -> 3
0 and 5         # -> 0
3 and 5         # -> 5
not 3           # -> False
name = ""
name or "anonymous"     # -> 'anonymous'
```

Use short-circuit evaluation to protect an index.

```
a = []
len(a) > 0 and a[0] == 1   # -> False
stack = []
not stack or stack[-1] < 5 # -> True
```

The left test runs first. If `stack` is empty, Python does not evaluate `stack[-1]`. The same guard protects a chain of attributes. In `fast and fast.next`, Python reads `fast.next` only when `fast` is not `None`.

```
class Node:
    def __init__(self, val, next=None):
        self.val, self.next = val, next
head = Node(1, Node(2, Node(3, Node(4, Node(5)))))
slow = fast = head
while fast and fast.next:  # fast moves two steps, slow moves one step
    slow, fast = slow.next, fast.next.next
slow.val                   # -> 3
```

`x or y` returns the first true operand. Use it to pick the part that is not empty, for example the rest of a linked list after a merge (`tail.next = l1 or l2`). A loop condition can combine several kinds of values.

```
l1, l2 = None, [3, 4]
l1 or l2                   # -> [3, 4]
None or None               # -> None
carry, steps = 1, 0
while l1 or l2 or carry:   # None, a list and an int in one test
    l2, carry = None, 0
    steps += 1
steps                      # -> 1
```

7.4 Comparison operators

Comparisons can be chained. `a < b < c` means `a < b and b < c`.

```
r, R = 2, 5
0 <= r < R            # -> True
1 < 3 > 2             # -> True
"apple" < "banana"    # -> True
"Z" < "a"             # -> True
"10" < "9"            # -> True
[1, 2] < [1, 3]       # -> True
(1, 2) < (1, 2, 0)    # -> True
1 < "a"               # -> raises TypeError
```

Strings, lists and tuples compare element by element (lexicographic order). Strings compare by Unicode code point, so all uppercase letters come before lowercase letters.

CAUTION

A tuple never equals a list, also when the elements are the same. Convert one side before you compare.

```
(1, 2) == [1, 2]              # -> False
list((1, 2)) == [1, 2]        # -> True
best, target = [2, 5], (2, 5)
best == list(target)          # -> True
```

7.5 Identity and equality

`==` compares values. `is` compares identity (the same object).

```
a = [1, 2]; b = [1, 2]
a == b        # -> True
a is b        # -> False
x = None
x is None     # -> True
p, q = None, None
p is q        # -> True
```

For two tree nodes, `p is q` is true only when both names refer to one node, or when both are `None`.

CAUTION

Use `is` only to compare with `None` (and `True`/`False` singletons if necessary). Do not use `is` to compare numbers or strings. The result depends on the implementation.

7.6 Membership

```
3 in [1, 2, 3]         # -> True
"b" in "abc"           # -> True
"bc" in "abcd"         # -> True
"a" in {"a": 1}        # -> True
1 in {"a": 1}          # -> False
4 not in {1, 2}        # -> True
5 in range(0, 10, 5)   # -> True
rows, r = 10, 9
r in (0, rows - 1)     # -> True
"*" in {"+", "-", "*", "/"}   # -> True
None in {(0, 0), (1, 2)}      # -> False
```

`in` on a dict checks the keys. `in` is $O(n)$ for a list and $O(1)$ on average for a set or dict. A small tuple literal is a short way to test for one of a few values.

7.7 Return a boolean

A comparison already gives a `bool`. Return it directly. `sum` of booleans counts the true values, because `True` equals 1.

```
def is_balanced(depth):
    return depth == 0          # not: if depth == 0: return True, else: return False
is_balanced(0)                 # -> True
sum(x > 2 for x in [1, 3, 5]) # -> 2
```

8 Control flow

8.1 if, elif, else

```
def sign(x):
    if x > 0:
        return 1
    elif x < 0:
        return -1
    else:
        return 0
[sign(-4), sign(0), sign(9)]   # -> [-1, 0, 1]
```

8.2 Conditional expression

`a if condition else b` gives `a` when the condition is true. Otherwise it gives `b`.

```
x = 7
"odd" if x % 2 else "even"     # -> 'odd'
```

8.3 for loops

`for` iterates over any iterable: list, string, tuple, dict, set, range, file, generator.

```
out = []
for ch in "hey":
    out.append(ch.upper())
out                    # -> ['H', 'E', 'Y']
```

Loop over a tuple literal of offsets to visit the neighbors of a grid cell. Unpack each pair in the header.

```
r, c, R, C = 0, 1, 2, 3
[(r + dr, c + dc) for dr, dc in ((1, 0), (-1, 0), (0, 1), (0, -1))
 if 0 <= r + dr < R and 0 <= c + dc < C]     # -> [(1, 1), (0, 2), (0, 0)]
```

8.4 range

`range(stop)`, `range(start, stop)`, `range(start, stop, step)`. The stop value is not included.

```
list(range(5))            # -> [0, 1, 2, 3, 4]
list(range(2, 5))         # -> [2, 3, 4]
list(range(0, 10, 3))     # -> [0, 3, 6, 9]
list(range(5, 0, -1))     # -> [5, 4, 3, 2, 1]
list(range(4, -1, -1))    # -> [4, 3, 2, 1, 0]
list(range(3, 3))         # -> []
```

To include the end value `n`, write `range(1, n + 1)`. A range with a start at or after its stop is empty, so the loop body does not run. Use `max(0, i - k)` as the start to clip a window at index 0.

```
n = 3
list(range(1, n + 1))             # -> [1, 2, 3]
list(range(5, 2))                 # -> []
i, k = 2, 4
list(range(max(0, i - k), i))     # -> [0, 1]
```

`range()` reads its arguments one time, before the loop starts. Elements that the loop adds to a list do not make the loop longer. A level-by-level BFS uses `for _ in range(len(q))` for this reason: it processes only the nodes of the current level.

```
q = [1]
for _ in range(len(q)):   # len(q) is 1 here, and it stays 1 for this loop
    q.append(q[-1] + 1)
q                         # -> [1, 2]
```

8.5 while loops

```
n, steps = 27, 0
while n != 1:
    n = n // 2 if n % 2 == 0 else 3 * n + 1
    steps += 1
steps                     # -> 111
```

8.6 while True (do-while) and a manual index

Python has no do-while loop. Use `while True:` with `break` or `return` as the exit. The body then runs at least one time. Use a `while` loop with a manual index when the step size changes.

```
n, steps = 19, 0
while True:                   # the body runs at least one time
    n = sum(int(d) ** 2 for d in str(n))
    steps += 1
    if n == 1 or steps > 50:
        break
(n, steps)                    # -> (1, 4)
s, i, runs = "aaabcc", 0, []
while i < len(s):             # run-length encoding: each step skips one run
    j = i
    while j < len(s) and s[j] == s[i]:
        j += 1
    runs.append(f"{s[i]}{j - i}")
    i = j
"".join(runs)                 # -> 'a3b1c2'
```

8.7 break, continue and loop else

`break` stops the loop. `continue` goes to the next iteration. The `else` block of a loop runs only when the loop ends without `break`.

```
def first_even(nums):
    for x in nums:
        if x % 2 == 0:
            result = x
            break
    else:
        result = None         # no break: no even number
    return result
first_even([3, 5, 8, 9])      # -> 8
first_even([3, 5])            # -> None
```

To stop two nested loops, put them in a function and use `return`.

```
def find(grid, target):
    for r, row in enumerate(grid):
        for c, v in enumerate(row):
            if v == target:
                return (r, c)
    return None
find([[1, 2], [3, 4]], 3)     # -> (1, 0)
```

8.8 enumerate, zip, reversed

`enumerate` gives pairs of (index, value). `zip` walks several sequences in parallel. `reversed` reads a sequence from the end. All three return iterators and make no copy.

```
list(enumerate("ab"))              # -> [(0, 'a'), (1, 'b')]
list(enumerate("ab", start=1))     # -> [(1, 'a'), (2, 'b')]
list(enumerate("ab", 1))           # -> [(1, 'a'), (2, 'b')]
list(zip([1, 2, 3], "ab"))         # -> [(1, 'a'), (2, 'b')]
list(reversed([1, 2, 3]))          # -> [3, 2, 1]
a = [5, 7, 9]
[y - x for x, y in zip(a, a[1:])]  # -> [2, 2]
```

`zip` stops at the shortest input. `zip(..., strict=True)` [Python 3.10+] raises `ValueError` if the lengths are different.

```
list(zip([1, 2], [3], strict=True))   # -> raises ValueError
```

8.9 Loop over a sequence backwards

```
a = [1, 2, 3]
[a[i] for i in range(len(a) - 1, -1, -1)]   # -> [3, 2, 1]
[x for x in reversed(a)]                    # -> [3, 2, 1]
a[::-1]                                     # -> [3, 2, 1]
```

8.10 Change a collection while you loop over it

CAUTION

Do not add or remove elements of a list, set or dict while you iterate over it. Iterate over a copy, or build a new collection.

```
d = {"a": 1, "b": 2}
for k in d:
    d[k + k] = 0          # -> raises RuntimeError
nums = [1, 2, 3, 4]
nums = [x for x in nums if x % 2 == 0]
nums                      # -> [2, 4]
d = {"a": 1, "b": 2}
for k in list(d):         # iterate over a copy of the keys
    if d[k] == 1:
        del d[k]
d                         # -> {'b': 2}
```

8.11 match statement

[Python 3.10+] `match` compares a value with patterns. The first pattern that matches runs.

```
def describe(cmd):
    match cmd:
        case "F" | "B":
            return "move"
        case "L" | "R":
            return "turn"
        case (x, y):
            return f"point {x},{y}"
        case {"op": op}:
            return f"op {op}"
        case [first, *rest] if len(rest) > 1:
            return f"list from {first}"
        case _:
            return "unknown"
[describe("F"), describe("R"), describe((1, 2))]   # -> ['move', 'turn', 'point 1,2']
[describe({"op": "add"}), describe([7, 8, 9]), describe(3)]   # -> ['op add', 'list from 7', 'unknown']
```

NOTE

A bare name in a `case` captures the value. It does not compare with a variable. Use a dotted name (for example `Color.RED`) or a literal to compare.

9 Functions

9.1 Define and call

A function without a `return` statement returns `None`.

```
def add(a, b):
    return a + b
add(2, 3)             # -> 5
def nothing():
    x = 1
nothing() is None     # -> True
```

9.2 Return more than one value

Return a tuple. Unpack it at the call.

```
def min_max(nums):
    return min(nums), max(nums)
lo, hi = min_max([4, 1, 9])
(lo, hi)              # -> (1, 9)
```

9.3 Default arguments

```
def greet(name, greeting="hello"):
    return f"{greeting} {name}"
greet("ana")               # -> 'hello ana'
greet("ana", "hi")         # -> 'hi ana'
greet(greeting="yo", name="bo")   # -> 'yo bo'
```

CAUTION

Python evaluates a default value one time, when it defines the function. All calls share a mutable default (list, dict, set). Use `None` and make a new object in the body.

```
def bad(x, acc=[]):
    acc.append(x)
    return acc
bad(1); bad(2)        # -> [1, 2]
def good(x, acc=None):
    if acc is None:
        acc = []
    acc.append(x)
    return acc
good(1); good(2)      # -> [2]
```

9.4 Positional, keyword, *args and **kwargs

- `*args` collects extra positional arguments into a tuple.
- `**kwargs` collects extra keyword arguments into a dict.
- Parameters after `*` are keyword-only.
- Parameters before `/` are positional-only [Python 3.8+].

```
def f(a, b=2, *args, c, d=4, **kwargs):
    return a, b, args, c, d, kwargs
f(1, c=3)                  # -> (1, 2, (), 3, 4, {})
f(1, 5, 6, 7, c=3, z=9)    # -> (1, 5, (6, 7), 3, 4, {'z': 9})
def g(x, /, y):
    return x + y
g(1, y=2)                  # -> 3
g(x=1, y=2)                # -> raises TypeError
```

9.5 Unpack arguments at the call

`*seq` in a call passes the elements of a sequence as positional arguments. `**d` passes the items of a dict as keyword arguments.

```
def add3(a, b, c):
    return a + b + c
nums = [1, 2, 3]
add3(*nums)               # -> 6
opts = {"a": 1, "b": 2, "c": 3}
add3(**opts)              # -> 6
print(*[1, 2, 3])         # prints: 1 2 3
print(*[1, 2, 3], sep=",")   # prints: 1,2,3
```

9.6 Lambda functions

A lambda is a small function with one expression. Use it for `key=` arguments.

```
square = lambda x: x * x
square(4)                        # -> 16
pairs = [(1, "b"), (2, "a")]
sorted(pairs, key=lambda p: p[1])   # -> [(2, 'a'), (1, 'b')]
```

9.7 Functions are objects

You can store a function in a variable, a list or a dict. You can pass it to another function.

```
ops = {"+": lambda a, b: a + b, "*": lambda a, b: a * b}
ops["*"](3, 4)                   # -> 12
def apply_twice(f, x):
    return f(f(x))
apply_twice(lambda v: v + 3, 1) # -> 7
```

9.8 Scope: LEGB

Python looks for a name in this order: Local, Enclosing function, Global, Built-in.

```
x = "global"
def outer():
    x = "enclosing"
    def inner():
        return x          # finds the enclosing x
    return inner()
outer()                   # -> 'enclosing'
```

9.9 global and nonlocal

An assignment to a name inside a function makes the name local. To assign to a name of an enclosing function, declare it `nonlocal`. To assign to a module name, declare it `global`.

```
def counter():
    count = 0
    def bump():
        nonlocal count
        count += 1
    bump(); bump()
    return count
counter()                 # -> 2
```

```
def broken():
    count = 0
    def bump():
        count += 1        # count is local here, but it has no value yet
    bump()
broken()                  # -> raises UnboundLocalError
```

```
calls = 0
def tick():
    global calls          # assign to the module-level name
    calls += 1
tick(); tick()
calls                     # -> 2
```

NOTE

You do not need `nonlocal` to change a list or dict in place (`res.append(x)`, `d[k] = v`). You need it only to assign a new object to the name.

```
def collect():
    res = []
    def visit(x):
        res.append(x)     # no nonlocal needed: res is not reassigned
    visit(1); visit(2)
    return res
collect()                 # -> [1, 2]
```

9.10 Closures and late binding

A closure remembers the variables of the enclosing function. It reads the variable when it runs, not when you create it.

```
fs = [lambda: i for i in range(3)]
[f() for f in fs]         # -> [2, 2, 2]
fs = [lambda i=i: i for i in range(3)]   # bind the value now
[f() for f in fs]         # -> [0, 1, 2]
```

9.11 Nested helper functions

Define a recursive helper inside the method. The helper reads the variables of the method directly (a closure). It can change a list, a set or a dict of the method without `nonlocal`. It needs `nonlocal` only when it assigns a new value to the name.

```
from typing import List

class Solution:
    def subsets(self, nums: List[int]) -> List[List[int]]:
        res, path = [], []
        def dfs(i: int) -> None:          # type hints on the helper are optional
            if i == len(nums):
                res.append(path[:])       # append a copy of the path
                return
            path.append(nums[i])          # take nums[i]
            dfs(i + 1)
            path.pop()                    # undo the choice
            dfs(i + 1)                    # skip nums[i]
        dfs(0)
        return res

    def longest_run(self, s: str) -> int:
```

```
        best = 0
        def update(length: int) -> None:
            nonlocal best                # best gets a new value, so nonlocal is necessary
            best = max(best, length)
        run = 0
        for i, ch in enumerate(s):
            run = run + 1 if i and s[i - 1] == ch else 1
            update(run)
        return best

Solution().subsets([1, 2])               # -> [[1, 2], [1], [2], []]
Solution().longest_run("abbbcc")         # -> 3
```

Put `@cache` on a nested helper. Each call of the method then makes a new helper with an empty cache. The cache does not keep results from an earlier input.

```
from functools import cache

class Solution:
    def climbStairs(self, n: int) -> int:
        @cache
        def ways(i: int) -> int:         # number of ways to reach step i
            if i <= 1:
                return 1
            return ways(i - 1) + ways(i - 2)
        return ways(n)

Solution().climbStairs(10)               # -> 89
Solution().climbStairs(3)                # -> 3
```

9.12 Recursion

```
def fact(n):
    return 1 if n <= 1 else n * fact(n - 1)
fact(10)                  # -> 3628800
import sys
sys.getrecursionlimit()   # -> 1000
```

CAUTION

The default recursion limit is 1000 frames. A deeper recursion raises `RecursionError`. Raise the limit with `sys.setrecursionlimit(10**6)`, or use a loop and an explicit stack. Section 30 gives more information.

9.13 Decorators

A decorator is a function that takes a function and returns a new function. `@name` above `def` applies it.

```
import functools
def logged(f):
    @functools.wraps(f)           # keeps the name and docstring of f
    def wrapper(*args, **kwargs):
        wrapper.calls += 1
        return f(*args, **kwargs)
    wrapper.calls = 0
    return wrapper
@logged
def double(x):
    return 2 * x
double(3); double(4)              # -> 8
double.calls                      # -> 2
double.__name__                   # -> 'double'
```

9.14 Memoization with functools.cache

`@cache` stores the result for each argument tuple. The arguments must be hashable.

```
from functools import cache
@cache
def fib(n):
    return n if n < 2 else fib(n - 1) + fib(n - 2)
fib(90)                  # -> 2880067194370816120
fib.cache_info().currsize # -> 91
fib.cache_clear()
```

NOTE

`@cache` [Python 3.9+] is the same as `@lru_cache(maxsize=None)`. On Python 3.8, use `@lru_cache(maxsize=None)`. Convert a list argument to a tuple before the call. A recursive `@cache` function can fail at a depth of a few thousand (Section 30).

9.15 Type hints

Python does not check type hints when the program runs. They are documentation.

```
from typing import Optional
def find(nums: list[int], target: int) -> Optional[int]:
    return nums.index(target) if target in nums else None
find([4, 5], 5)          # -> 1
find([4, 5], "x")        # -> None
```

NOTE

`list[int]` works on Python 3.9 and later. On old versions, use `from typing import List` and `List[int]`. LeetCode writes `Optional[TreeNode]` for a value that can be `None`.

9.15.1 Type hints in LeetCode signatures

LeetCode signatures use the `typing` forms. Section 2.6 shows the full format of a solution.

`typing` form (all versions)	Built-in form	Meaning
`List[int]`	`list[int]` [Python 3.9+]	list of int
`List[List[str]]`	`list[list[str]]`	2D grid of strings
`Dict[str, int]`	`dict[str, int]`	dict from str to int
`Set[int]`	`set[int]`	set of int
`Tuple[int, int]`	`tuple[int, int]`	pair of int
`Optional[TreeNode]`	`TreeNode | None` [Python 3.10+]	a node or `None`
`-> None`	`-> None`	the method changes its argument in place

```
from typing import List, Optional, Dict
def count_rows(grid: List[List[str]], k: int = 0) -> Dict[str, int]:
    return {"rows": len(grid)}
count_rows([["a"], ["b"]])              # -> {'rows': 2}
def stats(xs: list[int] | None = None) -> tuple[int, int]:   # X | None needs 3.10
    xs = xs or []
    return len(xs), sum(xs)
stats([1, 2])                           # -> (2, 3)
count_rows.__annotations__["k"]         # -> int
def same(node: "TreeNode") -> Optional["TreeNode"]:   # TreeNode is not defined yet
    return node
same(None) is None                      # -> True
def copy(head: "Optional[Node]") -> "Optional[Node]":  # no import of Optional is necessary
    return head
```

```
copy(None) is None                      # -> True
```

A hint in quotes is a forward reference. Python stores the string and does not evaluate it. Use it when the class is defined later in the file, or inside its own class body. LeetCode uses this form, for example `'Node'`, in some problems.

10 Iterators and generators

10.1 The iterator protocol

`iter(x)` gets an iterator. `next(it)` gets the next element. At the end, `next` raises `StopIteration`, or returns the default value if you give one.

```
it = iter([10, 20])
next(it)              # -> 10
next(it)              # -> 20
next(it, "done")      # -> 'done'
next(it)              # -> raises StopIteration
```

10.2 Share one iterator between recursive calls

An iterator remembers its position. Create it one time and read it from all recursive calls (through a closure or an argument). Each `next()` call takes the next token. You do not need an index variable.

```
def build(tokens):                      # preorder tokens; "N" marks an empty child
    it = iter(tokens)
    def rec():
        v = next(it)
        if v == "N":
            return None
        return (int(v), rec(), rec())     # (value, left, right)
    return rec()
build("1 2 N N 3 N N".split())            # -> (1, (2, None, None), (3, None, None))
```

10.3 Generator functions

A function with `yield` returns a generator. The generator makes values one at a time, when you ask.

```
def countdown(n):
    while n > 0:
        yield n
        n -= 1
list(countdown(3))    # -> [3, 2, 1]
def chain_all(*seqs):
    for s in seqs:
        yield from s
list(chain_all([1, 2], "ab"))   # -> [1, 2, 'a', 'b']
```

A generator can call itself with `yield from`. Use this for a lazy tree traversal. The caller can stop early, and the generator then does no more work. The recursion depth equals the height of the tree.

```
def inorder(t):                   # t is (value, left, right) or None
    if t:
        yield from inorder(t[1])
        yield t[0]
        yield from inorder(t[2])
tree = (2, (1, None, None), (3, None, None))
list(inorder(tree))               # -> [1, 2, 3]
it = inorder(tree)
next(it), next(it)                # -> (1, 2)
```

10.4 Generator expressions

A generator expression is like a list comprehension in parentheses. It does not build a list.

```
sum(x * x for x in range(4))        # -> 14
any(x > 2 for x in [1, 5])          # -> True
max(len(w) for w in ["a", "abc"])   # -> 3
```

10.5 Iterators run one time

CAUTION

A generator, `map`, `filter`, `zip` and file object give their values one time only. A second loop gets nothing. Convert to a list if you need the values again.

```
squares = (x * x for x in range(3))
list(squares)     # -> [0, 1, 4]
list(squares)     # -> []
z = zip([1, 2], [3, 4])
list(z); list(z)  # -> []
```

10.6 First element that matches

```
nums = [3, 8, 5, 10]
next(x for x in nums if x % 2 == 0)         # -> 8
next((x for x in nums if x > 100), None)    # -> None
```

11 Strings

11.1 String literals

```
'single' == "single"          # -> True
"it's"                        # -> "it's"
'say "hi"'                    # -> 'say "hi"'
"line1\nline2".count("\n")    # -> 1
len("tab\there")              # -> 8
r"C:\new"                     # -> 'C:\\new'
len(r"\n")                    # -> 2
"""two
lines"""                      # -> 'two\nlines'
"ab" "cd"                     # -> 'abcd'
```

A raw string (`r"..."`) does not process backslash escapes. Use it for regular expressions.

11.2 Strings are immutable

You cannot change a character in place. Build a new string.

```
s = "cat"
s[0] = "b"                    # -> raises TypeError
"b" + s[1:]                   # -> 'bat'
chars = list(s)
chars[0] = "h"
"".join(chars)                # -> 'hat'
code, i = "0000", 2
code[:i] + "9" + code[i + 1:] # -> '0090'
```

`s[:i] + ch + s[i + 1:]` makes a copy with one changed character. It takes $O(n)$ time. For many changes, convert the string to a list, change the list, and join it at the end.

11.3 Indexing and slicing

```
s = "python"
s[0], s[-1]           # -> ('p', 'n')
s[1:4]                # -> 'yth'
s[:2], s[2:]          # -> ('py', 'thon')
s[::-1]               # -> 'nohtyp'
s[::2]                # -> 'pto'
s[10:]                # -> ''
s[10]                 # -> raises IndexError
```

A slice never raises `IndexError`. It clips the bounds to the string.

11.4 Concatenation and repetition

```
"ab" + "cd"           # -> 'abcd'
"ab" * 3              # -> 'ababab'
"-" * 5               # -> '-----'
"n=" + str(5)         # -> 'n=5'
"n=" + 5              # -> raises TypeError
"." * 2 + "Q" + "." * 1   # -> '..Q.'
"ab" * 0              # -> ''
```

CAUTION

`s += c` in a loop can take $O(n^2)$ time. Collect the parts in a list. Then use `"".join(parts)` one time.

```
parts = []
for i in range(5):
    parts.append(str(i))
"".join(parts)        # -> '01234'
",".join(["a", "b"])  # -> 'a,b'
" ".join(map(str, [1, 2, 3]))   # -> '1 2 3'
",".join([1, 2])      # -> raises TypeError
```

`join` accepts only strings. Convert numbers with `map(str, ...)` first.

11.5 Find and count

```
s = "banana"
"nan" in s            # -> True
s.find("an")          # -> 1
s.rfind("an")         # -> 3
s.find("x")           # -> -1
s.index("an")         # -> 1
s.index("x")          # -> raises ValueError
s.count("a")          # -> 3
s.count("ana")        # -> 1
s.startswith("ba"), s.endswith("na")   # -> (True, True)
s.startswith(("x", "b"))               # -> True
```

`count` counts occurrences that do not overlap. To count overlapping occurrences, test each start position.

```
"aaaa".count("aa")                                  # -> 2
sum("aaaa".startswith("aa", i) for i in range(4))   # -> 3
```

`find`, `index`, `count` and `startswith` accept a start and an end position. The search starts at the start position, and the result is still an index in the full string.

```
"banana".find("a", 2)                # -> 3
"banana".find("a", 2, 3)             # -> -1
def decode(s):                       # each word is stored as length, "#", word
    res, i = [], 0
    while i < len(s):
        j = s.index("#", i)          # the next "#" at or after position i
        n = int(s[i:j])
        res.append(s[j + 1:j + 1 + n])
        i = j + 1 + n
    return res
encoded = "".join(f"{len(w)}#{w}" for w in ["a#b", "", "xyz"])
encoded                              # -> '3#a#b0#3#xyz'
decode(encoded)                      # -> ['a#b', '', 'xyz']
grid = ["..S", "T.."]
[(r, row.find("S")) for r, row in enumerate(grid) if "S" in row]   # -> [(0, 2)]
```

The length prefix makes the encoding safe for every character, also for the separator inside a word.

11.6 Change case

```
"Hello World".lower()        # -> 'hello world'
"Hello World".upper()        # -> 'HELLO WORLD'
"hello world".title()        # -> 'Hello World'
"hello world".capitalize()   # -> 'Hello world'
"Hello".swapcase()           # -> 'hELLO'
"HeLLo".casefold() == "hello"  # -> True
```

11.7 Split and join

```
"a b  c".split()             # -> ['a', 'b', 'c']
"a b  c".split(" ")          # -> ['a', 'b', '', 'c']
"a,b,,c".split(",")          # -> ['a', 'b', '', 'c']
"a,b,c".split(",", 1)        # -> ['a', 'b,c']
"a,b,c".rsplit(",", 1)       # -> ['a,b', 'c']
"l1\nl2\r\nl3".splitlines()  # -> ['l1', 'l2', 'l3']
"key=val=x".partition("=")   # -> ('key', '=', 'val=x')
"key=val=x".rpartition("=")  # -> ('key=val', '=', 'x')
"-".join("abc")              # -> 'a-b-c'
```

CAUTION

`split()` without an argument splits on any run of whitespace and ignores leading and trailing whitespace. `split(" ")` splits on each single space and keeps empty strings.

11.8 Strip, remove, replace

```
"  hi  ".strip()             # -> 'hi'
"  hi  ".lstrip()            # -> 'hi  '
"xxhixx".strip("x")          # -> 'hi'
"0042".lstrip("0")           # -> '42'
"test_case.py".removesuffix(".py")   # -> 'test_case'
"test_case".removeprefix("test_")    # -> 'case'
"a-b-c".replace("-", "+")    # -> 'a+b+c'
"a-b-c".replace("-", "", 1)  # -> 'ab-c'
```

CAUTION

`strip("abc")` removes any of the characters a, b, c from both ends. It does not remove the substring `"abc"`. Use `removeprefix` / `removesuffix` [Python 3.9+] for a substring.

```
"cabbage".strip("abc")       # -> 'ge'
"000".lstrip("0")            # -> ''
"000".lstrip("0") or "0"     # -> '0'
"0120".lstrip("0") or "0"    # -> '120'
```

CAUTION

`lstrip("0")` removes all the zeros of `"0"` and `"000"`. Add `or "0"` when the result must be a number.

11.9 Character tests

```
"123".isdigit(), "12a".isdigit()       # -> (True, False)
"abc".isalpha(), "ab1".isalnum()       # -> (True, True)
"  ".isspace(), "".isspace()           # -> (True, False)
"abc".islower(), "ABC".isupper()       # -> (True, True)
"-5".isdigit()                         # -> False
"--5".lstrip("-").isdigit()            # -> True
"--5".removeprefix("-").isdigit()      # -> False
"\u00b2".isdigit()                     # -> True
int("\u00b2")                          # -> raises ValueError
```

NOTE

`isdigit()` is false for `"-5"` and `"1.5"`. It is true for some non-ASCII digits, for example the superscript two, that `int()` rejects. To test for an integer, use `try: int(s)` and catch `ValueError`. For ASCII input, `s.removeprefix("-").isdigit()` [Python 3.9+] also works. Do not use `lstrip("-")`: it accepts `"-5"`.

11.10 Characters and codes

```
ord("a"), ord("A"), ord("0")   # -> (97, 65, 48)
chr(98)                        # -> 'b'
ord("c") - ord("a")            # -> 2
chr(ord("a") + 25)             # -> 'z'
[chr(ord("a") + i) for i in range(3)]   # -> ['a', 'b', 'c']
int("7") == ord("7") - ord("0")         # -> True
```

Use `ord(c) - ord("a")` to map a lowercase letter to 0 to 25. Use it as an index in a list of 26 counts.

```
counts = [0] * 26
for c in "hello":
    counts[ord(c) - ord("a")] += 1
counts[ord("l") - ord("a")]    # -> 2
```

Characters compare by code point. Use a chained comparison to test a range. Use `%` 26 to wrap around the alphabet.

```
c = "q"
"a" <= c <= "z"                # -> True
"0" <= "7" <= "9"              # -> True
def shift(c, k):               # move a lowercase letter k places, wrap after z
    return chr(ord("a") + (ord(c) - ord("a") + k) % 26)
shift("y", 3), shift("b", -3)  # -> ('b', 'y')
```

11.11 Padding and alignment

`zfill` adds zeros on the left and keeps a sign in front. `ljust`, `rjust` and `center` pad to a width with a fill character. The default fill character is a space.

```
"7".zfill(3)          # -> '007'
"-7".zfill(3)         # -> '-07'
"ab".rjust(5)         # -> '   ab'
"ab".ljust(5, ".")    # -> 'ab...'
"ab".center(6, "*")   # -> '**ab**'
```

11.12 translate

`str.maketrans(old, new)` makes a table that maps each character of `old` to the character at the same position in `new`. A third argument lists the characters to delete. `s.translate(table)` applies the table.

```
table = str.maketrans("abc", "xyz")
"aabbcc".translate(table)                    # -> 'xxyyzz'
"hello, world!".translate(str.maketrans("", "", ",!"))   # -> 'hello world'
```

11.13 The string module

```
import string
string.ascii_lowercase     # -> 'abcdefghijklmnopqrstuvwxyz'
string.ascii_uppercase[:3] # -> 'ABC'
string.digits              # -> '0123456789'
string.ascii_letters[-1]   # -> 'Z'
"!" in string.punctuation  # -> True
```

11.14 f-strings

Put an expression in braces inside an `f"..."` string. After a colon, add a format specification.

```
name, n, x = "ana", 42, 3.14159
f"{name} has {n} points"   # -> 'ana has 42 points'
f"{x:.2f}"                 # -> '3.14'
f"{n:5d}|"                 # -> '   42|'
f"{n:<5}|"                 # -> '42   |'
f"{n:^6}|"                 # -> '  42  |'
f"{n:*>6}"                 # -> '****42'
f"{n:05d}"                 # -> '00042'
f"{n:+d}"                  # -> '+42'
f"{1234567:,}"             # -> '1,234,567'
f"{1234567:_}"             # -> '1_234_567'
f"{0.256:.1%}"             # -> '25.6%'
f"{1e-5:.2e}"              # -> '1.00e-05'
f"{255:b} {255:x} {255:X} {255:o}"   # -> '11111111 ff FF 377'
f"{5:08b}"                 # -> '00000101'
f"{name!r}"                # -> "'ana'"
f"{n=}"                    # -> 'n=42'
f"{{literal braces}}"      # -> '{literal braces}'
width = 6
f"{n:>{width}}"            # -> '    42'
```

Format specification order: `[fill][align][sign][0][width][,][.precision][type]`. Align: `<` left, `>` right, `^` center. `f"{x=}"` [Python 3.8+] shows the name and the value.

11.15 Other formatting methods

```
"{} + {} = {}".format(1, 2, 3)   # -> '1 + 2 = 3'
"{0}{1}{0}".format("a", "b")     # -> 'aba'
"%d items, %.1f%%" % (3, 99.5)   # -> '3 items, 99.5%'
format(3.14159, ".3f")           # -> '3.142'
str(3.0), repr("a")              # -> ('3.0', "'a'")
```

11.16 String comparison and sorting

```
sorted(["banana", "apple", "Cherry"])                  # -> ['Cherry', 'apple', 'banana']
sorted(["banana", "apple", "Cherry"], key=str.lower)   # -> ['apple', 'banana', 'Cherry']
sorted(["bb", "a", "ccc"], key=len)                    # -> ['a', 'bb', 'ccc']
"".join(sorted("dcba"))                                # -> 'abcd'
max(["apple", "fig", "banana"], key=len)               # -> 'banana'
```

Without a key, all uppercase letters sort before all lowercase letters. Use `key=str.lower` to ignore case.

11.17 Common string idioms

```
s = "racecar"
s == s[::-1]                          # -> True
sorted("listen") == sorted("silent") # -> True
from collections import Counter
Counter("hello").most_common(1)      # -> [('l', 2)]
" ".join(reversed("the sky is blue".split()))   # -> 'blue is sky the'
"".join(c for c in "a1b2" if c.isdigit())        # -> '12'
[s[i:i + 3] for i in range(len(s) - 2)][:3]      # -> ['rac', 'ace', 'cec']
```

11.18 String method summary

All string methods return a new string (or another value). They never change the original string.

Task	Methods
Search	`in`, `find`, `rfind`, `index`, `rindex`, `count`, `startswith`, `endswith`
Change case	`lower`, `upper`, `title`, `capitalize`, `swapcase`, `casefold`
Test characters	`isdigit`, `isdecimal`, `isnumeric`, `isalpha`, `isalnum`, `isspace`, `islower`, `isupper`, `isascii`
Split and join	`split`, `rsplit`, `splitlines`, `partition`, `rpartition`, `join`
Trim and pad	`strip`, `lstrip`, `rstrip`, `removeprefix`, `removesuffix`, `zfill`, `ljust`, `rjust`, `center`
Replace	`replace`, `translate`, `str.maketrans`, `expandtabs`
Format	`format`, `format_map`, f-strings
Bytes	`encode` (and `bytes.decode`)

```
"banana".rindex("a")                    # -> 5
"12".isdecimal(), "x".isascii()         # -> (True, True)
"ab\n".rstrip("\n")                     # -> 'ab'
"a\tb".expandtabs(4)                    # -> 'a   b'
"{a}-{b}".format_map({"a": 1, "b": 2})  # -> '1-2'
"hi".encode(), b"hi".decode()           # -> (b'hi', 'hi')
```

11.19 Regular expressions

Use the `re` module for patterns. Use a raw string for the pattern.

```
import re
re.findall(r"\d+", "a12b345")              # -> ['12', '345']
re.sub(r"\s+", " ", "a   b \t c")          # -> 'a b c'
re.split(r"[,;]", "a,b;c")                 # -> ['a', 'b', 'c']
m = re.match(r"(\w+)@(\w+)\.com", "ana@mail.com")
m.group(1), m.group(2)                     # -> ('ana', 'mail')
bool(re.fullmatch(r"[a-z]+", "abc"))       # -> True
re.search(r"\d", "abc") is None            # -> True
```

`match` checks at the start of the string. `search` checks anywhere. `fullmatch` checks the full string.

12 Lists

12.1 Create a list

```
[]                          # -> []
[1, "a", None]              # -> [1, 'a', None]
list("abc")                 # -> ['a', 'b', 'c']
list(range(3))              # -> [0, 1, 2]
[0] * 4                     # -> [0, 0, 0, 0]
[x * x for x in range(4)]   # -> [0, 1, 4, 9]
[-1] * 3                    # -> [-1, -1, -1]
[False] * 2                 # -> [False, False]
[float("inf")] * 2          # -> [inf, inf]
list(range(1, 4))           # -> [1, 2, 3]
[1] + [0] * 3               # -> [1, 0, 0, 0]
```

When you fill a list by index, make it at its final size first. Common start values:

- `[0] * n` for counts, and `[-1] * n` for "no answer yet".
- `[False] * n` for flags, and `list(range(n))` for a parent array.

12.2 Create a 2D list

CAUTION

Do not use `[[0] * cols] * rows`. It repeats one inner list. All rows are the same object.

```
bad = [[0] * 3] * 2
bad[0][0] = 1
bad                         # -> [[1, 0, 0], [1, 0, 0]]
good = [[0] * 3 for _ in range(2)]
good[0][0] = 1
good                        # -> [[1, 0, 0], [0, 0, 0]]
```

NOTE

`[0] * 3` is safe because integers are immutable. `[[]] * 3` and `[set()] * 3` are not safe.

Use the same comprehension form for a list of empty lists (an adjacency list) or a list of sets.

```
adj = [[] for _ in range(3)]        # a new list for each node
adj[0].append(1)
adj                                 # -> [[1], [], []]
cols = [set() for _ in range(3)]    # a new set for each column
cols[2].add("5")
cols                                # -> [set(), set(), {'5'}]
shared = [[]] * 3                   # wrong: one list three times
shared[0].append(1)
shared                              # -> [[1], [1], [1]]
```

12.3 Index and slice

Negative indexes count from the end. A slice `a[start:stop:step]` makes a new list. The `stop` index is not included.

```
a = [10, 20, 30, 40, 50]
a[0], a[-1], a[-2]          # -> (10, 50, 40)
a[1:3]                      # -> [20, 30]
a[:2], a[3:]                # -> ([10, 20], [40, 50])
a[::2]                      # -> [10, 30, 50]
a[::-1]                     # -> [50, 40, 30, 20, 10]
a[-2:]                      # -> [40, 50]
a[3:1:-1]                   # -> [40, 30]
```

```
a[100:]                     # -> []
a[5]                        # -> raises IndexError
```

CAUTION

A negative index does not raise an error. `a[i - 1]` with `i = 0` silently reads the last element. Guard the index, for example `i > 0 and a[i - 1] == x`. Also, `a[-k:]` with `k = 0` gives the full list, not an empty list.

```
a = [10, 20, 30]
i = 0
a[i - 1]                    # -> 30
i > 0 and a[i - 1] == 30    # -> False
k = 0
a[-k:]                      # -> [10, 20, 30]
a[len(a) - k:]              # -> []
```

12.4 Assign to a slice and delete a slice

```
a = [0, 1, 2, 3, 4]
a[1:3] = ["x", "y", "z"]    # the new part can have a different length
a                           # -> [0, 'x', 'y', 'z', 3, 4]
del a[1:4]
a                           # -> [0, 3, 4]
a[::2] = [9, 9]             # an extended slice needs the same length
a                           # -> [9, 3, 9]
a[:] = []                   # clear in place
a                           # -> []
```

12.5 Add elements

```
a = [1, 2]
a.append(3)                 # add one element at the end
a.extend([4, 5])            # add all elements of an iterable
a.insert(0, 0)              # insert before index 0
a                           # -> [0, 1, 2, 3, 4, 5]
a.append([6, 7])            # append adds the list as ONE element
a[-1]                       # -> [6, 7]
```

12.6 Remove elements

```
a = [5, 3, 7, 3]
a.pop()                     # -> 3
a.pop(0)                    # -> 5
a                           # -> [3, 7]
a.remove(7)                 # remove the first element equal to 7
a                           # -> [3]
a.remove(99)                # -> raises ValueError
a.clear()
a                           # -> []
[].pop()                    # -> raises IndexError
```

12.7 Search and count

```
a = [4, 2, 4, 1]
a.index(4)                  # -> 0
a.index(4, 1)               # -> 2
a.count(4)                  # -> 2
4 in a                      # -> True
a.index(9)                  # -> raises ValueError
```

```
a.index(4, 1, 3)           # -> 2
0 in [3, 0, 5]             # -> True
```

`a.index(x, start, stop)` searches only `a[start:stop]`, but it returns an index in the full list.

12.8 Sort and reverse in place

`sort()` and `reverse()` change the list and return `None`. `sorted()` returns a new list. `reversed()` returns an iterator that reads the list from the end.

```
a = [3, 1, 2]
a.sort()
a                          # -> [1, 2, 3]
a.sort(reverse=True)
a                          # -> [3, 2, 1]
a.reverse()
a                          # -> [1, 2, 3]
b = [3, 1, 2]
sorted(b), b               # -> ([1, 2, 3], [3, 1, 2])
print(b.sort())            # prints: None
```

CAUTION

Do not write `a = a.sort()`. The result is `None`.

12.9 Operators and built-in functions

```
[1, 2] + [3]               # -> [1, 2, 3]
[1, 2] * 2                 # -> [1, 2, 1, 2]
len([1, 2, 3])             # -> 3
min([4, 2, 8]), max([4, 2, 8]), sum([4, 2, 8])   # -> (2, 8, 14)
max([], default=0)         # -> 0
max([])                    # -> raises ValueError
any([0, 0, 3]), all([1, 0])   # -> (True, False)
[1, 2] == [1, 2], [1, 2] < [1, 2, 0]   # -> (True, True)
```

12.10 min and max with a key

```
words = ["kiwi", "apple", "fig"]
max(words, key=len)                # -> 'apple'
min(words, key=lambda w: w[-1])    # -> 'apple'
points = [(1, 5), (3, 2), (0, 9)]
min(points, key=lambda p: p[1])    # -> (3, 2)
a = [7, 3, 9]
max(range(len(a)), key=lambda i: a[i])   # -> 2
max(["bb", "aa", "c"], key=len)    # -> 'bb'
b = [4, 9, 9]
max(range(len(b)), key=lambda i: b[i])   # -> 1
```

`max` and `min` return the first element if two elements have the same key. The `max(range(len(a)), key=...)` idiom gives the index of the largest element.

12.11 Use a list as a stack

`append` (amortized) and `pop()` at the end are $O(1)$.

```
stack = []
stack.append(1); stack.append(2)
stack[-1]                  # -> 2
stack.pop()                # -> 2
bool(stack)                # -> True
```

CAUTION

`pop(0)` and `insert(0, x)` are $O(n)$. For a queue, use `collections.deque`.

12.12 Copy a list

```
a = [1, [2, 3]]
b = a[:]                    # shallow copy (same as list(a) or a.copy())
b[0] = 99
a                           # -> [1, [2, 3]]
b[1].append(4)              # the inner list is shared
a                           # -> [1, [2, 3, 4]]
import copy
c = copy.deepcopy(a)
c[1].append(5)
a                           # -> [1, [2, 3, 4]]
grid = [[1, 2], [3, 4]]
g2 = [row[:] for row in grid]   # copy a 2D grid, fast
g2[0][0] = 0
grid[0][0]                  # -> 1
```

12.13 The loop variable is a name, not a slot

`for x in a` binds the name `x` to each element. An assignment to `x` does not change the list. Use an index to change the elements. In a list of lists, `row` is the inner list itself. A change through `row` changes the grid.

```
a = [1, 2, 3]
for x in a:
    x *= 10                 # rebinds the name x only
a                           # -> [1, 2, 3]
for i in range(len(a)):
    a[i] *= 10              # changes the list
a                           # -> [10, 20, 30]
grid = [[1, 2], [3, 4]]
for row in grid:
    row.reverse()           # changes each inner list in place
grid                        # -> [[2, 1], [4, 3]]
```

12.14 List methods and their cost

Method	Action	Time
`append(x)`	add `x` at the end	$O(1)$ amortized
`extend(it)`	add all elements of `it` at the end	$O(k)$
`insert(i, x)`	insert `x` before index `i`	$O(n)$
`pop()`	remove and return the last element	$O(1)$
`pop(i)`	remove and return the element at index `i`	$O(n)$
`remove(x)`	remove the first element equal to `x`; `ValueError` if there is none	$O(n)$
`index(x, start, stop)`	index of the first `x`; `ValueError` if there is none	$O(n)$
`count(x)`	number of elements equal to `x`	$O(n)$
`sort(key=, reverse=)`	sort in place (stable); returns `None`	$O(n \log n)$
`reverse()`	reverse in place; returns `None`	$O(n)$
`copy()`	shallow copy, the same as `a[:]`	$O(n)$
`clear()`	remove all elements	$O(n)$

12.15 Grid and matrix idioms

```
m = [[1, 2, 3],
     [4, 5, 6]]
len(m), len(m[0])                       # -> (2, 3)
[list(col) for col in zip(*m)]          # -> [[1, 4], [2, 5], [3, 6]]
[list(r) for r in zip(*m[::-1])]        # -> [[4, 1], [5, 2], [6, 3]]
[list(r) for r in zip(*m)][::-1]        # -> [[3, 6], [2, 5], [1, 4]]
[row[::-1] for row in m]                # -> [[3, 2, 1], [6, 5, 4]]
m[::-1]                                 # -> [[4, 5, 6], [1, 2, 3]]
[v for row in m for v in row]           # -> [1, 2, 3, 4, 5, 6]
sum(map(sum, m))                        # -> 21
[m[r][r] for r in range(2)]             # -> [1, 5]
```

- `zip(*m)` transposes. `zip(*m[::-1])` rotates 90 degrees clockwise. `list(zip(*m))[::-1]` rotates 90 degrees counterclockwise.
- On a diagonal from top-left to bottom-right, `r - c` is constant. On the other diagonal, `r + c` is constant.
- To store a cell as one integer, use `k = r * cols + c`. Get it back with `r, c = divmod(k, cols)`.
- In a 9×9 board, the 3×3 box of cell `(r, c)` has the number `(r // 3) * 3 + c // 3`.

```
cols = 4
k = 2 * cols + 3
divmod(k, cols)                         # -> (2, 3)
r, c = 4, 7
(r // 3) * 3 + c // 3                   # -> 5
(r // 3, c // 3)                        # -> (1, 2)
m = [[1, 2, 3], [4, 5, 6], [7, 8, 9]]
n = len(m)
for i in range(n):                      # transpose in place: swap across the diagonal
    for j in range(i + 1, n):
        m[i][j], m[j][i] = m[j][i], m[i][j]
for row in m:
    row.reverse()
m                                       # -> [[7, 4, 1], [8, 5, 2], [9, 6, 3]]
```

The last example rotates a square matrix 90 degrees clockwise in place: transpose, then reverse each row.

To read a rectangle, slice the rows, then slice each row. A nested generator expression adds the values without a temporary list.

```
m = [[1, 2, 3], [4, 5, 6], [7, 8, 9]]
sum(sum(row[1:3]) for row in m[0:2])    # -> 16
[row[1:3] for row in m[0:2]]            # -> [[2, 3], [5, 6]]
```

The example reads rows 0 to 1 and columns 1 to 2. For many rectangle queries, use a 2D prefix sum.

12.16 Grid of strings

A grid often comes as a list of strings. `grid[r][c]` reads one character. A string is immutable, so an assignment to `grid[r][c]` raises `TypeError`. Convert each row to a list when you must mark cells.

```
rows = ["#.", ".."]
rows[0][1]                              # -> '.'
rows[0][1] = "#"                        # -> raises TypeError
grid = [list(row) for row in rows]      # a list of lists of characters
grid[0][1] = "#"
["".join(row) for row in grid]          # -> ['##', '..']
```

12.17 Directions and turns in a grid

Store the four directions in clockwise order. An index `d` gives the current facing. `(d + 1) % 4` turns right, and `(d - 1) % 4` turns left. In Python, `(0 - 1) % 4` is 3, so a left turn from north correctly gives west.

```
DIRS = ((-1, 0), (0, 1), (1, 0), (0, -1))   # north, east, south, west
r, c, d = 0, 0, 0                            # at (0, 0), facing north
for cmd in "RFFLF":
    if cmd == "R":
        d = (d + 1) % 4                      # turn right
    elif cmd == "L":
        d = (d - 1) % 4                      # turn left
    else:
        dr, dc = DIRS[d]                     # move one step forward
        r, c = r + dr, c + dc
(r, c, d)                                    # -> (-1, 2, 0)
(0 - 1) % 4                                  # -> 3
```

13 Tuples

13.1 Create a tuple

A tuple is an immutable sequence. A comma makes a tuple, not the parentheses.

```
t = (1, 2, 3)
t2 = 1, 2
type(t2)                  # -> tuple
single = (5,)             # one element needs a comma
type((5))                 # -> int
type(single)              # -> tuple
tuple("ab")               # -> ('a', 'b')
() == tuple()             # -> True
```

13.2 Use tuples

```
t = (3, 1, 3)
t[0], t[-1], t[1:]        # -> (3, 3, (1, 3))
t.count(3), t.index(1)    # -> (2, 1)
t + (4,)                  # -> (3, 1, 3, 4)
t[0] = 9                  # -> raises TypeError
seen = {(0, 0), (1, 2)}   # tuples can be set elements and dict keys
(1, 2) in seen            # -> True
```

Convert a list to a tuple to use it as a key. For example, a tuple of 26 letter counts is a key that is the same for all anagrams.

```
def signature(word):
    counts = [0] * 26
    for c in word:
        counts[ord(c) - ord("a")] += 1
    return tuple(counts)      # a list is not hashable; a tuple is
groups = {}
for w in ["eat", "tea", "tan"]:
    groups.setdefault(signature(w), []).append(w)
list(groups.values())         # -> [['eat', 'tea'], ['tan']]
point = [3, 4]
{tuple(point): 1}             # -> {(3, 4): 1}
```

CAUTION

A tuple that contains a list is not hashable, and you can change the list.

```
t = (1, [2])
t[1].append(3)
t                           # -> (1, [2, 3])
hash(t)                     # -> raises TypeError
```

13.3 Tuples compare element by element

Use tuples to sort by more than one key and to store heap entries.

```
(1, "b") < (1, "c") < (2, "a")       # -> True
sorted([(2, "x"), (1, "z"), (1, "y")])   # -> [(1, 'y'), (1, 'z'), (2, 'x')]
```

13.4 namedtuple

```
from collections import namedtuple
Point = namedtuple("Point", ["x", "y"])
p = Point(1, 2)
p.x, p[1]                   # -> (1, 2)
p                           # -> Point(x=1, y=2)
p._replace(x=5)             # -> Point(x=5, y=2)
p._asdict()                 # -> {'x': 1, 'y': 2}
```

14 Ranges

A `range` does not store its elements. Length, index and membership of an `int` are $O(1)$. Membership of another type (for example a float) checks each element.

```
r = range(0, 20, 5)
len(r), r[2], r[-1]         # -> (4, 10, 15)
15 in r, 16 in r            # -> (True, False)
15.0 in r                   # -> True
list(reversed(r))           # -> [15, 10, 5, 0]
range(5)[1:3]               # -> range(1, 3)
```

15 Dictionaries

15.1 Create a dictionary

```
{"a": 1, "b": 2}                  # -> {'a': 1, 'b': 2}
dict(a=1, b=2)                    # -> {'a': 1, 'b': 2}
dict([("a", 1), ("b", 2)])        # -> {'a': 1, 'b': 2}
dict(zip("ab", [1, 2]))           # -> {'a': 1, 'b': 2}
{c: ord(c) for c in "ab"}         # -> {'a': 97, 'b': 98}
dict.fromkeys("ab", 0)            # -> {'a': 0, 'b': 0}
{}                                # -> {}
type({})                          # -> dict
```

CAUTION

`dict.fromkeys(keys, [])` gives every key the same list. Use a comprehension: `{k: [] for k in keys}`.

In a comprehension or a loop, a later value for the same key replaces the earlier value. So `{v: i for i, v in enumerate(a)}` keeps the last index of each value.

```
{v: i for i, v in enumerate("abca")}        # -> {'a': 3, 'b': 1, 'c': 2}
{a: b for a, b in [("x", 1), ("y", 2)]}     # -> {'x': 1, 'y': 2}
{c: set() for w in ["ab", "bc"] for c in w}   # -> {'a': set(), 'b': set(), 'c': set()}
MOVES = {"U": (-1, 0), "D": (1, 0), "L": (0, -1), "R": (0, 1)}
dr, dc = MOVES["L"]
(dr, dc)                                    # -> (0, -1)
OPEN = {")": "(", "]": "[", "}": "{"}       # closing bracket: its opening bracket
OPEN["]"]                                   # -> '['
```

15.2 Get values

```
d = {"a": 1}
d["a"]                    # -> 1
d["z"]                    # -> raises KeyError
d.get("z")                # -> None
d.get("z", 0)             # -> 0
"a" in d                  # -> True
```

15.3 Set, update and delete

```
d = {"a": 1}
d["b"] = 2                          # add or replace
d.update({"b": 3, "c": 4})
d                                   # -> {'a': 1, 'b': 3, 'c': 4}
d.setdefault("d", 0)                # -> 0
d.setdefault("a", 99)               # -> 1
d.pop("a")                          # -> 1
d.pop("zz", None)                   # -> None
del d["b"]
d                                   # -> {'c': 4, 'd': 0}
d.popitem()                         # -> ('d', 0)
d | {"x": 1}                        # -> {'c': 4, 'x': 1}
d |= {"c": 5}
d                                   # -> {'c': 5}
```

`setdefault(k, v)` returns the value of `k`. If `k` is not there, it first sets it to `v`. `popitem()` removes the last inserted item. `|` and `|=` [Python 3.9+] merge dictionaries.

CAUTION

Python evaluates the arguments before the call. `d.setdefault(k, Node())` makes a new `Node` on every call, also when `k` is already there. For an expensive default, test with `in` first, or use a `defaultdict`.

```
made = []
def new_node():
    made.append(1)
    return {}
children = {}
node = children.setdefault("a", new_node())
node = children.setdefault("a", new_node())   # "a" is there, but new_node() runs again
len(made)                                     # -> 2
if "b" not in children:
    children["b"] = new_node()
len(made)                                     # -> 3
```

15.4 Iterate over a dictionary

Dictionaries keep insertion order [Python 3.7+].

```
d = {"b": 2, "a": 1}
list(d)                    # -> ['b', 'a']
list(d.keys())             # -> ['b', 'a']
list(d.values())           # -> [2, 1]
list(d.items())            # -> [('b', 2), ('a', 1)]
[k for k, v in d.items() if v > 1]   # -> ['b']
```

`next(iter(d))` gives the oldest key in $O(1)$. `keys()`, `values()` and `items()` return views. A view shows later changes to the dict. Key views and item views support set operations.

```
d = {"x": 1, "y": 2, "z": 3}
next(iter(d))                  # -> 'x'
d.pop(next(iter(d)))           # -> 1
list(d)                        # -> ['y', 'z']
keys = d.keys()
d["w"] = 0
list(keys)                     # -> ['y', 'z', 'w']
d.keys() & {"y", "q"}          # -> {'y'}
{"a": 1, "b": 2}.items() - {("a", 1)}   # -> {('b', 2)}
max(d.values()), len(d)        # -> (3, 3)
```

NOTE

Many deletions at the front of a dict make `next(iter(d))` slower in CPython. For an LRU cache, use `OrderedDict` (Section 19) or a dict with a doubly linked list.

15.5 Sort and search by value

```
scores = {"ana": 7, "bo": 9, "cy": 7}
max(scores, key=scores.get)                    # -> 'bo'
sorted(scores, key=scores.get)                 # -> ['ana', 'cy', 'bo']
sorted(scores.items(), key=lambda kv: (-kv[1], kv[0]))   # -> [('bo', 9), ('ana', 7), ('cy', 7)]
dict(sorted(scores.items()))                   # -> {'ana': 7, 'bo': 9, 'cy': 7}
{v: k for k, v in {"a": 1, "b": 2}.items()}    # -> {1: 'a', 2: 'b'}
```

15.6 Counting and grouping idioms

```
counts = {}
for w in ["a", "b", "a"]:
    counts[w] = counts.get(w, 0) + 1
counts                         # -> {'a': 2, 'b': 1}
groups = {}
for w in ["eat", "tea", "tan"]:
    groups.setdefault("".join(sorted(w)), []).append(w)
groups                         # -> {'aet': ['eat', 'tea'], 'ant': ['tan']}
```

`collections.Counter` and `collections.defaultdict` make these idioms shorter (Section 19).

15.7 Dictionary rules

- Keys must be hashable: numbers, strings, tuples of hashable values, frozensets.
- `1`, `1.0` and `True` are the same key because they are equal. The first key object stays, and the last value replaces the earlier values.
- Two dictionaries are equal if they have the same items. The order does not matter.

```
{1: "a", 1.0: "b"}             # -> {1: 'b'}
{"a": 1, "b": 2} == {"b": 2, "a": 1}   # -> True
{[1]: 0}                       # -> raises TypeError
```

An object of your own class is hashable by identity when the class does not define `__eq__`. Use such objects as

keys to map each original node to its copy. The key `None` maps a missing pointer to `None`, so the copy code needs no test for `None`.

```
class Node:
    def __init__(self, val, next=None):
        self.val, self.next = val, next
a = Node(1, Node(2))
copies = {None: None}                # original node: its copy
n = a
while n:
    copies[n] = Node(n.val)
    n = n.next
n = a
while n:
    copies[n].next = copies[n.next]  # copies[None] is None
    n = n.next
b = copies[a]
(b.val, b.next.val, b.next.next)     # -> (1, 2, None)
b is not a, Node(1) in copies        # -> (True, False)
```

16 Sets

16.1 Create a set

```
{1, 2, 2, 3}              # -> {1, 2, 3}
set("aab") == {"a", "b"}  # -> True
set()                     # -> set()
type({})                  # -> dict
{x % 3 for x in range(10)}   # -> {0, 1, 2}
```

CAUTION

`{}` is an empty dict. Use `set()` for an empty set.

16.2 Add and remove

```
s = {1, 2}
s.add(3)
s.update([4, 5])
s.discard(10)             # no error if the element is not there
s.remove(1)
s                         # -> {2, 3, 4, 5}
s.remove(10)              # -> raises KeyError
len(s), 3 in s            # -> (4, True)
```

16.3 Set operations

```
a, b = {1, 2, 3}, {2, 3, 4}
a | b                     # -> {1, 2, 3, 4}
a & b                     # -> {2, 3}
a - b                     # -> {1}
a ^ b                     # -> {1, 4}
{1, 2} <= a               # -> True
{1, 2} < a                # -> True
a >= {1}                  # -> True
a.isdisjoint({7, 8})      # -> True
a.union([9]), a.intersection(b, {3})   # -> ({1, 2, 3, 9}, {3})
a |= {10}
a                         # -> {1, 2, 3, 10}
```

The methods (`union`, `intersection`, `difference`) accept any iterable. The operators need sets.

16.4 More set methods

Method	Same as / action
`s.issubset(t)`, `s.issuperset(t)`	`s <= t`, `s >= t`
`s.union(t)`, `s.update(t)`	`s | t`, `s |= t`
`s.intersection(t)`, `s.intersection_update(t)`	`s & t`, `s &= t`
`s.difference(t)`, `s.difference_update(t)`	`s - t`, `s -= t`
`s.symmetric_difference(t)`	`s ^ t`
`s.pop()`	remove and return an arbitrary element; `KeyError` if `s` is empty
`s.copy()`, `s.clear()`	shallow copy; remove all elements

```
s = {3, 1, 2}
t = s.copy()
x = t.pop()                         # removes an arbitrary element
len(t), x in s                      # -> (2, True)
{1, 2}.issubset([1, 2, 3]), {1, 2, 3}.issuperset({3})   # -> (True, True)
a = {1, 2, 3}
a.difference_update({1})
a.intersection_update({2, 9})
a                                   # -> {2}
{1, 2}.symmetric_difference([2, 3])   # -> {1, 3}
set().pop()                         # -> raises KeyError
```

16.5 frozenset

A `frozenset` is an immutable set. You can put it in a set or use it as a dict key.

```
regions = {frozenset({(0, 0), (0, 1)}), frozenset({(5, 5)})}
frozenset({(5, 5)}) in regions       # -> True
{{1, 2}}                             # -> raises TypeError
```

16.6 Set idioms

```
nums = [3, 1, 3, 2, 1]
len(set(nums))                       # -> 3
list(dict.fromkeys(nums))            # -> [3, 1, 2]
sorted(set(nums))                    # -> [1, 2, 3]
seen, dupes = set(), set()
for x in nums:
    (dupes if x in seen else seen).add(x)
sorted(dupes)                        # -> [1, 3]
```

`list(dict.fromkeys(nums))` removes duplicates and keeps the first order. `set` does not keep order.

```
nums = [1, 2, 2]
len(set(nums)) < len(nums)           # -> True
seen = set()
for r, c, v in [(0, 0, "5"), (1, 1, "5")]:   # one set of tuples replaces many sets
    keys = [("row", r, v), ("col", c, v), ("box", r // 3, c // 3, v)]
    dup = any(k in seen for k in keys)
    seen.update(keys)
dup                                  # -> True
```

`len(set(a)) < len(a)` tests for a duplicate in one line. A tuple that includes a tag (`"row"`, `"col"`, `"box"`) puts

several kinds of keys in one set.

NOTE

A set has no order. Its print order is not the sorted order (`{1, 2, 3, 10}` can print as `{10, 1, 2, 3}`). For strings, the order changes from run to run. Do not depend on it. Use `sorted()`.

17 Comprehensions

17.1 List, dict and set comprehensions

```
[x * 2 for x in range(4)]                  # -> [0, 2, 4, 6]
[x for x in range(10) if x % 3 == 0]       # -> [0, 3, 6, 9]
["even" if x % 2 == 0 else "odd" for x in range(3)]   # -> ['even', 'odd', 'even']
{x: x * x for x in range(3)}               # -> {0: 0, 1: 1, 2: 4}
{len(w) for w in ["a", "bb", "cc"]}        # -> {1, 2}
```

NOTE

A filter `if` goes at the end. A conditional value `a if c else b` goes at the start.

17.2 Nested comprehensions

The `for` clauses run in the same order as nested loops.

```
[(r, c) for r in range(2) for c in range(2)]    # -> [(0, 0), (0, 1), (1, 0), (1, 1)]
[[r * 3 + c for c in range(3)] for r in range(2)]   # -> [[0, 1, 2], [3, 4, 5]]
[x for row in [[1, 2], [3]] for x in row]       # -> [1, 2, 3]
[(a, b) for a in range(3) for b in range(a)]    # -> [(1, 0), (2, 0), (2, 1)]
```

17.3 Generator expressions as arguments

A function call with one argument can take a generator expression without extra parentheses. Python makes the values one at a time, so it builds no temporary list.

```
from collections import Counter, deque
board = [["a", "b"], ["b", "c"]]
Counter(ch for row in board for ch in row)["b"]   # -> 2
indeg = [0, 2, 0]
deque(i for i in range(3) if indeg[i] == 0)       # -> deque([0, 2])
offsets = [(dr, dc) for dr in (-1, 0, 1) for dc in (-1, 0, 1) if (dr, dc) != (0, 0)]
len(offsets)                                      # -> 8
words = ["apple", "fig"]
max(map(len, words))                              # -> 5
sum(len(w) for w in words if "p" in w)            # -> 5
[(w, n) for w in words if (n := len(w)) > 3]      # -> [('apple', 5)]
```

The walrus operator computes `len(w)` one time and uses it in the filter and in the value.

17.4 Comprehension scope

The loop variable of a comprehension does not leak out of the comprehension. A name that `:=` assigns inside a comprehension is different: it belongs to the enclosing scope.

```
x = "outer"
[x for x in range(3)]
x                                         # -> 'outer'
[y for v in [1, 2, 3] if (y := v * 10) > 15]   # -> [20, 30]
y                                         # -> 30
```

18 Mutability and copies

18.1 Mutable and immutable objects

You can change a mutable object (list, dict, set) in place. You cannot change an immutable object (int, float, str, tuple, frozenset). An operation on an immutable object makes a new object.

```
s = "ab"
t = s
s += "c"                  # makes a new string; t does not change
(s, t)                    # -> ('abc', 'ab')
a = [1]
b = a
a += [2]                  # changes the list in place; b sees it
b                         # -> [1, 2]
```

18.2 Arguments are passed by object reference

A function gets a reference to the same object. A change in place is visible to the caller. Assignment to the parameter name is not visible.

```
def mutate(lst):
    lst.append(99)
def rebind(lst):
    lst = [0]
a = [1]
mutate(a); rebind(a)
a                         # -> [1, 99]
```

To replace all the contents of an argument, assign to the full slice. A method that sorts its argument also changes the caller's list. Use `sorted(xs)` when the caller needs the old order.

```
def reset(lst):
    lst[:] = [0] * len(lst)   # changes the caller's list
a = [1, 2]
reset(a)
a                         # -> [0, 0]
def middle(xs):
    xs.sort()             # sorts the caller's list too
    return xs[len(xs) // 2]
data = [3, 1, 2]
middle(data), data        # -> (2, [1, 2, 3])
```

18.3 Shallow copy and deep copy

Operation	Copies	Notes
`b = a`	nothing	same object
`a[:]`, `list(a)`, `a.copy()`	outer list only	inner objects are shared
`[row[:] for row in a]`	two levels	correct for a 2D grid of numbers
`copy.deepcopy(a)`	everything	slow; avoid in loops
`d.copy()`, `dict(d)`	outer dict only	lists inside are shared

```
import copy
d = {"k": [1]}
shallow = d.copy()
deep = copy.deepcopy(d)
d["k"].append(2)
shallow["k"], deep["k"]   # -> ([1, 2], [1])
```

18.4 Keep the old values in a rolling update

Sometimes each new value must read only old values. Then write the new values to a separate list. Rebind the name to the new list at the end of the step. If one list is read and written in the same step, the update reads some new values by mistake.

```
prev = [1, 1, 1]
for _ in range(2):              # grid paths, one row at a time
    cur = [1] * 3               # a new list for each row
    for j in range(1, 3):
        cur[j] = cur[j - 1] + prev[j]
    prev = cur                  # rebind; no copy is necessary
prev                            # -> [1, 3, 6]
dist = [0, 5, 9]
new = dist.copy()               # read from dist, write to new
for u, v, w in [(0, 1, 1), (1, 2, 1)]:
    if dist[u] + w < new[v]:
        new[v] = dist[u] + w
new                             # -> [0, 1, 6]
```

In the second example, one round uses only the distances of the previous round. An in-place update gives 2 for the last node, because it uses the new value of node 1 in the same round.

18.5 Copy the current path in backtracking

CAUTION

`res.append(path)` stores a reference to the same list. Later changes to `path` change the stored result. Append a copy: `res.append(path[:])`.

```
res, path = [], []
for x in [1, 2]:
    path.append(x)
    res.append(path)      # wrong: the same list two times
res                       # -> [[1, 2], [1, 2]]
res, path = [], []
for x in [1, 2]:
    path.append(x)
    res.append(path[:])   # correct
res                       # -> [[1], [1, 2]]
```

19 The collections module

19.1 deque

A deque is a double-ended queue. Add and remove at both ends in $O(1)$ time.

```
from collections import deque
q = deque([1, 2, 3])
q.append(4)               # right end
q.appendleft(0)           # left end
q                         # -> deque([0, 1, 2, 3, 4])
q.pop(), q.popleft()      # -> (4, 0)
q[0], q[-1]               # -> (1, 3)
q.extend([7, 8]); q.extendleft([-1, -2])   # extendleft adds -1, then -2: the order reverses
q                         # -> deque([-2, -1, 1, 2, 3, 7, 8])
q.rotate(2)               # move 2 elements from right to left
q                         # -> deque([7, 8, -2, -1, 1, 2, 3])
q.rotate(-2)
len(q), 3 in q            # -> (7, True)
deque().popleft()         # -> raises IndexError
```

Use `maxlen` to keep only the last `n` elements.

```
from collections import deque
last3 = deque(maxlen=3)
for x in range(6):
    last3.append(x)
last3                       # -> deque([3, 4, 5], maxlen=3)
```

CAUTION

Indexing in the middle of a deque is $O(n)$. Do not use a deque for random access.

`deque(iterable)` accepts any iterable, also a generator expression. An empty deque is false, so `while q:` runs until the queue is empty. A deque also has `remove`, `reverse`, `index`, `count`, `insert` and `clear`.

```
from collections import deque
q = deque(x * x for x in range(4))
q                           # -> deque([0, 1, 4, 9])
bool(deque())               # -> False
q.remove(4); q.reverse()
q                           # -> deque([9, 1, 0])
q.index(1), q.count(9)      # -> (1, 1)
q.insert(1, 7)
q                           # -> deque([9, 7, 1, 0])
q.clear()
len(q)                      # -> 0
```

Process a breadth-first search one level at a time. Read the size of the level before the inner loop.

```
from collections import deque
children = {1: [2, 3], 2: [4], 3: [], 4: []}
q, levels = deque([1]), []
while q:
    level = []
    for _ in range(len(q)):           # only the nodes of this level
        u = q.popleft()
        level.append(u)
        q.extend(children[u])
    levels.append(level)
levels                      # -> [[1], [2, 3], [4]]
```

19.2 defaultdict

A `defaultdict` calls a factory function to make a value for a missing key.

```
from collections import defaultdict
graph = defaultdict(list)
for u, v in [(1, 2), (1, 3), (2, 3)]:
    graph[u].append(v)
    graph[v].append(u)
graph[1]                    # -> [2, 3]
counts = defaultdict(int)
for ch in "abca":
    counts[ch] += 1
counts["a"], counts["z"]    # -> (2, 0)
dict(counts)                # -> {'a': 2, 'b': 1, 'c': 1, 'z': 0}
nested = defaultdict(lambda: defaultdict(int))
nested["x"]["y"] += 5
nested["x"]["y"]            # -> 5
groups = defaultdict(set)
groups["k"].add(1)
groups["k"]                 # -> {1}
```

CAUTION

When you read a missing key, the `defaultdict` inserts it (see `"z"` above). To test for a key, use `key in d`. Do not use `d[key]`.

`d.get(key, default)` reads a `defaultdict` without an insert. A loop over `d[key]` inserts the key.

```
from collections import defaultdict
g = defaultdict(list)
g.get("x", [])            # -> []
"x" in g                  # -> False
for v in g["y"]:          # reading g["y"] inserts an empty list
    pass
"y" in g                  # -> True
```

19.3 Counter

A `Counter` counts hashable objects. A missing key has the count 0. It does not insert the key.

```
from collections import Counter
c = Counter("abracadabra")
c["a"], c["z"]            # -> (5, 0)
c.most_common(2)          # -> [('a', 5), ('b', 2)]
Counter([1, 1, 2])        # -> Counter({1: 2, 2: 1})
Counter({"x": 2, "y": 1}).total()   # -> 3
sorted(Counter("aab").elements())   # -> ['a', 'a', 'b']
c.update("aa")            # add counts
c["a"]                    # -> 7
c.subtract({"a": 6})      # subtract counts (can go negative)
c["a"]                    # -> 1
del c["a"]
"a" in c                  # -> False
```

```
from collections import Counter
a, b = Counter("aab"), Counter("abc")
a + b                     # -> Counter({'a': 3, 'b': 2, 'c': 1})
a - b                     # -> Counter({'a': 1})
a & b                     # -> Counter({'a': 1, 'b': 1})
a | b                     # -> Counter({'a': 2, 'b': 1, 'c': 1})
Counter("listen") == Counter("silent")    # -> True
Counter("ab") <= Counter("aabb")          # -> True
not (Counter("ab") - Counter("aabb"))     # -> True
```

- `a - b` keeps only positive counts. Use it to find what is missing.
- `most_common()` without an argument gives all items from the highest count to the lowest.
- `total()` [Python 3.10+] gives the sum of the counts. On old versions, use `sum(c.values())`.
- Comparisons `<=` and `>=` between counters [Python 3.10+] check multiset inclusion.

```
from collections import Counter
c = Counter("aab")
c["z"] -= 1                       # a missing key starts at 0
c["z"]                            # -> -1
+c                                # -> Counter({'a': 2, 'b': 1})
Counter(a=1, b=0) == Counter(a=1) # -> True
len(Counter("aab"))               # -> 2
sorted(Counter([3, 1, 3]))        # -> [1, 3]
Counter([(0, 0), (0, 0), (1, 2)])[(0, 0)]   # -> 2
max(Counter("aabbb").values())    # -> 3
Counter("aab").most_common()[-1]  # -> ('b', 1)
[k for k, v in Counter("abca").items() if v == 1]   # -> ['b', 'c']
c.get("q") is None, c.get("q", 0) # -> (True, 0)
```

- `c[k]` gives 0 for a missing key. `c.get(k)` gives `None`, like a plain dict. Write `c.get(k, 0)`.
- `+c` removes the keys with a count of 0 or less.
- Equality treats a count of 0 as a missing key [Python 3.10+]. On older versions, the two counters in the equality example above are not equal.
- `len(c)` is the number of keys. A key stays in the counter when its count falls to 0.

CAUTION

In a sliding window, `len(window)` counts the distinct values only if you delete each key whose count reaches 0.

```
from collections import Counter
window = Counter("ab")
window["a"] -= 1
len(window)                       # -> 2
if window["a"] == 0:
    del window["a"]
len(window)                       # -> 1
```

19.4 OrderedDict

A normal dict keeps insertion order. An `OrderedDict` also has `move_to_end` and `popitem(last=False)`. Use it for an LRU cache.

```
from collections import OrderedDict
cache = OrderedDict()
for k in "abc":
    cache[k] = k.upper()
cache.move_to_end("a")            # "a" is now the most recent
cache.popitem(last=False)         # -> ('b', 'B')
list(cache)                       # -> ['c', 'a']
cache.move_to_end("a", last=False)
list(cache)                       # -> ['a', 'c']
```

19.5 ChainMap

A `ChainMap` searches several dictionaries in order.

```
from collections import ChainMap
defaults, user = {"color": "red", "size": 1}, {"size": 3}
cfg = ChainMap(user, defaults)
cfg["size"], cfg["color"]         # -> (3, 'red')
```

20 heapq: priority queues

20.1 Basic operations

`heapq` makes a min-heap on a normal list. `h[0]` is always the smallest element.

```
import heapq
h = []
for x in [5, 1, 8, 3]:
    heapq.heappush(h, x)
h[0]                      # -> 1
heapq.heappop(h)          # -> 1
heapq.heappop(h)          # -> 3
len(h)                    # -> 2
a = [9, 4, 7, 1]
heapq.heapify(a)          # O(n), in place
a[0]                      # -> 1
heapq.heappushpop(a, 0)   # -> 0
```

```
heapq.heapreplace(a, 10)  # -> 1
sorted(a)                 # -> [4, 7, 9, 10]
heapq.heappop([])         # -> raises IndexError
```

- `heappush`, `heappop`: $O(\log n)$. `heapify`: $O(n)$. `h[0]`: $O(1)$.
- `heappushpop(h, x)`: push `x`, then pop the smallest. Faster than two calls.
- `heapreplace(h, x)`: pop the smallest, then push `x`.
- The list is not sorted. Only `h[0]` has a known position.
- `heapify` changes the list in place. Copy the input first if the caller needs it.
- A list with zero or one element is already a valid heap. It needs no `heapify`.

```
import heapq
nums = [5, 2, 8]
h = list(nums)            # copy, then heapify the copy
heapq.heapify(h)
nums, h[0]                # -> ([5, 2, 8], 2)
h = [(0, "src")]          # one element: already a heap
heapq.heappush(h, (3, "b"))
h[0][0]                   # -> 0
```

20.2 Max-heap

`heapq` has only a min-heap. Push negative values to get a max-heap.

```
import heapq
h = []
for x in [5, 1, 8]:
    heapq.heappush(h, -x)
-h[0]                     # -> 8
-heapq.heappop(h)         # -> 8
counts = {"a": 3, "b": 1}
mh = [(-n, ch) for ch, n in counts.items()]   # max-heap by count
heapq.heapify(mh)
n, ch = heapq.heappop(mh)
(-n, ch)                  # -> (3, 'a')
n += 1                    # use one copy: -3 becomes -2
if n:
    heapq.heappush(mh, (n, ch))
mh[0]                     # -> (-2, 'a')
```

NOTE

Python 3.14 adds `heapq.heappush_max`, `heapq.heappop_max` and `heapq.heapify_max`. Most judges run an older version. Use negated keys.

20.3 Priorities with tuples

Push tuples `(priority, value)`. Tuples compare by the first element, then the next one.

```
import heapq
tasks = []
heapq.heappush(tasks, (2, "write"))
heapq.heappush(tasks, (1, "read"))
heapq.heappush(tasks, (2, "test"))
[heapq.heappop(tasks) for _ in range(3)]   # -> [(1, 'read'), (2, 'test'), (2, 'write')]
```

CAUTION

If two priorities are equal, Python compares the next element. If that element cannot be compared (for example a dict or a node object), you get `TypeError`. Put a unique counter in the middle.

```
import heapq, itertools
counter = itertools.count()
h = []
heapq.heappush(h, (1, next(counter), {"id": "a"}))
heapq.heappush(h, (1, next(counter), {"id": "b"}))
heapq.heappop(h)[2]       # -> {'id': 'a'}
bad = []
heapq.heappush(bad, (1, {"id": "a"}))
heapq.heappush(bad, (1, {"id": "b"}))   # -> raises TypeError
```

20.4 k largest, k smallest and merge

```
import heapq
nums = [5, 1, 9, 3, 7]
heapq.nlargest(2, nums)                  # -> [9, 7]
heapq.nsmallest(2, nums)                 # -> [1, 3]
heapq.nlargest(1, ["aa", "b"], key=len)  # -> ['aa']
list(heapq.merge([1, 4], [2, 3], [0]))   # -> [0, 1, 2, 3, 4]
list(heapq.merge([4, 1], [2]))           # -> [2, 4, 1]
k, h = 3, []
for x in nums:                           # keep the k largest in a min-heap
    heapq.heappush(h, x)
    if len(h) > k:
        heapq.heappop(h)
sorted(h)                                # -> [5, 7, 9]
h[0]                                     # -> 5
```

`h[0]` of a size-k min-heap is the k-th largest element. `heapq.merge` needs sorted inputs. It does not check them, so unsorted inputs give an unsorted result.

20.5 Lazy deletion

`heapq` cannot change or remove an element inside the heap. Push a new entry instead. When you pop an entry that is out of date, skip it. Dijkstra's algorithm uses this method.

```
import heapq
best = {"a": 5, "b": 4}
h = [(4, "b"), (5, "a")]
best["a"] = 2                        # a better value for "a": push a new entry
heapq.heappush(h, (2, "a"))
out = []
while h:
    d, u = heapq.heappop(h)
    if d != best[u]:
        continue                     # an old entry: skip it
    out.append((u, d))
out                                  # -> [('a', 2), ('b', 4)]
```

20.6 Heap of custom objects

Define `__lt__` so that the heap can compare the objects.

```
import heapq
class Node:
    def __init__(self, val):
        self.val = val
    def __lt__(self, other):
        return self.val < other.val
h = [Node(3), Node(1)]
heapq.heapify(h)
```

```
h[0].val                  # -> 1
```

21 bisect: binary search on sorted lists

```
import bisect
a = [1, 3, 3, 3, 7]
bisect.bisect_left(a, 3)      # -> 1
bisect.bisect_right(a, 3)     # -> 4
bisect.bisect(a, 3)           # -> 4
bisect.bisect_left(a, 4)      # -> 4
bisect.bisect_left(a, 0)      # -> 0
bisect.bisect_left(a, 100)    # -> 5
bisect.bisect_right(a, 3) - bisect.bisect_left(a, 3)   # -> 3
bisect.insort(a, 5)
a                             # -> [1, 3, 3, 3, 5, 7]
```

- `bisect_left(a, x)`: the first index `i` with `a[i] >= x`.
- `bisect_right(a, x)`: the first index `i` with `a[i] > x`.
- The count of elements in `[lo, hi]` is `bisect_right(a, hi) - bisect_left(a, lo)`.
- `insort` is $O(n)$ because the list must shift elements.

21.1 Floor and ceiling in a sorted list

```
import bisect
a = [10, 20, 30]
def floor(x):                 # largest element <= x
    i = bisect.bisect_right(a, x)
    return a[i - 1] if i else None
def ceiling(x):               # smallest element >= x
    i = bisect.bisect_left(a, x)
    return a[i] if i < len(a) else None
floor(25), ceiling(25), floor(5), ceiling(35)   # -> (20, 30, None, None)
```

21.2 bisect with a key

[Python 3.10+]

```
import bisect
events = [(1, "a"), (4, "b"), (9, "c")]
bisect.bisect_left(events, 4, key=lambda e: e[0])   # -> 1
i = bisect.bisect_right(events, 5, key=lambda e: e[0]) - 1   # last time <= 5
events[i][1] if i >= 0 else ""                      # -> 'b'
```

`bisect_right(a, t) - 1` is the index of the last element with a time at or before `t`. The result `-1` means that no such element exists.

21.3 Search part of a list

`lo` and `hi` limit the search to `a[lo:hi]` without a slice copy. The result is an index in the full list. For a list in descending order, search a negated copy, or use `key=lambda x: -x` [Python 3.10+].

```
import bisect
a = [1, 3, 5, 7, 9]
i = 1
bisect.bisect_left(a, 7, lo=i + 1)        # -> 3
bisect.bisect_left(a, 7, 0, 2)            # -> 2
desc = [9, 7, 5]
bisect.bisect_left([-x for x in desc], -7)       # -> 1
bisect.bisect_left(desc, -7, key=lambda x: -x)   # -> 1
```

21.4 Binary search on the answer

`first_true` and `last_true` search a half-open range `[lo, hi)`. The search never calls `ok(hi)`.

```
def first_true(lo, hi, ok):   # first x in [lo, hi) with ok(x) True, or hi if no x passes
    while lo < hi:            # ok is False...False True...True on [lo, hi)
        mid = (lo + hi) // 2
        if ok(mid):
            hi = mid
        else:
            lo = mid + 1
    return lo

def last_true(lo, hi, ok):    # last x in [lo, hi) with ok(x) True, or lo - 1
    return first_true(lo, hi, lambda x: not ok(x)) - 1   # ok is True...True False...False

first_true(0, 100, lambda x: x * x >= 50)   # -> 8
first_true(0, 100, lambda x: x > 500)       # -> 100
last_true(0, 100, lambda x: x * x <= 50)    # -> 7
last_true(0, 100, lambda x: x < 0)          # -> -1
```

CAUTION

The direct form of the last true search uses a closed range `[lo, hi]` and needs `ok(lo)` True. It uses `mid = (lo + hi + 1) // 2` and `lo = mid`. If you round down there, the loop does not stop when `hi = lo + 1`. Do not mix this form with `last_true` above: for the same arguments, the results are different by one.

```
def last_true_closed(lo, hi, ok):   # last x in [lo, hi] with ok(x) True; ok(lo) must be True
    while lo < hi:
        mid = (lo + hi + 1) // 2    # round up
        if ok(mid):
            lo = mid
        else:
            hi = mid - 1
    return lo
last_true_closed(0, 100, lambda x: x * x <= 50)   # -> 7
last_true_closed(0, 100, lambda x: True)          # -> 100
```

22 Sorting

22.1 sort and sorted

`list.sort()` sorts in place. `sorted()` accepts any iterable and returns a new list. Both are stable and take $O(n \log n)$ time.

```
sorted([3, 1, 2])                # -> [1, 2, 3]
sorted("bca")                    # -> ['a', 'b', 'c']
sorted({"b": 1, "a": 2})         # -> ['a', 'b']
sorted([3, 1, 2], reverse=True)  # -> [3, 2, 1]
intervals = [[5, 6], [1, 3], [2, 4]]
intervals.sort(key=lambda iv: iv[0])   # in place, by start
intervals                        # -> [[1, 3], [2, 4], [5, 6]]
sorted([[2, 1], [1, 9], [1, 2]])  # -> [[1, 2], [1, 9], [2, 1]]
```

Lists and tuples sort element by element. Without a key, a list of intervals sorts by start, then by end.

Sort in reverse order, then use `pop()` from the end. Each `pop()` gives the next smallest element in $O(1)$.

```
stack = sorted(["b", "c", "a"], reverse=True)
stack.pop(), stack.pop()          # -> ('a', 'b')
```

22.2 Sort with a key

The key function gets each element and returns the value to compare.

```
words = ["banana", "Kiwi", "apple", "fig"]
sorted(words, key=len)                      # -> ['fig', 'Kiwi', 'apple', 'banana']
sorted(words, key=str.lower)                # -> ['apple', 'banana', 'fig', 'Kiwi']
sorted(words, key=lambda w: (len(w), w))    # -> ['fig', 'Kiwi', 'apple', 'banana']
sorted(words, key=lambda w: (-len(w), w))   # -> ['banana', 'apple', 'Kiwi', 'fig']
```

Return a tuple to sort by more than one key. Negate a number to sort that key in descending order.

22.3 Descending order for a string key

You cannot negate a string. Use two stable sorts: first the secondary key, then the primary key.

```
people = [("bo", 30), ("ana", 25), ("cy", 30)]
people.sort(key=lambda p: p[0], reverse=True)   # secondary: name descending
people.sort(key=lambda p: p[1])                 # primary: age ascending
people                    # -> [('ana', 25), ('cy', 30), ('bo', 30)]
```

22.4 itemgetter and attrgetter

```
from operator import itemgetter, attrgetter
rows = [("b", 2), ("a", 2), ("c", 1)]
sorted(rows, key=itemgetter(1))        # -> [('c', 1), ('b', 2), ('a', 2)]
sorted(rows, key=itemgetter(1, 0))     # -> [('c', 1), ('a', 2), ('b', 2)]
from collections import namedtuple
P = namedtuple("P", "name age")
min([P("x", 9), P("y", 3)], key=attrgetter("age"))   # -> P(name='y', age=3)
```

22.5 Argsort and ranks

```
x = [30, 10, 20]
order = sorted(range(len(x)), key=lambda i: x[i])
order                     # -> [1, 2, 0]
[x[i] for i in order]     # -> [10, 20, 30]
rank = [0] * len(x)
for r, i in enumerate(order):
    rank[i] = r
rank                      # -> [2, 0, 1]
```

`order[k]` is the original index of the k-th smallest value. `rank[i]` is the sorted position of `x[i]`. A bound method can be the key. `x.__getitem__` is the same as `lambda i: x[i]`, and it is faster.

```
x = [30, 10, 20]
sorted(range(len(x)), key=x.__getitem__)   # -> [1, 2, 0]
```

22.6 Sort by a custom alphabet

Map each character to its rank. The key of a word is the list of ranks of its characters.

```
order = "hlabcdefgijkmnopqrstuvwxyz"
rank = {c: i for i, c in enumerate(order)}
words = ["hello", "leetcode"]
words == sorted(words, key=lambda w: [rank[c] for c in w])   # -> True
sorted(["ab", "ha", "l"], key=lambda w: [rank[c] for c in w])   # -> ['ha', 'l', 'ab']
```

22.7 Custom comparison

Use `functools.cmp_to_key` when you cannot write a key. The function returns a negative number, zero, or a positive number.

```
from functools import cmp_to_key
def cmp(a, b):            # order numbers to make the largest concatenation
    return -1 if a + b > b + a else (1 if a + b < b + a else 0)
"".join(sorted(["3", "30", "34", "5", "9"], key=cmp_to_key(cmp)))   # -> '9534330'
```

22.8 Sort pitfalls

```
sorted([3, None, 1])      # -> raises TypeError
sorted([3, None, 1], key=lambda v: (v is None, v or 0))   # -> [1, 3, None]
sorted(["10", "9", "100"])                # -> ['10', '100', '9']
sorted(["10", "9", "100"], key=int)       # -> ['9', '10', '100']
```

23 Built-in iteration functions

```
list(map(int, ["1", "2"]))                # -> [1, 2]
list(map(lambda a, b: a * b, [1, 2], [3, 4]))   # -> [3, 8]
list(filter(None, [0, 1, "", "a"]))       # -> [1, 'a']
list(filter(lambda x: x > 1, [1, 2, 3]))  # -> [2, 3]
sum([[1], [2, 3]], [])                    # -> [1, 2, 3]
sum([1, 2, 3], 10)                        # -> 16
all([]), any([])                          # -> (True, False)
list(zip(*[(1, "a"), (2, "b")]))          # -> [(1, 2), ('a', 'b')]
dict(enumerate("ab"))                     # -> {0: 'a', 1: 'b'}
```

NOTE

`zip(*pairs)` unzips a list of pairs. `all([])` is `True`. `any([])` is `False`. The second argument of `sum` is the start value. `sum(lists, [])` copies the result for each list, so it takes $O(n^2)$ time for many lists. Use `list(chain.from_iterable(lists))` for many lists.

24 The itertools module

24.1 Infinite iterators

`count`, `cycle` and `repeat` without a count never stop. Stop them with `islice` or with `break`.

```
from itertools import count, cycle, repeat, islice
list(islice(count(10, 5), 3))     # -> [10, 15, 20]
list(islice(cycle("ab"), 5))      # -> ['a', 'b', 'a', 'b', 'a']
list(repeat(0, 3))                # -> [0, 0, 0]
```

24.2 accumulate

```
from itertools import accumulate
import operator
list(accumulate([1, 2, 3, 4]))                # -> [1, 3, 6, 10]
list(accumulate([1, 2, 3], initial=0))        # -> [0, 1, 3, 6]
list(accumulate([3, 1, 4, 1], max))           # -> [3, 3, 4, 4]
list(accumulate([1, 2, 3, 4], operator.mul))  # -> [1, 2, 6, 24]
```

`accumulate(a, initial=0)` [Python 3.8+] gives prefix sums where `P[i]` is the sum of `a[:i]`.

24.3 Combinatorics

```
from itertools import product, permutations, combinations, combinations_with_replacement
list(product("ab", [0, 1]))                # -> [('a', 0), ('a', 1), ('b', 0), ('b', 1)]
len(list(product(range(3), repeat=4)))     # -> 81
list(permutations([1, 2, 3], 2))           # -> [(1, 2), (1, 3), (2, 1), (2, 3), (3, 1), (3, 2)]
len(list(permutations(range(4))))          # -> 24
list(combinations("abc", 2))               # -> [('a', 'b'), ('a', 'c'), ('b', 'c')]
list(combinations_with_replacement("ab", 2))   # -> [('a', 'a'), ('a', 'b'), ('b', 'b')]
["".join(p) for p in product("01", repeat=2)]  # -> ['00', '01', '10', '11']
```

NOTE

These functions return tuples. They treat elements by position, not by value. Duplicates in the input give duplicate results.

24.4 Group, chain, slice, pair

```
from itertools import groupby, chain, islice, pairwise, zip_longest, batched
[(k, len(list(g))) for k, g in groupby("aaabcc")]   # -> [('a', 3), ('b', 1), ('c', 2)]
[k for k, _ in groupby([1, 1, 2, 1])]               # -> [1, 2, 1]
list(chain([1, 2], "ab"))                 # -> [1, 2, 'a', 'b']
list(chain.from_iterable([[1], [2, 3]]))  # -> [1, 2, 3]
list(islice(range(10), 2, 8, 2))          # -> [2, 4, 6]
list(pairwise([1, 2, 4]))                 # -> [(1, 2), (2, 4)]
list(zip_longest("ab", "x", fillvalue="-"))   # -> [('a', 'x'), ('b', '-')]
list(batched(range(5), 2))                # -> [(0, 1), (2, 3), (4,)]
```

CAUTION

`groupby` groups only consecutive equal elements. Sort the input first if you need one group for each key. `pairwise` needs [Python 3.10+]. `batched` needs [Python 3.12+].

Use `pairwise` with `all` or `any` to test adjacent elements. On old versions, use `zip(a, a[1:])`.

```
from itertools import pairwise
a = [1, 2, 2, 5]
all(x <= y for x, y in pairwise(a))      # -> True
any(x == y for x, y in zip(a, a[1:]))    # -> True
```

24.5 Filtering iterators

```
from itertools import takewhile, dropwhile, filterfalse, compress, starmap
list(takewhile(lambda x: x < 3, [1, 2, 5, 1]))    # -> [1, 2]
list(dropwhile(lambda x: x < 3, [1, 2, 5, 1]))    # -> [5, 1]
list(filterfalse(lambda x: x % 2, range(5)))      # -> [0, 2, 4]
list(compress("abcd", [1, 0, 1, 0]))              # -> ['a', 'c']
list(starmap(pow, [(2, 3), (3, 2)]))              # -> [8, 9]
```

25 The functools and operator modules

25.1 functools

`reduce(f, items, start)` combines the items into one value, from left to right. `partial(f, ...)` makes a new function with some arguments fixed.

```
from functools import reduce, partial, lru_cache, cache
reduce(lambda acc, x: acc * x, [1, 2, 3, 4], 1)   # -> 24
int2 = partial(int, base=2)
int2("101")                                       # -> 5
@lru_cache(maxsize=128)
def slow_square(n):
    return n * n
slow_square(4); slow_square(4)                    # -> 16
slow_square.cache_info().hits                     # -> 1
```

```
from functools import total_ordering
@total_ordering
class Version:
    def __init__(self, major, minor):
        self.key = (major, minor)
    def __eq__(self, other):
        return self.key == other.key
    def __lt__(self, other):
        return self.key < other.key
Version(1, 2) >= Version(1, 0)                    # -> True
```

`total_ordering` makes `<=`, `>` and `>=` from `__eq__` and `__lt__`.

25.2 operator

```
import operator
from functools import reduce
operator.add(2, 3), operator.mul(2, 3), operator.neg(4)   # -> (5, 6, -4)
reduce(operator.xor, [1, 2, 3])                   # -> 0
reduce(operator.or_, [{1}, {2}, {1, 3}])          # -> {1, 2, 3}
operator.itemgetter(0, 2)("abc")                  # -> ('a', 'c')
```

26 The math module

```
import math
math.sqrt(16), math.isqrt(17)          # -> (4.0, 4)
math.gcd(12, 18), math.lcm(4, 6)       # -> (6, 12)
math.gcd(12, 18, 30)                   # -> 6
math.factorial(5)                      # -> 120
math.comb(5, 2), math.perm(5, 2)       # -> (10, 20)
math.prod([2, 3, 4])                   # -> 24
math.floor(-1.5), math.ceil(-1.5)      # -> (-2, -1)
math.log2(1024), math.log10(1000)      # -> (10.0, 3.0)
math.log(math.e), math.log(8, 2)       # -> (1.0, 3.0)
math.hypot(3, 4)                       # -> 5.0
math.dist((0, 0), (3, 4))              # -> 5.0
math.inf > 10**300, -math.inf < 0      # -> (True, True)
math.isinf(math.inf), math.isnan(math.nan)   # -> (True, True)
round(math.pi, 4), round(math.e, 4)    # -> (3.1416, 2.7183)
math.isclose(1.0, 1.0 + 1e-12)         # -> True
math.fsum([0.1] * 10)                  # -> 1.0
total = 0.0
for x in [0.1] * 10:
```

```
    total += x
total                                   # -> 0.9999999999999999
round(math.degrees(math.atan2(1, 1)))   # -> 45
```

- `math.isqrt(n)` gives the exact integer square root. Use it, not `int(math.sqrt(n))`, for big integers.
- `math.lcm` needs [Python 3.9+]. A `math.gcd` call with more than two arguments also needs [Python 3.9+]. `math.comb`, `math.perm`, `math.prod`, `math.isqrt` and `math.dist` need [Python 3.8+].
- `math.fsum` adds floats with less rounding error. A plain loop with `+=` collects the error. `sum()` of floats also corrects the error [Python 3.12+], but not on older versions.

```
import math
n = 10**30 + 1
math.isqrt(n) ** 2 <= n                 # -> True
int(math.sqrt(10**30 - 1))              # -> 1000000000000000
math.isqrt(10**30 - 1)                  # -> 999999999999999
math.log(1000, 10), math.log(243, 3)    # -> (2.9999999999999996, 4.999999999999999)
int(math.log(243, 3))                   # -> 4
def is_power(n, b):                     # exact test with integers only
    while n > 1 and n % b == 0:
        n //= b
    return n == 1
is_power(243, 3), is_power(244, 3)      # -> (True, False)
```

CAUTION

`math.log(n, b)` is a float. It can be a little below the exact integer, so `int()` of it can be 1 too small. Do not use it to test for a power or to count digits. Use integer division, or compare with `b ** k`.

26.1 statistics

```
import statistics
statistics.mean([1, 2, 3, 4])           # -> 2.5
statistics.median([3, 1, 2])            # -> 2
statistics.median([1, 2, 3, 4])         # -> 2.5
statistics.mode([1, 2, 2, 3])           # -> 2
```

27 The random module

Use `random` to make test data. Set a seed to get the same values each run.

```
import random
random.seed(0)
a = random.randint(1, 6)        # 1 <= a <= 6 (both ends included)
1 <= a <= 6                     # -> True
b = random.randrange(10)        # 0 <= b < 10
c = random.choice("abc")
c in "abc"                      # -> True
deck = list(range(5))
random.shuffle(deck)            # in place
sorted(deck)                    # -> [0, 1, 2, 3, 4]
len(random.sample(range(100), 3))   # -> 3
0 <= random.random() < 1        # -> True
```

28 Classes and objects

28.1 Define a class

`__init__` sets up a new object. The first parameter of a method is the object itself (`self`).

```
class Account:
    bank = "PyBank"                 # class attribute: shared by all objects
    def __init__(self, owner, balance=0):
        self.owner = owner          # instance attributes
        self.balance = balance
    def deposit(self, amount):
        self.balance += amount
        return self.balance
acc = Account("ana")
acc.deposit(50)                     # -> 50
acc.owner, acc.bank                 # -> ('ana', 'PyBank')
isinstance(acc, Account)            # -> True
hasattr(acc, "balance"), getattr(acc, "missing", None)   # -> (True, None)
```

CAUTION

All objects share a mutable class attribute. Set lists and dicts in `__init__`.

```
class Bad:
    items = []
a, b = Bad(), Bad()
a.items.append(1)
b.items                             # -> [1]
```

28.2 String representation

`__repr__` gives the text for debugging. `__str__` gives the text for `print` and `str`.

```
class Point:
    def __init__(self, x, y):
        self.x, self.y = x, y
    def __repr__(self):
        return f"Point({self.x}, {self.y})"
    def __str__(self):
        return f"({self.x}, {self.y})"
p = Point(1, 2)
repr(p), str(p)                     # -> ('Point(1, 2)', '(1, 2)')
[p]                                 # -> [Point(1, 2)]
```

28.3 Equality, hashing and ordering

By default, `==` compares identity. Define `__eq__` and `__hash__` together to use objects as dict keys by value. Define `__lt__` to sort objects or to put them in a heap.

```
class Cell:
    def __init__(self, r, c):
        self.r, self.c = r, c
    def __eq__(self, other):
        return (self.r, self.c) == (other.r, other.c)
    def __hash__(self):
        return hash((self.r, self.c))
    def __lt__(self, other):
        return (self.r, self.c) < (other.r, other.c)
Cell(1, 2) == Cell(1, 2)            # -> True
len({Cell(1, 2), Cell(1, 2)})       # -> 1
[(c.r, c.c) for c in sorted([Cell(2, 0), Cell(1, 5)])]   # -> [(1, 5), (2, 0)]
```

NOTE

If you define `__eq__` but not `__hash__`, the objects become unhashable.

28.4 Dataclasses

`@dataclass` writes `__init__`, `__repr__` and `__eq__` for you.

```
from dataclasses import dataclass, field
@dataclass
class Item:
    name: str
    qty: int = 0
    tags: list = field(default_factory=list)   # a new list for each object
it = Item("pen", 3)
it                                  # -> Item(name='pen', qty=3, tags=[])
it == Item("pen", 3)                # -> True
@dataclass(order=True, frozen=True)
class Task:
    priority: int
    name: str = field(compare=False)
sorted([Task(2, "b"), Task(1, "a")])[0].name   # -> 'a'
hash(Task(1, "a")) == hash(Task(1, "zzz"))      # -> True
```

- `order=True` adds `<`, `<=`, `>`, `>=`. It compares the fields in order.
- `frozen=True` makes the objects immutable and hashable.
- `field(compare=False)` removes a field from comparisons.

28.5 Inheritance and super

`class Dog(Animal)` makes a subclass. The subclass gets all methods of `Animal`, and it can replace them. `super().__init__(...)` runs the `__init__` of the base class.

```
class Animal:
    def __init__(self, name):
        self.name = name
    def speak(self):
        return "..."
class Dog(Animal):
    def __init__(self, name, breed):
        super().__init__(name)
        self.breed = breed
    def speak(self):
        return "woof"
d = Dog("rex", "lab")
d.name, d.speak(), isinstance(d, Animal)   # -> ('rex', 'woof', True)
issubclass(Dog, Animal)                    # -> True
```

28.6 Properties, static methods and class methods

- `@property`: a method that you read like an attribute, without parentheses.
- `@staticmethod`: a function in the class body. It gets no `self`.
- `@classmethod`: it gets the class as `cls`. Use it for a second constructor.

```
class Circle:
    def __init__(self, r):
        self.r = r
    @property
    def diameter(self):             # read it like an attribute, without ()
        return 2 * self.r
    @staticmethod
    def unit():                     # no self, no cls
        return Circle(1)
    @classmethod
    def from_diameter(cls, d):      # cls is the class
        return cls(d / 2)
```

```
Circle(2).diameter                  # -> 4
Circle.unit().r                     # -> 1
Circle.from_diameter(10).r          # -> 5.0
```

28.7 Special methods

Method	Called by
`__len__`	`len(x)`
`__getitem__`, `__setitem__`	`x[k]`, `x[k] = v`
`__contains__`	`v in x`
`__iter__`	`for v in x`
`__call__`	`x(...)`
`__add__`, `__mul__`	`x + y`, `x * y`
`__eq__`, `__lt__`	`x == y`, `x < y`
`__hash__`	`hash(x)`, set, dict key
`__bool__`	`if x:`
`__repr__`, `__str__`	`repr(x)`, `str(x)`, `print(x)`

```
class Vec:
    def __init__(self, *xs):
        self.xs = list(xs)
    def __len__(self):
        return len(self.xs)
    def __getitem__(self, i):
        return self.xs[i]
    def __add__(self, other):
        return Vec(*[a + b for a, b in zip(self.xs, other.xs)])
    def __iter__(self):
        return iter(self.xs)
v = Vec(1, 2) + Vec(3, 4)
len(v), v[1], list(v), 4 in v       # -> (2, 6, [4, 6], True)
```

Python uses `__bool__` to decide if an object is true. If the class has no `__bool__`, Python uses `__len__`. An object with neither method is always true.

```
class Bag:
    def __init__(self):
        self.items = []
    def __len__(self):
        return len(self.items)
b = Bag()
bool(b)                             # -> False
b.items.append(1)
bool(b)                             # -> True
class Plain:
    pass
bool(Plain())                       # -> True
```

28.8 `__slots__`

`__slots__` saves memory and makes attribute access faster. The object cannot get new attributes.

```
class Node:
    __slots__ = ("val", "next")
    def __init__(self, val, next=None):
        self.val, self.next = val, next
n = Node(1)
n.other = 5                         # -> raises AttributeError
```

28.9 LeetCode node classes

```
class ListNode:
    def __init__(self, val=0, next=None):
        self.val = val
        self.next = next
class TreeNode:
    def __init__(self, val=0, left=None, right=None):
        self.val = val
        self.left = left
        self.right = right
def build_list(values):             # helper to test locally
    dummy = tail = ListNode()
    for v in values:
        tail.next = ListNode(v)
        tail = tail.next
    return dummy.next
def to_list(head):
    out = []
    while head:
        out.append(head.val)
        head = head.next
    return out
def reverse(head):
    prev = None
    while head:
        nxt = head.next             # save the rest of the list
        head.next = prev            # point this node backwards
        prev, head = head, nxt      # move both pointers one step
    return prev
to_list(reverse(build_list([1, 2, 3])))   # -> [3, 2, 1]
def reverse_short(head):            # the same loop in one line
    prev = None
    while head:
        head.next, prev, head = prev, head, head.next
    return prev
to_list(reverse_short(build_list([4, 5])))   # -> [5, 4]
root = TreeNode(1, TreeNode(2), TreeNode(3, TreeNode(4)))
def height(t):
    return 0 if t is None else 1 + max(height(t.left), height(t.right))
height(root)                        # -> 3
```

CAUTION

In a multiple assignment, Python evaluates the full right side first, then assigns from left to right. In the one-line form `head.next, prev, head = prev, head, head.next`, the order of the targets is important. If `head` changes before `head.next`, the code writes to the wrong node. The form with `nxt` is easier to get right.

A graph problem gives a node with a list of neighbors. The class uses `None` as the default, not `[]`, because a mutable default is shared by all calls. To copy a graph, map each original node to its copy in a dict. Objects without `__eq__` are dict keys by identity.

```
class Node:                         # LeetCode graph node
    def __init__(self, val=0, neighbors=None):
        self.val = val
        self.neighbors = neighbors if neighbors is not None else []
def clone(node):
    copies = {}                     # original node: its copy
    def dfs(n):
        if n in copies:
            return copies[n]
        copies[n] = Node(n.val)     # store the copy before the recursion
```

```
        copies[n].neighbors = [dfs(m) for m in n.neighbors]
        return copies[n]
    return dfs(node) if node else None
a, b = Node(1), Node(2)
a.neighbors, b.neighbors = [b], [a]
c = clone(a)
c.val, c.neighbors[0].val, c is not a  # -> (1, 2, True)
c.neighbors[0].neighbors[0] is c       # -> True
clone(None) is None                    # -> True
```

28.10 Design-problem classes

A design problem gives a class with `__init__` and several methods. Keep the state in instance attributes. The judge makes one object and calls its methods in order. The judge input is two lists: the method names and the argument lists. A short helper replays this input locally. `getattr(obj, name)` gets a method by its name.

```
class MinStack:
    def __init__(self):
        self.stack = []                 # pairs (value, minimum so far)
    def push(self, val: int) -> None:
        low = min(val, self.stack[-1][1]) if self.stack else val
        self.stack.append((val, low))
    def pop(self) -> None:
        self.stack.pop()
    def top(self) -> int:
        return self.stack[-1][0]
    def getMin(self) -> int:
        return self.stack[-1][1]

def replay(cls, ops, args):             # run the judge input format
    obj, out = None, []
    for op, a in zip(ops, args):
        if op == cls.__name__:
            obj = cls(*a)
            out.append(None)
        else:
            out.append(getattr(obj, op)(*a))
    return out

replay(MinStack, ["MinStack", "push", "push", "getMin", "pop", "top", "getMin"],
       [[], [3], [1], [], [], [], []])  # -> [None, None, None, 1, None, 3, 3]
```

28.11 A doubly linked list with sentinels

Two sentinel nodes (`head` and `tail`) remove the special cases at the ends. Every real node always has a `prev` and a `next`. A dict from key to node gives $O(1)$ access to any node. An LRU cache uses this design.

```
class Node:
    __slots__ = ("key", "prev", "next")
    def __init__(self, key=None):
        self.key, self.prev, self.next = key, None, None

class Recent:                           # the most recent key is at the front
    def __init__(self):
        self.head, self.tail = Node(), Node()
        self.head.next, self.tail.prev = self.tail, self.head
        self.nodes = {}
    def _remove(self, node):
        node.prev.next, node.next.prev = node.next, node.prev
    def _add_front(self, node):
        node.prev, node.next = self.head, self.head.next
```

```
        self.head.next.prev = node
        self.head.next = node
    def touch(self, key):
        if key in self.nodes:
            self._remove(self.nodes[key])
        else:
            self.nodes[key] = Node(key)
        self._add_front(self.nodes[key])
    def keys(self):
        out, cur = [], self.head.next
        while cur is not self.tail:
            out.append(cur.key)
            cur = cur.next
        return out

r = Recent()
for k in "abca":
    r.touch(k)
r.keys()                                # -> ['a', 'c', 'b']
r.tail.prev.key                         # -> 'b'
```

The least recent key is `tail.prev`. To evict it, call `_remove(tail.prev)` and delete its key from the dict.

28.12 Enum

An `Enum` gives names to a fixed set of constant values. Each member has a `name` and a `value`.

```
from enum import Enum
class Dir(Enum):
    UP = (-1, 0)
    DOWN = (1, 0)
Dir.UP.value, Dir.UP.name           # -> ((-1, 0), 'UP')
Dir((1, 0))                         # -> <Dir.DOWN: (1, 0)>
Dir((1, 0)) is Dir.DOWN, Dir["UP"] is Dir.UP   # -> (True, True)
[d.name for d in Dir]               # -> ['UP', 'DOWN']
```

29 Exceptions

29.1 try, except, else, finally

Python runs the first `except` clause that matches the exception type. A tuple of types catches any of them. The `else` block runs only when the `try` block raises nothing. The `finally` block always runs.

```
def parse(s):
    try:
        value = int(s)
    except ValueError:
        return "bad"
    except (TypeError, OverflowError) as e:
        return type(e).__name__
    else:
        return value * 2            # runs only if no exception
    finally:
        pass                        # always runs (close files, release locks)
parse("21"), parse("x"), parse(None)   # -> (42, 'bad', 'TypeError')
```

29.2 Raise an exception

```
def sqrt_int(n):
    if n < 0:
        raise ValueError(f"negative input: {n}")
    return int(n ** 0.5)
sqrt_int(-1)                        # -> raises ValueError
try:
    sqrt_int(-4)
except ValueError as e:
    msg = str(e)
msg                                 # -> 'negative input: -4'
```

29.3 Common exceptions

Exception	Typical cause
`IndexError`	list or string index out of range; `pop()` from an empty list
`KeyError`	missing dict key; `set.remove` of a missing element
`ValueError`	right type, wrong value: `int("x")`, `list.index` of a missing value, unpack count
`TypeError`	wrong type: `"a" + 1`, unhashable key, compare `None < 1`, wrong argument count
`AttributeError`	missing attribute, often a method call on `None`
`NameError`	name not defined (spelling, scope)
`UnboundLocalError`	local name used before assignment (missing `nonlocal`)
`ZeroDivisionError`	`x / 0`, `x // 0`, `x % 0`
`RecursionError`	recursion deeper than the limit
`StopIteration`	`next()` on an empty iterator
`RuntimeError`	dict or set changed size during iteration
`OverflowError`	float result too large, for example `math.exp(1000)`

```
[][0]                 # -> raises IndexError
{}["k"]               # -> raises KeyError
int("3.5")            # -> raises ValueError
None + 1              # -> raises TypeError
None.append(1)        # -> raises AttributeError
undefined_name        # -> raises NameError
1 % 0                 # -> raises ZeroDivisionError
import math
math.exp(1000)        # -> raises OverflowError
```

29.4 EAFP and LBYL

Python code often tries an operation and handles the exception (EAFP: easier to ask forgiveness than permission). The other style checks first (LBYL: look before you leap). Both are correct.

```
d = {"a": 1}
try:
    v = d["b"]
except KeyError:
    v = 0
v                     # -> 0
v = d["b"] if "b" in d else 0
v                     # -> 0
```

29.5 Context managers

A `with` block sets up a resource and always cleans it up, also after an exception.

```
from contextlib import suppress, contextmanager
with suppress(KeyError):          # ignore one exception type
    {}["missing"]
log = []
@contextmanager
def section(name):
    log.append(f"start {name}")
    try:
        yield name
    finally:
        log.append(f"end {name}")
with section("bfs") as s:
    log.append(f"inside {s}")
log                               # -> ['start bfs', 'inside bfs', 'end bfs']
```

29.6 assert

Use `assert` to check your own assumptions while you test. `python -O` removes all `assert` statements, so do not use `assert` to check input in real code.

```
def mean(xs):
    assert xs, "empty input"
    return sum(xs) / len(xs)
mean([1, 2])          # -> 1.5
mean([])              # -> raises AssertionError
```

30 Recursion

30.1 The recursion limit

The default limit is 1000 frames. A depth-first search on a large grid or a long linked list goes deeper than this.

```
def depth(n):
    return 0 if n == 0 else 1 + depth(n - 1)
depth(500)            # -> 500
depth(5000)           # -> raises RecursionError
import sys
sys.setrecursionlimit(10**6)
depth(5000)           # -> 5000
```

CAUTION

A higher limit is not always enough, because deep recursion can also use the C stack.

- On Python 3.10 and older, a depth of a few times 10^4 can crash the interpreter with no error message.
- A function with `@cache` or `@lru_cache` runs C code at each level. On 3.11, it can crash at a similar depth. On 3.12 and 3.13, it raises `RecursionError` at a depth of about 3000, also after `sys.setrecursionlimit`.

Find the largest possible depth. With `@cache`, the safe depth is about 3000. Without it, the safe depth is about 10^4. Above that, use a loop with an explicit stack or a bottom-up table.

30.2 Convert recursion to a loop

```
def dfs_recursive(graph, u, seen):
    seen.add(u)
    for v in graph[u]:
        if v not in seen:
            dfs_recursive(graph, v, seen)
def dfs_iterative(graph, start):
    seen = {start}
```

```
    stack = [start]
    while stack:
        u = stack.pop()
        for v in graph[u]:
            if v not in seen:
                seen.add(v)        # mark when you push
                stack.append(v)
    return seen
g = {0: [1, 2], 1: [3], 2: [], 3: []}
s = set(); dfs_recursive(g, 0, s)
s == dfs_iterative(g, 0)           # -> True
```

30.3 Memoized recursion

```
from functools import cache
@cache
def paths(r, c):                   # number of grid paths to (r, c)
    if r == 0 or c == 0:
        return 1
    return paths(r - 1, c) + paths(r, c - 1)
paths(16, 16)                      # -> 601080390
```

NOTE

Define a memoized helper inside the main function when the main function gets new input each call. Then the cache starts empty for each input. A global `@cache` keeps old results.

CAUTION

Do not put `@cache` on a method of `Solution`. The cache key then includes `self`, and the cache keeps every `Solution` object alive. Put `@cache` on a nested helper (Section 9.11).

30.4 Return several values from a recursive call

Return a tuple when the parent needs more than one value from a child. This avoids global state and `nonlocal` names.

```
def check(t):                          # t is (left, right) or None
    if t is None:
        return True, 0                 # (balanced, height)
    lb, lh = check(t[0])
    rb, rh = check(t[1])
    return lb and rb and abs(lh - rh) <= 1, 1 + max(lh, rh)
check(((None, None), None))            # -> (True, 2)
check((((None, None), None), None))    # -> (False, 3)
```

30.5 Run deep recursion in a thread

On some judges, a deep recursion crashes also after `sys.setrecursionlimit`, because the C stack is too small. Run the solution in a new thread with a larger stack. Use this only when you cannot write a loop. On Python 3.12 and 3.13, a larger stack does not raise the depth limit of a `@cache` function.

```
import sys, threading
def main():
    ...                                # the recursive solution
sys.setrecursionlimit(1 << 25)
threading.stack_size(1 << 27)          # 128 MB for threads made after this call
t = threading.Thread(target=main)
t.start()
t.join()
```

31 Input and output

31.1 print

```
print(1, 2, 3)                  # prints: 1 2 3
print(1, 2, sep="-")            # prints: 1-2
print("a", end="")              # prints: a
print(*["x", "y"])              # prints: x y
print("\n".join(["l1", "l2"]))  # prints: l1 | l2
print(f"{2/3:.3f}")             # prints: 0.667
print([1, 2], (3,), {"k": 1})   # prints: [1, 2] (3,) {'k': 1}
```

31.2 Read standard input

Some assessments give the input on standard input. These patterns read it. The examples use `io.StringIO` to act as the input.

```
import io, sys
sys.stdin = io.StringIO("3\n1 2 3\nhello\n")
n = int(input())                          # one integer
nums = list(map(int, input().split()))    # integers on one line
word = input().strip()
(n, nums, word)                           # -> (3, [1, 2, 3], 'hello')
sys.stdin = sys.__stdin__
```

```
import io, sys
sys.stdin = io.StringIO("2 3\n1 2 3\n4 5 6\n")
data = sys.stdin.read().split()           # all tokens at one time (fast)
r, c = int(data[0]), int(data[1])
grid = [list(map(int, data[2 + i * c: 2 + (i + 1) * c])) for i in range(r)]
grid                                      # -> [[1, 2, 3], [4, 5, 6]]
sys.stdin = sys.__stdin__
```

```
import io, sys
sys.stdin = io.StringIO("a b\nc\n")
lines = [line.rstrip("\n") for line in sys.stdin]   # read until end of input
lines                                     # -> ['a b', 'c']
sys.stdin = sys.__stdin__
```

NOTE

`input()` is slow for 10^5 or more lines. Use `sys.stdin.readline` or read all input at one time with `sys.stdin.buffer.read().split()`. `input()` removes the newline. `readline()` keeps it.

```
import io, sys
sys.stdin = io.StringIO("2\n5 1\n7 3\n")
input = sys.stdin.readline                # a common speed trick
n = int(input())                          # int() ignores the newline
pairs = [tuple(map(int, input().split())) for _ in range(n)]
pairs                                     # -> [(5, 1), (7, 3)]
sys.stdin.readline()                      # -> ''
sys.stdin = sys.__stdin__
```

At the end of the input, `readline()` returns `""` and `input()` raises `EOFError`. Use this to read when the input does not give the number of lines.

```
import io, sys
sys.stdin = io.StringIO("4\n9\n")
total = 0
while True:
    try:
```

```
        total += int(input())
    except EOFError:                        # no more lines
        break
total                                       # -> 13
sys.stdin = sys.__stdin__
```

31.3 Fast output

```
import sys, io
buf = io.StringIO()
results = [3, 1, 4]
buf.write("\n".join(map(str, results)) + "\n")   # one write instead of many prints
buf.getvalue()                              # -> '3\n1\n4\n'
```

For many lines, build one string and call `sys.stdout.write` one time.

```
import sys
sys.stdout.write("1 2\n")                   # prints: 1 2
sys.stdout.write("\n".join(map(str, [3, 1, 4])) + "\n")   # prints: 3 | 1 | 4
```

31.4 Debug output

Write debug text to standard error. The judge reads only standard output, so the debug text does not change the answer. Remove debug output before you submit, because it costs time.

```
import io, sys
sys.stderr = io.StringIO()                  # stands in for the real standard error here
print("debug: n =", 5, file=sys.stderr)     # not part of the answer
print(42)                                   # prints: 42
sys.stderr.getvalue()                       # -> 'debug: n = 5\n'
sys.stderr = sys.__stderr__
```

31.5 Files

```
import os, tempfile
path = os.path.join(tempfile.gettempdir(), "demo.txt")
with open(path, "w") as f:                  # the file closes at the end of the block
    f.write("line1\nline2\n")
with open(path) as f:
    lines = f.read().splitlines()
lines                                       # -> ['line1', 'line2']
os.remove(path)
```

32 Performance

32.1 Time budget

CPython does about 10^7 simple operations per second. Most judges give 1 to 4 seconds per test. Use the input size to choose the algorithm before you write code.

Largest n	Algorithm that fits in Python
≤ 10	$O(n!)$
≤ 20	$O(2^n)$, $O(n \cdot 2^n)$
≤ 200	$O(n^3)$; up to about 500 if the loops cover only $i < k < j$
≤ 5000	$O(n^2)$
$\leq 10^5$	$O(n \log n)$
$\leq 10^6$	$O(n)$; $O(n \log n)$ only with the built-in sort
$\leq 10^9$	$O(\sqrt{n})$, $O(\log n)$ or a formula
$\leq 10^{18}$	$O(\log n)$ or a formula

32.2 Cost of common operations

Type	Fast: $O(1)$ (average)	Slow: $O(n)$ or more
`list`	`a[i]`, `a[i] = x`, `append`, `pop()`, `len`	`x in a`, `insert(0, x)`, `pop(0)`, `remove`, `index`, slice, `a + b`, `min`, `max`, `sum`
`dict`	get, set, delete, `k in d`, `len`	iterate, copy
`set`	`add`, `discard`, `x in s`	iterate, copy; `a & b` is $O(\min(\|a\|, \|b\|))$
`deque`	`append`, `appendleft`, `pop`, `popleft`	`q[i]` in the middle
`heapq`	`h[0]`	`heappush`, `heappop`: $O(\log n)$; `heapify`: $O(n)$
`str`	`s[i]`, `len`	slice, `+`, `in`, `find`, `replace`, `join`
`sorted`	–	$O(n \log n)$

32.3 Speed rules

- Convert a list to a `set` before many membership tests.
- Use a list with integer indexes, not a dict, when the keys are `0..n-1`.
- Use built-in functions (`sum`, `max`, `sorted`, `str.join`, `Counter`, slicing). They run in C.
- Move work out of inner loops. Store `grid[r]` in a local name before the inner loop.
- Keep a running total. Do not call `sum(path)` in each recursive call.
- Do not call `copy.deepcopy` in a loop. Copy only what changes.
- Do not `print` inside a hot loop.
- Use a big integer as a bitset: `bits |= bits << x` finds subset sums fast.

```
nums = [3, 5, 7]
bits = 1                          # bit s is set when sum s is possible
for x in nums:
    bits |= bits << x
[s for s in range(16) if bits >> s & 1]   # -> [0, 3, 5, 7, 8, 10, 12, 15]
```

32.4 Make a hot loop faster

These changes do not change the complexity. They make a tight inner loop faster by a constant factor.

- Store a row in a local name: `row = grid[r]`, then read `row[c]`.
- Compute values that do not change in the inner loop before the loop.
- Use `if v > best: best = v` in place of `best = max(best, v)`. A function call costs more than a comparison.
- Store a bound method in a local name, for example `push = res.append`.
- Use a bound method as a sort key (`key=x.__getitem__`) in place of a lambda.
- Pass index bounds (`lo`, `hi`) to a recursive call. Do not pass slices: each slice copies its elements.

```
grid = [[1, 2, 3], [4, 5, 6]]
best = float("-inf")
for row in grid:                  # one lookup for each row
    for v in row:
        if v > best:              # compare and assign
            best = v
best                              # -> 6
```

```
vals, total = [2, 3, 4], 0
for l in range(3):
    left = vals[l]                # computed one time for each l
    for r in range(l + 1, 3):
        total += left * vals[r]
total                             # -> 26
res = []
push = res.append                 # a bound method in a local name
for i in range(3):
    push(i)
res                               # -> [0, 1, 2]
def total_slices(a):              # O(n^2): each call copies a[1:]
    return 0 if not a else a[0] + total_slices(a[1:])
def total_bounds(a, lo=0):        # O(n): the call passes an index
    return 0 if lo == len(a) else a[lo] + total_bounds(a, lo + 1)
total_slices([1, 2, 3]), total_bounds([1, 2, 3])   # -> (6, 6)
```

32.5 Measure time

```
import time, timeit
start = time.perf_counter()
total = sum(range(10**5))
elapsed = time.perf_counter() - start
elapsed < 5                       # -> True
t = timeit.timeit("x in s", setup="s = set(range(1000)); x = 999", number=1000)
t < 5                             # -> True
```

33 Common mistakes

Each item gives the mistake and the correct form. The sections above have more information.

Mistake	Wrong	Correct
2D list shares rows	`[[0] * c] * r`	`[[0] * c for _ in range(r)]`
Mutable default argument	`def f(a=[])`	`def f(a=None)`
Store a reference, not a copy	`res.append(path)`	`res.append(path[:])`
Sort result is None	`a = a.sort()`	`a.sort()` or `a = sorted(a)`
Empty set	`s = {}`	`s = set()`
One-element tuple	`(5)`	`(5,)`
Integer division	`7 / 2` gives `3.5`	`7 // 2`
Truncate toward zero	`-7 // 2` gives `-4`	`int(-7 / 2)`
Round half up	`round(2.5)` gives `2`	`math.floor(x + 0.5)`
Float equality	`a == b`	`math.isclose(a, b)`
Test for missing value	`if not x:`	`if x is None:`
Compare with None	`x == None`	`x is None`
Membership in a list in a loop	`x in lst`	`x in set_of_lst`
Queue with a list	`q.pop(0)`	`deque.popleft()`
String building	`s += c` in a loop	`"".join(parts)`
Read a defaultdict key	`if d[k]:`	`if k in d:`
Change dict while iterating	`for k in d: del d[k]`	`for k in list(d):`
Rebind enclosing name	`count += 1`	`nonlocal count` first
Lambda in a loop	`lambda: i`	`lambda i=i: i`
Strip a substring	`s.strip("abc")`	`s.removesuffix("abc")`
Split on spaces	`s.split(" ")`	`s.split()`
Iterator used two times	`list(z); list(z)`	`z = list(zip(...))`
Deep recursion	recursive DFS on a big grid	explicit stack

Mistake	Wrong	Correct
Heap ties with objects	`(p, node)`	`(p, counter, node)`
Max of an empty list	`max([])`	`max(a, default=0)`
Shadow a built-in	`sum = 0`, `list = []`	`total = 0`, `items = []`
Bitwise vs comparison	`x & 1 == 0` works, but	`(x & 1) == 0` is clearer

```
sum = 0                    # shadows the built-in sum
sum([1, 2])                # -> raises TypeError
del sum
sum([1, 2])                # -> 3
```

34 Features by Python version

Some judges use an old version. Check the version before you use these features. The examples in this document are tested on Python 3.13, so they do not use the 3.14 features.

Version	Features
3.6	f-strings; `_` in number literals
3.7	dict keeps insertion order (guaranteed); `dataclasses`
3.8	`:=`; `f"{x=}"`; positional-only `/`; `math.comb`, `math.perm`, `math.prod`, `math.isqrt`, `math.dist`; `pow(a, -1, m)`; `accumulate(initial=)`
3.9	`d1 | d2`; `list[int]` hints; `str.removeprefix`, `removesuffix`; `math.lcm`; `functools.cache`
3.10	`match`; `zip(strict=True)`; `itertools.pairwise`; `bisect key=`; `int.bit_count`; `Counter.total`; `X | Y` type hints
3.11	faster CPython; recursion does not use the C stack for Python calls; `ExceptionGroup`
3.12	`itertools.batched`; `sum()` of floats is more exact; nested quotes in f-strings
3.13	no change that matters for interview code; a better interactive interpreter (multi-line edit, color)
3.14	public max-heap functions: `heapq.heapify_max`, `heapq.heappush_max`, `heapq.heappop_max`, `heapq.heapreplace_max`, `heapq.heappushpop_max`

Part II

Solving unseen problems

35 A method for unseen problems

A new problem is usually an old technique behind new words. The method in this section finds that technique by reasoning, not by memory. It starts from a slow solution that is surely correct. Then it finds the waste in that solution and removes it. Each step has one question. Write a short answer to the question before you go to the next step.

Step	Question to ask	A good answer
1. Restate the task and the exact output	What goes in, what comes out, and what exactly does the output count or choose?	One sentence in your own words, the type of the output, and a decision for each word with two meanings.

Step	Question to ask	A good answer
2. Work small examples by hand	What is the answer on three to six elements, and what did my hand do?	Three or more input and output pairs, with edge cases. A note on the work that the hand repeats.
3. Read the constraints and set a time budget	Which complexity fits the largest input? What do the constraints give for free?	A target such as $O(n \log n)$ with the arithmetic behind it, and a list of useful facts (sorted, small values, negative values).
4. Write the brute force	What is the most direct correct solution?	Short code that follows the definition: try every candidate and test it.
5. Find the repeated or wasted work	What does the inner loop compute again and again? What does it search for?	One sentence: "for each X, the inner loop searches Y for Z". Then the kind of Z: an exact key, an extreme, a boundary, a repeated subproblem.
6. Choose the move that removes it	Which optimization move removes exactly that waste?	The name of a move from Section 37, and one reason why each other likely move fails.
7. State the invariant or the state definition	What does each variable hold at the top of each iteration? What does `dp[i]` mean?	One sentence for each variable, with a start value that makes the sentence true before the first iteration.
8. Code from the invariant	Which lines keep each sentence of step 7 true?	Code where each line maps to one part of the invariant. The order of the lines follows from it.
9. Test against the brute force	Do the fast and the slow solutions agree on many small inputs?	A seeded random test with small sizes and small values, and zero mismatches.

Short form to remember: restate, examples, budget, brute force, waste, move, invariant, code, test.

NOTE

Steps 1 to 3 take a few minutes, and they prevent most wrong turns. Steps 4 to 7 are where the idea appears. Do not skip step 4. The brute force is the reference for step 9. It is also a correct answer when no faster idea comes. A correct slow solution is worth more than a fast wrong one.

35.1 The running example

The steps below use one problem: Subarray Sums Divisible by K (LeetCode 974).

Statement. You get an integer list `nums` and an integer `k`. Return the number of non-empty subarrays whose sum is divisible by `k`. Constraints: $1 \le n \le 3 \cdot 10^4$, $-10^4 \le$ `nums[i]` $\le 10^4$, $2 \le k \le 10^4$. Example: `nums = [4, 5, 0, -2, -3, 1]` and `k = 5` give 7.

35.2 Step 1: Restate the task and the exact output

Ask. What is the input? What is the output? Which objects does the output count or choose?

A good answer. Write one sentence in your own words. Write the type of the output: a count, an index, a list, a boolean or a string. Write what to return when no answer exists. Then decide each word that has two meanings:

- contiguous (subarray, substring) or not (subsequence, subset);
- order matters (sequence, permutation) or not (combination, set);
- strict ("greater") or not strict ("at least");
- distinct values or distinct positions;
- empty answer permitted or not;
- ends that touch: do `[1, 2]` and `[2, 3]` overlap or not;

- indexes that start at 0 or at 1, in the input and in the output.

Example. "Count the pairs (i, j) with $i \le j$ such that $nums[i] + \cdots + nums[j]$ is a multiple of k. Return an int." Decisions: a subarray is contiguous and non-empty. The count is over positions, so two equal subarrays at different places count two times. A zero sum and a negative sum such as -5 are multiples of 5.

35.3 Step 2: Work small examples by hand

Ask. What is the answer on three to six elements? Which operations did my hand do?

A good answer. Three or more input and output pairs that you computed yourself. Include the edge cases: one element, all equal values, negative values, no answer. These pairs become your first tests. Also write down the work that your hand repeats. That work is often the key to the fast solution.

Example. For `[4, 5, 0, -2, -3, 1]` and $k = 5$, the good subarrays are `[5]`, `[5, 0]`, `[0]`, `[5, 0, -2, -3]`, `[0, -2, -3]`, `[-2, -3]` and the whole list: 7. To get each sum, the hand reuses a running total. The sum of $nums[i..j]$ is the running total after j minus the running total before i. Edge cases: `[5]` with $k = 9$ gives 0. `[-5]` with $k = 5$ gives 1. `[0, 0]` with $k = 3$ gives 3.

35.4 Step 3: Read the constraints and set a time budget

Ask. Which complexity fits the largest input? What else do the constraints allow or forbid?

A good answer. A target complexity, with the arithmetic that supports it. A list of facts that the constraints give for free: sorted input, small values, a small k, negative values. Section 36 has the full tables.

Example. $n \le 3 \cdot 10^4$, so there are about $n^2/2 = 4.5 \cdot 10^8$ subarrays. CPython does about 10^7 simple steps per second, so $O(n^2)$ needs about 45 seconds. The target is $O(n)$ or $O(n \log n)$. The values can be negative. Thus a running sum can go down, and a sliding window on sums does not work. Also $k \le 10^4$, so a list of size k is cheap.

35.5 Step 4: Write the brute force

Ask. What is the most direct correct solution, with no attention to speed?

A good answer. Code that follows the definition: try every candidate and test it. Keep it simple, because it is the reference for step 9. Remove only the obvious waste, such as a sum that you compute again from zero for each subarray.

Example. Try every left end. Extend the right end and keep a running sum. This is $O(n^2)$ time and $O(1)$ extra space: correct, and too slow for the limits.

```
def count_div_brute(nums, k):
    n, count = len(nums), 0
    for i in range(n):                  # left end
        total = 0
        for j in range(i, n):           # right end: extend the running sum
            total += nums[j]
            if total % k == 0:
                count += 1
    return count

count_div_brute([4, 5, 0, -2, -3, 1], 5)   # -> 7
count_div_brute([5], 9)                    # -> 0
count_div_brute([-5], 5)                   # -> 1
count_div_brute([0, 0], 3)                 # -> 3
```

35.6 Step 5: Find the repeated or wasted work

Ask. What does the inner loop compute again and again? What does it search for?

A good answer. One sentence of the form "for each X, the inner loop searches Y for Z". Then classify Z, because the kind of Z selects the move:

- an exact key or value: remember what you saw (hashing);

- an extreme of a range (a minimum or maximum): keep a running best or precompute it;
- a boundary in ordered data: binary search, two pointers or a monotonic stack;
- a subproblem that the recursion solves many times: memoization or DP.

The table at the start of Section 37 lists all twelve kinds of waste and the move for each one.

Example. Let $P[i]$ be the sum of the first i elements, with $P[0] = 0$. Then the sum of $nums[i..j]$ is $P[j+1] - P[i]$. This difference is a multiple of k exactly when $P[i]$ and $P[j+1]$ have the same remainder modulo k. So for each right end, the inner loop searches the earlier prefixes for one exact key: the remainder of the current prefix.

```
from collections import Counter
from itertools import accumulate
from math import comb

nums, k = [4, 5, 0, -2, -3, 1], 5
P = [0] + list(accumulate(nums))         # P[i] = sum of the first i elements
P                                        # -> [0, 4, 9, 9, 7, 4, 5]
rem = [p % k for p in P]
rem                                      # -> [0, 4, 4, 4, 2, 4, 0]
P[5] - P[1], rem[5] == rem[1]            # -> (0, True)
sum(comb(c, 2) for c in Counter(rem).values())   # -> 7
```

Each pair of equal remainders is one good subarray. The remainder 0 occurs 2 times (1 pair), and the remainder 4 occurs 4 times (6 pairs). The total is 7, as in the hand count of step 2.

35.7 Step 6: Choose the move that removes it

Ask. Which optimization move removes exactly that waste? Why do the other moves fail?

A good answer. The name of a move from Section 37, and one reason for each rejected alternative. A rejected alternative with a reason is useful: it shows which property of the problem matters.

Example. Two moves work together. *Precompute*: the running total gives each prefix sum in $O(1)$. *Remember what you saw*: a count of the remainders of earlier prefixes replaces the inner loop. Rejected moves: a sliding window needs a condition that stays true when the window shrinks. Divisibility does not: `[2, 3]` has the sum 5, but `[3]` has the sum 3. Negative values also let the sum go down. A sort loses the order $i < j$. A binary search needs sorted data, and the prefix sums are not sorted.

35.8 Step 7: State the invariant or the state definition

Ask. What does each variable hold at the top of each loop iteration? For a DP: what does one entry of the table mean, in words?

A good answer. One sentence for each variable, with its start value. The start value must make the sentence true before the first iteration. If you cannot write the sentence, you cannot yet write the code.

Example. At the top of the iteration for element j:

- `total` is $P[j]$, the sum of the first j elements;
- `seen[r]` is the number of prefixes $P[0], \ldots, P[j]$ with remainder r;
- `count` is the number of good subarrays that end before index j.

Start values: `total = 0`, `count = 0`, and `seen = {0: 1}`, because the empty prefix $P[0] = 0$ exists before the first element.

35.9 Step 8: Code from the invariant

Ask. Which lines keep each sentence of step 7 true?

A good answer. Code where each line maps to one part of the invariant. The invariant also gives the order of the lines: read the state, then update it.

Example. Add the element to `total`, so that `total` is $P[j+1]$. Each earlier prefix with the same remainder gives one good subarray that ends at j, so add `seen[r]` to `count`. Then add the new prefix to `seen`. Now the invariant is true for $j+1$. The result is $O(n)$ time and $O(\min(n, k))$ extra space.

```
from collections import defaultdict

def count_div(nums, k):
    seen = defaultdict(int)
    seen[0] = 1                          # the empty prefix P[0] = 0
    total = count = 0
    for x in nums:
        total += x                       # total is now the next prefix sum
        r = total % k                    # in Python, 0 <= r < k also for a negative total
        count += seen[r]                 # one good subarray for each earlier equal remainder
        seen[r] += 1                     # then add this prefix
    return count

count_div([4, 5, 0, -2, -3, 1], 5)       # -> 7
count_div([-5], 5)                       # -> 1
count_div([0, 0], 3)                     # -> 3
```

CAUTION

Read the state before you update it. If `seen[r] += 1` comes before `count += seen[r]`, each prefix pairs with itself, and the code counts empty subarrays.

CAUTION

In Python, `x % k` is in `0..k-1` when $k > 0$, also for a negative `x`. In C++ and Java the remainder of a negative value can be negative. There, use $((x \bmod k) + k) \bmod k$.

35.10 Step 9: Test against the brute force

Ask. Do the fast and the slow solutions agree on many small random inputs and on the edge cases?

A good answer. A short loop with a fixed seed. Use small sizes and a small value range, so that equal values, zeros and edge cases occur often. The loop finds zero mismatches. On a mismatch, keep the smallest failing input and trace it by hand.

```
import random
from collections import defaultdict

def count_div_brute(nums, k):            # step 4
    count = 0
    for i in range(len(nums)):
        total = 0
        for j in range(i, len(nums)):
            total += nums[j]
            count += total % k == 0
    return count

def count_div(nums, k):                  # step 8
    seen, total, count = defaultdict(int), 0, 0
    seen[0] = 1
    for x in nums:
        total += x
        count += seen[total % k]
        seen[total % k] += 1
    return count

random.seed(7)
bad = []
for _ in range(500):
    nums = [random.randint(-10, 10) for _ in range(random.randint(1, 8))]
    k = random.randint(2, 6)
    if count_div(nums, k) != count_div_brute(nums, k):
```

```
        bad.append((nums, k))
bad                                     # -> []
```

NOTE

Step 5 can find no waste that you can remove. Then do two checks. First, compare the brute force with the budget of step 3: sometimes the brute force already fits. Second, go to Section 40 and try the reframings in order.

36 Read the constraints

The constraints are part of the hint. The problem author picks the limits so that the intended solution passes and a slower solution fails. Read them before you choose an approach. They tell you the target complexity, and they often tell you which properties of the input you can use.

36.1 Input size and feasible complexity in CPython

CPython does about 10^7 simple steps per second. A simple step is one loop iteration with a few arithmetic operations, or one dict access. Built-in functions that run in C (`sorted`, `sum`, `Counter`, string methods, slicing) are 10 to 50 times faster for each element. Most judges give 1 to 4 seconds for each test. Plan for about 10^7 Python steps. Between $2 \cdot 10^7$ and $4 \cdot 10^7$ steps, the code passes only with simple loop bodies and built-ins. At 10^8 steps, the code takes about 10 seconds. The table extends the time-budget table in Section 32.1.

Largest n	Feasible in CPython	Steps at the largest n	Typical techniques
≤ 10	$O(n!)$; $O(n! \cdot n)$ is borderline	$10! \approx 3.6 \cdot 10^6$	All permutations; try every order (Section 54).
≤ 20	$O(2^n)$; $O(2^n \cdot n)$ is borderline	$2^{20} \approx 10^6$	All subsets, bitmask DP over the set of used items.
≤ 40	$O(2^{n/2} \cdot n)$	$2^{20} \cdot \log_2 2^{20} \approx 2 \cdot 10^7$	Meet in the middle: enumerate each half, then combine with sort or hashing.
≤ 200	$O(n^3)$; $O(n^4)$ only up to about 50	$8 \cdot 10^6$	Three nested loops, Floyd–Warshall, DP with a split point.
≤ 500	$O(n^3)$ only with loops over $i < k < j$; $O(n^2 \log n)$	$500^3/6 \approx 2 \cdot 10^7$	Interval DP (Burst Balloons has $n \leq 300$).
≤ 5000	$O(n^2)$	$2.5 \cdot 10^7$	Two-sequence DP tables, all pairs, DP over (index, previous index).
$\leq 10^5$	$O(n \log n)$; $O(n\sqrt{n})$ is borderline	$1.7 \cdot 10^6$	Sort, heap, binary search, two pointers, sliding window, monotonic stack.
$\leq 10^6$	$O(n)$; $O(n \log n)$ only with the built-in sort	10^6	One pass with $O(1)$ work for each element. Prefer built-ins.
$\leq 10^9$	$O(\sqrt{n})$, $O(\log n)$	$3 \cdot 10^4$	Math, divisor loops, binary search on the answer.
$\leq 10^{18}$	$O(\log n)$	60	Fast power, binary search with integer math, digit-by-digit methods.

Use the arithmetic, not a feeling. Multiply the sizes of nested loops. Count each query separately.

```python
import math
STEPS_PER_SECOND = 10**7                # simple CPython steps, a rough figure

def seconds(steps):
    return steps / STEPS_PER_SECOND

n = 10**5
seconds(n * n)                          # -> 1000.0
round(seconds(n * math.log2(n)), 2)     # -> 0.17
q = 10**5                               # q queries, each one O(n): too slow
seconds(q * n) > 60                     # -> True
seconds(n + q * math.log2(n)) < 1       # -> True
```

- With two inputs of sizes m and n, the budget uses both: $O(m \cdot n)$ with $m, n \le 10^3$ is 10^6 and fits.
- With q queries, the cost is the build cost plus q times the cost of one query.
- A table of $10^4 \times 10^4$ cells is 10^8 entries, about 800 MB of list slots in CPython. It does not fit a typical memory limit. Keep only the rows that the recurrence reads.
- A chain of 10^4 nodes (a list, a skewed tree, a path graph) is deeper than the default recursion limit of 1000. Use an explicit stack.

36.2 Other signals in the constraints and the statement

Some constraints say more than a size. They permit a structure, or they forbid one.

Signal	What to try	Why it helps
The input is sorted, or "non-decreasing"	Binary search; two pointers; a merge instead of a new sort	One comparison discards a half or one end.
Small values, for example $0 \le v \le 10^4$	A list indexed by value: counting sort, buckets, DP over values	No hashing and no comparison sort: $O(n + \text{range})$.
Values up to 10^9 and $n \le 10^5$	A dict or a set; sort plus coordinate compression	A list over the value range does not fit in memory.
A small alphabet (26 lowercase letters)	A list of 26 counts; a 26-bit mask; $O(26n)$ loops	A fixed-size state compares in constant time.
k much smaller than n (top k, window of k)	A heap of size k: $O(n \log k)$; a window of size k	Keep only k candidates, not all n.
$n \le 20$, or $n \le 16$	Subsets, bitmask DP, backtracking	$2^{20} \approx 10^6$ states fit.
$n \le 40$	Meet in the middle	Each half has only 2^{20} subsets.
A grid with $R \cdot C$ up to 10^5 or 10^6 cells	$O(1)$ work for each cell: BFS, flood fill, grid DP; an explicit stack	One region can hold every cell, and recursion then fails.
$R, C \le 300$ and the answer is a size	Binary search on the size plus 2-D prefix sums	$O(RC \min(R, C)) \approx 2.7 \cdot 10^7$ is slow in Python.
Up to 10^5 queries on static data	Precompute once: prefix sums, sort plus `bisect`, tables	Each query then costs $O(1)$ or $O(\log n)$.
You get all queries in advance	Answer offline: sort the queries and sweep	A pointer over sorted queries moves in one direction only.
Updates between the queries	A heap, a sorted list, union-find or a Fenwick tree (Section 39)	A precomputed table becomes wrong after an update.
"Return any valid answer", "in any order"	Build one answer directly: a greedy, any topological order	You do not search for a special answer.
"Return the answer modulo $10^9 + 7$"	Count with DP or combinatorics; take the modulo after each addition	The count is too large to enumerate.
"The answer fits in a 32-bit integer" (for a count)	Count with DP	The number of objects can be near $2 \cdot 10^9$.

Signal	What to try	Why it helps
Values can be negative	An exact sum: prefix sums plus a hash map. A shortest segment with a sum at least k: a deque of prefix indexes (Section 36.3). A best sum: Kadane; DP	A sliding window on sums and Dijkstra need non-negative values. A hash map finds only exact keys.
All values are positive	A sliding window on sums; binary search on prefix sums; an early break in backtracking	A sum grows when the range grows.
The values are distinct	Simple comparisons with no tie rules	Equal values break some binary searches and stack rules.
A step count or a budget up to 10^9 or 10^{18}	A formula, a division, cycle detection plus modulo, fast power	A step-by-step simulation does not finish.
"At most k" changes, deletions or stops	A budget inside a window; an extra DP dimension; $k+1$ rounds of relaxation	The budget is part of the state.
"Exactly k"	Count "at most k" minus "at most $k-1$"	"At most" stays true when a window shrinks. "Exactly" does not.
Follow-up: $O(1)$ extra space	Two pointers; marks in the input; the output list as storage; Floyd; XOR or sum	The bookkeeping goes into the input or into a few variables.
Follow-up: $O(n)$ when a sort gives $O(n \log n)$	Hashing, bucket sort, a monotonic stack, union-find	These remove the comparison sort.
Required $O(\log n)$	Binary search on the index or on the answer	A search that halves the range at each step takes logarithmic time.
A tiny n with many detailed rules	Simulate the rules literally; test each rule	Speed is not the risk. Correctness is.
V and E up to 10^5	Adjacency lists; $O(V+E)$ traversal; $O(E \log V)$ Dijkstra	An adjacency matrix needs V^2 cells.
A complete graph from a formula, $V \le 10^3$	$O(V^2)$ Prim with weights computed on demand	A list of about $5 \cdot 10^5$ edges is slow to build and sort.

36.3 A range-sum inequality with negative values

With negative values, a running sum can go down. Thus a sliding window on sums fails. A hash map also fails, because it finds only an exact key, not "at least k". For the shortest segment with a sum at least k, keep a deque of prefix indexes whose prefix values increase. Two rules keep the deque small:

- Front: while the current prefix minus the front prefix is at least k, record the length and remove the front. A later end gives only a longer segment for that start.
- Back: remove each index whose prefix is not lower than the current prefix. The current index starts later with a lower or equal prefix, so it is always a better or equal start.

Each index enters and leaves the deque one time, so the scan is $O(n)$ (Shortest Subarray with Sum at Least K, LeetCode 862).

```
import random
from collections import deque
from itertools import accumulate

def shortest_at_least(a, k):              # values can be negative
    P = [0] + list(accumulate(a))
    best = len(a) + 1
    starts = deque()                      # prefix indexes; their P values increase
```

```
    for j, p in enumerate(P):
        while starts and p - P[starts[0]] >= k:
            best = min(best, j - starts.popleft())   # no later end beats this start
        while starts and P[starts[-1]] >= p:
            starts.pop()                 # a later, lower prefix is a better start
        starts.append(j)
    return best if best <= len(a) else -1

def shortest_brute(a, k):
    n = len(a)
    lens = [j - i + 1 for i in range(n) for j in range(i, n) if sum(a[i:j + 1]) >= k]
    return min(lens, default=-1)

shortest_at_least([1, -5, 4, 3], 7)      # -> 2
shortest_at_least([84, -37, 32, 40, 95], 167)   # -> 3
random.seed(3)
cases = [([random.randint(-9, 9) for _ in range(random.randint(1, 8))], random.randint(1, 15))
         for _ in range(500)]
all(shortest_at_least(a, k) == shortest_brute(a, k) for a, k in cases)   # -> True
```

37 From brute force to fast: the optimization moves

A fast solution is a brute force minus some waste. The waste has few forms. This section gives twelve moves, and each move removes one form of waste. The table names the Part III sections by a short form of their titles and by their numbers. First find the waste (step 5 of the method). Then pick the move that removes it. Often two moves work together, as in the running example: precompute, then remember what you saw.

Move	Waste that it removes	Main sections in Part III (number)
Remember what you saw	An inner loop that searches earlier items for an exact value	Hashing (44)
Precompute	A range aggregate that you compute again for each query or index	Prefix sums (45)
Sort first	Comparisons between all pairs when only neighbors in order matter	Intervals (63); Two pointers (46); Greedy (62)
Exploit monotonic order	A scan over candidates that one comparison already ruled out	Two pointers (46); Binary search (49); Sliding window (47); Stacks and monotonic stacks (48)
Keep only the best candidates	Storage or a sort of all items when only the top or the current best matters	Heaps and priority queues (53); Greedy (62)
Reuse subproblems	A recursion that solves the same subproblem many times	The four DP sections (58 to 61)
Change the representation	A representation that hides the structure	Hashing (44); Graph traversal (55); Bit manipulation (66)
Work backward or from the complement	A search from every source, or a direct test of a hard property	Graph traversal (55); Intervals (63); DP on grids, intervals, games and states (61)
Fix one part and solve the rest	Nested loops over all parts of a tuple	Two pointers (46); Trees (51); Stacks and monotonic stacks (48)
Count by contribution	A list of all structures (subarrays, pairs, paths, moves) only to add up one value for each	Stacks and monotonic stacks (48); Bit manipulation (66); Trees (51)

Move	Waste that it removes	Main sections in Part III (number)
Track offsets or extremes	A simulation that moves every item at every step	Prefix sums (45); Stacks and monotonic stacks (48); Greedy (62)
Prune the search	Branches that cannot give a valid or better answer	Backtracking (54); Tries (52)

37.1 Remember what you saw (hashing)

Waste it removes. For each item, an inner loop searches the earlier items for one exact value: a partner, a duplicate, or an earlier prefix with the same key.

Signal. The partner of an item is fully known from the item: $t - x$, $x - 1$, the same remainder, the same sorted word. The relation is an equality, not "near" or "less than".

The move. Keep a set or a dict of what you saw. Ask it for the partner, then add the current item. Store the value that the question needs: a flag (set), an index (dict), or a count (`Counter`).

```
from collections import Counter

def count_pairs_brute(a, t):              # pairs i < j with a[i] + a[j] == t
    n = len(a)
    return sum(1 for i in range(n) for j in range(i + 1, n) if a[i] + a[j] == t)

def count_pairs(a, t):
    seen, count = Counter(), 0
    for x in a:
        count += seen[t - x]              # partners among the earlier items
        seen[x] += 1                      # then remember this item
    return count

a = [1, 5, 3, 3, 7, 1]
count_pairs_brute(a, 6), count_pairs(a, 6)    # -> (3, 3)
```

Why it is faster. The brute force compares each item with all earlier items: about $n^2/2$ tests. The partner is one exact key, so one dict lookup finds it in $O(1)$ on average. The dict holds all earlier items at the same time, so one scan is enough: $O(n)$. In the example, the brute force tests 15 pairs, and the scan does 6 lookups.

Problems. Two Sum; Contains Duplicate; Longest Consecutive Sequence; Subarray Sum Equals K; Continuous Subarray Sum (LeetCode 523).

CAUTION

Hashing finds only exact matches. When the inner loop looks for a value in a range ("the nearest", "at most d away"), sort the data or use buckets.

37.2 Precompute (prefix and suffix data)

Waste it removes. A sum, product, minimum or maximum over a range that you compute again for each query or for each index.

Signal. Many range sums on data that does not change. An answer for each index that uses "everything on the left" and "everything on the right". The words "except self".

The move. Build the prefix aggregate one time in $O(n)$. Then each range sum is a difference of two prefix values. For an operation with no inverse (max, min, a product with zeros), build a prefix list and a suffix list, and combine the two sides.

```python
from itertools import accumulate

a = [3, 1, 4, 1, 5, 9, 2]
P = [0] + list(accumulate(a))            # P[i] = sum(a[:i])

def range_sum(l, r):                     # sum of a[l..r], both ends included
    return P[r + 1] - P[l]

range_sum(2, 4), sum(a[2:5])             # -> (10, 10)
left = list(accumulate(a, max))                  # max of a[0..i]
right = list(accumulate(reversed(a), max))[::-1] # max of a[i..n-1]
left                                     # -> [3, 3, 4, 4, 5, 9, 9]
right                                    # -> [9, 9, 9, 9, 9, 9, 2]
```

Why it is faster. A range sum in the brute force adds up to n items. Ranges that overlap add the same items again and again. The prefix list adds each item one time, and all ranges share it. Thus q range sums cost $O(n+q)$, not $O(nq)$. In the example, `range_sum(2, 4)` is one subtraction: $P[5] - P[2] = 14 - 4 = 10$.

Problems. Product of Array Except Self; Trapping Rain Water; Range Sum Query - Immutable (LeetCode 303); Maximum Side Length of a Square with Sum Less than or Equal to Threshold (2-D prefix sums).

37.3 Sort first

Waste it removes. A comparison of all pairs, when after a sort only neighbors can matter.

Signal. The answer does not depend on the input order. The question is about overlaps, closest values, groups of equal values, or a greedy choice of "the smallest first". The budget permits $O(n \log n)$.

The move. Sort, then scan one time and compare each item with its neighbor or with the last group. Sort pairs `(value, index)` when the output needs the original indexes.

```python
def min_gap_brute(a):
    return min(abs(x - y) for i, x in enumerate(a) for y in a[i + 1:])

def min_gap(a):
    b = sorted(a)                        # the closest pair is now adjacent
    return min(y - x for x, y in zip(b, b[1:]))

a = [40, 3, 18, 25, 7, 31]
min_gap_brute(a), min_gap(a)             # -> (4, 4)
```

Why it is faster. After a sort, the closest partner of an item is one of its two neighbors. An item that is farther away in the sorted list is also farther away in value. Thus $n - 1$ neighbor pairs replace $n(n-1)/2$ pairs. The sort costs $O(n \log n)$, and the scan costs $O(n)$. In the example, the sorted list is `[3, 7, 18, 25, 31, 40]`. Its 5 neighbor gaps replace 15 pairs.

Problems. Merge Intervals; Meeting Rooms; 3Sum; Car Fleet; Minimum Absolute Difference (LeetCode 1200).

37.4 Exploit monotonic order

Waste it removes. A scan over candidates that one comparison already ruled out.

Signal. Something only grows or only shrinks. Examples are sorted data and a sum of positive values. Other examples are a feasibility that stays true for all larger parameters, and a validity that stays true when a window shrinks. Or a new item makes some old items useless for all later questions.

The move. Pick the tool from the shape of the order:

- two ends of sorted data, and each comparison discards one end: converging two pointers;
- a true-or-false property that changes one time along an index or a value: binary search;
- a window whose best left end never moves back: a sliding window;
- "the nearest greater or smaller item": a monotonic stack; the best item of a moving window: a monotonic deque.

```
def has_pair(b, t):                       # b is sorted
    lo, hi = 0, len(b) - 1
    while lo < hi:
        s = b[lo] + b[hi]
        if s == t:
            return True
        if s < t:
            lo += 1                       # b[lo] is too small for every partner
        else:
            hi -= 1                       # b[hi] is too big for every partner
    return False

has_pair([1, 3, 4, 6, 9], 10), has_pair([1, 3, 4, 6, 9], 11)   # -> (True, False)

def next_greater(a):
    res = [-1] * len(a)
    stack = []                            # indexes that wait for a greater value
    for i, x in enumerate(a):
        while stack and a[stack[-1]] < x:
            res[stack.pop()] = x          # x answers every smaller waiting item
        stack.append(i)
    return res

next_greater([2, 1, 2, 4, 3])             # -> [4, 2, 4, -1, -1]
```

Why it is faster. Each comparison removes one candidate for all later questions, and that candidate never comes back. Thus each pointer moves at most n times in total, and each index enters and leaves a stack one time: $O(n)$. A binary search removes half of the candidates with each check: $O(\log n)$ checks. In `has_pair` with $t = 11$, the sum $1 + 9 = 10$ is too small. Thus 1 is too small for every partner, and it leaves. The search ends after 4 pointer moves. The brute force tests 10 pairs.

Problems. Two Sum II - Input Array Is Sorted; Koko Eating Bananas; Longest Substring Without Repeating Characters; Daily Temperatures; Sliding Window Maximum.

37.5 Keep only the best candidates (heap, greedy)

Waste it removes. You store or sort all items, but the answer uses only the top k items, or only the current best item.

Signal. "k largest", "k closest", "k most frequent". A process that repeatedly takes the smallest or the largest item. A choice where one candidate dominates the others, for example the farthest reach or the earliest end.

The move. Keep a heap of size k whose root is the weakest kept item. For a greedy, keep one number that summarizes all candidates. Then prove with a small exchange argument that the dominated candidates never win.

```
import heapq

def k_largest(a, k):
    h = []                                # min-heap of the k largest items so far
    for x in a:
        heapq.heappush(h, x)
        if len(h) > k:
            heapq.heappop(h)              # drop the weakest kept item
    return sorted(h, reverse=True)

k_largest([5, 1, 9, 3, 7, 6], 3)          # -> [9, 7, 6]
heapq.nlargest(3, [5, 1, 9, 3, 7, 6])     # -> [9, 7, 6]

def can_reach_end(jumps):                 # jumps[i] = longest jump from i
    far = 0                               # the farthest reachable index dominates all others
    for i, j in enumerate(jumps):
        if i > far:
```

```
            return False
        far = max(far, i + j)
    return True

can_reach_end([2, 3, 1, 1, 4]), can_reach_end([3, 2, 1, 0, 4])   # -> (True, False)
```

Why it is faster. A full sort orders all n items, but the answer uses only k of them. In a heap of size k, each push and pop costs $O(\log k)$. The total is $O(n \log k)$ time and $O(k)$ memory. A greedy removes even more work: one number summarizes all candidates, so each item costs $O(1)$. In `can_reach_end`, every index up to `far` is reachable. Thus `far` replaces the set of reachable indexes.

Problems. Kth Largest Element in an Array; K Closest Points to Origin; Merge k Sorted Lists; Meeting Rooms II; Jump Game.

CAUTION

A greedy needs a reason. Before you code a greedy, try two or three small inputs where a different choice looks better. If you find a counterexample, use DP.

37.6 Reuse subproblems (memoization, DP)

Waste it removes. A recursion that solves the same subproblem many times through different paths.

Signal. "How many ways", "minimum cost", "is it possible". A choice at each step. The future depends only on a few numbers (an index, a remaining amount), not on the full history.

The move. Write the brute-force recursion. Find the arguments that alone decide the future. Cache the function on those arguments. Later, if necessary, turn the cache into a table filled in order.

```
from functools import cache

calls = 0
def ways(n):                            # steps of size 1, 2 or 3
    global calls
    calls += 1
    if n < 0:
        return 0
    if n == 0:
        return 1
    return ways(n - 1) + ways(n - 2) + ways(n - 3)

ways(20)                                # -> 121415
calls                                   # -> 433993

@cache
def ways_memo(n):
    if n < 0:
        return 0
    if n == 0:
        return 1
    return ways_memo(n - 1) + ways_memo(n - 2) + ways_memo(n - 3)

ways_memo(20)                           # -> 121415
ways_memo.cache_info().misses           # -> 23
```

Why it is faster. The plain recursion reaches the same argument through many paths, so the number of calls grows exponentially with n. The cache computes each distinct argument one time. The cost becomes the number of states times the work for each state. In the example, the plain recursion makes 433993 calls. The cached version solves each of the 23 subproblems one time, and each one reads at most 3 cached values.

Problems. Climbing Stairs; Coin Change; Word Break; Longest Common Subsequence; Edit Distance.

37.7 Change the representation

Waste it removes. Work that a better encoding makes cheap or removes. A comparison of two sets letter by letter, a search over pairs of states, a check of a diagonal cell by cell.

Signal. The items are equal "up to" some detail. The input is a configuration, not a graph, but the question is about moves between configurations. The objects are small sets. A grid has diagonals or blocks.

The move. Pick an encoding that makes the question one operation:

- a canonical key (sorted word, count tuple, `frozenset`) groups equal items in a dict;
- a graph whose nodes are states and whose edges are single moves turns "fewest moves" into BFS;
- a bitmask turns a small set into one integer: a test for common letters is one AND;
- coordinate keys: `r * C + c` for a cell id, $r - c$ and $r + c$ for the two diagonals, `(r // 3, c // 3)` for a block.

```
from collections import defaultdict

groups = defaultdict(list)
for w in ["eat", "tea", "tan", "ate", "nat", "bat"]:
    groups["".join(sorted(w))].append(w) # canonical key: the sorted letters
sorted(groups.values())                  # -> [['bat'], ['eat', 'tea', 'ate'], ['tan', 'nat']]

def max_product(words):                  # two words with no common letter
    masks = []
    for w in words:
        m = 0
        for ch in w:
            m |= 1 << (ord(ch) - ord("a"))
        masks.append(m)
    best = 0
    for i in range(len(words)):
        for j in range(i + 1, len(words)):
            if masks[i] & masks[j] == 0: # no shared letter: one AND
                best = max(best, len(words[i]) * len(words[j]))
    return best

max_product(["abcw", "baz", "foo", "bar", "xtfn", "abcdef"])   # -> 16
```

Why it is faster. You build each encoding one time, and then every comparison uses it. The sorted-letter key replaces a comparison of each word with each group: one dict lookup for each word. The 26-bit mask replaces a letter-by-letter test of two words: one AND, in place of up to $|w_1| \cdot |w_2|$ letter comparisons. A state graph replaces a search over all move sequences: BFS visits each state one time.

Problems. Group Anagrams; Word Ladder; N-Queens; Search a 2D Matrix; Maximum Product of Word Lengths (LeetCode 318).

37.8 Work backward or from the complement

Waste it removes. One search from each source toward a fixed target. A direct test of a property that is hard to test, when its opposite is easy.

Signal. Many sources and a few fixed targets (a border, an ocean, a gate). Words such as "cannot escape", "at least one", "minimum number to remove". A choice of the first action that leaves two coupled sides.

The move. Search once from the targets on reversed edges. Count the opposite and subtract it from the total. Turn "fewest removed" into "most kept". In interval DP, choose the last action, not the first.

```
def count_with_even_brute(a):            # subarrays with at least one even value
    n = len(a)
    return sum(any(x % 2 == 0 for x in a[i:j + 1])
               for i in range(n) for j in range(i, n))

def count_with_even(a):
    n = len(a)
```

```
    total = n * (n + 1) // 2            # all subarrays
    all_odd = run = 0
    for x in a:
        run = run + 1 if x % 2 else 0   # length of the odd run that ends here
        all_odd += run                  # subarrays with only odd values that end here
    return total - all_odd

a = [1, 3, 2, 5, 7, 9, 4]
count_with_even_brute(a), count_with_even(a)   # -> (19, 19)
```

Why it is faster. One search from the targets answers the question for every source at the same time. Thus the cost is one traversal, not one traversal for each source. The complement is often a simpler shape that one pass counts. In the example, a subarray with no even value lies inside a run of odd values. A run of length m holds $m(m+1)/2$ such subarrays, and a running length counts them in $O(n)$. The runs `[1, 3]` and `[5, 7, 9]` give $3+6=9$, and the answer is $28-9=19$.

Problems. Surrounded Regions; Pacific Atlantic Water Flow; Non-overlapping Intervals; Burst Balloons.

37.9 Fix one part and solve the rest

Waste it removes. Nested loops over all parts of a tuple: a pair, a triple or a path. Often one fixed part makes the rest a known, easier problem.

Signal. The answer is a tuple or a path of a fixed shape. One part has a special role: the middle element, the middle edge, the minimum of a range, the highest node of a tree path. Once you fix that part, the other parts become independent.

The move. Loop over the special part. Solve the rest with a count, a lookup, two pointers or a precomputed table. For independent sides, multiply the counts.

```
from itertools import combinations

def teams_brute(r):                     # triples i < j < k, strictly up or strictly down
    return sum(1 for x, y, z in combinations(r, 3) if x < y < z or x > y > z)

def teams(r):
    total = 0
    for j, mid in enumerate(r):         # fix the middle soldier
        left_less = sum(x < mid for x in r[:j])
        left_more = j - left_less       # the ratings are distinct
        right_less = sum(x < mid for x in r[j + 1:])
        right_more = len(r) - 1 - j - right_less
        total += left_less * right_more + left_more * right_less
    return total

teams_brute([2, 5, 3, 4, 1]), teams([2, 5, 3, 4, 1])   # -> (3, 3)
```

Why it is faster. With the middle fixed, the left choice and the right choice do not depend on each other. Thus the number of triples through this middle is a product of two counts, and you do not list the pairs. One loop level disappears: $O(n^3)$ becomes $O(n^2)$. In the example, fix the middle value 3. On the left, 2 is smaller and 5 is larger. On the right, 1 is smaller and 4 is larger. The middle gives $1 \cdot 1 + 1 \cdot 1 = 2$ teams.

Problems. 3Sum (fix the first value); Largest Rectangle in Histogram (fix the shortest bar); Binary Tree Maximum Path Sum (fix the highest node); Detect Squares (fix the diagonal corner); Count Number of Teams (LeetCode 1395).

37.10 Count by contribution

Waste it removes. You list every subarray, pair, path or move only to add up one value for each.

Signal. A sum over all subarrays, all substrings, all pairs or all paths. The total number of unit moves of items along the edges of a tree. There are $O(n^2)$ or more structures, but n is up to 10^5.

The move. Swap the order of the sum. For each element, bit or edge, count the structures where it counts. Multiply that count by the value of the element, and add the products. When the value of a structure is its minimum or maximum, a monotonic stack gives the reach of each element on each side.

```
def sum_all_subarrays_brute(a):
    n = len(a)
    return sum(sum(a[i:j + 1]) for i in range(n) for j in range(i, n))

def sum_all_subarrays(a):
    n = len(a)
    return sum(x * (i + 1) * (n - i) for i, x in enumerate(a))   # starts times ends

a = [3, 1, 4, 2]
sum_all_subarrays_brute(a), sum_all_subarrays(a)   # -> (50, 50)
```

Why it is faster. The brute force lists $n(n+1)/2$ subarrays and adds up to n items for each one. The element $a[i]$ is in each subarray that starts at or before i and ends at or after i. The start and the end are independent, so there are $(i+1)(n-i)$ such subarrays. Thus one pass with $O(1)$ work for each element gives the total: $O(n)$. In the example, the value 4 at index 2 is in $3 \cdot 2 = 6$ subarrays, so it adds 24.

Problems. Vowels of All Substrings (LeetCode 2063); Sum of All Odd Length Subarrays (LeetCode 1588); Count Unique Characters of All Substrings of a Given String (LeetCode 828: an occurrence counts in each substring that holds no other occurrence of its letter); Distribute Coins in Binary Tree (LeetCode 979: the coins that cross an edge equal the absolute value of the coins minus the nodes below that edge).

37.11 Track offsets or extremes instead of simulating every step

Waste it removes. A simulation that moves every item at every step. Or a step-by-step replay of a process when only a summary decides the answer.

Signal. The same move applies to all items at the same time. An item survives only if it stays inside bounds at every step. A step count or a budget is too large to simulate. Objects cannot pass each other, so only a finish value per object matters.

The move. Keep one shared offset instead of the positions. Keep its running minimum and maximum. The surviving starts form one interval. In other problems, replace the motion of each object by one number, such as a finish time, and scan with a running maximum.

```
def count_starts_brute(diffs, lower, upper):
    good = 0
    for start in range(lower, upper + 1):
        x, ok = start, True
        for d in diffs:
            x += d
            ok = ok and lower <= x <= upper
        good += ok
    return good

def count_starts(diffs, lower, upper):
    off = lo = hi = 0                    # offset from the start, and its extremes
    for d in diffs:
        off += d
        lo, hi = min(lo, off), max(hi, off)
    return max(0, (upper - hi) - (lower - lo) + 1)

count_starts_brute([1, -3, 4], 1, 6), count_starts([1, -3, 4], 1, 6)   # -> (2, 2)
```

Why it is faster. The brute force replays all steps for each start: starts times steps. All starts move by the same amount, so their distances never change, and one offset describes all of them. A start s is valid when $s + lo \geq lower$ and $s + hi \leq upper$. The cost is $O(n)$ for any range of starts. In the example, the offsets are 1, -2 and 2, so $lo = -2$ and $hi = 2$. With $lower = 1$ and $upper = 6$, the valid starts are $3 \leq s \leq 4$.

Problems. Count the Hidden Sequences (LeetCode 2145); Car Fleet; Gas Station; Best Time to Buy and Sell

Stock; Jump Game.

37.12 Prune the search

Waste it removes. Branches of a search that cannot give a valid or a better answer.

Signal. The task asks for all solutions, or the input is tiny, so a search is the plan. A partial answer can already break a rule: a sum above the target, a conflict, a prefix that no word has.

The move. Test the rule on the partial answer, and stop the branch at once. Sort the choices so that one failure ends the loop (`break`, not `continue`). Keep the conflict data in sets or a trie, so that each test is $O(1)$ or one step.

```
def subsets_with_sum(vals, target, prune):
    vals = sorted(vals)
    nodes = 0
    def dfs(start, remain):
        nonlocal nodes
        nodes += 1
        found = 1 if remain == 0 else 0
        for i in range(start, len(vals)):
            if prune and vals[i] > remain:
                break                    # sorted: every later value is too big too
            found += dfs(i + 1, remain - vals[i])
        return found
    return dfs(0, target), nodes         # (number of subsets, nodes visited)

subsets_with_sum(range(1, 13), 10, prune=False)   # -> (10, 4096)
subsets_with_sum(range(1, 13), 10, prune=True)    # -> (10, 43)
```

Why it is faster. A stopped branch removes its whole subtree, not one node. A cut near the root removes the most work. In the example, the plain search visits all $2^{12} = 4096$ subsets. The pruned search visits 43 nodes and finds the same 10 subsets.

Problems. Combination Sum; N-Queens; Word Search; Word Search II (the trie prunes); Partition to K Equal Sum Subsets.

38 Signal-to-technique table

Use this table after step 3 of the method. Find the rows that match the statement and the constraints. More than one row can match. Then confirm the technique with steps 4 to 6: a match of words is a hint, not a proof. The third column gives the section of Part III, by a short form of its title and by its number. DP stands for dynamic programming.

What you see in the statement or constraints	What to try	Section in Part III (number)
Pairs, duplicates and counts		
Does any value repeat? Are all values distinct?	A set of seen values; or sort and compare neighbors	Hashing (44)
A pair with an exact sum or difference; the output is indexes	A dict from value to index; look up $t - x$ before you insert x	Hashing (44)
Count the pairs whose sum is divisible by k	A `Counter` of remainders; the partner of x has the remainder $(-x) \bmod k$	Hashing (44)
Split all items into pairs whose sum is divisible by k	Count the remainders. The count of r must equal the count of $k - r$; the remainders 0 and $k/2$ need even counts	Hashing (44)

continued on the next page

What you see in the statement or constraints	What to try	Section in Part III (number)
Same items up to order: anagram, permutation, rearrange	A `Counter` or 26 counts as a signature	Hashing (44)
Group items that are equal up to a detail (order, shift, rotation, shape)	A canonical key that erases the detail; a dict of lists	Hashing (44)
Longest run of consecutive values in $O(n)$	A set; start a run only where $x - 1$ is absent	Hashing (44)
k most frequent items, better than $O(n \log n)$	Count, then buckets indexed by count	Hashing (44)
One item belongs to several groups (row, column, block)	One seen-set per group; group id by arithmetic	Hashing (44); Matrices and simulation (64)
Count axis-aligned squares or rectangles through a query point	Fix the diagonal corner; multiply the counts of the forced corners	Hashing (44); Math and number manipulation (65)
A sum over all subarrays, substrings, pairs or paths; the fewest unit moves of items along tree edges	Count by contribution: for each element, bit or edge, count the structures where it counts (Section 37.10)	Stacks and monotonic stacks (48); Bit manipulation (66); Trees (51)
Contiguous subarrays and substrings		
Many range sums on static data	Prefix sums: $P[r + 1] - P[l]$	Prefix sums (45)
Count subarrays with sum exactly k; values can be negative	Prefix sums plus a `Counter` of earlier prefixes	Prefix sums (45)
Sum divisible by k; equal numbers of two kinds of items	Store prefix remainders; map the kinds to $+1$ and -1	Prefix sums (45)
For each index, combine all other items; no division	Prefix and suffix aggregates	Prefix sums (45)
Many range additions, then read all values	Difference list: $+v$ at l, $-v$ at $r + 1$, then a prefix sum	Prefix sums (45)
The sum of any rectangle of a static grid	2-D prefix sums: four reads by inclusion and exclusion	Prefix sums (45)
The largest square (or side) under a sum limit; all values ≥ 0	Binary search on the side; check one side with 2-D prefix sums	Prefix sums (45); Binary search (49)
Water above each bar; a value that needs the maximum on its left and on its right	Prefix maximum and suffix maximum; or two pointers that move the lower side	Prefix sums (45); Two pointers (46)
Count the segments that contain no forbidden pair	A pair at positions $a < b$ with $b \leq r$ forces $l > a$; keep the running maximum of these bounds	Prefix sums (45); Sliding window (47)
The same move applies to all items; an item drops out at a bound	One shared offset with its running minimum and maximum	Prefix sums (45)
Maximum of $a[j] - a[i]$ with $i < j$	The running minimum of the prefix	Prefix sums (45); Sliding window (47)
Longest substring or subarray that satisfies a condition that survives shrinking	Variable window: grow on the right, shrink on the left while invalid	Sliding window (47)
"At most k" changes, flips or distinct values in a segment	A window with a cost counter; shrink while the cost is above k	Sliding window (47)

continued on the next page

What you see in the statement or constraints	What to try	Section in Part III (number)
A substring of fixed length: anagram of a pattern, average of k items	Fixed window: add one item, remove one item	Sliding window (47)
Shortest segment that covers all required items	Grow until valid, then shrink while valid; a counter of satisfied keys	Sliding window (47)
Count the segments whose property survives shrinking	Add `r - l + 1` for each right end	Sliding window (47)
Count the segments that contain all required items, or at least k of something (the property survives growing)	Shrink while the window is valid, then add `left` for each right end; or all segments minus the segments that fail	Sliding window (47)
Shortest segment with a sum at least k; values can be negative	A deque of prefix indexes with increasing prefix values; pop the front while the sum is large enough (Section 36.3)	Prefix sums (45); Sliding window (47)
"Exactly k" distinct or odd items in a segment	At most k minus at most $k-1$	Sliding window (47)
Maximum or minimum of every window of size k	A monotonic deque of indexes	Sliding window (47); Stacks and monotonic stacks (48)
Best contiguous sum with negative values	Kadane: extend or restart	DP: foundations and 1-D (58)
Best contiguous product with negatives and zeros	Carry the maximum and the minimum that end here	DP: foundations and 1-D (58)
Order, nearest items and search		
Sorted data and a pair or triple with a target sum	Converging two pointers; fix one value for k-sum	Two pointers (46)
Best pair with a score of $\min(h_i, h_j) \times (j-i)$	Start at the widest pair; move the shorter side	Two pointers (46)
Palindrome, mirror, compare from both ends	Two pointers from the ends	Two pointers (46)
Remove or compact items in place with $O(1)$ space	A read pointer and a write pointer	Two pointers (46)
Merge two sorted lists or arrays	One pointer in each; take the smaller front item	Two pointers (46); Linked lists (50)
$n+1$ values in $1..n$; find the repeated value with $O(1)$ space and no change to the input	Read $i \to a[i]$ as a linked list; Floyd finds the start of the cycle	Linked lists (50); Two pointers (46)
Nearest greater or smaller item on the left or right; "days until"	A monotonic stack of indexes	Stacks and monotonic stacks (48)
Remove items but keep the order of the rest, so that the result is the smallest or the largest (remove k digits; keep each letter once)	A stack that stays non-decreasing: pop a larger top while it can still go (removals are left, or it occurs again later); for "each letter once", skip a letter that is already on the stack	Stacks and monotonic stacks (48)
Count the smaller or larger items before or after each index; inversions; range sums with point updates	Rank the values, then a Fenwick tree of counts: $O(n \log n)$ (Section 39)	Prefix sums (45)
Score of a range = its minimum times its length	Fix the minimum; nearest smaller item on each side	Stacks and monotonic stacks (48)

continued on the next page

What you see in the statement or constraints	What to try	Section in Part III (number)
Nested open and close items: brackets, tags, function logs	A stack: push an opener, pop on a closer	Stacks and monotonic stacks (48)
Postfix expression; nested decode such as `3[a2[c]]`	A stack of pending values	Stacks and monotonic stacks (48)
Objects move to one target at different speeds and cannot pass; count the groups that arrive	Sort by position, nearest to the target first; a new group starts when the arrival time is larger than the time of the group ahead	Stacks and monotonic stacks (48)
Items vanish during a scan, and the neighbors change at once (collisions, adjacent duplicates)	A stack of the survivors: the top is the current left neighbor	Stacks and monotonic stacks (48); Matrices and simulation (64)
Sorted data and a required $O(\log n)$	Binary search with one invariant on `lo` and `hi`	Binary search (49)
Insert position; first or last position of a value; count of values in $[a, b]$	`bisect_left` and `bisect_right` on the sorted list	Binary search (49)
A matrix with sorted rows, where the first value of each row is greater than the last value of the row before it	Binary search on a flat index i; the cell is `divmod(i, C)`	Binary search (49)
A rotated or shifted sorted array	Binary search on the sorted half, or on a true-or-false predicate	Binary search (49)
Minimum speed, capacity or time such that a task is feasible; "minimize the largest"	Binary search on the answer with a greedy check	Binary search (49)
The latest value at or before time t	A dict of sorted lists plus `bisect_right` minus 1	Binary search (49); Design problems (67)
Median or k-th item of two sorted arrays in log time	Binary search on the cut	Binary search (49)
Longest increasing subsequence	`dp[i]` $= 1 +$ the best `dp[j]` with $j < i$ and a smaller value: $O(n^2)$; for n up to 10^5, a tails list plus `bisect_left`	Binary search (49); DP: foundations and 1-D (58)
Top k, streams and scheduling		
k largest, smallest, closest or most frequent items; the k-th largest item of a stream	A heap of size k with the opposite order	Heaps and priority queues (53)
Repeatedly take the largest items, combine them, put the result back	A heap simulation	Heaps and priority queues (53)
A running median, or a rank that moves with the size	Two heaps that face each other	Heaps and priority queues (53)
Merge k sorted lists or streams	A heap of the current fronts	Heaps and priority queues (53); Linked lists (50)
Equal tasks need a gap of n (cooldown)	A max-heap of counts, filled one window of $n + 1$ slots at a time; or `max(N, (m - 1) * (n + 1) + ties)` for N tasks, the top count m and `ties` tasks with that count; a queue of cooling tasks when the cooldowns differ	Heaps and priority queues (53); Greedy (62)

continued on the next page

What you see in the statement or constraints	What to try	Section in Part III (number)
Many point queries on fixed intervals, known in advance	Sort the queries, sweep, a heap with lazy deletion	Intervals (63); Heaps and priority queues (53)
Intervals and greedy choices		
Do any two intervals overlap? Can one person attend all meetings?	Sort by start; compare each start with the previous end	Intervals (63)
Merge overlapping ranges; total covered length	Sort by start; extend the last group or open a new one	Intervals (63)
Fewest removals so that no intervals overlap; most non-overlapping; fewest arrows	Sort by end; keep the interval that ends first	Intervals (63); Greedy (62)
Minimum number of rooms or groups so that no two intervals in one group overlap; maximum overlap at one time	Sweep events ($+1$ at a start, -1 at an end) or a heap of end times; for closed intervals, put the end event at $e + 1$	Intervals (63)
Insert one interval into a sorted, disjoint list	Three blocks: before, overlap, after	Intervals (63)
Can you reach the end, when each value is a maximum jump?	Track the farthest reachable index	Greedy (62)
Fewest jumps, or fewest intervals that cover a line	BFS by windows: extend the farthest reach inside the current window	Greedy (62)
A circular route with gains and costs; find the start	Total check; restart after the point where the balance goes negative	Greedy (62)
A rule toward both neighbors: an item must get more than each neighbor with a lower rating; minimum total	One pass for the rule toward the left neighbor, one pass for the rule toward the right neighbor, then the maximum of the two for each item	Greedy (62); Prefix sums (45)
Split all items into groups of a fixed shape, such as consecutive values	The smallest remaining value must start a group	Greedy (62)
Each letter must stay in one part; maximize the number of parts	Last occurrence of each letter; cut when the index reaches the farthest last occurrence	Greedy (62)
A bracket string with wildcards	A range of possible open counts $[lo, hi]$	Greedy (62)
Combine items with max (or OR) any number of times to reach a target	Drop each item that overshoots; combine the rest	Greedy (62)
A huge budget; unlock options in a fixed order, then repeat one action	Try each option as the repeated one: a running prefix cost plus one integer division; test the greedy against a brute force on small inputs	Greedy (62)
Enumerate all answers		
Return all subsets, combinations or permutations; $n \leq 10$ to 20	Backtracking: choose, explore, un-choose; when an item can repeat, recurse with i, not $i + 1$	Backtracking (54)
The input has duplicates, but the output must be unique	Sort; skip an equal value at the same level	Backtracking (54)

continued on the next page

What you see in the statement or constraints	What to try	Section in Part III (number)
One choice from a small set at each position (letters of phone keys)	A DFS by position, or `itertools.product`	Backtracking (54)
All valid bracket strings with n pairs	Add an open bracket while open $< n$; add a close bracket while close $<$ open	Backtracking (54); Stacks and monotonic stacks (48)
All ways to cut a string into valid pieces (palindromes, dictionary words)	For each end of the first piece, test the piece, then recurse on the rest	Backtracking (54)
All placements under pairwise conflicts (queens, sudoku)	One decision per row; sets for column, $r - c$, $r + c$	Backtracking (54)
Trace a word through a grid; no cell twice	DFS that marks on entry and unmarks on exit	Backtracking (54)
Many words in one grid; prefix queries; autocomplete	A trie of the words, walked during the search	Tries (52)
Search a word set with a wildcard letter	At most 1 or 2 wildcards: a set of masked patterns (mask one letter on add, fill one dot on search). Many wildcards, or prefix queries too: a trie plus DFS over all children at the wildcard	Tries (52)
Graphs and grids		
Count islands, regions, provinces or groups; the size of the largest region	Flood fill from each unvisited cell, and return its size; or union-find	Graph traversal (55)
Split the nodes into two groups so that every edge (a conflict, a dislike) goes between the groups	Two colors: a BFS from each node with no color gives each neighbor the other color; an edge inside one color means an odd cycle	Graph traversal (55)
Compare the regions of two grids: the same cells exactly	Collect each region of each grid as a `frozenset` of cells; compare the sets	Graph traversal (55)
Fewest steps in a grid, all steps of equal cost	BFS; mark a cell when you push it	Graph traversal (55)
Spread from many sources at one time; distance to the nearest X	Multi-source BFS with all sources at distance 0	Graph traversal (55)
Cells that can reach a border or an ocean; enclosed regions	Search backward from the targets	Graph traversal (55)
Fewest moves between two configurations (a lock, a word)	BFS over implicit states	Graph traversal (55)
Deep copy of a graph or of a list with random pointers	A dict from old node to new node	Graph traversal (55); Linked lists (50)
Prerequisites; "X before Y"; a build order; a cycle in a directed graph	Topological sort (Kahn, or DFS with three colors); a short order means a cycle	Graph structure (56)
Words sorted in an unknown alphabet	One edge from each first difference, then a topological sort	Graph structure (56)

continued on the next page

What you see in the statement or constraints	What to try	Section in Part III (number)
Edges arrive one at a time; "already connected?"; the extra edge	Union-find	Graph structure (56)
Is this undirected graph a tree?	$n - 1$ edges and connected	Graph structure (56)
Each node has one next node; does the walk loop?	A visited set, or fast and slow pointers	Linked lists (50); Graph structure (56)
Count the paths of a fixed short length, such as 3 edges with required labels	Fix the middle edge; multiply the counts of the matching neighbors on each side	Graph structure (56)
Non-negative weights; cheapest cost or earliest arrival	Dijkstra with a heap	Weighted and advanced graphs (57)
Path cost = the highest cell on the path; the earliest time two cells connect	Dijkstra with max; or binary search plus BFS; or union-find by value	Weighted and advanced graphs (57)
Cheapest path with at most k stops, that is $k + 1$ edges	Bellman-Ford: $k + 1$ rounds, each on a frozen copy	Weighted and advanced graphs (57)
Connect all points at a minimum total cost	Minimum spanning tree: Prim for dense, Kruskal for sparse	Weighted and advanced graphs (57)
Use every edge (ticket) exactly one time	Hierholzer: post-order, then reverse	Weighted and advanced graphs (57)
Optimize or count over choices		
"How many ways", or the minimum cost, to reach step n with a few step sizes	`dp[i]` from the few entries before it: a sum or a minimum over the last step	DP: foundations and 1-D (58)
Take or skip each item; neighbors conflict	`dp[i] = max(dp[i-1], dp[i-2] + v)`	DP: foundations and 1-D (58)
The circular version of a line problem	Two runs: without the first item, without the last item	DP: foundations and 1-D (58)
Split a string into valid pieces: is it possible, or how many ways (words, letter codes)?	DP over the end of the prefix: `dp[i]` from `dp[j]` and a valid `s[j:i]`	DP: foundations and 1-D (58)
Fewest coins, or number of ways, with unlimited items	Unbounded knapsack; the loop order selects combinations or orderings	DP: knapsack family (59)
Each item at most one time; reach a target sum; equal halves	0/1 knapsack, capacity loop downward; or a set of reachable sums	DP: knapsack family (59)
Count or optimize subsets with a total of at least T (profit, score), often with a second limit	Cap the total: write to `dp[min(T, t + w)]`; one dimension for each limit	DP: knapsack family (59)
A plus or minus sign on each number to hit a target	Memoized DFS on (index, running sum); or a subset sum with $P = (total + target)/2$	DP: knapsack family (59)
$n \leq 16$ to 20; assign items to k groups (equal sums) or find an order	Bitmask DP over the set of used items; or backtracking with items in decreasing order that skips equal empty groups	DP: knapsack family (59); Backtracking (54)
Two strings: align, edit, common subsequence, count matches	A table over prefix pairs (i, j) with three neighbors	DP on strings and sequences (60)

continued on the next page

What you see in the statement or constraints	What to try	Section in Part III (number)
A pattern with a repeat operator; interleave two strings	DP over (i, j); a repeat means skip it, or use one and stay	DP on strings and sequences (60)
Palindromic substrings (contiguous), n up to a few thousand	Expand around $2n - 1$ centers	DP on strings and sequences (60)
Fewest inserts or deletes that make a string a palindrome	An interval table on `s[i..j]`: equal ends go inside at no cost; else 1 plus the better of the two ranges with one end dropped	DP on strings and sequences (60)
Grid moves only right and down; count or best paths	Each cell from the cell above and the cell on the left	DP on grids, intervals, games and states (61)
Longest path where each move strictly increases a value	Visit the cells from the largest value down: 1 plus the best larger neighbor; or a memoized DFS on the implicit DAG	DP on grids, intervals, games and states (61)
Remove items one by one; the score uses the current neighbors	Interval DP; choose the last item to remove	DP on grids, intervals, games and states (61)
Two players take from the ends; both play optimally	Score difference: $f(i, j) = \max(\text{gain} - f(\text{rest}))$	DP on grids, intervals, games and states (61)
Buy and sell with cooldown, fee or k transactions	A state machine: one best value per status	DP on grids, intervals, games and states (61)
Trees and linked lists		
A value at a node from its children: height, balance, size	Post-order DFS that returns a summary	Trees (51)
Two trees: the same, mirror images, or one inside the other	Recurse on both roots together; for a subtree, run that check at each node	Trees (51)
Best path between any two nodes	Return the best one-sided chain; record the two-sided path in a global	Trees (51)
A condition that depends on all ancestors (path maximum, bounds)	Pass the state down as arguments	Trees (51)
One answer per level: level order, side view, level average	BFS with a snapshot of the level size	Trees (51)
A BST and an order question: k-th smallest, validate, ancestor	In-order is sorted; pass bounds; find the split point	Trees (51)
Rebuild a tree from traversals; serialize a tree	Root from preorder, split by an index map; null markers	Trees (51)
Reverse (all, or in groups of k), reorder or rotate a singly linked list in place	`prev`, `curr` and a saved `next`; a dummy head	Linked lists (50)
The k-th node from the end in one pass; the middle node	Two pointers with a gap; fast and slow pointers	Linked lists (50)
Numbers, bits and matrices		
Every value two times except one; $O(1)$ space	XOR of all values; if the input is sorted and $O(\log n)$ is required, binary search for the first pair index p with `nums[2*p] != nums[2*p+1]`	Bit manipulation (66); Binary search (49)
One missing value in $0..n$	Expected sum minus actual sum, or XOR	Bit manipulation (66); Math and number manipulation (65)

continued on the next page

What you see in the statement or constraints	What to try	Section in Part III (number)
Number of 1 bits of one value	`n &= n - 1` removes the lowest 1 bit; or `bin(n).count("1")`	Bit manipulation (66)
Number of 1 bits for every value 0..n	Answer of i = answer of $\lfloor i/2 \rfloor$ plus the last bit	Bit manipulation (66)
Reverse the bits of a 32-bit value	32 rounds: shift the result left, add `n & 1`, shift `n` right	Bit manipulation (66)
Add without +; fixed 32-bit width	XOR for the sum, AND shifted left for the carry, a 32-bit mask	Bit manipulation (66)
The digits of an integer: digit sums, reverse, palindrome number	`n, d = divmod(n, 10)` in a loop; remove the sign first	Math and number manipulation (65)
Numbers as digit lists or strings, longer than 64 bits	Column arithmetic with a carry from the least significant end	Math and number manipulation (65); Linked lists (50)
An exponent up to 2^{31}	Fast power by repeated squaring	Math and number manipulation (65)
Apply $x \to f(x)$ again and again: does it end or loop?	Cycle detection with a set or with fast and slow values	Math and number manipulation (65)
A fixed 32-bit range and no wider type	Test for overflow before the operation	Math and number manipulation (65)
The gradient of a function that sorts or selects items (sort, median, max)	Keep the argsort; send each upstream gradient back to its original index; add one term for each other path, such as a divisor	Math and number manipulation (65)
Rotate or flip a square matrix in place	Transpose, then reverse each row	Matrices and simulation (64)
Spiral or ring order	Four bounds that move inward	Matrices and simulation (64)
A walker moves and turns on a grid	A list of four direction vectors and an index into it; a right turn adds 1 to the index, modulo 4	Matrices and simulation (64)
Mark the row and the column of each zero with $O(1)$ extra space	Store the marks in the first row and the first column, plus one flag for the first column; apply those two last	Matrices and simulation (64)
All cells update at the same time, in place	Store old and new state in one cell (two bits)	Matrices and simulation (64)
Each output cell is a function of its neighbors	An offset list, a bounds check and a new grid	Matrices and simulation (64)
Change exactly one step of a process so that it succeeds	Forward state after each prefix; built backward, the set of states from which each suffix still reaches the goal; one lookup for each edit	Matrices and simulation (64)
Detailed rules, a tiny n, rounds that repeat until nothing changes	A literal simulation; stop after a round with no change	Matrices and simulation (64)
Design problems		
A class with a list of operations and a required cost for each	List the operations; pick a structure for each; link the structures (Section 39)	Design problems (67)

continued on the next page

What you see in the statement or constraints	What to try	Section in Part III (number)
$O(1)$ get and put with eviction of the least recently used item	A dict plus a doubly linked list, or `OrderedDict`	Design problems (67); Linked lists (50)
A stack with $O(1)$ minimum	Push the pair (value, minimum so far)	Design problems (67); Stacks and monotonic stacks (48)
Minimum or maximum over entries that change in place (update, increment, decrement)	A dict as the truth plus heaps with lazy deletion; for changes of ± 1, a linked list of count buckets	Design problems (67); Heaps and priority queues (53)
A feed of the 10 most recent items from many sources	A global timestamp plus a heap merge of the sources	Design problems (67); Heaps and priority queues (53)
Pack a list of any strings into one string	A length prefix before each item	Design problems (67); Hashing (44)

39 Choose a data structure from the operations

List the operations that the algorithm needs, and how often each one runs. Then pick the cheapest structure that supports all of them. If no single structure does, combine two structures that point into each other. For example, a dict finds a node, and a linked list keeps the order.

Operations you need	Structure	Cost in Python
Read and write by position; add and remove at the end	`list` (array)	`a[i]`, `append`, `pop()`: $O(1)$. Insert or delete in the middle, `x in a`: $O(n)$.
Membership; remove duplicates; a visited mark	`set`	`add`, `in`, `discard`: $O(1)$ average.
Key to value: an index, a last position, a parent, a memo	`dict`, `defaultdict`	Get, set, `in`, delete: $O(1)$ average. A `dict` keeps insertion order.
Count occurrences; compare two multisets	`Counter`	One update: $O(1)$. A missing key reads as 0 and adds nothing. `most_common(k)`: $O(n \log k)$.
Keys are $0..m-1$ with a small m	`list` of size m	$O(1)$, with a smaller constant than a `dict`.
A key made of several parts: a cell, a state, a pair, a set of letters	`tuple` or `frozenset`	Hashable, so it can be a `dict` key or a `set` item. A `list` or a `set` cannot. Hashing costs O(length).
Visited marks on a grid	A `set` of `(r, c)`, or a list of lists of `bool`	Both $O(1)$. The list of lists is faster and smaller when most cells get a mark.
Add and remove at both ends; a FIFO queue; BFS	`collections.deque`	`append`, `appendleft`, `pop`, `popleft`: $O(1)$. An index in the middle: $O(n)$.
Take the smallest item many times; insert at any time	`heapq` on a list	Push and pop: $O(\log n)$. Read `h[0]`: $O(1)$. `heapify`: $O(n)$. For a max-heap, push $-x$.
The k best items of a stream	A heap of size k	$O(n \log k)$ time, $O(k)$ space.
A moving middle rank (median)	Two heaps	Insert: $O(\log n)$. Read: $O(1)$.
Delete an arbitrary item from a heap	A heap plus a set or `Counter` of deleted items	Delete lazily when the item reaches the top: $O(\log n)$ amortized.
Sorted order with floor, ceiling and rank queries	A sorted `list` plus `bisect`	Search: $O(\log n)$. `insort`: $O(n)$ memory move, fast up to about 10^5 items.

Operations you need	Structure	Cost in Python
Last in, first out; matching; undo; next greater item	`list` as a stack	`append`, `pop()`, `a[-1]`: $O(1)$.
Lookup by key, move an item to the end, evict the oldest	`OrderedDict`, or a `dict` plus a doubly linked list	`move_to_end`, `popitem(last=False)`, get, set: $O(1)$.
A random item, plus insert and delete by value	A `list` plus a `dict` from value to index	Delete: swap with the last item, then `pop()`: $O(1)$.
Prefix queries over many strings	A trie (`dict` of `dict`)	Insert, search, prefix test: $O(L)$ for a string of length L.
Merge groups; "same group?"	Union-find (a parent list)	`find`, `union`: almost $O(1)$ amortized with path compression and union by size; $O(\log n)$ amortized with path compression only.
Range sums on static data	A prefix-sum list	Build: $O(n)$. Query: $O(1)$. In 2-D, a query is $O(1)$ by inclusion and exclusion.
Range sums with point updates	A Fenwick tree	Update and query: $O(\log n)$.
Maximum or minimum of a sliding window	A monotonic `deque`	$O(1)$ amortized per step.
Neighbors of each node	A `dict` of lists, or a list of lists	Build: $O(V + E)$. Neighbors of one node: $O(\text{degree})$.
Values for each key, ordered by time	A `dict` of lists plus `bisect`	Append: $O(1)$ when the times increase. Floor query: $O(\log k)$.

The structures below occur often. Each block shows the operations and their results.

```
from collections import OrderedDict

cache = OrderedDict()                    # the order of keys is the order of use
for key in ["a", "b", "c"]:
    cache[key] = key.upper()
cache.move_to_end("a")                   # "a" is now the most recently used key
cache.popitem(last=False)                # -> ('b', 'B')
list(cache)                              # -> ['c', 'a']
```

```
parent = list(range(6))                  # union-find: each item starts as its own group

def find(x):
    while parent[x] != x:
        parent[x] = parent[parent[x]]    # path halving keeps the trees flat
        x = parent[x]
    return x

def union(a, b):
    ra, rb = find(a), find(b)
    if ra == rb:
        return False                     # one group already: this edge closes a cycle
    parent[ra] = rb
    return True

[union(0, 1), union(1, 2), union(3, 4), union(2, 0)]   # -> [True, True, True, False]
len({find(x) for x in range(6)})         # -> 3
```

```python
import bisect

times = [1, 4, 4, 9, 12]                # sorted

def floor_index(t):                     # last index with times[i] <= t, or -1
    return bisect.bisect_right(times, t) - 1

floor_index(4), floor_index(0), floor_index(10)   # -> (2, -1, 3)
bisect.insort(times, 7)
times                                   # -> [1, 4, 4, 7, 9, 12]
```

Fenwick tree. A Fenwick tree keeps prefix sums that stay correct after a point update. Both `add` and `prefix` cost $O(\log n)$. The sum of `a[l..r]` is `prefix(r + 1) - prefix(l)`. To count the smaller items after each index, rank the values, scan from the right, and keep a count for each rank (Count of Smaller Numbers After Self, LeetCode 315). The same scan counts the inversions of a list.

```python
import random

class Fenwick:
    def __init__(self, n):
        self.tree = [0] * (n + 1)       # 1-based inside
    def add(self, i, v):                # a[i] += v, 0-based i
        i += 1
        while i < len(self.tree):
            self.tree[i] += v
            i += i & -i                 # the next node that covers i
    def prefix(self, i):                # sum of a[0..i-1]
        s = 0
        while i > 0:
            s += self.tree[i]
            i -= i & -i                 # drop the lowest set bit
        return s

def count_smaller_after(nums):
    ranks = {v: r for r, v in enumerate(sorted(set(nums)))}
    fw, out = Fenwick(len(ranks)), []
    for x in reversed(nums):
        out.append(fw.prefix(ranks[x]))  # smaller values already seen
        fw.add(ranks[x], 1)
    return out[::-1]

count_smaller_after([5, 2, 6, 1])       # -> [2, 1, 1, 0]
random.seed(5)
lists = [[random.randint(-5, 5) for _ in range(random.randint(0, 9))] for _ in range(300)]
all(count_smaller_after(a) == [sum(y < x for y in a[i + 1:]) for i, x in enumerate(a)]
    for a in lists)                     # -> True
```

CAUTION

Look for a hidden $O(n)$ operation inside a loop. It turns an $O(n)$ plan into $O(n^2)$. The usual ones are `x in some_list`, `list.pop(0)`, `list.insert(0, x)`, `list.remove(x)`, `list.index(x)`, a slice such as `a[1:]`, and `min`, `max`, `sum` or `sorted` over a whole list. Use a set, a `deque`, an index, or a running value instead.

40 When you are stuck

Sometimes step 5 finds the waste, but no move removes it. Sometimes the brute force itself is not clear. Then change the question, not the code. Try the reframings below in this order. Give each one a short, honest try before you go to the next one.

1. **Solve a smaller or special case.** Use $n = 1, 2, 3$, $k = 1$, all values equal, or only positive values. The answers

for small cases often show the recurrence. Example: count the binary strings of length n with no two adjacent 1s. A brute force gives 2, 3, 5, 8, 13. This is the Fibonacci pattern. A last character 0 follows any valid string of length $n-1$. A last character 1 follows a 0 and a valid string of length $n-2$.

```
from itertools import product

def brute(n):
    return sum("11" not in "".join(bits) for bits in product("01", repeat=n))

[brute(n) for n in range(1, 8)]          # -> [2, 3, 5, 8, 13, 21, 34]
```

2. **Draw it.** Draw the array as bars, the intervals on a line, the grid, or the graph. Example: draw the bars of Trapping Rain Water. The water over a bar rises to the lower of two walls: the highest bar on its left and the highest bar on its right.
3. **Model it as a graph.** Make each item or state a node, and each allowed move or relation an edge. Then "fewest moves" is a BFS, "an order" is a topological sort, and "groups" are components. Example: in Open the Lock (LeetCode 752), each 4-digit code is a node, and one turn of one wheel is an edge. The fewest turns is the BFS distance from `"0000"`.
4. **Sort it.** If the answer does not depend on the input order, sort, and look again. Example: in Meeting Rooms, after a sort by start, if any two meetings overlap, then two neighbors overlap. So one scan over the neighbors finds a conflict.
5. **Think about the last step.** Ask which choice comes last. The rest is a smaller version of the same problem. Example: in Coin Change, the last coin is one of the coins, so the best count for a is $1 + \min_c \text{best}(a-c)$. In Burst Balloons, the last balloon of a range splits it into two independent ranges.
6. **Fix one variable.** Loop over one part of the answer and solve the rest. Example: in Largest Rectangle in Histogram, fix the shortest bar of the rectangle. Then the width is the gap between the nearest shorter bars on each side.
7. **Reverse the process.** Run time backward, or search from the goal. Example: in Pacific Atlantic Water Flow, let the water climb from each ocean. When a process deletes edges, add them in reverse order with union-find.
8. **Look at the complement.** Count or test the opposite, then subtract. Example: in Non-overlapping Intervals, the fewest removals equal n minus the most intervals that you can keep.
9. **Add a dummy element.** A sentinel removes a special case at an edge. Examples: a dummy head node for list edits; $P[0] = 0$ before the prefix sums; a bar of height 0 at the end of a histogram to empty the stack; a 1 at each end in Burst Balloons.
10. **Bound the answer and binary search.** If you can check one candidate answer fast, and the check is monotone, search for the boundary. Example: Split Array Largest Sum (LeetCode 410). A greedy pass counts the parts for a cap. A larger cap never needs more parts. The problem asks for exactly k parts. A further cut never raises the largest sum, so "at most k parts" gives the same answer when $k \le n$.

```
def min_largest_sum(a, k):               # split a into at most k contiguous parts
    def parts_needed(cap):               # greedy: fill each part up to cap
        parts, cur = 1, 0
        for x in a:
            if cur + x > cap:
                parts, cur = parts + 1, 0
            cur += x
        return parts
    lo, hi = max(a), sum(a)              # the answer is in [lo, hi]
    while lo < hi:
        mid = (lo + hi) // 2
        if parts_needed(mid) <= k:
            hi = mid                     # mid is feasible: try a smaller cap
        else:
            lo = mid + 1
    return lo

min_largest_sum([7, 2, 5, 10, 8], 2)     # -> 18
```

11. **Ask what to remember from the prefix.** Scan from left to right. At each item, ask which summary of the items before it is enough to answer for this item. Example: in Best Time to Buy and Sell Stock, it is the minimum price so far. In Longest Substring Without Repeating Characters, it is the last index of each character.

NOTE

If no reframing works, go back to step 4. Improve the brute force one small step at a time, and test each version against the previous one. A memoized brute-force recursion is often the intended solution. A correct $O(n^2)$ solution also passes many tests when the intended one is $O(n \log n)$.

41 Pitfalls that cost points

Most lost points come from a small set of mistakes that repeat across problems. Often the algorithm is correct, but one detail is wrong: a bound, a tie, a copy, or one word of the statement. Each item in this section gives the wrong form and the right form. Most items also have a tested example that shows the bug and the fix.

The Common mistakes table in Part I lists the traps of the Python language. This section covers the traps one level higher: the specification, the boundaries of an algorithm, and the state that the code keeps.

41.1 Bug classes at a glance

Use this list when time is short. Each line names one class of bug and the subsection with the details.

Class	Check	Details
Specification	Strict or non-strict words; contiguous or not; order matters or not; "ends at" or "visits".	41.2
Rules on the method	Extra space, forbidden operations, change in place. The judge does not check them.	41.2
Off-by-one	Inclusive or exclusive ends; arrays of size $n + 1$; nodes or edges.	41.3
Negative indexes	Both bounds tested before each index: `0 <= i < n`.	41.3
Work after the loop	The last group, the rest of a stack, a final carry, the rest of a list.	41.3
Termination	Each path of a loop makes progress; the binary search midpoint; a repeated state stops the loop.	41.4
Shared objects	Rows made with `*`; a stored path; an alias of the input; a new value read too early.	41.5
Linked lists	`next` saved before a rewire; the tail moved after each link; `dummy.next` returned.	41.5
State	One fact in one place; three states for a cycle check; new state for each call.	41.6
Recursion	Depth above 1000; a memo key with all the state; a return on every path.	41.7
Greedy and DP	A small counterexample tried for each greedy; the base case; the meaning of `dp[i]`; the loop order.	41.8
Hashing	Lookup before insert; keys that keep all the information; no insert on a read.	41.9
Comparisons	Ties; the start value of a best; a comparison with the running value.	41.10
Numbers	Integer math for large values; `//` and the modulo of negative values; operator precedence.	41.11
Speed	No hidden $O(n)$ step in a loop; mark on push; no debug output.	41.12
Input shapes	Empty input, a `None` root, an empty stack; strings in a grid; rows and columns.	41.13
Output	The requested value; the impossible case; integer or float; return or print.	41.14

Then run the checks of Section 42: a hand trace, the edge cases, and a brute-force cross-check.

41.2 Read the specification

A wrong reading of the statement is the most frequent cause of a wrong answer. The code then solves a different problem, and it solves it correctly. Read the statement two times: one time for the idea, and one time for each rule and each word.

Word in the statement	Meaning in the code
at most k, no more than k	`<= k`. A value equal to k is valid.
less than k, fewer than k	`< k`. A value equal to k is not valid.
strictly larger, warmer, increasing	Equal values do not count.
non-decreasing, at least as large	Equal values count.
subarray, substring	Contiguous. You cannot skip elements.
subsequence	Keeps the order. You can skip elements.
subset, combination	The order does not matter: $[1, 2]$ and $[2, 1]$ are one answer. Check the examples: a title can say "combination" and still count orders.
permutation, sequence of choices	The order matters: $(1, 2)$ and $(2, 1)$ are two answers.
non-empty	The answer has at least one element. Do not start the best value at 0.
distinct	Distinct values, or distinct positions? Find the answer in the examples.
in place	Change the given object. The judge reads that object, not a return value.
1-indexed	Labels or answers start at 1. Size arrays $n + 1$, or subtract 1 from each label.
in any order	The judge accepts any order. Your own tests must sort before they compare.
ends at, final	Test only the last state.
reaches, visits, at some time	Test every state on the way.
every, all	`all(...)`: one failure gives `False`.
some, any, exists	`any(...)`: one success gives `True`.

CAUTION

The exact success condition. Wrong: return success as soon as a walk touches the target. Right: finish the walk, then test the final state. "Ends at the target" and "passes through the target" are two different problems.

```
MOVES = {"U": (-1, 0), "D": (1, 0), "L": (0, -1), "R": (0, 1)}
def cells_on_walk(start, program):
    r, c = start
    cells = [(r, c)]
    for ch in program:
        dr, dc = MOVES[ch]
        r, c = r + dr, c + dc
        cells.append((r, c))
    return cells
def reaches(start, goal, program):          # "the walk visits the goal"
    return goal in cells_on_walk(start, program)
def ends_at(start, goal, program):          # "the walk stops on the goal"
    return cells_on_walk(start, program)[-1] == goal
reaches((0, 0), (0, 1), "RR")               # -> True
ends_at((0, 0), (0, 1), "RR")               # -> False
```

CAUTION

Every rule implemented, every rule tested. Wrong: copy the code of one rule to make a similar rule, and miss one change. An example is a "move backward" command that moves forward. Right: list the rules of the statement. Write one small test for each rule. Raise an error on an input value that no rule covers, so that a missing rule fails loudly.

```
DIRS = [(-1, 0), (0, 1), (1, 0), (0, -1)]   # up, right, down, left: clockwise
def run(program):
    r, c, d = 0, 0, 0                       # start at (0, 0) and face up
    for ch in program:
        if ch == "L":
            d = (d - 1) % 4                 # turn counterclockwise
        elif ch == "R":
            d = (d + 1) % 4                 # turn clockwise
        elif ch == "F":
            r, c = r + DIRS[d][0], c + DIRS[d][1]
        elif ch == "B":                     # move backward and keep the facing
            r, c = r - DIRS[d][0], c - DIRS[d][1]
        else:
            raise ValueError(ch)
    return (r, c, d)
TESTS = {"L": (0, 0, 3), "R": (0, 0, 1), "F": (-1, 0, 0),
         "B": (1, 0, 0), "RF": (0, 1, 1), "RB": (0, -1, 1)}   # one test per rule
def failing(solution):
    return [p for p, want in TESTS.items() if solution(p) != want]
def run_wrong(program):                     # acts like a copied rule: B moves like F
    return run(program.replace("B", "F"))
failing(run_wrong)                          # -> ['B', 'RB']
failing(run)                                # -> []
run("X")                                    # -> raises ValueError
```

CAUTION

Rules on the method, not only on the answer. Wrong: ignore a rule that the judge cannot see. Examples are $O(1)$ extra space, "no division", "do not convert the input to an integer", and "relink the nodes, do not change their values". Right: list these rules together with the other rules of the statement. A judge often accepts a solution that breaks them, but an interviewer does not.

CAUTION

Assumptions that the statement does not give. Wrong: assume that the input is sorted, that the values are distinct or positive, or that a graph is connected, because the examples look like that. Right: check each assumption against the constraints. If the constraints do not promise it, handle the other case.

- Two pointers on unsorted input give wrong answers. Sort first, or use a hash map.
- A sliding window on sums needs sums that only grow when the window grows. Negative values break this. Use prefix sums with a hash map instead (Subarray Sum Equals K).
- A traversal from node 0 misses the other components. Start a traversal from every unvisited node.
- Isolated nodes do not appear in the edge list. Take the node count from n, not from the edges.
- Edge direction: a pair `[a, b]` can mean "b comes before a". Check the direction on the first example.

41.3 Indexing and off-by-one

Most off-by-one bugs come from a mixed convention. For each range, decide whether the end is included. Then use the same decision everywhere.

CAUTION

Inclusive and exclusive ends. Wrong: `s[i:j]` for the piece from i to j with both ends included. Right: `s[i:j + 1]`. A slice and `range` exclude the end. The number of elements from i to j is $j - i + 1$. The distance between them is $j - i$.

```
s = "abcdef"
i, j = 1, 3                     # the piece from index 1 to index 3, both included
s[i:j]                          # -> 'bc'
s[i:j + 1]                      # -> 'bcd'
len(range(i, j + 1)), j - i + 1 # -> (3, 3)
list(range(3, 0, -1))           # -> [3, 2, 1]
```

```
list(range(3, -1, -1))           # -> [3, 2, 1, 0]
```

A downward `range` also excludes its end. To include 0, stop at `-1`. Formulas that often go wrong by one:

- A window from `l` to `r`, both included, has length `r - l + 1`. The width between two walls at `l` and `r` is `r - l` (Container With Most Water).
- A prefix table has $n + 1$ entries. `P[i]` is the sum of the first i elements, so the sum of `a[i..j]` is `P[j + 1] - P[i]`.
- In a DP over prefix lengths, entry `dp[i]` covers `s[:i]`, so its last character is `s[i - 1]`.
- A path with k stops has $k+1$ edges. A path of k nodes has $k-1$ edges. Tree depth counts nodes, but a diameter counts edges.
- A cooldown of n steps between two runs puts the next run at time $t + n + 1$, not $t + n$.

CAUTION

Negative indexes do not fail. Wrong: test only the upper bound, `nr < R`. The index `-1` then reads the last row, and no error appears. Right: test both bounds with a chained comparison, `0 <= nr < R`, before you index. The same trap appears in `dp[a - c]` when `c > a`, in an index sentinel of `-1`, and in `s[i - 1:i + 1]` when `i` is 0.

```
grid = ["ab",
        "cd"]
R, C = len(grid), len(grid[0])
STEPS = ((1, 0), (-1, 0), (0, 1), (0, -1))
def neighbors_wrong(r, c):
    return sorted(grid[r + dr][c + dc] for dr, dc in STEPS
                  if r + dr < R and c + dc < C)          # no lower bound
def neighbors(r, c):
    return sorted(grid[r + dr][c + dc] for dr, dc in STEPS
                  if 0 <= r + dr < R and 0 <= c + dc < C)
neighbors_wrong(0, 0)       # -> ['b', 'b', 'c', 'c']
neighbors(0, 0)             # -> ['b', 'c']
dp = [10, 20, 30, 40]
a, coin = 1, 3
dp[a - coin]                # -> 30
```

The last line reads `dp[-2]` silently. Test `coin <= a` before you read `dp[a - coin]`. A guard that skips the move is correct only when the move must land exactly, as with a coin. When the move covers a span that can start before index 0, as with a 7-day pass, do not skip the move. Clamp the index with `max(0, i - k)` (Minimum Cost For Tickets).

```
def min_cost(days, costs, clamp=True):     # passes of 1, 7 and 30 days
    travel, last = set(days), days[-1]
    dp = [0] * (last + 1)                   # dp[i]: cheapest cover of the travel days up to day i
    for i in range(1, last + 1):
        if i not in travel:
            dp[i] = dp[i - 1]
        elif clamp:
            dp[i] = min(dp[max(0, i - k)] + c for k, c in zip((1, 7, 30), costs))
        else:                               # wrong: skips a pass that starts before day 1
            dp[i] = min(dp[i - k] + c for k, c in zip((1, 7, 30), costs) if i - k >= 0)
    return dp[last]
min_cost([1, 2, 3], [2, 3, 15], clamp=False)   # -> 6
min_cost([1, 2, 3], [2, 3, 15])                # -> 3
```

CAUTION

Empty and single-element input. Wrong: start a result at 1, or read `a[0]` before a length check. Right: decide the answer for an empty input and for one element first. Then check that the general code gives these answers, or add a guard.

```
def longest_run_wrong(a):          # longest run of equal neighbors
    best = cur = 1
    for i in range(1, len(a)):
        cur = cur + 1 if a[i] == a[i - 1] else 1
        best = max(best, cur)
    return best
def longest_run(a):
    if not a:
        return 0
    best = cur = 1
    for i in range(1, len(a)):
        cur = cur + 1 if a[i] == a[i - 1] else 1
        best = max(best, cur)
    return best
longest_run_wrong([])              # -> 1
longest_run([]), longest_run([7]), longest_run([4, 4, 1])   # -> (0, 1, 2)
```

CAUTION

Work left after the loop. Wrong: finish an item only when the next item starts. The last item then never gets into the output. Examples are the last group of a merge, the bars left on a stack, a final carry, and the rest of the longer list. Right: after the loop, handle what is left. For groups, keep the current group as the last element of the output, `out[-1]`. Then there is no step to forget.

```
def merge_wrong(intervals):
    intervals = sorted(intervals)
    out = []
    st, ed = intervals[0]
    for s, e in intervals[1:]:
        if s <= ed:
            ed = max(ed, e)
        else:
            out.append([st, ed])
            st, ed = s, e
    return out                          # the last group is never added
def merge(intervals):
    out = []
    for s, e in sorted(intervals):
        if out and s <= out[-1][1]:
            out[-1][1] = max(out[-1][1], e)
        else:
            out.append([s, e])          # a new list, not the input list
    return out
merge_wrong([[1, 3], [2, 4], [6, 7]])  # -> [[1, 4]]
merge([[1, 3], [2, 4], [6, 7]])        # -> [[1, 4], [6, 7]]
```

41.4 Loops and termination

CAUTION

Every loop makes progress. Wrong: a `while` loop with a path that does not change the loop variables. An example is a two-pointer swap that never moves the pointers. The program then hangs. Right: for every path through the body, find the variable that moves toward the exit condition. While you debug, add a step counter that raises an error after too many steps.

```
def reverse_wrong(a, limit=100):
    l, r, steps = 0, len(a) - 1, 0
    while l < r:
        steps += 1
        if steps > limit:                   # a debug guard against an endless loop
```

```
            raise RuntimeError("no progress")
        a[l], a[r] = a[r], a[l]              # l and r never move
    return a
def reverse(a):
    l, r = 0, len(a) - 1
    while l < r:
        a[l], a[r] = a[r], a[l]
        l += 1                               # both pointers move on every pass
        r -= 1
    return a
reverse_wrong([1, 2, 3])                     # -> raises RuntimeError
reverse([1, 2, 3])                           # -> [3, 2, 1]
```

CAUTION

The binary search midpoint. Wrong: `lo = mid` together with `mid = (lo + hi) // 2`. When `hi = lo + 1`, `mid` equals `lo`. If the test is true, `lo` does not change, and the loop never ends. Right: choose one template from the table below and keep all of its rules.

Goal	Loop	Midpoint	Moves	Result
Find an exact value	`lo <= hi`	`(lo + hi) // 2`	`lo = mid + 1` `hi = mid - 1`	`mid` on a match
First `True`	`lo < hi`	`(lo + hi) // 2`	`hi = mid` `lo = mid + 1`	`lo`
Last `True` (closed range, `ok(lo)` true)	`lo < hi`	`(lo + hi + 1) // 2`	`lo = mid` `hi = mid - 1`	`lo`

In the first row, the first move is for `a[mid] < x`, and a match returns at once. In the other rows, the first move is for a true test, and the second move is for a false test. Section 49 uses `first_true` on the half-open range `[lo, hi)`. It gets the last true value as the first false value minus 1, so it needs no second template.

```
def first_true(lo, hi, ok):      # on [lo, hi): returns hi if no x passes
    while lo < hi:
        mid = (lo + hi) // 2     # round down, because the update is hi = mid
        if ok(mid):
            hi = mid
        else:
            lo = mid + 1
    return lo
def last_true_closed(lo, hi, ok):   # closed [lo, hi]; ok(lo) must be True
    while lo < hi:
        mid = (lo + hi + 1) // 2   # round up, because the update is lo = mid
        if ok(mid):
            lo = mid
        else:
            hi = mid - 1
    return lo
def last_true_closed_wrong(lo, hi, ok, limit=100):
    steps = 0
    while lo < hi:
        steps += 1
        if steps > limit:
            raise RuntimeError("no progress")
        mid = (lo + hi) // 2     # rounds down: lo = mid can leave lo unchanged
        if ok(mid):
            lo = mid
        else:
            hi = mid - 1
    return lo
first_true(0, 100, lambda x: x * x >= 50)          # -> 8
```

```
last_true_closed(0, 10, lambda x: x <= 3)          # -> 3
last_true_closed_wrong(0, 10, lambda x: x <= 3)    # -> raises RuntimeError
```

Other binary search traps:

- The start range must contain the answer. Start at the smallest valid value: a speed of 0 divides by zero. Start at 0 when 0 is a possible answer.
- The test must be monotonic: all `False`, then all `True`. Negative values can break this for sums.
- `bisect_left` can return `len(a)`. Test `i < len(a)` before you read `a[i]`.
- After the loop of the exact-value template, `lo` is the insertion point, not a match.

CAUTION

Inner loops need the outer bound. Wrong: an inner `while` loop that skips elements and tests only the element. On an input with nothing to keep, the index runs off the end. Right: repeat the outer bound in the inner condition: `while i < j and ...`.

```
def is_pal_wrong(s):
    i, j = 0, len(s) - 1
    while i < j:
        while not s[i].isalnum():           # no i < j test
            i += 1
        while not s[j].isalnum():
            j -= 1
        if s[i].lower() != s[j].lower():
            return False
        i, j = i + 1, j - 1
    return True
def is_pal(s):
    i, j = 0, len(s) - 1
    while i < j:
        while i < j and not s[i].isalnum():
            i += 1
        while i < j and not s[j].isalnum():
            j -= 1
        if s[i].lower() != s[j].lower():
            return False
        i, j = i + 1, j - 1
    return True
is_pal_wrong("!!")                          # -> raises IndexError
is_pal("!!"), is_pal("A man, a plan, a canal: Panama")   # -> (True, True)
```

CAUTION

Loops that wait for a value. Wrong: `while n != 1: n = step(n)`. If the sequence enters a cycle without 1, the loop never ends. Right: stop on a repeated state too. Keep a set of seen states, or use fast and slow pointers.

```
def step(n):
    return sum(int(d) ** 2 for d in str(n))
def is_happy(n):
    seen = set()
    while n != 1 and n not in seen:      # stop on success or on a repeat
        seen.add(n)
        n = step(n)
    return n == 1
is_happy(19), is_happy(2)                # -> (True, False)
```

- Fast and slow pointers start on the same node. Test for equality after the first move. A test before the move ends the loop at once.
- A simulation in rounds stops only after a full round with no change. A round with fewer changes is not the end.
- When time can be large (up to 10^9), do not step one time unit per pass. Jump to the next event.

- A level-by-level BFS reads the queue size one time, before the inner loop: `for _ in range(len(q))`. A `while` loop that reads `len(q)` again mixes the levels.
- A level BFS that runs until the queue is empty also counts the last level, which changes nothing. For "minutes until all cells change", stop when no target is left, or subtract that last level.

CAUTION

A condition that can never be true. Wrong: a test inside a loop for a state that exists only after the loop, such as `i == len(s)` inside `for i in range(len(s))`. The test never passes, so its branch never runs, and no error appears. Right: put the final test after the loop. For each `if`, find one input that makes it true. If no such input exists, the condition is dead, and the logic around it is wrong.

```
def back_to_start_wrong(program):
    pos = 0
    for i in range(len(program)):
        pos += 1 if program[i] == "R" else -1
        if i == len(program):          # never true: i stops at len(program) - 1
            return pos == 0
    return False
def back_to_start(program):
    pos = 0
    for ch in program:
        pos += 1 if ch == "R" else -1
    return pos == 0                    # test the final state after the loop
back_to_start_wrong("RL"), back_to_start("RL")   # -> (False, True)
```

41.5 Mutation, copying and aliasing

A name refers to an object. An assignment, an append, or a call does not copy the object. Most state bugs have one of two causes: two names for one object, or a write that a later read sees too early.

CAUTION

Shared rows. Wrong: `[[0] * C] * R` or `[[]] * n` to make a table. Also wrong: `grid.copy()`, `grid[:]` or `list(grid)` to get an independent grid. With `[[0] * C] * R`, all rows are one list, so a write to one row changes every row. With `grid.copy()`, `grid[:]` or `list(grid)`, the copy shares each row with the original, so a write to the copy also changes the original. Right: `[[0] * C for _ in range(R)]`, `[[] for _ in range(n)]`, and `[row[:] for row in grid]`.

```
table = [[0] * 2] * 2              # one row object, two times
table[0][0] = 1
table                              # -> [[1, 0], [1, 0]]
table = [[0] * 2 for _ in range(2)]
table[0][0] = 1
table                              # -> [[1, 0], [0, 0]]
grid = [[1, 2], [3, 4]]
shallow = grid.copy()              # grid[:] and list(grid) do the same
shallow[0][0] = 9
grid[0][0]                         # -> 9
grid = [[1, 2], [3, 4]]
deep = [row[:] for row in grid]    # a new list for every row
deep[0][0] = 9
grid[0][0]                         # -> 1
```

CAUTION

Synchronous updates. Wrong: write new values into the grid while later cells still read their neighbors from that grid (Game of Life, an image blur). Later cells then read new values. Right: read only old values. Write into a new grid, or store the new state in extra bits and decode it in a second pass.

```python
def step_wrong(a):                # new a[i] = old a[i - 1] + old a[i + 1]
    n = len(a)
    for i in range(n):
        left = a[i - 1] if i > 0 else 0
        right = a[i + 1] if i + 1 < n else 0
        a[i] = left + right       # the next cell reads this new value
    return a
def step(a):
    n = len(a)
    return [(a[i - 1] if i > 0 else 0) + (a[i + 1] if i + 1 < n else 0)
            for i in range(n)]    # reads only the old list
step_wrong([1, 2, 3])             # -> [2, 5, 5]
step([1, 2, 3])                   # -> [2, 4, 2]
```

A one-dimensional DP table that is reused for each item has the same problem. For a choice that each item can make only one time, loop over the sums downward. Then each read sees the value from before this item.

```python
def can_sum(nums, target, downward):
    dp = [True] + [False] * target          # dp[s]: some subset has the sum s
    for x in nums:
        sums = range(target, x - 1, -1) if downward else range(x, target + 1)
        for s in sums:
            dp[s] = dp[s] or dp[s - x]
    return dp[target]
can_sum([3], 6, downward=False)             # -> True
can_sum([3], 6, downward=True)              # -> False
```

The upward loop uses the 3 two times. That is correct only when each item can repeat (Coin Change).

CAUTION

Rolling variables. Wrong: update two rolling variables in two statements, when the second statement needs the old value of the first. Right: use one tuple assignment. Python evaluates the full right side before it assigns. The same rule applies to a pair such as the current maximum and minimum (Maximum Product Subarray).

```python
def rob_wrong(nums):
    prev2 = prev1 = 0                 # best total up to i - 2, and up to i - 1
    for x in nums:
        prev2 = prev1                 # the next line reads this new prev2
        prev1 = max(prev1, prev2 + x)
    return prev1
def rob(nums):
    prev2 = prev1 = 0
    for x in nums:
        prev2, prev1 = prev1, max(prev1, prev2 + x)
    return prev1
rob_wrong([2, 1, 1, 2])               # -> 6
rob([2, 1, 1, 2])                     # -> 4
```

CAUTION

Rolling rows. Wrong: `prev = cur` at the end of a row, followed by writes into `cur` in the next row. The two names are now one list, so `prev[j - 1]` already holds a value of the new row. Right: `prev = cur[:]`, or build a new `cur` list for each row.

```python
def lcs(a, b, copy_row):              # longest common subsequence with two rows
    prev = [0] * (len(b) + 1)
    cur = [0] * (len(b) + 1)
    for i in range(1, len(a) + 1):
        for j in range(1, len(b) + 1):
            if a[i - 1] == b[j - 1]:
```

```
                cur[j] = prev[j - 1] + 1
            else:
                cur[j] = max(prev[j], cur[j - 1])
        prev = cur[:] if copy_row else cur
    return prev[-1]
lcs("ab", "bb", copy_row=False)       # -> 2
lcs("ab", "bb", copy_row=True)        # -> 1
```

NOTE

Tuple assignment evaluates the right side first, but it assigns the targets from left to right. A target such as `curr.next` uses the value of `curr` at the time of that assignment. Put `curr.next` before `curr` in the target list, or use a temporary name.

```
class Node:
    def __init__(self, val, next=None):
        self.val, self.next = val, next
def values(head):
    out = []
    while head:
        out.append(head.val)
        head = head.next
    return out
def reverse(head):
    prev, curr = None, head
    while curr:
        curr.next, prev, curr = prev, curr, curr.next   # curr.next is assigned first
    return prev
def reverse_wrong(head):
    prev, curr = None, head
    while curr:
        prev, curr, curr.next = curr, curr.next, prev   # curr changes first
    return prev
values(reverse(Node(1, Node(2, Node(3)))))      # -> [3, 2, 1]
reverse_wrong(Node(1, Node(2, Node(3))))        # -> raises AttributeError
```

CAUTION

Pointer changes in a linked list. Wrong: change `node.next` before you save the old value, or link a node and then not move the tail. Right: obey these rules on every change of a pointer.

- Save `nxt = curr.next` before you change `curr.next`. Otherwise the rest of the list is lost.
- Start an output list with a dummy node. Move the tail after each link. Return `dummy.next`, not `dummy`.
- After a split or a reorder, set the last `next` to `None`. An old link can make a cycle.
- Compare nodes with `is`, and use the node object as a map key. Two nodes can have the same value.
- Test `fast and fast.next` before a step of two nodes.

```
class Node:
    def __init__(self, val, next=None):
        self.val, self.next = val, next
def values(head):
    out = []
    while head:
        out.append(head.val)
        head = head.next
    return out
def merge(a, b, move_tail):
    dummy = tail = Node(0)
    while a and b:
        if a.val <= b.val:
            tail.next, a = a, a.next
        else:
```

```
            tail.next, b = b, b.next
        if move_tail:
            tail = tail.next
    tail.next = a or b                # attach the rest of the longer list
    return dummy.next
values(merge(Node(1, Node(3)), Node(2, Node(4)), move_tail=False))    # -> [4]
values(merge(Node(1, Node(3)), Node(2, Node(4)), move_tail=True))     # -> [1, 2, 3, 4]
```

CAUTION

A stored reference to the input. Wrong: append an input list to the output, and later change that output element. The caller's data changes too. Right: append a new object, such as `[s, e]`.

```
def merge_alias(intervals):
    out = []
    for iv in sorted(intervals):
        if out and iv[0] <= out[-1][1]:
            out[-1][1] = max(out[-1][1], iv[1])   # changes a list of the caller
        else:
            out.append(iv)                        # stores the input list itself
    return out
data = [[1, 3], [2, 4]]
merge_alias(data)        # -> [[1, 4]]
data                     # -> [[1, 4], [2, 4]]
```

CAUTION

A change to the caller's input. Wrong: sort, heapify, negate, or mark the input when the caller still needs it. Right: work on a copy (`sorted(nums)`, `[row[:] for row in grid]`), or keep a separate visited set. Some problems require a change in place. Then the opposite rule applies.

```
def kth_smallest_wrong(nums, k):
    nums.sort()                       # sorts the list of the caller
    return nums[k - 1]
def kth_smallest(nums, k):
    return sorted(nums)[k - 1]        # sorts a new list
data = [3, 1, 2]
kth_smallest_wrong(data, 1), data     # -> (1, [1, 2, 3])
data = [3, 1, 2]
kth_smallest(data, 1), data           # -> (1, [3, 1, 2])
```

CAUTION

A new object assigned to a parameter. Wrong: assign a new object to the parameter name when the function must change its input in place. The assignment changes only the local name. The caller still sees the old object. Right: change the object itself: `m[:] = new_rows`, or write into its cells.

```
def rotate_wrong(m):
    m = [list(row) for row in zip(*m[::-1])]      # binds the local name only
def rotate(m):
    m[:] = [list(row) for row in zip(*m[::-1])]   # changes the object of the caller
a = [[1, 2], [3, 4]]
rotate_wrong(a); a          # -> [[1, 2], [3, 4]]
rotate(a); a                # -> [[3, 1], [4, 2]]
```

CAUTION

A change to a collection during a loop over it. Wrong: add or remove elements of a list, a set or a dict in a `for` loop over that collection. A dict or a set raises `RuntimeError`. A list raises no error, but the loop skips elements. Right: loop over a copy (`list(d)`, `a[:]`), or build a new collection.

```
a = [1, 2, 2, 3]
for x in a:
    if x == 2:
        a.remove(x)                    # the next element moves into this slot and is skipped
a                                      # -> [1, 2, 3]
a = [1, 2, 2, 3]
[x for x in a if x != 2]               # -> [1, 3]
s = {1, 2}
for x in s:
    s.add(x + 10)
# -> raises RuntimeError
```

CAUTION

A stored path that keeps changing. Wrong: `res.append(path)` in a backtracking search. Every entry of `res` is then the same list, and the later pops make it empty. Right: `res.append(path[:])`, or append a new object, such as `"".join(path)` or `tuple(path)`.

```
def subsets(nums, copy):
    res, path = [], []
    def dfs(i):
        if i == len(nums):
            res.append(path[:] if copy else path)
            return
        path.append(nums[i])           # take nums[i]
        dfs(i + 1)
        path.pop()                     # undo, then skip nums[i]
        dfs(i + 1)
    dfs(0)
    return res
subsets([1, 2], copy=False)            # -> [[], [], [], []]
subsets([1, 2], copy=True)             # -> [[1, 2], [1], [2], []]
```

CAUTION

Backtracking state. Wrong: return from inside the loop before you undo a mark. The board then stays changed for the caller and for each later search on the same board, such as the next word. Right: compute the result, undo every change, and then return.

```
def exists_wrong(board, word):
    R, C = len(board), len(board[0])
    def dfs(r, c, i):
        if i == len(word):
            return True
        if not (0 <= r < R and 0 <= c < C) or board[r][c] != word[i]:
            return False
        board[r][c] = "#"                          # mark the cell as used
        for nr, nc in ((r + 1, c), (r - 1, c), (r, c + 1), (r, c - 1)):
            if dfs(nr, nc, i + 1):
                return True                        # returns before the undo
        board[r][c] = word[i]
        return False
    return any(dfs(r, c, 0) for r in range(R) for c in range(C))
def exists(board, word):
    R, C = len(board), len(board[0])
    def dfs(r, c, i):
        if i == len(word):
            return True
        if not (0 <= r < R and 0 <= c < C) or board[r][c] != word[i]:
            return False
        board[r][c] = "#"
```

```
        found = any(dfs(nr, nc, i + 1)
                    for nr, nc in ((r + 1, c), (r - 1, c), (r, c + 1), (r, c - 1)))
        board[r][c] = word[i]                       # undo on every path
        return found
    return any(dfs(r, c, 0) for r in range(R) for c in range(C))
board = [list("ab"), list("cd")]
exists_wrong(board, "ab"), board      # -> (True, [['#', '#'], ['c', 'd']])
board = [list("ab"), list("cd")]
exists(board, "ab"), board            # -> (True, [['a', 'b'], ['c', 'd']])
```

- Each change before a recursive call needs its undo after the call: `append` and `pop`, mark and unmark, add and remove.
- With equal values in the input, sort first. Then skip a value equal to the previous value only at the same level: `j > start`, not `j > 0`. The test `j > 0` also blocks answers with two equal values, such as $[2, 2]$.

```
def subsets_no_dup(nums, right):
    nums, res, path = sorted(nums), [], []
    def dfs(start):
        res.append(path[:])
        for j in range(start, len(nums)):
            first = start if right else 0      # right: j > start; wrong: j > 0
            if j > first and nums[j] == nums[j - 1]:
                continue                       # an equal value at the same level
            path.append(nums[j])
            dfs(j + 1)
            path.pop()
    dfs(0)
    return res
subsets_no_dup([2, 2], right=False)    # -> [[], [2]]
subsets_no_dup([2, 2], right=True)     # -> [[], [2], [2, 2]]
```

CAUTION

State that lives longer than one call. Wrong: keep results in a class attribute, a default argument, or a global name. A judge can call the method many times, and old results then leak into new answers. Right: create all state inside the method. Define a cached helper inside the method too, so that its cache starts empty for each input.

```
class SolutionWrong:
    found = []                      # a class attribute: one list for all calls
    def evens(self, nums):
        for x in nums:
            if x % 2 == 0:
                self.found.append(x)
        return self.found
class Solution:
    def evens(self, nums):
        found = []                  # new state for each call
        for x in nums:
            if x % 2 == 0:
                found.append(x)
        return found
SolutionWrong().evens([2, 3])      # -> [2]
SolutionWrong().evens([4])         # -> [2, 4]
Solution().evens([2, 3]), Solution().evens([4])   # -> ([2], [4])
```

41.6 State design

Keep each fact in one place, and give each state an explicit value. Clever shortcuts in the state save a line, but they are hard to check.

CAUTION

One fact, one place. Wrong: two parallel structures for one fact, such as a list of values and a list of flags, or a DP array together with a separate "reachable" array. A change to one structure without the other moves the facts out of line. Right: keep one list of records, or let one value encode every case, for example `float("inf")` for "not reachable".

```
names, alive = ["a", "b", "c"], [True, False, True]
del names[0]                          # the flag list still has its old first entry
list(zip(names, alive))               # -> [('b', True), ('c', False)]
items = [["a", True], ["b", False], ["c", True]]
del items[0]                          # one list: the facts move together
items                                 # -> [['b', False], ['c', True]]
```

The same rule applies to a design with two structures, such as a dict and a linked list in a cache. Change both structures in the same method, every time.

CAUTION

Explicit states, not destructive tricks. Wrong: one visited set for a cycle check in a directed graph, or a "done" mark that destroys the input, such as an emptied adjacency list. One set cannot tell "on the current path" from "finished", so two paths to one node look like a cycle. Right: three states: 0 for new, 1 for on the current path, 2 for finished. Kahn's algorithm (Section 56, Template 1) avoids the problem too.

```
def has_cycle_wrong(graph):
    seen = set()
    def dfs(u):
        if u in seen:
            return True                 # "seen before" is not "on the current path"
        seen.add(u)
        return any(dfs(v) for v in graph[u])
    return any(dfs(u) for u in graph if u not in seen)
def has_cycle(graph):
    state = {u: 0 for u in graph}       # 0 new, 1 on the current path, 2 finished
    def dfs(u):
        state[u] = 1
        for v in graph[u]:
            if state[v] == 1 or (state[v] == 0 and dfs(v)):
                return True
        state[u] = 2
        return False
    return any(state[u] == 0 and dfs(u) for u in graph)
diamond = {0: [1, 2], 1: [3], 2: [3], 3: []}
has_cycle_wrong(diamond), has_cycle(diamond)     # -> (True, False)
has_cycle({0: [1], 1: [0]})                      # -> True
```

Other rules for visited state:

- In an undirected graph, the edge back to the parent is not a cycle. Pass the parent to the search, and skip it.
- Union-find: compare `find(a)` with `find(b)`, never `parent[a]` with `parent[b]`. Link a root to a root.
- Two searches that answer two questions need two visited sets (Pacific Atlantic Water Flow).
- BFS marks a node when it pushes the node (Section 41.12). Dijkstra makes a node final only when it pops the node, because a later push can carry a smaller distance. Skip a popped entry that is larger than the best known distance.
- A search with a limit, such as at most k stops, can need a second visit to one node with fewer stops. Put the limit in the state, or do not mark the nodes.

CAUTION

Progress as an index. Wrong: build a new string or list in each step to record how far a search is, when an index already holds that information. Right: pass the index, and build the output one time at the end. The code is shorter, and each step costs $O(1)$ instead of $O(n)$.

41.7 Recursion

CAUTION

The depth limit. Wrong: a recursive search on a structure with a long path. The default limit is 1000 frames. Examples are a 300×300 grid (depth up to 90,000), a linked list with 5000 nodes, a skewed tree, and a memoized `f(i - 1)` for i up to 10^4. Right: use a loop with an explicit stack or queue.

```
def corridor_wrong(n):              # flood fill a row of n open cells
    seen = set()
    def dfs(c):
        if c < 0 or c >= n or c in seen:
            return
        seen.add(c)
        dfs(c + 1)
        dfs(c - 1)
    dfs(0)
    return len(seen)
def corridor(n):
    seen, stack = {0}, [0]
    while stack:
        c = stack.pop()
        for nc in (c + 1, c - 1):
            if 0 <= nc < n and nc not in seen:
                seen.add(nc)
                stack.append(nc)
    return len(seen)
corridor_wrong(300)                 # -> 300
corridor_wrong(5000)                # -> raises RecursionError
corridor(5000)                      # -> 5000
```

- A function that calls itself is a DFS with a depth limit, whatever its name. A BFS uses a queue and a loop, and it has no depth limit.
- The depth is the longest chain of nested calls, not the number of calls. Estimate it from the worst input shape: one long path, a grid that is all land, a tree that is a chain.
- Some judges raise the limit. Code can then pass on one platform and fail on another platform or on a local run.
- `sys.setrecursionlimit` raises the Python limit only. Very deep recursion can still overflow the C stack and stop the process with no Python traceback. The risk is highest on Python 3.10 and older, and when the calls go through C code, such as a `@cache` wrapper.

CAUTION

Recursion that should be a loop. Wrong: one recursive call for each step of a simulation that never branches. Python does not remove tail calls, so 3000 steps need 3000 frames. Right: a `for` loop.

```
def final_pos_wrong(program, i=0, pos=0):
    if i == len(program):
        return pos
    step = 1 if program[i] == "R" else -1
    return final_pos_wrong(program, i + 1, pos + step)   # one frame per command
def final_pos(program):
    pos = 0
    for ch in program:
        pos += 1 if ch == "R" else -1
    return pos
final_pos_wrong("R" * 3000)         # -> raises RecursionError
final_pos("R" * 3000)               # -> 3000
```

CAUTION

A missing memo. Wrong: a correct recurrence with overlapping states and no cache. The number of calls grows like 2^n. Right: add `@cache`, or fill a table. The cost with a cache is the number of distinct states times

the work for one state.

```
from functools import cache
def best_margin(nums, use_cache):   # first player's best score minus the second player's
    calls = [0]
    def f(i, j):
        calls[0] += 1
        if i > j:
            return 0
        return max(nums[i] - f(i + 1, j), nums[j] - f(i, j - 1))
    if use_cache:
        f = cache(f)                # the recursive calls also go through the cache
    return f(0, len(nums) - 1), calls[0]
nums = [3, 9, 1, 2] * 4             # 16 numbers
best_margin(nums, False)            # -> (28, 131071)
best_margin(nums, True)             # -> (28, 153)
```

CAUTION

The memo key and the "not computed" mark. Wrong: a key that leaves out state that the answer depends on. Also wrong: a test such as `if memo.get(i):`, which treats a stored 0 as "not computed". Right: put every changing input in the key. Test with `i in memo`, or use `None` as the empty mark.

- A path search that marks used cells cannot use `(r, c, i)` as a key. The answer also depends on the used cells (Word Search).
- A search that fills k groups must put the set of used elements in the key, for example as a bitmask.
- Check that a move is allowed before you use the cached answer of the next state. The cache does not know the move (Longest Increasing Path in a Matrix).

```
def calls_needed(test_with_in):
    memo, calls = {}, 0
    def ways(i):                        # every answer is 0 in this example
        nonlocal calls
        calls += 1
        if i < 2:
            return 0
        if test_with_in:
            if i in memo:               # right: a stored 0 is found
                return memo[i]
        elif memo.get(i):               # wrong: a stored 0 looks like "not computed"
            return memo[i]
        memo[i] = ways(i - 1) + ways(i - 2)
        return memo[i]
    ways(20)
    return calls
calls_needed(False), calls_needed(True)  # -> (21891, 39)
```

CAUTION

A value on every path. Wrong: a recursive call whose result is not returned. The function then falls off the end and returns `None`. Right: return a value on every path. Combine the results of the branches with `or`, `and`, `max` or `+`.

```
def contains_wrong(t, x):               # t is None or (value, left, right)
    if t is None:
        return False
    if t[0] == x:
        return True
    contains_wrong(t[1], x) or contains_wrong(t[2], x)    # the result is lost
def contains(t, x):
    if t is None:
        return False
```

```
    return t[0] == x or contains(t[1], x) or contains(t[2], x)
tree = (1, (2, None, None), None)
contains_wrong(tree, 2)                # -> None
contains(tree, 2)                      # -> True
```

41.8 Greedy choices and DP tables

CAUTION

A greedy with no proof. Wrong: take the choice that looks best now, such as the largest coin, the larger end, or the longest word that matches. Right: try to break the rule on a small input first. If a choice now removes a better choice later, use DP. A brute-force cross-check on all small inputs finds a counterexample quickly (Section 42.3).

```
def coins_greedy(coins, amount):           # the largest coin first
    count = 0
    for c in sorted(coins, reverse=True):
        count += amount // c
        amount %= c
    return count if amount == 0 else -1
def coins_dp(coins, amount):
    INF = float("inf")
    dp = [0] + [INF] * amount              # dp[a]: the fewest coins for the amount a
    for a in range(1, amount + 1):
        for c in coins:
            if c <= a:
                dp[a] = min(dp[a], dp[a - c] + 1)
    return dp[amount] if dp[amount] < INF else -1
coins_greedy([1, 3, 4], 6), coins_dp([1, 3, 4], 6)   # -> (3, 2)
```

CAUTION

The base case. Wrong: `dp[0] = 0` in a count of ways. Every later entry is then 0. Right: the empty case has one way, which is to do nothing. Use `dp[0] = 1` for a count, and `dp[0] = True` for a yes-or-no question. Fill the first row and the first column of a 2-D table from the meaning of an empty prefix. An example is `dp[i][0] = i` in an edit distance.

```
def ways(coins, amount, base):             # the number of coin combinations
    dp = [base] + [0] * amount
    for c in coins:
        for a in range(c, amount + 1):
            dp[a] += dp[a - c]
    return dp[amount]
ways([1, 2], 3, base=0), ways([1, 2], 3, base=1)   # -> (0, 2)
```

CAUTION

The meaning of dp[i], and the place of the answer. Wrong: define `dp[i]` as "the best result that ends at index i", and then return `dp[-1]`. Right: write the meaning of `dp[i]` in one sentence before you write code. If `dp[i]` ends at i, the answer is `max(dp)`. If `dp[i]` covers the first i elements, the answer is `dp[n]`.

```
def lis_ending_at(a):          # dp[i]: the longest increasing subsequence that ends at index i
    dp = [1] * len(a)
    for i in range(len(a)):
        for j in range(i):
            if a[j] < a[i]:
                dp[i] = max(dp[i], dp[j] + 1)
    return dp
dp = lis_ending_at([1, 2, 3, 0])
dp[-1], max(dp)                # -> (1, 3)
```

CAUTION

The loop order decides what the DP counts. Wrong: swap the two loops of a count of ways. Right: put the items in the outer loop to count combinations, where the order does not matter. Put the amounts in the outer loop to count ordered sequences.

```
def combinations(coins, amount):           # items outside: 1 + 2 and 2 + 1 are one answer
    dp = [1] + [0] * amount
    for c in coins:
        for a in range(c, amount + 1):
            dp[a] += dp[a - c]
    return dp[amount]
def sequences(coins, amount):              # amounts outside: 1 + 2 and 2 + 1 are two answers
    dp = [1] + [0] * amount
    for a in range(1, amount + 1):
        for c in coins:
            if c <= a:
                dp[a] += dp[a - c]
    return dp[amount]
combinations([1, 2], 3), sequences([1, 2], 3)      # -> (2, 3)
```

Other DP rules:

- Fill each entry after the entries that it reads. If `dp[i][j]` reads `dp[i + 1][j - 1]`, loop `i` downward, or fill by the length of the range (palindromes, interval DP, two-player games).
- A 1-D table that is reused for each item changes its meaning with the direction of the loop. See the synchronous updates in Section 41.5.
- Keep one convention for each state. For example, `dp[i]` includes `cost[i]`, or it does not. A mix of the two adds or drops one cost.

41.9 Hashing and membership

CAUTION

The order of lookup and insert. Wrong: add the current element to the map before you look up its partner. The element can then pair with itself. Right: look up first, then insert. For prefix-sum counts, start the map with `{0: 1}` for the empty prefix.

```
def two_sum_wrong(nums, target):
    pos = {}
    for i, x in enumerate(nums):
        pos[x] = i                         # insert before the lookup
        if target - x in pos:
            return [pos[target - x], i]
def two_sum(nums, target):
    pos = {}
    for i, x in enumerate(nums):
        if target - x in pos:              # look up first: x cannot pair with itself
            return [pos[target - x], i]
        pos[x] = i
two_sum_wrong([3, 2, 4], 6)                # -> [0, 0]
two_sum([3, 2, 4], 6), two_sum([3, 3], 6)   # -> ([1, 2], [0, 1])
```

CAUTION

Unhashable keys and keys that lose information. Wrong: a list as a set element or a dict key. This raises `TypeError`. Also wrong: a key that loses information, such as a set of letters, a sum of character codes, or numbers joined with no separator. Different inputs then get the same key. Right: a tuple, a sorted string, a tuple of counts, or a `frozenset` when the order does not matter and no element repeats.

```
seen = set()
seen.add([1, 2])                                # -> raises TypeError
seen.add((1, 2))
(1, 2) in seen                                  # -> True
set("aab") == set("abb")                        # -> True
sum(map(ord, "bc")) == sum(map(ord, "ad"))      # -> True
"".join(map(str, [1, 11])) == "".join(map(str, [11, 1]))   # -> True
def count_key(word):                            # a full count vector as a tuple
    counts = [0] * 26
    for ch in word:
        counts[ord(ch) - ord("a")] += 1
    return tuple(counts)
count_key("aab") == count_key("abb"), count_key("bc") == count_key("ad")   # -> (False, False)
```

CAUTION

A read that inserts. Wrong: `d[k]` on a `defaultdict` to test or read a key. The read inserts the key with a default value. Later membership tests then pass, and a loop over the dict can raise `RuntimeError`. Right: `k in d`, `d.get(k, default)`, or a `Counter`. A `Counter` returns 0 for a missing key and does not insert it.

```
from collections import defaultdict, Counter
d = defaultdict(list)
len(d["x"])                     # -> 0
"x" in d                        # -> True
d = defaultdict(int, {"a": 1})
for k in d:
    d[k + "!"] += 0
# -> raises RuntimeError
c = Counter()
c["x"]                          # -> 0
"x" in c                        # -> False
```

CAUTION

The truth value of a stored value. Wrong: `if cache.get(k):` when 0, `""` or `[]` are valid stored values. Right: `if k in cache:`.

```
cache = {"a": 0}
"hit" if cache.get("a") else "miss"      # -> 'miss'
"hit" if "a" in cache else "miss"        # -> 'hit'
```

CAUTION

Counter subtraction drops negative counts. Wrong: test that two strings have equal letter counts with `Counter(a) - Counter(b) == Counter()`. The subtraction keeps only positive counts, so the test proves only that `b` covers `a`. Right: `Counter(a) == Counter(b)`, or check that the lengths are equal too.

```
from collections import Counter
Counter("ab") - Counter("abc") == Counter()     # -> True
Counter("ab") == Counter("abc")                 # -> False
```

CAUTION

Equal-looking values of different types. Wrong: compare a tuple with a list, or a digit character with an integer. The result is always `False`, and no error appears. Right: convert one side, so that both sides have the same type.

```
(2, 7) == [2, 7]                # -> False
tuple([2, 7]) == (2, 7)         # -> True
"1" == 1                        # -> False
len({1, "1"})                   # -> 2
```

- Membership in a list costs $O(n)$. Convert the list to a set one time, before the loop. Look for hidden list scans: `x in nums[:i]`, `nums.index(x)`, and a test against an input list of forbidden states.
- A set lookup is $O(1)$ on average, not in the worst case. Say "expected time" in an interview.
- An array indexed by value works only for a small value range. For values up to 10^9, use a dict.
- The iteration order of a set of strings can change between runs. Sort before you print or compare.

41.10 Comparison, ties and sorting

CAUTION

Strict or non-strict. Wrong: `<` where the statement needs `<=`, or the opposite. Right: take the strictness from the statement, and then test an input with equal values. Equal values are the only inputs where the two forms differ.

```
def days_to_warmer(t, pop_on_equal):
    res, stack = [0] * len(t), []          # the stack keeps indexes
    for i, x in enumerate(t):
        while stack and (t[stack[-1]] <= x if pop_on_equal else t[stack[-1]] < x):
            j = stack.pop()
            res[j] = i - j
        stack.append(i)
    return res
days_to_warmer([70, 70, 71], pop_on_equal=True)    # -> [1, 1, 0]
days_to_warmer([70, 70, 71], pop_on_equal=False)   # -> [2, 1, 0]
```

"Warmer" is strict, so the second result is correct. Other places where strictness decides the answer:

- Intervals that touch: Merge Intervals joins `[1, 2]` and `[2, 3]`, so its overlap test is `s <= end`. A meeting that ends at t does not conflict with one that starts at t, so the conflict test is `s < end` (Meeting Rooms).
- Validate Binary Search Tree needs a strict order: an equal value in a subtree makes the tree invalid.
- "No ancestor is greater" means `>=`: a node equal to the path maximum counts (Count Good Nodes).
- Equal arrival times form one group (Car Fleet).
- A sliding window "at most k" shrinks while the window is above k, not while it equals k.

CAUTION

Ties in a best choice. Wrong: update the best index on `>=` when the statement asks for the first maximum. Right: `>` keeps the first maximum, and `>=` keeps the last. `max` and `min` return the first one.

```
sizes = [3, 5, 5]
best = 0
for i in range(1, len(sizes)):
    if sizes[i] >= sizes[best]:
        best = i
best                                          # -> 2
max(range(len(sizes)), key=lambda i: sizes[i])   # -> 1
```

CAUTION

The start value of a running best. Wrong: start a maximum at 0, or a minimum at 0. For all-negative input, a maximum of 0 is not a real answer. For all-positive input, a minimum of 0 never changes. Right: start from the first element, or from `float("-inf")` or `float("inf")`. Do not use a sentinel that can be a real value, such as `-2**31` when the input can hold `-2**31`.

```
def max_sub_wrong(a):                  # largest sum of a non-empty subarray
    best = cur = 0
    for x in a:
        cur = max(x, cur + x)
        best = max(best, cur)
    return best
def max_sub(a):
    best, cur = float("-inf"), 0
```

```
    for x in a:
        cur = max(x, cur + x)
        best = max(best, cur)
    return best
max_sub_wrong([-3, -1])                # -> 0
max_sub([-3, -1])                      # -> -1
```

CAUTION

Compare with the running value, not with the neighbor. Wrong: compare an element only with the previous element, or a tree node only with its parent. Right: compare with a summary of everything before it (Car Fleet, Count Good Nodes, Validate Binary Search Tree). Examples are the end of the current merged group, the running maximum, and the bounds from all ancestors.

```
def groups_wrong(iv):                  # iv is sorted by start; count the merged groups
    count = 1
    for k in range(1, len(iv)):
        if iv[k][0] > iv[k - 1][1]:    # compares with the previous interval only
            count += 1
    return count
def groups(iv):
    count, end = 1, iv[0][1]
    for s, e in iv[1:]:
        if s > end:
            count += 1
        end = max(end, e)              # the running end of the current group
    return count
iv = [[1, 10], [2, 3], [4, 5]]
groups_wrong(iv), groups(iv)           # -> (2, 1)
```

CAUTION

Sort keys and pairs. Wrong: sort two parallel lists separately, or sort the queries and lose their original positions. Right: sort pairs made with `zip`, or sort a list of indexes and write each answer at its original index.

```
position, speed = [10, 0, 5], [1, 3, 2]
sorted(zip(position, speed))           # -> [(0, 3), (5, 2), (10, 1)]
queries = [50, 10, 30]
order = sorted(range(len(queries)), key=lambda i: queries[i])
smaller = [0] * len(queries)           # how many queries are smaller
for rank, i in enumerate(order):
    smaller[i] = rank                  # the answer goes to the original index
smaller                                # -> [2, 0, 1]
```

- Python sorts are stable. Sort by a tuple key, or sort by the secondary key first and by the primary key second.
- A heap is not stable. Equal priorities compare the next field of the tuple. Put a counter before a payload that cannot be compared.
- `heapq` is a min-heap. For a max-heap, negate the key when you push and again when you pop. To keep the k largest values, keep a min-heap and pop its smallest value when the size goes above k.
- A greedy depends on its sort key. Sort intervals by start to merge them. Sort them by end to keep the most intervals that do not overlap. A wrong key gives a plausible wrong count.

41.11 Numbers

CAUTION

Floats in integer problems. Wrong: `a / b`, `math.ceil(a / b)` or `int(a / b)` with large integers. A float keeps 53 bits of precision, so integers above $2^{53} \approx 9 \times 10^{15}$ can lose digits. Right: stay in integers. Use `a // b`, `-(-a // b)` for the ceiling, and cross-multiplication to compare fractions. When the answer is a float, compare it in your tests with `math.isclose`, not with `==`.

```
import math
a = 10**17 + 1
int(a / 1) == a                          # -> False
a // 1 == a                              # -> True
math.ceil(a / 10) == -(-a // 10)         # -> False
-(-a // 10)                              # -> 10000000000000001
0.1 + 0.2 == 0.3                         # -> False
math.isclose(0.1 + 0.2, 0.3)             # -> True
p, q, r, s = 1, 3, 2, 6                  # compare p/q with r/s (q and s positive)
p * s == r * q                           # -> True
```

CAUTION

Negative numbers with // and the modulo operator. Wrong: expect `-7 // 2` to give `-3`, or `-123 % 10` to give 3. Python rounds toward negative infinity, and `%` takes the sign of the divisor. Right: for a quotient toward zero, divide the absolute values and restore the sign. For digits, work on `abs(n)` and keep the sign apart.

```
-7 // 2, -123 % 10, -123 // 10       # -> (-4, 7, -13)
def div_toward_zero(a, b):           # exact, also for very large integers
    q = abs(a) // abs(b)
    return q if (a < 0) == (b < 0) else -q
div_toward_zero(-7, 2)               # -> -3
def digits(n):
    sign, n = (-1 if n < 0 else 1), abs(n)
    out = []
    while n:
        n, d = divmod(n, 10)
        out.append(d)
    return sign, out[::-1]
digits(-123)                         # -> (-1, [1, 2, 3])
(0 - 1) % 10                         # -> 9
```

NOTE

The modulo of a negative number is useful for wrap-around: a wheel digit 0 turned down gives `(0 - 1) % 10`, which is 9. C, C++ and Java round toward zero, and there `-1 % 10` is `-1`. In those languages, write `(d + 9) % 10`.

CAUTION

Overflow in other languages. Python integers never overflow. But an interviewer can ask about a fixed-width language, and some problems define 32-bit limits. Wrong: assume that a value fits because each input value fits. Right: compare the largest intermediate value with $2^{31} - 1 \approx 2.1 \times 10^9$ and with $2^{63} - 1 \approx 9.2 \times 10^{18}$.

- A product of two values up to 10^5 is up to 10^{10}.
- A sum of 10^5 values up to 10^9 is up to 10^{14}.
- A count of pairs or segments, $n(n+1)/2$ for $n = 10^5$, is about 5×10^9.
- `lo + hi` in a binary search can overflow. Write `lo + (hi - lo) / 2` in those languages.
- Intermediate DP counts can be much larger than the final answer.
- For a problem that defines a 32-bit result, test the range in Python too: `-2**31 <= x <= 2**31 - 1`.

CAUTION

Negative numbers in bit tricks. Wrong: `while n: n &= n - 1` on a negative Python integer. Python integers have no fixed width, so a negative number has infinitely many leading 1 bits, and the loop never ends. Right: mask to 32 bits first with `n & 0xFFFFFFFF`. To get the sign back, subtract `2**32` when the masked value is above `0x7FFFFFFF`.

```
n = -5
bin(n)                                  # -> '-0b101'
m = n & 0xFFFFFFFF                      # the 32-bit pattern of -5
bin(m).count("1")                       # -> 31
m if m <= 0x7FFFFFFF else m - 2**32     # -> -5
2 ^ 3, 2 ** 3                           # -> (1, 8)
3 + 1 & 1, 3 + (1 & 1)                  # -> (0, 4)
1 << 2 + 1, (1 << 2) + 1                # -> (8, 5)
```

The last three lines show two more traps. First, `^` is XOR, not a power. Second, `+` and `-` bind more tightly than `<<` and `>>`, and these bind more tightly than `&`, `^` and `|`. Put parentheses around each bit operation in a mixed expression, for example `ans[i // 2] + (i & 1)`.

41.12 Performance traps

A time limit result often comes from one line that does hidden $O(n)$ work inside a loop. The code looks like $O(n)$, but it runs in $O(n^2)$. Part I lists the cost of each operation. The table below lists the patterns that hide this cost, and the fix for each one.

Code inside a loop	Hidden cost	Fix
`x in lst`, `lst.index(x)`	a scan of the list	a set, or a dict from value to index
`q.pop(0)`, `q.insert(0, x)`	a shift of all elements	`collections.deque`
`s[i + 1:]` in a recursive call	a copy of the rest	pass the index `i + 1`
`sum(path)`	a pass over the path	carry a running total as an argument
`len(set(window))`	a pass over the window	keep a count map of the window
`max(a[i:i + k])`	a copy and a pass	a monotonic deque
`s += ch`, `path + [x]`	a copy of the whole list, and often of the whole string	append to a list; `"".join` once at the end
a slice of the best answer at each update	a copy per update	keep the start and the length; slice once
`sorted(...)` for each query	$O(n \log n)$ per query	sort once, or keep a heap
a new copy of the grid in each step, when few cells change	$O(R \cdot C)$ per step	change only the cells that change
`print(...)`	slow output	remove it, or collect the output and print once

CAUTION

The same value computed again at every node. Wrong: call a helper such as `height(node)` at every node of a tree. On a chain-shaped tree this is $O(n^2)$. Right: one post-order pass that returns every value that the parent needs, for example `(height, best)`.

```
calls = 0
def height(t):                          # t is None or (left, right)
    global calls
    calls += 1
    return 0 if t is None else 1 + max(height(t[0]), height(t[1]))
def diameter_slow(t):                   # calls height() at every node
    if t is None:
        return 0
    through = height(t[0]) + height(t[1])
    return max(through, diameter_slow(t[0]), diameter_slow(t[1]))
def diameter_fast(t):                   # one pass returns (height, best)
    if t is None:
        return 0, 0
    lh, lb = diameter_fast(t[0])
    rh, rb = diameter_fast(t[1])
```

```
    return 1 + max(lh, rh), max(lb, rb, lh + rh)
t = None
for _ in range(200):
    t = (t, None)                       # a chain of 200 nodes
diameter_slow(t), calls                 # -> (199, 40200)
diameter_fast(t)[1]                     # -> 199
```

CAUTION

Mark a node when you push it. Wrong: mark a cell as visited when you pop it. The same cell can then enter the queue several times. The answer stays correct, but the work grows. Right: mark the cell when you push it.

```
from collections import deque
def pushes(R, C, mark_on_push):         # BFS over an open R x C grid
    seen = {(0, 0)} if mark_on_push else set()
    q, count = deque([(0, 0)]), 1
    while q:
        r, c = q.popleft()
        if not mark_on_push:
            if (r, c) in seen:
                continue
            seen.add((r, c))
        for nr, nc in ((r + 1, c), (r, c + 1), (r - 1, c), (r, c - 1)):
            if 0 <= nr < R and 0 <= nc < C and (nr, nc) not in seen:
                if mark_on_push:
                    seen.add((nr, nc))
                q.append((nr, nc))
                count += 1
    return count
pushes(30, 30, True), pushes(30, 30, False)     # -> (900, 1741)
```

CAUTION

Debug output in a hot loop. Wrong: `print` calls inside a loop that runs 10^5 times. The output is slow, and the run looks like a time limit. Right: remove debug output before you submit, or send it through one function that you can switch off (see the `log` helper in the next section).

Input shapes that make a correct solution slow:

- A quickselect with a fixed pivot is $O(n^2)$ on sorted input. Use a random pivot. Many equal values can make the parts uneven, even with a random pivot. Use a three-way partition: less than, equal to, and greater than the pivot.
- A loop over a list with many duplicates repeats work. A loop over a set does the work once for each value.
- A backtracking search that passes small tests can time out on a crafted input, such as many equal values or a board of one repeated letter. Prune early, and skip equal choices at the same level.
- A full table of $10^4 \times 10^4$ cells is 10^8 entries: too slow and too large in Python. Keep only the rows that the recurrence reads.

41.13 Input shapes

CAUTION

A grid of strings. Wrong: compare a cell with the integer 1, or assign to a cell of a string row. Strings are immutable. Right: compare with `"1"`. Convert with `[list(row) for row in grid]` when you must write, or keep a separate visited set.

```
grid = ["10",
        "01"]
grid[0][0] == 1             # -> False
grid[0][0] == "1"           # -> True
grid[0][0] = "#"            # -> raises TypeError
```

```
cells = [list(row) for row in grid]
cells[0][0] = "#"
cells[0]                        # -> ['#', '0']
```

CAUTION

Rows and columns. Wrong: use the row count where the column count belongs. A square test input hides this bug. Right: name `R = len(grid)` and `C = len(grid[0])` one time. For a flat index `i`, the cell is `(i // C, i % C)`. Test a 2×3 grid, a 3×2 grid, a $1 \times n$ grid and an $n \times 1$ grid.

```
m = [[1, 2, 3],
     [4, 5, 6]]                 # 2 rows, 3 columns
R, C = len(m), len(m[0])
[m[i // C][i % C] for i in range(R * C)]    # -> [1, 2, 3, 4, 5, 6]
m[4 // R][4 % R]                # -> raises IndexError
```

CAUTION

Empty input, None, and an empty stack. Wrong: read `grid[0]`, `nums[0]`, `root.val`, `stack[-1]` or `heap[0]` before a check. Right: guard first, for example `C = len(grid[0]) if grid else 0`, `if root is None: return 0`, or `while stack and ...`.

```
grid = []
len(grid[0])                    # -> raises IndexError
C = len(grid[0]) if grid else 0
C                               # -> 0
stack = []
stack[-1]                       # -> raises IndexError
bool(stack and stack[-1] == ")")      # -> False
```

CAUTION

A value of 0 is not a missing node. Wrong: `if not node.val` or `if not x` to find a missing node or a missing value. A node with the value 0 is still a node. Right: `if node is None`.

```
class TreeNode:
    def __init__(self, val, left=None, right=None):
        self.val, self.left, self.right = val, left, right
def size_wrong(node):
    if not node or not node.val:      # a node with value 0 looks missing
        return 0
    return 1 + size_wrong(node.left) + size_wrong(node.right)
def size(node):
    if node is None:
        return 0
    return 1 + size(node.left) + size(node.right)
root = TreeNode(0, TreeNode(1), TreeNode(2))
size_wrong(root), size(root)          # -> (0, 3)
```

CAUTION

Tokens. Wrong: find an operator by its first character or by a substring test. The number `"-11"` starts with `"-"`. Right: compare the whole token with a set of operators. Read a number as one whole token, not digit by digit.

```
tokens = ["-11", "3", "-"]
[t for t in tokens if t[0] in "+-*/"]         # -> ['-11', '-']
OPS = {"+", "-", "*", "/"}
[t for t in tokens if t in OPS]               # -> ['-']
text = "12,-3,7"
[int(t) for t in text.split(",")]             # -> [12, -3, 7]
[ch for ch in text if ch.isdigit()]           # -> ['1', '2', '3', '7']
```

The last line shows why a digit-by-digit read fails: it splits 12 and loses the sign of -3.

- Labels from 1 to n: size the arrays $n + 1$, or subtract 1 from each label.
- An undirected edge goes into both adjacency lists.
- Node ids are not always 0 to $n - 1$. Size an array by the largest id, or use a dict.
- The input is not sorted unless the statement says so, even when all the examples are sorted.
- A linked list or a tree can be empty. The head or the root is then `None`.

41.14 Output format

CAUTION

The requested value. Wrong: return values when the statement asks for indexes, a count when it asks for the items, or a margin when it asks for a score. Right: read the return type in the signature and in the examples before you write code. Frequent pairs: index or value, size or string, node or `node.val`, 0-based or 1-based index.

CAUTION

The impossible case. Wrong: return `None`, infinity, or an internal sentinel such as `amount + 1`. Right: convert to the value that the statement gives, such as `-1`, `[]`, `""` or `0`, in one place at the end.

CAUTION

Integer or float. Wrong: `(a + b) // 2` for a median or an average that can end in .5. Right: `(a + b) / 2`. Do not round a float result unless the statement asks for it. Decimal places in the examples are often only for display.

```
a, b = 2, 3
(a + b) // 2, (a + b) / 2            # -> (2, 2.5)
```

CAUTION

Return or print. Wrong: print the answer in a function problem, or return it in a program that reads standard input. Right: a function problem needs `return`. A standard-input problem needs output in the exact format: the separators, one answer on each line, and the spelling of booleans.

```
res = [3, 1, 4]
print(*res)                          # prints: 3 1 4
print(res)                           # prints: [3, 1, 4]
print("\n".join(map(str, res)))      # prints: 3 | 1 | 4
print(f"{2 / 3:.6f}")                # prints: 0.666667
print(str(True).lower())             # prints: true
```

- Keep the original order of the answers when the queries were sorted.
- When the judge accepts any order, sort the outputs in your own tests before you compare them.
- Use the exact method name and signature of the template, including names such as `startsWith`.
- Return lists when the checker compares lists. A strict local checker sees `(1, 2) != [1, 2]`.

42 Debug and verify a solution

Test a solution before you submit it. Three checks find most bugs: a hand trace of a small example, a list of edge cases, and a comparison with a brute force on many small inputs. A fourth check, a timed run on a maximum-size input, finds time limit problems.

42.1 Hand-trace a small example

1. Choose the smallest input that uses every branch of the code. Three to six elements are usually enough.
2. Make one column for each variable that changes, and one row for each loop pass.
3. Fill the rows from the code, not from what the code should do.
4. Compare each row with the value that you expect. The first row that is different shows the bug.

In an interview, trace the example of the statement aloud. The trace also finds rules that you read wrongly. A debug print gives the same table quickly:

```
def lower_bound(a, x, debug=False):
    lo, hi = 0, len(a)
    while lo < hi:
        mid = (lo + hi) // 2
        if debug:
            print(f"lo={lo} hi={hi} mid={mid} a[mid]={a[mid]}")
        if a[mid] < x:
            lo = mid + 1
        else:
            hi = mid
    return lo
lower_bound([1, 3, 5, 7], 5, debug=True)
# prints: lo=0 hi=4 mid=2 a[mid]=5 | lo=0 hi=2 mid=1 a[mid]=3
lower_bound([1, 3, 5, 7], 5)          # -> 2
```

Send debug output through one function. Then one flag switches all of it off, and standard error keeps it apart from the answer.

```
import sys
DEBUG = False
def log(*args):
    if DEBUG:
        print(*args, file=sys.stderr)
log("state:", [1, 2])                 # prints nothing while DEBUG is False
```

42.2 Test the edge cases

Run each case that fits the input type. Decide the expected answer from the statement before you run the code.

Input type	Cases to test
Array	empty; one element; two elements; all equal; all distinct; sorted; reverse sorted; negative values; zeros; duplicates of the answer value; the target at the first and the last index; no valid answer
String	empty; one character; all one letter; no letter repeats; upper and lower case; digits and punctuation; the pattern longer than the text
Grid	1×1; $1 \times n$; $n \times 1$; not square (2×3 and 3×2); all one value; no target cell; a region that touches the border; a long snake-shaped path
Graph	one node; no edges; isolated nodes; two components; a self loop; a cycle; a duplicate edge; a long chain (depth); a star (high degree); labels from 1 to n
Tree or list	empty (`None`); one node; a chain to the left; a chain to the right; a value of 0; negative values; duplicate values
Intervals	touching end points; one inside another; equal intervals; unsorted input; one interval
Numbers	0; 1; negative; the minimum and the maximum of the constraints; values near 2^{31}
Parameters	$k = 0$; $k = 1$; $k = n$; k larger than n; a target of 0; an answer of 0

A small generator keeps the edge cases in one place. Run the solution on each case, and compare each result with the answer that you expect.

```
def edge_arrays(n=6, lo=-3, hi=3):
    yield []                              # empty
    yield [lo]                            # one element
    yield [hi] * n                        # all equal
    yield [0] * n                         # all zero
    yield list(range(n))                  # sorted
    yield list(range(n, 0, -1))           # reverse sorted
    yield [lo, hi] * (n // 2)             # alternating
```

```
    yield [lo] * (n - 1) + [hi]           # the special value at the last index
def longest_run(a):                       # the solution under test
    if not a:
        return 0
    best = cur = 1
    for i in range(1, len(a)):
        cur = cur + 1 if a[i] == a[i - 1] else 1
        best = max(best, cur)
    return best
[longest_run(a) for a in edge_arrays()]   # -> [0, 1, 6, 6, 1, 1, 1, 5]
```

42.3 Cross-check with a brute force

Write the simplest correct solution: try all subarrays, all subsets, or all orders. It is slow, but it is easy to get right. Then compare it with the fast solution on every small input and on many random inputs. The template below returns the first input where the two solutions give different results.

```
import random
from itertools import product
def brute(a):                             # simple and slow: try every subarray
    return max(sum(a[i:j]) for i in range(len(a)) for j in range(i + 1, len(a) + 1))
def fast_wrong(a):                        # the solution under test
    best = cur = 0
    for x in a:
        cur = max(x, cur + x)
        best = max(best, cur)
    return best
def fast(a):
    best, cur = float("-inf"), 0
    for x in a:
        cur = max(x, cur + x)
        best = max(best, cur)
    return best
def all_small_inputs():                   # every list of length 1 to 4, values -2 to 2
    for n in range(1, 5):
        for t in product(range(-2, 3), repeat=n):
            yield list(t)
def random_inputs(count, seed=1):
    rng = random.Random(seed)             # a fixed seed: a failure repeats
    for _ in range(count):
        n = rng.randint(1, 8)
        yield [rng.randint(-5, 5) for _ in range(n)]
def first_failure(solution, inputs):
    for a in inputs:
        got, want = solution(a), brute(a)
        if got != want:
            return a, got, want
    return None
first_failure(fast_wrong, all_small_inputs())       # -> ([-2], 0, -2)
first_failure(fast, all_small_inputs()) is None     # -> True
first_failure(fast, random_inputs(500)) is None     # -> True
```

- Generate the inputs in order of size. The first failure is then the smallest one, and it is easy to trace.
- Keep the value range small, for example -2 to 2. Small ranges make duplicates, zeros and ties, which are the risky cases.
- Use `random.Random(seed)`, so that a failure repeats on the next run.
- When many answers are valid, compare a normal form (for example, the sorted output), or write a checker that tests the properties of the answer.
- For a greedy, the brute force is a practical proof check. If all small inputs agree, the greedy is probably correct.

42.4 Assert invariants

An invariant is a fact that must be true at a fixed point of each loop pass. Write it as an `assert` while you test. A failed assert stops at the first wrong step, not at a wrong final answer many steps later.

```
def longest_unique_wrong(s):           # longest substring with no repeated character
    last, left, best = {}, 0, 0
    for right, ch in enumerate(s):
        if ch in last:
            left = last[ch] + 1        # wrong: left can move back
        last[ch] = right
        window = s[left:right + 1]
        assert len(set(window)) == len(window), (s, left, right)   # O(n): test only
        best = max(best, right - left + 1)
    return best
def longest_unique(s):
    last, left, best = {}, 0, 0
    for right, ch in enumerate(s):
        if ch in last:
            left = max(left, last[ch] + 1)   # left never moves back
        last[ch] = right
        best = max(best, right - left + 1)
    return best
longest_unique_wrong("abba")           # -> raises AssertionError
longest_unique("abba")                 # -> 2
```

Invariants that are worth an assert:

- Sliding window: after the shrink step, the window satisfies the condition.
- Binary search: the answer stays in `[lo, hi]`, and the range gets smaller on every pass.
- Two heaps for a median: every value in the lower heap is at most every value in the upper heap, and the sizes differ by at most one.
- Monotonic stack: the values on the stack stay in order.
- Union-find: the number of components is n minus the number of successful unions.

Remove expensive asserts before you submit. An $O(n)$ assert inside a loop makes the solution $O(n^2)$.

42.5 Read the error message

Read the last line first. It gives the exception type and the message. Then find the lowest frame in the traceback that is in your own code. That line raised the error.

```
Traceback (most recent call last):
  File "solution.py", line 9, in <module>
    print(longest([]))
  File "solution.py", line 2, in longest
    best = nums[0]
IndexError: list index out of range
```

Error	Frequent cause
`IndexError`	an off-by-one bound; an empty input; `a[i + 1]` at the last index; `stack[-1]` on an empty stack
`KeyError`	a read of a missing key; use `in`, `get` or a `defaultdict`
`TypeError` that names `NoneType`	a function with no `return` on some path; `a = a.sort()`; a missing node
`TypeError`: unhashable type	a list or a dict used as a key or as a set element
`TypeError`: `<` not supported	a heap tie that compares payloads; a sort of mixed types or of `None`
`AttributeError` that names `NoneType`	a step past the end of a linked list or a tree; a missing `None` check
`RecursionError`	depth above 1000; a base case that the code never reaches

Error	Frequent cause
`UnboundLocalError`	an assignment to a name of an outer function without `nonlocal`, or to a module name without `global`
`RuntimeError`: changed size	an insert or a delete during a loop over a dict or a set; a `defaultdict` read in that loop
`ValueError` from `max` or `min`	an empty input; use `default=` or a guard
`ZeroDivisionError`	a count of 0, for example no neighbors or a speed of 0

42.6 Time a maximum-size input

Build an input at the largest size that the constraints allow. Choose the shape that is worst for the code. Then time the solution on that input.

```
import random, time
def solve(a):                           # the solution under test
    best, cur = float("-inf"), 0
    for x in a:
        cur = max(x, cur + x)
        best = max(best, cur)
    return best
rng = random.Random(7)
n = 10**5                               # the largest n in the constraints
big = [rng.randint(-10**4, 10**4) for _ in range(n)]
start = time.perf_counter()
answer = solve(big)
elapsed = time.perf_counter() - start
elapsed < 1.0                           # -> True
```

Shapes that often give the worst case:

- sorted, reverse sorted, and all-equal arrays;
- a graph that is one long chain (depth), and a star graph (one node with every edge);
- a grid that is all one value, and a snake-shaped path through a grid;
- a string of one repeated letter, and a pattern that almost matches everywhere;
- the largest values, to check overflow in a fixed-width language.

To find the slow part, run the profiler: `python3 -m cProfile -s cumtime solution.py`. It lists the functions, with the largest cumulative time first.

42.7 When a hidden test fails

A judge result names the kind of failure. Each kind has a short list of usual causes.

Result	Check first
Wrong answer	the success condition and each rule of the statement; strict or non-strict comparisons; the output format; empty and single-element input; negative indexes that wrap; shared rows or aliases; work left after the loop; a greedy with no proof; the DP base case. Then run the brute-force cross-check.
Runtime error	empty input; index bounds; `None` nodes; recursion depth; missing keys; division by zero.
Time limit	the complexity against the constraints; hidden $O(n)$ work in a loop; a loop with no progress on a special input; a recursion with no memo; debug output.
Memory limit	a full table where two rows are enough; stored copies of every path; one copy of the input per step; very deep recursion.

After a fix, search the rest of the code for the same pattern. A bug class, such as a missing lower bound check, usually appears in more than one place.

43 Final checklist

Do these checks in order after the code passes the given examples. Section 41.1 gives the detailed checklist by class of bug.

1. Read the output definition again. Check the type (int, bool, list, string) and the exact format.
2. Check 0-based and 1-based indexes.
3. Test the smallest input: empty list, one element, 1×1 grid, `n = 1`.
4. Test equal values, negative values and duplicates.
5. Find hidden $O(n)$ work inside loops: `in list`, `pop(0)`, slices, `sum`, string `+=`.
6. Check the recursion depth. Use a loop or raise the limit.
7. Check that BFS marks nodes when it adds them to the queue.
8. Check that backtracking appends copies.
9. Check that a simultaneous update reads only old values.
10. Check integer division, negative modulo and rounding rules.
11. Remove debug `print` calls.

Part III

Techniques

44 Hashing: counting, membership and canonical keys

44.1 The idea

A hash set or a hash map is a memory of the items that a scan has already seen. The hash function turns a key into a slot number. So a lookup goes straight to one slot and does not scan the items. This gives $O(1)$ average time for each question about one exact key: "did this key occur?", "where did it occur?", "how many times did it occur?" and "which items have it?". Many brute-force solutions have an inner loop that searches the earlier items for one exact value. Replace that inner loop with one lookup. Then $O(n^2)$ work becomes $O(n)$ work.

The second use is the canonical key. Often two items are "the same" under a rule that ignores one detail, for example the order of the letters. Compute a key that erases that detail. Then equal keys mean the same class. One dictionary insert per item replaces all pairwise comparisons.

Worked example: complement lookup. Find two indexes in `[3, 2, 4]` whose values add up to 6. At index 0, the value is 3 and its partner must be $6 - 3 = 3$. The map is empty, so store `3: 0`. At index 1, the value is 2 and its partner must be 4. The map has no 4, so store `2: 1`. At index 2, the value is 4 and its partner must be 2. The map holds `2: 1`, so the answer is `[1, 2]`. The brute force tests every pair. The map visits each element one time.

Worked example: canonical key. Group the words `eat`, `tea` and `tan` by anagram class. Sort the letters of each word. The keys are `aet`, `aet` and `ant`. The first two words have the same key, so they go into the same bucket.

The output of the problem tells you what to store for each key.

Question	Store	Example
Did this key occur?	a `set`	Contains Duplicate
Where did it occur?	a `dict` from key to index (first index for the longest range, last index for the nearest)	Two Sum
How many times?	a `Counter` from key to count	Valid Anagram, Subarray Sum Equals K
Which items have it?	a `defaultdict(list)` from key to items	Group Anagrams

44.2 Recognize it

- The statement asks if any value repeats, if all values are distinct, or for the first repeated value.
- The statement asks for a pair with an exact relation: a sum, a difference or a remainder equal to a target. One

element then fixes the exact key of its partner.

- The output is a pair of original indexes, not values. A sort loses the indexes, so a map from value to index fits.
- The words anagram, permutation of, rearrange or same characters appear. Compare counts, not order.
- The statement asks to group items, or to count distinct items, that are equal up to one detail: order, shift, rotation or position.
- The statement compares whole collections between two inputs, such as the regions of two grids. Store each collection as a `frozenset` and match them by hash.
- The statement asks for the k most frequent items. Count first, then select.
- The statement asks for runs of consecutive values in an unsorted array and demands $O(n)$ time. Sorting solves it in $O(n \log n)$, so set lookups must replace the sort.
- An item belongs to several groups at the same time (a row, a column, a box, a diagonal), and each group forbids repeats.
- The statement counts contiguous subarrays with an exact sum, and values can be negative. A range sum is a difference of two prefix sums, so this is a pair problem on prefix sums.
- A design problem stores points and counts shapes through a query point. The query and one more point force the rest of the shape.
- The statement packs many strings into one string and must recover them exactly. The strings can hold any character, so no delimiter is safe. Use a length header.
- The constraints give n up to 10^4 or 10^5. This rules out $O(n^2)$ pair checks in Python.
- Values go up to 10^9. An array indexed by value does not fit, but a hash map does. A small fixed alphabet (26 lowercase letters) allows a list of 26 counts.

NOTE

A hash finds only exact keys. If the relation is "at most", "nearest" or "inside a range", a lookup of the value itself does not help. Sort and use two pointers or binary search. Or hash each value to its bucket of a fixed width, and look up only the neighbor buckets.

44.3 Core template

Template A: the seen-before scan. Walk the items one time. For each item, compute the key that a valid partner must have, and ask the structure for that key. Then record the item under its own key. Only the type of the stored value changes from problem to problem.

```
from collections import Counter

def has_duplicate(nums):                # yes or no: a set
    seen = set()
    for x in nums:
        if x in seen:                   # 1. ask
            return True
        seen.add(x)                     # 2. record
    return False

def two_sum(nums, target):              # where: value -> index
    seen = {}
    for i, x in enumerate(nums):
        need = target - x               # the key of the partner
        if need in seen:                # 1. ask
            return [seen[need], i]
        seen[x] = i                     # 2. record
    return []

def count_pairs(nums, target):          # how many: value -> count
    seen = Counter()
    pairs = 0
    for x in nums:
        pairs += seen[target - x]       # 1. ask
        seen[x] += 1                    # 2. record
    return pairs
```

```
has_duplicate([1, 2, 3, 1])              # -> True
has_duplicate([1, 2, 3])                 # -> False
two_sum([3, 2, 4], 6)                    # -> [1, 2]
two_sum([3, 3], 6)                       # -> [0, 1]
count_pairs([1, 5, 3, 3, 3], 6)          # -> 4
```

Invariant: before step j, the structure holds facts about the items $0 \dots j-1$ only. Ask first, then record. So the scan tests every pair $i < j$ exactly one time, at step j, and an item never pairs with itself.

Template B: group by a key. Compute the key of each item and append the item to the bucket of that key.

```
from collections import defaultdict

def group_by(items, key):
    groups = defaultdict(list)           # key -> items with that key
    for item in items:
        groups[key(item)].append(item)
    return list(groups.values())

group_by(["eat", "tea", "tan", "ate", "nat", "bat"], lambda w: "".join(sorted(w)))
# -> [['eat', 'tea', 'ate'], ['tan', 'nat'], ['bat']]
group_by([1, 2, 3, 4, 5], lambda x: x % 2)   # -> [[1, 3, 5], [2, 4]]
```

Invariant: two items share a bucket exactly when their keys are equal. So the key must be equal for two items of one class, and different for two items of different classes. A dict keeps the insertion order, so the groups come out in the order of their first item.

44.4 Variations

44.4.1 Membership: seen before

The question for each element is "did an equal value occur earlier?". A set answers it. Return at the first hit (Contains Duplicate). **Memory hook:** *Ask the set if you saw it, then add it.*

- For a one-line answer, compare sizes: `len(set(nums)) < len(nums)`. This has no early exit.
- To avoid the set, sort and compare neighbors in $O(n \log n)$. Equal values become neighbors. A sorted copy costs $O(n)$ memory. An in-place sort saves the copy but changes the input.
- If the two equal values must be at most k indexes apart, store the last index of each value.
- If the values are $1 \dots n$, a list of $n+1$ flags replaces the set.

```
def first_repeat(nums):
    seen = set()
    for x in nums:
        if x in seen:
            return x
        seen.add(x)
    return None

def near_duplicate(nums, k):             # equal values at most k indexes apart
    last = {}                            # value -> last index
    for i, x in enumerate(nums):
        if x in last and i - last[x] <= k:
            return True
        last[x] = i
    return False

len(set([1, 2, 3, 1])) < 4               # -> True
first_repeat([2, 5, 1, 5, 2])            # -> 5
near_duplicate([1, 2, 3, 1], 3)          # -> True
near_duplicate([1, 2, 3, 1, 2, 3], 2)    # -> False
s = sorted([3, 1, 3])
```

```
any(s[i] == s[i + 1] for i in range(len(s) - 1))   # -> True
```

44.4.2 Complement lookup: the partner fixes its own key

A pair must satisfy an exact relation, so one element gives the exact key of its partner. Do not search for the partner. Look it up among the earlier elements (Two Sum). Store the index when the output is a position. Store a count when the output is a number of pairs. **Memory hook:** *Ask the map for the partner, then add yourself.*

- Sum equal to t: the partner key is $t - x$.
- Difference equal to $k > 0$: the partner keys are $x - k$ and $x + k$.
- Sum divisible by m: store $x \bmod m$. The partner key is $(-x) \bmod m$.
- Equal values such as `[3, 3]` need no special case. The lookup comes before the insert.
- To put every item into exactly one pair, consume the partner. When the lookup finds a waiting partner, subtract 1 from its count. Otherwise, add the item. At the end, no item may wait (Check If Array Pairs Are Divisible by k).
- The same test with counts only: remainder r needs as many items as remainder $m - r$. The remainders 0 and $m/2$ need even counts.
- A count of pairs does not answer "can all items pair up?". `[3, 3, 3, 1]` with $m = 6$ has 3 pairs, but no full pairing.
- The consume scan works in any order because the relation is symmetric: r needs $m - r$, and $m - r$ needs r. If the relation goes one way only, such as x with $2x$, the order matters. Process the values from the smallest absolute value up (Array of Doubled Pairs).

```
from collections import Counter

def pairs_sum_divisible(nums, m):        # count i < j with m dividing a[i] + a[j]
    seen = Counter()                     # remainder -> count
    pairs = 0
    for x in nums:
        pairs += seen[(-x) % m]          # remainder of the partner
        seen[x % m] += 1
    return pairs

def pairs_with_diff(nums, k):            # count i < j with |a[i] - a[j]| == k, k > 0
    seen = Counter()
    pairs = 0
    for x in nums:
        pairs += seen[x - k] + seen[x + k]
        seen[x] += 1
    return pairs

def can_pair_all(nums, m):               # every item in exactly one pair, m divides each sum
    waiting = Counter()                  # remainder -> items that wait for a partner
    for x in nums:
        need = (-x) % m
        if waiting[need]:
            waiting[need] -= 1           # consume one waiting partner
        else:
            waiting[x % m] += 1          # wait for a partner
    return not any(waiting.values())     # no item waits

pairs_sum_divisible([30, 20, 150, 100, 40], 60)   # -> 3
pairs_with_diff([1, 2, 2, 1], 1)                  # -> 4
can_pair_all([1, 2, 3, 4, 5, 10, 6, 7, 8, 9], 5)  # -> True
can_pair_all([-1, 1, -2, 2], 3)                   # -> True
can_pair_all([1, 2, 3, 4, 5, 6], 10)              # -> False
pairs_sum_divisible([3, 3, 3, 1], 6)              # -> 3
can_pair_all([3, 3, 3, 1], 6)                     # -> False
```

NOTE

If the input is sorted and the extra memory must be $O(1)$, use two converging pointers instead. If the relation is an inequality, sort and use two pointers. A hash map does not help there.

44.4.3 Complement lookup on running values

A sum over a contiguous range is a difference of two prefix sums: $\text{sum}(a[i..j-1]) = P[j] - P[i]$. So a range with sum k is a pair of prefixes with $P[j] - P[i] = k$. This is Two Sum on prefix sums (Subarray Sum Equals K). Seed the map with the empty prefix, `{0: 1}`, so that a range can start at index 0. This works with negative values, where a sliding window fails. **Memory hook:** *Two Sum on prefix sums: look up prefix minus k, then store prefix.*

- To count ranges, store the count of each prefix value.
- To find the longest range, store the first index of each prefix value. Seed it with `{0: -1}`.
- For a sum divisible by m, store the prefix mod m instead of the prefix.

```
from collections import Counter

def count_subarrays(nums, k):
    seen = Counter({0: 1})              # the empty prefix
    prefix = count = 0
    for x in nums:
        prefix += x
        count += seen[prefix - k]       # 1. ask
        seen[prefix] += 1               # 2. record
    return count

def longest_subarray(nums, k):
    first = {0: -1}                     # prefix value -> first index
    prefix = best = 0
    for i, x in enumerate(nums):
        prefix += x
        if prefix - k in first:
            best = max(best, i - first[prefix - k])
        first.setdefault(prefix, i)     # keep the first index only
    return best

count_subarrays([1, 1, 1], 2)           # -> 2
count_subarrays([1, 2, 3], 3)           # -> 2
count_subarrays([1, -1, 0], 0)          # -> 3
count_subarrays([0, 0], 0)              # -> 3
longest_subarray([1, -1, 5, -2, 3], 3)  # -> 4
```

44.4.4 Frequency counts: compare multisets

Two sequences are equal up to order exactly when their counts are equal (Valid Anagram). A `Counter` works for any alphabet. A list of 26 counts works for lowercase letters only. **Memory hook:** *Count both, compare counts.*

- The optimal form keeps one table. Check the lengths first. Count the letters of s, then subtract 1 for each letter of t, and return `False` at the first negative count. It has the same $O(n)$ time. In CPython the two-`Counter` form is faster, because `Counter` counts in C.
- `is_anagram26` below adds and subtracts in one pass over both strings. A count can go negative before the end there, so it checks for all zeros only at the end.
- For a one-sided check ("build the note from the letters"), subtract the counters. An empty result means that the letters cover the note. `a <= b` on counters also works [Python 3.10+].

```
from collections import Counter

def is_anagram(s, t):
    return Counter(s) == Counter(t)

def is_anagram26(s, t):                 # lowercase letters only, one table
```

```
    if len(s) != len(t):
        return False
    count = [0] * 26
    for a, b in zip(s, t):
        count[ord(a) - ord("a")] += 1
        count[ord(b) - ord("a")] -= 1
    return all(c == 0 for c in count)

def can_build(note, letters):            # one-sided: letters cover note
    return not (Counter(note) - Counter(letters))

is_anagram("anagram", "nagaram")         # -> True
is_anagram("rat", "car")                 # -> False
is_anagram26("aab", "abb")               # -> False
is_anagram26("listen", "silent")         # -> True
can_build("aab", "baac")                 # -> True
can_build("aa", "ab")                    # -> False
Counter("aab") <= Counter("aabc")        # -> True
```

44.4.5 Canonical keys: group items that are equal up to a detail

Map each item to a key that erases the ignored detail, then group by the key (Group Anagrams). The sorted word is the simplest key for anagrams. **Memory hook:** *Sort each word, group by the sorted word.* The optimal key is a tuple of 26 letter counts. It costs $O(k + 26)$ per word instead of $O(k \log k)$. In CPython the sorted key is as fast or faster, because `sorted` runs in C. Use the count tuple only when you must show $O(k)$ work per word. Before you trust a new key, test two items that must collide and two items that must not.

Items are equal up to	Key
the order of the letters	`"".join(sorted(w))`, or a tuple of 26 counts
the order, any alphabet	the sorted string, or `frozenset(Counter(w).items())`
the same shift of every letter	the tuple of differences of neighbor letters, modulo 26
order and repeats (a set of elements)	`frozenset(items)`
a rotation of a sequence	the smallest rotation
the position of a shape on a grid	a `frozenset` of offsets from the smallest cell

```
from collections import defaultdict

def group_anagrams(words):
    groups = defaultdict(list)
    for w in words:
        groups["".join(sorted(w))].append(w)
    return list(groups.values())

def count_key(w):                        # O(k) key for lowercase words
    count = [0] * 26
    for ch in w:
        count[ord(ch) - ord("a")] += 1
    return tuple(count)                  # a list is not hashable

def smallest_rotation(seq):              # O(k^2), fine for short items
    return min((seq[i:] + seq[:i] for i in range(len(seq))), default=seq)

group_anagrams(["eat", "tea", "tan", "ate", "nat", "bat"])
# -> [['eat', 'tea', 'ate'], ['tan', 'nat'], ['bat']]
group_anagrams(["", "b", ""])            # -> [['', ''], ['b']]
count_key("aab") == count_key("aba")     # -> True
count_key("aab") == count_key("abb")     # -> False
smallest_rotation("cab"), smallest_rotation("bca")   # -> ('abc', 'abc')
smallest_rotation("")                    # -> ''
```

44.4.6 Whole regions as keys: frozensets of cells

To compare whole collections, such as the regions of two grids, store each collection as a `frozenset`. Then one hash lookup finds an equal collection, with no pairwise comparisons. A set intersection counts the common collections in time linear in the total number of cells (for example, count the islands that are identical in two binary grids of the same size). **Memory hook:** *Collect each grid's islands as frozensets, then intersect.* Use an explicit stack for the flood fill. One region can hold the whole grid, and recursion fails at that depth. The optimal solution floods only the first grid and builds no sets. A region matches when the second grid has `1` on every region cell and `0` on every border cell. A connected region can grow only through a neighbor, so the border ring decides "no extra cell".

```
def regions(grid):                        # set of frozensets of (r, c)
    rows, cols = len(grid), len(grid[0])
    seen = [[False] * cols for _ in range(rows)]
    found = set()
    for r in range(rows):
        for c in range(cols):
            if grid[r][c] != "1" or seen[r][c]:
                continue
            seen[r][c] = True             # mark on push
            stack, cells = [(r, c)], []
            while stack:
                i, j = stack.pop()
                cells.append((i, j))
                for x, y in ((i - 1, j), (i + 1, j), (i, j - 1), (i, j + 1)):
                    inside = 0 <= x < rows and 0 <= y < cols
                    if inside and grid[x][y] == "1" and not seen[x][y]:
                        seen[x][y] = True
                        stack.append((x, y))
            found.add(frozenset(cells))
    return found

def shape(cells):                         # a key that ignores the position
    r0, c0 = min(cells)
    return frozenset((r - r0, c - c0) for r, c in cells)

len(regions(["001", "011", "100"]) & regions(["001", "011", "101"]))   # -> 1
len(regions(["101", "010", "101"]))       # -> 5
len({shape(cells) for cells in regions(["101", "010", "101"])})   # -> 1
len(regions(["000"]))                     # -> 0
```

44.4.7 Count, then select by count: bucket sort

Count each value with a hash map. Then select the k values with the largest counts (Top K Frequent Elements). A count is an integer from 1 to n. So a list of $n+1$ buckets, indexed by count, sorts the values by count in $O(n)$ time without comparisons. **Memory hook:** *Bucket values by count, read buckets top down, take the first k.* The general rule: when a sort key has a small known range, use the key as an index instead of comparing keys.

- `Counter(nums).most_common(k)` is the quick version. It uses a heap, $O(m \log k)$ for m distinct values.
- A size-k min-heap gives $O(n + m \log k)$ when the key is not a small integer.
- If ties at the boundary are possible, define a tie rule and sort inside the boundary bucket.

```
from collections import Counter

def top_k_frequent(nums, k):
    count = Counter(nums)
    buckets = [[] for _ in range(len(nums) + 1)]   # buckets[f]: values that occur f times
    for value, freq in count.items():
        buckets[freq].append(value)
    by_freq = []
    for freq in range(len(nums), 0, -1):           # highest count first
        by_freq.extend(buckets[freq])
    return by_freq[:k]
```

```
top_k_frequent([1, 1, 1, 2, 2, 3], 2)    # -> [1, 2]
top_k_frequent([4], 1)                   # -> [4]
[v for v, _ in Counter([1, 1, 1, 2, 2, 3]).most_common(2)]   # -> [1, 2]
```

44.4.8 Neighbors in value space: walk only from run starts

If a solution sorts only to find the neighbor of a value ($x+1$, $x+d$, $2x$), a set finds that neighbor in $O(1)$. Put the values in a set. A value starts a run exactly when $x-1$ is not in the set. Walk forward only from a run start (Longest Consecutive Sequence). Each value is then visited by one walk only, so the total work is $O(n)$. **Memory hook:** *Only start counting where x-1 is missing.* The general rule: to do the work of each group one time, find a cheap local test that is true for exactly one member of the group. Start the work only from that member.

```
def longest_run(nums):
    values = set(nums)
    longest = 0
    for start in values:                 # loop over the set: each start walks once
        if start - 1 in values:
            continue                     # not a run start
        length = 1
        while start + length in values:
            length += 1
        longest = max(longest, length)
    return longest

def count_runs(nums):
    values = set(nums)
    return sum(1 for x in values if x - 1 not in values)

longest_run([100, 4, 200, 1, 3, 2])      # -> 4
longest_run([0, 3, 7, 2, 5, 8, 4, 6, 0, 1])   # -> 9
longest_run([])                          # -> 0
count_runs([1, 2, 3, 10, 11, 20])        # -> 3
```

44.4.9 Several groups per item: group ids from coordinates

Each item belongs to several groups, and each group forbids repeats (Valid Sudoku). Give every group its own seen-set. Compute the group ids of a cell from its coordinates: the row is r, the column is c, and the $k \times k$ box is `(r // k, c // k)`. On a diagonal from top left to bottom right, $r - c$ is constant. On the other diagonal, $r + c$ is constant. **Memory hook:** *Each digit checks in at its row, its column and its box (r // 3, c // 3). A second check-in at any of the three fails.* The flat box id `(r // 3) * 3 + c // 3` lets you keep the boxes in a list of 9 sets. The sum `r // 3 + c // 3` is wrong: boxes `(0, 1)` and `(1, 0)` get the same id.

```
from collections import defaultdict

def valid_board(board, k):               # N x N board, k x k boxes, "." is empty
    seen = defaultdict(set)              # group id -> values in that group
    for r, row in enumerate(board):
        for c, v in enumerate(row):
            if v == ".":
                continue
            groups = (("row", r), ("col", c), ("box", r // k, c // k))
            if any(v in seen[g] for g in groups):
                return False
            for g in groups:
                seen[g].add(v)
    return True

valid_board(["12..", "..1.", "....", "...."], 2)   # -> True
valid_board(["1...", ".1..", "....", "...."], 2)   # -> False
valid_board(["1...", "....", "1...", "...."], 2)   # -> False
```

```
r, c = 4, 7
(r // 3, c // 3), (r // 3) * 3 + c // 3  # -> ((1, 2), 5)
```

The 9 by 9 board is the call with `k = 3`. The tag in each group id (`"row"`, `"col"`, `"box"`) keeps a row id and a column id apart in one dict.

44.4.10 Shapes through a query point: enumerate the free part, look up the forced part

Count the shapes that contain a query point (Detect Squares). Fix the query corner (q_x, q_y). Then one more point fixes an axis-aligned square: the diagonal corner (x, y), with $|x - q_x| = |y - q_y| \neq 0$. The other two corners are forced: (x, q_y) and (q_x, y). Look them up in a `Counter` of points. Duplicate points are different choices, so multiply the counts. **Memory hook:** *Pick the diagonal, the other two corners are forced, multiply the counts.* The general rule: find the smallest set of elements that fixes the whole shape. Enumerate only those elements, and look up the rest.

```
from collections import Counter

class DetectSquares:
    def __init__(self):
        self.points = Counter()          # (x, y) -> number of copies

    def add(self, point):
        self.points[tuple(point)] += 1

    def count(self, point):
        qx, qy = point
        total = 0
        for (x, y), copies in self.points.items():
            if x == qx or abs(x - qx) != abs(y - qy):
                continue                 # not the diagonal corner of a real square
            total += copies * self.points.get((x, qy), 0) * self.points.get((qx, y), 0)
        return total

ds = DetectSquares()
for p in ([3, 10], [11, 2], [3, 2]):
    ds.add(p)
ds.count([11, 10])                       # -> 1
ds.count([14, 8])                        # -> 0
ds.add([11, 2])
ds.count([11, 10])                       # -> 2
```

44.4.11 Length-prefixed encoding

Pack a list of strings into one string and get the list back (Encode and Decode Strings). A delimiter fails when the data can contain it: `["a,b"]` and `["a", "b"]` join to the same text. Write each item as its length in digits, then a marker, then the item. The header holds only digits, so the first marker after the header start ends the header. Then take exactly that many characters and jump over them. Never search inside the payload. **Memory hook:** *Write length, hash, string; read length, then jump.* The general rule: put the data that a reader needs before the item that it describes. Write that header in an alphabet that cannot contain the marker.

```
def encode(strs):
    return "".join(f"{len(s)}#{s}" for s in strs)

def decode(s):
    res, i = [], 0
    while i < len(s):
        j = s.index("#", i)              # the marker that ends the header
        length = int(s[i:j])
        start = j + 1
        res.append(s[start:start + length])
        i = start + length               # jump over the payload
```

```
    return res

encode(["a#b", "", "12"])                # -> '3#a#b0#2#12'
decode(encode(["a#b", "", "12"]))        # -> ['a#b', '', '12']
decode(encode([""])), decode(encode([]))   # -> ([''], [])
",".join(["a,b"]) == ",".join(["a", "b"])  # -> True
```

Move an index. Do not cut the used prefix off the string in each step. Each cut copies the rest, and decode becomes $O(n^2)$.

44.5 Adapt it to a new problem

Ask these questions in this order.

1. What does the inner loop of the brute force search for? If it searches for one exact key, replace the loop with a lookup. If it searches for a range or the nearest value, use sorting instead.
2. What is the key? It can be the value itself, the partner key (`target - x`), a running value (`prefix - k`, the prefix modulo m), a canonical form (sorted, counts, offsets), or a group id (`(r // k, c // k)`).
3. Does the relation mix two elements, or an index and a value? Move all terms of one element to one side. For example, `a[j] - a[i] == j - i` becomes `a[i] - i == a[j] - j`. Then each side is a key of one element, and equal keys form the pairs.
4. What is the stored value? Read it from the output: a set for yes or no, an index for a position, the first index for a longest range, a count for a number of pairs, a list for groups.
5. When do you record? For pairs $i < j$, ask first and record second. Put the start state (the empty prefix) into the map before the loop.
6. Is the key hashable and exact? Use a tuple, a `frozenset` or a string. Keep the counts. Put a separator between numbers in a string key. Test two items that must collide and two that must not.
7. Which container? Use a list when the keys are small dense integers (26 letters, values $1 \dots n$, counts $0 \dots n$). Use a dict or a set for all other keys.
8. Do duplicates count as different choices? If yes, store counts and multiply them. If no, use a set, and loop over the set.
9. Must every item go into exactly one pair? Then a pair count is not the answer. On a hit, consume the waiting partner. At the end, check that no item waits.
10. Does every member of a group repeat the same work? Find a cheap test that selects one member per group, such as "$x - 1$ is absent". Start the work only there.
11. Is the last step a selection by count? Bucket by count (a count is at most n), keep a size-k heap, or call `most_common(k)`.
12. Does each query force most of a shape? Enumerate the free part only, and look up the forced part in a counter.
13. Must many strings go into one string and come back? Write a length header before each item. Read the header, then jump by the length.

44.6 Pitfalls

CAUTION

Record before you ask. The current item then finds itself. In Two Sum it pairs with itself. In prefix counting with $k = 0$ it counts empty ranges. Always ask first, then record.

```
def two_sum_wrong(nums, target):         # records before it asks
    seen = {}
    for i, x in enumerate(nums):
        seen[x] = i
        if target - x in seen:
            return [seen[target - x], i]

two_sum_wrong([3, 2, 4], 6)              # -> [0, 0]
```

CAUTION

A missing start state. In a prefix scan, the empty prefix is an earlier prefix too. Without the seed `{0: 1}`, every range that starts at index 0 is lost.

```
from collections import Counter

def count_subarrays(nums, k, seed):
    seen = Counter(seed)
    prefix = count = 0
    for x in nums:
        prefix += x
        count += seen[prefix - k]
        seen[prefix] += 1
    return count

count_subarrays([1, 2, 3], 3, {0: 1})    # -> 2
count_subarrays([1, 2, 3], 3, {})        # -> 1
```

CAUTION

A key that loses information. A set of letters loses the counts. A sum or an XOR of character codes collides. Numbers joined with no separator collide. Each wrong key merges two different classes.

```
set("aab") == set("abb")                 # -> True
sum(map(ord, "bc")) == sum(map(ord, "ad"))   # -> True
"".join(map(str, [1, 11])) == "".join(map(str, [11, 1]))   # -> True
",".join(map(str, [1, 11])) == ",".join(map(str, [11, 1]))   # -> False
```

CAUTION

A mutable key. A list or a set cannot be a dict key or a set element. Convert it to a tuple or a `frozenset`.

```
d = {}
d[[1, 2]] = "x"                          # -> raises TypeError
d[(1, 2)] = "x"
frozenset({1, 2}) in {frozenset({2, 1})}   # -> True
```

CAUTION

A read that inserts. A read of a missing key in a `defaultdict` inserts that key. Inside a loop over the same dict, this raises `RuntimeError`. A `Counter` read and `dict.get` do not insert.

```
from collections import Counter, defaultdict
d = defaultdict(int, {1: 1})
d[5]                                     # -> 0
len(d)                                   # -> 2
def look_ahead(table):
    for key in table:
        table[key + 100]                 # inserts a key during the loop
look_ahead(d)                            # -> raises RuntimeError
c = Counter({1: 1})
c[5], len(c)                             # -> (0, 1)
look_ahead(c)
len(c)                                   # -> 1
```

CAUTION

A hidden linear scan. `x in some_list`, `nums.index(x)` and `x in nums[:i]` each cost $O(n)$. Inside a loop, the total becomes $O(n^2)$ again. Keep the seen items in a set or a dict.

CAUTION

Shared buckets. `[[]] * n` repeats one list n times. Build buckets with a comprehension.

```
buckets = [[]] * 3
buckets[1].append(7)
buckets                                  # -> [[7], [7], [7]]
buckets = [[] for _ in range(3)]
buckets[1].append(7)
buckets                                  # -> [[], [7], []]
```

CAUTION

Counter comparison details. Subtraction drops counts that are zero or negative. So an empty `a - b` proves only that `b` covers `a`, not that they are equal. Check the lengths too. Since Python 3.10, `Counter` equality treats a missing key as count 0. A plain dict does not.

```
from collections import Counter
Counter("ab") - Counter("abc")           # -> Counter()
Counter("ab") == Counter("abc")          # -> False
Counter(a=1, b=0) == Counter(a=1)        # -> True
{"a": 1, "b": 0} == {"a": 1}             # -> False
```

CAUTION

Run starts over the list, not over the set. If one run start occurs many times in the input, a loop over the list walks the same run many times. The time becomes $O(n^2)$. Loop over the set. Do not add to or remove from a set while a loop runs over it. Python raises `RuntimeError`.

NOTE

Hash operations are $O(1)$ on average only. The order of a set of strings changes from run to run. Sort a result before you compare it in a test, and do not print a set as an answer.

44.7 Problems in this family

Problem	What changes	Memory hook
Contains Duplicate	A set of seen values. Return at the first hit.	Ask the set if you saw it, then add it.
Two Sum	A map from value to index. Look up `target - x`.	Ask the map for the partner, then add yourself.
Subarray Sum Equals K	Two Sum on prefix sums. A count map seeded with the empty prefix.	Two Sum on prefix sums: look up prefix minus k, then store prefix.
Valid Anagram	Two count tables, compared.	Count both, compare counts.
Group Anagrams	Canonical key: the sorted word. A dict of lists.	Sort each word, group by the sorted word.
Top K Frequent Elements	Count, then bucket by count. A count is at most n.	Bucket values by count, read buckets top down, take the first k.
Longest Consecutive Sequence	A set of values. Walk only from a run start.	Only start counting where x-1 is missing.
Valid Sudoku	One seen-set per row, column and box. Box id `(r // 3, c // 3)`.	Each digit checks in at its row, its column and its box (r // 3, c // 3). A second check-in at any of the three fails.
Detect Squares	A counter of points. Enumerate the diagonal corner, look up the forced corners, multiply the counts.	Pick the diagonal, the other two corners are forced, multiply the counts.
Encode and Decode Strings	A length header and a marker. Jump by the length, never search the payload.	Write length, hash, string; read length, then jump.

The transfer drills for this technique are in Section 69.2 (page 418). The answers are in Section A.1 (page 428).

45 Prefix sums, running aggregates and offsets

45.1 The idea

A brute force often computes the same aggregate many times. For each index, it adds, multiplies or compares all the earlier elements again. A prefix sum, a running minimum or a running offset does that work one time. One pass carries a summary of everything seen so far. Each later step reads the summary in $O(1)$ instead of a new inner loop. This usually changes $O(n^2)$ into $O(n)$. In this section, a *range* is a contiguous part of an array, also called a subarray. `nums[l..r]` includes both ends.

Worked example. Take `nums = [3, -1, 4, 2]`. The prefix sums are `P = [0, 3, 2, 6, 8]`. `P[i]` is the sum of the first `i` elements, and `P[0] = 0` is the empty prefix. The sum of `nums[1..2]` is `P[3] - P[1]`, that is $6 - 3 = 3$. Check: $-1 + 4 = 3$. So a range sum is the difference of two prefix values. A question about a range becomes a question about a pair of prefixes.

A second example. Take the prices `[7, 1, 5, 3, 6, 4]` and sell on day 4 at price 6. The best earlier day to buy is the cheapest day so far, at price 1. The profit is 5. The cheapest price so far is one variable. It replaces the loop over all earlier days.

Three questions pick the main tool:

- **Does the operation have an inverse?** Addition and XOR have one. Remove one prefix from another (subtract a sum, XOR an XOR) to get any range. Maximum, minimum, gcd and a product with zeros have no usable inverse. For them, a prefix pass and a suffix pass answer "all except `i`" and "left and right of `i`". They do not give an arbitrary range.
- **What do you need from the past?** A count or the first index of each prefix value needs a hash map. The best earlier partner needs one running minimum or maximum.
- **Does everything move together?** Then one shared offset describes all the items. Move the frame, not the items. Track the offset and its extremes.

Summary of the past	Build step	Answers
Prefix sum `P`	`P[i + 1] = P[i] + a[i]`	Any range sum in $O(1)$. With a hash map: count or find ranges with an exact sum.
Prefix and suffix aggregate	A left pass and a right pass	"All except `i`", "left of `i` and right of `i`".
Running minimum or maximum	`low = min(low, x)`	The best pair `i < j` when the best left partner is an extreme.
2D prefix table	Cell + top + left − overlap	Any rectangle sum in $O(1)$.
Difference array	`+v` at `l`, `-v` at `r + 1`	Many range updates, then one read.
Shared offset and its extremes	`off += step`, then min and max	Rigid moves of all items, with loss at a boundary.
Running left bound	`left = max(left, ...)`	Count the ranges whose property survives shrinking.

45.2 Recognize it

- The statement asks for the sum of a contiguous range, or for many range sums of static data.
- The task counts ranges with an exact sum, a sum divisible by m, or a balance of two kinds of items. The values can be negative or zero, so a sliding window fails.
- The answer at index `i` uses "all elements except `i`", or "the elements left of `i` and right of `i`". The statement forbids the inverse operation, for example division.
- A per-index formula uses `min(best on the left, best on the right)`, as with walls and water.
- The task maximizes a score of a pair with the order `i < j`: "buy before sell", "an earlier day".
- A static grid needs many rectangle or square sums.
- The task asks for the largest size of a region under a sum limit, and all values are non-negative.
- Many updates add a value to a range: bookings, trips, shifts over an index range. The task reads the final values, or the peak value, one time.
- The same move applies to every item at the same time, and items that cross a boundary are lost. The number of steps times the number of items is far above 10^7.

- The input gives moves or differences, not positions. Each position is a running sum of the moves, and every position must stay inside bounds.
- The task counts contiguous ranges that must not contain something. Every range inside a valid range is valid.
- n is 10^4 or more, so $O(n^2)$ is too slow in Python. The target is one or a few linear passes.

45.3 Core template

The prefix array. Pad it with one zero at the front. Then no range needs a special case.

```
from itertools import accumulate

nums = [3, -1, 4, 2]
P = [0]
for x in nums:
    P.append(P[-1] + x)
P                                   # -> [0, 3, 2, 6, 8]
list(accumulate(nums, initial=0))   # -> [0, 3, 2, 6, 8]

def range_sum(l, r):                # sum of nums[l..r], both ends included
    return P[r + 1] - P[l]

range_sum(1, 2)                     # -> 3
range_sum(0, 3)                     # -> 8
range_sum(2, 2)                     # -> 4
```

Invariant: `P[i]` is the sum of `nums[0..i-1]`, and `P[0] = 0` is the empty prefix. The sum of `nums[l..r]` is `P[r + 1] - P[l]`. The `initial` argument of `accumulate` needs [Python 3.8+].

The one-pass sweep. Most problems in this family do not store `P`. They sweep one time and keep a summary of the earlier prefixes. Each step has four parts: seed, extend, ask, add.

```
from collections import defaultdict

def count_ranges_with_sum(nums, k):
    seen = defaultdict(int)
    seen[0] = 1                     # 1. seed: the empty prefix is an earlier prefix
    prefix = 0
    count = 0
    for x in nums:
        prefix += x                 # 2. extend: the prefix that ends here
        count += seen[prefix - k]   # 3. ask: earlier prefixes p with prefix - p == k
        seen[prefix] += 1           # 4. add: only now is this prefix in the past
    return count

count_ranges_with_sum([1, 1, 1], 2)       # -> 2
count_ranges_with_sum([1, 2, 3], 3)       # -> 2
count_ranges_with_sum([3, -1, 4, 2], 3)   # -> 2
count_ranges_with_sum([0, 0], 0)          # -> 3
```

Invariant: before the step for index `j`, the summary holds exactly the prefixes that end before `j`, plus the empty prefix. Ask before you add. Then a range never pairs a prefix with itself, and every range is non-empty.

The variations fill the same four slots in different ways:

Problem shape	Summary of the past	Seed	Ask at each step
Count ranges with sum `k`	Count of each prefix value	`{0: 1}`	`seen[prefix - k]`
Longest range with sum `k`	First index of each prefix value	`{0: -1}`	`j - first[prefix - k]`
Best pair with `i < j`	Running minimum or maximum	`a[0]`, or $\pm\infty$	`a[j] - low`
Rigid moves, loss at a bound	Minimum and maximum of the offset	`0` and `0`	After the loop: the interval of surviving starts
Ranges that survive shrinking	Smallest valid left end	The first index	`r - left + 1`

45.4 Variations

45.4.1 Ranges with an exact sum: prefix sums and a hash map

The core template above is **Subarray Sum Equals K**. **Memory hook:** Two Sum on prefix sums: look up prefix minus k, then store prefix. A range `nums[i..j]` has sum `k` exactly when an earlier prefix equals `prefix - k`. A sliding window does not work here, because negative values make the window sum go up and down.

Change the key or the stored value for related shapes:

- **Divisible by m:** use `prefix % m` as the key. Two equal remainders bound a divisible range. Python gives a non-negative remainder for $m > 0$. (Subarray Sums Divisible by K; Continuous Subarray Sum needs a length of at least 2, so it stores first indexes.)
- **Longest range:** store the first index of each prefix value, and seed `{0: -1}`. Do not overwrite an index that is already in the map. (Maximum Size Subarray Sum Equals k)
- **Balance of two kinds of items:** map one kind to $+1$ and the other kind to -1. A balanced range has sum 0, so it starts and ends at equal prefix values.
- **Paths in a tree:** run the same map along the root-to-node path. Undo the insert when you backtrack. (Path Sum III)
- **Submatrices:** fix a pair of rows, collapse the columns into one array, and run the 1D sweep. (Number of Submatrices That Sum to Target)

```python
def count_divisible(nums, m):
    seen = {0: 1}
    prefix = count = 0
    for x in nums:
        prefix = (prefix + x) % m             # never negative for m > 0
        count += seen.get(prefix, 0)
        seen[prefix] = seen.get(prefix, 0) + 1
    return count

def longest_with_sum(nums, k):
    first = {0: -1}                           # the empty prefix ends at index -1
    prefix = best = 0
    for j, x in enumerate(nums):
        prefix += x
        if prefix - k in first:
            best = max(best, j - first[prefix - k])
        if prefix not in first:               # keep the FIRST index only
            first[prefix] = j
    return best

count_divisible([4, 5, 0, -2, -3, 1], 5)      # -> 7
longest_with_sum([1, -1, 5, -2, 3], 3)        # -> 4
longest_with_sum([-2, -1, 2, 1], 1)           # -> 2
longest_with_sum([1, 2], 7)                   # -> 0
```

NOTE

When all values are positive and the condition is "sum at least k" or "sum at most k", a sliding window is simpler and uses $O(1)$ space. The hash map handles exact targets and negative values.

45.4.2 Everything except one index: a prefix pass and a suffix pass

The answer at `i` is the aggregate of the left side combined with the aggregate of the right side. In **Product of Array Except Self**, division is forbidden, and zeros break it anyway. **Memory hook:** Answer = left product times right product. In each pass, use the running product, then multiply in nums[i].

Write the left products into the output. Then sweep from the right with one scalar and multiply it in. This uses $O(1)$ extra space, because the output array does not count. The same two passes work for any associative operation. Start each pass at the identity, which is the value of an empty side. The identity is 1 for a product, 0 for a sum, `-inf` for a maximum, `inf` for a minimum, and 0 for gcd and OR. The two passes matter most when the operation has no inverse.

```
def product_except_self(nums):
    n = len(nums)
    answer = [1] * n
    prefix = 1
    for i in range(n):                  # answer[i] = product of nums[0..i-1]
        answer[i] = prefix
        prefix *= nums[i]
    suffix = 1
    for i in range(n - 1, -1, -1):      # multiply in the product of nums[i+1..n-1]
        answer[i] *= suffix
        suffix *= nums[i]
    return answer

product_except_self([1, 2, 3, 4])       # -> [24, 12, 8, 6]
product_except_self([-1, 1, 0, -3, 3])  # -> [0, 0, 9, 0, 0]
product_except_self([0, 0, 5])          # -> [0, 0, 0]
```

General rule: when a result needs two directional aggregates, store one direction and stream the other.

45.4.3 Two walls: prefix maximum and suffix maximum

In **Trapping Rain Water**, the water at index `i` rises to the lower of two walls. One wall is the tallest bar in `height[0..i]`. The other wall is the tallest bar in `height[i..n-1]`. **Memory hook:** Water at i is the lower wall minus the bar: min(left max, right max) - height. Both maxima include `height[i]`, so no term is negative and the end bars add 0.

```
from itertools import accumulate

def trap(height):
    n = len(height)
    left_max = [0] * n
    left_max[0] = height[0]
    for i in range(1, n):
        left_max[i] = max(left_max[i - 1], height[i])
    right_max = [0] * n
    right_max[n - 1] = height[n - 1]
    for i in range(n - 2, -1, -1):
        right_max[i] = max(right_max[i + 1], height[i])
    return sum(min(left_max[i], right_max[i]) - height[i] for i in range(n))

trap([0, 1, 0, 2, 1, 0, 1, 3, 2, 1, 2, 1])       # -> 6
trap([4, 2, 0, 3, 2, 5])                         # -> 9
trap([5])                                        # -> 0
a = [0, 2, 1, 3, 1]
list(accumulate(a, max))                         # -> [0, 2, 2, 3, 3]
list(accumulate(reversed(a), max))[::-1]         # -> [3, 3, 3, 3, 1]
```

`accumulate(a, max)` builds a running maximum in one line. Apply it to `reversed(a)` and reverse the result to get the suffix maximum.

NOTE

The optimal solution uses two converging pointers and $O(1)$ extra space. Only the lower wall matters. If `left_max < right_max`, the true right wall of the next left index is at least `right_max`. So `left_max` alone sets its level. Settle the side with the smaller running maximum and move that pointer inward. Use this form only when the problem asks for constant extra space.

45.4.4 The best earlier partner: a running minimum or maximum

Best Time to Buy and Sell Stock asks for the largest `prices[j] - prices[i]` with `i < j`, or 0. Fix the sell day `j`. The best buy day is the cheapest day at or before `j`. **Memory hook:** Remember the cheapest day, sell today.

In general, maximize a score of a pair `i < j`. If the score splits into `f(i) + g(j)`, keep the running best of `f(i)` and combine it with `g(j)` at each step. For the stock, `f(i) = -prices[i]` and `g(j) = prices[j]`. If the best left partner is not an extreme but an exact value, keep a hash map instead (Two Sum).

```
def max_profit(prices):
    min_price = prices[0]           # cheapest buy price so far
    best = 0                        # 0 means "do not trade"
    for price in prices:
        min_price = min(min_price, price)
        best = max(best, price - min_price)
    return best

max_profit([7, 1, 5, 3, 6, 4])      # -> 5
max_profit([7, 6, 4, 3, 1])         # -> 0
max_profit([2, 4, 1])               # -> 2
```

Here the update of the minimum comes before the ask. That is safe, because a pair of one day with itself gives profit 0. That value never beats the start value 0.

The same scan on prefix sums gives the largest range sum (Maximum Subarray). A range sum is `P[j] - P[i]`, so keep the smallest earlier prefix. Here the order matters: ask before you add, so the range is not empty. This is Kadane's algorithm seen through prefix sums.

```
def max_subarray(nums):             # the subarray must not be empty
    prefix, low, best = 0, 0, float("-inf")
    for x in nums:
        prefix += x
        best = max(best, prefix - low)   # ask first
        low = min(low, prefix)           # then add this prefix to the past
    return best

max_subarray([-2, 1, -3, 4, -1, 2, 1, -5, 4])   # -> 6
max_subarray([-3, -1])                          # -> -1
```

45.4.5 2D prefix sums and the largest square under a limit

A direct sum of a $k \times k$ square reads k^2 cells. A padded table `P` with `(rows + 1) x (cols + 1)` cells gives any rectangle sum with four lookups. `P[r][c]` is the sum of the block above row `r` and left of column `c`.

- **Build:** the cell, plus the block above, plus the block to the left, minus their overlap.
- **Query:** the whole block, minus the top strip, minus the left strip, plus the corner that you subtracted two times.

Memory hook: whole, minus top, minus left, plus the corner. When all values are non-negative, a larger square has a larger or equal sum than a square inside it. So the condition "a square of side `k` fits" is monotone in `k`. Binary search the largest side. Each check scans all corners in $O(R \cdot C)$. The total is $O(R \cdot C \log \min(R, C))$.

- **Maximum Side Length of a Square with Sum Less than or Equal to Threshold** needs *some* square of

side k to fit: any. **Memory hook:** Prefix table for O(1) squares, then binary search the biggest side that fits.
- **A largest side where all squares must fit** needs *every* square of side k to fit: all. **Memory hook:** Cells are non-negative, so if size k fails, every bigger size fails. Binary search k, and check all k x k squares with 2D prefix sums.

```
def build(grid):
    rows, cols = len(grid), len(grid[0])
    P = [[0] * (cols + 1) for _ in range(rows + 1)]
    for r in range(rows):
        for c in range(cols):
            P[r + 1][c + 1] = grid[r][c] + P[r][c + 1] + P[r + 1][c] - P[r][c]
    return P

def rect_sum(P, r1, c1, r2, c2):        # corners (r1, c1) and (r2, c2), both included
    return P[r2 + 1][c2 + 1] - P[r1][c2 + 1] - P[r2 + 1][c1] + P[r1][c1]

def largest_side(grid, limit, every):
    rows, cols = len(grid), len(grid[0])
    P = build(grid)
    def fits(k):
        ok = (rect_sum(P, r, c, r + k - 1, c + k - 1) <= limit
              for r in range(rows - k + 1) for c in range(cols - k + 1))
        return all(ok) if every else any(ok)
    lo, hi, best = 1, min(rows, cols), 0    # side 0 always fits
    while lo <= hi:
        mid = (lo + hi) // 2
        if fits(mid):
            best, lo = mid, mid + 1         # mid fits: try bigger
        else:
            hi = mid - 1                    # mid fails: try smaller
    return best

P = build([[1, 2], [3, 4]])
P                                       # -> [[0, 0, 0], [0, 1, 3], [0, 4, 10]]
rect_sum(P, 1, 0, 1, 1), rect_sum(P, 0, 1, 1, 1)   # -> (7, 6)
mat = [[1, 1, 3, 2, 4, 3, 2] for _ in range(3)]
largest_side(mat, 4, every=False)       # -> 2
largest_side([[2] * 5 for _ in range(5)], 1, every=False)   # -> 0
largest_side([[1] * 3 for _ in range(3)], 4, every=True)    # -> 2
largest_side([[1, 2], [3, 4]], 4, every=True)               # -> 1
largest_side([[5, 0], [0, 0]], 4, every=True)               # -> 0
largest_side([[5, 0], [0, 0]], 4, every=False)              # -> 1
```

NOTE

Both problems have an $O(R \cdot C)$ refinement. For "some square", scan the bottom-right corners in row-major order and test only side `best + 1` at each corner. A valid square of side K contains valid squares of sides 1 to K along its diagonal, and the scan meets them in order. For "every square", a two-pointer walk along each diagonal removes the log factor. The binary search is fast enough for grids up to 300×300.

45.4.6 Difference arrays: many range updates, one read

A direct update touches every index of its range, so q updates cost $O(q \cdot n)$. A difference array is the inverse of a prefix sum. It stores only where the value changes. To add `v` to `a[l..r]`, do `diff[l] += v` and `diff[r + 1] -= v`. Each update costs $O(1)$. One prefix sum pass at the end gives the final array. **Memory hook:** plus at the start, minus just past the end, then run the sum. Use it for the final values after many range additions and for the peak number of overlapping intervals. Also use it to check that the load never goes above a capacity. (Corporate Flight Bookings, Range Addition)

```python
from itertools import accumulate

def apply_updates(n, updates):
    diff = [0] * (n + 1)                # one extra slot for r + 1 == n
    for l, r, v in updates:             # add v to a[l..r], both ends included
        diff[l] += v
        diff[r + 1] -= v
    return list(accumulate(diff[:n]))

apply_updates(5, [(1, 3, 2), (0, 1, 5), (3, 4, -1)])   # -> [5, 7, 2, 1, -1]
intervals = [(1, 3), (2, 5), (4, 6)]                    # closed integer intervals
max(apply_updates(7, [(s, e, 1) for s, e in intervals]))   # -> 2
```

For a half-open interval `[s, e)`, put the `-v` at `e`, not at `e + 1`. In 2D, to add `v` to the rectangle from `(r1, c1)` to `(r2, c2)`, add `v` at `(r1, c1)` and at `(r2 + 1, c2 + 1)`. Subtract `v` at `(r1, c2 + 1)` and at `(r2 + 1, c1)`. Then run the 2D prefix sum build.

45.4.7 A shared offset and its extremes instead of a simulation

A simulation moves every item at every step, so it costs the number of steps times the number of items. But when the same move applies to every item, all items keep their relative positions. One shared offset per axis describes every item. Move the frame, not the items. Take a grid of leaves where each gust of wind moves every leaf one cell. A leaf that leaves the garden never comes back. **Memory hook:** Move the frame, not the leaves: track the offset's min and max, then sum the surviving box.

A leaf that starts at row `r` survives if `0 <= r + d <= rows - 1` for the offset `d` after every prefix of gusts. This depends only on the minimum and the maximum of the offset. So the surviving start rows are exactly `-min_dr` to `rows - 1 - max_dr`. Columns follow the same rule. In general, with bounds `lower` and `upper`, the valid start values form the interval `[lower - min_off, upper - max_off]`. It is empty when the extremes are too far apart.

```python
def remaining_leaves(garden, gusts):
    rows, cols = len(garden), len(garden[0])
    step = {"U": (-1, 0), "D": (1, 0), "L": (0, -1), "R": (0, 1)}
    dr = dc = 0
    min_dr = max_dr = min_dc = max_dc = 0     # the empty prefix counts
    for g in gusts:
        sr, sc = step[g]
        dr += sr
        dc += sc
        min_dr, max_dr = min(min_dr, dr), max(max_dr, dr)
        min_dc, max_dc = min(min_dc, dc), max(max_dc, dc)
    top, bottom = -min_dr, rows - 1 - max_dr
    left, right = -min_dc, cols - 1 - max_dc
    if top > bottom or left > right:          # guard: negative slice ends count from the end
        return 0
    return sum(sum(row[left:right + 1]) for row in garden[top:bottom + 1])

g = [[1, 2, 3], [4, 5, 6]]
remaining_leaves(g, "R")                      # -> 12
remaining_leaves(g, "RL")                     # -> 12
remaining_leaves(g, "")                       # -> 21
remaining_leaves(g, "UD")                     # -> 15
remaining_leaves(g, "UU")                     # -> 0
remaining_leaves(g, "DLL")                    # -> 3
```

The net offset of `"RL"` is 0, but the `R` gust already removed the right column. So track the extremes over all prefixes, not only the final offset. Each axis is independent, so the survivors form one rectangle. For many gust strings on the same garden, build a 2D prefix table one time. Then each rectangle sum costs $O(1)$.

45.4.8 A monotone left bound while sweeping

A brute force checks all $n^2/2$ ranges one by one. Some properties survive shrinking: if a range is valid, every range inside it is valid. Then, for each right end `r`, the valid starts form one block `[left, r]`, and `left` never moves back. Count `r - left + 1` ranges for each right end. The total work is $O(n)$. **Memory hook:** valid stays valid when it shrinks, so keep the smallest left and add `r - left + 1`.

Take the task to count the ranges of `1..n` that contain no forbidden pair. A range is invalid when it contains both numbers of a forbidden pair. A pair `{a, b}` is the interval `[lo, hi]`. A range contains the pair when it covers that interval. **Memory hook:** A pair closes at its max and pushes left past its min.

Many constraints have the form "a range that ends at or after `p` must start after `q`". Bucket them by `p` and keep the largest `q` in each bucket. A running maximum during the sweep applies them. This replaces a search over the constraints with $O(1)$ work per step.

```
def count_valid_sequences(n, pairs):
    block = [0] * (n + 1)                # block[hi] = largest lo of a pair that closes at hi
    for a, b in pairs:
        lo, hi = min(a, b), max(a, b)
        block[hi] = max(block[hi], lo)
    total = 0
    left = 1                             # smallest valid start for the current right end
    for right in range(1, n + 1):
        left = max(left, block[right] + 1)    # left never moves back
        total += right - left + 1
    return total

count_valid_sequences(4, [[1, 3]])            # -> 8
count_valid_sequences(3, [])                  # -> 6
count_valid_sequences(5, [[1, 2], [4, 5]])    # -> 8
count_valid_sequences(4, [[3, 1], [1, 3]])    # -> 8
```

The running maximum is essential. A pair that closed earlier still blocks every later right end. The same sweep counts ranges of positive values with a product below k, or ranges with at most k distinct values (see Sliding window).

45.5 Adapt it to a new problem

Ask these questions in order:

1. What is the brute force? Write the quantity for one range or one index as a formula.
2. Is it a contiguous range? Write it as `f(j) - f(i)` with a running value `f`. Then the task is about pairs of prefixes.
3. Is the condition an exact value, a remainder or a balance? Keep a hash map of earlier prefix values: a count to count ranges, the first index to find the longest range.
4. Is the condition an inequality? With non-negative values, use a sliding window or a binary search on `P`. With negative values, use a sorted list of earlier prefixes or a monotonic deque.
5. Does the answer at `i` need both sides of `i`? Use a prefix pass and a suffix pass. If the operation has an inverse, the total minus one part is simpler.
6. Is it the best pair with `i < j`? Split the score into `f(i) + g(j)` and keep the running best of `f(i)`.
7. Are there many range updates before one read? Use a difference array.
8. Does every item get the same move? Track one shared offset. If items drop out at a bound, track its minimum and maximum too.
9. Does validity survive shrinking? Keep the smallest valid left end and add `r - left + 1`.
10. Is it a grid? Build a padded 2D table. If the question asks for a size, check that the condition is monotone, then binary search.
11. What is the empty prefix? Seed it: 0 for a sum, 1 for a product, `{0: 1}` for counts, `{0: -1}` for first indexes, 0 for an offset.
12. Ask before you add. Decide if the problem allows a pair of one position with itself.
13. Test the small cases by hand: one element, a zero value, `k = 0`, and a range that starts at index 0.

45.6 Pitfalls

CAUTION

Off-by-one in the prefix array. `P` has `n + 1` entries. The sum of `nums[l..r]` is `P[r + 1] - P[l]`, not `P[r] - P[l]`. A padded table (one zero row and one zero column in 2D) removes all bounds checks.

CAUTION

The empty prefix. Without `seen[0] = 1`, the sweep misses every range that starts at index 0. Without `first[0] = -1`, the longest range from index 0 is lost. Offsets and their extremes start at 0, not at the first move.

CAUTION

Add before ask. An insert before the lookup pairs a prefix with itself. With `k = 0`, this counts empty ranges.

```
def wrong(nums, k):
    seen, prefix, count = {0: 1}, 0, 0
    for x in nums:
        prefix += x
        seen[prefix] = seen.get(prefix, 0) + 1    # add first: wrong
        count += seen.get(prefix - k, 0)
    return count

wrong([0, 0], 0)                                  # -> 5
wrong([1, 1, 1], 2)                               # -> 2
```

The correct answer for `[0, 0]` and `k = 0` is 3. The bug is invisible when `k` is not 0.

CAUTION

Sliding window on negative values. A window needs a sum that never drops when the window grows. Negative values break that. For an exact count, zeros also break a simple window, because several starts give the same sum. Use prefix sums with a hash map.

CAUTION

The wrong identity. A running product starts at 1, a running minimum at `inf` or the first element, a running maximum at `-inf`. A minimum price that starts at 0 never changes.

CAUTION

Global extremes ignore the order. The global minimum can come after the global maximum. For prices `[2, 4, 1]` the answer is 2, not 3. Keep the minimum of the prefix only.

CAUTION

The final offset is not enough. With permanent loss, an item can leave and the offset can come back. Track the minimum and the maximum over all prefixes.

CAUTION

2D tables. `[[0] * n] * m` makes all rows one list. Use a comprehension for any table that you write to. Check the signs: subtract the top strip and the left strip, then add back the corner.

CAUTION

Binary search on a size. The search for a maximum with `lo = mid` needs the upper mid `(lo + hi + 1) // 2`. The upper bound is `min(rows, cols)`, not `max(rows, cols)`. The answer can be 0, so side 0 must be a possible result. The search is valid only for non-negative values.

CAUTION

Empty slices. When a computed range is empty, its end can be negative. A negative slice end counts from the end of the list, so the slice returns real elements. Guard the empty case first.

CAUTION

Difference array size. An update that ends at index `n - 1` writes to `diff[n]`. Allocate `n + 1` slots.

NOTE

A `defaultdict` lookup inserts the missing key. The count stays correct, but the map grows. Use `dict.get` or `Counter` to read a missing key without an insert.

45.7 Problems in this family

Problem	What changes	Memory hook
Product of Array Except Self	Exclude one index. No division. A left pass writes prefix products. A right pass multiplies in a scalar suffix.	Answer = left product times right product. In each pass, use the running product, then multiply in nums[i].
Subarray Sum Equals K	Exact range sum with negative values. Count earlier prefixes `prefix - k` in a map seeded with `{0: 1}`.	Two Sum on prefix sums: look up prefix minus k, then store prefix.
Maximum Side Length of a Square with Sum Less than or Equal to Threshold	2D prefix table. *Some* square of side `k` fits. Monotone in `k`, so binary search the side.	Prefix table for O(1) squares, then binary search the biggest side that fits.
Best Time to Buy and Sell Stock	Best pair with `i < j`. The best partner is the running minimum. The answer starts at 0.	Remember the cheapest day, sell today.
Trapping Rain Water	Per index: `min` of prefix maximum and suffix maximum, minus the bar. Two pointers give $O(1)$ space.	Water at i is the lower wall minus the bar: min(left max, right max) - height.

The transfer drills for this technique are in Section 69.3 (page 418). The answers are in Section A.2 (page 430).

46 Two pointers

46.1 The idea

A pointer is an index into an array or a reference to a list node. Two pointers walk the data at the same time, and each pointer moves in one direction only. Each pointer makes $O(n)$ moves, so the total work is $O(n)$. A brute force over pairs checks about $n^2/2$ pairs. The two pointers do less work because each step proves a fact about one element. The element cannot be part of any answer, or of a better answer than the best so far. That element is then discarded for good.

Worked example. Find two values with the sum 9 in the sorted list $[1, 3, 4, 6, 9]$. Put `lo` on 1 and `hi` on 9. The sum 10 is too large. The value 9 is too large even with the smallest partner, 1. So no pair uses 9: move `hi` to 6. The sum $1 + 6 = 7$ is too small. The value 1 is too small even with the largest partner that is left, 6. So no pair uses 1: move `lo` to 3. The sum $3 + 6 = 9$ is the answer. Three steps replace the ten pairs of the brute force.

The pointers can move in four shapes. Each shape keeps a different fact true.

Shape	Moves	Use it for	Fact that stays true
Converging	`lo` from the left, `hi` from the right, toward each other	pair sums on sorted data, symmetric checks, the best pair with a bottleneck	Every answer lies inside `[lo, hi]`, or the best answer outside it is already recorded.
Fast and slow	same direction, two speeds or a fixed gap	cycles, the middle, the kth node from the end on forward-only data	After t steps, slow is at position t and fast is at $2t$. Or fast is k nodes ahead of slow.
Read and write	same direction, write never ahead of read	filter, remove, deduplicate or partition in place	`a[:w]` is the finished output.
One per sequence	each pointer walks its own sorted input	merge, intersection, subsequence check	The output so far is final and sorted.

Only the converging shape removes pairs. The other three shapes remove memory or a second pass. Fast and slow pointers replace a visited set, a node array or a length count. Read and write pointers replace a second output array. One pointer per sorted input replaces a new sort of all the values.

46.2 Recognize it

- The input is sorted. Or you can sort it, because neither the output nor the score uses the original positions.
- The task asks for a pair or a triple with a sum condition, and n is too large to try every pair ($n \geq 10^4$) or every triple (n in the thousands).
- The score of a pair depends only on the two endpoints. One endpoint caps the score (a minimum of the two), and the distance between them grows toward the ends.
- The task compares element k with element $n-1-k$: a palindrome, a mirror, a front-back order.
- The task asks for $O(1)$ extra space, and the easy answer uses a hash set or a cleaned copy.
- The data is a singly linked list, and the question is about a cycle, the middle or the kth node from the end.
- Each value is a valid index into the same array, or one deterministic step repeats on a finite set of states. The data is then a functional graph, and a walk must enter a cycle.
- The task says "in place" and asks to remove, filter or deduplicate elements of an array.
- The task groups the elements in place and in one pass. The keys are few: 0, 1 and 2, or below, equal to and above a pivot.
- Two or more sorted inputs give one sorted output, or one check walks two sequences together.

46.3 Core template

The converging skeleton fits all pair searches on sorted data and all "best pair" searches. Only the use of the pair and the discard rule change from problem to problem.

```
lo, hi = 0, len(a) - 1
while lo < hi:
    use the pair (lo, hi): return it, count it, or update the best
    if the element at lo cannot be in a better answer:
        lo += 1
    else:
        hi -= 1
```

The sorted pair sum is the basic instance.

```
def pair_with_sum(a, target):        # a is sorted; return (i, j) with i < j, or None
    lo, hi = 0, len(a) - 1
    while lo < hi:
        s = a[lo] + a[hi]
        if s == target:
            return lo, hi
        if s < target:
            lo += 1                  # a[lo] is too small even with the largest partner
        else:
            hi -= 1                  # a[hi] is too large even with the smallest partner
    return None
```

```
pair_with_sum([1, 3, 4, 6, 9], 9)    # -> (1, 3)
pair_with_sum([1, 3, 4, 6, 9], 2)    # -> None
pair_with_sum([3, 3], 6)             # -> (0, 1)
pair_with_sum([1, 2, 5], 10)         # -> None
```

Invariant. Every answer pair (i, j) stays inside the range `[lo, hi]`: `lo <= i` and `j <= hi`. Each step discards one end and keeps the invariant. So `hi - lo` decreases by one, and the loop ends in at most $n - 1$ steps.

46.4 Variations

46.4.1 Pair search on sorted data: find, count, closest

Two Sum II - Input Array Is Sorted is the core template with 1-based output indices. The template returns 0-based indices. For this problem, return `[lo + 1, hi + 1]`. Memory hook: *Sorted pair sum: start at both ends. Too small, move left up. Too big, move right down.* The same loop answers three more questions. Only the action on the pair changes.

- **Count the pairs below a target.** If `a[lo] + a[hi] < target`, then `a[lo]` pairs with every index in `lo+1..hi`. Add `hi - lo` at once, then move `lo`.
- **The closest sum.** Keep the sum with the smallest `abs(s - target)`. Move the pointers as in the template.
- **Unsorted input.** Sort first in $O(n \log n)$ when the output needs values only. When it needs the original indices, sort `(value, index)` pairs, or use a hash map.

```
from itertools import combinations

def count_pairs_below(a, target):    # pairs i < j with a[i] + a[j] < target
    a = sorted(a)
    lo, hi, count = 0, len(a) - 1, 0
    while lo < hi:
        if a[lo] + a[hi] < target:
            count += hi - lo         # a[lo] works with every index in lo+1..hi
            lo += 1
        else:
            hi -= 1
    return count

count_pairs_below([-1, 1, 2, 3, 1], 2)                          # -> 3
a = [5, -2, 7, 0, 3, 3, -4]
count_pairs_below(a, 4) == sum(x + y < 4 for x, y in combinations(a, 2))
# -> True
```

(Count Pairs Whose Sum is Less than Target, 3Sum Closest, Two Sum IV on a BST.)

46.4.2 Same-direction pointers for a pair difference

A difference needs another shape. From the two ends, a difference that is too large does not tell you which end to move: both moves make it smaller. Start both pointers at the left instead. If the gap `a[j] - a[i]` is too small, move `j`. If it is too large, move `i`. Each move changes the gap in one known direction, so each comparison again discards one index for good. Memory hook: *Difference: both start left. Gap too small, move the leading pointer. Gap too big, move the trailing pointer.*

```
def pair_with_difference(a, k):      # a is sorted, k >= 0; i != j and a[j] - a[i] == k
    i, j = 0, 1
    while j < len(a):
        d = a[j] - a[i]
        if i == j or d < k:
            j += 1                   # the gap is too small: widen it
        elif d > k:
            i += 1                   # the gap is too large: shrink it
        else:
            return i, j
```

```
    return None

pair_with_difference([1, 3, 5, 8], 3)    # -> (2, 3)
pair_with_difference([1, 4, 6], 5)       # -> (0, 2)
pair_with_difference([1, 1, 4], 0)       # -> (0, 1)
pair_with_difference([1, 2], 0)          # -> None
```

(K-diff Pairs in an Array; the pair count inside Find K-th Smallest Pair Distance.)

46.4.3 k-sum: fix one element, then squeeze

3Sum asks for all value triples with sum 0 and no repeated triples. Sort, fix an anchor `a[i]`, and run the sorted pair search on the suffix with target `-a[i]`. The cost is n anchors times $O(n)$, so $O(n^2)$. Memory hook: *Sort, fix one, squeeze two pointers, skip repeats.*

Sorting also removes duplicate triples without a set. At each choice level, use a value only the first time it appears. Compare with the previous element (`a[i] == a[i - 1]`), not the next one. Then the first copy still pairs with its own repeats, as in $[-1, -1, 2]$. After a match, move `lo` past all copies of the value just used. When the anchor is positive, every later value is positive too, so stop.

```
def three_sum(nums):
    a = sorted(nums)                 # sorted() keeps the caller's list unchanged
    n, res = len(a), []
    for i in range(n - 2):
        if a[i] > 0:
            break                    # every later value is positive too
        if i > 0 and a[i] == a[i - 1]:
            continue                 # this anchor value was already used
        lo, hi = i + 1, n - 1
        while lo < hi:
            s = a[i] + a[lo] + a[hi]
            if s < 0:
                lo += 1
            elif s > 0:
                hi -= 1
            else:
                res.append([a[i], a[lo], a[hi]])
                lo += 1
                hi -= 1
                while lo < hi and a[lo] == a[lo - 1]:
                    lo += 1          # skip copies of the left value just used
    return res

three_sum([-1, 0, 1, 2, -1, -4])     # -> [[-1, -1, 2], [-1, 0, 1]]
three_sum([0, 0, 0, 0])              # -> [[0, 0, 0]]
three_sum([0, 1, 1])                 # -> []
```

The same idea works for any k: fix one element and solve $(k-1)$-sum on the rest. The base case is the pair search. The cost is $O(n^{k-1})$.

```
def k_sum(a, k, target, start=0):    # a is sorted; value lists of size k, no repeats
    res = []
    if k == 2:
        lo, hi = start, len(a) - 1
        while lo < hi:
            s = a[lo] + a[hi]
            if s < target:
                lo += 1
            elif s > target:
                hi -= 1
            else:
                res.append([a[lo], a[hi]])
                lo += 1
```

```
                hi -= 1
                while lo < hi and a[lo] == a[lo - 1]:
                    lo += 1
        return res
    for i in range(start, len(a) - k + 1):
        if i > start and a[i] == a[i - 1]:
            continue                # same value at the same level: skip it
        for rest in k_sum(a, k - 1, target - a[i], i + 1):
            res.append([a[i]] + rest)
    return res

k_sum(sorted([1, 0, -1, 0, -2, 2]), 4, 0)
# -> [[-2, -1, 1, 2], [-2, 0, 0, 2], [-1, 0, 0, 1]]
k_sum(sorted([-1, 0, 1, 2, -1, -4]), 3, 0)    # -> [[-1, -1, 2], [-1, 0, 1]]
```

(4Sum, 3Sum Closest.) To count index triples instead of value triples, add `hi - lo` at once, as in the pair count (3Sum Smaller). For an exact sum with many repeated values, count each value and use combinatorics (3Sum With Multiplicity).

46.4.4 Symmetric checks from both ends

A palindrome check compares the kth item from the left with the kth item from the right. **Valid Palindrome** keeps only letters and digits and ignores case. The intuitive solution builds the cleaned list and compares it with its reverse. Memory hook: *Clean it, flip it, compare.* The converging form is the aside. It needs $O(1)$ extra space and stops at the first mismatch. Use it when the task asks for $O(1)$ extra space, or when the check must react to the first mismatch. Each loop step does one thing: it skips one invalid character, or it consumes one matched pair. So the loop needs no inner skip loops and no extra bound checks.

```
def is_pal_clean(s):
    cleaned = [ch.lower() for ch in s if ch.isalnum()]
    return cleaned == cleaned[::-1]

def is_pal_pointers(s):
    i, j = 0, len(s) - 1
    while i < j:
        if not s[i].isalnum():
            i += 1                  # skip an invalid character on the left
        elif not s[j].isalnum():
            j -= 1                  # skip an invalid character on the right
        elif s[i].lower() != s[j].lower():
            return False
        else:
            i += 1                  # one matched pair is done
            j -= 1
    return True

s = "A man, a plan, a canal: Panama"
is_pal_clean(s), is_pal_pointers(s)           # -> (True, True)
is_pal_clean("OP"), is_pal_pointers("OP")     # -> (False, False)
is_pal_pointers(".,"), is_pal_pointers("aA")  # -> (True, True)
```

Swap instead of compare, and the same loop reverses an array in place. Add the same skips, and it reverses only selected characters. (Reverse String, Reverse Only Letters, Palindrome Number.)

46.4.5 Discard the weaker endpoint

Some problems ask for the best pair, and the score is a bottleneck times a distance. In **Container With Most Water**, the area of lines $i < j$ is `min(h[i], h[j]) * (j - i)`. Start with the widest pair. Suppose `h[lo] < h[hi]`. Every inner partner of `lo` is narrower, and the height stays at most `h[lo]`. So `lo` has already met its best partner. Record the area and discard `lo`. On a tie, discard either line. Memory hook: *Start widest, keep the best area, and move the shorter wall in.*

```
def max_area(h):
    lo, hi, best = 0, len(h) - 1, 0
    while lo < hi:
        best = max(best, min(h[lo], h[hi]) * (hi - lo))
        if h[lo] < h[hi]:
            lo += 1                 # the shorter wall is done
        else:
            hi -= 1
    return best

max_area([1, 8, 6, 2, 5, 4, 8, 3, 7])    # -> 49
max_area([1, 1])                         # -> 1
max_area([2, 9, 9, 2])                   # -> 9
```

The general rule: move the pointer whose element cannot appear in a better answer. Write a one-sentence proof that every skipped pair is no better than a pair already checked. If the score uses a maximum instead of a minimum, this proof fails. Think again about which end is dominated.

46.4.6 Settle the side with the smaller bound

In **Trapping Rain Water**, the water above bar i is the lower of two walls minus the bar: `min(left_max[i], right_max[i]) - h[i]`. The intuitive solution computes both walls with a prefix maximum pass and a suffix maximum pass. Memory hook: *Water at i is the lower wall minus the bar: min(left max, right max) - height.*

The converging form removes both arrays. Keep `left_max` for `h[0..lo]` and `right_max` for `h[hi..n-1]`. If `left_max < right_max`, the true right wall of index `lo + 1` is at least `right_max`. So `left_max` alone decides the level there. Settle that index now, then do the mirror step on the other side. The general rule: if the answer is `min(A, B)` and you know $A \leq$ a lower bound of B, the answer is A.

```
def trap_arrays(h):                  # intuitive: two passes, O(n) space
    n = len(h)
    left_max, right_max = h[:], h[:]
    for i in range(1, n):
        left_max[i] = max(left_max[i - 1], h[i])
    for i in range(n - 2, -1, -1):
        right_max[i] = max(right_max[i + 1], h[i])
    return sum(min(left_max[i], right_max[i]) - h[i] for i in range(n))

def trap_pointers(h):                # converging: O(1) space
    lo, hi = 0, len(h) - 1
    left_max, right_max = h[lo], h[hi]
    water = 0
    while lo < hi:
        if left_max < right_max:
            lo += 1
            left_max = max(left_max, h[lo])    # update first, then add
            water += left_max - h[lo]
        else:
            hi -= 1
            right_max = max(right_max, h[hi])
            water += right_max - h[hi]
    return water

h = [0, 1, 0, 2, 1, 0, 1, 3, 2, 1, 2, 1]
trap_arrays(h), trap_pointers(h)         # -> (6, 6)
trap_pointers([4, 2, 0, 3, 2, 5])        # -> 9
trap_pointers([5])                       # -> 0
trap_arrays([3, 0, 1]), trap_pointers([3, 0, 1])    # -> (1, 1)
```

Use the arrays first. Switch to the pointers only when the task asks for $O(1)$ extra space.

46.4.7 Read and write pointers: compact in place

The read pointer `r` visits every element. The write pointer `w` marks the end of the kept part. Copy `a[r]` to `a[w]` only when the element stays. **Invariant:** `a[:w]` holds the kept elements in their original order, and $w \leq r$. So a write never destroys an element that is not read yet. To keep a condition on the output itself, compare with the written part, for example `a[w - 1]`. Memory hook: *Read every element, write only the keepers. The prefix before w is done.*

```
def remove_value(a, val):            # keep every element that is not val
    w = 0
    for r in range(len(a)):
        if a[r] != val:
            a[w] = a[r]
            w += 1
    return w

def dedupe_sorted(a):                # keep the first copy of each value
    w = 0
    for r in range(len(a)):
        if w == 0 or a[r] != a[w - 1]:
            a[w] = a[r]
            w += 1
    return w

a = [3, 2, 2, 3, 4]
k = remove_value(a, 3)
a[:k]                                # -> [2, 2, 4]
b = [0, 0, 1, 1, 1, 2, 2, 3]
k = dedupe_sorted(b)
b[:k]                                # -> [0, 1, 2, 3]
```

To keep the removed elements at the end, swap `a[w]` and `a[r]` instead of a copy. (Remove Element, Remove Duplicates from Sorted Array, Move Zeroes, String Compression.)

46.4.8 Three-way partition: three pointers

Some tasks put each element into one of three groups, in place: below, equal to and above a pivot. **Sort Colors** (values 0, 1 and 2) is the usual example. The easy answer counts each value and writes the counts back. That takes two passes. For one pass, use three pointers. `lo` and `hi` are write pointers at the two ends. `mid` is the read pointer. **Invariant:** `a[:lo]` holds the small keys, `a[lo:mid]` the middle keys and `a[hi + 1:]` the large keys. `a[mid:hi + 1]` is not read yet, so the loop runs while `mid <= hi`. A small key at `mid` goes to `lo`, and both pointers move: the value that comes back was read already. A large key goes to `hi`, and only `hi` moves. After a swap with `hi`, do not move `mid`: the new `a[mid]` is not read yet. Memory hook: *Small to the front, large to the back, middle stays. After a swap with the back, look again.*

```
def partition3(a, pivot):                # in place, one pass
    lo, mid, hi = 0, 0, len(a) - 1
    while mid <= hi:
        if a[mid] < pivot:
            a[lo], a[mid] = a[mid], a[lo]
            lo += 1
            mid += 1                     # the old a[lo] was read already
        elif a[mid] > pivot:
            a[mid], a[hi] = a[hi], a[mid]
            hi -= 1                      # mid stays: the new a[mid] is not read yet
        else:
            mid += 1
    return lo, hi + 1                    # a[lo:hi + 1] holds the keys equal to pivot

a = [2, 0, 2, 1, 1, 0]
partition3(a, 1)                         # -> (2, 4)
a                                        # -> [0, 0, 1, 1, 2, 2]
```

```
b = [5, 1, 5, 9, 5, 3, 7]
i, j = partition3(b, 5)
sorted(b[:i]), b[i:j], sorted(b[j:])     # -> ([1, 3], [5, 5, 5], [7, 9])
partition3([], 1)                        # -> (0, 0)
```

For Sort Colors, use the pivot 1. The swaps do not keep the original order inside a group. (Sort Colors, the partition step of quickselect when many values are equal.)

46.4.9 Fast and slow pointers: cycle and middle

Linked List Cycle asks if a list loops, with $O(1)$ extra memory. Move `slow` one node and `fast` two nodes per step. Without a cycle, `fast` reaches the end. With a cycle, both pointers enter it. Inside the cycle, `fast` gains exactly one node per step, so the gap counts down to 0 and cannot skip it. Memory hook: *Hare runs two, tortoise runs one. If they meet, it loops. If the hare falls off the end, it does not.* The same loop finds the middle: when `fast` reaches the end, `slow` is halfway.

```
class ListNode:
    def __init__(self, val=0, next=None):
        self.val, self.next = val, next

def build(vals):
    head = None
    for v in reversed(vals):
        head = ListNode(v, head)
    return head

def has_cycle(head):
    slow = fast = head
    while fast and fast.next:            # fast can take two steps
        slow = slow.next
        fast = fast.next.next
        if slow is fast:                 # the same node object
            return True
    return False                         # fast fell off the end

def middle(head):
    slow = fast = head
    while fast and fast.next:
        slow = slow.next
        fast = fast.next.next
    return slow                          # the right middle for an even length

head = build([1, 2, 3, 4])
middle(head).val                         # -> 3
middle(build([1, 2, 3, 4, 5])).val       # -> 3
has_cycle(head), has_cycle(None)         # -> (False, False)
head.next.next.next.next = head.next     # the tail now links back to node 2
has_cycle(head)                          # -> True
```

Start `fast` at `head.next` to get the left middle, the last node of the front half. Use the left middle when you cut a list into two halves. (Middle of the Linked List, Happy Number, Palindrome Linked List.)

46.4.10 Cycle entry: an array as a linked list

Find the Duplicate Number has $n+1$ values in $[1, n]$, a read-only array and $O(1)$ extra space. Read the array as a linked list: index `i` points to index `nums[i]`. No value is 0, so nothing points to index 0. A walk from index 0 must enter a cycle. The cycle entry has two incoming edges, so two indexes hold its value: the entry is the duplicate.

Floyd's algorithm finds the entry in two phases. Phase 1 runs fast and slow pointers until they meet. Phase 2 puts one pointer back at the head and moves both pointers one step at a time. They meet at the entry. The reason: at the meeting, `slow` walked m steps and `fast` walked $2m$ steps. Both stand on the same node, so the extra m steps

are a whole number of laps. Let μ be the tail length, the number of steps from the head to the entry. After μ more steps, the pointer from the meeting point is $\mu + m$ steps from the head: the entry plus whole laps. The pointer from the head is on the entry after the same μ steps. Memory hook: *Index points to value: tortoise and hare from 0, then reset one to 0, walk together, and meet at the duplicate.*

```
def find_duplicate(nums):
    slow, fast = nums[0], nums[nums[0]]  # both pointers after their first move
    while slow != fast:                  # phase 1: meet inside the cycle
        slow = nums[slow]
        fast = nums[nums[fast]]
    start = 0
    while start != slow:                 # phase 2: meet at the cycle entry
        start = nums[start]
        slow = nums[slow]
    return slow

find_duplicate([1, 3, 4, 2, 2])          # -> 2
find_duplicate([3, 1, 3, 4, 2])          # -> 3
find_duplicate([3, 3, 3, 3, 3])          # -> 3
```

The same two phases find the entry of a cycle in a real linked list (Linked List Cycle II). A second $O(1)$-space approach: binary search on the value. If more than `mid` values are at most `mid`, the duplicate is in `[1, mid]`.

46.4.11 Fixed gap: the kth node from the end

Remove Nth Node From End of List gives a position from the end, but a singly linked list only walks forward. The intuitive solution lines up the nodes in a Python list behind a dummy node. Then `nodes[-n - 1]` is the predecessor of the target. Memory hook: *Line up the nodes behind a dummy, then `nodes[-n - 1]` skips its next.*

The pointer form needs $O(1)$ extra memory. Give `fast` a lead of n nodes and then move both pointers. The gap stays n. Stop when `fast` is on the last node: `slow` is then the predecessor of the target. Start both pointers at a dummy node, so that the removal of the head needs no special code.

```
class ListNode:
    def __init__(self, val=0, next=None):
        self.val, self.next = val, next

def build(vals):
    head = None
    for v in reversed(vals):
        head = ListNode(v, head)
    return head

def to_list(head):
    out = []
    while head:
        out.append(head.val)
        head = head.next
    return out

def remove_nth_array(head, n):           # intuitive: O(n) extra memory
    dummy = ListNode(0, head)
    nodes, node = [], dummy
    while node:
        nodes.append(node)
        node = node.next
    prev = nodes[-n - 1]
    prev.next = prev.next.next
    return dummy.next

def remove_nth_gap(head, n):             # gap pointers: O(1) extra memory
    dummy = ListNode(0, head)
```

```
    slow = fast = dummy
    for _ in range(n):
        fast = fast.next                # fast leads by n nodes
    while fast.next:                    # stop with fast on the last node
        slow = slow.next
        fast = fast.next
    slow.next = slow.next.next          # slow is the predecessor of the target
    return dummy.next

to_list(remove_nth_gap(build([1, 2, 3, 4, 5]), 2))     # -> [1, 2, 3, 5]
to_list(remove_nth_gap(build([1]), 1))                 # -> []
to_list(remove_nth_gap(build([1, 2]), 2))              # -> [2]
to_list(remove_nth_array(build([1, 2, 3, 4, 5]), 5))   # -> [2, 3, 4, 5]
```

Run until `fast` is past the last node, and `slow` lands on the target itself. (Kth node from the end, Rotate List, Swapping Nodes in a Linked List.)

46.4.12 Mirror pairs on a forward-only list

Reorder List changes $L_0, L_1, \ldots, L_{n-1}$ into $L_0, L_{n-1}, L_1, L_{n-2}, \ldots$. Node i pairs with node $n-1-i$, but a singly linked list has no backward step. The intuitive solution copies the node references into an array. Two converging indexes then give the order front, back, front, back. Chain the nodes and end the list with `None`. Memory hook: *Put the nodes in an array, take front then back, chain them, end with None.*

```
class ListNode:
    def __init__(self, val=0, next=None):
        self.val, self.next = val, next

def build(vals):
    head = None
    for v in reversed(vals):
        head = ListNode(v, head)
    return head

def to_list(head):
    out = []
    while head:
        out.append(head.val)
        head = head.next
    return out

def reorder(head):
    nodes = []
    while head:
        nodes.append(head)
        head = head.next
    order = []
    lo, hi = 0, len(nodes) - 1
    while lo <= hi:                     # lo <= hi: the middle node needs a step
        order.append(nodes[lo])
        if lo < hi:
            order.append(nodes[hi])
        lo += 1
        hi -= 1
    for a, b in zip(order, order[1:]):
        a.next = b
    order[-1].next = None               # end the list, or it has a cycle

head = build([1, 2, 3, 4, 5])
reorder(head)
to_list(head)                           # -> [1, 5, 2, 4, 3]
head = build([1, 2, 3, 4])
```

```
reorder(head)
to_list(head)                           # -> [1, 4, 2, 3]
```

The $O(1)$-space form composes three linked-list routines. Find the left middle with fast and slow pointers and cut the list there. Reverse the back half in place, so that a forward walk reads it from the last node. Then weave the two halves, front first. The same three steps check a palindrome list and find the maximum twin sum. Only the last step changes.

46.4.13 Merge two sorted sequences

Each input is sorted, so the smallest value that is left is always one of the current fronts. Keep one pointer per input. Compare the fronts, take the smaller one, and move that pointer. When one input runs out, append the rest of the other input.

```
def merge(a, b):
    i = j = 0
    out = []
    while i < len(a) and j < len(b):
        if a[i] <= b[j]:                # <= takes from a first on a tie: stable
            out.append(a[i])
            i += 1
        else:
            out.append(b[j])
            j += 1
    out.extend(a[i:])                   # at most one of these is not empty
    out.extend(b[j:])
    return out

merge([1, 4, 6], [2, 3, 7, 8])          # -> [1, 2, 3, 4, 6, 7, 8]
merge([], [5])                          # -> [5]
```

Merge Two Sorted Lists does the same on linked lists. A dummy node removes the special case for the first node. A `tail` pointer marks the end of the output. Memory hook: *Dummy and tail. Take the smaller head, move the tail. Glue the rest.*

```
class ListNode:
    def __init__(self, val=0, next=None):
        self.val, self.next = val, next

def build(vals):
    head = None
    for v in reversed(vals):
        head = ListNode(v, head)
    return head

def to_list(head):
    out = []
    while head:
        out.append(head.val)
        head = head.next
    return out

def merge_lists(l1, l2):
    dummy = tail = ListNode()
    while l1 and l2:
        if l1.val <= l2.val:
            tail.next, l1 = l1, l1.next
        else:
            tail.next, l2 = l2, l2.next
        tail = tail.next                # move the tail after each link
    tail.next = l1 or l2                # glue the rest
    return dummy.next
```

```
to_list(merge_lists(build([1, 2, 4]), build([1, 3, 4])))   # -> [1, 1, 2, 3, 4, 4]
to_list(merge_lists(None, build([0])))                     # -> [0]
to_list(merge_lists(None, None))                           # -> []
```

For an intersection, move the pointer at the smaller front. When the two fronts are equal, emit the value and move both pointers. For k inputs, keep the k fronts in a min-heap. A subsequence check is also a merge. The pointer of the long sequence moves on every step. The pointer of the short sequence moves only on a match. Sometimes the two sorted runs sit at the two ends of one array. Then merge from the ends inward and take the larger value first. For example, the squares of a sorted array with negative values are largest at the two ends. (Squares of a Sorted Array, Intersection of Two Arrays II, Is Subsequence, Sort List.)

46.5 Adapt it to a new problem

Ask these questions in order.

1. What does the brute force check? If it checks all pairs, can one comparison rule out a full row of pairs?
2. Is the order useful? Is the input sorted, or can you sort it? Sorting is safe when neither the output nor the score uses the original positions. It is not safe when the output needs original indices, or when the score uses a distance between indices, as in Container With Most Water.
3. Which shape fits? A sum, a bottleneck or a mirror: converging. A position from the end, the middle or a cycle on forward-only data: fast and slow. An in-place filter: read and write. Three groups in place: three pointers. Several sorted inputs: one pointer per input.
4. Write the discard rule as one sentence: "the element at `lo` cannot be in a better answer, because ...". If you cannot prove it for either end, the two ends are the wrong start. Try the same direction, or another technique.
5. Decide the action on a match: return it, count `hi - lo` pairs at once, or record it and skip the repeats.
6. Choose the loop condition. Use `lo < hi` when the two pointers must be different elements. Use `lo <= hi` when the middle element needs its own step.
7. More than two elements in a group? Fix $k - 2$ elements with loops and run the pair search inside.
8. A linked list? Add a dummy node when the head can change. Aim the trailing pointer at the predecessor of the node to change.
9. Test the edges: an empty input or one element, all values equal, no answer, and an answer at the two ends.

46.6 Pitfalls

CAUTION

Move exactly one pointer per comparison, and move the right one. Moving both pointers on a step without a match can skip the answer. Moving the taller wall can skip the optimum. Settling the side with the larger running maximum adds too much water: on $[3, 0, 1]$ it gives 5, not 1. Move the pointer that your proof discards.

CAUTION

`lo <= hi` pairs an element with itself. The pair search then uses one element two times when no real pair exists.

```
def pair_bad(a, target):
    lo, hi = 0, len(a) - 1
    while lo <= hi:                      # wrong: lo == hi is one element
        s = a[lo] + a[hi]
        if s == target:
            return lo, hi
        if s < target:
            lo += 1
        else:
            hi -= 1
    return None

pair_bad([1, 2, 5], 10)                  # -> (2, 2)
```

CAUTION

Bound every inner skip loop. Write `while lo < hi and a[lo] == a[lo - 1]`. Without `lo < hi`, the input $[0, 0, 0, 0]$ walks off the end. In a palindrome check with inner skip loops, each loop needs `i < j`, or `"!!"` raises `IndexError`.

CAUTION

Skip repeats by a comparison with the previous element. `a[i] == a[i + 1]` skips the first copy of a value, and $[-1, -1, 2]$ is lost. `a[i] == a[i - 1]` uses the first copy and skips only the later ones.

CAUTION

Fast pointers: check before you jump, compare after you move. Test `fast and fast.next` before `fast.next.next`, or a `None` raises `AttributeError`. Both pointers start on the head, so compare them only after the first move. Compare nodes with `is`. Two nodes with equal values are not the same node.

CAUTION

The first meeting point of Floyd's algorithm is, in general, not the cycle entry. Run phase 2. Put one pointer back on the same head where phase 1 started, not on another node. In the duplicate problem, the head is index 0, which is never inside the cycle.

```
nums = [1, 3, 4, 2, 2]
slow, fast = nums[0], nums[nums[0]]
while slow != fast:
    slow = nums[slow]
    fast = nums[nums[fast]]
slow                                    # -> 4
start = 0
while start != slow:
    start = nums[start]
    slow = nums[slow]
slow                                    # -> 2
```

CAUTION

Linked lists: end the list and return the real head. After you rewire nodes, set the next link of the new last node to `None`. Otherwise the list has a cycle. Return `dummy.next`, not the old head: the old head can move or be removed.

CAUTION

Sorting has side effects. `nums.sort()` changes the caller's list, and it loses the original indices. Use `sorted(nums)`, or sort `(value, index)` pairs when the output needs indices.

NOTE

Converging pointers need a rule that names the end to discard. For a sum search, a move of `lo` must push the sum up and a move of `hi` must push it down. For a best pair, one end must be dominated, like the shorter wall. A difference, a product of signed values or a score with `max` in place of `min` often breaks this rule. Then use same-direction pointers, a hash map, or another technique.

46.7 Problems in this family

Problem	What changes	Memory hook
Valid Palindrome	Keep the letters and digits in lowercase and compare the list with its reverse. Aside: converging pointers for $O(1)$ space.	Clean it, flip it, compare.
Two Sum II - Input Array Is Sorted	The core pair search on sorted data. Discard the end that cannot pair. 1-based indices.	Sorted pair sum: start at both ends. Too small, move left up. Too big, move right down.

Problem	What changes	Memory hook
3Sum	Sort, fix an anchor, run the pair search on the suffix. Skip a repeated value at each level.	Sort, fix one, squeeze two pointers, skip repeats.
Container With Most Water	Best pair with a min-times-width score. The shorter wall has met its best partner.	Start widest, keep the best area, and move the shorter wall in.
Trapping Rain Water	Water at each index is min(prefix max, suffix max) minus the bar. Aside: settle the side with the smaller running maximum for $O(1)$ space.	Water at i is the lower wall minus the bar: min(left max, right max) - height.
Linked List Cycle	Fast pointer two steps, slow pointer one step. A meeting proves a cycle.	Hare runs two, tortoise runs one. If they meet, it loops. If the hare falls off the end, it does not.
Find the Duplicate Number	The array is a functional graph `i -> nums[i]`. Floyd phase 2 finds the cycle entry.	Index points to value: tortoise and hare from 0, then reset one to 0, walk together, and meet at the duplicate.
Remove Nth Node From End of List	Line up the nodes behind a dummy; `nodes[-n - 1]` is the predecessor. Aside: a gap of n nodes for $O(1)$ memory.	Line up the nodes behind a dummy, then `nodes[-n - 1]` skips its next.
Reorder List	Mirror pairs i and $n-1-i$ on a forward-only list. Converging indexes on an array of nodes.	Put the nodes in an array, take front then back, chain them, end with None.
Merge Two Sorted Lists	One pointer per sorted input. Take the smaller front; a dummy and a tail build the output.	Dummy and tail. Take the smaller head, move the tail. Glue the rest.

The transfer drills for this technique are in Section 69.4 (page 419). The answers are in Section A.3 (page 431).

47 Sliding window

47.1 The idea

A window is a contiguous part `a[left..right]` of an array or a string. Both ends of the window move only to the right. In each step, the right end moves one position and one element enters the window. When the window breaks the rule of the problem, the left end moves and elements leave the window. Each element enters the window one time and leaves it at most one time. So the scan does $O(n)$ updates in total.

A brute force checks all $n(n+1)/2$ contiguous segments, and it often scans each segment again. That is $O(n^2)$ or $O(n^3)$ work. The window saves this work for two reasons:

- **The left end never moves back.** A brute force restarts the left end at 0 for each right end. The window keeps the left end where it is. Take a "longest" or a "count" goal, and suppose `a[l..r]` is invalid. Then every longer window `a[l..r2]` contains it and is invalid too. So a start that fails one time fails for all later right ends. The scan never tries it again. For a "shortest" goal, suppose `a[l..r]` is valid. Then every longer window from `l` is worse. So the scan records `a[l..r]` and drops `l`.
- **The window keeps a state.** The state is a set, a count map, a sum or a deque. It changes in $O(1)$ when one element enters or leaves. The validity test reads the state. It does not scan the window again.

The first reason needs a *monotone* rule. For a "longest" goal, a valid window must stay valid when it shrinks. For a "shortest" goal, a valid window must stay valid when it grows. A "count" goal can use either direction. The table in the core template shows the two loop shapes. If the rule is not monotone, the correct left end can be behind the current left end. Then the method gives wrong answers.

Worked example. Find the longest substring of `"pwwkew"` with no repeated character. The state is the set of characters in the window.

- `right = 0` adds `p`. The window is `"p"`, length 1.
- `right = 1` adds `w`. The window is `"pw"`, length 2.
- `right = 2` reads `w`, which is already in the set. Remove `p` and then `w` from the left. Now `left = 2`. Add the new `w`. The window is `"w"`.
- `right = 3` and `right = 4` add `k` and `e`. The window is `"wke"`, length 3.
- `right = 5` reads `w` again. Remove one `w` from the left. The window is `"kew"`, length 3.

The answer is 3. Six characters entered the window and three characters left it. The work is linear.

47.2 Recognize it

- The answer is a **contiguous** segment: a subarray, a substring, a run of days, a block of consecutive values. A subsequence or a subset is not a window problem.
- The goal is the longest, the shortest, the number of, or the existence of a segment with a property. Or the goal is one value (a maximum, a sum, a match) for every segment of a fixed size k.
- The property is monotone. Words such as "at most k", "no repeated", "does not contain" and "less than" (with non-negative values) usually give a rule that survives shrinking. Words such as "contains all of", "at least" and "covers" usually give a rule that survives growing.
- A budget limits the bad elements in the segment: at most k changes, flips, zeros, deletions or distinct values.
- The statement says "anagram" or "permutation" of a pattern, together with "substring". The length is then fixed, and only the counts matter.
- The statement asks for the number of segments with *exactly* k of something (odd numbers, distinct values, ones). Count two windows: at most k, and at most $k - 1$.
- The statement asks for the maximum or the minimum of every window of size k. Or the rule uses the maximum or the minimum of the window, such as `max - min <= limit`.
- The length is up to about 10^5. An $O(n^2)$ scan of all segments is too slow, and the target is $O(n)$.
- The statement asks for the best pair `i < j`, such as "buy before you sell". The left end of the window then jumps to the best earlier element.

NOTE

Sums with negative values are not monotone: a shrink can make the sum larger or smaller. For such problems, use prefix sums with a hash map, or a monotonic deque of prefix sums.

47.3 Core template

Answer three questions before you write code.

1. **State.** What data tells you if the window is valid? It must update in $O(1)$ when one element enters and when one element leaves.
2. **Rule.** Write the validity test as an expression of the state.
3. **Record.** Where in the loop is the window an answer candidate? See the table below.

```
left = 0
for right, x in enumerate(a):
    add x to the state                      # grow on the right
    while the window a[left..right] is invalid:
        remove a[left] from the state       # shrink on the left
        left += 1
    record the answer for a[left..right]    # longest: right - left + 1
```

The tested template below finds the longest valid window. The state is a `Counter` of the window. Only the rule changes from problem to problem, so the rule is a parameter.

```
from collections import Counter

def longest_window(a, invalid):
    count = Counter()                   # value -> count in a[left..right]
    left = best = 0
    for right, x in enumerate(a):
        count[x] += 1                   # grow: x enters on the right
```

```
        while invalid(count, right - left + 1):
            out = a[left]              # shrink: a[left] leaves on the left
            count[out] -= 1
            if count[out] == 0:
                del count[out]         # keep len(count) = number of distinct values
            left += 1
        best = max(best, right - left + 1)   # a[left..right] is valid here
    return best

def has_repeat(count, size):
    return len(count) < size           # fewer distinct values than elements

def more_than_two_kinds(count, size):
    return len(count) > 2

def needs_more_than_one_change(count, size):
    return size - max(count.values()) > 1

longest_window("pwwkew", has_repeat)                    # -> 3
longest_window([1, 2, 1, 2, 3], more_than_two_kinds)    # -> 4
longest_window("AABABBA", needs_more_than_one_change)   # -> 4
longest_window("", has_repeat)                          # -> 0
```

Invariant. After the shrink loop, `a[left..right]` is valid, and `left` is the smallest valid start for this right end. The left end never moves back, so the total time is $O(n)$ times the cost of one rule test.

The goal decides the loop shape and the place of the record step.

Goal	Left end moves	Record	Needs
Longest valid	`while` the window is invalid	after the loop: `best = max(best, right - left + 1)`	a rule that stays true when the window shrinks
Count valid, rule survives shrinking	`while` the window is invalid	after the loop: `total += right - left + 1`	a rule that stays true when the window shrinks
Count valid, rule survives growing	`while` the window is valid	after the loop: `total += left`	a rule that stays true when the window grows
Shortest valid	`while` the window is valid	inside the loop, before the removal: `best = min(best, right - left + 1)`	a rule that stays true when the window grows
Fixed size k	remove `a[right - k]` when `right >= k`	when `right >= k - 1`	a state with add and remove
Max or min of each window	pop the front index when it leaves	when the window is full: `a[dq[0]]`	a monotonic deque of indices
Best pair `i < j`	jump to a better start	after the update of the start	one summary of the past, such as a minimum

47.4 Variations

47.4.1 The left end jumps: a running minimum

In Best Time to Buy and Sell Stock, the left end is the buy day and the right end is the sell day. When a price is lower than the price at the left end, no later sell day prefers the old buy day. So the left end jumps to the new low. Only the price at the left end matters, so keep the running minimum. Memory hook: *Remember the cheapest day, sell today.*

```python
def max_profit(prices):
    min_price = prices[0]                    # cheapest buy price so far
    best = 0                                 # 0 means "do not trade"
    for price in prices:
        min_price = min(min_price, price)    # buy at the lowest price so far
        best = max(best, price - min_price)  # sell today
    return best

max_profit([7, 1, 5, 3, 6, 4])   # -> 5
max_profit([7, 6, 4, 3, 1])      # -> 0
max_profit([2, 4, 1])            # -> 2
```

The general idea is the best pair with an order `i < j`. Fix the right element `j`. Keep one summary of the prefix that gives the best partner: a minimum, a maximum, or a hash map. For example, Two Sum keeps a hash map of the earlier values.

47.4.2 Longest window with a set

Longest Substring Without Repeating Characters is the base case of the template. The state is the set of characters in the window. The window is invalid when the new character is already in the set. Shrink until it is not, then add it. Memory hook: *Drop from the left until the new character is unique, then add it on the right.*

```python
def longest_unique(s):
    window = set()                 # characters in s[left..right]
    left = best = 0
    for right, ch in enumerate(s):
        while ch in window:        # shrink until ch is unique
            window.remove(s[left])
            left += 1
        window.add(ch)             # grow
        best = max(best, right - left + 1)
    return best

longest_unique("abcabcbb")   # -> 3
longest_unique("bbbbb")      # -> 1
longest_unique("pwwkew")     # -> 3
longest_unique("")           # -> 0
```

A second form stores the last index of each character. The left end then jumps past the old copy in one step. This form needs `left = max(left, last[ch] + 1)`. See the pitfalls.

47.4.3 Longest window under a budget

Many problems allow at most k changes, flips or deletions. Write the cost of a window as an expression of the state. The cost must not decrease when the window grows. Then shrink while the cost is more than k.

In Longest Repeating Character Replacement, the cheapest way to make a window uniform keeps its most common letter and changes all other letters. So the cost is the window length minus the top count. Memory hook: *Window length minus top count must be at most k.*

```python
from collections import Counter

def character_replacement(s, k):
    count = Counter()
    left = best = 0
    for right, ch in enumerate(s):
        count[ch] += 1
        # changes needed = window length - count of the most common letter
        while (right - left + 1) - max(count.values()) > k:
            count[s[left]] -= 1
            left += 1
        best = max(best, right - left + 1)
```

```
    return best

character_replacement("ABAB", 2)      # -> 4
character_replacement("AABABBA", 1)   # -> 4
character_replacement("ABCD", 0)      # -> 1
```

The scan `max(count.values())` costs $O(26)$ per step. That is fast for a small alphabet. For other budgets, the cost is often simpler. In Max Consecutive Ones III, the cost is the number of zeros in the window. For at most k distinct characters, the cost is the number of keys in the `Counter`.

NOTE

The optimal solution keeps `max_freq` as a running maximum that never decreases. It removes the factor of 26. A stale `max_freq` can keep a window that is not valid now. That window is never longer than the best valid window so far. The answer grows only when a count goes above the old `max_freq`. This trick works only when the goal is the maximum length.

In the optimal form, the window slides and never shrinks. When the window is too long, exactly one element leaves, so an `if` replaces the `while`.

```
def character_replacement_fast(s, k):
    count = {}
    left = max_freq = best = 0
    for right, ch in enumerate(s):
        count[ch] = count.get(ch, 0) + 1
        max_freq = max(max_freq, count[ch])      # never decreases
        if (right - left + 1) - max_freq > k:    # slide: the length never shrinks
            count[s[left]] -= 1
            left += 1
        best = max(best, right - left + 1)
    return best

character_replacement_fast("AABABBA", 1)   # -> 4
character_replacement_fast("ABAB", 2)      # -> 4
character_replacement_fast("ABCD", 0)      # -> 1
```

The general rule: when the goal is the maximum length, a bound that can only be too high is safe. The condition is that the answer can grow only when that bound grows.

47.4.4 Fixed-size window

When the length of the answer is known, the window has a fixed size k. In each step, one element enters at `right`, and the element `a[right - k]` leaves when `right >= k`. The window is full when `right >= k - 1`.

In Permutation in String, a permutation of `s1` is any string with the same letter counts. So slide a window of length `len(s1)` over `s2` and compare two count arrays. Memory hook: *Slide a window of len(s1) over s2 and compare its letter counts to s1's counts.*

```
def check_inclusion(s1, s2):
    k = len(s1)
    need = [0] * 26                # letter counts of s1
    window = [0] * 26              # letter counts of the last k letters of s2
    for ch in s1:
        need[ord(ch) - ord("a")] += 1
    for right, ch in enumerate(s2):
        window[ord(ch) - ord("a")] += 1                # enters on the right
        if right >= k:
            window[ord(s2[right - k]) - ord("a")] -= 1  # leaves on the left
        if window == need:                             # same counts: a permutation
            return True
    return False

check_inclusion("ab", "eidbaooo")   # -> True
```

```
check_inclusion("ab", "eidboaoo")    # -> False
check_inclusion("abc", "ab")         # -> False
```

The compare `window == need` costs $O(26)$. That is fast for lowercase letters. **Optimal aside.** For a large or unknown alphabet, keep one map `diff`. It holds the window count minus the pattern count for each key. Also keep an integer `bad`: the number of keys with a nonzero `diff`. Only the entering key and the leaving key can change `bad`. The window matches when `bad == 0`.

```
def check_inclusion_fast(s1, s2):
    k = len(s1)
    diff = {}                        # count in window - count in s1
    for ch in s1:
        diff[ch] = diff.get(ch, 0) - 1
    bad = len(diff)                  # keys with a nonzero diff
    def update(ch, delta):
        nonlocal bad
        old = diff.get(ch, 0)
        diff[ch] = old + delta
        bad += (old == 0) - (old + delta == 0)    # +1 if it left 0, -1 if it came back
    for right, ch in enumerate(s2):
        update(ch, 1)
        if right >= k:
            update(s2[right - k], -1)
        if bad == 0:
            return True
    return False

check_inclusion_fast("ab", "eidbaooo")    # -> True
check_inclusion_fast("aab", "abba")       # -> False
check_inclusion_fast("aab", "cbaa")       # -> True
```

A fixed window also gives the sum, the average or the number of vowels of each block of k elements. Add the entering value and subtract the leaving value.

47.4.5 Shortest window: shrink while valid

For a "shortest" goal, flip the loop. Grow the window until it is valid. Then shrink it while it stays valid, and record the window *inside* the shrink loop, before each removal. The simplest case uses a sum of non-negative values and a positive target.

```
def shortest_sum_at_least(a, target):     # values >= 0, target > 0
    total = left = 0
    best = float("inf")
    for right, x in enumerate(a):
        total += x
        while total >= target:            # valid: record, then shrink
            best = min(best, right - left + 1)
            total -= a[left]
            left += 1
    return 0 if best == float("inf") else best

shortest_sum_at_least([2, 3, 1, 2, 4, 3], 7)   # -> 2
shortest_sum_at_least([1, 1, 1], 5)            # -> 0
```

When the rule is "the window contains every required item with its count", use a **need/have** pair of counters. `need` holds the required counts. `have` counts the keys that meet their requirement. The window is valid when `have == required`. Change `have` only when one count crosses its threshold: up when `window[c]` becomes equal to `need[c]`, down when it falls below `need[c]`. This is Minimum Window Substring. Memory hook: *Grow right until have equals required, then shrink left while it holds.*

```python
from collections import Counter

def min_window(s, t):
    need = Counter(t)
    window = Counter()
    have, required = 0, len(need)     # keys that meet their count; keys in t
    best_len, best_start = float("inf"), 0
    left = 0
    for right, c in enumerate(s):
        window[c] += 1
        if c in need and window[c] == need[c]:          # c just reached its count
            have += 1
        while have == required:                         # valid: record, then shrink
            if right - left + 1 < best_len:
                best_len, best_start = right - left + 1, left
            out = s[left]
            window[out] -= 1
            if out in need and window[out] < need[out]:  # out just fell short
                have -= 1
            left += 1
    if best_len == float("inf"):
        return ""
    return s[best_start:best_start + best_len]

min_window("ADOBECODEBANC", "ABC")   # -> 'BANC'
min_window("a", "aa")                # -> ''
min_window("aa", "aa")               # -> 'aa'
```

Store the start and the length of the best window, and slice one time at the end. A slice at each improvement copies the window each time.

47.4.6 Counting windows: add right - left + 1

Count the valid segments when the rule survives shrinking. Fix the right end. After the shrink loop, every start from `left` to `right` gives a valid segment. So `right - left + 1` valid segments end at `right`. Add this number for each right end. This counts up to $n(n+1)/2$ segments in $O(n)$. Memory hook: *Every start from left to right is valid, so add right - left + 1.*

In a problem with forbidden pairs, the segments are runs `l, l + 1, ..., r` of the numbers `1..n`. A run is invalid when it contains both numbers of a forbidden pair. So the pair $\{a, b\}$ acts as the interval `[lo, hi]`, and a run is invalid when it covers that interval. A pair has no effect until the right end reaches `hi`. From then on, every valid start is after `lo`. Keep `block[hi]`, the largest `lo` for each `hi`, and move `left` with a running maximum. Memory hook: *A pair closes at its max and pushes left past its min.*

```python
def count_valid_sequences(n, pairs):
    block = [0] * (n + 1)          # block[r] = largest lo of a pair with hi = r
    for a, b in pairs:
        lo, hi = min(a, b), max(a, b)
        block[hi] = max(block[hi], lo)
    total = 0
    left = 1                       # smallest valid start for this right end
    for right in range(1, n + 1):
        left = max(left, block[right] + 1)   # left never moves back
        total += right - left + 1            # starts left, left + 1, ..., right
    return total

count_valid_sequences(4, [[1, 3]])           # -> 8
count_valid_sequences(3, [])                 # -> 6
count_valid_sequences(5, [[1, 2], [4, 5]])   # -> 8
```

The trick is reusable. Many constraints can say "a window that ends at or after `p` must start after `q`". Bucket them

by `p`, keep the largest `q` per bucket, and take a running maximum in the sweep.

A rule with "exactly k" does not survive shrinking. Count two windows that do: *exactly k = at most k minus at most $k-1$*. The example counts the subarrays with exactly k distinct values ($k \geq 1$).

```
from collections import Counter

def count_at_most(a, k):           # subarrays with at most k distinct values
    count = Counter()
    left = total = 0
    for right, x in enumerate(a):
        count[x] += 1
        while len(count) > k:
            count[a[left]] -= 1
            if count[a[left]] == 0:
                del count[a[left]]
            left += 1
        total += right - left + 1
    return total

def count_exactly(a, k):
    return count_at_most(a, k) - count_at_most(a, k - 1)

count_at_most([1, 2, 1, 2, 3], 2)    # -> 12
count_exactly([1, 2, 1, 2, 3], 2)    # -> 7
count_exactly([1, 2, 1, 3, 4], 3)    # -> 3
```

Some count rules survive growing: "contains all of", "at least k". Then flip the loop. Shrink while the window is valid. After the loop, `a[left..right]` is invalid, but every start before `left` gives a valid window. So add `left` for each right end. Memory hook: *Shrink while valid, then every start before left is valid, so add left.*

```
from collections import Counter

def count_with_all(s, letters):    # substrings that contain every letter
    count = Counter()
    left = total = 0
    for right, ch in enumerate(s):
        count[ch] += 1
        while all(count[c] > 0 for c in letters):   # valid: shrink
            count[s[left]] -= 1
            left += 1
        total += left              # starts 0, 1, ..., left - 1 are valid
    return total

count_with_all("abcabc", "abc")    # -> 10
count_with_all("aaacb", "abc")     # -> 3
count_with_all("ab", "abc")        # -> 0
```

Another way: count the invalid windows with `right - left + 1` and subtract them from $n(n+1)/2$. (Number of Substrings Containing All Three Characters, Count Subarrays Where Max Element Appears at Least K Times.)

47.4.7 Window maximum: a monotonic deque

A running maximum cannot handle a removal. When the maximum leaves the window, the next maximum is unknown. Use dominance: if an older element is not larger than a newer element, the older one leaves first and never wins again. Delete it when the newer element arrives. The remaining candidates have decreasing values from front to back, so the front is the maximum. This is Sliding Window Maximum. Memory hook: *Kick out the old from the front, kick out the weak from the back, and the front is the max.*

```python
from collections import deque

def max_sliding_window(nums, k):
    candidates = deque()     # indices; their values go down from front to back
    result = []
    for i, x in enumerate(nums):
        if candidates and candidates[0] <= i - k:        # old: left the window
            candidates.popleft()
        while candidates and nums[candidates[-1]] <= x:  # weak: never a max again
            candidates.pop()
        candidates.append(i)
        if i >= k - 1:                                   # the window is full
            result.append(nums[candidates[0]])
    return result

max_sliding_window([1, 3, -1, -3, 5, 3, 6, 7], 3)   # -> [3, 3, 5, 5, 6, 7]
max_sliding_window([1], 1)                          # -> [1]
max_sliding_window([4, 2, 2, 1], 2)                 # -> [4, 2, 2]
```

Store indices, not values. The index tells you when the front leaves the window. Each index enters and leaves the deque at most one time, so the time is $O(n)$.

- For the window minimum, pop the back while its value is `>= x`.
- In a variable window, the left end can pass several indices. Pop the front with a `while` loop while its index is less than `left`.
- A rule such as `max - min <= limit` needs two deques: one for the maximum and one for the minimum.
- The same deque speeds up a DP of the form `dp[i] = f(i) + max(dp[j])` over the last k indices.

47.5 Adapt it to a new problem

Ask these questions in this order.

1. Is the answer one contiguous segment? If the answer can skip elements, use another technique.
2. If the segment is not visible, reframe the input. If the order does not matter, sort first: the best group is then often a window of the sorted array. If you remove items from both ends, the items that stay form one window of fixed size.
3. Is the length fixed? Then use a fixed window: one element in and one element out in each step.
4. Write the rule for a valid window in one sentence. Is it monotone? Try a small example. For a longest goal, a valid window must stay valid when it shrinks. For a shortest goal, it must stay valid when it grows. For a count goal whose rule survives growing (contains all of, at least k), shrink while the window is valid, then add `left`: every start before `left` gives a valid window. Or count the invalid windows with `right - left + 1` and subtract them from $n(n+1)/2$. Use prefix sums only when no direction is monotone, for example sums with negative values.
5. Choose the state. It must update in $O(1)$ for one entering and one leaving element: a set, a `Counter`, a sum, a count of bad elements, a need/have pair, or a deque.
6. Write the rule as an expression of the state. Examples: "at most k changes" gives `length - top_count <= k`. "Contains all of `t`" gives `have == required`.
7. Choose the loop shape from the table in the core template. Put the record step in the correct place.
8. If the rule says "exactly k", count "at most k" minus "at most $k-1$".
9. If the rule needs the maximum or the minimum of the window, keep a monotonic deque.
10. If constraints connect two positions, turn each one into an interval. Keep the largest blocking start per right end.
11. Check the edges: an empty input, k larger than n, no valid window. Use `float("inf")` as the start value of a shortest length, and return the "none" value at the end.
12. Compare with a brute force over all $O(n^2)$ segments on small random inputs.

47.6 Pitfalls

CAUTION

A stale index moves the left end back. In the last-index form of the longest unique substring, the last copy of a character can be before `left`. Then `left = last[ch] + 1` moves the window back and includes a repeat. Use `left = max(left, last[ch] + 1)`.

```
def jump_bad(s):
    last, left, best = {}, 0, 0
    for right, ch in enumerate(s):
        if ch in last:
            left = last[ch] + 1              # wrong: can move left back
        last[ch] = right
        best = max(best, right - left + 1)
    return best

def jump_good(s):
    last, left, best = {}, 0, 0
    for right, ch in enumerate(s):
        if ch in last:
            left = max(left, last[ch] + 1)   # left never moves back
        last[ch] = right
        best = max(best, right - left + 1)
    return best

jump_bad("abba"), jump_good("abba")          # -> (3, 2)
```

CAUTION

A `Counter` keeps keys with a count of 0. Then `len(count)` is not the number of distinct values in the window. Delete a key when its count reaches 0, or keep a separate counter of distinct values.

```
from collections import Counter
count = Counter("aab")
count["b"] -= 1              # "b" left the window
len(count)                   # -> 2
del count["b"]
len(count)                   # -> 1
```

CAUTION

Negative values break a sum window. With negative values, a shrink can increase the sum. The window below misses the full array, which has a sum of 0. Use prefix sums for such problems.

```
def longest_sum_at_most(a, k):        # correct only when all values are >= 0
    total = left = best = 0
    for right, x in enumerate(a):
        total += x
        while left <= right and total > k:
            total -= a[left]
            left += 1
        best = max(best, right - left + 1)
    return best

longest_sum_at_most([1, 2, 3, 1], 4)  # -> 2
longest_sum_at_most([5, -10, 5], 0)   # -> 2
```

CAUTION

The record step in the wrong place. For a shortest window, record inside the shrink loop, before the removal. After the loop, the window is not valid. For a longest window, record after the loop. Inside the loop, the window is not valid yet.

CAUTION

`if` instead of `while`. One removal does not always make the window valid. A repeated character can be far inside the window. Use `while` for the shrink step. The fixed window is an exception, because exactly one element leaves in each step. The slide form with a stale `max_freq` is also an exception, because its window never shrinks.

CAUTION

Off-by-one errors. The window length is `right - left + 1`. In a fixed window, the leaving element is `a[right - k]`, and it leaves only when `right >= k`. The first full window ends at `right = k - 1`.

CAUTION

An empty window. If one element alone can break the rule, the shrink loop can empty the window. Then `left` is `right + 1`, and a test such as `max(count.values())` can raise `ValueError`. Add `left <= right` to the loop condition, or make sure the rule is true for one element.

CAUTION

The have counter counts crossings, not elements. Increase `have` only when a count becomes equal to its requirement. Decrease it only when a count falls below its requirement. A surplus copy must not change `have`. A key that is not in `need` must not change `have`.

CAUTION

Hidden $O(k)$ work. Do not call `sum(a[left:right + 1])`, `max(window)` or `Counter(s[left:right + 1])` inside the loop. Each call rescans the window and gives $O(nk)$. Update the state with the one entering and the one leaving element.

CAUTION

The empty answer. Start a longest length at 0 and a shortest length at `float("inf")`. In a profit problem, start at 0 when "no trade" is allowed. Return the value the statement gives for "no valid window", such as `""`, `0` or `-1`.

47.7 Problems in this family

Problem	What changes	Memory hook
Best Time to Buy and Sell Stock	The left end jumps to a new low. The state is the running minimum.	Remember the cheapest day, sell today.
Longest Substring Without Repeating Characters	Longest window. The state is a set. Shrink while the new character is in the set.	Drop from the left until the new character is unique, then add it on the right.
Longest Repeating Character Replacement	Longest window under a budget. The cost is the length minus the top count.	Window length minus top count must be at most k.
Permutation in String	Fixed window of length `len(s1)`. Compare two count arrays.	Slide a window of len(s1) over s2 and compare its letter counts to s1's counts.
Minimum Window Substring	Shortest window. Need/have counters. Record inside the shrink loop.	Grow right until have equals required, then shrink left while it holds.
Sliding Window Maximum	Fixed window. The state is a monotonic deque of indices.	Kick out the old from the front, kick out the weak from the back, and the front is the max.

The transfer drills for this technique are in Section 69.5 (page 419). The answers are in Section A.4 (page 433).

48 Stacks and monotonic stacks

48.1 The idea

A stack keeps the items that are still *pending*: openers that wait for a closer, values that wait for an operator, or indexes that wait for an answer. Each new item looks only at the newest pending item. It finishes the newest pending items that it can finish (pop). Then, if it must wait too, it joins the pile (push). This is last in, first out order. A brute force rescans the earlier items for each new item and costs $O(n^2)$. The stack removes each finished item for good, so each item is pushed once and popped at most once. The total cost is $O(n)$, even with a `while` loop inside the `for` loop.

There are two kinds of stack problems.

- **Matching stack.** Pairs nest but do not cross. The next closer can match only the most recent unmatched opener. Brackets, tags, postfix expressions and undo work this way.
- **Monotonic stack.** Each element waits for the first later element that beats it (greater or smaller). A new element resolves every waiting element that it beats. No element on the stack beats an element below it, so the values on the stack stay sorted.

Worked example (matching). Check `"([)]"`. Push `(`, then push `[`. Next comes `)`. The top is `[`, which is the wrong opener, so the string is not valid. Two counters (one per bracket type) accept this string, because counters forget the order of the openers.

Worked example (monotonic). For each day of `[73, 74, 75, 71, 69, 72, 76]`, find the number of days until a warmer day. Day 0 (73) waits. Day 1 (74) is warmer, so day 0 gets 1 and leaves; day 1 waits. Day 2 (75) resolves day 1. Days 3 (71) and 4 (69) are colder, so they wait on top of day 2. Day 5 (72) pops day 4 (gap 1) and day 3 (gap 2). It stops at day 2, because 75 is not colder. Day 6 (76) pops day 5 (gap 1) and day 2 (gap 4). The result is `[1, 1, 4, 2, 1, 1, 0]`. The waiting temperatures never increase from bottom to top. So when the pops reach an element that is not colder, no element below it is colder, and the pops can stop.

The same pass answers "previous" questions. Take the previous smaller element. A new element hides every earlier element that is greater than or equal to it, because the new element is closer and not larger. Those earlier elements can never be an answer again, so the pass pops them. What stays below the new element is its answer.

48.2 Recognize it

- Items open and close. Each closer pairs with an earlier opener. Pairs nest and do not cross (brackets, HTML tags, start and end entries of function logs).
- An operator or a closing token acts on the most recent pending items (postfix expressions, nested decoding such as `3[a2[c]]`, path simplification, backspace or undo).
- A design asks for an aggregate (min, max, sum) in $O(1)$ next to push and pop. Removals are LIFO, so every earlier state comes back after a pop.
- "For each element, find the nearest element to the left or right that is greater or smaller", or the distance to it, or the value of it.
- The output has one entry per element (an index, a distance or a value). A default such as `-1` or `0` marks an element with no answer.
- The score of a range is the minimum (or maximum) of the range times its length or sum. As the minimum, each element limits a range that stops just before the nearest smaller element on each side. For a maximum, use the nearest greater element.
- The amount at each index depends on the tallest element on its left and the tallest element on its right (water between bars).
- Remove items and keep the order of the others, so that the result is the smallest (or largest) possible. The limit is a count k, or a rule such as "keep each letter exactly once".
- Objects on a line cannot pass each other, and the question counts the groups that form.
- The task asks for all valid bracket strings (or other nested structures) of size n, and n is small (about 8 or less). This is backtracking, and the state is two counts: openers used and closers used.
- n is up to about 10^5, and the brute force scans right from every index. That is $O(n^2)$.

48.3 Core template

Matching stack. Push each opener. A closer must find its own opener on top, and then it pops it. At the end, the stack must be empty.

```
def is_valid(s):
    opener_of = {")": "(", "]": "[", "}": "{"}
    stack = []                          # openers that wait for a closer
    for c in s:
        if c in opener_of:              # c is a closer
            if not stack or stack[-1] != opener_of[c]:
                return False
            stack.pop()
        else:
            stack.append(c)
    return not stack                    # leftover openers make it invalid

is_valid("([]{})")   # -> True
is_valid("([)]")     # -> False
is_valid("((")       # -> False
is_valid(")")        # -> False
```

Invariant: before each character, the stack holds the unmatched openers of the prefix, in their original order. A closer is legal only if it matches the top.

Monotonic stack. Keep the indexes that still wait for an answer. Pop every waiting index that the new element beats. There are two ways to read an answer. At pop time, the popped index gets the current index as its *next* element. After the pops, the index on top is the *previous* element of the current one.

```
def next_greater(a):
    # ans[i] = index of the first j > i with a[j] > a[i], else -1
    ans = [-1] * len(a)
    stack = []                          # indexes that wait for an answer
    for i, x in enumerate(a):
        while stack and a[stack[-1]] < x:
            ans[stack.pop()] = i        # x resolves each smaller waiting index
        stack.append(i)
    return ans

def prev_smaller(a):
    # ans[i] = index of the last j < i with a[j] < a[i], else -1
    ans = [-1] * len(a)
    stack = []
    for i, x in enumerate(a):
        while stack and a[stack[-1]] >= x:
            stack.pop()                 # never a "previous smaller" again
        if stack:
            ans[i] = stack[-1]
        stack.append(i)
    return ans

a = [2, 1, 2, 4, 3]
next_greater(a)       # -> [3, 2, 3, -1, -1]
prev_smaller(a)       # -> [-1, -1, 1, 2, 2]

# check: compare with a brute force on all small arrays
from itertools import product

def brute_next_greater(a):
    return [next((j for j in range(i + 1, len(a)) if a[j] > a[i]), -1) for i in range(len(a))]

all(next_greater(list(t)) == brute_next_greater(list(t))
    for n in range(6) for t in product(range(3), repeat=n))   # -> True
```

Invariant: the stack holds the indexes that are not popped yet, in increasing index order, and their values are sorted. These indexes still wait for a *next* answer, and only they can still be a *previous* answer. Each index is pushed once and popped at most once, so the pass is $O(n)$. Choose the pop condition from the question:

For each `i`, find	Pop while	Read the answer	Stack values
next greater	`a[top] < x`	popped index gets `i`	non-increasing
next greater or equal	`a[top] <= x`	popped index gets `i`	strictly decreasing
next smaller	`a[top] > x`	popped index gets `i`	non-decreasing
previous greater	`a[top] <= x`	top after the pops	strictly decreasing
previous smaller	`a[top] >= x`	top after the pops	strictly increasing

Rule: pop what can never be an answer again. One pass gives two answers. The popped indexes get their *next* answer, and the top after the pops is the *previous* answer with the opposite strictness. For example, the pass with `a[top] < x` gives the next greater and the previous greater-or-equal element.

48.4 Variations

48.4.1 Match and validate nested pairs

Push openers and pop on a matching closer (Valid Parentheses). Memory hook: *push openers; each closer must pop its own opener off the top; end with an empty stack.*

- One bracket type and only a yes or no answer: a balance counter replaces the stack. Add 1 for an opener, subtract 1 for a closer, fail below 0, and require 0 at the end.
- Positions or lengths matter: push indexes instead of characters. The indexes that stay unmatched mark the parts to remove or the borders of valid runs.
- Tags or log entries: push the name or id on start, and check it on end.
- A token cancels the newest kept token (`..` in a file path, a backspace, an undo): push the normal tokens, and pop on the cancel token. The stack holds the result.

48.4.2 Evaluate an expression: a stack of pending values

In postfix order, each operator acts on the two newest pending values (Evaluate Reverse Polish Notation). Push numbers. On an operator, pop the right operand first, then the left operand, and push the result. Memory hook: *push numbers; an operator pops b, then a, and pushes a op b; divide with* ***`int(a / b)`****.*

```
OPS = {
    "+": lambda a, b: a + b,
    "-": lambda a, b: a - b,
    "*": lambda a, b: a * b,
    "/": lambda a, b: int(a / b),    # truncate toward zero
}

def eval_rpn(tokens):
    stack = []                       # values that wait for an operator
    for tok in tokens:
        if tok in OPS:               # compare the whole token: "-11" is a number
            b = stack.pop()          # the right operand is on top
            a = stack.pop()
            stack.append(OPS[tok](a, b))
        else:
            stack.append(int(tok))
    return stack[0]

eval_rpn(["2", "1", "+", "3", "*"])      # -> 9
eval_rpn(["4", "13", "5", "/", "+"])     # -> 6
eval_rpn(["7", "-2", "/"])               # -> -3
eval_rpn(["-11"])                        # -> -11
-7 // 2, int(-7 / 2)                     # -> (-4, -3)
```

- Prefix (Polish) notation: scan from the right with the same stack. Pop the left operand first.
- Infix with precedence and no parentheses: keep the previous operator. Push `+num` or `-num`. For `*` and `/`, pop the top, combine it with `num`, and push the result. The answer is the sum of the stack.
- Infix with parentheses and only `+` and `-`: on `(`, push the current result and sign, and start a new result. On `)`, pop and combine. With all four operators and parentheses, use recursive descent: evaluate each part in parentheses with the method of the previous item.

NOTE

`int(a / b)` is exact for 32-bit operands. For very large integers, use `abs(a) // abs(b)` and negate the result when the signs differ.

48.4.3 A stack that carries extra state

Store a snapshot of the aggregate with each push (Min Stack). Each entry is the pair (value, minimum of this value and all values below it). A pop removes the value and its snapshot together, so the old minimum comes back with no work. Memory hook: *push the pair (value, min so far); pop removes both.*

```
class MinStack:
    def __init__(self):
        self.stack = []   # (value, min of this value and all values below it)

    def push(self, val):
        low = min(val, self.stack[-1][1]) if self.stack else val
        self.stack.append((val, low))

    def pop(self):
        self.stack.pop()

    def top(self):
        return self.stack[-1][0]

    def getMin(self):
        return self.stack[-1][1]

s = MinStack()
for v in [5, 3, 7, 3]:
    s.push(v)
s.getMin()              # -> 3
s.pop()
s.pop()
s.getMin(), s.top()     # -> (3, 3)
s.pop()
s.getMin(), s.top()     # -> (5, 5)
```

The same idea works for any aggregate that extends by one element: max, sum, count, gcd. Min and max have no inverse: the current minimum and the popped value do not give back the old minimum. So save the old value with each level instead. Nested decoding uses the same idea. On an opener, push the current state (for example the text so far and a repeat count), and start fresh. On the closer, pop the state and combine. A common alternative splits the pairs into two parallel lists, with the same cost. Then each pop must remove from both lists.

48.4.4 Generate balanced structures

Generate every valid string of `n` bracket pairs (Generate Parentheses). For one bracket type, the stack is only a count of open brackets. So backtrack with two counts and allow only moves that keep the prefix completable. Memory hook: *open while opens remain; close only what is open; save at length 2n.*

```python
def generate_parentheses(n):
    result = []

    def build(text, opens, closes):
        if len(text) == 2 * n:
            result.append(text)
            return
        if opens < n:                # an opener is still free
            build(text + "(", opens + 1, closes)
        if closes < opens:           # close only what is open
            build(text + ")", opens, closes + 1)

    build("", 0, 0)
    return result

generate_parentheses(3)
# -> ['((()))', '(()())', '(())()', '()(())', '()()()']
len(generate_parentheses(4))   # -> 14
```

Every branch ends in a valid string, so the work is proportional to the output size. The output has $C(n)$ strings (the Catalan number) of length $2n$, so the total is $O(n \cdot C(n)) = O(4^n/\sqrt{n})$. With several bracket types, the state must hold the stack of open types. The optimal version uses one shared list as the path (append, recurse, pop) and joins it at a leaf. It saves a string copy per node. That is only a constant factor here.

48.4.5 Next greater element: the answer at pop time

Each new element is the answer for every smaller element that still waits (Daily Temperatures). Memory hook: *today pops every colder waiting day and writes the gap.*

```python
def daily_temperatures(temps):
    answer = [0] * len(temps)
    waiting = []                     # days that still wait for a warmer day
    for today, t in enumerate(temps):
        while waiting and temps[waiting[-1]] < t:
            day = waiting.pop()
            answer[day] = today - day
        waiting.append(today)
    return answer

daily_temperatures([73, 74, 75, 71, 69, 72, 76, 73])
# -> [1, 1, 4, 2, 1, 1, 0, 0]
daily_temperatures([30, 30, 31])   # -> [2, 1, 0]
```

- Store indexes, not values. An index gives both the distance and the value.
- The question asks for the next value instead of the distance: write `a[i]` into the popped slot.
- Circular array: loop `i` over `range(2 * n)`, read `a[i % n]`, and push only when `i < n`.
- A stream of queries for the previous greater element: keep (value, span) pairs. A new value absorbs the spans of the values that it pops.

```python
def next_greater_circular(nums):
    n = len(nums)
    ans = [-1] * n
    stack = []
    for i in range(2 * n):
        x = nums[i % n]
        while stack and nums[stack[-1]] < x:
            ans[stack.pop()] = x     # the value, not the distance
        if i < n:                    # the second lap only resolves
            stack.append(i)
    return ans
```

```
next_greater_circular([1, 2, 1])       # -> [2, -1, 2]
next_greater_circular([5, 4, 3, 4])    # -> [-1, 5, 4, 5]
```

48.4.6 Smallest subsequence: a greedy increasing stack

Some problems remove items, keep the order of the others, and ask for the smallest result (Remove K Digits, Remove Duplicate Letters). An earlier position weighs more than all later positions together. So a smaller item that arrives replaces a larger item before it, if that larger item may still go. The stack holds the result so far. Memory hook: *pop a bigger top that you can still afford to lose.* The problem sets the permission to pop.

- A budget of k removals (Remove K Digits): pop while removals remain, and count each pop. After the scan, cut the unused removals from the end, because the end holds the largest digits. Then strip the leading zeros.
- Each letter exactly once (Remove Duplicate Letters): pop the top only when it occurs again later. Keep the last index of each letter for this test. Also skip a letter that is already on the stack. The result needs it only once, and the copy on the stack already has its best position.
- The largest result: reverse the comparison, and pop a smaller top.

```
def remove_k_digits(num, k):
    stack = []                          # the result so far
    for d in num:
        while k and stack and stack[-1] > d:
            stack.pop()                 # a smaller digit replaces a larger one before it
            k -= 1
        stack.append(d)
    if k:
        stack = stack[:-k]              # unused removals come off the end
    return "".join(stack).lstrip("0") or "0"

def smallest_subsequence(s):            # each letter exactly once
    last = {ch: i for i, ch in enumerate(s)}    # last index of each letter
    stack, on_stack = [], set()
    for i, ch in enumerate(s):
        if ch in on_stack:
            continue                    # the result needs it only once
        while stack and stack[-1] > ch and last[stack[-1]] > i:
            on_stack.discard(stack.pop())   # the top comes again later
        stack.append(ch)
        on_stack.add(ch)
    return "".join(stack)

remove_k_digits("1432219", 3)           # -> '1219'
remove_k_digits("10200", 1)             # -> '200'
remove_k_digits("10", 2)                # -> '0'
smallest_subsequence("bcabc")           # -> 'abc'
smallest_subsequence("cbacdcbc")        # -> 'acdb'

# check: compare with a brute force on all small inputs
from itertools import combinations, product

def brute_remove(num, k):
    kept = ("".join(t) for t in combinations(num, len(num) - k))
    return min((t.lstrip("0") or "0" for t in kept), key=int)

def brute_smallest(s):
    m = len(set(s))
    return min("".join(t) for t in combinations(s, m) if len(set(t)) == m)

all(remove_k_digits(s, k) == brute_remove(s, k)
    for n in range(1, 6) for s in map("".join, product("0123", repeat=n))
    for k in range(n + 1))                                  # -> True
all(smallest_subsequence(s) == brute_smallest(s)
```

```
    for n in range(1, 7) for s in map("".join, product("abc", repeat=n)))   # -> True
```

48.4.7 Sort first, then a stack of groups

Objects on a line cannot pass each other, and the question counts groups (Car Fleet). Reduce each object to one number: its finish time on an empty road. The general rule: when only the end state matters, compute the outcome of each item alone first. Then apply the interaction rule in one ordered pass.

Here a car cannot pass the fleet ahead, so its real time is the maximum of its own time and the time of the fleet ahead. Sort by position and scan from the front, because blocking flows from the front to the back. A car joins the fleet ahead if it would arrive at the same time or earlier. Memory hook: *front car first: a car that arrives later than the fleet ahead starts a new fleet.*

```
def car_fleet(target, position, speed):
    cars = sorted(zip(position, speed), reverse=True)   # front car first
    fleets = 0
    fleet_time = 0.0                # arrival time of the fleet just ahead
    for pos, spd in cars:
        arrive = (target - pos) / spd    # time on an empty road
        if arrive > fleet_time:          # cannot catch the fleet ahead
            fleets += 1
            fleet_time = arrive
    return fleets

car_fleet(12, [10, 8, 0, 5, 3], [2, 4, 1, 1, 3])   # -> 3
car_fleet(10, [3], [3])                             # -> 1
car_fleet(100, [0, 2, 4], [4, 2, 1])               # -> 1
```

The usual stack version pushes each new fleet time onto a stack and returns the stack size. The stack only uses its top, so it is a running maximum. Keep the stack when the question asks about each fleet, for example its size. Then push (time, count), and add to the top count when a car joins. The same scan without the sort finds the buildings with an ocean view. Scan from the ocean side, and keep each strict new maximum.

48.4.8 Nearest smaller on both sides: largest rectangle

Enumerate the limiting element instead of the range (Largest Rectangle in Histogram). Suppose bar `i` is the shortest bar of the rectangle. Then the rectangle spans the gap between the nearest strictly shorter bar on the left and the nearest strictly shorter bar on the right. The best rectangle has a shortest bar, so this search finds it. It replaces $O(n^2)$ ranges with n candidates. Find both borders with one helper, called once in each direction. Memory hook: *each bar is the shortest: stretch it to the nearest shorter bar on both sides.*

```
def largest_rectangle(heights):
    n = len(heights)

    def nearest_smaller(order, default):
        # result[i] = closest strictly shorter bar before i in this order
        result = [default] * n
        stack = []
        for i in order:
            while stack and heights[stack[-1]] >= heights[i]:
                stack.pop()
            if stack:
                result[i] = stack[-1]
            stack.append(i)
        return result

    left = nearest_smaller(range(n), -1)
    right = nearest_smaller(range(n - 1, -1, -1), n)
    return max(heights[i] * (right[i] - left[i] - 1) for i in range(n))

largest_rectangle([2, 1, 5, 6, 2, 3])   # -> 10
largest_rectangle([2, 4])               # -> 4
```

```
largest_rectangle([4, 0, 4])           # -> 4
```

The optimal version does the work in one pass. A popped bar gets its area when the shorter bar arrives. The new bar inherits the start index of the last bar that it popped, because it extends left over all of them. A 0-height sentinel at the end flushes the stack. Use this "absorb the span" trick when the popped element's range must pass to the new element.

```
def largest_rectangle_one_pass(heights):
    best = 0
    stack = []                        # (start, height); heights never decrease
    for i, h in enumerate(heights + [0]):    # the 0 bar pops every bar
        start = i
        while stack and stack[-1][1] > h:
            start, top = stack.pop()
            best = max(best, top * (i - start))
        stack.append((start, h))     # h reaches left to the last popped start
    return best

largest_rectangle_one_pass([2, 1, 5, 6, 2, 3])   # -> 10

# check: compare with a brute force on all small arrays
from itertools import product

def brute(hs):
    return max(min(hs[l:r + 1]) * (r - l + 1)
               for l in range(len(hs)) for r in range(l, len(hs)))

all(largest_rectangle_one_pass(list(t)) == brute(list(t))
    for n in range(1, 6) for t in product(range(4), repeat=n))   # -> True
```

Some problems count over all subarrays, for example the sum of the subarray minimums. There, use a strict comparison on one side and a non-strict comparison on the other side. Then each subarray with equal minimums is counted once. When the score is the minimum times the sum (Maximum Subarray Min-Product), keep the same borders and read the range sum from prefix sums. For a binary matrix, build one histogram per row and run this on each row (Maximal Rectangle).

48.4.9 Fill between walls: trapping water

The water above index `i` rises to the lower of two walls (Trapping Rain Water). The left wall is the tallest bar on its left, and the right wall is the tallest bar on its right. A brute force scans both sides for every index, which is $O(n^2)$. The intuitive solution needs no stack: precompute prefix maxima and suffix maxima, then sum the levels. Memory hook: *water at i is the lower wall minus the bar: min(left max, right max) - height.* The stack version fills water in horizontal layers. Keep a stack of indexes with non-increasing heights. A taller bar pops the floor bar. The new top is the left wall, and the current bar is the right wall. The layer has height `min(left wall, right wall) - floor` and width `i - left - 1`.

```
def trap_walls(height):
    n = len(height)
    left_max, right_max = height[:], height[:]
    for i in range(1, n):
        left_max[i] = max(left_max[i - 1], height[i])
    for i in range(n - 2, -1, -1):
        right_max[i] = max(right_max[i + 1], height[i])
    return sum(min(left_max[i], right_max[i]) - height[i] for i in range(n))

def trap_stack(height):
    water = 0
    stack = []                        # indexes; heights never increase
    for i, h in enumerate(height):
        while stack and height[stack[-1]] < h:
            floor = height[stack.pop()]
```

```
            if not stack:
                break                # no left wall: water runs off
            left = stack[-1]
            level = min(height[left], h)
            water += (level - floor) * (i - left - 1)
        stack.append(i)
    return water

hs = [0, 1, 0, 2, 1, 0, 1, 3, 2, 1, 2, 1]
trap_walls(hs), trap_stack(hs)       # -> (6, 6)
trap_stack([4, 2, 0, 3, 2, 5])       # -> 9

# check: compare the two methods on all small arrays
from itertools import product
all(trap_stack(list(t)) == trap_walls(list(t))
    for n in range(1, 7) for t in product(range(4), repeat=n))   # -> True
```

The optimal solution uses two converging pointers with running maxima and $O(1)$ extra space. Move the side with the smaller running maximum, because that maximum already sets the level there. Use it only when the problem asks for constant extra space.

48.5 Adapt it to a new problem

Ask these questions in order.

1. Does each new item interact only with the newest pending item? If yes, use a stack. Name what "pending" means: an opener, a value that waits for an operator, or an index without an answer.
2. Does the score of a range depend on its minimum or maximum? Then fix the limiting element. Find its nearest smaller (or greater) element on each side, and score the range between them.
3. What do you push? Push an index when you need a distance or a width. Push a tuple when each level needs extra state. Examples are a snapshot of the minimum, a count, a start index and the text so far.
4. When do you pop? Write the pop condition from the question: pop the items that can never be an answer again. Next greater pops smaller items. Previous smaller pops greater-or-equal items. A smallest subsequence pops a larger top that can still go.
5. When do you read the answer? At pop time for the popped item ("next"), or after the pops for the current item ("previous").
6. How do ties work? Read the tie rule in the statement and choose `<` or `<=`. For counts over all subarrays, use strict on one side and non-strict on the other side.
7. What stays on the stack at the end? Those items have no answer (keep the default), or they reach the end of the array (flush them, or add a sentinel).
8. Does the input give objects on a line in no order? Sort by position first. Scan from the side that blocks.
9. Does the stack only use its top? Then replace it with a variable (running maximum) or a counter (one bracket type). To generate balanced strings of one type, two counts replace the stack.
10. Is the array circular? Loop twice with `i % n` and push only in the first lap.

48.6 Pitfalls

CAUTION

Check that the stack is not empty before `stack[-1]` or `stack.pop()`. A closer can arrive when the stack is empty, as in `")"`. Write `while stack and ...`, with the emptiness test first.

CAUTION

Return `not stack` at the end of a matching check. Leftover openers, as in `"(("`, make the input invalid. Separate counters per bracket type accept crossed pairs such as `"([)]"`.

CAUTION

In an expression, the top of the stack is the *right* operand. Pop `b` first, then `a`. Python `//` rounds toward negative infinity: `-7 // 2` is `-4`. Use `int(a / b)` to truncate toward zero. Detect operators by the whole

token, because `"-11"` starts with `-`.

CAUTION

Read the tie rule. An equal temperature is not warmer, so pop only on a strictly lower value. Two cars with equal arrival times meet at the target and form one fleet, so start a new fleet only on a strictly later time.

CAUTION

Do not forget the items that stay on the stack after the loop. In the largest rectangle, they reach the end of the array and often hold the answer (for example, an increasing array). Iterate over `heights + [0]` for a sentinel. Do not append to the caller's list.

CAUTION

One cached minimum variable fails after a pop removes the current minimum. Store the minimum per level. In Car Fleet, compare with the fleet time (the running maximum), not with the car directly ahead. Zip position with speed before you sort.

CAUTION

In Remove Duplicate Letters, skip a letter that is already on the stack. Without the skip, `"cbacdcbc"` gives `"acdbc"` instead of `"acdb"`. In Remove K Digits, cut the unused removals from the end, and return `"0"` for an empty result.

NOTE

A `while` loop inside a `for` loop looks like $O(n^2)$. It is $O(n)$ in total, because each index is pushed once and popped at most once.

48.7 Problems in this family

Problem	What changes	Memory hook
Valid Parentheses (20)	Matching stack of openers; three bracket types need the stack, not counters.	Push openers. Each closer must pop its own opener off the top. End with an empty stack.
Min Stack (155)	Each level stores a snapshot of the minimum, so a pop restores the old minimum.	Push the pair (value, min so far). Pop removes both.
Evaluate Reverse Polish Notation (150)	Stack of pending values; an operator pops two and pushes one.	Push numbers. An operator pops b, then a, and pushes a op b. Divide with int(a / b).
Generate Parentheses (22)	Generate instead of check; one bracket type, so two counts replace the stack.	Open while opens remain. Close only what is open. Save at length 2n.
Daily Temperatures (739)	Next greater element; the answer is the distance, written at pop time.	Today pops every colder waiting day and writes the gap.
Car Fleet (853)	Sort by position, scan from the front; the stack of fleet times is a running maximum.	Front car first: a car that arrives later than the fleet ahead starts a new fleet.
Largest Rectangle in Histogram (84)	Nearest strictly smaller bar on both sides; area = height times gap.	Each bar is the shortest: stretch it to the nearest shorter bar on both sides.
Trapping Rain Water (42)	Lower of two walls per index; a decreasing stack fills water in layers.	Water at i is the lower wall minus the bar: min(left max, right max) - height.

The transfer drills for this technique are in Section 69.6 (page 420). The answers are in Section A.5 (page 435).

49 Binary search

49.1 The idea

Binary search finds the place where a yes/no question changes its answer. Put the candidates on a line in order. The candidates can be indexes, values, sizes, times or split points. Ask one question `ok(x)` about each candidate. If the answers are *monotone*, they form one block of "no" and then one block of "yes": F F F T T T T. Then one question at the middle removes half of the candidates. A brute force asks about every candidate. Binary search asks about $\log_2$ of their count: 20 questions for 10^6 candidates, 30 questions for 10^9 candidates.

Worked example 1: a sorted array. Find the first value that is at least 7 in `[2, 4, 7, 7, 9, 12, 15]`. The question "is `a[i] >= 7`?" gives F F T T T T T on indexes 0 to 6. Ask at index 3. The value 7 passes, so the answer is index 3 or an index to its left. Keep indexes 0 to 3. Ask at index 1. The value 4 fails, so the answer is to its right. Keep indexes 2 to 3. Ask at index 2. The value 7 passes. Only index 2 is left, so the answer is index 2. Three questions replace up to seven.

Worked example 2: no array at all. Find the smallest eating speed that finishes the piles `[3, 6, 7, 11]` in 8 hours. One pile of p bananas takes $\lceil p/k \rceil$ hours at speed k. Ask "does speed k finish in time?". Speed 6 needs $1+1+2+2=6$ hours: yes. Speed 3 needs $1+2+3+4=10$ hours: no. Speed 4 needs $1+2+2+3=8$ hours: yes. Speed 3 fails and speed 4 passes, so the answer is 4. A faster speed never needs more hours. Thus the answers are monotone, and binary search applies. A brute force tries speeds 1, 2, 3, and so on. Binary search tests only about $\log_2$ of the largest pile.

So binary search does not need a sorted array. It needs three things:

- a range of candidates that surely holds the answer;
- a question `ok(x)` that you can answer fast for one candidate;
- a reason why `ok` changes from false to true at most one time along the range.

The work saved is large. A check that costs $O(n)$ and a range of 10^9 values give about $30n$ steps, not $10^9 n$.

49.2 Recognize it

- The input is sorted, and the statement asks for $O(\log n)$ time. A scan is then not accepted.
- The output is a position in sorted data: an insertion point, the first or the last copy of a value, or the count of values below a limit.
- The input is sorted, but it is rotated, shifted or cut into rows. It is still a few sorted runs.
- The question asks for the minimum or the maximum value of a parameter that still meets a limit: a speed, a capacity, a size, a distance, a day. Typical phrases: "the minimum x such that it is possible", "minimize the largest", "maximize the smallest".
- You can test one candidate fast, but no formula gives the answer directly.
- The statement asks for $O(\log n)$ time, but the values are not sorted. Look for a test at `mid` and its neighbor that flips one time.
- The answer range is very large (up to 10^9 or more). A scan over all values is too slow, but $\log_2$ of the range is only about 30.
- The test flips only one time. Typical reasons: sorted data, non-negative values, a bigger region that contains a smaller region, more capacity that always helps.
- A query asks for "the latest value at or before time t", a version, a snapshot, or a value "as of" a date. This is a floor query on a sorted history.
- The question asks for the median or the k-th smallest value of sorted inputs, and a full merge is too slow or not allowed.
- The question asks for the longest strictly increasing chain (a subsequence, or pairs that nest), and n is up to 10^5. The $O(n^2)$ DP is then too slow. A sorted helper array of chain tails gives $O(n \log n)$.

49.3 Core template

Two loops cover almost every problem. Use the first loop to find one exact value. Use the second loop to find a boundary: the first candidate that passes a test.

49.3.1 Exact match on a closed range

Use this loop to find one value when any index that holds it is a correct answer. Unique values give this case (Binary Search).

```
def search(nums, target):
    lo, hi = 0, len(nums) - 1          # closed range [lo, hi] can hold target
    while lo <= hi:                    # the range is not empty
        mid = (lo + hi) // 2
        if nums[mid] == target:
            return mid
        if nums[mid] < target:         # mid and everything left of it are too small
            lo = mid + 1
        else:                          # mid and everything right of it are too big
            hi = mid - 1
    return -1                          # the range is empty: target is absent

nums = [-1, 0, 3, 5, 9, 12]
search(nums, 9)                        # -> 4
search(nums, 2)                        # -> -1
search([5], 5), search([5], 7)         # -> (0, -1)
```

Invariant: if `target` is present, its index is in the closed range `[lo, hi]`. Both updates drop `mid`, so the range shrinks at each step and the loop stops. Memory hook: *closed range, check the middle, drop the half that cannot hold the target.*

49.3.2 First true on a half-open range

Use this loop by default for a boundary question. It is the general template.

```
def first_true(lo, hi, ok):
    # First x in [lo, hi) with ok(x) True, or hi if no x passes.
    # ok must be False ... False True ... True on [lo, hi).
    while lo < hi:
        mid = (lo + hi) // 2
        if ok(mid):
            hi = mid             # mid passes: the answer is mid or left of it
        else:
            lo = mid + 1         # mid fails: the answer is right of it
    return lo

a = [2, 4, 7, 7, 9, 12, 15]
first_true(0, len(a), lambda i: a[i] >= 7)     # -> 2
first_true(0, len(a), lambda i: a[i] > 7)      # -> 4
first_true(0, len(a), lambda i: a[i] >= 99)    # -> 7
first_true(1, 10**9, lambda x: x * x >= 50)    # -> 8
```

Invariant: every candidate below `lo` fails, and `hi` passes or is the end of the range. The lower midpoint gives `mid < hi`, so `hi = mid` always shrinks the range. The loop stops when `lo == hi`. That candidate is the first one that passes. Memory hook: *if mid passes, keep it; if mid fails, skip it.*

The `bisect` module is this template on a sorted list. `bisect_left(a, x)` is the first index with `a[i] >= x`. `bisect_right(a, x)` is the first index with `a[i] > x`. Both return `len(a)` when no index passes.

```
from bisect import bisect_left, bisect_right
a = [2, 4, 7, 7, 9, 12, 15]
bisect_left(a, 7), bisect_right(a, 7)       # -> (2, 4)
bisect_right(a, 7) - bisect_left(a, 7)      # -> 2
bisect_left(a, 99)                          # -> 7
```

49.3.3 Last true

Some questions are true first and false after: T T T F F. A "largest value that works" gives this shape. The last true candidate is one position before the first false candidate. So reuse `first_true` with the negated question.

```
def first_true(lo, hi, ok):
    while lo < hi:
        mid = (lo + hi) // 2
        if ok(mid):
            hi = mid
        else:
            lo = mid + 1
    return lo

def last_true(lo, hi, ok):
    # Last x in [lo, hi) with ok(x) True, or lo - 1 if no x passes.
    # ok must be True ... True False ... False on [lo, hi).
    return first_true(lo, hi, lambda x: not ok(x)) - 1

last_true(0, 100, lambda x: x * x <= 50)    # -> 7
last_true(0, 100, lambda x: x < 0)          # -> -1
last_true(0, 100, lambda x: True)           # -> 99
```

NOTE

Two other forms find the last true. The direct form uses `while lo < hi`, the upper midpoint `(lo + hi + 1) // 2`, `lo = mid` and `hi = mid - 1`. The "best" form uses `while lo <= hi`. On a pass it saves `best = mid` and sets `lo = mid + 1`. On a fail it sets `hi = mid - 1`. When at least one candidate passes, all forms give the same result. They differ only in the value that they return when no candidate passes. Use `first_true` by default. The "best" form is a safe alternative for a last-true question, and some examples below use it. Do not mix the update rules of two forms in one loop.

49.4 Variations

49.4.1 Boundaries in a sorted list: lower bound, upper bound, floor, ceiling

What changes: values can repeat, or the target can be absent, and the question asks for a position. Do not stop on a match. Search for a boundary. The `bisect` functions give each boundary in one line. (Search Insert Position, Find First and Last Position of Element in Sorted Array.)

Question	Code	If none exists
first index with `a[i] >= x` (insertion point)	`bisect_left(a, x)`	`len(a)`
first index with `a[i] > x`	`bisect_right(a, x)`	`len(a)`
count of values `< x`	`bisect_left(a, x)`	0
count of values `<= x`	`bisect_right(a, x)`	0
last index with `a[i] <= x` (floor)	`bisect_right(a, x) - 1`	-1
last index with `a[i] < x`	`bisect_left(a, x) - 1`	-1
count of values in `[lo, hi]`	`bisect_right(a, hi) - bisect_left(a, lo)`	0
first and last index of `x`	`bisect_left(a, x)`, `bisect_right(a, x) - 1`	test `first < len(a)`, then `a[first] == x`

```
from bisect import bisect_left, bisect_right
def first_last(a, x):
    first = bisect_left(a, x)
    if first == len(a) or a[first] != x:       # x is absent
        return [-1, -1]
    return [first, bisect_right(a, x) - 1]
```

```
a = [1, 3, 3, 3, 8]
first_last(a, 3)                        # -> [1, 3]
first_last(a, 5), first_last(a, 9)      # -> ([-1, -1], [-1, -1])
def floor_index(a, x):                  # last index with a[i] <= x, or -1
    return bisect_right(a, x) - 1
floor_index(a, 5), floor_index(a, 0)    # -> (3, -1)
```

Memory hook: *left gives the first* `>= x`*, right gives the first* `> x`*; minus one gives the last one before it.*

49.4.2 A virtual sorted array: a flat index into a grid

What changes (Search a 2D Matrix): each row is sorted, and each row starts above the end of the previous row. Read row by row, the grid is one sorted list of length `rows * cols`. Do not copy the grid into a list. That costs $O(mn)$. Binary search the flat index `k`, and convert it to a cell with `divmod(k, cols)`. Memory hook: *the grid is one sorted list cut into rows, so binary search the flat index and divmod by cols.*

```
def search_matrix(matrix, target):
    rows, cols = len(matrix), len(matrix[0])
    lo, hi = 0, rows * cols - 1
    while lo <= hi:
        mid = (lo + hi) // 2
        r, c = divmod(mid, cols)       # divide by the number of columns
        val = matrix[r][c]
        if val == target:
            return True
        if val < target:
            lo = mid + 1
        else:
            hi = mid - 1
    return False

m = [[1, 3, 5, 7], [10, 11, 16, 20], [23, 30, 34, 60]]
search_matrix(m, 3), search_matrix(m, 13)       # -> (True, False)
search_matrix([[1], [3]], 3)                    # -> True
```

The general rule: binary search works on any structure that gives its k-th element in sorted order in $O(1)$. You do not have to build the sorted list.

NOTE

Sometimes rows and columns are sorted, but a row does not start above the end of the previous row (Search a 2D Matrix II). Then the flat order is not sorted. Walk a staircase from the top-right corner in $O(m+n)$: move left when the value is too big, and move down when it is too small.

49.4.3 Rotated sorted arrays: find the drop

What changes: a rotated sorted array such as `[4, 5, 6, 7, 0, 1, 2]` is two sorted runs, a high run and then a low run. One comparison with `nums[mid]` no longer tells which side holds a target.

To find the minimum (Find Minimum in Rotated Sorted Array), use a fixed reference value on a known side of the drop. The last element is always in the low run. Every value of the high run is greater than the last element. Every value of the low run is not. So the test `nums[i] <= nums[-1]` is F ... F T ... T, and the minimum is at its first true index. An array that is not rotated passes the test everywhere, so the answer is index 0 with no special case. Memory hook: *low run means at most the last value; find the first one.*

CAUTION

Do not compare with `nums[lo]`. When the window is already sorted, the minimum is at `lo`, but the test sends the search to the right. On `[1, 2, 3]`, the loop below returns 3. The general rule: compare with a reference value that is surely on one known side of the boundary.

```python
def find_min_wrong(nums):            # compares with the left end: wrong
    lo, hi = 0, len(nums) - 1
    while lo < hi:
        mid = (lo + hi) // 2
        if nums[mid] >= nums[lo]:
            lo = mid + 1
        else:
            hi = mid
    return nums[lo]

find_min_wrong([4, 5, 1, 2, 3])      # -> 1
find_min_wrong([1, 2, 3])            # -> 3
```

To search for a target (Search in Rotated Sorted Array), split the problem into two searches that you know. First find the drop, which is the index of the minimum. A target `<= nums[-1]` can only be in the low run `nums[pivot:]`. A bigger target can only be in `nums[:pivot]`. Then run a lower-bound search on that one sorted run. Memory hook: *find the drop, then binary search the side that can hold the target.*

```python
from bisect import bisect_left
def first_true(lo, hi, ok):
    while lo < hi:
        mid = (lo + hi) // 2
        if ok(mid):
            hi = mid
        else:
            lo = mid + 1
    return lo

def find_min(nums):
    last = nums[-1]
    return nums[first_true(0, len(nums), lambda i: nums[i] <= last)]

def search_rotated(nums, target):
    n, last = len(nums), nums[-1]
    pivot = first_true(0, n, lambda i: nums[i] <= last)    # index of the minimum
    lo, hi = (pivot, n) if target <= last else (0, pivot)
    i = bisect_left(nums, target, lo, hi)                   # searches only nums[lo:hi]
    return i if i < hi and nums[i] == target else -1

nums = [4, 5, 6, 7, 0, 1, 2]
find_min(nums), find_min([1, 2, 3]), find_min([2, 1])     # -> (0, 1, 1)
search_rotated(nums, 0), search_rotated(nums, 5)          # -> (4, 1)
search_rotated(nums, 3), search_rotated([1], 0)           # -> (-1, -1)
arrays = [[1, 3, 5, 7][k:] + [1, 3, 5, 7][:k] for k in range(4)]
all(search_rotated(a, t) == (a.index(t) if t in a else -1)
    for a in arrays for t in range(9))                    # -> True
```

NOTE

The common one-pass form finds the target in a single loop. At each step, one half of `[lo, hi]` is sorted: the left half when `nums[lo] <= nums[mid]`. Keep that half only when the target is inside its value range: `nums[lo] <= target < nums[mid]` for the left half, or `nums[mid] < target <= nums[hi]` for the right half. It has the same $O(\log n)$ time and about half the steps. With duplicate values, the test against `nums[-1]` is not always monotone: `[1, 1, 2, 1]` gives T T F T. For Find Minimum in Rotated Sorted Array II, compare `nums[mid]` with `nums[hi]`. On a tie, drop `hi` with `hi -= 1`. For Search in Rotated Sorted Array II, use the one-pass form. When `nums[lo] == nums[mid]`, you cannot tell which half is sorted, so set `lo += 1`. This is safe, because `nums[lo]` equals `nums[mid]`, which is not the target. Do this test before the sorted-half test. The two-pass form fails here, because its pivot test is not monotone. In both problems, the worst case becomes $O(n)$.

49.4.4 No sorted values: a test from local structure

What changes: the values are not sorted, but the statement still asks for $O(\log n)$ time. Binary search does not need sorted values. It needs a test that flips one time. Build the test from `mid` and its neighbor.

- A slope (Peak Index in a Mountain Array, LeetCode 852): the values go up and then down. A downhill step `a[i] > a[i + 1]` is F ... F T ... T. The peak is the first true index.
- A broken pairing (Single Element in a Sorted Array, LeetCode 540): each value occurs two times, except one value. Before the single value, each pair of indexes `(2p, 2p + 1)` holds two equal values. At and after the single value, each pair is broken. So the test `nums[2p] != nums[2p + 1]` is F ... F T ... T over the pairs `p`.

Memory hook: *no sorted values? Find a test at mid that flips one time: a downhill step, a broken pair.*

```
def first_true(lo, hi, ok):
    while lo < hi:
        mid = (lo + hi) // 2
        if ok(mid):
            hi = mid
        else:
            lo = mid + 1
    return lo

def peak_index(a):                    # a goes up, then down
    return first_true(0, len(a) - 1, lambda i: a[i] > a[i + 1])

def single_in_pairs(nums):            # sorted; each value two times except one
    p = first_true(0, len(nums) // 2, lambda p: nums[2 * p] != nums[2 * p + 1])
    return nums[2 * p]                 # the first broken pair starts with the single value

peak_index([0, 2, 5, 9, 4, 1]), peak_index([0, 10, 5, 2])    # -> (3, 1)
single_in_pairs([3, 3, 7, 7, 10, 11, 11])                    # -> 10
single_in_pairs([1, 1, 2, 3, 3, 4, 4, 8, 8])                 # -> 2
single_in_pairs([5]), single_in_pairs([1, 1, 2])             # -> (5, 2)
```

CAUTION

The XOR of all values also finds the single value, but it costs $O(n)$. It does not meet a required $O(\log n)$ time. XOR is the correct tool when the values are not sorted (Single Number).

49.4.5 Binary search on the answer: the smallest value that works

What changes: no array exists. The candidates are the possible answers. Use four steps.

1. Write `ok(x)`: can you do the task with the value `x`? It is often a sum or a greedy pass.
2. Prove that `ok` is monotone: if `x` works, `x + 1` also works.
3. Set the bounds: `lo` is the smallest possible answer, and `hi` surely works.
4. Find the first true value. The total cost is the cost of one check times $\log_2(\text{hi} - \text{lo})$.

In Koko Eating Bananas, `ok(k)` is "the hours at speed `k` are at most `h`". Each pile needs $\lceil p/k \rceil$ hours, and that value never grows when `k` grows. Speed 1 is the slowest. Speed `max(piles)` always works, because each pile then takes one hour and `h >= n`. Memory hook: *guess a speed, count the hours, and binary search for the first speed that fits.*

```
def min_eating_speed(piles, h):
    def hours_needed(k):
        return sum((p + k - 1) // k for p in piles)    # ceil(p / k) in integers
    lo, hi = 1, max(piles)        # hi always fits: one pile per hour
    while lo < hi:
        mid = (lo + hi) // 2
        if hours_needed(mid) <= h:
            hi = mid              # mid fits: it can be the answer
        else:
            lo = mid + 1          # mid is too slow
```

```
    return lo

min_eating_speed([3, 6, 7, 11], 8)          # -> 4
min_eating_speed([30, 11, 23, 4, 20], 5)    # -> 30
min_eating_speed([30, 11, 23, 4, 20], 6)    # -> 23
min_eating_speed([10**9], 10**9)            # -> 1
```

Only the check and the bounds change in the problems of this group: the capacity to ship packages in D days, the smallest largest sum of k pieces, the smallest divisor under a threshold, the first day with enough bouquets. For a "maximize the smallest" question, the check flips in the other direction. Then search for the last true value.

NOTE

For a real-valued answer, run a fixed number of steps and set `lo = mid` or `hi = mid` with no `+ 1`. Each step halves the range, but the range cannot become smaller than the gap between two neighbor floats. From a range of 10^9, about 80 steps make `lo` and `hi` neighbor floats for an answer near 1. More steps do not make the answer more exact. Thus 100 steps give an answer that is as exact as a float allows, which is enough for normal limits. A very large range needs more steps: with 100 steps, the cube root below is wrong by 58% for $v = 10^{45}$.

```
def cube_root(v):                   # v >= 0
    lo, hi = 0.0, max(1.0, v)       # the answer is in [lo, hi]
    for _ in range(100):
        mid = (lo + hi) / 2
        if mid ** 3 >= v:
            hi = mid
        else:
            lo = mid
    return hi

round(cube_root(2.0), 6), round(cube_root(27.0), 6)    # -> (1.259921, 3.0)
# 100 steps are too few for a huge range: the true ratio is 1.0
round(cube_root(1e45) / 1e15, 2)                       # -> 1.58
```

49.4.6 Binary search on the answer: the largest value that works

What changes: the test is T T T F F, so search for the last true value. Two grid problems show this shape. Both have non-negative cells, and both ask for the largest side k of a square under a sum limit.

- Maximum Side Length of a Square with Sum Less than or Equal to Threshold: *some* $k \times k$ square must fit. It is monotone because a fitting square contains a fitting smaller square.
- A variant that asks for the largest side where *every* $k \times k$ square must fit. It is monotone because a square that fails makes each bigger square around it fail.

A 2D prefix table gives each square sum in $O(1)$, so one check costs $O(RC)$. The total cost is $O(RC \log \min(R, C))$. The loop below uses the "best" form: if `mid` fits, save it and try bigger. The only difference between the two problems is `any` or `all`. Memory hooks: *prefix table for* $O(1)$ *squares, then binary search the biggest side that fits* (some square); *cells are non-negative, so if size* k *fails, every bigger size fails; binary search* k*, and check all* $k \times k$ *squares with 2D prefix sums* (every square).

```
def largest_side(grid, limit, need_all):
    rows, cols = len(grid), len(grid[0])
    pre = [[0] * (cols + 1) for _ in range(rows + 1)]     # pre[r][c] = sum of the top-left r x c block
    for r in range(rows):
        for c in range(cols):
            pre[r + 1][c + 1] = grid[r][c] + pre[r][c + 1] + pre[r + 1][c] - pre[r][c]

    def square_sum(r, c, k):            # k x k square with top-left cell (r, c)
        return pre[r + k][c + k] - pre[r][c + k] - pre[r + k][c] + pre[r][c]

    def fits(k):
        tests = (square_sum(r, c, k) <= limit
                 for r in range(rows - k + 1) for c in range(cols - k + 1))
```

```
        return all(tests) if need_all else any(tests)

    lo, hi, best = 1, min(rows, cols), 0    # best stays 0 if no side fits
    while lo <= hi:
        mid = (lo + hi) // 2
        if fits(mid):
            best = mid                  # mid fits: save it and try bigger
            lo = mid + 1
        else:
            hi = mid - 1                # mid fails: try smaller
    return best

mat = [[1, 1, 3, 2, 4, 3, 2]] * 3       # read only, so shared rows are safe here
largest_side(mat, 4, need_all=False)    # -> 2
largest_side(mat, 4, need_all=True)     # -> 1
largest_side([[2] * 5] * 5, 1, need_all=False)   # -> 0
largest_side([[5, 5, 5]], 4, need_all=True)      # -> 0
```

NOTE

Both problems also have an $O(RC)$ solution. For "some square", scan the bottom-right corners in row order and test only the side `best + 1` at each corner. For "every square", walk each diagonal with two pointers. The limits (300×300) do not need them.

49.4.7 Sorted history: the latest value at or before a time

What changes (Time Based Key-Value Store): the data groups by an exact key, and each key has a timeline. A query asks for the value in effect at time t. Split the query into two parts. A hash map finds the exact key. A sorted list per key handles the time. The set timestamps strictly increase, so `append` keeps each list sorted for free. A get is a floor query: `bisect_right(times, t)` counts the times `<= t`, and the answer is one position to the left. Memory hook: *two lists per key, times and values;* ***`bisect_right`*** *counts the times at or before* t*, so take the value one to the left.*

```
from bisect import bisect_right
from collections import defaultdict

class TimeMap:
    def __init__(self):
        self.times = defaultdict(list)     # key -> timestamps, increasing
        self.values = defaultdict(list)    # key -> values, same order as times

    def set(self, key, value, timestamp):
        self.times[key].append(timestamp)  # timestamps increase: the list stays sorted
        self.values[key].append(value)

    def get(self, key, timestamp):
        times = self.times.get(key, [])    # .get does not add a missing key
        i = bisect_right(times, timestamp) # i = count of times <= timestamp
        return self.values[key][i - 1] if i > 0 else ""

tm = TimeMap()
tm.set("foo", "bar", 1)
tm.get("foo", 1), tm.get("foo", 3)      # -> ('bar', 'bar')
tm.set("foo", "bar2", 4)
tm.get("foo", 4), tm.get("foo", 5)      # -> ('bar2', 'bar2')
tm.get("foo", 0), tm.get("baz", 9)      # -> ('', '')
"baz" in tm.times                       # -> False
```

The same pattern answers snapshot arrays, election leaders by time, and prices "as of" a date. For the earliest value at or after t, use `i = bisect_left(times, t)` and read index `i` when `i < len(times)`. Without `bisect`, write the floor search by hand with the "best" form. If the time at `mid` is `<= t`, save its value and set `lo = mid +`

1. Else set `hi = mid - 1`.

NOTE

Always check whether the input order already gives sorted data. Here it removes the need for a balanced tree. If inserts can arrive out of order, insert at `i = bisect_right(times, t)` in both lists (`bisect.insort` does this for one list). Each insert costs $O(k)$, because the list shifts. For many such inserts, use an ordered container.

49.4.8 A sorted helper array: longest increasing subsequence

What changes (Longest Increasing Subsequence): the input is not sorted. The algorithm builds a sorted array as it goes. For each chain length, keep only the smallest tail value seen so far. A smaller tail is always at least as easy to extend, so the other chains of that length do not matter. These tails stay strictly increasing. For a new value `x`, `bisect_left` finds the first tail `>= x`. Replace that tail with `x`, or append `x` when no tail is `>= x`. The length of `tails` is the answer. For `[10, 9, 2, 5, 3, 7, 101, 18]`, the tails change as follows: `[10]`, `[9]`, `[2]`, `[2, 5]`, `[2, 3]`, `[2, 3, 7]`, `[2, 3, 7, 101]`, `[2, 3, 7, 18]`. The length is 4. Memory hook: *smallest tail per length:* `bisect_left`*, then replace or append.*

```
from bisect import bisect_left, bisect_right
def length_of_lis(nums, strict=True):
    find = bisect_left if strict else bisect_right   # right lets equal values chain
    tails = []        # tails[k] = smallest last value of a chain of length k + 1
    for x in nums:
        pos = find(tails, x)
        if pos == len(tails):
            tails.append(x)       # x extends the longest chain
        else:
            tails[pos] = x        # x is a smaller tail for length pos + 1
    return len(tails)

length_of_lis([10, 9, 2, 5, 3, 7, 101, 18])     # -> 4
length_of_lis([0, 1, 0, 3, 2, 3])               # -> 4
length_of_lis([7, 7, 7, 7])                     # -> 1
length_of_lis([7, 7, 7, 7], strict=False)       # -> 4
```

CAUTION

`tails` is not a real subsequence. Only its length is correct. To rebuild the sequence, store the index of each tail and a parent index for each element.

The $O(n^2)$ DP ("the longest chain that ends at index i") is the simple fallback when n is a few thousand.

49.4.9 Partition search across two sorted arrays

What changes (Median of Two Sorted Arrays): the answer is a cut, not a value. Put the smallest half of all values on the left side of the cut. The left side has a fixed size `half = (m + n + 1) // 2`. If it takes `i` values from array `A`, it must take `j = half - i` values from array `B`. So only `i` is free. Binary search `i` over the shorter array, so that `j` always stays in range. Check a cut with the four border values. Use $-\infty$ and $+\infty$ for a border outside an array.

- `A[i - 1] > B[j]`: the cut takes too many values from `A`. Move `i` left.
- `B[j - 1] > A[i]`: the cut takes too few values from `A`. Move `i` right.
- Else the cut is correct. The median comes from the largest left border and the smallest right border.

Memory hook: *cut the shorter array so the left half is full: too many from A, go left; too few, go right; just right, read the borders.*

```
def find_median(nums1, nums2):
    A, B = nums1, nums2
    if len(A) > len(B):
        A, B = B, A                    # search the shorter array
    m, n = len(A), len(B)
    half = (m + n + 1) // 2            # size of the left side
```

```
    def at(arr, k):                     # arr[k], with -inf and +inf outside the array
        if k < 0:
            return float("-inf")
        if k >= len(arr):
            return float("inf")
        return arr[k]

    lo, hi = 0, m                       # i = values that the left side takes from A
    while lo <= hi:
        i = (lo + hi) // 2
        j = half - i                    # the rest of the left side comes from B
        if at(A, i - 1) > at(B, j):
            hi = i - 1                  # too many from A
        elif at(B, j - 1) > at(A, i):
            lo = i + 1                  # too few from A
        else:                           # just right
            left_max = max(at(A, i - 1), at(B, j - 1))
            if (m + n) % 2 == 1:
                return float(left_max)
            return (left_max + min(at(A, i), at(B, j))) / 2

find_median([1, 3], [2])                # -> 2.0
find_median([1, 2], [3, 4])             # -> 2.5
find_median([], [5]), find_median([1, 1, 1], [1, 1])    # -> (5.0, 1.0)
import statistics
pairs = [([1, 4, 9], [2, 3]), ([5, 6], [1, 2, 3, 4]), ([], [1, 2]), ([0, 0], [0, 7])]
all(find_median(a, b) == statistics.median(a + b) for a, b in pairs)    # -> True
```

The same cut finds the k-th smallest value of two sorted arrays: make the left side size k, and keep `i` between `max(0, k - n)` and `min(k, m)`. The answer is the largest left border. With more than one free count (many sorted arrays, a sorted matrix), binary search the answer value instead. The answer is the first value `v` where the count of values `<= v` is at least k.

49.5 Adapt it to a new problem

Ask these questions in this order.

1. **What are the candidates?** Indexes, answer values, sizes, times, a split count, or a flat index.
2. **What is the question for one candidate?** Write `ok(x)` in words first.
3. **Why does `ok` flip only one time?** Name the reason: sorted data, non-negative values, a bigger region that contains a smaller one, more capacity that always helps, or a local pattern at `mid` (a downhill step, a broken pair). If you cannot name a reason, compare with a brute force on small inputs.
4. **Which boundary do you need?** The smallest value that works is a first true. The largest value that works is a last true. One exact value uses the closed-range loop.
5. **What are the bounds?** `lo` is the smallest possible answer. `hi` surely works, or it is one past the end. Decide the return value when no candidate works.
6. **What does one check cost?** The total is the cost of one check times $\log_2$ of the range. Make the check cheap with prefix sums, a greedy pass, or `bisect` counts.
7. **Is one comparison at `mid` ambiguous?** Find a second fact that makes it safe (which half is sorted, which array gives too many values). Or split the problem into two searches that you know.
8. **Test the edges:** one element, two elements, the answer at each end of the range, and no answer.

49.6 Pitfalls

CAUTION

Mixed loop conventions. Each loop form has one condition and one pair of updates. A mix skips the last candidate or loops forever. With `lo = mid` and the lower midpoint, a range of two values never shrinks, because `mid == lo`.

Goal	Start	Loop	Updates
Exact match	`lo, hi = 0, n - 1`	`while lo <= hi`	`lo = mid + 1`, `hi = mid - 1`
First true	`lo, hi = 0, n` (hi is a sentinel)	`while lo < hi`, lower mid	`hi = mid`, `lo = mid + 1`
Last true, direct	`ok(lo)` is true	`while lo < hi`, upper mid	`lo = mid`, `hi = mid - 1`
Last true, best	`best` = the "none" value	`while lo <= hi`	pass: `best = mid`, `lo = mid + 1`; fail: `hi = mid - 1`

```
lo, hi = 4, 5
(lo + hi) // 2            # -> 4
(lo + hi + 1) // 2        # -> 5
```

CAUTION

A boundary index can be outside the list. `bisect_left` can return `len(a)`. Test `i < len(a)` before you read `a[i]`. A floor index can be −1, and `a[-1]` silently reads the last element. Test `i >= 0` first. In the count form `i = bisect_right(a, x)`, test `i > 0` before you read `a[i - 1]`.

```
from bisect import bisect_left, bisect_right
a = [10, 20, 30]
bisect_left(a, 99)                # -> 3
a[bisect_left(a, 99)]             # -> raises IndexError
i = bisect_right(a, 5) - 1        # no value <= 5
i, a[i]                           # -> (-1, 30)
```

CAUTION

A search on a question that is not monotone gives a wrong answer and no error. A search on the size of a region (a square, a window) needs non-negative values. With negative values, a size can fail while a bigger size passes. Prove monotonicity first, or test it against a brute force.

CAUTION

Wrong bounds. Make the smallest possible answer reachable, including 0 when nothing can fit. Start `lo` at 0 in the direct form, or start `best` at 0 in the "best" form. A speed or a divisor starts at 1, because 0 divides by zero. Do not set `hi` above the largest real candidate. For a side above `min(rows, cols)`, no square exists, so an "every square" check passes with no work.

```
all(s <= 4 for s in [])     # -> True
any(s <= 4 for s in [])     # -> False
```

CAUTION

Ceiling division. A partial pile or a partial load still costs a full unit. Floor division undercounts. Use `(p + k - 1) // k` or `-(-p // k)`. Do not use `math.ceil(p / k)` for large values, because float division loses precision above 2^{53}.

```
p, k = 7, 2
p // k, (p + k - 1) // k, -(-p // k)    # -> (3, 4, 4)
int((2**53 + 1) / 1) == 2**53 + 1       # -> False
```

CAUTION

`bisect` on tuples. A probe `(t,)` sorts before `(t, "x")`, so `bisect_right` misses an exact match. Search a separate list of times, or pass `key=` [Python 3.10+].

```python
from bisect import bisect_right
pairs = [(1, "a"), (4, "b")]
pairs[bisect_right(pairs, (4,)) - 1]                    # -> (1, 'a')
pairs[bisect_right(pairs, 4, key=lambda p: p[0]) - 1]   # -> (4, 'b')
```

- **Strict or non-strict.** `bisect_left` keeps equal values apart (strictly increasing). `bisect_right` lets them chain (non-decreasing).
- **"Some" or "every".** "Some square fits" and "every square fits" are two different checks. Read the statement again before you write `any` or `all`.
- **The wrong divisor in a flat index.** Divide by the number of columns, not by the number of rows.
- **A table with shared rows.** `[[0] * (cols + 1)] * (rows + 1)` makes every row the same list. Build a prefix table with `[[0] * (cols + 1) for _ in range(rows + 1)]`.
- **A read that adds a key.** `d[key]` on a `defaultdict` adds an empty entry. Use `d.get(key, [])` in a query.
- **Integer division of a median.** Use `/ 2` for the average of two middle values, not `// 2`.
- **A scan that meets the limits but breaks the rule.** A merge or a linear scan can pass the tests, but it does not meet a required $O(\log n)$ time.

49.7 Problems in this family

Problem	What changes	Memory hook
Binary Search	Exact match on a closed range; return -1 when the range is empty.	Closed range, check the middle, drop the half that cannot hold the target.
Search a 2D Matrix	The grid is one sorted list; convert the flat index with `divmod(k, cols)`.	The grid is one sorted list cut into rows, so binary search the flat index and divmod by cols.
Koko Eating Bananas	Search the answer (the speed); the check sums $\lceil p/k \rceil$ hours.	Guess a speed, count the hours, and binary search for the first speed that fits.
Find Minimum in Rotated Sorted Array	First index with `nums[i] <= nums[-1]`.	Low run means at most the last value; find the first one.
Search in Rotated Sorted Array	Find the drop, then a lower bound in the one run that can hold the target.	Find the drop, then binary search the side that can hold the target.
Time Based Key-Value Store	Floor query per key: `bisect_right` on the times, then one to the left.	Two lists per key, times and values; `bisect_right` counts the times at or before t, so take the value one to the left.
Median of Two Sorted Arrays	Search the cut in the shorter array; check four border values.	Cut the shorter array so the left half is full: too many from A, go left; too few, go right; just right, read the borders.
Maximum Side Length of a Square with Sum Less than or Equal to Threshold	Last true over the side; *some* square must fit; 2D prefix sums for the check.	Prefix table for $O(1)$ squares, then binary search the biggest side that fits.
Longest Increasing Subsequence	The sorted array is the `tails` list that the loop builds.	Smallest tail per length: `bisect_left`, then replace or append.

The transfer drills for this technique are in Section 69.7 (page 420). The answers are in Section A.6 (page 438).

50 Linked lists

50.1 The idea

A singly linked list is a chain of nodes. Each node holds a value and one link, `next`, to the node after it. The list has no index, no stored length and no link back. In this section, a *link* is a field of a node that refers to another node: `next`, and in some problems `prev` or `random`. A *pointer* is a variable that refers to a node. A list algorithm is a plan of link changes: which links change, and in which order. The values do not move.

One rule controls all list code: a node that no pointer and no link refers to is lost. Before you change a link, put a pointer on the node at its other end. This rule explains the saved `nxt` in a reversal and the saved `after` in a block reversal. It also explains the dummy node. The link `dummy.next` always refers to the current first node, even when the first node changes.

Worked example. Reverse `1 -> 2 -> 3`. Start with `prev = None` and `curr` on node 1. Save `nxt` on node 2. Point the link of node 1 at `prev`. Step: `prev` is now node 1 and `curr` is node 2. Do the same for node 2, then for node 3. Now `curr` is `None` and `prev` is node 3, the head of `3 -> 2 -> 1`. Each node changes one time, so the time is $O(n)$ and the extra memory is $O(1)$.

What the techniques save. The brute force copies the nodes into a Python list. That gives index access, negative indexes and access from both ends, for $O(n)$ extra memory. This copy is often the fastest correct solution to write, and the constraints often allow it. The list techniques remove that memory. They also remove a node from the middle in $O(1)$. A singly linked list needs a pointer on the predecessor. A doubly linked list needs only a pointer on the node itself. An array cannot do that: a removal from the middle shifts all later items, $O(n)$.

Five moves. Almost every list problem is a composition of two or three of these moves:

- **Walk**: `while node: ... node = node.next`.
- **Build**: a dummy node in front, and a tail pointer on the last node of the output.
- **Reverse**: `prev`, `curr` and a saved `nxt`.
- **Find a position**: fast and slow pointers (the middle, a cycle), or two pointers with a fixed gap (the k-th node from the end).
- **Map**: a dict keyed by node object (a deep copy), or a dict from key to node (an LRU cache).

50.2 Recognize it

- The input or the output is a chain of nodes. The statement says "in place", "do not change the values" or "$O(1)$ extra memory". Plan the link changes.
- The first node of the output can change: removal of the head, insertion at the front, a reversal, a merge. Put a dummy node in front of the head.
- The task needs a position from the end, the middle, or both ends at the same time. Use fast and slow pointers, a fixed gap, or reverse one half. If memory is free, copy the nodes into a Python list.
- The task changes a block or a range of the list: reverse in groups, swap pairs, rotate. Name four nodes: `before`, `first`, `last` and `after`.
- Two or more inputs are sorted, and the output must be sorted. Merge the fronts (the first unused node of each input): compare two fronts, or keep k fronts in a min-heap.
- A number is stored as a chain of digits. Add column by column with a carry.
- The task asks for a deep copy of nodes with extra links (random, child) that can point to any node. Map each old node to its new node.
- One deterministic step repeats on a finite set of states: "does it loop", "where does the loop start". Use fast and slow pointers. An array whose values are valid indexes into the same array is such a walk: index `i` links to index `nums[i]`.
- Every operation must be $O(1)$, and you need lookup by key plus an order that changes (most recent, oldest). The statement asks you to design a class with methods such as `get` and `put`. Use a hash map from key to node and a doubly linked list.
- The constraint sizes. A list has at most about 10^5 nodes. An $O(n)$ copy into a Python list fits, unless the statement or a follow-up asks for $O(1)$ extra memory. Python stops recursion near depth 1000. When the list can be longer, use loops, not one recursive call per node.

50.3 Core template

Most examples in this section start with the same three helpers: a node class, `build` (Python list to linked list) and `to_list` (linked list to Python list). Each block repeats them, because each block runs alone.

Reverse. This is the base move for all link rewiring.

```
class ListNode:
    def __init__(self, val=0, next=None):
        self.val, self.next = val, next
def build(vals):
    head = None
    for v in reversed(vals):
        head = ListNode(v, head)
    return head
def to_list(head):
    out = []
    while head:
        out.append(head.val)
        head = head.next
    return out

def reverse(head):
    prev, curr = None, head          # prev: reversed part, curr: the rest
    while curr:
        nxt = curr.next              # 1. save the rest
        curr.next = prev             # 2. flip the link
        prev = curr                  # 3. step prev
        curr = nxt                   # 4. step curr
    return prev                      # curr is None; prev is the old tail

to_list(reverse(build([1, 2, 3])))   # -> [3, 2, 1]
to_list(reverse(build([7])))         # -> [7]
reverse(None) is None                # -> True
```

Invariant: `prev` is the head of a reversed list of all visited nodes, and `curr` is the head of the untouched rest. The two lists are disjoint. Each step moves one node from the front of the rest to the front of the reversed part.

Build with a dummy and a tail. This is the base move for every output list. The example keeps the nodes that pass a test and splices them into a new chain.

```
class ListNode:
    def __init__(self, val=0, next=None):
        self.val, self.next = val, next
def build(vals):
    head = None
    for v in reversed(vals):
        head = ListNode(v, head)
    return head
def to_list(head):
    out = []
    while head:
        out.append(head.val)
        head = head.next
    return out

def keep(head, test):
    dummy = tail = ListNode()        # tail: the last node of the output
    while head:
        if test(head.val):
            tail.next = head         # link the node
            tail = head              # move the tail
        head = head.next
```

```
    tail.next = None                # end the output; cut the old links
    return dummy.next               # the head can change

to_list(keep(build([5, 1, 6, 2, 7]), lambda v: v < 3))   # -> [1, 2]
keep(build([5, 6]), lambda v: v < 3) is None             # -> True
```

Invariant: the chain from `dummy.next` to `tail` is the finished output. The link of `tail` is not part of the output until the end, so set it to `None` after the loop. The dummy removes all special code for the first node and for an empty output.

Escape to an array. When $O(n)$ extra memory is allowed, copy the node references into a Python list. Write the new order with indexes, then relink the nodes in that order. This handles any reorder that you can write as a list of indexes.

```
class ListNode:
    def __init__(self, val=0, next=None):
        self.val, self.next = val, next
def build(vals):
    head = None
    for v in reversed(vals):
        head = ListNode(v, head)
    return head
def to_list(head):
    out = []
    while head:
        out.append(head.val)
        head = head.next
    return out

def to_nodes(head):
    nodes = []
    while head:
        nodes.append(head)
        head = head.next
    return nodes
def relink(nodes):                  # chain the nodes in this order
    for a, b in zip(nodes, nodes[1:]):
        a.next = b
    if not nodes:
        return None
    nodes[-1].next = None           # end the list; no old link stays
    return nodes[0]

nodes = to_nodes(build([1, 2, 3, 4, 5]))
to_list(relink(nodes[1::2] + nodes[::2]))   # -> [2, 4, 1, 3, 5]
```

50.4 Variations

50.4.1 Reverse a whole list

To reverse a whole list, use the core template with no change (Reverse Linked List). Memory hook: *Save next, flip the arrow, step both, return prev.* Use the loop, not recursion. A recursive reversal goes one call deep per node, and Python stops near depth 1000. The same loop is a subroutine in harder problems. Reverse one half to read it backward (Reorder List). Reverse a block (Reverse Nodes in k-Group). Reverse digit lists to add from the low end (Add Two Numbers II).

50.4.2 Merge two sorted lists

Merge two sorted lists into one sorted list (Merge Two Sorted Lists). The *front* of an input is its first node that is not in the output yet. Each input is sorted, so the smallest remaining value is at one of the two fronts. Compare the two fronts, link the smaller one after the tail, and move the tail. When one input is empty, link the rest of the

other input in one step. Memory hook: *Dummy and tail. Take the smaller head, move the tail. Glue the rest.*

```
class ListNode:
    def __init__(self, val=0, next=None):
        self.val, self.next = val, next
def build(vals):
    head = None
    for v in reversed(vals):
        head = ListNode(v, head)
    return head
def to_list(head):
    out = []
    while head:
        out.append(head.val)
        head = head.next
    return out

def merge(a, b):
    dummy = tail = ListNode()
    while a and b:
        if a.val <= b.val:           # <= takes the node of a first on a tie
            tail.next = a
            a = a.next
        else:
            tail.next = b
            b = b.next
        tail = tail.next
    tail.next = a or b               # glue the rest (or None)
    return dummy.next

to_list(merge(build([1, 2, 4]), build([1, 3, 4])))   # -> [1, 1, 2, 3, 4, 4]
to_list(merge(None, build([0])))                     # -> [0]
merge(None, None) is None                            # -> True
```

The same two-front comparison merges sorted arrays and gives the intersection or union of two sorted sequences. It is also the merge step of merge sort on a linked list.

50.4.3 Digit lists: add with a carry

Two numbers are stored as lists of digits, and the output is their sum as a new list (Add Two Numbers). Here the digits come least significant first, which is the order of column addition. Walk both lists together. Use 0 for a list that has ended. Write the total mod 10 and carry the total div 10. Keep the loop alive while a carry is left, so a final carry becomes a new node. Memory hook: *Add digits plus carry while anything is left; write mod 10, carry div 10.*

```
class ListNode:
    def __init__(self, val=0, next=None):
        self.val, self.next = val, next
def build(vals):
    head = None
    for v in reversed(vals):
        head = ListNode(v, head)
    return head
def to_list(head):
    out = []
    while head:
        out.append(head.val)
        head = head.next
    return out

def add(l1, l2):
    dummy = tail = ListNode()
```

```
    carry = 0
    while l1 or l2 or carry:          # a final carry makes one more node
        total = carry
        if l1:
            total += l1.val
            l1 = l1.next
        if l2:
            total += l2.val
            l2 = l2.next
        carry, digit = divmod(total, 10)
        tail.next = ListNode(digit)
        tail = tail.next
    return dummy.next

to_list(add(build([2, 4, 3]), build([5, 6, 4])))   # -> [7, 0, 8]
to_list(add(build([9, 9]), build([1])))            # -> [0, 0, 1]
```

The carry is 0 or 1, because the largest column total is $9 + 9 + 1 = 19$. When the digits come most significant first, reverse both lists first, or push the digits on two stacks.

50.4.4 Both ends at once: an array of nodes, or split, reverse and weave

The new order is first, last, second, second to last, and so on (Reorder List). Node `i` pairs with node `n - 1 - i`. A singly linked list reads only forward, so the task needs access from the back. The intuitive solution copies the nodes into a Python list. Two indexes move inward and give the order. Then chain the nodes and end the last one with `None`. Memory hook: *Put the nodes in an array, take front then back, chain them, end with None.*

```
class ListNode:
    def __init__(self, val=0, next=None):
        self.val, self.next = val, next
def build(vals):
    head = None
    for v in reversed(vals):
        head = ListNode(v, head)
    return head
def to_list(head):
    out = []
    while head:
        out.append(head.val)
        head = head.next
    return out

def reorder(head):
    nodes = []
    node = head
    while node:
        nodes.append(node)
        node = node.next
    order = []
    left, right = 0, len(nodes) - 1
    while left <= right:
        order.append(nodes[left])
        if left < right:              # the middle node of an odd list goes in once
            order.append(nodes[right])
        left += 1
        right -= 1
    for a, b in zip(order, order[1:]):
        a.next = b
    order[-1].next = None             # end the list, or it has a cycle

head = build([1, 2, 3, 4, 5])
reorder(head)
```

```
to_list(head)                          # -> [1, 5, 2, 4, 3]
head = build([1, 2, 3, 4])
reorder(head)
to_list(head)                          # -> [1, 4, 2, 3]
```

The $O(1)$ memory form composes three moves. Find the last node of the front half with fast and slow pointers. Cut the list there and reverse the back half, so the back reads forward. Then weave the two halves. The same split-and-reverse answers any mirror-pair question on a list: compare the pairs, add them, or weave them.

```
class ListNode:
    def __init__(self, val=0, next=None):
        self.val, self.next = val, next
def build(vals):
    head = None
    for v in reversed(vals):
        head = ListNode(v, head)
    return head
def to_list(head):
    out = []
    while head:
        out.append(head.val)
        head = head.next
    return out
def reverse(head):
    prev = None
    while head:
        nxt = head.next
        head.next = prev
        prev, head = head, nxt
    return prev

def middle(head):                      # the left middle for an even length
    slow, fast = head, head.next
    while fast and fast.next:
        slow = slow.next
        fast = fast.next.next
    return slow
def reorder(head):
    mid = middle(head)
    back = reverse(mid.next)           # the back half reads forward now
    mid.next = None                    # cut: the front half ends here
    front = head
    while back:                        # the back half is never longer
        f_next, b_next = front.next, back.next
        front.next = back
        back.next = f_next
        front, back = f_next, b_next

middle(build([1, 2, 3, 4, 5])).val   # -> 3
middle(build([1, 2, 3, 4])).val      # -> 2
head = build([1, 2, 3, 4, 5, 6])
reorder(head)
to_list(head)                          # -> [1, 6, 2, 5, 3, 4]
```

With `slow, fast = head, head.next`, `slow` stops on node `ceil(n/2) - 1`, the last node of the front half. The front half is then never shorter than the back half. With `slow = fast = head`, `slow` stops on the right middle.

50.4.5 A position from the end: line up the nodes, or keep a gap

The task names a node by its position from the end, for example to delete it (Remove Nth Node From End of List). To delete a node, you need its predecessor. The head has no predecessor, so put a dummy in front. The intuitive solution lines up all nodes, dummy included, in a Python list. Then `nodes[-n]` is the target and `nodes[-n - 1]`

is its predecessor. Memory hook: *Line up the nodes behind a dummy, then* `nodes[-n - 1]` *skips its next.*

```
class ListNode:
    def __init__(self, val=0, next=None):
        self.val, self.next = val, next
def build(vals):
    head = None
    for v in reversed(vals):
        head = ListNode(v, head)
    return head
def to_list(head):
    out = []
    while head:
        out.append(head.val)
        head = head.next
    return out

def remove_nth_from_end(head, n):
    dummy = ListNode(0, head)        # the head gets a predecessor
    nodes = []
    node = dummy
    while node:
        nodes.append(node)
        node = node.next
    prev = nodes[-n - 1]             # nodes[-n] is the target
    prev.next = prev.next.next
    return dummy.next

def remove_with_gap(head, n):        # one pass, O(1) memory
    dummy = ListNode(0, head)
    slow = fast = dummy
    for _ in range(n):               # fast is n nodes ahead of slow
        fast = fast.next
    while fast.next:                 # stop when fast is on the last node
        slow = slow.next
        fast = fast.next
    slow.next = slow.next.next       # slow is the predecessor
    return dummy.next

to_list(remove_nth_from_end(build([1, 2, 3, 4, 5]), 2))   # -> [1, 2, 3, 5]
remove_nth_from_end(build([1]), 1) is None                # -> True
to_list(remove_with_gap(build([1, 2, 3, 4, 5]), 2))       # -> [1, 2, 3, 5]
to_list(remove_with_gap(build([1, 2]), 2))                # -> [2]
```

The optimal form keeps a gap of n nodes between two pointers. When the lead pointer is on the last node, the trail pointer is on the predecessor. To land on the target itself, run until the lead pointer is `None`. The same gap finds the k-th node from the end and the cut point to rotate a list.

50.4.6 Reverse a block in place: before, first, last, after

The task reverses each block of k nodes, and a short last block stays as it is (Reverse Nodes in k-Group). Each full block is a plain reversal with two splice points. Probe k steps to find `last`. If the probe reaches `None`, the block is short: stop and change nothing. Save `first` and `after`. Reverse the block with `prev` starting at `after`, so the reversed block is already attached at its tail. Then link `before` to `last`, and move `before` to `first`, the new tail of the block. Memory hook: *Before, first, last, after: probe k, reverse with prev = after, link before to last, step to first.*

```
class ListNode:
    def __init__(self, val=0, next=None):
        self.val, self.next = val, next
def build(vals):
    head = None
```

```
    for v in reversed(vals):
        head = ListNode(v, head)
    return head
def to_list(head):
    out = []
    while head:
        out.append(head.val)
        head = head.next
    return out

def reverse_k_group(head, k):
    dummy = ListNode(0, head)
    before = dummy                    # the tail of the finished part
    while True:
        last = before
        for _ in range(k):            # 1. probe: is there a full block?
            last = last.next
            if last is None:
                return dummy.next     # a short block stays as it is
        first, after = before.next, last.next
        prev, curr = after, first     # 2. reverse; first ends up linked to after
        while curr is not after:
            nxt = curr.next
            curr.next = prev
            prev = curr
            curr = nxt
        before.next = last            # 3. splice the block in
        before = first                # first is the tail of the block now

to_list(reverse_k_group(build([1, 2, 3, 4, 5]), 2))   # -> [2, 1, 4, 3, 5]
to_list(reverse_k_group(build([1, 2, 3, 4, 5]), 3))   # -> [3, 2, 1, 4, 5]
to_list(reverse_k_group(build([1, 2]), 1))            # -> [1, 2]
```

The four names fit most changes to a contiguous part of a list: reverse a range, swap pairs ($k = 2$), or move a block (rotate). Save the outside neighbors first, change the inside, then splice. Start an accumulator with the value that the boundary must have at the end. Here, `prev = after` makes the tail link correct with no extra step.

50.4.7 Merge many sorted lists: a heap of fronts

Merge k sorted lists into one sorted list (Merge k Sorted Lists). The next output node is the smallest of the current fronts. Keep only the fronts in a min-heap. Pop the smallest front, link it after the tail, and push its successor. Each node costs one pop and at most one push, $O(\log k)$. With the $O(k)$ heap build, the total is $O(k + N \log k)$ for N nodes. Memory hook: *Heap the fronts; pop the smallest, push its next.*

```
import heapq
class ListNode:
    def __init__(self, val=0, next=None):
        self.val, self.next = val, next
def build(vals):
    head = None
    for v in reversed(vals):
        head = ListNode(v, head)
    return head
def to_list(head):
    out = []
    while head:
        out.append(head.val)
        head = head.next
    return out

def merge_k(lists):
    heap = [(node.val, i, node) for i, node in enumerate(lists) if node]
```

```
    heapq.heapify(heap)              # i breaks ties, so nodes never compare
    dummy = tail = ListNode()
    while heap:
        _, i, node = heapq.heappop(heap)
        tail.next = node
        tail = node
        if node.next:
            heapq.heappush(heap, (node.next.val, i, node.next))
    return dummy.next

to_list(merge_k([build([1, 4, 5]), build([1, 3, 4]), build([2, 6])]))   # -> [1, 1, 2, 3, 4, 4, 5, 6]
merge_k([]) is None, merge_k([None]) is None    # -> (True, True)
heapq.heappush([(1, ListNode())], (1, ListNode()))   # -> raises TypeError
```

The last line shows the reason for the index: two equal values make Python compare the nodes, and nodes have no order. The list index is unique in the heap, because the heap holds at most one front per list. An alternative merges the lists in pairs, round after round. There are $\log k$ rounds of N nodes each. Do not merge the lists one by one into a growing result. That re-reads the result and costs $O(Nk)$.

50.4.8 Copy a list with extra links: map old node to new node

The task asks for a deep copy of a list (Copy List with Random Pointer). Each node has an extra link that can point to any node. A `random` link can point forward, so the copy of its target may not exist yet. Use two passes. Pass 1 makes a copy of every node and stores old node to new node in a dict. Pass 2 sets each `next` and `random` of the copies through the dict. Key the dict by the node object, because values can repeat. The entry `{None: None}` copies a missing link as `None`, with no special case. Memory hook: *Make every copy first, then link copies through the old-to-new map.*

```
class Node:
    def __init__(self, val, next=None, random=None):
        self.val, self.next, self.random = val, next, random

def copy_random_list(head):
    old_to_new = {None: None}        # a missing link maps to None
    cur = head
    while cur:                       # pass 1: make every copy
        old_to_new[cur] = Node(cur.val)
        cur = cur.next
    cur = head
    while cur:                       # pass 2: link the copies
        copy = old_to_new[cur]
        copy.next = old_to_new[cur.next]
        copy.random = old_to_new[cur.random]
        cur = cur.next
    return old_to_new[head]

a, b, c = Node(7), Node(7), Node(5)  # equal values: key by node, not by value
a.next, b.next = b, c
a.random, b.random, c.random = c, a, c
h = copy_random_list(a)
(h.val, h.next.val, h.next.next.val)             # -> (7, 7, 5)
h.random is h.next.next, h.next.random is h      # -> (True, True)
h is not a and h.random is not c                 # -> True
copy_random_list(None) is None                   # -> True
```

This is the pattern to clone any structure with arbitrary references: graphs, trees with parent links, objects with cycles. In a DFS or BFS clone, put the new node in the map before you visit its neighbors, so a cycle ends.

The $O(1)$ memory form stores the map inside the list. Insert each copy right after its original, so `orig.next` is the copy. Set `copy.random = orig.random.next` when `orig.random` is not `None`. Then split the two lists and restore the original.

The general rule: when you need an extra map and the structure has a free slot or a change that you can undo, store the map inside the structure. Undo the change at the end. Index marking on arrays uses the same idea: to record value `x`, negate `nums[abs(x) - 1]`. Morris traversal of a tree also uses it: a free right link points back to an ancestor for a short time.

50.4.9 Detect a cycle: fast and slow pointers

The task asks if a list loops back on itself (Linked List Cycle). A visited set of nodes works with $O(n)$ memory. For $O(1)$ memory, move `slow` one node and `fast` two nodes per step. Without a cycle, `fast` reaches the end. With a cycle, both pointers enter it. Inside the cycle, `fast` gains exactly one node per step, so the gap counts down to 0 and cannot skip it. Memory hook (`fast` is the hare, `slow` is the tortoise): *Hare runs two, tortoise runs one. If they meet, it loops. If the hare falls off the end, it does not.*

```
class ListNode:
    def __init__(self, val=0, next=None):
        self.val, self.next = val, next
def build(vals):
    head = None
    for v in reversed(vals):
        head = ListNode(v, head)
    return head

def has_cycle(head):
    slow = fast = head
    while fast and fast.next:        # fast can take two steps
        slow = slow.next
        fast = fast.next.next
        if slow is fast:             # the same node object: a loop
            return True
    return False                     # fast fell off the end

head = build([3, 2, 0, -4])
has_cycle(head)                      # -> False
head.next.next.next.next = head.next # the tail links back to node 1
has_cycle(head)                      # -> True
has_cycle(None)                      # -> False
```

Compare after the move. Both pointers start on the head, so a test before the first move is always true. The meeting comes within n steps, so the time is $O(n)$.

The general rule: when two walkers move on one cycle, reason about their relative speed. A relative speed of 1 shrinks the gap by 1 per step, so the walkers meet within one lap. A relative speed of 2 or more can jump over the slow walker. From some start positions, the walkers never meet. For example, on a cycle of 4 nodes with a gap of 1, a relative speed of 2 gives the gaps 1, 3, 1, 3, and so on. Keep the speeds 1 and 2.

Fast and slow pointers work on any sequence that applies one deterministic step to a finite set of states. An example is the sequence of digit-square sums in Happy Number.

50.4.10 An array as a linked list: the cycle entrance

An array has $n + 1$ values in the range $1..n$, so one value repeats (Find the Duplicate Number). The array is read-only, and the extra memory must be $O(1)$. Sorting and a set are not allowed. Read the array as a linked list: index `i` links to index `nums[i]`. Every value is a valid index. No value is 0, so nothing links to index 0, and the walk from index 0 starts outside any cycle. A walk on a finite set of indexes must enter a cycle. The cycle entrance has two incoming links: one from the path and one from inside the cycle. Two indexes that hold the same value make those two links, so the entrance is the duplicate value. Memory hook: *Index points to value: tortoise and hare from 0, then reset one to 0, walk together, and meet at the duplicate.*

```python
def find_duplicate(nums):
    slow, fast = nums[0], nums[nums[0]]   # both after their first move
    while slow != fast:                   # phase 1: meet inside the cycle
        slow = nums[slow]
        fast = nums[nums[fast]]
    start = 0
    while start != slow:                  # phase 2: meet at the cycle entrance
        start = nums[start]
        slow = nums[slow]
    return slow

find_duplicate([1, 3, 4, 2, 2])      # -> 2
find_duplicate([3, 1, 3, 4, 2])      # -> 3
find_duplicate([3, 3, 3, 3, 3])      # -> 3
```

Why phase 2 works: let μ be the path length before the cycle and L the cycle length. At the meeting, `slow` walked m steps and `fast` walked $2m$ steps. Both stand on the same node. The extra m steps of `fast` are whole laps of the cycle, so m is a multiple of L. After μ more steps, `slow` has walked $\mu + m$ steps: the path, then whole laps. So it stands on the entrance. A pointer from the start also reaches the entrance after μ steps. The two pointers meet there. The same two phases find the start of the cycle in a real linked list. The first meeting point is not the answer. Phase 2 is necessary.

NOTE

Other $O(1)$ memory route: binary search on the value. If more than `mid` elements are `<= mid`, the duplicate is in `1..mid`. This is $O(n \log n)$. A sum or XOR formula fails, because the duplicate can appear more than two times.

50.4.11 Fast lookup plus fast reorder: hash map and doubly linked list

A cache needs `get` and `put` in $O(1)$ and an order by last use (LRU Cache). When the cache is full, it evicts the least recently used key. A dict gives lookup but no order. A Python list gives order, but a move from the middle is $O(n)$. In Python, `collections.OrderedDict` is a dict plus a doubly linked list. It keeps the oldest key first. On every use, move the key to the end. On overflow, pop the first key. Memory hook: *Ordered dict: move to end on use, pop the front on overflow.*

```python
from collections import OrderedDict
class LRUCache:
    def __init__(self, capacity):
        self.capacity = capacity
        self.cache = OrderedDict()   # the oldest key first, the newest key last
    def get(self, key):
        if key not in self.cache:
            return -1
        self.cache.move_to_end(key)  # a get is a use
        return self.cache[key]
    def put(self, key, value):
        self.cache[key] = value
        self.cache.move_to_end(key)  # a put is a use
        if len(self.cache) > self.capacity:
            self.cache.popitem(last=False)   # drop the oldest key

lru = LRUCache(2)
lru.put(1, 1)
lru.put(2, 2)
lru.get(1)                           # -> 1
lru.put(3, 3)                        # evicts key 2
lru.get(2)                           # -> -1
lru.put(4, 4)                        # evicts key 1
(lru.get(1), lru.get(3), lru.get(4))   # -> (-1, 3, 4)
```

Build the structure by hand when `OrderedDict` is not allowed, or when the task asks for the list itself. The dict maps key to node. The doubly linked list keeps the nodes from least recent (left) to most recent (right). Two sentinel nodes at the ends remove all checks for an empty list. Each node stores its key, so an eviction can delete its dict entry. Hook for this form: *map finds the node, list keeps the order; remove then insert at the right; evict `left.next`.*

```
class Node:
    def __init__(self, key=0, val=0):
        self.key, self.val = key, val    # the key lets an eviction clean the dict
        self.prev = self.next = None
class LRUCache:
    def __init__(self, capacity):
        self.cap, self.map = capacity, {}          # key: node
        self.left, self.right = Node(), Node()     # sentinels: LRU end, MRU end
        self.left.next, self.right.prev = self.right, self.left
    def _remove(self, node):
        prev, nxt = node.prev, node.next
        prev.next, nxt.prev = nxt, prev
    def _insert(self, node):                       # just before the right sentinel
        prev, nxt = self.right.prev, self.right
        prev.next = nxt.prev = node
        node.prev, node.next = prev, nxt
    def get(self, key):
        if key not in self.map:
            return -1
        node = self.map[key]
        self._remove(node)
        self._insert(node)
        return node.val
    def put(self, key, value):
        if key in self.map:
            self._remove(self.map[key])            # no old node stays in the list
        node = Node(key, value)
        self.map[key] = node
        self._insert(node)
        if len(self.map) > self.cap:
            lru = self.left.next
            self._remove(lru)
            del self.map[lru.key]

lru = LRUCache(2)
lru.put(1, 1)
lru.put(2, 2)
lru.get(1)                          # -> 1
lru.put(3, 3)
lru.put(1, 10)                      # an existing key: update, no eviction
(lru.get(2), lru.get(3), lru.get(1))   # -> (-1, 3, 10)
```

NOTE

A plain dict also keeps insertion order, and `next(iter(d))` gives the oldest key. But many deletions at the front make that call slow in CPython. Use `OrderedDict` or the hand-built list.

The general rule: when one structure gives only half of the required operations, combine two structures that point into each other. Each item must carry enough data to find itself in the other structure.

50.5 Adapt it to a new problem

Ask these questions in order.

1. Draw the target for a list of 5 or 6 nodes. Mark each link that changes.
2. Is $O(n)$ extra memory allowed? If yes, copy the nodes into a Python list, write the new order with indexes, and

relink. End the last node with `None`.

3. Can the head change? If yes, start with a dummy node and return `dummy.next`.
4. Which nodes do you need after a link change? Name them (`nxt`, `before`, `after`, the tail) and save them first.
5. Do you need a position? Use a 2-to-1 speed for the middle and a fixed gap for the k-th node from the end. To delete or insert, stop on the predecessor.
6. Do you need to read part of the list backward? Reverse that part in place. Reverse it again if the caller needs the original.
7. Are there several sorted inputs and one sorted output? Merge the fronts. Compare two fronts, or keep k fronts in a min-heap.
8. Can a link point to any node (random, child, graph edge)? Map old node to new node. Make all nodes first, link them second.
9. Does one deterministic step repeat on a finite set? Expect a cycle. Use fast and slow pointers for $O(1)$ memory, a visited set otherwise.
10. Must every operation be $O(1)$ by key in an order that changes? Combine a dict with a doubly linked list.
11. Test the empty list, one node, two nodes, odd and even lengths, and a change of the head. Check the last node of the result: its link must be `None`.

50.6 Pitfalls

CAUTION

A lost rest of the list. Save `curr.next` before you overwrite it. After the change, nothing refers to the rest.

CAUTION

Tuple assignment order. Python evaluates the right side first, then assigns the targets from left to right. If `curr` changes before `curr.next` is assigned, the write goes to the wrong node. A temporary variable is safer.

```
class ListNode:
    def __init__(self, val=0, next=None):
        self.val, self.next = val, next
def build(vals):
    head = None
    for v in reversed(vals):
        head = ListNode(v, head)
    return head
def to_list(head):
    out = []
    while head:
        out.append(head.val)
        head = head.next
    return out

def reverse_ok(head):
    prev, curr = None, head
    while curr:
        curr.next, prev, curr = prev, curr, curr.next
    return prev
def reverse_bad(head):
    prev, curr = None, head
    while curr:
        prev, curr, curr.next = curr, curr.next, prev   # curr moves first
    return prev

to_list(reverse_ok(build([1, 2, 3])))   # -> [3, 2, 1]
reverse_bad(build([1, 2, 3]))           # -> raises AttributeError
```

CAUTION

The wrong return value. Return `prev` after a reversal, not `head`: the old head is now the tail. Return `dummy.next`, not `dummy` (an extra 0 node) and not the old `head` (it can be removed or moved).

CAUTION

A tail that does not move. After `tail.next = node`, also move `tail`. Otherwise each link overwrites the same field.

CAUTION

A cycle in the output. The last node of a rebuilt list keeps its old link. Set it to `None`. After a split at the middle, cut the front half. Without the cut, the result has a cycle.

CAUTION

A read of `next` on `None`. Test `fast and fast.next` before you move `fast` two nodes. Treat an ended input as 0 or as empty before you read `.val`. In a block reversal, probe for k nodes before you change a link.

CAUTION

Values instead of nodes. Compare nodes with `is`. Key a map by the node object. Equal values do not mean the same node.

CAUTION

Recursion on a long list. A recursive reversal or merge goes one call deep per node. Python stops near depth 1000.

```
class ListNode:
    def __init__(self, val=0, next=None):
        self.val, self.next = val, next
def build(vals):
    head = None
    for v in reversed(vals):
        head = ListNode(v, head)
    return head

def reverse_rec(head):
    if head is None or head.next is None:
        return head
    new_head = reverse_rec(head.next)
    head.next.next = head
    head.next = None
    return new_head

reverse_rec(build([1, 2, 3])).val          # -> 3
reverse_rec(build(list(range(5000))))      # -> raises RecursionError
```

CAUTION

Heap entries without a tie-breaker. Push `(val, i, node)`, not `(val, node)`. Equal values make Python compare nodes, which raises `TypeError`.

CAUTION

Floyd details. Move first, then compare. The first meeting point is not the cycle entrance: run phase 2, with both pointers at one step. For an array walk, start at an index that nothing links to (index 0 when no value is 0).

CAUTION

LRU details. A successful `get` is a use. A `put` on an existing key is a use and must not evict. Store the key in each node. Test membership with `in`, not with the truth value of a stored value, because 0 is a legal value.

50.7 Problems in this family

Problem	What changes	Memory hook
Reverse Linked List	The core template: flip every link.	Save next, flip the arrow, step both, return prev.
Merge Two Sorted Lists	Build with a dummy and a tail; compare two fronts.	Dummy and tail. Take the smaller head, move the tail. Glue the rest.
Reorder List	Both ends at once: array of nodes (or middle, reverse, weave).	Put the nodes in an array, take front then back, chain them, end with None.
Remove Nth Node From End of List	A position from the end; stop on the predecessor; dummy for the head.	Line up the nodes behind a dummy, then `nodes[-n - 1]` skips its next.
Copy List with Random Pointer	Pointers go anywhere: two passes and a map keyed by node.	Make every copy first, then link copies through the old-to-new map.
Add Two Numbers	Build new nodes; column addition with a carry.	Add digits plus carry while anything is left; write mod 10, carry div 10.
Linked List Cycle	Fast and slow pointers; meet means loop.	Hare runs two, tortoise runs one. If they meet, it loops. If the hare falls off the end, it does not.
Find the Duplicate Number	Array as a list (`i` links to `nums[i]`); Floyd phase 2 finds the entrance.	Index points to value: tortoise and hare from 0, then reset one to 0, walk together, and meet at the duplicate.
LRU Cache	Dict plus doubly linked list (`OrderedDict`).	Ordered dict: move to end on use, pop the front on overflow.
Merge k Sorted Lists	Many fronts: a min-heap with a tie-breaker.	Heap the fronts; pop the smallest, push its next.
Reverse Nodes in k-Group	Block surgery with four names; probe before you change.	Before, first, last, after: probe k, reverse with prev = after, link before to last, step to first.

The transfer drills for this technique are in Section 69.8 (page 420). The answers are in Section A.7 (page 440).

51 Trees

51.1 The idea

A binary tree is a recursive structure. A tree is empty, or it is a node with a left subtree and a right subtree. So a question about a tree splits into the same question about each subtree. Write a function that answers the question for one node. Let it use the answers of the two children, or the facts that come down from the ancestors. Each node then does $O(1)$ work, and one traversal costs $O(n)$.

The brute force repeats work at each node. It scans the whole subtree again to get a height, or it walks the path from the root again to see the ancestors. On a skewed tree (each node has at most one child, so the height is n), both cost $O(n^2)$. The traversal removes this repeated work: each node reuses the answers of its children and the facts from its parent.

Information moves in one of three directions.

- **Down (top-down).** A parent gives each child a fact about the path above it: the depth, the largest value so far, the allowed value range. Pass the fact as an argument, or keep it in the stack entry with the node.
- **Up (bottom-up).** A child gives its parent a summary of its subtree: the height, the size, the best path down. Return the summary from the call. The parent combines the two summaries after both calls return (postorder).
- **Across (level order).** A queue visits the nodes one level at a time (BFS). Use it when the question is about depth, levels or rows.

Worked example. Take the tree `[3, 1, 4, 3, None, 1, 5]` (LeetCode level order):

```
      3
     / \
    1   4
   /   / \
  3   1   5
```

A node is "good" when no value on the path above it is larger. This is a fact about the ancestors, so pass the path maximum down. The root gets no maximum and is good. It gives 3 to its children. Node 1 gets 3, so it is not good. The leaf 3 below it gets 3, so it is good. Node 4 gets 3, so it is good, and it gives 4 to its children. Leaf 1 is not good. Leaf 5 is good. The count is 4.

Now find the height. This is a fact about the subtree, so return it up. Each leaf returns 1. Node 1 returns $1 + \max(1, 0) = 2$. Node 4 returns $1 + \max(1, 1) = 2$. The root returns $1 + \max(2, 2) = 3$.

Some questions need a returned value and a separate answer. The longest path between two nodes has one highest node, where the path bends. A chain is a path that starts at a node and goes only down. At the highest node, the path uses one chain from each child. But the node can give only one chain to its parent, because a path cannot branch. So the function returns the one-sided chain and records the two-sided path in an outer variable. In the example tree, the root records $2 + 2 = 4$ edges (leaf 3 to leaf 5) and returns 3 to its (missing) parent.

51.2 Recognize it

- The input is a root node, or data where each item has one parent: a file system, an org chart, nested expressions.
- The answer for a node comes from the answers of its children: height, size, sum, "balanced", the best chain. Use bottom-up DFS.
- The rule for a node uses all its ancestors: "no ancestor is larger", "the path sum so far", a value range. Use top-down DFS.
- The words "path between any two nodes" or "may not pass through the root". Return one value to the parent and record the answer at each node.
- The words "level", "depth", "row", "view from the side", "minimum depth". Use BFS by levels. The output is a list of lists (one list for each depth), or one value for each depth.
- The input is a binary search tree (BST) and the question is about order: the kth value, a successor, a range, the closest value, a split point. Compare with `node.val` to discard one side, or read the in-order stream, which is sorted.
- Two trees go into one question (equal, mirror, contains). Recurse on pairs of nodes.
- The input is two traversals or a string, and the task is to rebuild the tree. One traversal names the root; null markers keep the shape.
- The tree has up to 10^4 or 10^5 nodes. Plan one $O(n)$ traversal. A scan of the subtree at each node is $O(n^2)$ on a skewed tree, which is too slow.
- The tree can be skewed, with a height of 10^4 or more. The recursion depth then passes the CPython limit of 1000. Plan an explicit stack or BFS.

51.3 Core template

All examples use the LeetCode node class. LeetCode defines it for you. Build a small test tree with nested calls: `T(2, T(1), T(3))` is the root 2 with the children 1 and 3. For the empty child in the middle of a call, write `None`: `T(1, None, T(2))`.

51.3.1 The three DFS orders

A DFS visits each node one time. The order tells when a node does its own work, relative to its two subtrees.

- **Preorder** (node, left, right). Top-down work goes here: the node passes its state before it visits the children.
- **In-order** (left, node, right). In a BST, this order gives the values in ascending order.
- **Postorder** (left, right, node). Bottom-up work goes here: the node needs both child results first.

```
class TreeNode:
    def __init__(self, val=0, left=None, right=None):
        self.val, self.left, self.right = val, left, right
T = TreeNode

def orders(node, pre, ino, post):
    if node is None:
        return
    pre.append(node.val)                # before both subtrees
    orders(node.left, pre, ino, post)
    ino.append(node.val)                # between the subtrees
    orders(node.right, pre, ino, post)
    post.append(node.val)               # after both subtrees

pre, ino, post = [], [], []
orders(T(1, T(2, T(4)), T(3, T(5), T(6))), pre, ino, post)
pre, ino, post      # -> ([1, 2, 4, 3, 5, 6], [4, 2, 1, 5, 3, 6], [4, 2, 5, 6, 3, 1])
```

51.3.2 Bottom-up: return a summary of the subtree

Write the contract first, in one sentence: "`f(node)` returns X for the subtree at `node`". Then write the value for the empty tree. Then combine the two child values.

```
class TreeNode:
    def __init__(self, val=0, left=None, right=None):
        self.val, self.left, self.right = val, left, right
T = TreeNode

def height(node):                   # nodes on the longest downward path
    if node is None:
        return 0                    # the empty tree
    left = height(node.left)        # trust the contract for each child
    right = height(node.right)
    return 1 + max(left, right)     # combine after both calls return

def total(node):                    # sum of all values in the subtree
    if node is None:
        return 0
    return node.val + total(node.left) + total(node.right)

root = T(3, T(1, T(3)), T(4, T(1), T(5)))
height(root)                        # -> 3
total(root)                         # -> 17
height(None), height(T(7))          # -> (0, 1)
```

Invariant: when `f(node)` returns, its value is correct for the whole subtree, because both child values were correct. The empty case starts the induction.

51.3.3 Top-down: pass the path state down

Keep the state in each stack entry, next to its node. Each branch then has its own copy. Compute the state of a child from the state of its parent in $O(1)$. The example lists the sums of all root-to-leaf paths.

```
class TreeNode:
    def __init__(self, val=0, left=None, right=None):
        self.val, self.left, self.right = val, left, right
T = TreeNode

def leaf_sums(root):
    out = []
    stack = [(root, 0)]                    # (node, sum of the path above it)
```

```
    while stack:
        node, above = stack.pop()
        if node is None:
            continue
        s = above + node.val                # the state for this node
        if node.left is None and node.right is None:
            out.append(s)                   # a leaf ends a path
        stack.append((node.left, s))        # each child gets its own copy
        stack.append((node.right, s))
    return sorted(out)

def leaf_sums_rec(node, above=0):           # recursive form: the state is an argument
    if node is None:
        return []
    s = above + node.val
    if node.left is None and node.right is None:
        return [s]
    return leaf_sums_rec(node.left, s) + leaf_sums_rec(node.right, s)

root = T(3, T(1, T(3)), T(4, T(1), T(5)))
leaf_sums(root)                             # -> [7, 8, 12]
leaf_sums_rec(root)                         # -> [7, 8, 12]
leaf_sums(None)                             # -> []
```

Invariant: each stack entry carries the exact state of the path from the root to the parent of its node.

51.3.4 Level by level: BFS

Keep the nodes of one whole level in a list. Read what you need from the level. Then replace the list with the children of its nodes, left child first.

```
class TreeNode:
    def __init__(self, val=0, left=None, right=None):
        self.val, self.left, self.right = val, left, right
T = TreeNode

def levels(root):
    out = []
    level = [root] if root else []
    while level:
        out.append([node.val for node in level])   # use the whole level here
        next_level = []
        for node in level:
            if node.left:
                next_level.append(node.left)
            if node.right:
                next_level.append(node.right)
        level = next_level                          # swap in the children
    return out

root = T(3, T(1, T(3)), T(4, T(1), T(5)))
levels(root)                                # -> [[3], [1, 4], [3, 1, 5]]
levels(None)                                # -> []
len(levels(root))                           # -> 3
```

Invariant: at the start of each loop, `level` holds exactly the nodes of one depth, from left to right. The number of loops is the height.

51.3.5 Deep trees: no recursion

CPython stops a recursion at a depth of about 1000. A skewed tree with 5000 nodes makes a recursive DFS raise `RecursionError`. LeetCode raises the limit, but a local run or another judge may not.

- Top-down and BFS: the loops above use no recursion.
- Bottom-up: list the nodes with a stack so that each parent comes before its children. Then walk the list backward. Both children of a node are then done before the node. Keep the child results in a dict that starts as `h = {None: 0}`, so an empty child has a value.

```
class TreeNode:
    def __init__(self, val=0, left=None, right=None):
        self.val, self.left, self.right = val, left, right

def height_rec(node):
    return 0 if node is None else 1 + max(height_rec(node.left), height_rec(node.right))

def parents_first(root):
    order, stack = [], [root]
    while stack:
        node = stack.pop()
        if node:
            order.append(node)
            stack.append(node.left)
            stack.append(node.right)
    return order

def height_iter(root):
    h = {None: 0}                                # the empty child has height 0
    for node in reversed(parents_first(root)):   # children before parents
        h[node] = 1 + max(h[node.left], h[node.right])
    return h[root]

skewed = None
for v in range(5000):
    skewed = TreeNode(v, skewed)                 # skewed to the left, 5000 levels deep
height_rec(skewed)                               # -> raises RecursionError
height_iter(skewed)                              # -> 5000
height_iter(None)                                # -> 0
```

NOTE

`sys.setrecursionlimit(10**6)` also helps (Section 30). The explicit stack is safe on all judges. The recursive form is shorter. Use it when the height is small.

51.4 Variations

51.4.1 The same rule at every node

Apply one local change at each node and let the traversal reach all nodes. The order of the visits does not matter, so recursion, a stack or a queue all work. Invert Binary Tree swaps the two children of every node. Memory hook: *swap the two children, then invert each child.*

```
class TreeNode:
    def __init__(self, val=0, left=None, right=None):
        self.val, self.left, self.right = val, left, right
T = TreeNode

def invert(root):
    if root is None:
        return None
    root.left, root.right = root.right, root.left   # one statement: both sides at once
    invert(root.left)
    invert(root.right)
    return root

def preorder(n):
```

```
    return [] if n is None else [n.val] + preorder(n.left) + preorder(n.right)

preorder(invert(T(4, T(2, T(1), T(3)), T(7, T(6), T(9)))))   # -> [4, 7, 9, 6, 2, 3, 1]
invert(None) is None                                         # -> True
```

Another problem of this shape: mirror an N-ary tree (reverse the list of children). Symmetric Tree and Merge Two Binary Trees look similar, but they walk two trees in lockstep (see below).

51.4.2 Bottom-up with a check at every node

The rule must hold at every node, and each node needs a number from its subtrees. A helper that scans the subtree again at each node costs $O(n^2)$ on a skewed tree. Merge the helper and the check into one bottom-up pass. Balanced Binary Tree compares the two child heights at each node. The main solution uses the parents-first list from the core template. Memory hook: *list parents first, walk backward, compare child heights.*

```
class TreeNode:
    def __init__(self, val=0, left=None, right=None):
        self.val, self.left, self.right = val, left, right
T = TreeNode

def is_balanced(root):
    order, stack = [], [root] if root else []
    while stack:                            # each parent before its children
        node = stack.pop()
        order.append(node)
        if node.left:
            stack.append(node.left)
        if node.right:
            stack.append(node.right)
    height = {None: 0}
    for node in reversed(order):            # children before parents
        left, right = height[node.left], height[node.right]
        if abs(left - right) > 1:
            return False
        height[node] = 1 + max(left, right)
    return True

def height_or_fail(node):                   # recursive form: -1 means "not balanced"
    if node is None:
        return 0
    left = height_or_fail(node.left)
    if left == -1:
        return -1                           # pass the failure up at once
    right = height_or_fail(node.right)
    if right == -1 or abs(left - right) > 1:
        return -1
    return 1 + max(left, right)

ok = T(3, T(9), T(20, T(15), T(7)))
bad = T(1, T(2, T(3, T(4), T(4)), T(3)), T(2))
is_balanced(ok), is_balanced(bad), is_balanced(None)   # -> (True, False, True)
height_or_fail(ok), height_or_fail(bad)                # -> (3, -1)
two_chains = T(1, T(2, T(3, T(4))), T(2, None, T(3, None, T(4))))
# the root sees heights 3 and 3; node 2 below it sees 2 and 0
is_balanced(two_chains)                                # -> False
```

The recursive form returns a value outside the valid range (`-1`) as the failure signal. It stops at the first failure and needs no dict. A tuple `(ok, height)` also works. Use the same shape when a rule at every node needs the size, the sum, the minimum or the maximum of the subtrees.

51.4.3 Answer through the node, value up to the parent

This split is the most important tree idea. Every path between two nodes has one highest node. At that node the path is a left chain, the node and a right chain. So enumerate the highest node:

- **Record** the two-sided path at the node in an outer best.
- **Return** only the one-sided chain, because the parent can extend only one side.

The brute force tries each node as the bend and computes its two chains from scratch: $O(n^2)$ on a skewed tree. The bottom-up pass gets every chain from the child values, so the whole search is one $O(n)$ walk. Diameter of Binary Tree counts edges: the candidate is `left + right`, and the node stores `1 + max(left, right)`. Memory hook: *parents first, then walk backward: add the two heights for best, store one plus the bigger height.* Binary Tree Maximum Path Sum adds values, which can be negative. Cut a negative child chain to 0: it is better to not take it. Memory hook: *list parents first, then walk back. At each node, cut negative sides to 0, record left plus me plus right, and store me plus the better side.*

```
class TreeNode:
    def __init__(self, val=0, left=None, right=None):
        self.val, self.left, self.right = val, left, right
T = TreeNode

def parents_first(root):
    order, stack = [], [root]
    while stack:
        node = stack.pop()
        if node:
            order.append(node)
            stack.append(node.left)
            stack.append(node.right)
    return order

def diameter(root):
    height, best = {None: 0}, 0
    for node in reversed(parents_first(root)):
        left, right = height[node.left], height[node.right]
        best = max(best, left + right)              # record: edges of the bend here
        height[node] = 1 + max(left, right)         # return: one side only
    return best

def max_path_sum(root):
    gain, best = {None: 0}, root.val                # a single node is a valid path
    for node in reversed(parents_first(root)):
        left = max(gain[node.left], 0)              # drop a negative branch
        right = max(gain[node.right], 0)
        best = max(best, node.val + left + right)   # record: left + me + right
        gain[node] = node.val + max(left, right)    # return: me + the better side
    return best

off_root = T(1, T(2, T(3, T(5)), T(4, None, T(6))))     # longest path avoids the root
diameter(off_root), diameter(T(1))                       # -> (4, 0)
max_path_sum(T(-10, T(9), T(20, T(15), T(7))))           # -> 42
max_path_sum(T(-3)), max_path_sum(T(-2, T(-1)))          # -> (-3, -1)
```

Aside: the usual textbook solution is the recursive form of the same idea. It is shorter and has the same $O(n)$ time. Use it when the height is small. The outer best needs `nonlocal`.

```
class TreeNode:
    def __init__(self, val=0, left=None, right=None):
        self.val, self.left, self.right = val, left, right
T = TreeNode

def max_path_sum(root):
    best = root.val
```

```
    def gain(node):                          # best downward chain that starts at node
        nonlocal best                        # without it: UnboundLocalError
        if node is None:
            return 0
        left = max(gain(node.left), 0)
        right = max(gain(node.right), 0)
        best = max(best, node.val + left + right)
        return node.val + max(left, right)
    gain(root)
    return best

max_path_sum(T(-10, T(9), T(20, T(15), T(7))))   # -> 42
max_path_sum(T(1, T(2), T(3)))                   # -> 6
```

Other problems of this shape: the diameter of an N-ary tree (add the two largest child chains), and the longest path in a general tree. The array version is Maximum Subarray: a tree path is two Kadane chains that meet at the highest node.

Moves along edges use the same split, with a sum in place of a maximum. In Distribute Coins in Binary Tree (LeetCode 979), each node must end with one coin. One move sends one coin along one edge. The balance of a subtree is its coins minus its nodes. A surplus must leave the subtree and a deficit must come in, so exactly the absolute balance crosses the edge above it. Return the balance to the parent, and add the absolute balance of each child to the total. Memory hook: *return coins minus nodes; each edge costs the absolute balance below it.*

```
class TreeNode:
    def __init__(self, val=0, left=None, right=None):
        self.val, self.left, self.right = val, left, right
T = TreeNode

def distribute_coins(root):
    order, stack = [], [root]
    while stack:
        node = stack.pop()
        if node:
            order.append(node)
            stack.append(node.left)
            stack.append(node.right)
    balance, moves = {None: 0}, 0                    # balance = coins minus nodes
    for node in reversed(order):                     # children before parents
        left, right = balance[node.left], balance[node.right]
        moves += abs(left) + abs(right)              # record: coins that cross each child edge
        balance[node] = node.val - 1 + left + right  # return: the balance for the parent edge
    return moves

distribute_coins(T(3, T(0), T(0)), distribute_coins(T(0, T(3), T(0)))   # -> (2, 3)
distribute_coins(T(1, T(0, None, T(3)), T(0)))                          # -> 4
```

51.4.4 Two trees in lockstep

Walk two trees together, one pair of nodes at a time. Treat `None` as a real value that must match `None`. Same Tree checks the empty pairs first, then the values, then the two child pairs. Memory hook: *both null is true, one null or different values is false, else check left pair and right pair.* Subtree of Another Tree runs the same check from every node of the large tree. It is pattern search: one loop lists the start positions, and one exact check runs at each start. Memory hook: *try every node as a start, and ask Same Tree at each one. Both loops pop, skip the empty case, check, and push the children.*

```
class TreeNode:
    def __init__(self, val=0, left=None, right=None):
        self.val, self.left, self.right = val, left, right
T = TreeNode

def same(p, q):                              # Same Tree, recursive
```

```
    if p is None and q is None:
        return True
    if p is None or q is None or p.val != q.val:
        return False
    return same(p.left, q.left) and same(p.right, q.right)

def same_iter(a, b):                        # the same check with a stack of pairs
    pairs = [(a, b)]
    while pairs:
        x, y = pairs.pop()
        if x is None and y is None:
            continue
        if x is None or y is None or x.val != y.val:
            return False
        pairs.append((x.left, y.left))
        pairs.append((x.right, y.right))
    return True

def is_subtree(root, sub):
    stack = [root]                          # every node of root is a possible start
    while stack:
        node = stack.pop()
        if node is None:
            continue
        if same_iter(node, sub):
            return True
        stack.append(node.left)
        stack.append(node.right)
    return False

same(T(1, T(2)), T(1, None, T(2)))                      # -> False
sub = T(4, T(1), T(2))
is_subtree(T(3, T(4, T(1), T(2)), T(5)), sub)           # -> True
is_subtree(T(3, T(4, T(1), T(2, T(0))), T(5)), sub)     # -> False
```

The check is $O(n \cdot m)$. Aside: for large trees, serialize both trees in preorder. Write a null marker for each empty child and a delimiter before each value. Then search for the small string in the large string with a linear matcher. Other problems of this shape: Symmetric Tree (pair the left child of one side with the right child of the other), Flip Equivalent Binary Trees (accept the straight or the swapped pairing), Merge Two Binary Trees.

51.4.5 A path state from the ancestors

The rule for a node uses all its ancestors. Reduce the path to a small summary, such as a maximum, a minimum, a sum, a bitmask or a value range. Each parent computes the summary of its children in $O(1)$. Count Good Nodes in Binary Tree carries the largest value above each node. Memory hook: *carry the path max down; count a node when it is at least the max.* Validate Binary Search Tree carries an open window `(low, high)`. A left turn sets a new ceiling, and a right turn sets a new floor. A check of each parent against its children is not enough. Memory hook: *every node lives in a window: going left lowers the ceiling, going right raises the floor.*

```
class TreeNode:
    def __init__(self, val=0, left=None, right=None):
        self.val, self.left, self.right = val, left, right
T = TreeNode

def good_nodes(root):
    count = 0
    stack = [(root, root.val)]          # (node, biggest value above it on the path)
    while stack:
        node, path_max = stack.pop()
        if node is None:
            continue
```

```
        if node.val >= path_max:
            count += 1
        path_max = max(path_max, node.val)
        stack.append((node.left, path_max))
        stack.append((node.right, path_max))
    return count

def is_bst(root):
    stack = [(root, float("-inf"), float("inf"))]
    while stack:
        node, low, high = stack.pop()
        if node is None:
            continue
        if not (low < node.val < high):
            return False
        stack.append((node.left, low, node.val))    # left side: new ceiling
        stack.append((node.right, node.val, high))  # right side: new floor
    return True

good_nodes(T(3, T(1, T(3)), T(4, T(1), T(5))))   # -> 4
good_nodes(T(3, T(3, T(4), T(2))))               # -> 3
is_bst(T(2, T(1), T(3)))                         # -> True
is_bst(T(5, T(4), T(6, T(3), T(7))))             # -> False
is_bst(T(2, T(2)))                               # -> False
```

In the second BST test, 3 is a correct left child of 6, but it is in the right subtree of 5. Other problems of this shape: Path Sum (carry the remaining sum), Sum Root to Leaf Numbers (carry the number so far), Pseudo-Palindromic Paths (carry a parity bitmask).

51.4.6 One answer per level

Use the BFS level loop of the core template and reduce each level to what the question asks for. Binary Tree Level Order Traversal keeps the whole level. This is the core template. Memory hook: *take the whole level, save its values, swap in its children.* Maximum Depth of Binary Tree counts the levels. Memory hook: *depth is 1 plus the deeper child; for a very deep tree, count the BFS levels instead.* The short recursive form `1 + max(depth(left), depth(right))` is the base shape of most bottom-up problems. Binary Tree Right Side View keeps the last node of each level. Following only the right pointers fails when the left subtree is deeper. Memory hook: *level-order BFS; the last node of each level is the one you see.* The example uses the second common form of BFS: one deque, and a loop over the level size, read one time. Read the answer for a level before the inner loop removes its nodes. At that moment `queue[-1]` is the rightmost node and `queue[0]` is the leftmost node, so you need no extra variable. Use the same read for any first or last value of each level.

```
from collections import deque
class TreeNode:
    def __init__(self, val=0, left=None, right=None):
        self.val, self.left, self.right = val, left, right
T = TreeNode

def max_depth(root):
    depth, level = 0, [root] if root else []
    while level:
        depth += 1
        level = [c for n in level for c in (n.left, n.right) if c]
    return depth

def right_side_view(root):
    if root is None:
        return []
    view, queue = [], deque([root])
    while queue:
        view.append(queue[-1].val)          # the queue holds one level, left to right
```

```
        for _ in range(len(queue)):         # the range reads the size one time
            node = queue.popleft()
            if node.left:
                queue.append(node.left)
            if node.right:
                queue.append(node.right)
    return view

max_depth(T(3, T(9), T(20, T(15), T(7)))), max_depth(None)    # -> (3, 0)
right_side_view(T(1, T(2, None, T(5)), T(3, None, T(4))))    # -> [1, 3, 4]
right_side_view(T(1, T(2)))                                   # -> [1, 2]
```

A DFS can also give one value per depth. Carry the depth, visit the right child first, and append a value when `depth == len(view)`. The first node at each new depth is the rightmost one. Other problems of this shape: Zigzag Level Order (reverse every other level), Average of Levels, Largest Value in Each Row, Minimum Depth (stop at the first leaf).

51.4.7 BST: compare and go one way

In a BST, one comparison with `node.val` tells which subtree holds a value. So a search follows one path and costs $O(h)$, not $O(n)$. Lowest Common Ancestor of a Binary Search Tree walks down while both targets are on the same side. The first node where they split (or where one target sits) is the answer. Memory hook: *both smaller go left, both bigger go right, else this node is the split.* The example takes the two target values. LeetCode passes two nodes, so compare `p.val` and `q.val`.

```
class TreeNode:
    def __init__(self, val=0, left=None, right=None):
        self.val, self.left, self.right = val, left, right
T = TreeNode

def lca_bst(root, p, q):                # p and q are values that exist in the tree
    node = root
    while node:
        if p < node.val and q < node.val:
            node = node.left            # both targets are in the left subtree
        elif p > node.val and q > node.val:
            node = node.right           # both targets are in the right subtree
        else:
            return node                 # they split here, or one target is here
    return None

bst = T(6, T(2, T(0), T(4, T(3), T(5))), T(8, T(7), T(9)))
lca_bst(bst, 2, 8).val, lca_bst(bst, 2, 4).val, lca_bst(bst, 5, 3).val   # -> (6, 2, 4)
```

Use strict `<` and `>` to move. A node is its own ancestor, so an equal value stops the walk. The same descent finds a value, an insert position, a floor, a ceiling or the closest value. In a tree without order, this descent does not work, because one comparison does not tell the side.

51.4.8 BST: the in-order stream is sorted

An in-order traversal (left, node, right) of a BST gives the values in ascending order. Read the BST as a sorted list, one value at a time, and stop early. A sort of all values costs $O(n \log n)$. The stack walk stops after k values, so it costs $O(h + k)$. Kth Smallest Element in a BST pops k values from an iterative in-order walk. The left spine of a node is the node, its left child, the left child of that child, and so on. The smallest value of a subtree is at the bottom of its left spine. Memory hook: *push the left spine; pop, count, push the right child's left spine.*

```
class TreeNode:
    def __init__(self, val=0, left=None, right=None):
        self.val, self.left, self.right = val, left, right
T = TreeNode

def kth_smallest(root, k):
```

```
    stack = []
    def push_left(node):
        while node:
            stack.append(node)
            node = node.left
    push_left(root)                     # the top of the stack is the smallest value
    while True:
        node = stack.pop()
        k -= 1
        if k == 0:
            return node.val
        push_left(node.right)           # the next smallest value is now on top

def inorder(root):                      # the same walk as a generator: a sorted stream
    stack, node = [], root
    while stack or node:
        while node:
            stack.append(node)
            node = node.left
        node = stack.pop()
        yield node.val
        node = node.right

bst = T(5, T(3, T(2, T(1)), T(4)), T(6))
kth_smallest(bst, 3), kth_smallest(bst, 1)      # -> (3, 1)
list(inorder(bst))                              # -> [1, 2, 3, 4, 5, 6]
vals = list(inorder(T(5, T(4), T(6, T(3), T(7)))))
all(a < b for a, b in zip(vals, vals[1:]))      # -> False
```

The last two lines are a second way to validate a BST: the in-order stream must be strictly increasing. For the kth largest, walk in reverse in-order (right, node, left). Other problems of this shape: Binary Search Tree Iterator, Two Sum IV (a forward and a backward iterator as two pointers).

51.4.9 Build a tree from two traversals

One traversal names the root: preorder starts with the root, and postorder ends with it. In-order splits the other values: the values before the root form the left subtree. Every subtree is a contiguous block in preorder, in-order and postorder (not in level order), so recurse on the blocks. Construct Binary Tree from Preorder and Inorder Traversal slices both lists and keeps a stack of jobs. Memory hook: *preorder names the root. In-order splits at the root. Slice both lists and push each side as a job.*

```
class TreeNode:
    def __init__(self, val=0, left=None, right=None):
        self.val, self.left, self.right = val, left, right

def build(preorder, inorder):
    root = TreeNode(preorder[0])
    stack = [(root, preorder, inorder)]     # job: node, preorder and in-order of its subtree
    while stack:
        node, pre, ino = stack.pop()
        mid = ino.index(node.val)           # the size of the left subtree
        left_pre, left_ino = pre[1:mid + 1], ino[:mid]
        right_pre, right_ino = pre[mid + 1:], ino[mid + 1:]
        if left_pre:
            node.left = TreeNode(left_pre[0])
            stack.append((node.left, left_pre, left_ino))
        if right_pre:
            node.right = TreeNode(right_pre[0])
            stack.append((node.right, right_pre, right_ino))
    return root

def pre(n):
```

```
    return [] if n is None else [n.val] + pre(n.left) + pre(n.right)
def ino(n):
    return [] if n is None else ino(n.left) + [n.val] + ino(n.right)

root = build([3, 9, 20, 15, 7], [9, 3, 15, 20, 7])
root.val, root.left.val, root.right.val, root.right.left.val   # -> (3, 9, 20, 15)
pre(root), ino(root)            # -> ([3, 9, 20, 15, 7], [9, 3, 15, 20, 7])
```

The slices and `index` make this $O(n^2)$ on a skewed tree. Aside: the optimal form is $O(n)$ with two changes. Look up the root position in a dict from value to in-order index. Pass in-order bounds `(lo, hi)` instead of lists, and read the roots with one shared preorder pointer.

```
class TreeNode:
    def __init__(self, val=0, left=None, right=None):
        self.val, self.left, self.right = val, left, right

def build(preorder, inorder):
    pos = {v: i for i, v in enumerate(inorder)}
    roots = iter(preorder)                  # one shared cursor into preorder
    def make(lo, hi):                       # the subtree that holds inorder[lo..hi]
        if lo > hi:
            return None
        node = TreeNode(next(roots))
        mid = pos[node.val]
        node.left = make(lo, mid - 1)       # left first: preorder lists it first
        node.right = make(mid + 1, hi)
        return node
    return make(0, len(inorder) - 1)

def pre(n):
    return [] if n is None else [n.val] + pre(n.left) + pre(n.right)
root = build([3, 9, 20, 15, 7], [9, 3, 15, 20, 7])
pre(root), root.right.right.val             # -> ([3, 9, 20, 15, 7], 7)
```

The shared cursor works only because the code builds the left subtree before the right subtree. That is the order in which preorder lists the nodes. The dict and the in-order split need unique values. The general rule: when a stream lists a recursive structure in the same order as your recursion visits it, read the stream with one shared cursor. You then compute no offsets. The preorder decoder for serialization (next variation) uses the same rule.

51.4.10 Serialize with null markers

A list of values loses the shape. The root 1 with the left child 2 gives the values 1, 2. The root 1 with the right child 2 gives the same values. Write a marker `N` for every empty child. Then one preorder, postorder or level-order traversal fully defines the tree. The decoder reads the tokens in the same order as the encoder wrote them. In-order with markers is not enough: `T(1, T(2))` and `T(2, None, T(1))` both give `N,2,N,1,N`. Serialize and Deserialize Binary Tree uses level order, like the LeetCode list format. Memory hook: *write it level by level like the example list, with N for every null; each real node in the queue takes the next two tokens as its children.*

```
from collections import deque
class TreeNode:
    def __init__(self, val=0, left=None, right=None):
        self.val, self.left, self.right = val, left, right
T = TreeNode

def serialize(root):
    out, queue = [], deque([root])
    while queue:
        node = queue.popleft()
        if node is None:
            out.append("N")                 # the null marker keeps the shape
            continue
        out.append(str(node.val))
```

```
        queue.append(node.left)             # the queue holds None children too
        queue.append(node.right)
    return ",".join(out)

def deserialize(data):
    tokens = iter(data.split(","))
    def make(tok):
        return None if tok == "N" else TreeNode(int(tok))
    root = make(next(tokens))
    queue = deque([root])
    while queue:
        node = queue.popleft()
        if node is None:
            continue                        # a null has no children to read
        node.left = make(next(tokens, "N")) # the next two tokens are its children
        node.right = make(next(tokens, "N"))
        queue.append(node.left)
        queue.append(node.right)
    return root

s = serialize(T(1, T(2), T(3, T(4), T(5))))
s                                           # -> '1,2,3,N,N,4,5,N,N,N,N'
serialize(deserialize(s)) == s              # -> True
serialize(None), deserialize("N") is None   # -> ('N', True)
serialize(deserialize("-1,N,20"))           # -> '-1,N,20,N,N'

def inorder_marks(n):                       # in-order with markers: not reversible
    if n is None:
        return "N"
    return inorder_marks(n.left) + "," + str(n.val) + "," + inorder_marks(n.right)
inorder_marks(T(1, T(2))), inorder_marks(T(2, None, T(1)))   # -> ('N,2,N,1,N', 'N,2,N,1,N')
```

The default `"N"` in `next(tokens, "N")` also reads a LeetCode list, which drops the trailing nulls (write `N` for `null`). So `deserialize` is a good helper to build test trees. Join the tokens with a delimiter and split on it, so negative and multi-digit values stay whole. Aside: the usual textbook solution writes preorder with null markers. It decodes with the grammar *tree = N, or a value, a left tree and a right tree*, and reads the tokens with one shared iterator. Part I ("Share one iterator between recursive calls") shows that decoder. The grammar form is the base for problems that check or reuse a preorder encoding. Other problems of this shape: the N-ary version (write the child count after each value), Encode and Decode Strings (length prefixes), Find Duplicate Subtrees (the serialized subtree is a dict key).

51.5 Adapt it to a new problem

Ask these questions in this order.

1. Write the brute force in one sentence. Find the work that it repeats at each node.
2. Does the answer for a node need its ancestors (down), its subtree (up), or its level (across)?
3. Write the contract in one sentence: "`f(node)` returns ... for the subtree", or "each stack entry carries ... of the path above".
4. Write the empty case first. Decide what `None` gives: 0, `True`, `float("-inf")` or a sentinel.
5. Does the parent need two facts from each child? Return a tuple, for example `(ok, height)`, or use a sentinel such as `-1` for "failed".
6. Is the answer always at the root, or can it be at any node? If at any node, record a best at each node and return only the part that the parent can extend.
7. Can a part of the answer make it worse (negative values)? Clamp that part at 0.
8. Is the tree a BST? Compare with `node.val` to go one way, or read the sorted in-order stream.
9. Do two structures move together? Recurse on pairs, and handle the `None` pairs first.
10. Can the height reach n with $n > 1000$? Use a stack with state, BFS, or the parents-first list.
11. Test the empty tree, one node, a tree skewed to the left, a tree skewed to the right, negative values and equal values.

51.6 Pitfalls

CAUTION

Return and record are two different values. Do not return the two-sided value (`left + right` or `node.val + left + right`) to the parent. A path cannot branch, so return `1 + max(left, right)` or `node.val + max(left, right)`. Also, the answer at the root only is wrong when the best path avoids the root.

CAUTION

The start value of a best. When the values can be negative and the answer is not empty, start the best at `root.val` or `float("-inf")`, not 0. Clamp only the child chains at 0. The node itself is always on its own path.

CAUTION

A node, not a value. Test `node is None`. A node with the value 0 is still a node. Read `node.val` only after the `None` test.

CAUTION

All ancestors, not only the parent. A BST rule compares a node with every ancestor. A check of each node against its two children accepts `[5, 4, 6, None, None, 3, 7]`. Pass the window `(low, high)` down, use strict comparisons, and start with infinite bounds, not with the smallest or largest integer of the input range.

```
class TreeNode:
    def __init__(self, val=0, left=None, right=None):
        self.val, self.left, self.right = val, left, right
T = TreeNode

def local_check(n):                     # wrong: compares only with the children
    if n is None:
        return True
    if n.left and not n.left.val < n.val:
        return False
    if n.right and not n.val < n.right.val:
        return False
    return local_check(n.left) and local_check(n.right)

def window_check(n, low=float("-inf"), high=float("inf")):
    if n is None:
        return True
    if not (low < n.val < high):
        return False
    return window_check(n.left, low, n.val) and window_check(n.right, n.val, high)

t = T(5, T(4), T(6, T(3), T(7)))
local_check(t), window_check(t)         # -> (True, False)
```

CAUTION

State that two branches share. In a top-down walk, one global "path maximum" is wrong after the walk returns from a branch. Keep the state in the argument or in the stack entry. In a recursive walk, a nested function that assigns to an outer counter needs `nonlocal`. Without it, Python raises `UnboundLocalError`.

```
def count_wrong(values):
    count = 0
    def visit(i):
        if i < len(values):
            count += 1                  # assignment makes count a local name
            visit(2 * i + 1)
    visit(0)
    return count
```

```
def count_right(values):
    count = 0
    def visit(i):
        nonlocal count
        if i < len(values):
            count += 1
            visit(2 * i + 1)
    visit(0)
    return count

count_wrong([1, 2, 3, 4])              # -> raises UnboundLocalError
count_right([1, 2, 3, 4])              # -> 3
```

CAUTION

The level size and the queue. Read the level size one time before the inner loop: `for _ in range(len(queue))`. A `while` loop that reads `len(queue)` again also takes the children of this level. Do not use `list.pop(0)` as a queue: each pop is $O(n)$. Push the left child before the right child, or the order inside a level breaks.

CAUTION

Nodes or edges. Height in nodes gives `height(None) = 0`, and `left + right` is then an edge count. Height in edges needs `height(None) = -1`. Then `-1` cannot be a failure sentinel. A single node has depth 1 and diameter 0.

CAUTION

Two assignments for a swap. `root.left = root.right` followed by `root.right = root.left` loses the old left child. Swap in one statement: `root.left, root.right = root.right, root.left`.

CAUTION

Recursion depth. A skewed tree with n nodes makes a recursion n calls deep. CPython stops at about 1000. For n up to 10^4 or 10^5, use an explicit stack, BFS, or the parents-first list.

CAUTION

Encodings that lose information. Values alone lose the shape. Write a null marker for each empty child. Put a delimiter between tokens, or `"-1"` splits into two characters. For a substring search, put the delimiter before every token, or `"2,N,N"` matches inside `"12,N,N"`. The rebuild from preorder plus in-order needs unique values. With repeats, `T(1, T(1))` and `T(1, None, T(1))` give the same two lists.

51.7 Problems in this family

Problem	What changes	Memory hook
Invert Binary Tree	The same local rule (swap the children) at every node; any order of visits.	Swap the two children, then invert each child.
Maximum Depth of Binary Tree	The bottom-up height template itself; BFS counts the levels with no recursion.	Depth is 1 plus the deeper child; for a very deep tree, count the BFS levels instead.
Diameter of Binary Tree	Record `left + right` at each node; return `1 + max(left, right)`.	Parents first, then walk backward: add the two heights for best, store one plus the bigger height.
Balanced Binary Tree	Bottom-up heights from the parents-first list; compare the two child heights at every node. Aside: the recursive form returns `-1` on failure.	List parents first, walk backward, compare child heights.

Problem	What changes	Memory hook
Same Tree	Two trees in lockstep; the `None` pairs first.	Both null is true, one null or different values is false, else check left pair and right pair.
Subtree of Another Tree	Same Tree from every node of the large tree.	Try every node as a start, and ask Same Tree at each one. Both loops pop, skip the empty case, check, and push the children.
Lowest Common Ancestor of a Binary Search Tree	One guided descent in a BST; stop where the targets split.	Both smaller go left, both bigger go right, else this node is the split.
Binary Tree Level Order Traversal	BFS; keep the whole level.	Take the whole level, save its values, swap in its children.
Binary Tree Right Side View	BFS; keep the last node of each level.	Level-order BFS; the last node of each level is the one you see.
Count Good Nodes in Binary Tree	Top-down; carry the path maximum.	Carry the path max down; count a node when it is at least the max.
Validate Binary Search Tree	Top-down; carry an open window `(low, high)`.	Every node lives in a window: going left lowers the ceiling, going right raises the floor.
Kth Smallest Element in a BST	Iterative in-order is a sorted stream; stop after k values.	Push the left spine; pop, count, push the right child's left spine.
Construct Binary Tree from Preorder and Inorder Traversal	Preorder gives the root; in-order splits the rest.	Preorder names the root. In-order splits at the root. Slice both lists and push each side as a job.
Binary Tree Maximum Path Sum	Record `left + me + right`; return `me + max(left, right)`; clamp at 0.	List parents first, then walk back. At each node, cut negative sides to 0, record left plus me plus right, and store me plus the better side.
Serialize and Deserialize Binary Tree	Level order with a marker for each null child.	Level by level with N for every null; each real node takes the next two tokens as its children.

The transfer drills for this technique are in Section 69.9 (page 421). The answers are in Section A.8 (page 442).

52 Tries

52.1 The idea

Many stored words share their first letters. A trie (prefix tree) stores each shared beginning one time only. Each edge holds one letter. Each node stands for one prefix: the letters on the path from the root to that node. The root stands for the empty string. A flag on a node tells whether its prefix is also a whole stored word.

Worked example. Insert `car`, `cat`, `cart` and `dog`. The root gets two children, `c` and `d`. Three words start with `ca`, but the path c-a exists one time. Below `ca`, the path splits into `r` and `t`. The four nodes where a word ends carry the end flag.

```
(root)
 +-- c
 |   +-- a
 |       +-- r  *        * = end flag (car)
 |       |   +-- t  *    (cart)
 |       +-- t  *        (cat)
 +-- d
```

```
    +-- o
        +-- g  *        (dog)
```

Now ask about `ca`. The walk takes two steps and reaches a node, so some stored word starts with `ca`. That node has no end flag, so `ca` itself is not a stored word. Ask about `cab`. The walk takes two steps, and then stops: the node for `ca` has no child `b`. The number of stored words does not change the cost of the walk.

What a trie saves.

- Brute force compares the query with every stored word: $O(N \cdot L)$ for each query.
- A hash set of every prefix answers a prefix query in $O(L)$. But each insert builds L prefix strings, so it copies $O(L^2)$ characters.
- A trie does each insert and each query in $O(L)$ steps. It has at most one node for each stored letter, and shared beginnings cost nothing extra.

The larger gain appears when a search branches. A wildcard letter or the neighbors of a grid cell give several next letters at one position. At each step, the search must ask: "is this string still the start of some target word?" The current trie node answers this in $O(1)$: the next letter is a child, or it is not. A missing child stops the branch at once. This pruning is the main reason to use a trie.

Trie or set. A trie is not always the best tool. Use this table to choose.

The task needs	Use
Exact lookups only	A hash set.
Prefix queries, with inserts and queries mixed	A trie.
Prefix queries on a list that does not change	Sort the list and use `bisect`, or a trie.
Wildcard queries with few wildcards (1 or 2)	A set of masked patterns (one letter replaced by a dot). Fill a second dot with each letter.
Many wildcards, or a search that grows a string (grid, stream)	A trie and a DFS that walks it.
All stored words below a prefix (autocomplete)	A trie, or a sorted list and `bisect`.
The shortest or longest stored prefix of a query	A trie walk that watches the end flags.

A sorted list answers a static prefix query with one binary search. All words with prefix p are next to each other, and the first one is at `bisect_left(words, p)`.

```
from bisect import bisect_left
words = sorted(["mouse", "mobile", "moneypot", "monitor", "mousepad"])
def has_prefix(p):
    i = bisect_left(words, p)          # the first word that is >= p
    return i < len(words) and words[i].startswith(p)
has_prefix("mou"), has_prefix("mox")   # -> (True, False)
i = bisect_left(words, "mon")
words[i:i + 2]                         # -> ['moneypot', 'monitor']
```

52.2 Recognize it

- The input has many strings (a dictionary, a word list), and the queries ask about prefixes: "starts with", autocomplete, "shortest root", "common prefix".
- Inserts and queries interleave, so one sort at the start does not work.
- A query contains a wildcard that matches any one letter. One query then stands for many strings. A small bound on the wildcards ("at most 2 dots") also allows a set of masked patterns.
- A search builds a string one letter at a time: a path in a grid, a stream of letters, a split of a string. At each step, the search must test many target words.
- Many patterns must be found in one search space. You know the search for one pattern, but a repeat of it for each pattern is too slow.
- The output is a subset of a given word list: "return all words that appear on the board".
- The sizes: 10^4 or more stored words, and 10^4 or more calls. Each word is short, often 10 to 30 letters. Each call must then cost $O(L)$ for its string, not $O(N \cdot L)$ for all stored strings.
- The keys are sequences of small parts: lowercase letters, digits, the bits of an integer, the parts of a file path.

52.3 Core template

Give each node a dict of children and an end flag. Write one helper that walks a string down from the root. Then `search` and `startsWith` differ only in the last check.

```
class TrieNode:
    def __init__(self):
        self.children = {}      # letter to TrieNode
        self.is_end = False     # True if a whole word ends here

class Trie:
    def __init__(self):
        self.root = TrieNode()

    def insert(self, word):
        node = self.root
        for ch in word:
            if ch not in node.children:
                node.children[ch] = TrieNode()
            node = node.children[ch]
        node.is_end = True      # flag the last node only

    def _walk(self, s):         # the node for prefix s, or None
        node = self.root
        for ch in s:
            node = node.children.get(ch)
            if node is None:
                return None
        return node

    def search(self, word):     # is word a whole stored word?
        node = self._walk(word)
        return node is not None and node.is_end

    def startsWith(self, prefix):   # does a stored word start with prefix?
        return self._walk(prefix) is not None

t = Trie()
t.insert("apple")
t.search("apple"), t.search("app"), t.startsWith("app")   # -> (True, False, True)
t.insert("app")
t.search("app")                                           # -> True
t.startsWith("apple"), t.startsWith("b")                  # -> (True, False)
```

Invariant. A node other than the root exists if and only if its string is a prefix of an inserted word. `is_end` is `True` if and only if its string was inserted as a whole word. Each operation costs $O(L)$ for a string of length L. The memory is O(total letters inserted).

NOTE

The judge uses the method names of the statement, often in camelCase: `startsWith`, `addWord`. A snake_case name fails the judge.

A shorter form uses nested dicts. Each node is a dict, and a special key marks the end of a word. The end key must never be a letter of the alphabet. Write this form fast when the trie is only a helper inside a larger solution.

```
END = "$"                       # a key that is never a letter

def insert(root, word):
    node = root
    for ch in word:
        node = node.setdefault(ch, {})   # create the child only on insert
    node[END] = True
```

```
def find(root, s):              # the node for prefix s, or None
    node = root
    for ch in s:
        if ch not in node:
            return None
        node = node[ch]
    return node

root = {}
for w in ["car", "cat", "cart", "dog"]:
    insert(root, w)
END in find(root, "car"), END in find(root, "ca")   # -> (True, False)
find(root, "cab") is None                           # -> True
sorted(find(root, "ca"))                            # -> ['r', 't']
```

52.4 Variations

52.4.1 Whole word or prefix

This is the core template (Implement Trie (Prefix Tree)). A path can exist only because a longer word passes through it. So `search` returns the end flag of the last node, and `startsWith` returns "the walk reached a node". A word counts as a prefix of itself. A second insert of the same word walks the same nodes and changes nothing. Memory hook: *One node per prefix: walk the letters, flag the word end.*

52.4.2 Store data on the nodes

Put more data on each node. The same walk then answers richer questions.

- A pass count (the number of words that go through the node) answers "how many words start with p". An end count answers "how many copies of w are stored". An erase decrements the counts along the path.
- The whole word at its end node gives the answer at once. You do not rebuild the path string.
- A best value below the node (a sum, a minimum, the 3 smallest words) answers a query in one walk. Update the value at each node on the path during the insert.
- The sum of the pass counts on the query walk (without the root) answers more than one prefix at a time. It adds up, for each prefix of the query, the number of stored words with that prefix (Sum of Prefix Scores of Strings, LeetCode 2416).

Memory hook: *Update every node on the insert path; read the last node of the query walk, or add up all its nodes.*

```
class Node:
    def __init__(self):
        self.children = {}
        self.pass_count = 0     # words that go through this node
        self.end_count = 0      # words that end at this node

class CountTrie:
    def __init__(self):
        self.root = Node()

    def insert(self, word):
        node = self.root
        node.pass_count += 1
        for ch in word:
            node = node.children.setdefault(ch, Node())
            node.pass_count += 1
        node.end_count += 1

    def _walk(self, s):
        node = self.root
        for ch in s:
            node = node.children.get(ch)
```

```
            if node is None:
                return None
        return node

    def count_prefix(self, p):
        node = self._walk(p)
        return node.pass_count if node else 0

    def count_equal(self, w):
        node = self._walk(w)
        return node.end_count if node else 0

    def prefix_score(self, w):  # add up count_prefix over every prefix of w
        node, total = self.root, 0
        for ch in w:
            node = node.children.get(ch)
            if node is None:
                break
            total += node.pass_count
        return total

    def erase(self, word):      # word must be stored
        node = self.root
        node.pass_count -= 1
        for ch in word:
            child = node.children[ch]
            child.pass_count -= 1
            if child.pass_count == 0:
                del node.children[ch]   # no word is left below: cut the branch
                return
            node = child
        node.end_count -= 1

t = CountTrie()
for w in ["apple", "apple", "app"]:
    t.insert(w)
t.count_equal("apple"), t.count_prefix("app")    # -> (2, 3)
t.erase("apple")
t.count_equal("apple"), t.count_prefix("app")    # -> (1, 2)
t.erase("apple")
t.count_prefix("appl"), t.count_equal("app")     # -> (0, 1)
t._walk("app").children                          # -> {}
s = CountTrie()
for w in ["abc", "ab", "bc", "b"]:
    s.insert(w)
[s.prefix_score(w) for w in ["abc", "ab", "bc", "b"]]   # -> [5, 4, 3, 2]
```

52.4.3 Shortest stored prefix and autocomplete

Two more walks come up often. The first walk follows a query word and stops at the first end flag. It gives the shortest stored prefix of the word (Replace Words). For the longest one, keep the last end flag. The second walk goes down to the node of a prefix. Then a DFS visits the children in sorted order. It lists all words with that prefix in lexicographic order (autocomplete). Memory hook: *Stop at the first flag; list below the prefix node in sorted order.*

```
def build(words):
    root = {}
    for w in words:
        node = root
        for ch in w:
            node = node.setdefault(ch, {})
```

```
        node["$"] = w           # store the whole word at its end node
    return root

def shortest_root(root, word):  # the shortest stored prefix of word, else word
    node = root
    for ch in word:
        if ch not in node:
            break
        node = node[ch]
        if "$" in node:
            return node["$"]
    return word

def words_below(node, out):     # all words at or below node, sorted
    if "$" in node:
        out.append(node["$"])   # a word comes before its extensions
    for ch in sorted(k for k in node if k != "$"):
        words_below(node[ch], out)
    return out

roots = build(["cat", "bat", "rat", "ca"])
[shortest_root(roots, w) for w in "the cattle was rattled".split()]
# -> ['the', 'ca', 'was', 'rat']
shop = build(["mouse", "mobile", "moneypot", "monitor", "mousepad"])
words_below(shop["m"]["o"]["u"], [])     # -> ['mouse', 'mousepad']
words_below(shop, [])[:3]                # -> ['mobile', 'moneypot', 'monitor']
```

52.4.4 Wildcard search by branching

A query can contain a wildcard: a dot that matches any one letter (Design Add and Search Words Data Structure). The query and the stored word must then have the same length. The statement often bounds the number of dots, for example at most 2 for each query. Inserts and queries interleave, so the structure must grow one word at a time.

Intuitive solution: a set of masked patterns. When the number of dots is small, move the work to the insert. Store each word, and each copy of it with one letter replaced by a dot. A query with at most one dot is then one set lookup. A query with two dots fills its first dot with each of the 26 letters and searches again. An insert costs $O(L^2)$. A query with $k \geq 1$ dots costs 26^{k-1} set lookups. Memory hook: *Mask one letter on add, fill one dot on search.*

```
class WordDictionary:
    def __init__(self):
        self.patterns = set()   # each word, and each copy with one letter masked

    def addWord(self, word):
        self.patterns.add(word)
        for i in range(len(word)):
            self.patterns.add(word[:i] + "." + word[i + 1:])

    def search(self, word):
        if word.count(".") <= 1:
            return word in self.patterns
        i = word.index(".")     # fill the first dot, then search again
        return any(self.search(word[:i] + letter + word[i + 1:])
                   for letter in "abcdefghijklmnopqrstuvwxyz")

d = WordDictionary()
for w in ["bad", "dad", "mad"]:
    d.addWord(w)
d.search("pad"), d.search("bad"), d.search(".ad"), d.search("b..")   # -> (False, True, True, True)
d.search("..."), d.search("ba"), d.search("....")                    # -> (True, False, False)
```

General solution: DFS over the trie. A letter in the query follows one edge. A dot follows every edge at that depth. The search succeeds if one branch succeeds. At the end of the query, the walk succeeds only if the node has the end flag. The state is `dfs(i, node)`: `node` matches `word[:i]`, and the call matches the rest. For example, with `bad` and `dad` stored, the query `.ad` tries the children `b` and `d` of the root. The branch through `b` reaches the end flag of `bad`, so the search returns `True`. Memory hook: *A letter follows one edge; a dot follows every edge.*

```
class TrieNode:
    def __init__(self):
        self.children = {}
        self.is_end = False

class WordDictionary:
    def __init__(self):
        self.root = TrieNode()

    def addWord(self, word):
        node = self.root
        for ch in word:
            if ch not in node.children:
                node.children[ch] = TrieNode()
            node = node.children[ch]
        node.is_end = True

    def search(self, word):
        def dfs(i, node):               # node matches word[:i]
            if i == len(word):
                return node.is_end
            ch = word[i]
            if ch == ".":               # branch: try every child
                return any(dfs(i + 1, child) for child in node.children.values())
            child = node.children.get(ch)
            return child is not None and dfs(i + 1, child)
        return dfs(0, self.root)

d = WordDictionary()
for w in ["bad", "dad", "mad"]:
    d.addWord(w)
d.search("pad"), d.search("bad"), d.search(".ad"), d.search("b..")   # -> (False, True, True, True)
d.search("..."), d.search("ba"), d.search("....")                    # -> (True, False, False)
```

Use the trie form when the statement asks for a trie, when you also need prefix queries, or when a query can have many dots. A trie search never visits more nodes than the trie has. In CPython the trie does not save memory here, because each node is an object with its own dict.

NOTE

The bound "at most 2 dots" does not make the trie search fast in the worst case. A crafted input with all 676 two-letter prefixes and two leading dots keeps 676 branches alive to the last letter.

The same walk handles other branching rules. A query position can allow a set of letters: follow each child in the set. A query can allow a budget of mismatches: carry the budget in the state, and spend it on a child with a different letter.

52.4.5 Trie plus grid backtracking

The task: find every word of a long list that a path on a letter grid spells (Word Search II). A path moves up, down, left or right, and it uses each cell at most one time in one word. One grid DFS for each word repeats the same work for shared prefixes, and $3 \cdot 10^4$ words make it too slow. Start from the one-word solution: a DFS from each cell that marks the cell and unmarks it on return. Then replace "compare with `word[i]`" by "step to the trie child". One DFS now tests all words at the same time, and a missing child ends the path. For example, with the words `oath` and `oat`, the path o-a-t reaches the end node of `oat`. The DFS records `oat` and continues, because the node still has the child `h`. Two tricks keep the search fast.

- Store the whole word at its end node. When the DFS reaches it, record the word and set the field to `None`. The DFS then reports each word one time, and it does not build path strings.
- On the way back, delete a node that has no children left. Later searches never enter a branch with nothing left to find.

Memory hook: *Word Search I, but walk a trie instead of one word, and cut empty leaves on the way back.*

```
class TrieNode:
    def __init__(self):
        self.children = {}      # letter to TrieNode
        self.word = None        # the whole word that ends here, if any

def find_words(board, words):
    root = TrieNode()
    for word in words:
        node = root
        for ch in word:
            node = node.children.setdefault(ch, TrieNode())
        node.word = word
    rows, cols = len(board), len(board[0])
    found = []

    def dfs(r, c, parent):
        if r < 0 or c < 0 or r == rows or c == cols:
            return                          # off the board
        ch = board[r][c]
        if ch not in parent.children:
            return                          # no word goes on ('#' marks a used cell)
        node = parent.children[ch]
        if node.word:
            found.append(node.word)
            node.word = None                # report each word once
        board[r][c] = "#"                   # mark the cell as used on this path
        dfs(r + 1, c, node)
        dfs(r - 1, c, node)
        dfs(r, c + 1, node)
        dfs(r, c - 1, node)
        board[r][c] = ch                    # backtrack: restore the cell
        if not node.children:
            del parent.children[ch]         # prune: nothing left to find here

    for r in range(rows):
        for c in range(cols):
            dfs(r, c, root)
    return found

board = [list("oaan"), list("etae"), list("ihkr"), list("iflv")]
sorted(find_words(board, ["oath", "pea", "eat", "rain"]))   # -> ['eat', 'oath']
board[0]                                                    # -> ['o', 'a', 'a', 'n']
find_words([list("ab"), list("cd")], ["abdc", "abcb"])      # -> ['abdc']
sorted(find_words([list("aa")], ["a", "aa", "aaa"]))        # -> ['a', 'aa']
```

The DFS state is (cell, trie node). The letters on the current path spell the string from the root to the node. The marker `#` is never a trie key, so the child test also blocks a second visit to a cell. The recursion depth is at most one more than the length of the longest word. The general rule: when a search consumes its targets, shrink the target structure as you find them. The worst case is still exponential in the word length. An all-`a` board with the words `aaaaaaab` to `aaaaaaak` keeps long paths alive. The DFS finds no word, so it prunes nothing. A form that checks the bounds and the trie child before each call makes fewer calls. In CPython it is not clearly faster, so write the form that you remember. Keep the leaf prune in both forms.

NOTE

Other searches use the same pair (search state, trie node). A Boggle grid with 8 directions changes only the neighbor list. If cells can repeat, drop the used mark: the longest word still limits the depth. To count paths instead of words, do not clear words and do not prune.

52.4.6 Walk the trie from each start index

To find many dictionary words inside one string, walk the trie from each start index i. Stop at the first missing child. Each end flag on the way marks a match `s[i:j + 1]`. The cost is $O(n \cdot L_{\max})$, where $L_{\max}$ is the length of the longest word. A set of words must test every end index j. The trie walk stops at the first missing child. Word Break uses this walk to fill a DP table over the indexes. Memory hook: *From each start, walk until the trie says no.*

```
def word_break(s, words):
    root = {}
    for w in words:
        node = root
        for ch in w:
            node = node.setdefault(ch, {})
        node["$"] = True
    n = len(s)
    ok = [True] + [False] * n           # ok[i]: s[:i] splits into words
    for i in range(n):
        if not ok[i]:
            continue
        node = root
        for j in range(i, n):           # walk the trie along s[i:]
            node = node.get(s[j])
            if node is None:
                break
            if "$" in node:
                ok[j + 1] = True
    return ok[n]

word_break("leetcode", ["leet", "code"])                          # -> True
word_break("catsandog", ["cats", "dog", "sand", "and", "cat"])    # -> False
```

When the questions are about suffixes, insert the reversed keys and walk the string from its end. For a very long text and many patterns, the Aho-Corasick automaton adds failure links to the trie and matches in linear time. Interview problems rarely need it.

52.4.7 Binary trie for XOR

The keys can be integers. Write each integer as a fixed number of bits, from the highest bit to the lowest. The trie then has at most two children per node. To get the largest `x ^ y` over the stored `y`, walk down from the root. At each bit, take the child with the opposite bit if it exists, because a high bit outweighs all lower bits. For example, $3 = 00011$ and $25 = 11001$ differ in the top bit, so their XOR is at least $2^4 = 16$. Memory hook: *Bits from the top; take the opposite bit when it exists.*

```
def max_xor_pair(nums):
    B = max(nums).bit_length()          # the number of bits to use
    root, best = {}, 0
    for x in nums:
        node = root
        for b in range(B - 1, -1, -1):  # insert x, highest bit first
            node = node.setdefault((x >> b) & 1, {})
        node, cur = root, 0
        for b in range(B - 1, -1, -1):  # query: prefer the opposite bit
            bit = (x >> b) & 1
            if 1 - bit in node:
                cur |= 1 << b
```

```
                node = node[1 - bit]
            else:
                node = node[bit]
        best = max(best, cur)
    return best

max_xor_pair([3, 10, 5, 25, 2, 8])      # -> 28
max_xor_pair([7])                       # -> 0
```

52.5 Adapt it to a new problem

Ask these questions in this order.

1. What are the keys, and what is the alphabet? Letters, digits, bits from the highest bit, the parts of a path. If the questions are about suffixes, reverse the keys.
2. Which operations need prefixes? If every query is an exact lookup, use a hash set and stop.
3. Do the stored strings change? A list that does not change can use a sorted list and `bisect`.
4. What must each node hold? An end flag, a pass count, the whole word, an index, a sum, a minimum. Update this data at each node on the path during the insert.
5. Does a query branch? A wildcard, a grid neighbor and a budget of edits all branch. Write a DFS over the state (position in the query or cell, trie node). A missing child ends the branch.
6. Is the number of branches small and bounded? Then a set of masked patterns can be simpler than a trie.
7. Do found targets leave the search? Clear the stored word, and delete empty leaves on the way back.
8. Check the cost. The trie has at most one node per inserted letter. Each walk is $O(L)$. The recursion depth grows with the length of the longest key, not with the number of keys.
9. Write the code in layers. First the insert and the plain walk of the core template. Then the node data. Then the branching DFS. Last, the pruning.

52.6 Pitfalls

CAUTION

A path is not a word. After an insert of `apple` only, `search("app")` must return `False`. Return the end flag of the last node, not "the walk finished".

CAUTION

A query that creates nodes. Use `setdefault` or a `defaultdict` only in the insert. In a query, they add empty paths. A later `startsWith` then returns `True` for a word that was never inserted.

```
from collections import defaultdict
def make_node():
    return defaultdict(make_node)
root = make_node()
node = root
for ch in "cat":
    node = node[ch]                     # insert "cat"
node["$"] = True
def starts_with_bad(root, p):
    node = root
    for ch in p:
        node = node[ch]                 # creates a node when ch is missing
    return True
def starts_with_good(root, p):
    node = root
    for ch in p:
        if ch not in node:
            return False
        node = node[ch]
    return True
starts_with_good(root, "dog")           # -> False
```

```
starts_with_bad(root, "dog")          # -> True
starts_with_good(root, "dog")         # -> True
```

The last line shows the damage: the bad query added the path d-o-g, so the correct query now returns `True` too.

CAUTION

One dict for all nodes. A class attribute `children = {}` or a mutable default argument shares one dict between all nodes. Create a new dict in `__init__` for each node.

```
class BadNode:
    children = {}                     # one dict for the class, not for each node
a, b = BadNode(), BadNode()
a.children["x"] = BadNode()
"x" in b.children                     # -> True
```

CAUTION

Wildcard rules. A dot matches exactly one letter, so the lengths must be equal. At a dot, try every child, and return `True` if any branch succeeds. Return that result at once: the recursive calls already match the rest of the query. Check for the dot before you compute a list index such as `ord(ch) - ord("a")`.

CAUTION

Grid search with a trie.

- Restore the cell after the four recursive calls. Otherwise later searches see a damaged board.
- Do not return after the first word at a node. A longer word can continue the path (`cat` and `cats`).
- A word can be reachable along several paths. Clear the stored word, or use a set, so each word appears once.
- Delete a node only from its parent, and only after its own recursion returns.
- Use a used-cell marker that is never a trie key, such as `#` for a trie of lowercase letters.
- Record a one-letter word at the start cell, before any step to a neighbor.

NOTE

Python node objects are large. A list of 26 children per node wastes memory when few words share prefixes. Prefer dict children, or add `__slots__` to the node class. A nested-dict trie is often smaller and faster than a trie of node objects.

52.7 Problems in this family

Problem	What changes	Memory hook
Implement Trie (Prefix Tree)	The core template: insert, a whole-word search that reads the end flag, and a prefix search that does not.	One node per prefix: walk the letters, flag the word end.
Design Add and Search Words Data Structure	A dot matches any one letter, at most 2 dots. Store masked patterns in a set, or run a trie DFS that tries every child at a dot.	Mask one letter on add, fill one dot on search. (Trie: a letter follows one edge; a dot follows every edge.)
Word Search II	Many words on one grid. The grid DFS walks a trie, stores each word at its end node, clears it when found, and cuts empty leaves.	Word Search I, but walk a trie instead of one word, and cut empty leaves on the way back.

The transfer drills for this technique are in Section 69.10 (page 421). The answers are in Section A.9 (page 443).

53 Heaps and priority queues

53.1 The idea

A heap is a list with one rule: each parent is less than or equal to its children. This rule puts the smallest item at index 0. The other items are only partly ordered. A push or a pop moves one item along one path between the root and a leaf, so each costs $O(\log n)$. A new sort costs $O(n \log n)$ and a new scan costs $O(n)$ each time the data changes. The heap does only the work that is necessary to know the next smallest item. Use a heap when a collection changes again and again, and each step needs only its best item.

Most heap solutions have one of three shapes:

- **Keep the k best.** A heap of size k holds the best items so far. Its root is the weakest kept item. The root is the gate that a new item must beat.
- **Pop the best, push what follows.** The heap holds the candidates for the next step. Each pop can make new candidates: the next item of the same list, a combined stone, a task that is ready again.
- **Two heaps at a boundary.** A max-heap holds the lower part and a min-heap holds the upper part. The two roots face each other, so a middle rank is always at the top.

Worked example. Find the 3rd largest value after each value of the stream 5, 1, 8, 3, 9. Keep a min-heap of at most 3 values. After 5, 1 and 8, the heap holds $\{1, 5, 8\}$, and the root 1 is the answer. Then 3 arrives. Push it, so the heap holds 4 values. Pop the root 1. The value 1 can never be in the top 3 again, because values are only added. The heap is $\{3, 5, 8\}$, and the answer is the root 3. Then 9 arrives. Push it and pop 3. The heap is $\{5, 8, 9\}$, and the answer is 5. Each step costs $O(\log k)$. A sort of the full history at each step costs $O(n \log n)$.

53.2 Recognize it

- The statement asks for the kth largest, the kth smallest, the top k, or the k closest items. It does not ask for the full order.
- Values arrive one at a time, and a query follows each arrival. One sort at the end is not possible.
- A process repeatedly takes the largest or the smallest item, and then it puts a new item back.
- Equal items must stay at least n positions apart. The output is the shortest length or one valid arrangement.
- Several inputs are each sorted, and the output is one combined order, or only the first k items of it.
- The query asks for the median or another rank that moves as the data grows.
- Items become active at one time and expire at a later time. Each query needs the best active item.
- The statement asks for the minimum number of resources (rooms, servers, machines) for overlapping jobs.
- A scarce resource (ladders, refuel stops, days before a deadline) can cover only some steps of a sequence. The goal is to go as far as possible, or to fit as many items as possible.
- The limits allow $O(n \log n)$ but not $O(n^2)$, and k or the number of sources is much smaller than n.

53.3 Core template

53.3.1 The heapq toolkit

The module `heapq` works on a plain list. Its classic functions give only a min-heap. `heap[0]` is the smallest item. The rest of the list is not sorted.

```
import heapq
h = []
for x in [5, 1, 8, 3]:
    heapq.heappush(h, x)           # O(log n)
h[0]                               # -> 1
heapq.heappop(h)                   # -> 1
h[0]                               # -> 3
a = [9, 4, 7, 1]
heapq.heapify(a)                   # O(n), in place, returns None
a[0]                               # -> 1
heapq.heapreplace(a, 6)            # -> 1
heapq.heappushpop(a, 0)            # -> 0
sorted(a)                          # -> [4, 6, 7, 9]
heapq.nlargest(2, [5, 1, 8, 3])                    # -> [8, 5]
heapq.nsmallest(2, ["bb", "a", "ccc"], key=len)    # -> ['a', 'bb']
```

- `heapreplace(h, x)`: pop the root, then push `x`. It returns the old root, even when `x` is smaller.
- `heappushpop(h, x)`: push `x`, then pop. It returns `x` at once when `x` is not larger than the root.

Each call does one sift instead of two.

To get a max-heap, store the negated key. Negate again when you read it. To sort by a key and keep a payload, push a tuple. Tuples compare field by field.

```
import heapq
h = [-x for x in [5, 1, 8, 3]]      # store -x to get a max-heap
heapq.heapify(h)
-h[0]                               # -> 8
-heapq.heappop(h)                   # -> 8
-h[0]                               # -> 5
jobs = [(2, "write"), (1, "read"), (2, "exec")]
heapq.heapify(jobs)
heapq.heappop(jobs)                 # -> (1, 'read')
heapq.heappop(jobs)                 # -> (2, 'exec')
```

If two keys are equal, Python compares the next field. An object without an order then raises `TypeError`. Put a unique number between the key and the payload: an index or a counter. A value that you cannot negate, such as a string, needs a small wrapper class with a reversed `__lt__`.

```
import heapq
class Node:
    pass
h = []
heapq.heappush(h, (1, Node()))
heapq.heappush(h, (1, Node()))      # -> raises TypeError
h = []
for i, node in enumerate([Node(), Node()]):
    heapq.heappush(h, (1, i, node))    # the unique index i breaks the tie
heapq.heappop(h)[1]                 # -> 0

class Desc:
    """Wrap a value so that the heap pops the largest value first."""
    def __init__(self, v):
        self.v = v
    def __lt__(self, other):
        return self.v > other.v
words = [Desc(w) for w in ["pear", "apple", "fig"]]
heapq.heapify(words)
heapq.heappop(words).v              # -> 'pear'
```

NOTE

Python 3.14 adds public max-heap functions, for example `heapq.heappush_max` and `heapq.heappop_max` [Python 3.14+]. Many judges run an older version. Use negation for portable code.

53.3.2 Template 1: keep the k best

```
import heapq

def k_largest(items, k):
    top = []                        # min-heap: the k largest items so far
    for x in items:
        heapq.heappush(top, x)
        if len(top) > k:
            heapq.heappop(top)      # drop the weakest; it cannot be in the top k
    return top                      # top[0] is the kth largest
```

```
top = k_largest([3, 2, 3, 1, 2, 4, 5, 5, 6], 4)
sorted(top)                          # -> [4, 5, 5, 6]
top[0]                               # -> 4
k_largest([7], 1)                    # -> [7]
```

Invariant: after each item, the heap holds the k largest items so far, duplicates included. The root is the smallest of them, so it is the kth largest. The heap has the opposite polarity of the request. For the k largest, keep a min-heap. For the k smallest or the k closest, keep a max-heap with negated keys. The cost is $O(n \log k)$ time and $O(k)$ memory, and one pass is enough, so it also works on a stream.

A faster gate is optional: when the heap is full, compare `x` with `top[0]` first. A value that cannot enter then costs $O(1)$. A value that enters uses one `heapreplace`.

53.3.3 Template 2: pop the best, push what follows

This template merges sorted sources. The heap holds one cursor per source.

```
import heapq

def merge_sorted(lists, limit=None):
    # One entry per source that still has items: (value, source, index).
    heap = [(lst[0], s, 0) for s, lst in enumerate(lists) if lst]
    heapq.heapify(heap)
    out = []
    while heap and (limit is None or len(out) < limit):
        value, s, i = heapq.heappop(heap)       # the smallest front
        out.append(value)
        if i + 1 < len(lists[s]):               # push the next item of the same source
            heapq.heappush(heap, (lists[s][i + 1], s, i + 1))
    return out

merge_sorted([[1, 4, 5], [1, 3, 4], [2, 6]])      # -> [1, 1, 2, 3, 4, 4, 5, 6]
merge_sorted([[1, 4, 5], [], [2, 6]], limit=3)    # -> [1, 2, 4]
merge_sorted([])                                  # -> []
list(heapq.merge([1, 4], [2, 3], [0]))            # -> [0, 1, 2, 3, 4]
```

Invariant: the heap holds exactly one entry for each source with unread items. That entry is the front of the source. No unread item is smaller than the front of its own source, so the smallest front is the next output. With N items in k sources, the cost is $O(N \log k)$. The same loop shape also runs simulations and sweeps: pop the best item, use it, and push the items that it makes. `heapq.merge` is a lazy built-in version for plain iterables.

53.4 Variations

53.4.1 The kth largest, the top k and the k closest

Kth Largest Element in an Array is Template 1. The root is the answer. Memory hook: *Top k in a min-heap: push, trim to k, return the root.* Quickselect gives $O(n)$ average time. A fixed first or last pivot degrades to $O(n^2)$ on sorted input, so pick a random pivot. Many equal values also need a three-way partition.

Kth Largest Element in a Stream keeps the same heap alive between calls. The constructor sends each initial value through `add`, so one gate handles every value. Memory hook: *Keep the top k in a min-heap. Every value goes through add: push, pop if over k, return the root.*

```
import heapq

class KthLargest:
    def __init__(self, k, nums):
        self.k = k
        self.top = []               # min-heap of the k largest values so far
        for x in nums:
            self.add(x)             # every value goes through the same gate

    def add(self, val):
```

```
        heapq.heappush(self.top, val)
        if len(self.top) > self.k:
            heapq.heappop(self.top)
        return self.top[0]

s = KthLargest(3, [4, 5, 8, 2])
[s.add(v) for v in [3, 5, 10, 9, 4]]     # -> [4, 5, 5, 8, 8]
KthLargest(1, []).add(-3)                # -> -3
```

K Closest Points to Origin asks for the k smallest keys. The simplest solution sorts and slices. Memory hook: *Sort by x*x + y*y, keep the first k.* The size-k heap version keeps a max-heap with negated keys, so the root is the farthest kept point. It costs $O(n \log k)$ instead of $O(n \log n)$, and it works on a stream.

```
import heapq

def k_closest_sorted(points, k):
    return sorted(points, key=lambda p: p[0] * p[0] + p[1] * p[1])[:k]

def k_closest_heap(points, k):
    heap = []                           # max-heap by distance: root = farthest kept point
    for i, (x, y) in enumerate(points):
        heapq.heappush(heap, (-(x * x + y * y), i, [x, y]))
        if len(heap) > k:
            heapq.heappop(heap)                # drop the farthest point
    return [p for _, _, p in heap]

pts = [[3, 3], [5, -1], [-2, 4]]
k_closest_sorted(pts, 2)                       # -> [[3, 3], [-2, 4]]
sorted(k_closest_heap(pts, 2))                 # -> [[-2, 4], [3, 3]]
heapq.nsmallest(2, pts, key=lambda p: p[0] * p[0] + p[1] * p[1])   # -> [[3, 3], [-2, 4]]
```

The squared distance is a reusable trick. You can replace a key with any strictly increasing function of it. The order does not change, and integer math has no rounding error.

53.4.2 Simulation: take the largest, put back the result

A process takes the largest (or smallest) items, combines them, and puts a result back. The order changes after each step, so a sort at the start is not enough. A heap gives each step in $O(\log n)$. Last Stone Weight is the model. Memory hook: *Negate into a heap, smash the top two, push back only a nonzero difference.*

```
import heapq

def last_stone_weight(stones):
    heap = [-s for s in stones]                # max-heap by negation
    heapq.heapify(heap)
    while len(heap) > 1:
        y = -heapq.heappop(heap)               # heaviest
        x = -heapq.heappop(heap)               # second heaviest
        if y != x:
            heapq.heappush(heap, -(y - x))     # push back only a nonzero rest
    return -heap[0] if heap else 0

last_stone_weight([2, 7, 4, 1, 8, 1])         # -> 1
last_stone_weight([3, 3])                      # -> 0
last_stone_weight([1])                         # -> 1
```

Bound the number of steps: here each step removes at least one item, so there are at most $n - 1$ steps. The same frame joins the two smallest items at a minimum total cost: use a min-heap and push the sum.

CAUTION

The heap greedy is correct here because the statement gives the rule. If the problem lets you choose the items, the greedy can be wrong. Last Stone Weight II is a subset-sum problem. Test a greedy rule against a brute force on small inputs first.

53.4.3 Greedy scheduling with a cooldown

Between two runs of the same kind, at least n other intervals must pass. Only the count of each kind matters. The most frequent kind is the bottleneck, so always run the most frequent kind that is ready. Task Scheduler is the model. The simple form works in rounds: one round is one window of $n+1$ slots. In one window, each kind can run at most one time. So fill each round with the $n+1$ most frequent kinds, or with all kinds if there are fewer. A full round costs $n+1$, and the last round costs only the tasks in it. Example: for AAABBB with $n=2$, the rounds are A B idle, A B idle, and A B. The total is $3+3+2=8$. Memory hook: *Fill each window of $n+1$ with the most frequent different tasks. Only the last window has no idle tail.*

```
import heapq
from collections import Counter

def least_interval(tasks, n):
    heap = [-c for c in Counter(tasks).values()]    # max-heap of remaining counts
    heapq.heapify(heap)
    time = 0
    while heap:
        ran = []
        while heap and len(ran) < n + 1:            # one round = one window of n + 1 slots
            ran.append(heapq.heappop(heap) + 1)     # run one copy (counts are negative)
        for c in ran:
            if c:                                   # copies remain
                heapq.heappush(heap, c)
        time += n + 1 if heap else len(ran)         # the last round has no idle tail
    return time

def least_interval_formula(tasks, n):
    counts = Counter(tasks).values()
    m = max(counts)
    ties = sum(1 for c in counts if c == m)
    return max(len(tasks), (m - 1) * (n + 1) + ties)

tests = [("AAABBB", 2), ("ACABDB", 1), ("AAABBB", 3), ("AAABBBCCCDDE", 2), ("AAAAB", 0)]
[least_interval(list(t), n) for t, n in tests]            # -> [8, 6, 10, 12, 5]
[least_interval_formula(list(t), n) for t, n in tests]    # -> [8, 6, 10, 12, 5]
```

The closed form is the fastest to type: $\max(N, (m-1)(n+1)+t)$. Here N is the number of tasks, m is the top count, and t is the number of kinds with count m.

The event form runs one interval at a time. Ready kinds wait in a max-heap. Kinds that cool down wait in a FIFO queue with the last interval in which they are still blocked. A kind that ran in interval t is blocked up to interval $t+n$, and it can run again in interval $t+n+1$. The queue is FIFO because every kind cools for the same time. With different cooldowns, use a min-heap by the last blocked interval. When nothing is ready, jump the clock to the last blocked interval of the next kind. Do not step through each idle interval. Use this form when the problem asks for the schedule itself.

```
import heapq
from collections import Counter, deque

def least_interval_events(tasks, n):
    heap = [-c for c in Counter(tasks).values()]
    heapq.heapify(heap)
    cooling = deque()                   # (negated count left, last blocked interval)
    time = 0                            # intervals used so far
    while heap or cooling:
```

```
        if not heap:
            time = cooling[0][1]        # nothing is ready: skip the idle intervals
        else:
            time += 1                   # run the most frequent ready kind
            c = heapq.heappop(heap) + 1
            if c:
                cooling.append((c, time + n))
        while cooling and cooling[0][1] <= time:
            heapq.heappush(heap, cooling.popleft()[0])
    return time

least_interval_events(list("AAABBB"), 2)            # -> 8
least_interval_events(list("AAABBBCCCDDE"), 2)      # -> 12
least_interval_events(list("AAAAB"), 0)             # -> 5
least_interval_events(["A"] * 3, 100)               # -> 203
```

53.4.4 Two heaps for a running median

The median depends only on the boundary between the lower half and the upper half. Keep the lower half in a max-heap (`small`) and the upper half in a min-heap (`large`). The two roots are the two values next to the boundary. A sorted list with `bisect.insort` costs $O(n)$ for each insert. The two heaps cost $O(\log n)$. Example: after 5, 15 and 1, `small` holds 1 and 5, and `large` holds 15. The root of `small` is 5, and 5 is the median. Find Median from Data Stream is the model. Memory hook: *Push into small, pass its top to large, give one back if large gets bigger.*

```
import heapq

class MedianFinder:
    def __init__(self):
        self.small = []     # max-heap (negated) of the lower half; it holds the extra item
        self.large = []     # min-heap of the upper half

    def addNum(self, num):
        heapq.heappush(self.small, -num)                          # route num through small
        heapq.heappush(self.large, -heapq.heappop(self.small))   # its largest moves up
        if len(self.large) > len(self.small):
            heapq.heappush(self.small, -heapq.heappop(self.large))

    def findMedian(self):
        if len(self.small) > len(self.large):
            return float(-self.small[0])
        return (-self.small[0] + self.large[0]) / 2

m = MedianFinder()
out = []
for x in [5, 15, 1, 3, 8]:
    m.addNum(x)
    out.append(m.findMedian())
out                                 # -> [5.0, 10.0, 5.0, 4.0, 5.0]
```

Invariants: every value in `small` is less than or equal to every value in `large`. `len(small)` is equal to `len(large)` or to `len(large) + 1`. The insert routes the new value through one side, and that side passes its extreme value to the other side. So the insert never compares the new value with a root. Reuse this routing step for any split into two parts.

The size relation sets the rank. For a fixed rank k from one end, one heap of size k is enough (Template 1). For a percentile p, keep the lower heap at $\lceil p \cdot n \rceil$ items. A related shape moves items from one heap to the other when a threshold grows. For example, a min-heap by required capital releases projects into a max-heap by profit (IPO).

53.4.5 K-way merge: one cursor per sorted source

Merge k Sorted Lists is Template 2 on linked lists. The tuple is `(value, list index, node)`. The index breaks ties, so Python never compares two nodes. Memory hook: *Heap the fronts; pop the smallest, push its next.*

```
import heapq

class ListNode:
    def __init__(self, val=0, next=None):
        self.val = val
        self.next = next

def merge_k_lists(lists):
    heap = [(node.val, i, node) for i, node in enumerate(lists) if node]
    heapq.heapify(heap)
    dummy = tail = ListNode()
    while heap:
        _, i, node = heapq.heappop(heap)
        tail.next = node
        tail = node
        if node.next:
            heapq.heappush(heap, (node.next.val, i, node.next))
    return dummy.next

def build(values):
    dummy = tail = ListNode()
    for v in values:
        tail.next = ListNode(v)
        tail = tail.next
    return dummy.next

def to_list(node):
    out = []
    while node:
        out.append(node.val)
        node = node.next
    return out

to_list(merge_k_lists([build([1, 4, 5]), build([1, 3, 4]), build([2, 6])]))
# -> [1, 1, 2, 3, 4, 4, 5, 6]
merge_k_lists([]) is None, merge_k_lists([None]) is None     # -> (True, True)
```

Merging the lists one at a time into a growing result costs $O(Nk)$. Merging them in pairs, round after round, costs $O(N \log k)$, as the heap does.

Design Twitter is a k-way merge inside a design problem. The feed is the first 10 items of a merge of the timelines of the user and the followees. Each timeline is sorted by time, because posts are appended in time order. Tweet ids do not show the post order, so stamp each tweet with a global counter. The heap holds one cursor `(-time, tweet id, user, index)` per user and walks each list from the end. Memory hook: *Stamp each tweet with a global clock. Merge the timelines with a heap: one cursor per user, pop the newest, push that user's previous tweet, stop at ten.*

```
import heapq
from collections import defaultdict

class Twitter:
    def __init__(self):
        self.time = 0
        self.tweets = defaultdict(list)     # user -> [(time, tweet id)], oldest first
        self.following = defaultdict(set)

    def postTweet(self, userId, tweetId):
        self.tweets[userId].append((self.time, tweetId))   # global clock stamp
        self.time += 1
```

```
    def getNewsFeed(self, userId):
        heap = []
        for uid in self.following[userId] | {userId}:
            ts = self.tweets.get(uid)
            if ts:                                          # cursor at the newest tweet
                i = len(ts) - 1
                heap.append((-ts[i][0], ts[i][1], uid, i))
        heapq.heapify(heap)
        feed = []
        while heap and len(feed) < 10:
            _, tweet_id, uid, i = heapq.heappop(heap)
            feed.append(tweet_id)
            if i > 0:                                       # the previous tweet of uid
                t, tid = self.tweets[uid][i - 1]
                heapq.heappush(heap, (-t, tid, uid, i - 1))
        return feed

    def follow(self, followerId, followeeId):
        if followerId != followeeId:
            self.following[followerId].add(followeeId)

    def unfollow(self, followerId, followeeId):
        self.following[followerId].discard(followeeId)

tw = Twitter()
tw.postTweet(1, 5)
tw.follow(1, 2)
tw.postTweet(2, 6)
tw.getNewsFeed(1)                   # -> [6, 5]
tw.unfollow(1, 2)
tw.getNewsFeed(1)                   # -> [5]
for t in range(12):
    tw.postTweet(3 if t % 2 else 1, 100 + t)
tw.follow(1, 3)
tw.getNewsFeed(1)                   # -> [111, 110, 109, 108, 107, 106, 105, 104, 103, 102]
```

A feed costs $O(F + 10 \log F)$ for F followees. It does not depend on the total number of tweets. The sources can also be implicit: rows of a sorted matrix, or the sums of one fixed element with a sorted array. Push the first item of each source, and create the next item only when its predecessor is popped.

53.4.6 Intervals: count the overlap, or keep a heap of end times

The minimum number of resources for intervals equals the largest number of intervals that overlap at one time. Meeting Rooms II is the model. The simplest solution needs no heap: a sweep over $+1$ and -1 events. Memory hook: *Start is +1, end is -1, sort the events, and keep the peak.* The heap form sorts the meetings by start and keeps a min-heap of the end times of the rooms. The root is the room that is free first. Use it when you must know which room frees first, for example to give room numbers or to delay a meeting.

```
import heapq

def min_rooms_sweep(intervals):
    events = []
    for start, end in intervals:
        events.append((start, 1))
        events.append((end, -1))
    events.sort()                       # at equal times, (t, -1) comes before (t, 1)
    busy = most = 0
    for _, change in events:
        busy += change
        most = max(most, busy)
    return most
```

```
def min_rooms_heap(intervals):
    ends = []                           # min-heap: end time of the last meeting in each room
    for start, end in sorted(intervals):
        if ends and ends[0] <= start:
            heapq.heapreplace(ends, end)    # reuse the room that is free first
        else:
            heapq.heappush(ends, end)       # all rooms are busy: open a new room
    return len(ends)

meetings = [[0, 30], [5, 10], [15, 20]]
min_rooms_sweep(meetings), min_rooms_heap(meetings)      # -> (2, 2)
min_rooms_sweep([[7, 10], [2, 4]])                       # -> 1
min_rooms_heap([[1, 5], [5, 8]])                         # -> 1
min_rooms_heap([[1, 5], [2, 6], [3, 7], [6, 9]])         # -> 3
```

The tie rule depends on the endpoints. Here a meeting is the half-open range $[s, e)$, so a meeting that ends at t frees its room for a meeting that starts at t. For closed intervals, change `<=` to `<` and put the start events first.

53.4.7 Offline queries: sort, sweep, release into a heap

Many queries ask for the best item that is active at a point. A scan of all items for each query costs $O(nm)$. All queries are known in advance, so you can answer them in any order. Sort the queries, and sort the items by start. A pointer releases each item into a heap when the sweep passes its start. The heap key is the quality to optimize. The queries only grow. So an item that ended before one query also ended before all later queries. An item that ended stays in the heap until it reaches the root. Then pop it. Minimum Interval to Include Each Query is the model. Memory hook: *Sweep sorted queries, push started intervals by size, pop ended tops, read the top.*

```
import heapq

def min_interval(intervals, queries):
    intervals = sorted(intervals)                          # by left end
    order = sorted(range(len(queries)), key=lambda j: queries[j])
    ans = [-1] * len(queries)
    heap = []                                          # (size, right) of started intervals
    i = 0
    for j in order:                                        # queries from small to large
        q = queries[j]
        while i < len(intervals) and intervals[i][0] <= q:    # release started intervals
            left, right = intervals[i]
            heapq.heappush(heap, (right - left + 1, right))
            i += 1
        while heap and heap[0][1] < q:                     # the top has ended: remove it now
            heapq.heappop(heap)
        if heap:
            ans[j] = heap[0][0]
    return ans

min_interval([[1, 4], [2, 4], [3, 6], [4, 4]], [2, 3, 4, 5])        # -> [3, 3, 1, 4]
min_interval([[2, 3], [2, 5], [1, 8], [20, 25]], [2, 19, 5, 22])    # -> [2, -1, 4, 6]
```

Each interval enters the heap one time and leaves it at most one time. The cost is $O(n \log n + m \log m)$. Sort a list of query indexes, not the queries, so each answer goes to its original position. The same frame works with a clock instead of queries: jobs arrive at given times, and a free machine takes the best job that has arrived. When the heap is empty, jump the clock to the next arrival.

53.4.8 Take back the cheapest choice

A scarce resource can cover only some steps of a sequence. The other steps cost a second resource. The best steps for the scarce resource depend on steps that come later, so a choice at each step can be wrong. Do not try to decide early. Give each step the scarce resource first, and keep those steps in a heap. When the resource runs out,

take back the step that gains the least, and pay it another way. The heap holds the best use of the resource for the prefix so far. Furthest Building You Can Reach (LeetCode 1642) is the model: a climb of d costs d bricks or one ladder. Memory hook: *Ladders go to the largest climbs so far.*

```
import heapq

def furthest_building(heights, bricks, ladders):
    on_ladder = []                          # min-heap: the climbs that use a ladder now
    for i in range(len(heights) - 1):
        d = heights[i + 1] - heights[i]
        if d <= 0:
            continue
        heapq.heappush(on_ladder, d)        # give the climb a ladder first
        if len(on_ladder) > ladders:
            bricks -= heapq.heappop(on_ladder)   # take back the smallest climb: pay it with bricks
            if bricks < 0:
                return i                    # the climb from i to i + 1 fails
    return len(heights) - 1

def schedule_course(courses):               # [duration, last day]
    taken = []                              # max-heap (negated) of the durations taken
    end = 0
    for duration, last_day in sorted(courses, key=lambda c: c[1]):
        heapq.heappush(taken, -duration)    # take the course first
        end += duration
        if end > last_day:
            end += heapq.heappop(taken)     # take back the longest course
    return len(taken)

furthest_building([4, 2, 7, 6, 9, 14, 12], 5, 1)            # -> 4
furthest_building([4, 12, 2, 7, 3, 18, 20, 3, 19], 10, 2)   # -> 7
schedule_course([[100, 200], [200, 1300], [1000, 1250], [2000, 3200]])   # -> 3
schedule_course([[3, 2], [4, 3]])                            # -> 0
```

The building form is Template 1 with k equal to the number of ladders. The heap keeps the largest climbs so far, and each climb that leaves the heap is paid with bricks. For each prefix, the best plan gives the ladders to its largest climbs. So the heap plan fails only when no plan can pass. The cost is $O(n \log k)$. A binary search on the answer also works, but it sorts the climbs in each check.

The same frame has two other common forms:

- **Drop the worst item** (Course Schedule III, LeetCode 630). Sort the courses by last day. Take each course, and keep the durations in a max-heap. When the end passes the last day, drop the longest course. The count does not decrease, and the end becomes as early as possible for that count.
- **Choose late** (Minimum Number of Refueling Stops, LeetCode 871). Drive past the stations, and push their fuel into a max-heap. When the fuel cannot reach the next point, refuel at the best station already passed. If the heap is empty, the answer is -1.

53.4.9 Lazy deletion

`heapq` has no remove operation, and a search for an item in the middle costs $O(n)$. Do not search for the item. Mark it as dead, and pop it only when it reaches the root. Only the root is ever read, so a dead item deeper down does no harm. Memory hook: *Do not remove it now. Mark it dead, and pop it when it reaches the root.* There are three common forms:

- **Monotone expiry.** An item expires at a time, and time only grows. Pop the root while it has expired. The offline sweep above uses this form. A sliding window can store `(value, index)` and pop roots with an old index.
- **Removal by value.** Count the pending removals in a `Counter`. Before you read the root, pop it while its count is positive. Keep a separate count of the live items, because `len(heap)` includes dead items.
- **Changed priority.** Push a new entry and keep the old one. At the root, compare the entry with the current value (or a version number) in a dict. Pop it if it is old.

```python
import heapq
from collections import Counter

class LazyMinHeap:
    """A min-heap with remove(x) in O(log n) amortized time."""
    def __init__(self):
        self.heap = []
        self.dead = Counter()      # value -> copies to skip when they reach the root
        self.size = 0              # live items only

    def push(self, x):
        heapq.heappush(self.heap, x)
        self.size += 1

    def remove(self, x):           # x must be a live item
        self.dead[x] += 1
        self.size -= 1

    def top(self):
        while self.dead[self.heap[0]]:
            self.dead[heapq.heappop(self.heap)] -= 1
        return self.heap[0]

h = LazyMinHeap()
for x in [5, 1, 3, 1]:
    h.push(x)
h.remove(1)
h.top(), h.size                    # -> (1, 3)
h.remove(1)
h.top(), h.size                    # -> (3, 2)

price = {"a": 5, "b": 3}           # changed priority: a max-heap of current prices
heap = [(-p, name) for name, p in price.items()]
heapq.heapify(heap)
price["a"] = 1
heapq.heappush(heap, (-1, "a"))    # push the new entry, keep the old one
while -heap[0][0] != price[heap[0][1]]:
    heapq.heappop(heap)            # the root is old: drop it
heap[0][1]                         # -> 'b'
```

The cost stays $O(\log n)$ amortized, because each entry is pushed one time and popped at most one time.

53.5 Adapt it to a new problem

Ask these questions in this order.

1. What does the brute force repeat? Usually it sorts or scans again after each change to find the best item. The heap replaces that repeated step.
2. What does each step need: the minimum, the maximum, the kth item, or a middle rank? If the data does not change between queries, a sort or quickselect can be simpler than a heap.
3. Which item must leave first? Put that item at the root. For the k best, the root is the weakest kept item, so the heap has the opposite polarity of the request.
4. What is the key, and what breaks ties? Use `(key, unique id, payload)`. Use the cheapest key with the same order, for example a squared distance.
5. How many candidates are necessary at one time? Often one per source is enough (one front per list, one cursor per user).
6. Does the rank move as the data grows? Use two heaps and fix their size relation to the rank.
7. Do items leave for a reason other than a pop? Use lazy deletion: expiry at the root, a dead counter, or a version check.
8. Are all queries known in advance? Sort them, sort the items, and release items into the heap with a pointer.

9. Can a choice become wrong when later items arrive? Commit each choice, and keep the choices in a heap. When a budget breaks, take back the weakest choice.
10. Is the greedy choice proved? A rule in the statement is safe. A rule that you invent needs a brute-force check.
11. Count the heap operations. Each item should enter one time and leave one time. Jump the clock over idle time, so the loop count does not depend on the size of the time values.

53.6 Pitfalls

CAUTION

heappush and heappop build a min-heap. Negate the key on each push and negate it again on each read. A missing negation on one push, for example when a result goes back into the heap, puts one wrong value in the heap.

CAUTION

Wrong polarity for the k best. A max-heap of size k for the k largest evicts the best item, not the weakest. For the k largest, use a min-heap. For the k smallest or the k closest, use a max-heap.

CAUTION

Ties reach the payload. `(key, node)` raises `TypeError` when two keys are equal and the nodes have no order. Put a unique index after the key. Without a key in front, tuples of points compare by their coordinates, not by distance.

CAUTION

A heap is not sorted. `heap[1]` is not always the second smallest item, and a loop over the heap does not give sorted order. Pop the items, or sort them, to get an order. `heapify` returns `None` and changes the list in place.

CAUTION

Do not change an item inside the heap. A changed key breaks the heap rule without an error. Push a new entry and skip the old one when it reaches the root.

CAUTION

Empty heap. `heap[0]` and `heappop` raise `IndexError` on an empty heap. Test the heap before each read, for example after a loop that pops expired roots.

CAUTION

Integer division in a median. The median of an even count is `(a + b) / 2`. Do not use `//`. It rounds down, so the median of 2 and 3 becomes 2, and `-5 // 2` is `-3`.

CAUTION

Boundaries in time. With a gap of n between equal tasks, a task that runs at t can run again at $t + n + 1$. A half-open meeting that ends at t frees its room for a start at t: use `<=`. An inclusive interval with `right == q` still contains `q`: pop only when `right < q`.

CAUTION

Lost order and lost state. Sort query indexes, not the query list, or the answers go to the wrong positions. Do not reset the release pointer for each query, or the cost becomes $O(n \cdot m)$. With lazy deletion, `len(heap)` counts dead items. Keep a live count.

53.7 Problems in this family

Problem	What changes	Memory hook
Kth Largest Element in a Stream	Template 1 kept alive between calls; the constructor uses `add`.	Keep the top k in a min-heap. Every value goes through add: push, pop if over k, return the root.
Last Stone Weight	Simulation with a max-heap; pop two, push back the difference.	Negate into a heap, smash the top two, push back only a nonzero difference.
K Closest Points to Origin	The k smallest by squared distance; sort and slice, or a size-k max-heap.	Sort by x*x + y*y, keep the first k.
Kth Largest Element in an Array	Template 1 on a static array; the root is the answer.	Top k in a min-heap: push, trim to k, return the root.
Task Scheduler	Max-heap of counts in rounds of $n + 1$; a closed form also exists.	Fill each window of n + 1 with the most frequent different tasks. Only the last window has no idle tail.
Design Twitter	K-way merge of timelines with a global clock; stop at 10.	Stamp each tweet with a global clock. Merge the timelines with a heap: one cursor per user, pop the newest, push that user's previous tweet, stop at ten.
Find Median from Data Stream	Two heaps that face each other at the median.	Push into small, pass its top to large, give one back if large gets bigger.
Merge k Sorted Lists	Template 2 on linked lists; an index breaks ties.	Heap the fronts; pop the smallest, push its next.
Meeting Rooms II	Peak overlap; an event sweep, or a min-heap of end times.	Start is +1, end is -1, sort the events, and keep the peak.
Minimum Interval to Include Each Query	Offline sweep; release by start, key by size, pop ended roots.	Sweep sorted queries, push started intervals by size, pop ended tops, read the top.

The transfer drills for this technique are in Section 69.11 (page 421). The answers are in Section A.10 (page 446).

54 Backtracking

54.1 The idea

Some tasks ask for every answer: every subset, every ordering, every valid board. Build each answer as a sequence of decisions. These decisions form a tree. The root is the empty answer. Each level makes one decision, and each branch is one option for that decision. A node where the answer is complete is a leaf. It gives one answer.

Backtracking walks this tree depth first with one shared *path*. The path holds the decisions on the route from the root to the current node. For each option at a node, do three steps:

1. **Choose**: append the option to the path, and mark it as used if the problem needs marks.
2. **Explore**: call the function for the next level.
3. **Un-choose**: pop the option and remove its mark.

After the un-choose step, the state is the same as before the choose step. So the next option starts from a clean state.

Brute force builds every complete candidate and then tests it. Backtracking tests the rules on a *partial* answer. If a partial answer breaks a rule, no completion can repair it. So the search cuts the full subtree at once. This cut is *pruning*. With an exact pruning test, every branch ends in an answer. The work is then proportional to the size of the output.

Worked example. Find the multisets of values from `[2, 3, 6, 7]` with the sum 7. A value can repeat. Build each multiset in sorted order: the next value is never smaller than the last value. This rule gives each multiset exactly one route, so no answer comes out two times.

- Path `[2]`, remainder 5. Path `[2, 2]`, remainder 3. Path `[2, 2, 2]`, remainder 1. All values are larger than 1,

so prune.
- Un-choose the last 2. Choose 3: path `[2, 2, 3]`, remainder 0. Record a copy of the path.
- Back at `[2]`: choose 3, remainder 2. The value 3 is larger than 2, so prune. The value 6 is larger than 5, so stop the loop.
- Back at the root: `[3]`, then `[3, 3]` with remainder 1. Prune. Then `[6]` with remainder 1. Prune.
- Last, `[7]` gives remainder 0. Record it.

The result is `[[2, 2, 3], [7]]`. The search never builds `[3, 2, 2]`, because 2 cannot come after 3.

Size of the tree. The output alone is often exponential, so the input bounds are small. Estimate the tree before you write code. CPython does about 10^7 simple steps per second (Section 36). A backtracking call does more work than a simple step, so count each call as several steps.

Answer type	Number of answers	Usual bound on the input
Subsets of n elements	2^n	n up to about 20
Combinations of k from n	$\binom{n}{k}$	depends on k
Permutations of n elements	$n!$	n up to about 9
Balanced strings of n pairs	Catalan number, about $4^n/(n^{1.5}\sqrt{\pi})$	n up to about 12
One option for each of n slots	product of the option counts	product up to about 10^6
Grid paths of L cells, no cell twice	at most $4 \cdot 3^{L-2}$ from one start cell	small board, short word

54.2 Recognize it

- The task says "return all", "list every" or "generate all". The output can be exponential.
- The bounds are tiny: $n \leq 10$ to 20, a board of at most 6×6, a word of at most 15 letters.
- An answer is a sequence of decisions, and the number of decisions depends on the input. A fixed number of nested loops cannot build it.
- A partial answer can already break a rule: a sum above the target, an attacked square, a wrong letter, a piece that is not a palindrome. Then you can prune early.
- A path must not use an item or a cell two times. The state then includes the set of used items or cells. That set is usually too large to memoize. A bitmask memo works only for about 20 items or fewer.
- A yes or no question with such a path-dependent state: search, and stop at the first success.
- Counter-signal: the task asks only for a count, a minimum or a yes or no answer. If the rest of the search depends on a small state, use DP instead (Section 54.4.12). Examples of a small state are an index, a remaining sum and a mask of used items.

54.3 Core template

Every backtracking solution answers three questions. What is the state? Which options are legal at this state? When is the state complete?

```
def solve(data):
    res, path = [], []
    def backtrack(state):
        if is_complete(state):            # goal test
            res.append(path[:])           # record a COPY of the path
            return
        for option in options(state):
            if not legal(option, state):  # prune: continue, or break on sorted input
                continue
            path.append(option)           # choose (and mark)
            backtrack(next_state(state, option))   # explore
            path.pop()                    # un-choose (and unmark)
    backtrack(start_state)
    return res
```

The two most common shapes build the subsets of a list. The *take or skip* shape decides one element per level and records at the leaves. The *start index* shape picks the next element from the elements after the last pick. Every node of its tree is a valid subset, so it records at every node.

```python
def subsets_take_skip(nums):
    res, path = [], []
    def dfs(i):
        if i == len(nums):              # every element is decided
            res.append(path[:])
            return
        path.append(nums[i])            # take nums[i]
        dfs(i + 1)
        path.pop()
        dfs(i + 1)                      # skip nums[i]
    dfs(0)
    return res

def subsets_start(nums):
    res, path = [], []
    def dfs(start):
        res.append(path[:])             # every node is a subset
        for j in range(start, len(nums)):
            path.append(nums[j])        # choose
            dfs(j + 1)                  # explore: only later elements
            path.pop()                  # un-choose
    dfs(0)
    return res

subsets_take_skip([1, 2, 3])
# -> [[1, 2, 3], [1, 2], [1, 3], [1], [2, 3], [2], [3], []]
subsets_start([1, 2, 3])
# -> [[], [1], [1, 2], [1, 2, 3], [1, 3], [2], [2, 3], [3]]
```

Invariant. When a call returns, the path and every mark are the same as when the call started. Each answer matches exactly one route from the root, so the search finds it exactly one time.

Pick the shape from the kind of answer. The table maps each kind to the decision at one level.

Shape	One level decides	Next call; record when	Example
Take or skip	take or skip `nums[i]`	`i + 1`; `i == n`	Subsets
Start index	any `j >= start`	`j + 1`; every node, or a goal	Subsets, Combination Sum II
Start index, reuse	any `j >= start`	`j`; remainder 0	Combination Sum
Pool	any unused element	mark it; path length n	Permutations
Slots	one option of slot `i`	`i + 1`; `i == n`	Letter Combinations
Pieces	the end of the next piece	after the piece; at the end	Palindrome Partitioning
Placement	a value for the next slot, free of conflicts	next slot; all slots full	N-Queens
Grid path	a neighbor cell not on the path	next cell; word complete	Word Search
Buckets	a bucket for the next item	next item; all items placed	Partition to K Equal Sum Subsets

NOTE

When there is no pruning, the `itertools` module gives the same enumerations as tuples. Use it to write a brute force that checks a backtracking solution.

```python
from itertools import combinations, combinations_with_replacement, permutations, product
list(combinations([1, 2, 3], 2))        # -> [(1, 2), (1, 3), (2, 3)]
list(permutations([1, 2, 3], 2))        # -> [(1, 2), (1, 3), (2, 1), (2, 3), (3, 1), (3, 2)]
list(product("ab", "xy"))               # -> [('a', 'x'), ('a', 'y'), ('b', 'x'), ('b', 'y')]
list(combinations_with_replacement([2, 3], 2))   # -> [(2, 2), (2, 3), (3, 3)]
[c for r in range(4) for c in combinations([2, 3, 6, 7], r) if sum(c) == 9]
```

```
# -> [(2, 7), (3, 6)]
```

54.4 Variations

54.4.1 Subsets: take or skip each element

Each element is in or out, so there are 2^n subsets. The core template solves it. A shorter form is the *cascade*. Start with the empty subset. For each element, keep every old subset and add a copy of it with the element. **Memory hook** (Subsets): *Each number doubles the list. Keep the old subsets and add each one plus the number.*

```
def subsets(nums):
    res = [[]]                                  # the empty subset
    for x in nums:
        res += [subset + [x] for subset in res]  # old subsets, plus each one with x
    return res

subsets([1, 2, 3])
# -> [[], [1], [2], [1, 2], [3], [1, 3], [2, 3], [1, 2, 3]]
len(subsets(list(range(10))))                   # -> 1024
nums = [1, 2, 3]                                # the same order from a bitmask count
[[nums[i] for i in range(3) if mask >> i & 1] for mask in range(1 << 3)]
# -> [[], [1], [2], [1, 2], [3], [1, 3], [2, 3], [1, 2, 3]]
```

The comprehension builds the full new list before `+=` adds it, so it reads only the old subsets. A loop that appends to `res` while it reads `res` does not stop. The cascade cannot prune, because it builds every subset. Learn the backtracking form too: Combination Sum, Subsets II and Permutations add rules to it.

A fixed size k (Combinations) needs two changes in the start index shape. Record and return when `len(path) == k`. Loop only while enough elements remain: `range(start, n - (k - len(path)) + 1)`. **Memory hook** (Combinations): *Record at size k. Stop the loop when too few elements remain.*

54.4.2 Combinations under a sum, with reuse

The answer is a multiset with a target sum, and a value can repeat (Combination Sum). Use the start index shape so that each multiset has one sorted route. Recurse with the same index `j` to allow reuse. Recurse with `j + 1` for single use. Carry the remainder `remain = target - sum(path)`, and record when it is 0. All values are positive, so sort them first. Then the first value above the remainder ends the loop with `break`. **Memory hook** (Combination Sum): *Sort first. In dfs, save a copy at zero, then loop j from start: break if too big, pick, recurse with the same j, pop.*

```
def combination_sum(candidates, target):
    candidates = sorted(candidates)           # a copy: the caller's list stays the same
    res, path = [], []
    def dfs(start, remain):
        if remain == 0:
            res.append(path[:])
            return
        for j in range(start, len(candidates)):
            c = candidates[j]
            if c > remain:
                break                         # sorted: every later value is too big
            path.append(c)
            dfs(j, remain - c)                # the same j: c can repeat
            path.pop()
    dfs(0, target)
    return res

combination_sum([2, 3, 6, 7], 7)              # -> [[2, 2, 3], [7]]
combination_sum([2, 3, 5], 8)                 # -> [[2, 2, 2, 2], [2, 3, 3], [3, 5]]
combination_sum([2], 1)                       # -> []
```

The depth is at most `target // min(candidates)`. The run time grows with the number of partial combinations

(multisets with a sum of at most the target), not only with the answers.

54.4.3 Duplicate values: sort and skip equal siblings

The input can repeat a value, but the output must not repeat a group (Subsets II, Combination Sum II). Two equal values at different indexes give the same subtree. Sort the input so that equal values are adjacent. Then, inside one loop of one call, try each value only one time: skip index `j` when `j > start and nums[j] == nums[j - 1]`. A deeper call can still take the next copy, because that copy is at `j == start` in the child call. So `[1, 1, 6]` stays possible. **Memory hook** (Combination Sum II): *Sort, then each level tries each value once.*

```
def combination_sum2(candidates, target):
    candidates = sorted(candidates)
    res, path = [], []
    def backtrack(start, remain):
        if remain == 0:
            res.append(path[:])
            return
        for i in range(start, len(candidates)):
            if i > start and candidates[i] == candidates[i - 1]:
                continue                        # same level, same value: already tried
            if candidates[i] > remain:
                break
            path.append(candidates[i])
            backtrack(i + 1, remain - candidates[i])   # i + 1: each index once
            path.pop()
    backtrack(0, target)
    return res

combination_sum2([10, 1, 2, 7, 6, 1, 5], 8)
# -> [[1, 1, 6], [1, 2, 5], [1, 7], [2, 6]]
combination_sum2([1] * 100, 30) == [[1] * 30]   # one route, not C(100, 30) routes
# -> True
```

A subset with duplicates is fully described by the count of each distinct value. So the cascade also works with counts: every old subset takes 0, 1, ..., `count` copies of the next value. **Memory hook** (Subsets II): *Count each value. Every old subset takes 0 to count copies of it.*

```
from collections import Counter
def subsets_with_dup(nums):
    res = [[]]
    for value, count in sorted(Counter(nums).items()):
        res = [subset + [value] * k for subset in res for k in range(count + 1)]
    return res

subsets_with_dup([1, 2, 2])
# -> [[], [2], [2, 2], [1], [1, 2], [1, 2, 2]]
subsets_with_dup([0])                           # -> [[], [0]]
```

Assign the new list to `res`. Do not extend `res`, because `k = 0` already keeps each old subset. The backtracking form with the skip rule is the template for Combination Sum II, where a sum must prune the search.

The general rule: choose one *canonical form* for each answer, and generate only that form. The same idea has other forms:

- Permutations of values with duplicates: sort, and skip `nums[i]` when it equals `nums[i - 1]` and `nums[i - 1]` is not used on the path. Or loop over the distinct values of a `Counter`.
- The order of the input must stay (subsequences): you cannot sort. Keep a set of the values that the loop of this call already tried.
- Fixed positions in 3Sum and 4Sum: skip a value that equals the value before it at the same position.

54.4.4 Permutations: pick from the unused pool

The order matters, and each element appears exactly one time (Permutations). At each level, any unused element can come next. The loop starts at 0, not at a start index. The short form passes a new path and a smaller pool to each call. **Memory hook** (Permutations): *Pick one from the pool. Recurse on the rest. An empty pool means a finished permutation.*

```
def permute(nums):
    result = []
    def build(path, remaining):
        if not remaining:
            result.append(path)               # path is a new list in each call
            return
        for i in range(len(remaining)):
            build(path + [remaining[i]], remaining[:i] + remaining[i + 1:])
    build([], nums)
    return result

def permute_shared(nums):                     # one shared path and a used flag per index
    res, path, used = [], [], [False] * len(nums)
    def backtrack():
        if len(path) == len(nums):
            res.append(path[:])
            return
        for i in range(len(nums)):
            if used[i]:
                continue
            used[i] = True                    # choose
            path.append(nums[i])
            backtrack()                       # explore
            path.pop()                        # un-choose, in reverse order
            used[i] = False
    backtrack()
    return res

permute([1, 2, 3])
# -> [[1, 2, 3], [1, 3, 2], [2, 1, 3], [2, 3, 1], [3, 1, 2], [3, 2, 1]]
permute_shared([1, 2, 3]) == permute([1, 2, 3])   # -> True
```

Both forms take $O(n \cdot n!)$ time. The shared form needs only $O(n)$ extra space, not $O(n^2)$, along the recursion. It is also the usual template for a search with pruning, as in N-Queens.

54.4.5 One option list per slot

Each position has its own list of options, and the options do not limit each other (Letter Combinations of a Phone Number). The answer is a Cartesian product. A variable number of nested loops is a recursion with one level per slot. Without pruning, a layer expansion is enough: grow every prefix by every option of the next slot. **Memory hook** (Letter Combinations of a Phone Number): *Start with one empty string, and grow every prefix by every letter of the next digit.*

```
LETTERS = {"2": "abc", "3": "def", "4": "ghi", "5": "jkl",
           "6": "mno", "7": "pqrs", "8": "tuv", "9": "wxyz"}
def letter_combinations(digits):
    if not digits:
        return []                             # not [""]
    combos = [""]
    for d in digits:
        combos = [prefix + ch for prefix in combos for ch in LETTERS[d]]
    return combos

letter_combinations("23")
# -> ['ad', 'ae', 'af', 'bd', 'be', 'bf', 'cd', 'ce', 'cf']
```

```
len(letter_combinations("79"))                # -> 16
from itertools import product
["".join(p) for p in product(*(LETTERS[d] for d in "23"))] == letter_combinations("23")
# -> True
```

Use the backtracking form (append, recurse, pop) when an option at one slot limits the next slots, or when the search can stop early.

54.4.6 Generation with a prefix rule

A rule that every prefix must obey decides the legal options (Generate Parentheses). A string of one bracket type is well formed when no prefix has more closes than opens, and the counts are equal at the end. So the state is only two counts, not the full string. Make the pruning exact: allow a move only if the prefix can still become a valid answer. Then every branch reaches a valid leaf, and no final check is necessary. **Memory hook** (Generate Parentheses): *Open while opens remain. Close only what is open. Save at length 2n.*

```
def generate_parenthesis(n):
    result = []
    def build(text, opens, closes):
        if len(text) == 2 * n:
            result.append(text)
            return
        if opens < n:
            build(text + "(", opens + 1, closes)
        if closes < opens:
            build(text + ")", opens, closes + 1)
    build("", 0, 0)
    return result

generate_parenthesis(3)
# -> ['((()))', '(()())', '(())()', '()(())', '()()()']
[len(generate_parenthesis(n)) for n in range(1, 7)]   # -> [1, 2, 5, 14, 42, 132]
```

The new string at each call copies $O(n)$ characters. A shared list with one `join` at the leaf saves a constant factor only. For several bracket types, the state must hold the stack of open types, not only a count.

54.4.7 Partition a sequence into valid pieces

Split a string into contiguous pieces, and each piece must pass a local test (Palindrome Partitioning). One decision is where the next piece ends. Recurse on the rest of the string. Keep a piece only if it passes the test. This prunes a branch at its first bad piece. **Memory hook** (Palindrome Partitioning): *Cut a palindrome off the front, recurse on the rest.*

```
def partition(s):
    result, path = [], []
    def backtrack(start):
        if start == len(s):                   # the whole string is cut
            result.append(path[:])
            return
        for end in range(start + 1, len(s) + 1):
            piece = s[start:end]
            if piece == piece[::-1]:          # keep only palindrome pieces
                path.append(piece)
                backtrack(end)
                path.pop()
    backtrack(0)
    return result

partition("aab")                              # -> [['a', 'a', 'b'], ['aa', 'b']]
partition("a")                                # -> [['a']]
len(partition("a" * 10))                      # -> 512
```

A string of n equal letters has 2^{n-1} partitions, so no algorithm is polynomial here. The optimal form fills a table `pal[i][j]` before the search, so each test is one lookup. Fill `i` from `n - 1` down to 0: `pal[i][j] = s[i] == s[j] and (j - i < 2 or pal[i + 1][j - 1])`. It changes the bound from $O(n^2 \cdot 2^n)$ to $O(n \cdot 2^n)$. The same shape solves Word Break II, where each piece must be a dictionary word. When the answer needs exactly m pieces, add the number of open pieces to the state. Then prune a branch when the rest of the string is too short or too long for those pieces.

54.4.8 Constraint placement with conflict keys

Place items so that no two items conflict (N-Queens). Choose the level so that one rule holds by construction: one queen per row, so one level per row. Then turn every other conflict line into a key that is constant on that line. A column has the key `c`. On a diagonal from top-left to bottom-right, `r - c` is constant. On the other diagonal, `r + c` is constant. Keep one set of used keys for each kind of line. **Memory hook** (N-Queens): *One queen per row. Block the column, r minus c and r plus c.*

```
def solve_n_queens(n):
    cols, diag, anti = set(), set(), set()    # c, r - c, r + c of the placed queens
    queens, res = [], []                      # queens[r] = column of the queen in row r
    def backtrack(r):
        if r == n:
            res.append(["." * c + "Q" + "." * (n - c - 1) for c in queens])
            return
        for c in range(n):
            if c in cols or r - c in diag or r + c in anti:
                continue                      # attacked: prune
            cols.add(c)
            diag.add(r - c)
            anti.add(r + c)
            queens.append(c)
            backtrack(r + 1)
            queens.pop()
            cols.remove(c)
            diag.remove(r - c)
            anti.remove(r + c)
    backtrack(0)
    return res

solve_n_queens(4)
# -> [['.Q..', '...Q', 'Q...', '..Q.'], ['..Q.', 'Q...', '...Q', '.Q..']]
[len(solve_n_queens(n)) for n in range(1, 9)]   # -> [1, 0, 0, 2, 10, 4, 40, 92]
```

The general trick: find an expression of `(r, c)` that is constant on each conflict region. Examples are `r`, `c`, `r - c`, `r + c`, and `(r // 3) * 3 + c // 3` for a Sudoku box. Use the key in a set, as a bit position, or as an array index. The key `r - c` can be negative, so shift it by `n - 1` for an array. For a count only (N-Queens II), return 1 at a full board and add the results.

54.4.9 Grid path search with marking

Trace a word through a grid, and use each cell at most one time on a path (Word Search). The answer depends on the cells that the path already uses, so the state is too large to memoize. Compare the letter when you enter a cell, and return at once on a mismatch. Mark the cell before the recursive calls, and restore it after them. Then one grid shows exactly the current path. Return at the first success: `or` and `any` stop at the first true value. **Memory hook** (grid path): *Check the cell, mark it, try four neighbors, unmark it.*

```
def exist(board, word):
    R, C = len(board), len(board[0])
    def dfs(r, c, i):
        if i == len(word):
            return True                       # every letter matched
        if not (0 <= r < R and 0 <= c < C) or board[r][c] != word[i]:
            return False                      # off the grid, used, or a wrong letter
        board[r][c] = "#"                     # choose: mark the cell as used
```

```
        found = (dfs(r + 1, c, i + 1) or dfs(r - 1, c, i + 1)
                 or dfs(r, c + 1, i + 1) or dfs(r, c - 1, i + 1))
        board[r][c] = word[i]                # un-choose: restore before the return
        return found
    return any(dfs(r, c, 0) for r in range(R) for c in range(C))

board = [list("ABCE"), list("SFCS"), list("ADEE")]
exist(board, "ABCCED"), exist(board, "SEE"), exist(board, "ABCB")   # -> (True, True, False)
board == [list("ABCE"), list("SFCS"), list("ADEE")]               # -> True
```

This plain search is correct, but crafted boards make it slow. An example is a board of one letter and a word whose last letter is not on the board. The fix is a *guard table* from a relaxed problem that allows a cell to repeat. The relaxed problem has a small state (cell, index), so a short DP solves it from the last letter to the first. A cell that fails the relaxed problem also fails the real one, so the search never enters it. **Memory hook** (Word Search): *Fill can-finish from the back, then backtrack only on can-finish cells.*

```
def exist_guarded(board, word):
    rows, cols = len(board), len(board[0])
    cells = [(r, c) for r in range(rows) for c in range(cols)]
    last = len(word) - 1
    def neighbors(r, c):
        return [(r + 1, c), (r - 1, c), (r, c + 1), (r, c - 1)]
    can = [set() for _ in word]              # can[i]: cells that could spell word[i:] with reuse
    for i in range(last, -1, -1):
        for r, c in cells:
            if board[r][c] == word[i] and (
                    i == last or any(nb in can[i + 1] for nb in neighbors(r, c))):
                can[i].add((r, c))
    used = set()                             # cells on the current path
    def dfs(r, c, i):
        if (r, c) not in can[i] or (r, c) in used:
            return False                     # one test: bounds, letter and future
        if i == last:
            return True
        used.add((r, c))
        found = any(dfs(nr, nc, i + 1) for nr, nc in neighbors(r, c))
        used.remove((r, c))
        return found
    return any(dfs(r, c, 0) for r, c in cells)

board = [list("ABCE"), list("SFCS"), list("ADEE")]
[exist_guarded(board, w) for w in ("ABCCED", "SEE", "ABCB", "A")]   # -> [True, True, False, True]
exist_guarded([["A"] * 6 for _ in range(6)], "A" * 14 + "B")       # -> False
```

The optimal form adds two cheap checks before the search. It rejects the word when the board has too few copies of a letter. It reverses the word when its first letter is more common on the board than its last letter. General rule for a constraint search: expand the most constrained choice first, and reject early with necessary conditions from a relaxed problem.

54.4.10 Many words on one grid: walk a trie

Find all words of a large list on one board (Word Search II). One search per word repeats the work for shared prefixes. Put all words in a trie, and run one grid search from each cell. Each step into a cell also steps one trie edge. A missing trie child proves that no word continues, so the search returns at once. Store each word at its end node, and clear it when you find it. Then each word is reported one time. On the way back, delete a trie node that has no children left. Later searches then skip dead branches. **Memory hook** (Word Search II): *Word Search I, but walk a trie instead of one word, and cut empty leaves on the way back.*

```python
class TrieNode:
    def __init__(self):
        self.children = {}                    # letter to child node
        self.word = None                      # the word that ends here, if any

def find_words(board, words):
    root = TrieNode()
    for word in words:
        node = root
        for ch in word:
            node = node.children.setdefault(ch, TrieNode())
        node.word = word
    rows, cols = len(board), len(board[0])
    found = []
    def dfs(r, c, parent):
        if r < 0 or c < 0 or r == rows or c == cols:
            return
        ch = board[r][c]
        if ch not in parent.children:
            return                            # no word continues; a used cell holds '#'
        node = parent.children[ch]
        if node.word:
            found.append(node.word)
            node.word = None                  # report each word once
        board[r][c] = "#"
        dfs(r + 1, c, node)
        dfs(r - 1, c, node)
        dfs(r, c + 1, node)
        dfs(r, c - 1, node)
        board[r][c] = ch
        if not node.children:
            del parent.children[ch]           # prune: nothing left to find here
    for r in range(rows):
        for c in range(cols):
            dfs(r, c, root)
    return found

board = [list("oaan"), list("etae"), list("ihkr"), list("iflv")]
sorted(find_words(board, ["oath", "pea", "eat", "rain"]))   # -> ['eat', 'oath']
find_words([list("ab"), list("cd")], ["abcb"])              # -> []
find_words([list("ab")], ["a", "ab", "ba"])                 # -> ['a', 'ab', 'ba']
```

Do not return after the first word at a node. A longer word can continue it, as `cat` and `cats`. The general rule: when a search consumes its targets, shrink the target structure as you find them.

54.4.11 Fill k buckets: symmetry and memoization

Split all items into k groups with equal sums (Partition to K Equal Sum Subsets). Each group must reach `target = total // k`. The buckets are interchangeable. A search that puts each item into any bucket visits each partition up to $k!$ times. Remove this symmetry: fill one bucket at a time, and start the next bucket only when the open bucket is full. Then the fill of the open bucket is the sum of the used items modulo `target`. So the set of used items alone is the full state. Store it as a bitmask, and memoize on it. There are at most 2^n masks, so the search does at most $O(n \cdot 2^n)$ work. **Memory hook** (Partition to K Equal Sum Subsets): *Mask of used numbers, fill is used sum mod target, try every unused number that fits.*

```python
from functools import cache
def can_partition_k(nums, k):
    total = sum(nums)
    if total % k:
        return False
    target = total // k
```

```
    n = len(nums)
    all_used = (1 << n) - 1
    @cache                                  # a new cache for each call
    def can_finish(used, fill):
        if used == all_used:
            return True                     # the used sum is k * target: all buckets full
        for i in range(n):
            if used >> i & 1 or fill + nums[i] > target:
                continue                    # already placed, or it does not fit
            if can_finish(used | 1 << i, (fill + nums[i]) % target):
                return True                 # % target opens the next bucket at 0
        return False
    return can_finish(0, 0)

can_partition_k([4, 3, 2, 3, 5, 2, 1], 4)     # -> True
can_partition_k([1, 2, 3, 4], 3)              # -> False
can_partition_k([1, 1, 1, 1, 2, 2, 2, 2], 4)  # -> True
```

The search fills one bucket at a time, but it does not commit to a bucket. A greedy split that keeps the first group with the target sum fails. An example is `[1, 1, 1, 1, 2, 2, 2, 2]` with $k = 4$: the group `1 + 1 + 1` leaves no valid split. The optimal form adds four prunings. It sorts the numbers, and it rejects a number above `target`. It breaks at the first number that overflows the bucket. It skips an equal value whose earlier copy is still unused. These steps do not change the $O(n \cdot 2^n)$ bound.

The other form places each item, from the largest down, into one of k buckets. Two buckets with the same load give the same subtree, so try only one of them. This rule also tries only one empty bucket. The form also works for "minimize the largest bucket". **Memory hook** (bucket form): *Largest item first. Try each distinct load one time.*

```
def can_partition_buckets(nums, k):
    total = sum(nums)
    if total % k:
        return False
    target = total // k
    nums = sorted(nums, reverse=True)         # large items first: dead ends show early
    if nums[0] > target:
        return False
    load = [0] * k
    def place(i):
        if i == len(nums):
            return True                       # every load is at most target, and the sum is k * target
        tried = set()                         # loads already tried for item i
        for b in range(k):
            if load[b] + nums[i] > target or load[b] in tried:
                continue                      # overflow, or the same as a tried bucket
            tried.add(load[b])
            load[b] += nums[i]                # choose
            if place(i + 1):
                return True
            load[b] -= nums[i]                # un-choose
        return False
    return place(0)

can_partition_buckets([4, 3, 2, 3, 5, 2, 1], 4)    # -> True
can_partition_buckets([1, 2, 3, 4], 3)             # -> False
can_partition_buckets([1, 1, 1, 1, 2, 2, 2, 2], 4) # -> True
```

54.4.12 Count, best value or yes-no: switch to DP

Backtracking lists answers one by one. When the task asks only for a number, look for a small state first. Ask: after a decision, what does the rest of the search depend on? **Memory hook**: *To list the answers, backtrack. To count them or find the best one, look for a small state and use DP.*

The rest depends on	Size of the state	Use
the full path (cells or items used, in a large space)	exponential	backtracking with pruning (Word Search)
an index and a remaining sum	$n \cdot target$	knapsack DP (Coin Change II, Partition Equal Subset Sum)
the start of the rest of a string	n	DP over prefixes (Word Break, Decode Ways)
the set of used items, $n \leq 20$	2^n	bitmask DP (Partition to K Equal Sum Subsets)
the set of used items, n near 40	$2^{n/2}$ per half	meet in the middle

For example, the number of answers of Combination Sum is an unbounded knapsack count. Put the items in the outer loop, so that each multiset counts one time.

```
def count_combinations(candidates, target):
    ways = [1] + [0] * target                # ways[s]: multisets with the sum s
    for c in candidates:                     # items outer: each multiset once
        for s in range(c, target + 1):
            ways[s] += ways[s - c]
    return ways[target]

count_combinations([2, 3, 6, 7], 7)          # -> 2
count_combinations(list(range(2, 32)), 40)   # -> 6131
```

With the sums in the outer loop, the same code counts ordered sequences, not multisets.

54.5 Adapt it to a new problem

Ask these questions in this order.

1. Does the task want every answer, or only a count, a best value or yes or no? For a number, find the state that the rest depends on. If it is small, use DP.
2. What is one decision? Take or skip an element, pick the next element, pick an unused element, pick an option for a slot, pick the end of a piece, step to a neighbor, or put an item in a bucket.
3. Does the order inside an answer matter? If not, use a start index so that each answer has one sorted route. If it does, loop over all elements and mark the used ones.
4. Can an element repeat? Recurse with the same index for reuse, and with the next index for single use.
5. Can the input repeat a value? Sort and skip equal siblings, branch on the distinct values of a counter, or keep a set of tried values for each call.
6. Which rule can a partial answer break? Test it before the recursive call. Sort the options if a failed test then proves that all later options fail, and use `break`.
7. Which shared state changes? List the path, the marks, the sets and the counters. Undo each change after the call, in reverse order. Record a copy.
8. How large is the tree? If it is too large, prune harder: most constrained choice first, a guard from a relaxed problem, one try for interchangeable choices, or a memo on a complete state.
9. How do you test it? Compare with an `itertools` brute force on small inputs. Sort both outputs, because the order is free.

54.6 Pitfalls

CAUTION

Record a copy of the path. Wrong: `res.append(path)`. Each entry is then the same list object, and the pops at the end make it empty. Right: `res.append(path[:])`, or `"".join(path)` for strings.

```python
def subsets_no_copy(nums):
    res, path = [], []
    def dfs(start):
        res.append(path)                        # BUG: no copy
        for j in range(start, len(nums)):
            path.append(nums[j])
            dfs(j + 1)
            path.pop()
    dfs(0)
    return res

subsets_no_copy([1, 2])                         # -> [[], [], [], []]
```

CAUTION

Undo every change, and undo it before the return. A missing `pop`, unmark or set removal leaks the choice into the next branch. In a grid search, compute the result, restore the cell, and only then return. A stale mark blocks valid cells, and the caller sees a changed board.

CAUTION

The right loop start. Combinations and subsets loop from `start`. A loop from 0 builds every ordering of one group, so the output has duplicates. Permutations loop from 0 with a used mark. A loop from `start` misses orderings. For reuse, recurse with `j`, not with `start`: `start` lets the path take a smaller index again.

CAUTION

The skip rule tests `j > start`, not `j > 0`. The test `j > 0` also skips the first copy in a deeper call. Then groups with equal values, such as `[2, 2]`, disappear. The rule also needs sorted input.

```python
def subsets_dup(nums, low_is_zero=False):
    nums = sorted(nums)
    res, path = [], []
    def dfs(start):
        res.append(path[:])
        for j in range(start, len(nums)):
            low = 0 if low_is_zero else start
            if j > low and nums[j] == nums[j - 1]:
                continue
            path.append(nums[j])
            dfs(j + 1)
            path.pop()
    dfs(0)
    return res

subsets_dup([2, 1, 2])                          # -> [[], [1], [1, 2], [1, 2, 2], [2], [2, 2]]
subsets_dup([2, 1, 2], low_is_zero=True)        # -> [[], [1], [1, 2], [2]]
```

CAUTION

`break` needs sorted input and positive values. On unsorted input, one value above the remainder does not prove that later values are too large. Use `continue`, or sort first. With negative values, the remainder does not only go down, so the pruning is not valid.

CAUTION

The memo key must hold all the state. Do not memoize `dfs(r, c, i)` in Word Search: the answer depends on the used cells. Do not memoize on `(index, fill)` for buckets: different used sets share these values. Put the used mask in the key. Define the cached function inside the solving function, so that each call gets a new cache.

CAUTION

Negative indexes do not fail. In a grid, test `0 <= r < R` before you index, because `board[-1]` reads the last row. The diagonal key `r - c` is negative for some cells. Use a set, or shift it by `n - 1` for a list index.

CAUTION

Side effects on the input. `nums.sort()` changes the caller's list, and a grid search writes marks into the board. Use `sorted(nums)`, and restore every mark.

NOTE

These problems usually accept the answers in any order. So sort both outputs before a test compares them. The recursion depth equals the number of decisions on one route, so it is small for these bounds.

54.7 Problems in this family

Problem	What changes	Memory hook
Subsets	The core shape: take or skip each element; all values distinct.	Each number doubles the list. Keep the old subsets and add each one plus the number.
Combination Sum	Start index with reuse: recurse with the same `j`; sort and break when a value exceeds the remainder.	Sort first. In dfs, save a copy at zero, then loop j from start: break if too big, pick, recurse with the same j, pop.
Permutations	Order matters: any unused element at each level; loop from 0.	Pick one from the pool. Recurse on the rest. An empty pool means a finished permutation.
Subsets II	Duplicate values: build from value counts, or sort and skip equal siblings.	Count each value. Every old subset takes 0 to count copies of it.
Combination Sum II	Duplicate values and single use: skip equal siblings, recurse with `i + 1`, break on the remainder.	Sort, then each level tries each value once.
Word Search	Grid path without a repeated cell: mark, recurse, unmark; a guard table from the relaxed problem.	Fill can-finish from the back, then backtrack only on can-finish cells.
Palindrome Partitioning	One decision is the end of the next piece; the piece must be a palindrome.	Cut a palindrome off the front, recurse on the rest.
Letter Combinations of a Phone Number	One option list per slot, no pruning: a Cartesian product.	Start with one empty string, and grow every prefix by every letter of the next digit.
N-Queens	One level per row; conflicts as keys `c`, `r - c`, `r + c` in sets.	One queen per row. Block the column, r minus c and r plus c.
Generate Parentheses	A prefix rule on two counts gives exact pruning; every branch is valid.	Open while opens remain. Close only what is open. Save at length 2n.
Word Search II	Many words: walk a trie with the grid search; clear found words and delete empty nodes.	Word Search I, but walk a trie instead of one word, and cut empty leaves on the way back.
Partition to K Equal Sum Subsets	Interchangeable buckets: fill one at a time; the used mask is the full state, so memoize.	Mask of used numbers, fill is used sum mod target, try every unused number that fits.

The transfer drills for this technique are in Section 69.12 (page 422). The answers are in Section A.11 (page 448).

55 Graph traversal: grids and implicit graphs

55.1 The idea

Many problems contain a graph that the input does not show. A grid is a graph: each cell is a node, and each pair of adjacent cells is an edge. A word, a lock code or a board is also a node: each legal one-step change is an edge. When you name the nodes and the edges, two traversals answer most questions.

- A **flood fill** (DFS or BFS, in any order) answers "which nodes connect". Use it to count regions, to measure them, or to test a property of each region.
- A **BFS** answers "how many steps". BFS takes nodes from a queue in order of distance. When each step costs the same, the first time BFS reaches a node gives its shortest distance.

The visited mark saves the work. A brute force starts a new search from each cell, which costs $O((mn)^2)$ on a grid. With a visited mark, each node is expanded (its edges are read) at most one time. The total work is then $O(V + E)$, or $O(mn)$ on a grid.

Worked example: count components. Take the grid with the rows `110`, `010` and `001`. Scan the cells row by row. Cell (0, 0) is land and is not visited, so count one island. A flood fill from (0, 0) marks (0, 0), (0, 1) and (1, 1). The scan skips (0, 1) and (1, 1), because they are marked. Cell (2, 2) touches (1, 1) only at a corner, and corner contact does not connect. So (2, 2) starts a second island, and the answer is 2.

Worked example: BFS layers. In an open 3×3 grid, start a BFS at (0, 0). Layer 0 holds only (0, 0). Layer 1 holds (1, 0) and (0, 1). Layer 2 holds (2, 0), (1, 1) and (0, 2). The far corner (2, 2) is in layer 4, so the fewest steps to it are 4. A DFS can reach (2, 2) first by a longer path, so a DFS does not give the shortest distance. A brute force tries every path and keeps the shortest one, and the number of paths grows exponentially. BFS removes that work. It reaches each node one time, and that first time is by a shortest path.

For each problem, decide three things: what a node is, what an edge is, and what the traversal returns (a count, a size, a flag, a distance, or a copy).

55.2 Recognize it

- The input is a grid, and cells connect "horizontally or vertically". The words island, region, group, blob or province name connected components.
- The question asks for the minimum number of moves, steps or minutes, and each move costs the same. This is a shortest path in an unweighted graph. Use BFS, not DFS.
- A process spreads from many cells at the same time, one step per unit of time. Use one BFS with all sources in the queue at distance 0.
- The question asks which cells can reach a border, an ocean or an exit, or which regions are surrounded. Many sources share a few fixed targets. Search backward from the targets.
- The statement asks you to split the nodes into two groups, and each conflict or dislike must join the two groups. Color the nodes with two colors in a BFS.
- The input is one configuration (a word, a code, a board), and a move changes it a little. Each configuration is a node of an implicit graph.
- The input gives n nodes and a list of pairs: edges, friendships, prerequisites or equal pairs. Build an adjacency list, then traverse it.
- The input gives objects and a rule that relates two of them: a distance, a radius or a one-letter difference. Each object is a node, and each pair that obeys the rule is an edge. If the rule is not symmetric ("i reaches j"), the edges are directed.
- The statement says clone or deep copy, and the structure has shared nodes or cycles. Traverse it and keep a map from each old node to its copy.
- The sizes allow one linear pass: about 10^5 cells or 10^6 states. One search from each cell is too slow.
- The answer is -1, 0 or INF when a target is unreachable. A traversal with a reachability check gives this case.

55.3 Core template

55.3.1 Flood fill: scan and sink

Scan every cell. At each land cell that is not visited, add one to the count and flood fill its whole component. The flood fill marks each cell that it reaches, so the scan never counts that component again. This version pushes all

four neighbors without a check, and it checks each cell when it pops the cell. The check after the pop rejects cells off the grid, water cells, and cells that are already sunk. Use this push-blindly form only for a flood fill with a stack, where no distance and no counter depend on the pushes. In a BFS with a queue, mark each node when you push it (see the BFS template below).

```
def num_islands(grid):
    rows, cols = len(grid), len(grid[0])

    def sink(r, c):
        stack = [(r, c)]                # iterative DFS: no recursion limit
        while stack:
            i, j = stack.pop()
            if 0 <= i < rows and 0 <= j < cols and grid[i][j] == "1":
                grid[i][j] = "0"        # sink: the grid is the visited set
                stack += [(i + 1, j), (i - 1, j), (i, j + 1), (i, j - 1)]

    islands = 0
    for r in range(rows):
        for c in range(cols):
            if grid[r][c] == "1":
                islands += 1
                sink(r, c)
    return islands

grid = [list("110"), list("010"), list("001")]
num_islands(grid)                       # -> 2
grid[0]                                 # -> ['0', '0', '0']
num_islands([list("11000"), list("11000"), list("00100"), list("00011")])   # -> 3
num_islands([["1"] * 200 for _ in range(200)])                              # -> 1
```

Invariant. Before the scan reaches a cell, every cell of each counted island is already `"0"`. So each `"1"` that the scan finds starts a new island. Memory hook (Number of Islands): *count each new land cell, then sink its whole island; push blindly, check on pop.*

Aside. The optimal form checks each neighbor before it pushes it, and it sinks the neighbor at once (see Max Area of Island below). Then each cell enters the stack at most one time. Both forms cost $O(mn)$. The sink-on-push form also works with a deque, so it is the form to use in a BFS.

CAUTION

Use an explicit stack or a queue, not recursion. A recursive DFS on an all-land 300×300 grid goes 90,000 calls deep. The default Python limit is 1000 calls, so the code raises `RecursionError`.

55.3.2 BFS: fewest steps

Put the start nodes in a deque. Pop from the left. Give each new neighbor the distance of its parent plus 1, and mark it at that moment. The dict `dist` does two jobs: it is the visited set, and it holds the answer. The function takes a `neighbors` function, so the same code runs on a grid, on an adjacency list, or on a state graph.

```
from collections import deque

def bfs(starts, neighbors):
    dist = {s: 0 for s in starts}       # visited set and answer in one dict
    queue = deque(starts)
    while queue:
        node = queue.popleft()
        for nxt in neighbors(node):
            if nxt not in dist:         # mark on push
                dist[nxt] = dist[node] + 1
                queue.append(nxt)
    return dist

maze = ["..#",
```

```
        ".##",
        "..."]
rows, cols = len(maze), len(maze[0])

def open_cells(cell):
    r, c = cell
    for nr, nc in ((r + 1, c), (r - 1, c), (r, c + 1), (r, c - 1)):
        if 0 <= nr < rows and 0 <= nc < cols and maze[nr][nc] == ".":
            yield nr, nc

dist = bfs([(0, 0)], open_cells)
dist[(2, 2)]                            # -> 4
dist.get((0, 2), -1)                    # -> -1
len(dist)                               # -> 6
```

Invariant. The queue holds nodes in nondecreasing order of distance, and two queued distances differ by at most 1. So the first distance that a node gets is its shortest distance. Mark a node when you push it. A node that you mark only when you pop it can enter the queue many times.

NOTE

There are three ways to keep the distance: a dict as above, (node, distance) pairs in the queue, or a level loop `for _ in range(len(queue))` with one counter. Use the level loop when you must act on one whole level at a time, for example one minute of a spread.

55.3.3 Build an adjacency list from edges

When the input is a list of edges, build an adjacency list first. For an undirected edge, store both directions. Then the flood fill and the BFS above run without a change. Loop over all node ids, not over the dict keys, so that a node with no edges still counts.

```
from collections import defaultdict

n = 6
edges = [(0, 1), (1, 2), (3, 4)]
adj = defaultdict(list)
for a, b in edges:
    adj[a].append(b)
    adj[b].append(a)                    # undirected: store both directions
adj[1]                                  # -> [0, 2]

def count_components(n, adj):
    seen = set()
    count = 0
    for start in range(n):              # every node id, also node 5 with no edges
        if start in seen:
            continue
        count += 1
        seen.add(start)
        stack = [start]
        while stack:
            node = stack.pop()
            for nxt in adj[node]:
                if nxt not in seen:
                    seen.add(nxt)
                    stack.append(nxt)
    return count

count_components(n, adj)                # -> 3
```

Union-find, cycle checks and topological order are in the section on graph structure.

55.4 Variations

55.4.1 Measure each component

What changes: the flood fill returns a value for its component, and the scan combines the values. In **Max Area of Island**, the flood fill returns the cell count, and the scan keeps the largest count. Sink each cell when you push it, and count each cell when you pop it. Each cell enters the stack one time, so the number of pops is the area. Memory hook: *scan for land; sink on push, count on pop, keep the max.*

```
def max_area(grid):
    rows, cols = len(grid), len(grid[0])

    def island_size(r, c):
        grid[r][c] = 0                  # sink on push
        stack = [(r, c)]
        size = 0
        while stack:
            i, j = stack.pop()
            size += 1                   # count on pop
            for x, y in ((i + 1, j), (i - 1, j), (i, j + 1), (i, j - 1)):
                if 0 <= x < rows and 0 <= y < cols and grid[x][y] == 1:
                    grid[x][y] = 0
                    stack.append((x, y))
        return size

    best = 0
    for r in range(rows):
        for c in range(cols):
            if grid[r][c] == 1:
                best = max(best, island_size(r, c))
    return best

max_area([[1, 1, 0], [0, 1, 0], [1, 0, 1]])   # -> 3
max_area([[0, 0]])                            # -> 0
```

The flood fill can return other values. Two examples are the number of sides that touch water (a perimeter) and the offsets from the first cell (a shape key). A third example is a flag that stays `True` only if every cell passes a test.

55.4.2 Compare regions by their cell sets

What changes: there are two grids of the same shape. A region of one grid matches a region of the other grid only if both have exactly the same coordinates. To **count the regions that two binary grids share**, flood fill every region of each grid and store it as a `frozenset` of coordinates. The answer is the size of the intersection of the two sets of regions. A `frozenset` is hashable, so it can be a member of a set, and two frozensets with the same cells are equal. Memory hook: *collect each grid's islands as frozensets, then intersect.*

```
def count_matching_regions(grid1, grid2):
    def regions(grid):
        rows, cols = len(grid), len(grid[0])
        seen = [[False] * cols for _ in range(rows)]
        found = set()
        for r in range(rows):
            for c in range(cols):
                if grid[r][c] != "1" or seen[r][c]:
                    continue
                seen[r][c] = True
                stack = [(r, c)]
                cells = []
                while stack:
                    i, j = stack.pop()
                    cells.append((i, j))
```

```
                    for x, y in ((i - 1, j), (i + 1, j), (i, j - 1), (i, j + 1)):
                        if (0 <= x < rows and 0 <= y < cols
                                and grid[x][y] == "1" and not seen[x][y]):
                            seen[x][y] = True
                            stack.append((x, y))
                found.add(frozenset(cells))
        return found

    return len(regions(grid1) & regions(grid2))

count_matching_regions(["110", "000", "011"], ["110", "000", "010"])   # -> 1
count_matching_regions(["10"], ["11"])                                 # -> 0
```

The faster form: check the border ring. A maximal region can grow only through a neighbor cell. So a region A of `grid1` equals a region of `grid2` exactly when two conditions are true. First, `grid2` has `"1"` on every cell of A. Second, `grid2` has `"0"` on every neighbor of A outside A (the border ring). Check both conditions with one flag during one flood fill of `grid1`. This needs no sets of coordinates. The general idea: to compare a component with another structure, check the frontier where the component can leak.

```
def count_matching_regions(grid1, grid2):
    rows, cols = len(grid1), len(grid1[0])
    seen = [[False] * cols for _ in range(rows)]
    count = 0
    for r in range(rows):
        for c in range(cols):
            if grid1[r][c] != "1" or seen[r][c]:
                continue
            seen[r][c] = True
            stack = [(r, c)]
            matches = True
            while stack:                            # finish the fill even after a mismatch
                i, j = stack.pop()
                if grid2[i][j] != "1":
                    matches = False                 # grid2 misses a region cell
                for x, y in ((i - 1, j), (i + 1, j), (i, j - 1), (i, j + 1)):
                    if not (0 <= x < rows and 0 <= y < cols):
                        continue
                    if grid1[x][y] == "1":
                        if not seen[x][y]:
                            seen[x][y] = True
                            stack.append((x, y))
                    elif grid2[x][y] == "1":
                        matches = False             # the grid2 region leaks out
            count += matches
    return count

count_matching_regions(["110", "000", "011"], ["110", "000", "010"])   # -> 1
count_matching_regions(["10"], ["11"])                                 # -> 0
count_matching_regions(["101"], ["101"])                               # -> 2
```

CAUTION

Do not stop a flood fill at the first mismatch. The rest of the region then stays unmarked, and the scan later starts a new search inside it. That search can count a false match. Finish the fill, then decide.

55.4.3 Search backward from the targets

What changes: many sources ask if they can reach a small fixed set of targets. Reverse each edge, and run one search that starts at all targets together. The cells that this search reaches are exactly the cells that can reach a target. One reverse search replaces one forward search per cell.

In **Pacific Atlantic Water Flow**, water flows to a neighbor that is lower or at the same height. So the reverse

search climbs to a neighbor that is higher or at the same height. Run one search from the Pacific border: the top row and the left column. Run one more search from the Atlantic border: the bottom row and the right column. Return the cells that are in both results. Memory hook: *water flows down, so flood uphill from each ocean and keep the cells both floods reach.*

```
def pacific_atlantic(heights):
    rows, cols = len(heights), len(heights[0])

    def flood_uphill(starts):
        reached = set(starts)
        stack = list(starts)
        while stack:
            r, c = stack.pop()
            for nr, nc in ((r + 1, c), (r - 1, c), (r, c + 1), (r, c - 1)):
                if (0 <= nr < rows and 0 <= nc < cols
                        and (nr, nc) not in reached
                        and heights[nr][nc] >= heights[r][c]):   # reverse of "flows down"
                    reached.add((nr, nc))
                    stack.append((nr, nc))
        return reached

    top_left = [(0, c) for c in range(cols)] + [(r, 0) for r in range(rows)]
    bottom_right = ([(rows - 1, c) for c in range(cols)]
                    + [(r, cols - 1) for r in range(rows)])
    both = flood_uphill(top_left) & flood_uphill(bottom_right)
    return [[r, c] for r, c in sorted(both)]

heights = [[1, 2, 2, 3, 5],
           [3, 2, 3, 4, 4],
           [2, 4, 5, 3, 1],
           [6, 7, 1, 4, 5],
           [5, 1, 1, 2, 4]]
pacific_atlantic(heights)
# -> [[0, 4], [1, 3], [1, 4], [2, 2], [3, 0], [3, 1], [4, 0]]
pacific_atlantic([[7]])                 # -> [[0, 0]]
```

Each ocean has its own `reached` set. A shared set stops the second search early on cells that the first search marked.

In **Surrounded Regions**, a region of `"O"` cells survives if one of its cells is on the border. Flood from all border cells, and change each `"O"` that the flood reaches to a third symbol `"T"`. Then sweep the board one time: `"O"` becomes `"X"` (captured), and `"T"` becomes `"O"` (safe). The third symbol replaces a visited set. Use this trick when you may change the input and need a temporary third state. Memory hook: *flood from the border, O becomes T; then O to X, T to O.*

```
def solve(board):
    rows, cols = len(board), len(board[0])
    stack = [(r, c) for r in range(rows) for c in range(cols)
             if r in (0, rows - 1) or c in (0, cols - 1)]
    while stack:
        r, c = stack.pop()
        if 0 <= r < rows and 0 <= c < cols and board[r][c] == "O":
            board[r][c] = "T"           # safe: the border reaches it
            stack += [(r + 1, c), (r - 1, c), (r, c + 1), (r, c - 1)]
    for r in range(rows):
        for c in range(cols):
            if board[r][c] == "O":
                board[r][c] = "X"       # captured
            elif board[r][c] == "T":
                board[r][c] = "O"       # restored

board = [list("XXXX"), list("XOOX"), list("XXOX"), list("XOXX")]
```

```
solve(board)
["".join(row) for row in board]         # -> ['XXXX', 'XXXX', 'XXXX', 'XOXX']
```

The same border-first flood counts the land cells that cannot reach the edge, or the islands that do not touch it.

55.4.4 Multi-source BFS

What changes: the search starts from many cells at the same time. Put all sources in the queue at distance 0 before the loop. This is the same as a BFS from one virtual source that connects to every real source. So the first distance that a cell gets is its distance to the nearest source.

In **Rotting Oranges**, put all rotten oranges in the queue, and count the fresh ones. Process one BFS level per minute. Stop when no fresh orange is left or when the queue is empty. Return -1 if a fresh orange remains. Memory hook: *all rotten in the queue, count the fresh, one level is one minute, stop when fresh is zero.*

```
from collections import deque

def oranges_rotting(grid):
    rows, cols = len(grid), len(grid[0])
    queue = deque()
    fresh = 0
    for r in range(rows):
        for c in range(cols):
            if grid[r][c] == 2:
                queue.append((r, c))
            elif grid[r][c] == 1:
                fresh += 1
    minutes = 0
    while queue and fresh > 0:          # "fresh > 0" stops before an extra empty minute
        for _ in range(len(queue)):     # one level is one minute
            r, c = queue.popleft()
            for nr, nc in ((r + 1, c), (r - 1, c), (r, c + 1), (r, c - 1)):
                if 0 <= nr < rows and 0 <= nc < cols and grid[nr][nc] == 1:
                    grid[nr][nc] = 2    # mark on push
                    fresh -= 1
                    queue.append((nr, nc))
        minutes += 1
    return minutes if fresh == 0 else -1

oranges_rotting([[2, 1, 1], [1, 1, 0], [0, 1, 1]])   # -> 4
oranges_rotting([[2, 1, 1], [0, 1, 1], [1, 0, 1]])   # -> -1
oranges_rotting([[0, 2]])                            # -> 0
```

In **Walls and Gates**, put all gates in the queue, then run a plain BFS. Write the parent distance plus 1 into each room that still holds INF. INF means "not reached yet", so the grid is the visited set. Memory hook: *push all gates first, then BFS and fill only INF rooms.*

```
from collections import deque

INF = 2**31 - 1

def walls_and_gates(rooms):
    rows, cols = len(rooms), len(rooms[0])
    queue = deque((r, c) for r in range(rows) for c in range(cols) if rooms[r][c] == 0)
    while queue:
        r, c = queue.popleft()
        for nr, nc in ((r + 1, c), (r - 1, c), (r, c + 1), (r, c - 1)):
            if 0 <= nr < rows and 0 <= nc < cols and rooms[nr][nc] == INF:
                rooms[nr][nc] = rooms[r][c] + 1   # first visit is the shortest
                queue.append((nr, nc))

rooms = [[INF, -1, 0, INF],
         [INF, INF, INF, -1],
```

```
        [INF, -1, INF, -1],
        [0, -1, INF, INF]]
walls_and_gates(rooms)
rooms
# -> [[3, -1, 0, 1], [2, 2, 1, -1], [1, -1, 2, -1], [0, -1, 3, 4]]
```

NOTE

Multi-source BFS gives a minimum over the sources. When the answer for a cell is a sum over all sources, run one BFS per source and add the distances. When the steps have different costs, push all sources into a heap at cost 0 and run Dijkstra (see the section on weighted graphs).

55.4.5 Two sides: two-coloring

What changes: the statement asks you to split the nodes into two groups, and each edge (a conflict or a dislike) must join the two groups. Such a graph is bipartite. A graph is bipartite exactly when it has no cycle of odd length. Run the BFS template with a dict `color` in place of `dist`. Give each new neighbor the other color, `1 - color[u]`. If an edge joins two nodes of the same color, return `False`: that edge closes an odd cycle. The graph can have many components, so start a BFS from every node that has no color yet. The first color of each component is free, because you can swap the two colors of a component. Memory hook: *give each new neighbor the other color; a same-color edge means an odd cycle.*

```
from collections import deque

def is_bipartite(graph):                # graph[u] = list of neighbors
    color = {}
    for start in range(len(graph)):     # every component
        if start in color:
            continue
        color[start] = 0
        queue = deque([start])
        while queue:
            u = queue.popleft()
            for v in graph[u]:
                if v not in color:
                    color[v] = 1 - color[u]      # the other side, marked on push
                    queue.append(v)
                elif color[v] == color[u]:
                    return False                 # an edge inside one side: an odd cycle
    return True

def possible_bipartition(n, dislikes):  # people 1..n, pairs that must split
    graph = [[] for _ in range(n + 1)]   # node 0 is not used and stays alone
    for a, b in dislikes:
        graph[a].append(b)
        graph[b].append(a)
    return is_bipartite(graph)

is_bipartite([[1, 3], [0, 2], [1, 3], [0, 2]])         # -> True
is_bipartite([[1, 2, 3], [0, 2], [0, 1, 3], [0, 2]])   # -> False
is_bipartite([[1], [0]])                               # -> True
is_bipartite([[1], [0], [3, 4], [2, 4], [2, 3]])       # -> False
possible_bipartition(4, [[1, 2], [1, 3], [2, 4]])      # -> True
possible_bipartition(3, [[1, 2], [1, 3], [2, 3]])      # -> False
```

A DFS with a stack works the same way, because only the color matters, not the distance. Example problems: Is Graph Bipartite (785) and Possible Bipartition (886).

CAUTION

Do not union the two ends of each edge. The two ends of an edge must go to different sides. A union puts them in one group, so the check fails even on the single edge `[[1], [0]]`, which is bipartite. Union-find works only in this form: union all neighbors of one node with each other, and return `False` if the node is in the same group as one of its neighbors.

55.4.6 Implicit state graphs

What changes: a node is a whole configuration, and a move generator lists its neighbors. You do not build the graph in advance. Define three things: the state, the move generator, and the forbidden or allowed states. Then count the states. Up to about 10^6 states fit a plain BFS in Python. Use a hashable state as the key of the visited set: a string, a tuple, or an integer.

In **Open the Lock**, a state is a 4-digit string. One move turns one wheel by $+1$ or -1 modulo 10, so each state has 8 neighbors and there are 10^4 states. Put the dead ends in the `seen` set before the search. Then one membership test blocks both the revisits and the forbidden states. Memory hook: *BFS from 0000 with (code, turns) pairs; dead ends start in seen; 8 neighbors are each wheel plus or minus 1 mod 10.*

```
from collections import deque

def open_lock(deadends, target):
    def neighbors(code):
        for i in range(4):
            digit = int(code[i])
            for step in (1, -1):
                yield code[:i] + str((digit + step) % 10) + code[i + 1:]

    seen = set(deadends)                # dead ends are walls: BFS never enters them
    if "0000" in seen:
        return -1
    seen.add("0000")
    queue = deque([("0000", 0)])        # (code, turns from "0000")
    while queue:
        code, turns = queue.popleft()
        if code == target:              # test on pop: also handles target == "0000"
            return turns
        for nxt in neighbors(code):
            if nxt not in seen:
                seen.add(nxt)
                queue.append((nxt, turns + 1))
    return -1

open_lock(["0201", "0101", "0102", "1212", "2002"], "0202")    # -> 6
open_lock(["8888"], "0009")                                    # -> 1
ring = ["8887", "8889", "8878", "8898", "8788", "8988", "7888", "9888"]
open_lock(ring, "8888")                                        # -> -1
open_lock(["0000"], "8888")                                    # -> -1
open_lock([], "0000")                                          # -> 0
```

Aside. The optimal code counts levels and tests the target when it generates a neighbor. It stops about one level sooner, but it needs its own check for a target of `"0000"`. The test on pop is fast enough for 10^4 states.

In **Word Ladder**, a state is a word, and a move changes one letter to give a word of the list. Do not compare all pairs of words: that is $O(N^2L)$. Generate the $26L$ candidates of each word and look each one up in a set. Remove a word from the set when you enqueue it. The answer counts words, not moves, so the start has length 1. Memory hook: *BFS of (word, length) pairs: change one letter, keep the word only if it is still in the unvisited set.*

```
from collections import deque
from string import ascii_lowercase

def ladder_length(begin, end, word_list):
    unvisited = set(word_list)
```

```
    unvisited.discard(begin)            # the start counts as visited
    queue = deque([(begin, 1)])         # (word, words in the ladder so far)
    while queue:
        word, length = queue.popleft()
        if word == end:
            return length
        for i in range(len(word)):
            for c in ascii_lowercase:
                nxt = word[:i] + c + word[i + 1:]
                if nxt in unvisited:
                    unvisited.remove(nxt)       # mark visited when enqueued
                    queue.append((nxt, length + 1))
    return 0

ladder_length("hit", "cog", ["hot", "dot", "dog", "lot", "log", "cog"])   # -> 5
ladder_length("hit", "cog", ["hot", "dot", "dog", "lot", "log"])          # -> 0
```

The time is $O(26 \cdot N \cdot L^2)$: N words, $26L$ candidates per word, and $O(L)$ to build and hash each one. The optimal code is the same BFS. It processes one level per pass with one counter, and it returns 0 at once when the end word is not in the list. For a much larger graph, use bidirectional BFS: search from both ends, and always expand the smaller frontier.

Trick: wildcard buckets. When an edge means "equal except at one position", mask that position. All words with the same masked key are neighbors. This turns an all-pairs search into hash lookups. Building the buckets costs $O(NL^2)$ and has no loop over the alphabet.

```
from collections import defaultdict

buckets = defaultdict(list)
for w in ["hot", "dot", "dog", "lot"]:
    for i in range(len(w)):
        buckets[w[:i] + "*" + w[i + 1:]].append(w)
buckets["*ot"]                          # -> ['hot', 'dot', 'lot']
buckets["do*"]                          # -> ['dot', 'dog']
```

55.4.7 Return the path, not only the distance

Store the parent of each node when you push it. The parent dict is also the visited set. At the goal, follow the parents back to the start, then reverse the list. The example is an implicit graph on numbers: the moves are $+1$ and $\times 2$. Memory hook: *the parent dict is the visited set; walk back from the goal, then reverse.*

```
from collections import deque

def shortest_path(start, goal, neighbors):
    parent = {start: None}
    queue = deque([start])
    while queue:
        node = queue.popleft()
        if node == goal:
            path = []
            while node is not None:
                path.append(node)
                node = parent[node]
            return path[::-1]
        for nxt in neighbors(node):
            if nxt not in parent:
                parent[nxt] = node
                queue.append(nxt)
    return None

moves = lambda x: [y for y in (x + 1, x * 2) if y <= 20]
shortest_path(2, 9, moves)              # -> [2, 4, 8, 9]
```

```
shortest_path(5, 3, moves) is None      # -> True
```

55.4.8 Clone a graph

What changes: the traversal builds one new node for each old node. A dict from each old node to its copy does two jobs. It is the visited set, and it translates old references into new references. Create the copy and store it in the dict before you visit the neighbors. Then a cycle that comes back to the node finds the copy and stops. In **Clone Graph**, `dfs(cur)` returns the copy of `cur`. Memory hook: *copy a tree, but check the map first and register the copy before you recurse.*

```
class Node:
    def __init__(self, val=0, neighbors=None):
        self.val = val
        self.neighbors = neighbors if neighbors is not None else []

def clone_graph(node):
    old_to_new = {}

    def dfs(cur):
        if cur in old_to_new:
            return old_to_new[cur]
        copy = Node(cur.val)
        old_to_new[cur] = copy          # register before recursing
        for nei in cur.neighbors:
            copy.neighbors.append(dfs(nei))
        return copy

    return dfs(node) if node else None

a, b, c, d = Node(1), Node(2), Node(3), Node(4)      # the cycle 1-2-3-4-1
a.neighbors, b.neighbors = [b, d], [a, c]
c.neighbors, d.neighbors = [b, d], [c, a]
a2 = clone_graph(a)
[n.val for n in a2.neighbors]           # -> [2, 4]
a2 is a, a2.neighbors[0] is b           # -> (False, False)
a2.neighbors[0].neighbors[0] is a2      # -> True
clone_graph(None) is None               # -> True
```

The same map copies a linked list with random pointers, and `copy.deepcopy` keeps a memo dict for the same reason. Recursion is safe here for 100 nodes. For a long graph, use BFS: create the copy and store it when you enqueue the node.

55.5 Adapt it to a new problem

Ask these questions in this order.

1. **What is a node?** A cell `(r, c)`, an index, a word, or a tuple that holds the whole state. It must be hashable if a set or a dict holds it.
2. **What is an edge?** Write the move generator. List the offsets: 4 or 8 directions, a knight move, one wheel, one letter. Test the bounds first. Then apply the rule of the edge: a wall, a height rule, a wrap-around with modulo. Ask if the edge is directed. A rule such as "i reaches j" is not always true in reverse. If the input is a list of pairs or objects, build the adjacency list now.
3. **What does the question ask?** Connectivity, a count, a size or a flag per region: flood fill in any order. Fewest steps with equal costs: BFS. Different costs: Dijkstra. Two groups with every edge between them: two-coloring. An order or a cycle: see the section on graph structure.
4. **How many sources and targets?** Many sources that act together: multi-source BFS. Many sources and a few fixed targets: search backward from the targets. One start and one goal in a large graph: plain BFS, then bidirectional BFS if necessary.
5. **How do you mark a visited node?** An unused value in the input (sink, `"T"`, INF) if you may change it. A set if you may not. A dict if each node needs a value: a distance, a parent, or a copy. Mark on push.

6. **How large is the graph?** Nodes times moves must be about 10^7 or less in Python. Use an explicit stack or a queue if the depth can pass 1000.
7. **What are the edge cases?** The start is the goal. The start is forbidden. There is no source. A target is unreachable. If the answer can be -1, count the targets before the search and check the count after it.
8. **Is one traversal enough?** Some problems chain two. For example, a flood fill collects the cells of one region, and a multi-source BFS from those cells measures the distance to something else.

55.6 Pitfalls

CAUTION

Negative indexes. Python reads `grid[-1]` as the last row, with no error. Test `0 <= nr < rows and 0 <= nc < cols` first in the `and` chain, then read the cell.

```
grid = [[1, 2], [3, 4]]
r, c = 0, 0
grid[r - 1][c]                          # -> 3
0 <= r - 1 < len(grid)                  # -> False
```

CAUTION

Mark on push in a BFS. If you mark a node only when you pop it, the node can enter the queue many times. The work grows, and a counter that you change on each push (the fresh oranges) becomes wrong. The push-blindly flood fill with a stack is the only exception. It has no distance and no counter, and it tests each cell when it pops it. So a duplicate entry does no harm there.

CAUTION

Recursion depth. A recursive DFS can go as deep as the number of cells. Above 1000 calls, Python raises `RecursionError`. Use an explicit stack or a deque.

CAUTION

DFS for a shortest path. DFS finds a path, not the shortest path. Use BFS when each step costs the same.

CAUTION

Nodes or moves. Word Ladder counts words, so the start has length 1. Open the Lock counts moves, so the start has distance 0. You can test the goal when you generate a neighbor. Then the answer is the parent value plus 1, and the case "start equals goal" needs its own check.

CAUTION

An extra level. A level loop that runs until the queue is empty also processes the last level, which reaches nothing new. Stop when the targets are done, or subtract the last level.

CAUTION

The wrong direction in a reverse search. The reverse of "flows to a lower or equal cell" is "climbs to a higher or equal cell": use `>=`, not `<=` and not `>`. Give each search its own visited set.

CAUTION

Cell types. A grid of strings holds `"1"`, not `1`. The test `grid[r][c] == 1` is always `False` there. Diagonal cells connect only if the statement says so.

CAUTION

The queue and the visited matrix. `list.pop(0)` is $O(n)$; use `collections.deque`. Build a visited matrix with a comprehension: `[[False] * cols for _ in range(rows)]`.

CAUTION

Changes to the input. Sinking cells destroys the input. Copy the grid first if the caller needs it. If you use a temporary marker such as `"T"`, restore it in a final sweep. A function that must change the input in place must not rebind the name to a new object.

55.7 Problems in this family

Problem	What changes	Memory hook
Number of Islands	Count the flood fill starts.	Count each new land cell, then sink its whole island; push blindly, check on pop.
Max Area of Island	The flood fill returns its cell count; keep the maximum.	Scan for land; sink on push, count on pop, keep the max.
Pacific Atlantic Water Flow	Two reverse searches that climb from each ocean border; intersect the results.	Water flows down, so flood uphill from each ocean and keep the cells both floods reach.
Surrounded Regions	Flood from the border with a third symbol, then one sweep decodes the board.	Flood from the border, O becomes T; then O to X, T to O.
Rotting Oranges	Multi-source BFS by levels with a counter of fresh cells.	All rotten in the queue, count the fresh, one level is one minute, stop when fresh is zero.
Walls and Gates	Multi-source BFS from all gates; INF marks the cells not reached yet.	Push all gates first, then BFS and fill only INF rooms.
Open the Lock	BFS on 10^4 codes with 8 moves each; dead ends start in the visited set.	BFS from 0000 with (code, turns) pairs; dead ends start in seen; 8 neighbors are each wheel plus or minus 1 mod 10.
Word Ladder	BFS on words; generate one-letter edits and look them up in a set; count words, not moves.	BFS of (word, length) pairs: change one letter, keep the word only if it is still in the unvisited set.
Clone Graph	The visited set becomes a dict from each old node to its copy.	Copy a tree, but check the map first and register the copy before you recurse.

The transfer drills for this technique are in Section 69.13 (page 422). The answers are in Section A.12 (page 449).

56 Graph structure: ordering, cycles and connectivity

The problems in this section ask about the shape of a graph, not about a distance. Is there an order that respects all the rules? Which items belong together? Does an extra link close a loop? Where does a forced walk stop? Each question has a short template that visits every node and every edge a constant number of times.

56.1 The idea

Many statements give a set of items and pairwise facts about them. A fact "u comes before v" is a directed edge $u \to v$. A fact "a is linked to b" is an undirected edge. A brute force tries all $n!$ orders, or it starts a new search from every node. The templates below do one pass over the graph, so they run in $O(V + E)$ or close to it (union-find adds a small factor).

- **Ordering (directed).** An order that respects every edge exists if and only if the graph has no directed cycle. In an acyclic graph, some node always waits for nothing, so it can go next. Kahn's algorithm takes such a node, removes its edges, and repeats. If it stops before it takes every node, the remaining nodes sit on a cycle or wait for one.
- **Grouping (undirected).** "Linked to" is symmetric. "Connected by a path" is also transitive, so the items fall into connected components. Flood fill from each unvisited node finds them. Union-find merges groups as the edges arrive, and it tells you when an edge joins a group to itself. That edge closes a cycle.
- **Forced walks (functional graph).** When each node has at most one next node, a walk has no choices. It is a chain that stops at a dead end, or a tail that runs into a cycle (a "rho" shape). The first repeated node proves a cycle, because from there the walk repeats the same steps forever. So one visited set stops the walk after at most V steps.

- **Counting patterns.** A small pattern, such as a path of 3 edges, has exactly one center. Fix the center and multiply the independent choices on each side. A brute force lists every path, and the number of paths can be in the billions. The product counts them without listing them.

Worked example (ordering). Take courses 0 to 3 with the rules 0 before 1, 0 before 2, 1 before 3 and 2 before 3. Count the unmet rules of each course: course 0 has 0, courses 1 and 2 have 1, course 3 has 2. Take course 0. Now courses 1 and 2 have 0 unmet rules. Take course 1: course 3 drops to 1. Take course 2: course 3 drops to 0. Take course 3. All four courses are taken, so the order 0, 1, 2, 3 is valid. Now add the rule "3 before 0". Course 0 then has 1 unmet rule, and no course starts at 0. The process takes no course, so a cycle exists.

Worked example (grouping). Take the undirected edges (1, 2), (1, 3), (2, 3) in this order. Edge (1, 2) joins two groups. Edge (1, 3) joins group $\{1, 2\}$ and group $\{3\}$. Edge (2, 3) finds both ends in the same group already. So this edge closes a cycle.

Worked example (forced walk). Take the next nodes $0 \to 1$, $1 \to 2$, $2 \to 3$ and $3 \to 1$. The walk from 0 visits 0, 1, 2, 3. The next node of 3 is 1, and 1 is already visited. So the walk has a tail of one node (0) and a cycle of three nodes (1, 2, 3).

Worked example (counting). Fix one middle edge (a, b) of a path $p_0 - a - b - p_3$. Node a has 3 neighbors that can be p_0, and node b has 4 neighbors that can be p_3. The two choices are independent, so $3 \times 4 = 12$ paths use this middle edge. One multiplication replaces 12 listed paths.

Question	Graph	Tool	Time
Does an order exist? Give one.	directed	Kahn's algorithm (or DFS with three colors)	$O(V + E)$
How many groups? Which group?	undirected	flood fill from each unvisited node	$O(V + E)$
Does this new edge close a cycle?	undirected	union-find: same root means a cycle	$O(E \log V)$ or better
Is it a tree?	undirected	$n - 1$ edges and one flood reaches all nodes	$O(V + E)$
Where does the walk stop?	out-degree ≤ 1	successor map and a visited set	$O(V)$
How many paths with 2 or 3 edges?	any	fix the middle, multiply side counts	$O(V + E)$

56.2 Recognize it

- The input gives pairs such as "take b before a", "a depends on b", prerequisites, a build order, an install order or a deadlock. Each pair is a directed edge. Think topological sort.
- The task says "return any valid order" and "return an empty result if it is impossible". Impossible means a directed cycle. Kahn's algorithm finds both in one pass.
- A list is sorted by an unknown rule, and you must recover the rule. Neighbors in the list give the edges.
- The task asks how many groups, provinces, networks or clusters exist, or if two items are connected. The relation is symmetric and transitive. Think connected components.
- Edges arrive one at a time, and the task asks which edge creates a cycle or when two items become connected. Think union-find.
- The task asks if an undirected graph is a tree. Use the edge count $n - 1$ plus one connectivity check.
- Each node has at most one outgoing edge: a next pointer, an array `i -> a[i]`, or a function applied again and again. Think functional graph: follow the only next node until a dead end or a repeat.
- The task counts paths of a small fixed length, n and m go up to 10^5, and "the answer can be large". You cannot list the paths. Fix the middle and multiply.
- Groups come from a relation on values, such as x and $x + 1$, and the task demands $O(n)$. Put the values in a hash set and start the work only at one member of each group.
- Chains can be thousands of nodes long. Avoid recursion, because CPython stops at a depth of about 1000.

56.3 Core template

Three templates cover most problems in this family. Learn them as three fixed shapes.

56.3.1 Template 1: Kahn's topological sort

Count the unmet requirements (the indegree) of each node. Put every node with indegree 0 in a queue. Pop a node, append it to the order, and lower the indegree of each node that it points to. Push a node when its indegree reaches 0. At the end, a short order means a cycle.

```
from collections import deque

def topo_order(n, edges):               # edge (u, v): u must come before v
    graph = [[] for _ in range(n)]
    indegree = [0] * n                  # unmet requirements of each node
    for u, v in edges:
        graph[u].append(v)
        indegree[v] += 1
    queue = deque(v for v in range(n) if indegree[v] == 0)
    order = []
    while queue:
        u = queue.popleft()
        order.append(u)
        for v in graph[u]:              # u is done: v waits for one node less
            indegree[v] -= 1
            if indegree[v] == 0:
                queue.append(v)
    return order if len(order) == n else []   # short order: a cycle blocks some nodes

topo_order(4, [(0, 1), (0, 2), (1, 3), (2, 3)])            # -> [0, 1, 2, 3]
topo_order(4, [(0, 1), (0, 2), (1, 3), (2, 3), (3, 0)])    # -> []
topo_order(3, [])                                          # -> [0, 1, 2]
topo_order(2, [(1, 1)])                                    # -> []
```

Invariant. `indegree[v]` equals the number of requirements of `v` that are not in the order yet. The queue holds exactly the nodes with indegree 0 that are not in the order yet. A node on a cycle always waits for another node of the cycle, so it never reaches 0. So `len(order) == n` holds if and only if the graph has no cycle.

56.3.2 Template 2: flood fill to count components

Loop over all nodes. Each unvisited node starts a new component. Mark everything that it reaches with a stack.

```
def count_components(n, edges):
    graph = [[] for _ in range(n)]
    for a, b in edges:
        graph[a].append(b)              # undirected: add both directions
        graph[b].append(a)
    seen = [False] * n
    count = 0
    for start in range(n):
        if seen[start]:
            continue                    # an earlier flood already reached it
        count += 1                      # a new start is a new component
        seen[start] = True
        stack = [start]
        while stack:
            node = stack.pop()
            for nxt in graph[node]:
                if not seen[nxt]:
                    seen[nxt] = True    # mark on push, not on pop
                    stack.append(nxt)
    return count

count_components(5, [(0, 1), (1, 2), (3, 4)])              # -> 2
count_components(5, [(0, 1), (1, 2), (2, 3), (3, 4)])      # -> 1
count_components(3, [])                                    # -> 3
```

Invariant. Every node in `seen` is reachable from a start that the loop already counted. Each node enters the stack at most once. When the stack is empty, the flood has marked the whole component of `start`. To get the group of each node, store the component number in a list instead of `True`.

56.3.3 Template 3: union-find

Each group is a tree of parent links. The root names the group. `find` walks up to the root. `union` links one root under the other root. A union of two nodes with the same root fails: the edge closes a cycle.

```
class DSU:
    def __init__(self, n):
        self.parent = list(range(n))   # every node starts as its own root
        self.groups = n

    def find(self, x):
        while self.parent[x] != x:
            self.parent[x] = self.parent[self.parent[x]]   # path halving
            x = self.parent[x]
        return x

    def union(self, a, b):
        ra, rb = self.find(a), self.find(b)
        if ra == rb:
            return False               # same root already: this edge closes a cycle
        self.parent[ra] = rb           # link root to root
        self.groups -= 1
        return True

d = DSU(5)
[d.union(a, b) for a, b in [(0, 1), (1, 2), (2, 0), (3, 4)]]
# -> [True, True, False, True]
d.groups                                          # -> 2
d.find(0) == d.find(2), d.find(0) == d.find(3)    # -> (True, False)
```

Invariant. After the first k edges, the sets are exactly the connected components of the graph with those k edges. `groups` equals n minus the number of successful unions. Path halving is a one-pass form of path compression: each step points a node at its grandparent.

NOTE

For the tight bound, also keep a `size` list and link the smaller root under the larger root (union by size). Then each operation costs almost $O(1)$ ($O(\alpha(n))$ amortized). Path halving alone gives $O(\log n)$ amortized, which is enough for n up to about 10^5. The recursive form `parent[x] = find(parent[x])` also compresses paths, but a long chain can pass the recursion limit before the first compression.

56.4 Variations

56.4.1 Can every task finish?

Course Schedule gives pairs `[a, b]`: take `b` before `a`. It asks only if all courses can finish. Run Template 1 and compare the number of taken courses with the number of courses. Memory hook: *take what is ready, free its dependents, check that you took everything.* Disconnected parts and self-loops need no special code: the queue starts with every free node, and a self-loop never reaches indegree 0. A DFS with three colors (below) also works, but a chain of 2000 courses needs a recursion depth of 2000.

56.4.2 Return one valid order

Course Schedule II returns the order itself, or an empty list. Now the edge direction matters. The pair `[a, b]` means the edge `b -> a`, because `b` comes first. Reversed edges give a reversed order. Memory hook: *count unmet prerequisites, take any course at zero; a short order means a cycle.* Two small changes to the queue answer two common follow-up questions.

- **Minimum number of rounds** (semesters, parallel steps): process the queue one level at a time. The number of levels is the answer. It equals the number of nodes on the longest path.
- **Lexicographically smallest order**: replace the deque with a min-heap of free nodes.

```
import heapq
from collections import deque

def build(n, edges):
    graph = [[] for _ in range(n)]
    indegree = [0] * n
    for u, v in edges:
        graph[u].append(v)
        indegree[v] += 1
    return graph, indegree

def rounds(n, edges):                   # minimum number of rounds; -1 on a cycle
    graph, indegree = build(n, edges)
    level = [v for v in range(n) if indegree[v] == 0]
    count = done = 0
    while level:
        count += 1
        done += len(level)
        next_level = []
        for u in level:
            for v in graph[u]:
                indegree[v] -= 1
                if indegree[v] == 0:
                    next_level.append(v)
        level = next_level
    return count if done == n else -1

def smallest_order(n, edges):           # the smallest order in dictionary order
    graph, indegree = build(n, edges)
    heap = [v for v in range(n) if indegree[v] == 0]
    heapq.heapify(heap)
    order = []
    while heap:
        u = heapq.heappop(heap)
        order.append(u)
        for v in graph[u]:
            indegree[v] -= 1
            if indegree[v] == 0:
                heapq.heappush(heap, v)
    return order if len(order) == n else []

def fifo_order(n, edges):               # Template 1, for comparison
    graph, indegree = build(n, edges)
    queue = deque(v for v in range(n) if indegree[v] == 0)
    order = []
    while queue:
        u = queue.popleft()
        order.append(u)
        for v in graph[u]:
            indegree[v] -= 1
            if indegree[v] == 0:
                queue.append(v)
    return order

rounds(4, [(0, 3), (1, 3), (3, 2)])     # -> 3
rounds(2, [(0, 1), (1, 0)])             # -> -1
fifo_order(3, [(1, 0)])                 # -> [1, 2, 0]
smallest_order(3, [(1, 0)])             # -> [1, 0, 2]
```

The order of Template 1 depends on when a node becomes free. Node 0 becomes free after node 2 is already in the queue. The heap always takes the smallest free node. The same idea of peeling free nodes works on an undirected tree: peel the leaves (degree 1) layer by layer to find the center (Minimum Height Trees). To compute a value for each node from its predecessors, such as an earliest finish time, update the value of `v` when you lower its indegree. Every input of a node is final before the node leaves the queue.

56.4.3 DFS with three colors

A DFS also finds a directed cycle and an order. Give each node one of three colors: white (not seen), gray (on the current path) and black (finished). An edge to a gray node is a back edge, so it closes a cycle. An edge to a black node is safe. A node finishes after every node that it points to, so the reversed finish order is a topological order. Memory hook: *gray is the current path; an edge to gray is a cycle; reverse the finish order.*

```
def dfs_topo(n, edges):                # edge (u, v): u before v; [] on a cycle
    graph = [[] for _ in range(n)]
    for u, v in edges:
        graph[u].append(v)
    WHITE, GRAY, BLACK = 0, 1, 2
    color = [WHITE] * n
    finished = []
    def visit(u):
        color[u] = GRAY
        for v in graph[u]:
            if color[v] == GRAY:
                return False           # back edge: a cycle
            if color[v] == WHITE and not visit(v):
                return False
        color[u] = BLACK
        finished.append(u)             # u finishes after all nodes that it points to
        return True
    for u in range(n):
        if color[u] == WHITE and not visit(u):
            return []
    return finished[::-1]

dfs_topo(4, [(0, 1), (0, 2), (1, 3), (2, 3)])              # -> [0, 2, 1, 3]
dfs_topo(3, [(0, 1), (1, 2), (2, 1)])                      # -> []
dfs_topo(3000, [(i, i + 1) for i in range(2999)])          # -> raises RecursionError
```

Use the colors when you need the set of nodes that can reach a cycle (Find Eventual Safe States). Then do not stop at the first cycle: run `visit` from every node. A node is safe if its DFS finishes black, and a node that stays gray can reach a cycle. Prefer Kahn's algorithm when a chain can be long. If the edges point from a node to its prerequisites (`a -> b`), the finish order is already correct; do not reverse it.

56.4.4 Derive the edges from a sorted list

In Alien Dictionary, the words are sorted by an unknown letter order, and the task is to recover that order. Compare only neighboring words: the order is transitive, so other pairs add nothing new. In each neighboring pair, only the first differing letters give a fact: the letter of the first word comes first. The letters after that position give nothing. If the shared prefix is equal and the first word is longer, no order can sort the list. Every letter is a node, also when no fact mentions it. Memory hook: *compare neighbors: a longer word before its prefix fails, else the first difference gives one edge. Kahn gives the order. A short order means a cycle.*

```
from collections import deque

def alien_order(words):
    graph = {c: set() for w in words for c in w}   # every letter is a node
    indegree = {c: 0 for c in graph}
    for w1, w2 in zip(words, words[1:]):
        if len(w1) > len(w2) and w1.startswith(w2):
```

```
            return ""                                # a longer word before its own prefix
        for a, b in zip(w1, w2):
            if a != b:
                if b not in graph[a]:                # count each edge one time only
                    graph[a].add(b)
                    indegree[b] += 1
                break                                # only the first difference counts
    queue = deque(c for c in graph if indegree[c] == 0)
    order = []
    while queue:
        c = queue.popleft()
        order.append(c)
        for nxt in graph[c]:
            indegree[nxt] -= 1
            if indegree[nxt] == 0:
                queue.append(nxt)
    return "".join(order) if len(order) == len(graph) else ""

alien_order(["wrt", "wrf", "er", "ett", "rftt"])   # -> 'wertf'
alien_order(["z", "x", "z"])                       # -> ''
alien_order(["abc", "ab"])                         # -> ''
alien_order(["ab", "ab"])                          # -> 'ab'
```

The general rule for a lexicographic comparison has three parts. The first difference gives a strict fact. An equal prefix gives a length fact. Everything after the first difference gives nothing. When many orders are valid, test an answer by checking every edge, not by comparing it with one fixed string.

56.4.5 Cycle detection: directed or undirected

The tool depends on the direction of the edges. Union-find ignores direction, so it reports false cycles in a directed graph. Memory hook: *directed: Kahn or colors; undirected: union-find or skip the parent.*

Graph	Tool	Sign of a cycle
directed	Kahn's algorithm	the order has fewer than n nodes
directed	DFS with three colors	an edge to a gray node
undirected	union-find	an edge whose ends have the same root
undirected	DFS that skips the edge back to its parent	a neighbor that is seen and is not the parent
undirected	counting	a connected graph with n nodes and at least n edges

```
def undirected_cycle(n, edges):        # iterative DFS with a parent check
    graph = [[] for _ in range(n)]
    for a, b in edges:
        graph[a].append(b)
        graph[b].append(a)
    seen = [False] * n
    for s in range(n):
        if seen[s]:
            continue
        seen[s] = True
        stack = [(s, -1)]
        while stack:
            node, parent = stack.pop()
            for nxt in graph[node]:
                if nxt == parent:
                    continue           # the edge back to the parent is not a cycle
                if seen[nxt]:
                    return True
                seen[nxt] = True
```

```
                stack.append((nxt, node))
    return False

def union_find_cycle(n, edges):
    parent = list(range(n))
    def find(x):
        while parent[x] != x:
            parent[x] = parent[parent[x]]
            x = parent[x]
        return x
    for a, b in edges:
        ra, rb = find(a), find(b)
        if ra == rb:
            return True
        parent[ra] = rb
    return False

triangle = [(0, 1), (1, 2), (2, 0)]
undirected_cycle(3, [(0, 1), (1, 2)])  # -> False
undirected_cycle(3, triangle)          # -> True
union_find_cycle(3, triangle)          # -> True
dag = [(0, 1), (1, 2), (0, 2)]         # directed edges: no directed cycle
union_find_cycle(3, dag)               # -> True
```

The last line is wrong for a directed graph. Two paths from 0 to 2 are not a cycle when the edges have a direction.

56.4.6 First edge that closes a cycle

Redundant Connection gives a tree on nodes 1 to n plus one extra edge. It asks for the cycle edge that occurs last in the input. Add the edges in input order. The first edge whose ends already share a root is the answer. Memory hook: *same root already? That edge closes the cycle.*

```
def redundant_connection(edges):       # nodes 1..n and n == len(edges)
    parent = list(range(len(edges) + 1))   # index 0 is not used
    def find(x):
        while parent[x] != x:
            parent[x] = parent[parent[x]]
            x = parent[x]
        return x
    for a, b in edges:
        ra, rb = find(a), find(b)
        if ra == rb:
            return [a, b]              # already connected: this edge closes the cycle
        parent[ra] = rb
    return []

redundant_connection([[1, 2], [1, 3], [2, 3]])                    # -> [2, 3]
redundant_connection([[1, 2], [2, 3], [3, 4], [1, 4], [1, 5]])    # -> [1, 4]
```

The tie-break "the last one in the input" needs no extra work. A cycle is complete only when its last edge arrives, so the first conflict in a forward scan is the last cycle edge. This is a reusable idea: when a rule asks for the last element that completes a structure, scan forward and stop at the moment of completion. The optimal form adds union by size for the $O(n\alpha(n))$ bound. For $n \leq 1000$, path halving alone is enough. The same loop is the core of Kruskal's minimum spanning tree: sort the edges by weight and skip each edge that closes a cycle.

56.4.7 Count the groups

Number of Connected Components in an Undirected Graph is Template 2 without changes. Memory hook: *count how many times you must start a new flood.* Isolated nodes are components too, so loop over all n nodes, not only over the nodes in the edges. The optimal solution uses union-find: the count is n minus the successful unions (Template 3). It needs no adjacency list, and it reports the count after each new edge when edges arrive over time.

A grid is the same problem with an implicit graph (Number of Islands). Other aggregates come from the same flood: the size of each group, or a list of its members.

56.4.8 Is it a tree?

Graph Valid Tree asks if n nodes and an undirected edge list form one tree. A graph on n nodes is a tree when any two of these three facts hold: it has $n-1$ edges, it is connected, it has no cycle. Check the cheap fact first (the edge count). Then check connectivity with one flood. With $n-1$ edges, connected means acyclic, so you need no cycle logic. Memory hook: *a tree has $n-1$ edges and reaches everyone.*

```
def valid_tree(n, edges):
    if len(edges) != n - 1:
        return False                    # a tree on n nodes has exactly n - 1 edges
    adj = [[] for _ in range(n)]
    for a, b in edges:
        adj[a].append(b)
        adj[b].append(a)
    seen = {0}
    stack = [0]
    while stack:
        node = stack.pop()
        for nei in adj[node]:
            if nei not in seen:
                seen.add(nei)
                stack.append(nei)
    return len(seen) == n               # n - 1 edges and connected: no cycle

valid_tree(5, [[0, 1], [0, 2], [0, 3], [1, 4]])             # -> True
valid_tree(5, [[0, 1], [1, 2], [2, 3], [1, 3], [1, 4]])     # -> False
valid_tree(4, [[0, 1], [1, 2], [2, 0]])                     # -> False
valid_tree(1, [])                                           # -> True
```

The third call has $n-1$ edges, but it is a triangle plus an isolated node. So the count alone is not enough. A related counting fact: a forest (no cycle) with n nodes and E edges has exactly $n-E$ components.

56.4.9 Groups on values: runs of consecutive numbers

Longest Consecutive Sequence asks for the longest run of values $x, x+1, x+2, \ldots$ in an unsorted array, in $O(n)$ time. Think of an edge between x and $x+1$. Each component is a run. A walk up from every value repeats work, because each member of a long run walks the rest of the run again. Instead, start the walk only at one member of each group: the value x with $x-1$ missing. Memory hook: *only start counting where $x-1$ is missing.*

```
def longest_consecutive(nums):
    values = set(nums)
    longest = 0
    for start in values:                # loop over the set, not over nums
        if start - 1 in values:
            continue                    # not the start of a run
        length = 1
        while start + length in values:
            length += 1
        longest = max(longest, length)
    return longest

longest_consecutive([100, 4, 200, 1, 3, 2])           # -> 4
longest_consecutive([0, 3, 7, 2, 5, 8, 4, 6, 0, 1])   # -> 9
longest_consecutive([])                               # -> 0
```

Each value is visited by exactly one walk, so the total work is $O(n)$ on average. The general rule: to process each group once, find a cheap local test that is true for exactly one member of each group, and start the work only there. Union-find on values (union x with $x+1$) also works, and it handles values that arrive one at a time. With a different neighbor rule, such as $x \to x^2$ on positive values, the same idea applies: start only where the predecessor

$\sqrt{x}$ is missing.

56.4.10 Functional graphs: follow the only next node

In a functional graph, each node has at most one outgoing edge. Linked lists, arrays read as `i -> a[i]`, and a function applied again and again are all functional graphs. A typical task walks from a start node through a network where each node has at most one next node. It returns the dead end, or the last node before the walk repeats a node. Build a successor map, keep a visited set, and look at the next node before you move. Then the current node is the answer in both stop cases. Memory hook: *map next, mark seen, peek before you step.*

```
def find_endpoint(edges, start):
    nxt = {u: v for u, v in edges}     # each node has at most one next node
    seen = {start}
    cur = start
    while cur in nxt:                  # stop at a node with no next node
        step = nxt[cur]
        if step in seen:               # the next step would repeat a node
            return cur
        seen.add(step)
        cur = step
    return cur

find_endpoint([[1, 3], [7, 3], [3, 4], [4, 6], [2, 6], [6, 9], [9, 5]], 1)   # -> 5
find_endpoint([[0, 1], [1, 2], [2, 0]], 0)              # -> 2
find_endpoint([[0, 1], [1, 2], [2, 3], [3, 1]], 0)      # -> 3
find_endpoint([[4, 4]], 4), find_endpoint([], 8)        # -> (4, 8)
```

To measure the rho shape, store the step index of each node instead of a plain visited set. The first repeated node is the cycle entry. Its stored index is the tail length, and the difference of the two indexes is the cycle length. Call these two numbers t and c. For $k \geq t$, the node after k steps is the node after $t + ((k - t) \bmod c)$ steps. So a huge k needs no long walk.

```
def rho(nxt, start):                   # nxt: dict from node to next node
    step_of = {}                       # node: its step index on the walk
    cur, step = start, 0
    while cur in nxt and cur not in step_of:
        step_of[cur] = step
        cur, step = nxt[cur], step + 1
    if cur not in step_of:
        return step, 0                 # dead end after step moves: no cycle
    return step_of[cur], step - step_of[cur]   # (tail length, cycle length)

rho({0: 1, 1: 2, 2: 3, 3: 1}, 0)       # -> (1, 3)
rho({4: 4}, 4)                         # -> (0, 1)
rho({0: 1, 1: 2}, 0)                   # -> (2, 0)
```

For $O(1)$ extra memory, use Floyd's tortoise and hare (Linked List Cycle, Find the Duplicate Number). When many starts share one graph, mark each node as unvisited, in progress or done, so each node is resolved once.

56.4.11 Count fixed-length paths: fix the middle

Take a road network where each shop has one of the types 0, 1, 2 and 3. The task counts the paths $p_0 - p_1 - p_2 - p_3$ in an undirected graph that use each type exactly once. A path and its reverse count separately. With n and m up to 10^5, the number of paths can reach billions, so you cannot list them. Every path with 3 edges has exactly one middle edge. Fix the middle edge (a, b). Then p_0 is any neighbor of a with one missing type, and p_3 is any neighbor of b with the other missing type. The two choices are independent, so multiply. Precompute `near[v][t]`, the number of neighbors of `v` with type `t`, in one pass. All four types differ, so the four shops are different automatically. Memory hook: *fix the middle road, multiply the end choices, double for reverse.*

```python
from itertools import permutations

def count_routes(shops, roads):
    near = [[0] * 4 for _ in shops]    # near[v][t]: neighbors of v with type t
    for a, b in roads:
        near[a][shops[b]] += 1
        near[b][shops[a]] += 1
    total = 0
    for a, b in roads:                 # each road as the middle p1 - p2
        if shops[a] == shops[b]:
            continue
        x, y = {0, 1, 2, 3} - {shops[a], shops[b]}   # types of the two ends
        total += near[a][x] * near[b][y] + near[a][y] * near[b][x]
    return 2 * total                   # every route also runs backwards

def brute(shops, roads):
    edge = {(a, b) for a, b in roads} | {(b, a) for a, b in roads}
    return sum(1 for p in permutations(range(len(shops)), 4)
               if {shops[v] for v in p} == {0, 1, 2, 3}
               and all((p[i], p[i + 1]) in edge for i in range(3)))

count_routes([0, 1, 2, 3], [[0, 1], [1, 2], [2, 3]])            # -> 2
count_routes([0, 1, 2, 3], [[0, 1], [1, 2], [2, 3], [3, 0]])    # -> 8
shops = [0, 1, 2, 3, 1, 2, 0]
roads = [[0, 1], [1, 2], [2, 3], [3, 0], [0, 4], [4, 5], [5, 3], [1, 5], [6, 4], [6, 2]]
count_routes(shops, roads) == brute(shops, roads)                # -> True
```

The general rule: pick the unique center of the pattern and multiply the counts on its sides. A path with 2 edges has a middle node. A path with 3 edges has a middle edge. A triple $i < j < k$ in an array has a middle index j. When labels can repeat, the side choices can overlap, and you must subtract the overlaps. For example, count the unlabeled simple paths with 3 edges (a path and its reverse count once). Start with $\sum(\deg(a)-1)(\deg(b)-1)$ over the edges (a, b). This sum also counts the case $p_0 = p_3$, which is a triangle and not a path. Each triangle occurs 3 times, one time for each of its edges as the middle. So subtract 3 times the number of triangles. For longer paths with distinct labels, use a bitmask DP over the set of used labels.

56.5 Adapt it to a new problem

1. **Name the nodes and the edges.** Is a fact "before" (directed), "linked" (undirected), or "next" (at most one outgoing edge)? Write the direction of one sample edge in a comment.
2. **Find the edges.** Are they given, or must you derive them? Derived edges come from neighbors in a sorted list, from value neighbors such as $x + 1$, or from a shared key such as an email address.
3. **Name the question.** An order or "is it possible" needs Template 1. Groups or a count of groups need Template 2. A first edge that breaks a property, or edges that arrive over time, need Template 3. A forced walk needs a successor map. A count of small patterns needs a fixed center.
4. **Use the counting facts.** A tree has $n - 1$ edges. A connected graph with n edges has exactly one cycle. Components equal n minus the successful unions.
5. **Cover every node.** Seed the queue or the outer loop with all n nodes, not only the nodes in the edges.
6. **Check the depth.** If a chain can be longer than about 1000 nodes, use a queue, an explicit stack or union-find with an iterative `find`.
7. **Check the extra rules.** A required order (smallest first) needs a heap. A number of rounds needs levels. A value for each node needs an update in topological order.
8. **Test on small inputs.** Compare with a brute force: all permutations for an order, a fresh search for connectivity, or all tuples for a path count.

56.6 Pitfalls

CAUTION

Edge direction. The pair `[a, b]` in "take `b` before `a`" is the edge `b -> a`. Reversed edges keep the yes/no answer of a cycle check, but they reverse the order. Write the direction in a comment before you build the graph.

CAUTION

One visited set in a directed DFS. A single set cannot tell "on the current path" from "finished". On a diamond, two paths reach the same finished node, and the DFS reports a false cycle. Use three colors.

```
def wrong_has_cycle(n, edges):          # one set for "on the path" and "finished"
    graph = [[] for _ in range(n)]
    for u, v in edges:
        graph[u].append(v)
    visited = set()
    def visit(u):
        if u in visited:
            return True                 # wrong: u can be finished, not on the path
        visited.add(u)
        return any(visit(v) for v in graph[u])
    return any(visit(u) for u in range(n) if u not in visited)

diamond = [(0, 1), (0, 2), (1, 3), (2, 3)]   # no cycle
wrong_has_cycle(4, diamond)             # -> True
```

CAUTION

Union-find on a directed graph. Union-find ignores direction. It reports a cycle for `0 -> 1 -> 2` plus `0 -> 2`, which has none. Use it only for undirected connectivity.

CAUTION

Compare roots and link roots. Two nodes of one group can have different parent entries. Compare `find(a)` with `find(b)`, not `parent[a]` with `parent[b]`. Link `parent[ra] = rb`. The assignment `parent[a] = b` moves only node `a` and the nodes below it, and it cuts them away from the rest of their old group. Compare the roots before the union, not after it.

```
parent = [1, 2, 2]                      # 0 -> 1 -> 2: one group with root 2
def find(x):
    while parent[x] != x:
        x = parent[x]
    return x
parent[0] == parent[2]                  # -> False
find(0) == find(2)                      # -> True
```

CAUTION

The final count check. Nodes on a cycle never reach indegree 0. Kahn's algorithm then returns a short order and raises no error. Compare `len(order)` with n and return the empty result.

CAUTION

Duplicate derived edges. When you derive edges (for example from word pairs), the same edge can appear two times. If the successors sit in a set, each copy still adds 1 to the indegree, but the set holds the node only one time. The indegree then never reaches 0, and a valid input returns "impossible". Test `b not in graph[a]` before you add the edge and raise the indegree.

CAUTION

Isolated nodes. A node that appears in no edge is still a node. It is a component, it is a free course, and it must be in the order. Take the node count from n (or from all letters), not from the edges.

CAUTION

Recursion depth. A recursive DFS or a recursive `find` on a chain of 2000 nodes passes the CPython limit of 1000 and raises `RecursionError` (see the `dfs_topo` example). Use Kahn's algorithm, an explicit stack, or an iterative `find`. `sys.setrecursionlimit` helps only up to the size of the C stack (Section 30).

CAUTION

Undirected details. Add each undirected edge in both directions. In a DFS cycle check, skip the edge back to the parent, or every edge looks like a cycle. Build the adjacency list with `[[] for _ in range(n)]`, never with `[[]] * n`.

CAUTION

Group starts. In the run-start pattern, loop over the set, not over the input list. Many copies of one run start in the list repeat the full walk many times. Start `longest` at 0 for an empty input.

CAUTION

Forced walks. Put the start node in the visited set before the first step. Test the next node before you move, or you return the repeated node instead of the node before it. A node that is never a source has no key in the successor map; test with `in` before you index.

CAUTION

Middle-edge counting. Use both assignments of the two missing types. Multiply by 2 only if the loop visits each undirected edge one time. Precompute the neighbor counts. A scan of the neighbor lists for each edge costs the sum of the squared degrees, and that is too slow on a star.

CAUTION

Queues. `list.pop(0)` costs $O(n)$. Use `collections.deque` and `popleft()`.

56.7 Problems in this family

Problem	What changes	Memory hook
Course Schedule	Only feasibility: Template 1 and the count `taken == n`.	Take what is ready, free its dependents, check that you took everything.
Course Schedule II	Return the order; the pair `[a, b]` is the edge `b -> a`.	Count unmet prerequisites. Take any course at zero. A short order means a cycle.
Alien Dictionary	Derive the edges from the first difference of neighboring words; a longer word before its prefix fails.	Compare neighbors: a longer word before its prefix fails, else the first difference gives one edge. Kahn gives the order. A short order means a cycle.
Redundant Connection	Union-find in input order; return the first edge with the same root at both ends.	Same root already? That edge closes the cycle.
Number of Connected Components in an Undirected Graph	Count the flood starts (or n minus the successful unions).	Count how many times you must start a new flood.
Graph Valid Tree	Check $n - 1$ edges, then one flood that reaches all nodes.	A tree has $n - 1$ edges and reaches everyone.
Longest Consecutive Sequence	Groups on values (x and $x + 1$); walk only from the start of each run.	Only start counting where $x - 1$ is missing.

The transfer drills for this technique are in Section 69.14 (page 423). The answers are in Section A.13 (page 451).

57 Weighted and advanced graphs

57.1 The idea

BFS finds the path with the fewest edges. When edges have different costs, this is not enough: a path with more edges can cost less. Dijkstra fixes this with one change. The FIFO queue becomes a min-heap that orders the nodes by their distance so far.

Think of a wave that starts at the source and moves along all edges at the same time. The wave reaches each node at its shortest distance, and it reaches the nodes in order of that distance. The heap replays the wave. Each pop gives the next node that the wave reaches.

Worked example. The directed edges are A→B (4), A→C (1), C→B (2) and B→D (1). Start at A with the heap `[(0, A)]`.

1. Pop `(0, A)`. A is final at 0. Push `(4, B)` and `(1, C)`.
2. Pop `(1, C)`. C is final at 1. Push `(3, B)`, because $1 + 2 = 3$.
3. Pop `(3, B)`. B is final at 3. Push `(4, D)`.
4. Pop `(4, B)`. B is already final, so skip this old copy.
5. Pop `(4, D)`. D is final at 4. The heap is empty, so stop.

BFS puts B one edge from A, at cost 4. The cheaper route A→C→B has two edges.

Why is the first pop of a node final? Every entry that is still in the heap is at least as large. An edge with a non-negative weight cannot make a longer path cheaper. A brute force tries every path, and the number of paths can grow exponentially. Dijkstra handles each edge one time, so it costs $O(E \log E)$.

The other problems of this family change one part of this picture.

Question	What changes	Tool
Shortest time from one source; time until all nodes are reached	nothing	Dijkstra with `d + w`
Lowest possible highest point on a route	combine with `max` instead of `+`	Dijkstra with `max`, or binary search on the level
Cheapest set of edges that connects all nodes	the cost is the total of the chosen edges, not a path	minimum spanning tree: Prim or Kruskal
Cheapest path with at most k edges	a second limit on the path	Bellman-Ford for k rounds from a copy, or BFS by levels
Cheapest path under a limit on another resource	the state is (node, resource)	Dijkstra on states
Use every edge exactly one time	the answer is an order of edges, not a cost	Hierholzer (Euler path)

57.2 Recognize it

- Edges carry a cost, time, price or distance, and the task asks for the minimum total from a source. Different weights rule out BFS. Non-negative weights allow Dijkstra.
- "When do all nodes receive the signal": each node gets it at its shortest distance. The answer is the maximum of these distances, not their sum.
- A route is usable only when every cell or edge on it is at or below some level. Phrases: "earliest time that A and B connect", "minimum effort", "smallest limit so that a path exists". This is a bottleneck (minimax) path. A higher level never removes a route, so binary search on the level also works.
- "Connect all points at minimum total cost", "exactly one path between any two points", "no redundant link": a minimum spanning tree (MST).
- A formula gives the cost of every pair, for example the distance between two points. The graph is complete and implicit. Compute each weight when you need it.
- "At most k stops, moves or edges": the cheapest path overall can break the limit. Run one Bellman-Ford round, or one BFS level, for each edge that the path can use.
- A limit on another resource (fuel, time, coupons, obstacles to remove): put the resource in the state.
- "Use every ticket, pair or domino exactly one time", and the end of one item is the start of the next: an Euler path. Nodes can repeat; edges cannot.

- "Visit every node exactly one time" is a different problem (a Hamiltonian path). It needs backtracking or bitmask DP, and it is exponential.
- A negative weight rules out Dijkstra. Use Bellman-Ford.
- Sizes: a heap suits E up to about 10^5 to 10^6. A complete graph with $V \approx 10^3$ has $5 \cdot 10^5$ edges. Do not store them; use an $O(V^2)$ scan.

57.3 Core template

Dijkstra with lazy deletion. Do not decrease a key in the heap. Push a new entry instead, and skip an entry at pop time when its node is already final.

```
import heapq
from collections import defaultdict

def dijkstra(graph, src):
    # graph[u] = list of (v, w), every w >= 0. Returns {node: shortest distance}.
    dist = {}                           # final distances; also the visited set
    heap = [(0, src)]                   # (distance so far, node): distance first
    while heap:
        d, u = heapq.heappop(heap)
        if u in dist:                   # an old copy: u is already final
            continue
        dist[u] = d                     # the first pop gives the shortest distance
        for v, w in graph[u]:
            if v not in dist:
                heapq.heappush(heap, (d + w, v))
    return dist

graph = defaultdict(list)
for u, v, w in [("A", "B", 4), ("A", "C", 1), ("C", "B", 2), ("B", "D", 1)]:
    graph[u].append((v, w))             # directed edge from u to v
dijkstra(graph, "A")                    # -> {'A': 0, 'C': 1, 'B': 3, 'D': 4}

def network_delay_time(times, n, k):    # Network Delay Time: nodes 1 .. n
    graph = defaultdict(list)
    for u, v, w in times:
        graph[u].append((v, w))
    dist = dijkstra(graph, k)
    return max(dist.values()) if len(dist) == n else -1

network_delay_time([[2, 1, 1], [2, 3, 1], [3, 4, 1]], 4, 2)   # -> 2
network_delay_time([[1, 2, 1]], 2, 1)                         # -> 1
network_delay_time([[1, 2, 1]], 2, 2)                         # -> -1
network_delay_time([], 1, 1)                                  # -> 0
```

Invariant. `dist` holds only final nodes, and each value is the true shortest distance. The heap holds candidates, and some of them are old copies. When a node pops for the first time, no other path can be shorter: every other entry is at least as large, and no weight is negative.

Three small changes cover most uses.

- **One target.** Return `d` when the target pops. Do not return when it is pushed.
- **Several sources.** Start the heap with `(0, s)` for each source.
- **Implicit graph.** Replace `graph[u]` with code that makes the neighbors: the four cells around a grid cell, or the next states of a puzzle.

NOTE

The tuple puts the distance first, so the heap orders by distance. On a tie, Python compares the second item. A one-item list `[(0, src)]` is already a valid heap, so no `heapify` call is necessary.

57.4 Variations

57.4.1 Time until every node is reached

Something spreads from one node along all edges at the same time. Each node gets it at its shortest distance. So the answer is the largest final distance, or -1 when a node is not reached (Network Delay Time). The core template already solves it. Compare the number of final nodes with n. A node with no edges is not in the edge list, but it still counts. Memory hook: *the first pop of a node is its arrival time.*

57.4.2 Bottleneck paths: the highest point on a route

Here a route is usable at level t only when every cell on it is at most t. So the cost of a route is its highest cell, not a sum. The answer is the minimum over all routes of that maximum. Dijkstra still works with `max` in place of `+`. The reason: `max` never makes a path cheaper when the path grows, so the first pop is still final (Swim in Rising Water). Memory hook: *Dijkstra, but the path cost is the highest cell, so use max instead of plus.*

```
import heapq

def swim_in_water(grid):                # Swim in Rising Water
    n = len(grid)
    heap = [(grid[0][0], 0, 0)]         # (highest cell on the path, row, col)
    visited = set()
    while heap:
        t, r, c = heapq.heappop(heap)
        if (r, c) in visited:
            continue                    # an old copy with a key that is not smaller
        visited.add((r, c))             # t is final for (r, c)
        if (r, c) == (n - 1, n - 1):
            return t
        for dr, dc in ((1, 0), (-1, 0), (0, 1), (0, -1)):
            nr, nc = r + dr, c + dc
            if 0 <= nr < n and 0 <= nc < n and (nr, nc) not in visited:
                heapq.heappush(heap, (max(t, grid[nr][nc]), nr, nc))   # max, not +
    return -1

swim_in_water([[0, 2], [1, 3]])         # -> 3
swim_in_water([[0, 1, 2, 3, 4], [24, 23, 22, 21, 5], [12, 13, 14, 15, 16],
               [11, 17, 18, 19, 20], [10, 9, 8, 7, 6]])   # -> 16
swim_in_water([[0]])                    # -> 0
```

The heap key has a direct meaning: it is the earliest time at which you can reach that cell. The start cell counts, so the heap starts with `grid[0][0]`, not with 0. For a cost on the edges, for example the height difference between two cells, change only the key formula. A shorter form marks a cell when it is pushed. It has the same $O(n^2 \log n)$ cost, but it is correct only because the cost to enter a cell depends on the cell alone. With a cost on the edges, it can give wrong answers.

Other combine rules. Dijkstra works when the combine step never makes a path better as it grows. A sum of non-negative weights, a maximum, and a product of factors that are at least 1 all satisfy this. To maximize a minimum, or a product of probabilities in $[0, 1]$, use a max-heap: push the negated key.

Alternative: binary search on the level. If the corners connect at level t, they connect at every larger level. So binary search for the first level that connects them, with one BFS for each check. This costs $O(n^2 \log n)$, the same as Dijkstra.

```
from collections import deque

def connected_at(grid, t):              # can you go corner to corner on cells <= t?
    n = len(grid)
    if grid[0][0] > t:
        return False
    seen = {(0, 0)}
    queue = deque([(0, 0)])
    while queue:
```

```
        r, c = queue.popleft()
        if (r, c) == (n - 1, n - 1):
            return True
        for dr, dc in ((1, 0), (-1, 0), (0, 1), (0, -1)):
            nr, nc = r + dr, c + dc
            if (0 <= nr < n and 0 <= nc < n and (nr, nc) not in seen
                    and grid[nr][nc] <= t):
                seen.add((nr, nc))
                queue.append((nr, nc))
    return False

def swim_by_level(grid):
    lo, hi = grid[0][0], max(map(max, grid))   # the answer is in [lo, hi]
    while lo < hi:                             # find the first level that connects
        mid = (lo + hi) // 2
        if connected_at(grid, mid):
            hi = mid
        else:
            lo = mid + 1
    return lo

swim_by_level([[0, 2], [1, 3]])          # -> 3
grid = [[0, 1, 2, 3, 4], [24, 23, 22, 21, 5], [12, 13, 14, 15, 16],
        [11, 17, 18, 19, 20], [10, 9, 8, 7, 6]]
swim_by_level(grid)                      # -> 16
```

A third method adds the cells in increasing order of value and joins each new cell to its active neighbors with union-find. The answer is the value at which the start and the end first share a root. This is Kruskal's order (next subsection). The path between two nodes in an MST is always a minimax path.

NOTE

When the values are a permutation of 0 to $N-1$, store the position of each value in an array. Then you can add the cells in value order without a sort.

57.4.3 Minimum spanning tree: connect everything at minimum total cost

The goal is the total weight of the chosen edges, not the length of one path. With non-negative weights, a cheapest connected set never needs a cycle: remove the heaviest edge of a cycle, and all nodes stay connected. So the answer is a minimum spanning tree (MST). The **cut property** makes greedy growth safe: for any split of the nodes into two groups, the cheapest edge across the split belongs to some MST.

Prim grows one tree from any node. Each round adds the outside node with the cheapest edge to the tree. On a complete graph, keep one value for each outside node and scan the values. This costs $O(V^2)$ time and $O(V)$ memory, and it stores no edges. A dict that holds only the outside nodes is also the to-do list (Min Cost to Connect All Points). Memory hook: *keep a dict of outside points: pop the nearest, then let it lower the others.*

```
def min_cost_connect_points(points):     # Min Cost to Connect All Points
    # cost[v] = cheapest edge from the outside point v to the tree
    cost = {v: float("inf") for v in range(len(points))}
    cost[0] = 0
    total = 0
    while cost:
        u = min(cost, key=cost.get)      # the nearest outside point joins the tree
        total += cost.pop(u)
        ux, uy = points[u]
        for v in cost:                   # the new tree point can lower the others
            d = abs(ux - points[v][0]) + abs(uy - points[v][1])
            if d < cost[v]:
                cost[v] = d
    return total

min_cost_connect_points([[0, 0], [2, 2], [3, 10], [5, 2], [7, 0]])   # -> 20
```

```
min_cost_connect_points([[3, 12], [-2, 5], [-4, 1]])                # -> 18
min_cost_connect_points([[0, 0]])                                   # -> 0
```

Compare it with Dijkstra. Both grow a set from one node. Dijkstra keys a node by the whole path cost `d + w`. Prim keys a node by the single edge `w`.

Kruskal suits an explicit, sparse edge list. Sort the edges by weight. Keep an edge when its two ends are in different components of a union-find. Stop after $V-1$ edges. If fewer than $V-1$ edges join components, the graph is not connected and no spanning tree exists. Memory hook: *cheapest edge first; keep it only when it joins two components.*

```
def kruskal(n, edges):                    # edges = [(w, u, v)], nodes 0 .. n-1
    parent = list(range(n))
    def find(x):
        while parent[x] != x:
            parent[x] = parent[parent[x]]   # path halving
            x = parent[x]
        return x
    total = used = 0
    for w, u, v in sorted(edges):         # cheapest edge first
        ru, rv = find(u), find(v)
        if ru != rv:                      # the edge joins two components: keep it
            parent[ru] = rv
            total += w
            used += 1
            if used == n - 1:
                break
    return total if used == n - 1 else -1   # -1: the graph is not connected

kruskal(4, [(1, 0, 1), (4, 1, 2), (3, 0, 2), (2, 2, 3), (5, 1, 3)])   # -> 6
kruskal(3, [(1, 0, 1)])                   # -> -1
pts = [[0, 0], [2, 2], [3, 10], [5, 2], [7, 0]]
edges = [(abs(a[0] - b[0]) + abs(a[1] - b[1]), i, j)
         for i, a in enumerate(pts) for j, b in enumerate(pts) if i < j]
kruskal(len(pts), edges)                  # -> 20
```

Choose by density. A complete or formula-based graph: $O(V^2)$ Prim. A sparse edge list: Kruskal in $O(E \log E)$, or Prim with a heap. On the complete graph of Min Cost to Connect All Points ($n \le 1000$), heap Prim and Kruskal cost $O(n^2 \log n)$ and are slower than the scan. A list-based scan with an `in_tree` flag array has the same $O(n^2)$ cost as the dict form.

NOTE

A node can sometimes connect to an outside source with its own cost ("build a well here"). Add one virtual node, give it an edge to each such node, and run the MST on the larger graph. For a maximum spanning tree, sort the edges in descending order.

57.4.4 Shortest path with at most k edges

A limit on the number of edges breaks plain Dijkstra. Dijkstra keeps one best cost for each node, but the cheapest arrival can use too many edges to continue. **Bellman-Ford** relaxes every edge in rounds. If each round reads only the values of the previous round, then after round i each value is the cheapest cost with at most i edges. So run exactly $k+1$ rounds for k stops (Cheapest Flights Within K Stops). Memory hook: $k+1$ *rounds: relax every flight from last round's copy.*

```
def cheapest_price(n, flights, src, dst, k):   # Cheapest Flights Within K Stops
    INF = float("inf")
    prices = [INF] * n
    prices[src] = 0
    for _ in range(k + 1):                # k stops = k + 1 flights (edges)
        next_prices = prices.copy()       # read from prices, write to next_prices
        for u, v, cost in flights:
            if prices[u] == INF:
```

```
                continue
            if prices[u] + cost < next_prices[v]:
                next_prices[v] = prices[u] + cost
        prices = next_prices
    return -1 if prices[dst] == INF else prices[dst]

def cheapest_in_place(n, flights, src, dst, k):   # WRONG: no copy
    INF = float("inf")
    prices = [INF] * n
    prices[src] = 0
    for _ in range(k + 1):
        for u, v, cost in flights:
            if prices[u] + cost < prices[v]:
                prices[v] = prices[u] + cost       # a later edge reads this new value
    return -1 if prices[dst] == INF else prices[dst]

flights = [[0, 1, 100], [1, 2, 100], [2, 0, 100], [1, 3, 600], [2, 3, 200]]
cheapest_price(4, flights, 0, 3, 1)      # -> 700
short = [[0, 1, 100], [1, 2, 100], [0, 2, 500]]
cheapest_price(3, short, 0, 2, 1)        # -> 200
cheapest_price(3, short, 0, 2, 0)        # -> 500
cheapest_in_place(3, short, 0, 2, 0)     # -> 200
```

The last line is wrong. With $k = 0$, only the direct flight (500) is allowed. The in-place round read the new price of city 1 and chained two flights in one round. The cost is $O(k(V + E))$.

NOTE

The two-buffer rule applies to every layered computation in which layer i must depend only on layer $i - 1$. Two-row knapsack DP and synchronous grid updates use the same rule. For "exactly $k + 1$ edges", start each new round at infinity instead of a copy of the old round.

Alternative: BFS by stops. A BFS level is the same as a round. Level i holds the paths with exactly i edges, as (node, cost) pairs. Expand only the current level, and stop after $k+1$ levels. Push a neighbor only when its new cost is lower than the best cost so far. A path that is cheaper and has fewer edges makes the other path useless.

```
from collections import defaultdict

def cheapest_bfs(n, flights, src, dst, k):
    graph = defaultdict(list)
    for u, v, p in flights:
        graph[u].append((v, p))
    best = [float("inf")] * n
    best[src] = 0
    level = [(src, 0)]                   # (node, cost) for paths with exactly i edges
    for _ in range(k + 1):               # one level for each flight
        next_level = []
        for u, cost in level:
            for v, p in graph[u]:
                if cost + p < best[v]:   # better than every path with fewer edges
                    best[v] = cost + p
                    next_level.append((v, cost + p))
        level = next_level
    return -1 if best[dst] == float("inf") else best[dst]

flights = [[0, 1, 100], [1, 2, 100], [2, 0, 100], [1, 3, 600], [2, 3, 200]]
cheapest_bfs(4, flights, 0, 3, 1)        # -> 700
short = [[0, 1, 100], [1, 2, 100], [0, 2, 500]]
cheapest_bfs(3, short, 0, 2, 0)          # -> 500
cheapest_bfs(3, short, 0, 2, 1)          # -> 200
```

Do not return when `dst` first appears. A later level can reach it at a lower cost.

57.4.5 Add a resource to the state

The general tool for any limit is a larger state: (node, resource used). Run Dijkstra on these states and order the heap by cost. A state is useless when the same node already popped with a resource use that is not larger. That earlier pop had a cost that is not larger, so it is at least as good in both ways. This handles limits on fuel, total time, coupons or removed obstacles. The edge-count limit is one example. Memory hook: *put the limit in the state; pop by cost; skip a state that uses no less of the limit.*

```
import heapq
from collections import defaultdict

def cheapest_by_state(n, flights, src, dst, k):
    graph = defaultdict(list)
    for u, v, p in flights:
        graph[u].append((v, p))
    fewest = {}                          # node -> fewest edges among its popped states
    heap = [(0, src, 0)]                 # (cost, node, edges used)
    while heap:
        cost, u, used = heapq.heappop(heap)
        if u == dst:
            return cost
        if u in fewest and fewest[u] <= used:
            continue                     # an earlier pop was cheaper and used fewer edges
        fewest[u] = used
        if used == k + 1:
            continue                     # no edges left in the budget
        for v, p in graph[u]:
            heapq.heappush(heap, (cost + p, v, used + 1))
    return -1

flights = [[0, 1, 100], [1, 2, 100], [2, 0, 100], [1, 3, 600], [2, 3, 200]]
cheapest_by_state(4, flights, 0, 3, 1)   # -> 700
cheapest_by_state(4, flights, 0, 3, 2)   # -> 400
short = [[0, 1, 100], [1, 2, 100], [0, 2, 500]]
cheapest_by_state(3, short, 0, 2, 0)     # -> 500
```

A node can pop more than one time here, each time with fewer edges used. A plain visited set is wrong.

57.4.6 Use every edge exactly one time: Euler path

Model the items as edges. An airport, a number or a letter is a node, and each ticket or pair is a directed edge. “Use every ticket one time” is then an Euler path. A greedy walk can get stuck too early. The key fact: the first node where the walk gets stuck must be the last node of the path. So write a node down when it has no unused edges (post-order), and reverse the list at the end. After the reverse, a detour that the walk skipped comes before the dead end, so no choice needs an undo. This is **Hierholzer’s algorithm** (Reconstruct Itinerary). Memory hook: *take the smallest ticket, write the airport down when stuck, then reverse.*

```
from collections import defaultdict

def find_itinerary(tickets):             # Reconstruct Itinerary
    graph = defaultdict(list)
    for src, dst in sorted(tickets, reverse=True):
        graph[src].append(dst)           # each list ends with the smallest destination
    route = []
    def visit(airport):
        while graph[airport]:
            visit(graph[airport].pop())  # use the smallest unused ticket
        route.append(airport)            # stuck: write the airport down
    visit("JFK")
    return route[::-1]

find_itinerary([["MUC", "LHR"], ["JFK", "MUC"], ["SFO", "SJC"], ["LHR", "SFO"]])
```

```
# -> ['JFK', 'MUC', 'LHR', 'SFO', 'SJC']
find_itinerary([["JFK", "SFO"], ["JFK", "ATL"], ["SFO", "ATL"], ["ATL", "JFK"], ["ATL", "SFO"]])
# -> ['JFK', 'ATL', 'JFK', 'SFO', 'ATL', 'SFO']
find_itinerary([["JFK", "KUL"], ["JFK", "NRT"], ["NRT", "JFK"]])
# -> ['JFK', 'NRT', 'JFK', 'KUL']
```

In the last example, the greedy walk takes KUL first and gets stuck. KUL goes to the end of the route. The walk then continues from JFK with the other tickets. Sort each list in descending order and `pop()` from the end: this removes the smallest destination in $O(1)$. The total cost is $O(E \log E)$ for the sort and $O(E)$ for the walk. Both tricks are reusable. A list sorted in descending order is a cheap static priority queue. Use it when you take items only in ascending order and never add new items. The post-order append and the final reverse also give a topological order by finish time (DFS with three colors, Section 56).

General form. When the start is not given, start at the node with out-degree minus in-degree equal to 1. If all nodes are balanced, the path is a circuit, so start at any node with an edge. Use an explicit stack when E is large, because the recursion depth is $E + 1$.

```
from collections import defaultdict

def euler_path(edges):                    # directed edges (u, v); a path is known to exist
    graph = defaultdict(list)
    balance = defaultdict(int)            # out-degree minus in-degree
    for u, v in edges:
        graph[u].append(v)
        balance[u] += 1
        balance[v] -= 1
    start = next((x for x in balance if balance[x] == 1), edges[0][0])
    stack, path = [start], []
    while stack:
        u = stack[-1]
        if graph[u]:
            stack.append(graph[u].pop())  # walk along an unused edge
        else:
            path.append(stack.pop())      # stuck: u takes the next place from the end
    return path[::-1]

euler_path([(3, 1), (1, 2), (2, 3), (3, 4)])          # -> [3, 1, 2, 3, 4]
euler_path([("a", "b"), ("b", "c"), ("c", "a")])      # -> ['a', 'b', 'c', 'a']
euler_path([("a", "b"), ("a", "c")])                  # -> ['a', 'b', 'c']
```

In the first example, the walk goes $3 \to 4$ first and gets stuck. The cycle $3 \to 1 \to 2 \to 3$ is put before it.

A directed Euler path exists when at most one node has out $-$ in $= 1$, at most one node has in $-$ out $= 1$, all other nodes are balanced, and all edges are in one connected piece. For an undirected graph, the number of odd-degree nodes must be 0 or 2. Start at an odd one, and remove each edge from the lists of both ends. When existence is not given, first check the degree conditions. Then check that the result has $E + 1$ nodes. Fewer nodes mean that the edges are not in one connected piece. The length check alone is not enough. The edges a→b and a→c have no Euler path, but the last call above returns $E + 1 = 3$ nodes, and b→c is not an edge.

57.5 Adapt it to a new problem

1. Name the nodes and the edges. Are the edges an explicit list, a grid, or a formula for every pair?
2. Name the goal. A set of edges that connects all nodes: an MST, so go to step 7 to pick Prim or Kruskal. An order that uses every edge: an Euler path. Find the start from the degrees, then run Hierholzer. One path from a source: continue with step 3.
3. For a path, find how the cost combines along the path: sum, max, min or product. Check that the combine never makes a path better as it grows. Then Dijkstra works. To maximize, use a max-heap. For max or min, binary search on the level also works.
4. Check the weights. All equal: BFS. Only 0 and 1: 0-1 BFS with a deque. Non-negative: Dijkstra. Some negative: Bellman-Ford.
5. Look for a second limit. A limit on the number of edges: Bellman-Ford rounds from a copy. A limit on another

resource: Dijkstra on (node, resource) states.

6. Decide when to stop. One target: return at its first pop. All nodes: take the maximum, and check that every node is reached.
7. Check the density. A complete graph with $V \approx 10^3$: an $O(V^2)$ scan with no stored edges. A sparse graph: a heap, or Kruskal.
8. Test the small and impossible cases: one node, an unreachable node, a disconnected graph, a limit of 0.

57.6 Pitfalls

CAUTION

Final on pop, not on push. Wrong: mark a node as done when it is first pushed. A later push can carry a smaller distance. Right: make a node final at its first pop, and skip its later copies. Marking on push is correct only when the cost to enter a node does not depend on the edge.

```python
import heapq
from collections import defaultdict

def build(edges):
    graph = defaultdict(list)
    for u, v, w in edges:
        graph[u].append((v, w))
    return graph

def dijkstra_mark_on_push(graph, src):   # WRONG
    dist = {src: 0}
    heap = [(0, src)]
    while heap:
        d, u = heapq.heappop(heap)
        for v, w in graph[u]:
            if v not in dist:            # the first push fixes v too early
                dist[v] = d + w
                heapq.heappush(heap, (d + w, v))
    return dist

def dijkstra(graph, src):                # right: final at the first pop
    dist = {}
    heap = [(0, src)]
    while heap:
        d, u = heapq.heappop(heap)
        if u in dist:
            continue
        dist[u] = d
        for v, w in graph[u]:
            if v not in dist:
                heapq.heappush(heap, (d + w, v))
    return dist

g = build([("A", "B", 1), ("A", "C", 5), ("B", "C", 1)])
dijkstra_mark_on_push(g, "A")["C"]       # -> 5
dijkstra(g, "A")["C"]                    # -> 2
```

CAUTION

Heap entries that cannot be compared. Wrong: push `(distance, obj)` where `obj` is a dict, a node object or a list of mixed types. On a tie in the distance, Python compares the second items and can raise `TypeError`. Right: put a counter between the key and the object, or push a node id instead of the object.

```python
import heapq
from itertools import count

heap = []
heapq.heappush(heap, (3, {"name": "x"}))
heapq.heappush(heap, (3, {"name": "y"}))     # -> raises TypeError
tie = count()
heap = []
heapq.heappush(heap, (3, next(tie), {"name": "x"}))
heapq.heappush(heap, (3, next(tie), {"name": "y"}))
heapq.heappop(heap)[2]                       # -> {'name': 'x'}
```

CAUTION

Read the graph correctly. Directed edges get one entry; undirected edges get two. Labels from 1 to n need arrays of size $n + 1$. Compare the number of reached nodes with n, not with the nodes that appear in the edge list. Return -1 (or what the statement asks) when a node is not reached, not infinity.

CAUTION

Negative weights. Dijkstra can give wrong answers with a negative edge, because a later path can be cheaper than a node that is already final. Use Bellman-Ford. Zero weights are safe.

CAUTION

Bottleneck paths. Do not sum the values; combine with `max`. Start the heap with the value of the start cell, not with 0. The value of the end cell also counts. In a binary search, the lowest level is at least the start value.

CAUTION

Edge limits. k stops means $k + 1$ edges, so run $k + 1$ rounds. Read from the old array and write to a copy. With a heap, do not keep one best cost for each node and do not use a plain visited set: a node can need a second visit with fewer edges. In a level BFS, do not return at the first arrival at the target: a later level can be cheaper.

CAUTION

MST is not a shortest-path tree. Dijkstra minimizes the distance from one source. An MST minimizes the total of all chosen edges. Example: edges A–B (2), B–C (2) and A–C (3). The MST costs 4 (A–B, B–C). The shortest-path tree from A uses A–B and A–C, and costs 5. On a graph that can be disconnected, Prim can pick a node with cost infinity. Then no spanning tree exists.

CAUTION

Euler paths. Track used edges, not visited nodes: nodes repeat. Keep duplicate edges in a list, not a set. Append a node after its loop, not before it, and reverse the list at the end. `list.pop(0)` costs $O(E)$; sort in descending order and pop from the end, or use a heap. The recursion depth is $E + 1$. Above about 900 edges, use the explicit stack.

57.7 Problems in this family

Problem	What changes	Memory hook
Network Delay Time (743)	Plain Dijkstra; the answer is the largest final distance, or -1 if a node is not reached.	The first pop of a node is its arrival time.
Swim in Rising Water (778)	The path cost is the highest cell, so combine with `max`. Binary search on the level also works.	Dijkstra, but the path cost is the highest cell, so use max instead of plus.
Min Cost to Connect All Points (1584)	The goal is the total of the chosen edges: an MST on a complete graph, with an $O(n^2)$ Prim scan.	Keep a dict of outside points: pop the nearest, then let it lower the others.
Cheapest Flights Within K Stops (787)	At most $k+1$ edges: Bellman-Ford rounds that read from the previous round, or BFS by stops.	$k+1$ rounds: relax every flight from last round's copy.
Reconstruct Itinerary (332)	Use every edge one time, smallest order first: Hierholzer with post-order and a reverse.	Take the smallest ticket, write the airport down when stuck, then reverse.

The transfer drills for this technique are in Section 69.15 (page 423). The answers are in Section A.14 (page 453).

58 Dynamic programming: foundations and 1-D

58.1 The idea

A brute force tries every sequence of choices. It explores a tree of calls, and many branches reach the same subproblem. Dynamic programming (DP) gives each distinct subproblem a name, the *state*. DP solves each state one time and stores the answer. The cost drops from the number of paths, which is often exponential, to the number of states times the work per state.

Worked example. Count the ways to climb 5 steps when each move climbs 1 step or 2 steps. Look at the last move. It lands on step 5 from step 4 or from step 3. These two groups of ways do not overlap, and together they contain every way. So $ways(5) = ways(4) + ways(3)$, and the same rule holds for every step. The plain recursion computes $ways(3)$ two times and $ways(2)$ three times. At $n = 45$ it makes about $3.7 \cdot 10^9$ calls. A table that you fill from step 0 upward computes each value one time: 1, 1, 2, 3, 5, 8. The answer 8 costs 4 additions.

Every DP answers the five questions below. Write the answers in words before you write code.

Part	Question	Climbing Stairs
State	What does one entry of the table mean, in words?	`ways[i]` is the number of move sequences that end exactly on step `i`.
Transition	What is the last decision? How do its cases combine?	The last move is 1 step or 2 steps: `ways[i] = ways[i-1] + ways[i-2]`.
Base case	What is the answer for the smallest input, often the empty prefix?	`ways[0] = 1` (the empty climb is one way) and `ways[1] = 1`.
Order	Which states must be ready before this state?	`i` from 2 up to `n`.
Answer	Which state, or which combination of states, is the answer?	`ways[n]`.

NOTE

The question gives the combine operator. Use a sum for "how many ways", `min` or `max` for "the best total", and `any` (logical or) for "is it possible". The state and the order do not change. Only the value of the base case changes with the operator: 1 way, cost 0, or true.

The transition must use only the state, not the path to it. In Climbing Stairs, the ways to finish from step 3 do not depend on how you reached step 3. When the future depends on more than the state, add that information to the state. Examples are a flag "the first house is robbed" or a second value such as the minimum product.

58.2 Recognize it

- The question asks "how many ways", "the minimum or maximum total", or "is it possible". It does not ask for the list of all solutions.
- Each step has a small set of choices: a move of 1 or 2, take or skip, a piece of length 1 or 2, extend or restart.
- A rule forbids two neighbors, such as "no two adjacent items". Decide take or skip for each item.
- The input must be cut into consecutive pieces (decode, segment, split), and each piece must pass a test. Use a state over the prefix length and list the cases of the last piece.
- A greedy choice fails on a small input. For example, the best houses in `[2, 1, 1, 2]` are not every second house.
- The plain recursion calls the same arguments many times. Overlapping subproblems point to a cache or a table.
- The answer grows exponentially with n, and the statement promises that it fits in 32 bits or asks for it modulo $10^9 + 7$. You cannot list the solutions one by one, so you count them.
- The constraints allow $O(n)$ or $O(n^2)$, but not $O(2^n)$. A 1-D DP over $n \leq 10^5$ needs $O(1)$ or $O(\log n)$ work per state. A limit of a few thousand allows $O(n)$ work per state, for example a look-back over all earlier indices.
- The answer is the best *contiguous* segment (subarray, substring). Use a state that ends at index `i`.
- A step can reverse the order of the candidates, for example a product with negative values. Then the best new result can come from the worst old result. Keep the minimum and the maximum that end at index `i`.
- The answer is the longest *subsequence* where each element relates to the previous chosen element. Use a chain that ends at index `i`.
- The items form a circle, so the first item and the last item are neighbors. Cut the circle into lines by cases.

58.3 Core template

Write the same recurrence in one of two forms.

	Memoized DFS (top-down)	Tabulation (bottom-up)
How to write it	Write the brute-force recursion. Put `@cache` on it. The arguments are the state.	Make a list for all states. Fill it in a loop from the base case upward.
Order of states	The recursion finds the order.	You choose the order. Each state must come after the states that it reads.
States computed	Only the states that the start state reaches.	All states.
Limits	CPython stops at about 1000 nested calls with `RecursionError`. With `@cache` on older versions, the limit is about 500.	No recursion limit.
Space compression	Not practical: the cache keeps every state that it computes.	Keep only the last few entries when the transition reads only them.
Best use	Many dimensions, an unclear order, or a sparse set of reachable states.	A clear order, such as prefix length, and large n.

58.3.1 Tabulation over prefixes

The state `dp[i]` describes the prefix of length `i`. Pad the table by one, so that `dp[0]` is the empty prefix. The template below counts the ways to reach step `n` with moves from a set `steps`. Climbing Stairs is this template with `steps = [1, 2]`.

```
def count_ways(n, steps):
    ways = [0] * (n + 1)          # state: ways[i] = move sequences that end on step i
    ways[0] = 1                   # base case: the empty climb
    for i in range(1, n + 1):     # order: each state reads smaller states only
        for s in steps:           # transition: cases on the last move
            if i - s >= 0:
                ways[i] += ways[i - s]
    return ways[n]                # answer
```

```
count_ways(3, [1, 2])             # -> 3
count_ways(5, [1, 2])             # -> 8
count_ways(45, [1, 2])            # -> 1836311903
count_ways(4, [1, 3, 5])          # -> 3
```

Invariant: when the loop reaches `i`, every entry `ways[0..i-1]` is final. The cases of the last move do not overlap, so each sequence is counted exactly one time.

58.3.2 The same recurrence as a memoized DFS

Start from the brute force: "to finish at `i`, try each last move". Then cache by the arguments.

```
from functools import cache
def count_ways_memo(n, steps):
    @cache
    def ways(i):                  # the state is the argument list
        if i == 0:
            return 1              # base case
        return sum(ways(i - s) for s in steps if i - s >= 0)
    return ways(n)
count_ways_memo(5, [1, 2])        # -> 8
count_ways_memo(45, [1, 2])       # -> 1836311903
```

Define the cached function inside the solver. Then each call of the solver gets a new, empty cache.

58.3.3 State that ends at index i

Some problems do not fit "the best over the prefix". A contiguous segment, or a chain of chosen elements, must connect to its last element. For these problems, let `end[i]` be the best answer that *ends exactly at* index `i`. The answer is then the best value over all `i`, not the last entry. The template below finds the longest chain where each element fits after the previous chosen element.

```
def longest_chain(a, fits):
    end = [1] * len(a)            # end[i] = longest chain that ends at index i
    for i in range(len(a)):
        for j in range(i):        # a[j] is the element before a[i] in the chain
            if fits(a[j], a[i]):
                end[i] = max(end[i], end[j] + 1)
    return max(end)               # the chain can end at any index
nums = [10, 9, 2, 5, 3, 7, 101, 18]
longest_chain(nums, lambda x, y: x < y)                  # -> 4
longest_chain([1, 2, 4, 8, 3], lambda x, y: y % x == 0)  # -> 4
```

This is $O(n^2)$. For a contiguous segment, only index `i - 1` can come before `i`, so one pass of $O(n)$ is enough (Kadane, below).

58.3.4 Compress the table to a few variables

When `dp[i]` reads only the last k entries, keep k variables instead of the table. Update them in one tuple assignment, so that every new value uses the old values.

```
def climb(n):
    prev, curr = 1, 1             # ways[0], ways[1]
    for _ in range(n - 1):
        prev, curr = curr, prev + curr    # both new values use the old ones
    return curr
[climb(n) for n in range(1, 7)]   # -> [1, 2, 3, 5, 8, 13]
def climb_bug(n):
    prev, curr = 1, 1
    for _ in range(n - 1):
        prev = curr               # overwrites prev too early
        curr = prev + curr
    return curr
```

```
[climb_bug(n) for n in range(1, 7)]   # -> [1, 2, 4, 8, 16, 32]
```

Write the full table first. Compress it only when the task asks for $O(1)$ extra space. The table is easier to print, to check by hand, and to walk back for the chosen items.

58.4 Variations

58.4.1 Count the ways: sum over the last move

The core template is the whole solution. Name the last move, add the counts of the states that it comes from, and set the empty prefix to 1. Memory hook (Climbing Stairs): *the last move is a 1-step or a 2-step, so add the ways for the two steps below.*

- Steps from a set S: sum `ways[i - s]` over $s \in S$ with `i - s >= 0`.
- A forbidden step: set its entry to 0 and keep the same transition.
- A large count: apply the modulo after each addition.
- Up to k steps per move: keep a running sum of the last k entries, so that each state costs $O(1)$.
- The positions are the outer loop, so the template counts ordered sequences (1+2 and 2+1 are two ways). For unordered combinations, put the move types in the outer loop. The knapsack section covers this case.

58.4.2 Best cost: min over the last move

Replace the sum with `min` and add the cost of the move. Define exactly what the state includes. Below, `dp[i]` is the cost to stand on step `i` before you pay `cost[i]`. Both start steps are free, and the top is index `n`, one past the last step. Memory hook (Min Cost Climbing Stairs): *to stand on step i, come from one back or two back, and pay the step you leave.*

```
def min_cost_climb(cost):
    n = len(cost)
    dp = [0] * (n + 1)            # dp[i] = cheapest cost to stand on step i; the top is i = n
    for i in range(2, n + 1):     # dp[0] = dp[1] = 0: both starts are free
        from_one_back = dp[i - 1] + cost[i - 1]
        from_two_back = dp[i - 2] + cost[i - 2]
        dp[i] = min(from_one_back, from_two_back)
    return dp[n]
min_cost_climb([10, 15, 20])                          # -> 15
min_cost_climb([1, 100, 1, 1, 1, 100, 1, 1, 100, 1])  # -> 6
min_cost_climb([1, 2, 3])                             # -> 2
```

The last case shows that greedy fails: the cheaper start step 0 costs at least 3, but the start step 1 costs 2. For moves of size 1 to k, take the minimum of `dp[j] + cost[j]` over the last k indices `j`. For a large k, keep that minimum with a monotonic deque.

Some moves cover a span, not a fixed step. An example is a pass that covers 7 days. The span can start before the first day. Then clamp the source index to the base state: `dp[max(0, d - 7)] + cost`. Do not skip the move with the guard `if d - 7 >= 0`. That loses every plan where the first pass starts before day 1. Memory hook (Minimum Cost For Tickets): *the pass that covers today started up to 6 days ago; clamp at day 0.*

```
def min_cost_tickets(days, costs):
    travel, last = set(days), days[-1]
    dp = [0] * (last + 1)         # dp[d] = cheapest cost to cover the travel days up to day d
    for d in range(1, last + 1):
        if d not in travel:
            dp[d] = dp[d - 1]     # no travel today: no new pass
            continue
        dp[d] = min(dp[d - 1] + costs[0],             # last move: a 1-day pass
                    dp[max(0, d - 7)] + costs[1],     # clamp: the pass can start before day 1
                    dp[max(0, d - 30)] + costs[2])
    return dp[last]
min_cost_tickets([1, 4, 6, 7, 8, 20], [2, 7, 15])   # -> 11
min_cost_tickets([1, 2, 3], [5, 6, 100])            # -> 6
```

With the guard instead of the clamp, the last case gives 15: three 1-day passes.

58.4.3 Take or skip each item

Each item is taken or skipped, and a rule forbids two neighbors. Decide only about the last item. Skip it and keep the best of the shorter prefix. Or take it and add it to the best of the prefix that ends before its forbidden neighbor. Memory hook (House Robber): *skip it and keep the last best, or rob it and add its money to the best two back.*

```
def rob(nums):
    best = [0, 0]                 # two zeros for "no houses yet", so best[-2] exists
    for money in nums:
        skip = best[-1]           # leave this house: keep the best so far
        take = best[-2] + money   # rob it: the house before stays empty
        best.append(max(skip, take))
    return best[-1]
rob([1, 2, 3, 1])                 # -> 4
rob([2, 7, 9, 3, 1])              # -> 12
rob([2, 1, 1, 2])                 # -> 4
rob([5])                          # -> 5
```

The two leading zeros remove all base cases, so one house needs no special code. When a take forbids the next k items, the take reads `best[-k-1]`. Start `best` with $k+1$ zeros, so that this entry exists. When an item can be in one of several modes, keep one value per mode (a state machine). The optimal form keeps `best[-2]` and `best[-1]` in two variables for $O(1)$ space.

58.4.4 Recover the choices, not only the value

Keep the full table. Then walk back from the answer and ask which case produced each entry. Memory hook: *walk back from the answer: which case made this entry?*

```
def rob_plan(nums):
    n = len(nums)
    dp = [0] * (n + 1)            # dp[i] = best total from the first i houses
    for i in range(1, n + 1):
        take = (dp[i - 2] if i >= 2 else 0) + nums[i - 1]
        dp[i] = max(dp[i - 1], take)
    plan, i = [], n
    while i > 0:                  # walk back: which case made dp[i]?
        if dp[i] == dp[i - 1]:
            i -= 1                # house i-1 was skipped
        else:
            plan.append(i - 1)    # house i-1 was robbed
            i -= 2
    return dp[n], plan[::-1]
rob_plan([2, 7, 9, 3, 1])         # -> (12, [0, 2, 4])
rob_plan([2, 1, 1, 2])            # -> (4, [0, 3])
```

For a state that ends at `i`, store a parent index next to each entry. Then follow the parents from the best entry.

58.4.5 Circular array: cut the circle by cases

A circle is a line plus one extra link between the two ends. Remove that link by cases on one end: one of the two end items is always unused. Solve each case with the line solver and take the best result. Memory hook (House Robber II): *drop the first or drop the last, rob each row, take the max.*

```
def rob_row(houses):
    two_back, one_back = 0, 0
    for money in houses:
        two_back, one_back = one_back, max(one_back, two_back + money)
    return one_back
def rob_circle(nums):
    if len(nums) == 1:
        return nums[0]            # one house has no neighbor
```

```
    return max(rob_row(nums[1:]), rob_row(nums[:-1]))   # drop the first, or drop the last
rob_circle([2, 3, 2])             # -> 3
rob_circle([1, 2, 3, 1])          # -> 4
rob_circle([5, 1, 1, 5])          # -> 6
rob_circle([7])                   # -> 7
rob_row([2, 3, 2])                # -> 4
```

The last line shows the bug of one run on the whole array: it robs both ends. The general rule: fix the few possible states of one end item, solve each case as a line, and combine. A best segment on a circle needs a different split: the segment wraps past the end, or it does not. A wrapped segment is the whole array minus one segment that does not wrap. The slices copy the list. Pass index bounds instead when the array is very large.

58.4.6 Moves with validity guards

The shape is Climbing Stairs, but each move is allowed only when a local test passes. Put each test on its own transition. Do not handle the special values outside the DP. Memory hook (Decode Ways): *Climbing Stairs with guards: one step if the digit is not 0, two steps if the pair is 10 to 26.*

```
def num_decodings(s):
    n = len(s)
    dp = [0] * (n + 1)            # dp[i] = ways to decode s[:i]
    dp[0] = 1                     # the empty prefix has one way
    for i in range(1, n + 1):
        if s[i - 1] != "0":                         # last piece: one digit
            dp[i] += dp[i - 1]
        if i >= 2 and 10 <= int(s[i - 2:i]) <= 26:   # last piece: two digits
            dp[i] += dp[i - 2]
    return dp[n]
num_decodings("12")               # -> 2
num_decodings("226")              # -> 3
num_decodings("11106")            # -> 2
num_decodings("06")               # -> 0
num_decodings("30")               # -> 0
```

A `"0"` cannot start a piece. It is valid only as the second digit of `"10"` or `"20"`. The test `10 <= value` rejects a two-digit piece with a leading zero, such as `"05"`. With `dp` over prefix length, the last digit is `s[i-1]`, and the last two digits are `s[i-2:i]`.

58.4.7 Split into pieces from a set

The input must be cut into consecutive pieces, and each piece must come from an allowed set. The rest of the string depends only on where it starts, not on the earlier cuts. So cache the recursion by the start index. Memory hook (Word Break): *cut off a first word, ask if the rest can split, and cache by start index.*

```
from functools import cache
def word_break(s, word_dict):
    words = set(word_dict)
    @cache
    def can_split(start):         # True when s[start:] splits into words
        if start == len(s):
            return True           # nothing is left to split
        for end in range(start + 1, len(s) + 1):
            if s[start:end] in words and can_split(end):
                return True
        return False
    return can_split(0)
word_break("leetcode", ["leet", "code"])                        # -> True
word_break("applepenapple", ["apple", "pen"])                   # -> True
word_break("catsandog", ["cats", "dog", "sand", "and", "cat"])  # -> False
word_break("cars", ["car", "ca", "rs"])                         # -> True
```

The optimal form is bottom-up over prefixes: `dp[i]` is true when `s[:i]` splits. It looks back at most L characters,

where L is the longest word length. So the slice work drops from $O(n^3)$ to $O(nL^2)$. It also has no recursion limit, which matters for strings of thousands of characters. The bottom-up form is the general partition template. Change only the combine operator.

```
def split_count(s, word_dict):    # number of ways to split s into words
    words = set(word_dict)
    longest = max(map(len, words))
    n = len(s)
    dp = [0] * (n + 1)            # dp[i] = ways to split the prefix s[:i]
    dp[0] = 1
    for i in range(1, n + 1):
        for j in range(max(0, i - longest), i):   # the last piece is s[j:i]
            if s[j:i] in words:
                dp[i] += dp[j]
    return dp[n]
split_count("catsanddog", ["cat", "cats", "and", "sand", "dog"])   # -> 2
split_count("aaaa", ["a", "aa"])                                   # -> 5
split_count("catsandog", ["cats", "dog", "sand", "and", "cat"])    # -> 0
```

For a yes or no answer, use `dp[i] = dp[i] or dp[j]`. For the fewest pieces, use `dp[i] = min(dp[i], dp[j] + 1)` with `inf` as the start value.

58.4.8 Best segment that ends here (Kadane)

For the best contiguous segment, let `cur` be the best sum of a segment that ends at the current index. That segment is the current element alone, or the current element added to the best segment that ends one index earlier. A negative earlier sum never helps, so clamp it at 0 before the addition. Memory hook (Maximum Subarray): *a negative running sum only hurts, so drop it and restart.*

```
def max_subarray(nums):
    best = nums[0]                # non-empty: do not start at 0
    cur = 0                       # best sum that ends at the previous index
    for x in nums:
        cur = max(cur, 0) + x     # extend a positive run, or restart at x
        best = max(best, cur)
    return best
max_subarray([-2, 1, -3, 4, -1, 2, 1, -5, 4])   # -> 6
max_subarray([5, 4, -1, 7, 8])                  # -> 23
max_subarray([-3, -1, -2])                      # -> -1
```

Restart when the running sum is negative, not when the current element is negative. In `[4, -1, 2, 1]` the `-1` is part of the best segment. When one state is not enough (a deletion is allowed, the sign flips), keep a small fixed set of states per index.

58.4.9 Carry the max and the min

A multiplication by a negative number reverses the order of the candidates. The smallest product becomes the largest. So carry both the largest and the smallest product that end at the current index. Memory hook (Maximum Product Subarray): *Kadane, but carry the biggest and the smallest, because a negative swaps them.*

```
def max_product(nums):
    best = nums[0]
    big = small = 1               # largest and smallest product that ends here
    for x in nums:
        candidates = (x, big * x, small * x)          # x alone, or an old run times x
        big, small = max(candidates), min(candidates)   # both from the OLD values
        best = max(best, big)
    return best
max_product([2, 3, -2, 4])        # -> 6
max_product([-2, 3, -4])          # -> 24
max_product([-2, 0, -1])          # -> 0
max_product([-2])                 # -> -2
```

The general rule applies to any step that is monotone but can reverse the order. For such a step, the extremes of the new values come from the extremes of the old values. So keep the range `[min, max]` as the state. The candidate `x` alone restarts the run after a zero.

58.4.10 Longest chain: increasing subsequence

The $O(n^2)$ form is the chain template above with `fits = x < y`. It is enough for n up to a few thousand. The $O(n \log n)$ form keeps, for each chain length, only the smallest last value of any chain with that length. A smaller last value is always at least as easy to extend. These best tails stay sorted, so a binary search finds the place of each new value. Memory hook (Longest Increasing Subsequence): *smallest tail per length:* ***`bisect_left`****, then replace or append.*

```
from bisect import bisect_left, bisect_right
def lis_tails(nums, strict=True):
    find = bisect_left if strict else bisect_right
    tails = []                    # tails[k] = smallest last value of a chain of length k + 1
    for x in nums:
        pos = find(tails, x)      # first tail >= x (strict) or > x
        if pos == len(tails):
            tails.append(x)       # x extends the longest chain
        else:
            tails[pos] = x        # x is a smaller tail for length pos + 1
    return tails
len(lis_tails([10, 9, 2, 5, 3, 7, 101, 18]))   # -> 4
len(lis_tails([0, 1, 0, 3, 2, 3]))             # -> 4
len(lis_tails([7, 7, 7]))                      # -> 1
len(lis_tails([7, 7, 7], strict=False))        # -> 3
lis_tails([3, 4, 1])                           # -> [1, 4]
```

The last line shows that `tails` is not a real subsequence: `1` comes after `4` in the input. Only its length is correct. To rebuild a real chain, store a parent index for each element. The general idea is dominance: of two partial solutions with the same score, keep the one that is easier to extend. Many 2-D chain problems become this problem after a sort, when both keys must be strictly larger. Sort by the first key going up. For equal first keys, sort the second key going down. Then two items with the same first key cannot both be in a strictly increasing chain. Run the tails loop on the second key. If equal first keys are sorted going up on the second key, items with the same first key chain, and the answer is too large.

58.5 Adapt it to a new problem

Ask these questions in order.

1. Does the question ask for a count, a best value, or a yes or no answer? If it asks for all solutions, use backtracking instead.
2. Write the brute force as a recursion over choices. Its arguments are a first try at the state.
3. Does the rest of the problem depend only on these arguments? If it also depends on the path, add the missing fact to the state.
4. Is the answer a contiguous segment or a chain? Then use a state that ends at index `i`. Otherwise use the best over the prefix of length `i`.
5. What is the last decision: the last move, the last piece, take or skip the last item, extend or restart? List its cases. For a count, the cases must not overlap, and together they must cover every solution.
6. Choose the combine operator: a sum, `min` or `max`, or `any`.
7. Set the base case. The empty prefix usually has 1 way, cost 0, or the value true. Pad the table by one to hold it.
8. Choose the order, so that each state comes after the states that it reads.
9. Find the answer: `dp[n]` for a prefix state, or the best over all `i` for an "ends at i" state.
10. Multiply the number of states by the work per state. Compare the result with the constraints. If it is too slow, look for a dominance rule (keep only the best of equal candidates) or a bound on the look-back.
11. Is there an extra link between the ends (a circle)? Fix the state of one end item, solve each case as a line, and combine.
12. Check the code against a brute force on all small inputs. Then compress the table if the task asks for less space.

58.6 Pitfalls

CAUTION

A wrong base case. The empty prefix has 1 way to count, not 0. With 0, every count is 0. Word Break needs `dp[0] = True` for the same reason.

CAUTION

A non-empty answer that starts at 0. When the segment must be non-empty and values can be negative, start the best at `nums[0]`. A start value of 0 returns 0 for `[-3, -1, -2]`. For a product, starting both states at `nums[0]` and then looping over all of `nums` counts `nums[0]` two times: `[2]` gives 4.

CAUTION

The last entry of an "ends at i" table. The longest chain or the best segment can end anywhere. Return the best over all `i`, not `end[n-1]`.

CAUTION

Greedy instead of DP. Every second item, the largest item first, or the longest word first can all fail. Try a small counterexample before you trust a greedy rule.

```
def rob(nums):
    best = [0, 0]
    for money in nums:
        best.append(max(best[-1], best[-2] + money))
    return best[-1]
def rob_every_other(nums):        # greedy: the better of the even and the odd positions
    return max(sum(nums[0::2]), sum(nums[1::2]))
rob_every_other([2, 1, 1, 2]), rob([2, 1, 1, 2])   # -> (3, 4)
```

CAUTION

The recursion limit. A memoized DFS goes as deep as the longest chain of states. CPython raises `RecursionError` at about 1000 nested calls. On Python 3.11 and older, and on early 3.12 releases, the `@cache` wrapper also counts. Then the limit is about 500 calls. Use tabulation when the chain can be that long.

```
from functools import cache
@cache
def down(i):
    return 0 if i == 0 else down(i - 1)
down(400)                         # -> 0
down(5000)                        # -> raises RecursionError
```

CAUTION

A cache that outlives its input. A module-level `@cache` function that reads a global list keeps old answers for the next input. The list is a hidden part of the state. Define the cached function inside the solver, so that each input gets a new cache.

```
from functools import cache
nums = [1, 2, 3]
@cache
def best_from(i):                 # reads the global nums: a hidden part of the state
    if i >= len(nums):
        return 0
    return max(best_from(i + 1), nums[i] + best_from(i + 2))
best_from(0)                      # -> 4
nums = [10, 1, 10]
best_from(0)                      # -> 4
best_from.cache_clear()
best_from(0)                      # -> 20
```

CAUTION

A negative slice start. With `dp` over prefix length, the two-digit piece `s[i-2:i]` at `i = 1` is `s[-1:1]`. It is an empty string, and `int` of it raises `ValueError`. Test `i >= 2` first.

```
s = "12"
i = 1
s[i - 2:i]                      # -> ''
int(s[i - 2:i])                 # -> raises ValueError
```

CAUTION

Two separate updates of rolling variables. `prev = curr` followed by `curr = prev + curr` reads the new `prev`. Use one tuple assignment, as in `prev, curr = curr, prev + curr`. The same rule applies to the max and min of a product.

CAUTION

The circle edge cases. With one item, both rows of the case split are empty, so return that item. Do not exclude both ends in one run: that loses every plan that uses exactly one end, such as `[5, 1, 1, 5]`.

CAUTION

Strict or non-strict chains. `bisect_left` gives a strictly increasing chain. `bisect_right` lets equal values chain. In the $O(n^2)$ form, `<` and `<=` make the same difference.

58.7 Problems in this family

Problem	What changes	Memory hook
Climbing Stairs	Count the ways: a sum over the last move (1 or 2 steps). The empty climb is 1 way.	The last move is a 1-step or a 2-step, so add the ways for the two steps below.
Min Cost Climbing Stairs	Best cost: a `min` over the last move, plus the cost of the step you leave. The top is index `n`.	To stand on step i, come from one back or two back, and pay the step you leave.
House Robber	Take or skip each item. A take reads the best two back.	Skip it and keep the last best, or rob it and add its money to the best two back.
House Robber II	A circle: run the line solver without the first item and without the last item. One item is a special case.	Drop the first or drop the last, rob each row, take the max.
Decode Ways	Count with guards: one digit if it is not `"0"`, two digits if they are 10 to 26.	Climbing Stairs with guards: one step if the digit is not 0, two steps if the pair is 10 to 26.
Word Break	Yes or no over the last piece from a set. Memoized DFS on the start index, or bottom-up with a bounded look-back.	Cut off a first word, ask if the rest can split, and cache by start index.
Maximum Subarray	The best sum that ends at `i`: extend or restart. The answer is the best over all `i`.	A negative running sum only hurts, so drop it and restart.
Maximum Product Subarray	Two states that end at `i`: the max and the min product.	Kadane, but carry the biggest and the smallest, because a negative swaps them.
Longest Increasing Subsequence	A chain that ends at `i` in $O(n^2)$, or the smallest tail per length with binary search in $O(n \log n)$.	Smallest tail per length: `bisect_left`, then replace or append.

The transfer drills for this technique are in Section 69.16 (page 424). The answers are in Section A.15 (page 454).

59 Dynamic programming: knapsack family

59.1 The idea

A knapsack problem picks items so that their sizes add up to a *total*. The total can be an amount of money, a capacity, a subset sum, or a position in a string. Brute force tries every subset or every sequence of items. That is 2^n choices or more. But the future depends only on the total so far, not on the items that made it. So merge all the choices with the same total into one cell. Keep one cell for each total $0, 1, \ldots, T$, and add the items one at a time. The work drops to $O(n \cdot T)$.

Each cell holds one answer for its total: the fewest items, the number of ways, the best value, or only yes or no. Three questions fix the code:

Question	Answer	Code
Can an item repeat?	no: each item at most once (0/1)	Inner loop over the totals goes down: `range(T, w - 1, -1)`.
	yes: unlimited copies (unbounded)	Inner loop goes up: `range(w, T + 1)`.
What does the cell hold?	fewest items	`dp[0] = 0`, other cells `inf`; `dp[t] = min(dp[t], dp[t - w] + 1)`.
	number of ways	`dp[0] = 1`, other cells `0`; `dp[t] += dp[t - w]`.
	possible or not	`dp[0] = True`, other cells `False`; `dp[t] = dp[t] or dp[t - w]`.
	best value	`dp[t] = max(dp[t], dp[t - w] + v)`.
Does the order count? (counting only)	no: combinations	Items in the outer loop, totals inside.
	yes: ordered sequences	Totals in the outer loop, items inside. Only with unlimited copies.

Worked example (fewest items). Make the amount 6 from coins 1, 3 and 4, with unlimited copies. Let `dp[a]` be the fewest coins for the amount `a`. Fix the last coin: `dp[a]` is 1 plus the best of `dp[a - 1]`, `dp[a - 3]` and `dp[a - 4]`. Start with `dp[0] = 0`. Then `dp[1] = 1`, `dp[2] = 2`, `dp[3] = 1`, `dp[4] = 1` and `dp[5] = 2` $(4 + 1)$. Last, `dp[6]` is 1 plus the best of `dp[5]`, `dp[3]` and `dp[2]`, so it is 2 $(3 + 3)$. A greedy choice takes the coin 4 first and needs three coins $(4 + 1 + 1)$.

Worked example (loop direction). One item of size 2, totals 0 to 4, yes or no. Loop up: `t = 2` reads `dp[0]` and becomes `True`. Then `t = 4` reads `dp[2]`, which is already `True` in this pass. So the one item counts two times. Loop down: `t = 4` reads `dp[2]` while it still holds the old `False`. Then `t = 2` becomes `True`. The item counts one time only.

A 1-D table is the 2-D table `dp[i][t]` ("first `i` items, total `t`") with only one row kept. The downward loop reads cells that are still from the row before item `i`. That is the 0/1 rule. The upward loop reads cells that already include item `i`. That is the "take one more copy" rule.

59.2 Recognize it

- Items must "make an amount", "reach exactly a target sum" or "fit a capacity".
- The question asks for the fewest items, the number of ways, the best value, or only yes or no.
- The target, or the sum of all values, is small (up to about 10^4), and $n \cdot T$ is a few million at most. Index the table by the total, even when single values are huge.
- The total must be *at least* a bound P, such as a profit of at least P. Cap the total at P. The table then has $P + 1$ cells, also when the sum of all values is large.
- The statement says that each item is used once, or it says "unlimited supply" or "any number of times".
- The statement says "combinations" (order does not matter), or "different orders count as different" (ordered sequences). Read the examples to be sure.
- Each item goes into one of two groups, or gets a plus or a minus sign. Rewrite it as a subset sum.
- A string must be cut into pieces from a set. The position in the string plays the total.

- A greedy choice fails on a small counterexample, such as coins 1, 3, 4 and the amount 6.
- $n \leq 20$ and the items go into $k > 2$ groups. The total alone is not enough state. Use a bitmask of the used items.

59.3 Core template

Remember this skeleton. Only the start values, the loop direction and the combine step change.

```
dp = [START] + [EMPTY] * T          # min: 0, inf   count: 1, 0   yes or no: True, False
for w in items:                     # items outside
    for t in range(T, w - 1, -1):   # each item once: down. Unlimited: range(w, T + 1)
        dp[t] = COMBINE(dp[t], dp[t - w])   # min(a, b + 1)   a + b   a or b
answer = dp[T]
```

One tested function covers the three cell types and both loop directions. The items are in the outer loop.

```
from itertools import combinations, combinations_with_replacement

INF = float("inf")

def knapsack(items, T, reuse, kind):
    # dp[t] = answer for the total exactly t, with the items seen so far
    start = {"min": 0, "count": 1, "can": True}[kind]       # the empty choice
    empty = {"min": INF, "count": 0, "can": False}[kind]    # not reached yet
    dp = [start] + [empty] * T
    for w in items:                          # items outside: one build order
        if reuse:
            totals = range(w, T + 1)         # up: dp[t - w] may already use w
        else:
            totals = range(T, w - 1, -1)     # down: dp[t - w] is from before w
        for t in totals:
            if kind == "min":
                dp[t] = min(dp[t], dp[t - w] + 1)
            elif kind == "count":
                dp[t] += dp[t - w]
            else:
                dp[t] = dp[t] or dp[t - w]
    return dp[T]

knapsack([1, 3, 4], 6, True, "min")          # -> 2
knapsack([1, 3, 4], 6, False, "min")         # -> inf
knapsack([1, 3, 4], 7, False, "min")         # -> 2
knapsack([1, 2, 3], 3, True, "count")        # -> 3
knapsack([1, 2, 3], 3, False, "count")       # -> 2
knapsack([1, 5, 11, 5], 11, False, "can")    # -> True
knapsack([1, 3], 2, False, "can")            # -> False
knapsack([1, 3], 2, True, "can")             # -> True

def brute_subsets(items, T):                 # each item at most once
    return sum(sum(c) == T for r in range(len(items) + 1)
               for c in combinations(items, r))

def brute_multisets(items, T):               # unlimited copies
    return sum(sum(c) == T for r in range(T + 1)
               for c in combinations_with_replacement(items, r))

tests = [([1, 2, 3], 5), ([2, 3, 5, 7], 10), ([1, 4, 6], 8), ([3, 5], 7)]
all(knapsack(a, T, False, "count") == brute_subsets(a, T) and
    knapsack(a, T, True, "count") == brute_multisets(a, T) for a, T in tests)   # -> True
```

Invariant: after the outer loop finishes item `i`, `dp[t]` is the answer for the total `t` with only the items `0..i`. A total that no choice reaches keeps the "empty" value. Time $O(n \cdot T)$, space $O(T)$.

Best value with weights. The classic knapsack gives each item a weight and a value. The cell holds the best value. Start every cell at 0 for "total weight at most `c`". Start only `dp[0]` at 0 and the other cells at minus infinity for "total weight exactly `c`".

```
def best_value(items, cap, reuse=False, exact=False):
    # items: (weight, value) pairs. dp[c] = best value for the weight c
    NEG = float("-inf")
    dp = [0] + [NEG if exact else 0] * cap
    for w, v in items:
        totals = range(w, cap + 1) if reuse else range(cap, w - 1, -1)
        for c in totals:
            dp[c] = max(dp[c], dp[c - w] + v)
    return dp[cap]

items = [(2, 3), (3, 4), (4, 5)]
best_value(items, 7)                  # -> 9
best_value(items, 7, reuse=True)      # -> 10
best_value(items, 1)                  # -> 0
best_value(items, 1, exact=True)      # -> -inf
```

NOTE

$O(n \cdot T)$ is *pseudo-polynomial*. It grows with the value of T, not with its number of digits. A table for $T = 10^9$ does not fit. Then look for a small n (bitmask or meet in the middle) or for a different state.

59.4 Variations

59.4.1 Fewest items with unlimited copies

Fix the last item. The rest is the same problem for a smaller total (Coin Change). The cell for `a` is 1 plus the best cell for `a - c` over every coin `c <= a`. Fill the amounts upward. Memory hook: *pick the last coin: `dp[a] = 1 + min of dp[a - coin]`, with `dp[0] = 0`.*

```
def coin_change(coins, amount):
    INF = float("inf")
    dp = [0] + [INF] * amount          # dp[a] = fewest coins for the amount a
    for a in range(1, amount + 1):
        for c in coins:
            if c <= a:                 # guard: a negative index reads the end of the list
                dp[a] = min(dp[a], dp[a - c] + 1)
    return dp[amount] if dp[amount] != INF else -1

coin_change([1, 3, 4], 6)     # -> 2
coin_change([1, 2, 5], 11)    # -> 3
coin_change([2], 3)           # -> -1
coin_change([1], 0)           # -> 0
```

- For a min (or a max, or yes or no), the loop order does not change the result. The amounts can be outside, as here, or the coins can be outside, as in the core template.
- The items have a price: add the price instead of 1.
- The items are a fixed family, such as the squares $1, 4, 9, \ldots$: generate them, then run the same loop.
- Each amount is a node and each coin is an edge of length 1. A BFS from 0 finds the same answer.

59.4.2 Count combinations or count ordered sequences

When the cell counts ways, the loop order decides *what* it counts. With the items outside, the items join in one fixed order, so each multiset is built once: these are combinations (Coin Change II). With the totals outside, each total tries every item as the last pick, so $1 + 2$ and $2 + 1$ are two different paths: these are ordered sequences (Combination Sum IV). In both, `dp[0] = 1` counts the empty choice. Memory hooks: *coins outside, amounts going up inside: `ways[a] += ways[a - coin]`* (combinations); *ways to make a total: try every last number, add the ways for what is left* (ordered sequences).

```
def change(amount, coins):              # combinations: coins outside
    ways = [1] + [0] * amount
    for coin in coins:
        for a in range(coin, amount + 1):     # up: the coin can repeat
            ways[a] += ways[a - coin]
    return ways[amount]

def combination_sum4(nums, target):     # ordered sequences: totals outside
    ways = [1] + [0] * target
    for total in range(1, target + 1):
        for last in nums:                     # every number can be the last pick
            if last <= total:
                ways[total] += ways[total - last]
    return ways[target]

change(4, [1, 2, 3])             # -> 4
combination_sum4([1, 2, 3], 4)   # -> 7
change(5, [1, 2, 5])             # -> 4
change(0, [7])                   # -> 1
combination_sum4([9], 3)         # -> 0
combination_sum4([1, 2], 5)      # -> 8
```

The last line is Climbing Stairs with 5 steps: ordered sequences of steps 1 and 2. General rule: the outer loop of a counting DP fixes the order in which the choices are made. Choose it from the question: are two orders of the same items one answer or two answers?

CAUTION

The title Combination Sum IV says "combination", but its examples count $(1, 1, 2)$, $(1, 2, 1)$ and $(2, 1, 1)$ as three answers. Trust the examples, not the title.

59.4.3 Each item at most once: subset sum and equal split

Two equal halves exist only if one subset reaches `total / 2`. The other items form the second half (Partition Equal Subset Sum). An odd total gives `False` at once. The intuitive solution keeps the *set* of reachable sums. Memory hook: *odd total means no; otherwise grow the set of reachable sums and ask if half the total is in it.* The optimal solution keeps the same facts in a boolean list and loops the totals down. It can stop early when the target cell becomes `True`.

```
def can_partition(nums):                # set of reachable sums
    total = sum(nums)
    if total % 2:
        return False
    sums = {0}                          # the empty subset
    for x in nums:
        sums |= {s + x for s in sums}   # skip x (old sums) or take x (s + x)
    return total // 2 in sums

def can_partition_table(nums):          # boolean list, totals go down
    total = sum(nums)
    if total % 2:
        return False
    target = total // 2
    dp = [True] + [False] * target
    for x in nums:
        for j in range(target, x - 1, -1):
            dp[j] = dp[j] or dp[j - x]
    return dp[target]

def wrong_upward(nums, target):         # BUG: one x is used many times
    dp = [True] + [False] * target
    for x in nums:
```

```
        for j in range(x, target + 1):
            dp[j] = dp[j] or dp[j - x]
    return dp[target]

can_partition([1, 5, 11, 5]), can_partition_table([1, 5, 11, 5])   # -> (True, True)
can_partition([1, 2, 3, 5]), can_partition_table([1, 2, 3, 5])     # -> (False, False)
wrong_upward([1, 3], 2)                # -> True
```

The set comprehension reads only the old set, so each number is used once. The last line shows the bug: the subsets of `[1, 3]` cannot make 2, but the upward loop uses the 1 two times.

- Two groups with the smallest difference: find the largest reachable sum `s <= total // 2`. The difference is `total - 2 * s`.
- Count the subsets with a given sum: same loop, with `+=` and `dp[0] = 1`.

59.4.4 Subset sums as a bitset

For yes or no questions, store all the reachable sums as the bits of one Python integer. Bit `s` is 1 when the sum `s` is reachable. A shift by `x` adds `x` to every sum at the same time. CPython does the shift and the OR in C, 30 bits per step. So this is much faster than a loop over a list. It works only for yes or no. It cannot hold counts or minimums. Memory hook: *one integer holds every sum; shift by* x *and OR to add* x *to all of them.*

```
def reachable_bits(nums):
    bits = 1                          # bit 0: the empty subset has the sum 0
    for x in nums:
        bits |= bits << x             # each old sum s also gives s + x
    return bits

bits = reachable_bits([1, 5, 11, 5])
bits >> 11 & 1                        # -> 1
[s for s in range(bits.bit_length()) if bits >> s & 1]
# -> [0, 1, 5, 6, 10, 11, 12, 16, 17, 21, 22]
total = 22
max(s for s in range(total // 2 + 1) if bits >> s & 1)   # -> 11

def reachable_upto(nums, target):
    mask = (1 << (target + 1)) - 1    # keep the bits 0..target only
    bits = 1
    for x in nums:
        bits = (bits | bits << x) & mask
    return [s for s in range(target + 1) if bits >> s & 1]

reachable_upto([3, 4, 5], 7)          # -> [0, 3, 4, 5, 7]
```

The shift binds tighter than `&`, so `bits >> s & 1` reads bit `s`. The mask keeps the integer small when sums above the target can never help.

59.4.5 Signs on every item: rewrite as a subset sum

Each number gets a plus or a minus sign, and the question counts the sign choices that hit a target (Target Sum). The intuitive solution is a memoized DFS on (index, running sum). The running sum stays in $[-total, total]$, so there are at most $(n+1)\cdot(2\cdot total+1)$ states. Memory hook: *plus or minus each number, cache on (index, running sum).*

The optimal solution removes the signs. Let P be the sum of the plus group and N the sum of the minus group. Then $P - N = target$ and $P + N = total$, so $P = (total + target)/2$. Now count the subsets with the sum P: a 0/1 counting knapsack with the totals going down.

```
from functools import cache

def target_sum_memo(nums, target):
    @cache
    def ways(i, running):              # sign choices for nums[i:] that end at target
```

```
        if i == len(nums):
            return 1 if running == target else 0
        return ways(i + 1, running + nums[i]) + ways(i + 1, running - nums[i])
    return ways(0, 0)

def target_sum_subset(nums, target):
    total = sum(nums)
    if abs(target) > total or (total + target) % 2:
        return 0
    goal = (total + target) // 2       # the sum of the plus group
    dp = [1] + [0] * goal
    for x in nums:
        for s in range(goal, x - 1, -1):    # down: each number once
            dp[s] += dp[s - x]
    return dp[goal]

target_sum_memo([1, 1, 1, 1, 1], 3), target_sum_subset([1, 1, 1, 1, 1], 3)   # -> (5, 5)
target_sum_memo([0, 0, 1], 1), target_sum_subset([0, 0, 1], 1)               # -> (4, 4)
target_sum_subset([1, 2], -4), target_sum_subset([1, 2], 2)                  # -> (0, 0)
```

The general trick: when a rule mixes a quantity with its complement, write the complement as the total minus the quantity. This removes one unknown and turns a two-sided choice into a subset choice. The same move turns "two equal groups" into "one subset with half the total".

59.4.6 Segmentation: the total is a position

Cut a string into dictionary words (Word Break). The position in the string plays the total. The words are items with unlimited copies. The cell holds yes or no, so the combine step is `or`. The intuitive solution is a memoized DFS on the start index. Memory hook: *cut off a first word, ask if the rest can split, and cache by start index.* The optimal solution is the same idea bottom-up over prefixes: `dp[i]` is `True` when `s[:i]` splits. It looks back at most the longest word length and needs no recursion.

```
from functools import cache

def word_break(s, word_dict):
    words = set(word_dict)
    @cache
    def can_split(start):              # True when s[start:] splits into words
        if start == len(s):
            return True
        for end in range(start + 1, len(s) + 1):
            if s[start:end] in words and can_split(end):
                return True
        return False
    return can_split(0)

def word_break_table(s, word_dict):    # dp[i]: the prefix s[:i] splits
    words = set(word_dict)
    longest = max(map(len, words))
    dp = [True] + [False] * len(s)
    for i in range(1, len(s) + 1):
        for j in range(max(0, i - longest), i):     # the last word is s[j:i]
            if dp[j] and s[j:i] in words:
                dp[i] = True
                break
    return dp[len(s)]

def count_splits(s, word_dict):        # the same table with a sum
    words = set(word_dict)
    dp = [1] + [0] * len(s)
    for i in range(1, len(s) + 1):
        dp[i] = sum(dp[j] for j in range(i) if s[j:i] in words)
```

```
    return dp[len(s)]

word_break("leetcode", ["leet", "code"])                          # -> True
word_break("catsandog", ["cats", "dog", "sand", "and", "cat"])    # -> False
word_break("cars", ["car", "ca", "rs"]), word_break_table("cars", ["car", "ca", "rs"])   # -> (True,
    True)
count_splits("catsanddog", ["cat", "cats", "and", "sand", "dog"])   # -> 2
```

Change the combine step for related questions: a sum counts the splits, and `min(dp[j] + 1)` finds the fewest words. A greedy longest match fails: `"cars"` needs `"ca" + "rs"`, not `"car"`.

59.4.7 More than two groups: a bitmask of used items

With two groups, one subset fixes the other, so a table over sums works. With $k > 2$ groups, one valid group does not guarantee the rest (Partition to K Equal Sum Subsets). A greedy fails: for `[1, 1, 1, 1, 2, 2, 2, 2]` and $k = 4$, the group $1 + 1 + 1$ leaves no valid split, but $1 + 2$ four times works. So the state must be the *set* of used items, as an n-bit mask. Fill one group at a time to remove the symmetry between groups. The fill of the open group is the used sum modulo the group target, so the mask alone fixes the state. Memory hook: *mask of used numbers, fill is used sum mod target, try every unused number that fits.*

```
from functools import cache

def can_partition_k(nums, k):
    total = sum(nums)
    if total % k:
        return False
    target = total // k
    n = len(nums)
    all_used = (1 << n) - 1

    @cache
    def can_finish(used, fill):        # fill == (sum of the used numbers) % target
        if used == all_used:
            return True                # every group closed at exactly target
        for i in range(n):
            if used >> i & 1 or fill + nums[i] > target:
                continue               # already placed, or does not fit
            if can_finish(used | 1 << i, (fill + nums[i]) % target):
                return True
        return False

    return can_finish(0, 0)

can_partition_k([4, 3, 2, 3, 5, 2, 1], 4)       # -> True
can_partition_k([1, 2, 3, 4], 3)                # -> False
can_partition_k([1, 1, 1, 1, 2, 2, 2, 2], 4)    # -> True
```

There are at most 2^n masks with n steps each, so the cost is $O(n \cdot 2^n)$. In CPython that is under half a second for $n \leq 16$, but several seconds near $n = 20$. The optimal version adds pruning. Sort the numbers, and return `False` when the largest number exceeds the target. Stop the loop at the first number that does not fit. Skip a value when an earlier equal copy is still unused. The pruning helps most when values repeat. It does not change the bound.

The same state works for other "assign $n \leq 20$ items" problems. Keep the used mask. Add only what the future still needs: a fill, a count of closed groups, or the last item.

59.4.8 Rebuild the chosen items

The table gives a number. To list the items, store which item last improved each cell. Then walk back from the target. Memory hook: *a second list remembers the last item of each cell; follow it back to 0.*

```python
def fewest_coins_list(coins, amount):
    INF = float("inf")
    dp = [0] + [INF] * amount
    last = [0] * (amount + 1)          # last[a] = the coin that set dp[a]
    for a in range(1, amount + 1):
        for c in coins:
            if c <= a and dp[a - c] + 1 < dp[a]:
                dp[a] = dp[a - c] + 1
                last[a] = c
    if dp[amount] == INF:
        return None
    picked = []
    while amount:
        picked.append(last[amount])
        amount -= last[amount]
    return picked

fewest_coins_list([1, 3, 4], 6)    # -> [3, 3]
fewest_coins_list([2], 3)          # -> None
```

For a 0/1 problem, the 1-D table forgets which row a cell came from. Keep the full 2-D table `dp[i][t]` and walk back from `i = n` to 1: if `dp[i][t] != dp[i - 1][t]`, item `i` is in the answer, so subtract its size from `t`. Otherwise skip item `i`.

59.4.9 More limits, groups and bounded copies

Memory hook: *one table dimension per limit; one new row per group.*

- **Two capacities** (for example weight and volume): use one table dimension per capacity. For 0/1 items, loop every capacity down.
- **Exactly one item from each group**: build a new row for each group from the old row. Do not update in place. An in-place update lets two items of one group both count. It also keeps the old value, so a group can be skipped.
- **A fixed number of items**: add the count to the state, `dp[count][t]`.
- **Item `i` at most `k` times**: for yes or no, min or max, split `k` into the parts 1, 2, 4, ... and the rest, and run the 0/1 loop on the parts. A part of `c` copies has the size `c * w`, the value `c * v`, and adds `c` to an item count. For counts, do not split. The parts build some multisets in two ways. Use a sliding window sum over `t - j * w` for `j = 0..k`.
- **Huge values but** $n \leq 40$: meet in the middle. List the sums of each half and match them with a hash map or with sorting.
- **A lower bound on one total** (a profit of at least P with at most G members): use one dimension per limit, and cap the lower-bounded total at P (Profitable Schemes, LeetCode 879). The next variation shows the code.

59.4.10 At least the total: cap the index

Some questions ask for a total of *at least* P, for example a profit of at least P. An exact table needs one cell for each sum up to the sum of all values. In Profitable Schemes, that is 10^4 profit cells for each of 101 member counts and 100 items: about 10^8 steps. But all totals of P or more give the same result: they pass. So merge them into one cell. Write to `dp[min(P, t + w)]`, so the last cell means "P or more". The table stays $P + 1$ cells wide. Memory hook: *at least P: cap the index at P; the last cell means P or more.*

```python
from itertools import combinations

MOD = 10**9 + 7

def count_at_least(items, P):
    # dp[t] = subsets with the sum t; dp[P] = subsets with the sum P or more
    dp = [1] + [0] * P
    for w in items:
        for t in range(P, -1, -1):          # down, and every t is a source
```

```
            dp[min(P, t + w)] += dp[t]       # take w: the total t + w, capped at P
    return dp[P]

def brute(items, P):
    return sum(sum(c) >= P for r in range(len(items) + 1)
               for c in combinations(items, r))

count_at_least([2, 3, 5], 5)                 # -> 5
tests = [([2, 3, 5], 5), ([0, 1, 4], 3), ([6], 2), ([], 0), ([1, 1, 2, 7], 4)]
all(count_at_least(a, P) == brute(a, P) for a, P in tests)   # -> True

def profitable_schemes(n, min_profit, group, profit):
    P = min_profit
    # dp[m][p] = item sets with m members and the profit p (p == P means P or more)
    dp = [[0] * (P + 1) for _ in range(n + 1)]
    dp[0][0] = 1
    for g, pr in zip(group, profit):
        for m in range(n - g, -1, -1):       # members: an upper limit, exact index
            for p in range(P, -1, -1):       # profit: a lower bound, capped index
                q = min(P, p + pr)
                dp[m + g][q] = (dp[m + g][q] + dp[m][p]) % MOD
    return sum(row[P] for row in dp) % MOD

profitable_schemes(5, 3, [2, 2], [2, 3])           # -> 2
profitable_schemes(10, 5, [2, 3, 5], [6, 7, 8])    # -> 7
```

The step pushes from the cell `t` to a later cell. So the loop visits every `t` from `P` down to 0, not only `t >= w`. Each write goes to an index of `t` or more. The loop goes down, so that cell is already read in this pass, and each item counts one time only. At `t = P` the step doubles `dp[P]`: with or without the item, the total stays P or more. General rule: an upper limit drops the cells above it, and a lower bound merges the cells above it into one.

59.5 Adapt it to a new problem

Ask these questions in order.

1. Do the items get signs, or go into two groups? Write the complement as the total minus the quantity. Then solve one subset sum, for example with the sum `(total + target) // 2`.
2. What is the total? A sum, a capacity, a length, a string position or a day. Is $n \cdot T$ at most a few million? If not, and $n \leq 20$ (or $n \leq 40$), use a mask (or meet in the middle).
3. Is the total enough state? If $k > 2$ groups share the items, it is not. With $n \leq 20$, use the mask of used items.
4. Write the recurrence. Fix the last item (or the last piece of a string): `dp[t]` comes from `dp[t - w]`. Check the step on a tiny case by hand.
5. Can one item repeat? No: loop the totals down. Yes: loop them up.
6. What does the cell hold? Fewest items (min, start `inf`), number of ways (sum, `dp[0] = 1`), yes or no (or, `dp[0] = True`), or best value (max).
7. For counts: are two orders of the same items one answer or two? One answer: items outside. Two answers: totals outside (this needs unlimited copies).
8. Exactly the total, at most the total, or at least the total? Exactly: only `dp[0]` starts as reached. At most: solve for each exact total, then combine `dp[0..T]` at the end (the best cell, or the sum for counts). For a best value, every cell can also start at 0. At least `P`: cap the total at `P` and write to `dp[min(P, t + w)]`, so the last cell means "`P` or more". Loop `t` down from `P` to 0. The table stays `P + 1` cells wide, also when the sum of all values is large.
9. Can an item be zero or negative? Zero items in a 0/1 count double the ways. Negative items in an unbounded count can give infinitely many sequences. Then add the length to the state.
10. Are the items themselves part of the answer? Store the last item for each cell, or keep the 2-D table.
11. Choose the form. A memoized DFS is fine when the depth stays below about 1000. Otherwise use the bottom-up loop. Test it against a brute force over subsets on small inputs.

59.6 Pitfalls

CAUTION

Loop direction. A 0/1 problem with the totals going up uses one item many times. An unbounded problem with the totals going down uses each item only once. Both bugs give wrong answers and no error.

CAUTION

Loop order for counts. Items outside counts combinations. Totals outside counts ordered sequences. For `[1, 2, 3]` and the target 4, the two orders give 4 and 7.

CAUTION

Base case. A count needs `dp[0] = 1`, a yes or no needs `dp[0] = True`, and a min needs `dp[0] = 0`. With `dp[0] = 0` in a count, every cell stays 0. The amount 0 has one way (the empty choice) and needs zero coins.

CAUTION

Negative indexes. Start the inner loop at `w`, or test `w <= t`, before you read `dp[t - w]`. A negative index reads from the end of the list and gives a silent wrong answer.

- The sentinel for "not reachable" must be larger than any real answer: `inf` or `amount + 1`, not `amount`. Convert it to `-1` (or what the statement asks) at the end.
- Greedy by the largest item fails on sets such as coins 1, 3, 4 and the amount 6. Test a small counterexample before you trust a greedy.
- Target Sum: test `abs(target) > total` and the parity of `total + target` before you size the table. A zero has two signs, so the inner loop must include `s = x` (it then doubles `dp[s]`).
- A memoized DFS over amounts up to 10^4 goes deeper than the Python recursion limit of 1000. Use the bottom-up loop.
- Do not change a set while you iterate over it. Build the new sums in a comprehension first.
- A bitmask memo must have the mask in its key. A key of (index, fill) merges different used sets and gives wrong answers.
- Put the cached helper inside the function. A cache at module level keeps results from an earlier input.
- Turn the word list into a set before the DP. A list lookup scans every word.

59.7 Problems in this family

Problem	What changes	Memory hook
Coin Change (322)	Unbounded, fewest items (min); sentinel for "not reachable".	Pick the last coin: dp[a] = 1 + min of dp[a - coin], with dp[0] = 0.
Coin Change II (518)	Unbounded count of combinations: coins outside, amounts up inside.	Coins outside, amounts going up inside: ways[a] += ways[a - coin].
Combination Sum IV (377)	Unbounded count of ordered sequences: totals outside, items inside.	Ways to make a total: try every last number, add the ways for what is left.
Partition Equal Subset Sum (416)	0/1 yes or no; equal halves become one subset with half the total.	Odd total means no. Otherwise grow the set of reachable sums and ask if half the total is in it.
Target Sum (494)	Signs become a subset with sum (total + target) / 2; 0/1 count.	Plus or minus each number, cache on (index, running sum).
Partition to K Equal Sum Subsets (698)	$k > 2$ groups: the state is the mask of used items, with fill = used sum mod target.	Mask of used numbers, fill is used sum mod target, try every unused number that fits.
Word Break (139)	The total is a string position; words are reusable items; combine with or.	Cut off a first word, ask if the rest can split, and cache by start index.

The transfer drills for this technique are in Section 69.17 (page 424). The answers are in Section A.16 (page 456).

60 Dynamic programming on strings and sequences

60.1 The idea

Many problems compare two sequences. They ask for the longest common part, the cheapest edit, the number of ways that one sequence fits in the other, or a yes or no match. A brute force tries every subsequence or every edit script. That work is exponential.

The work that remains after some choices depends only on how much of each sequence is used. So a pair of positions (i, j), one in each sequence, is a full state. The brute force reaches the same pair by many different choice paths, and it solves that pair again each time. There are only $(m + 1)(n + 1)$ pairs. Solve each pair one time and store the result in a table. Cell `dp[i][j]` holds the answer for the prefixes `a[:i]` and `b[:j]`.

To fill a cell, look only at the last element of each prefix: `a[i - 1]` and `b[j - 1]`. Each move removes one or two of these elements and points to a smaller cell:

- Diagonal, cell $(i - 1, j - 1)$: use both last elements together (a match or a replace).
- Up, cell $(i - 1, j)$: drop the last element of `a`.
- Left, cell $(i, j - 1)$: drop the last element of `b`.

The problems in this family differ mainly in two things: which moves are legal, and how the moves combine. Use `max` for a longest value, `min` for a cost, a sum for a number of ways, and `or` for a yes or no answer.

Worked example. Find the length of the longest common subsequence (LCS) of `"abcde"` and `"ace"`. Row 0 and column 0 stand for an empty prefix, so they hold 0. Cell $(3, 2)$ compares `c` with `c`. They match, so the cell is the diagonal value plus 1, which is 2. Cell $(4, 3)$ compares `d` with `e`. They do not match, so at least one of them is not in the answer. Drop each one in turn: the cell is the larger of up (2) and left (2). The bottom-right cell holds the answer: 3, for the subsequence `"ace"`.

	`""`	`a`	`c`	`e`
`""`	0	0	0	0
`a`	0	1	1	1
`b`	0	1	1	1
`c`	0	1	**2**	2
`d`	0	1	2	**2**
`e`	0	1	2	**3**

A second group of problems works on one string and asks about palindromes. A window of a string is a palindrome when its two end characters are equal and the window just inside them is a palindrome. So each window depends on one smaller window with the same center. Start at a center and grow outward. Stop at the first mismatch, because no larger window on that center can be a palindrome. A string of length n has $2n - 1$ centers: n characters for odd lengths and $n - 1$ gaps for even lengths. This takes $O(n^2)$ time. The brute force checks each of the $O(n^2)$ windows from scratch and takes $O(n^3)$.

Worked example. Count the palindromic substrings of `"aaa"`. The center on index 1 grows to `"a"` and then to `"aaa"`, so it gives 2 palindromes. Each of the two gaps gives `"aa"` one time. Each end character gives `"a"` one time. The total is $2 + 2 + 2 = 6$.

60.2 Recognize it

- Two strings or arrays, and the question is how well they match, align, merge or transform. Typical words: "longest common", "minimum number of operations to convert", "number of ways to form", "can it be formed from", "does the pattern match the entire string".
- The word *subsequence*: you can skip elements, but you must keep their order. A skip is one move and a match is another move.
- Both lengths are up to about 10^3. A table of about 10^6 cells fits the budget.
- Lengths of about 20 with wildcard or repeat symbols: a memoized search on (i, j) is enough.
- At some step two choices both look valid. For example, both sources match the next character, or a star can stop or continue. Greedy fails, so keep every reachable state. Counter-signal: when each step has only one legal choice (Is Subsequence), greedy two pointers are enough.
- The answer type selects the combine step: longest or cheapest (`max`, `min`), how many (sum), is it possible (`or`).

- The output is one number, or true or false: you need only the value, so you can roll the rows. The output is the sequence itself (a common subsequence, an edit script): keep the full table and walk back.
- One string, the words *palindromic substring*, and n up to a few thousand: expand around each center.
- One string, and the words *palindromic subsequence* or "fewest inserts to make a palindrome": use an interval table on `s[i:j + 1]`.
- Many questions of the form "is this range a palindrome", for example cuts into palindromes: build the palindrome table one time.

60.3 Core template

Decide five things before you write the code:

1. **Meaning.** One sentence: `dp[i][j]` is the answer for `a[:i]` and `b[:j]`.
2. **Border.** The values for an empty prefix: row 0 and column 0.
3. **Moves.** What can happen to `a[i - 1]` and `b[j - 1]`: diagonal, up or left.
4. **Combine.** `max`, `min`, a sum, or `or`.
5. **Answer and order.** Usually `dp[m][n]`. Fill the rows top to bottom and each row left to right.

The template is the LCS. It returns the full table, so you can read the cells of the worked example.

```
def lcs_table(a, b):
    m, n = len(a), len(b)
    # dp[i][j] = LCS length of the prefixes a[:i] and b[:j]
    dp = [[0] * (n + 1) for _ in range(m + 1)]   # row 0 and column 0: empty prefix
    for i in range(1, m + 1):
        for j in range(1, n + 1):
            if a[i - 1] == b[j - 1]:             # cell (i, j) compares a[i-1] and b[j-1]
                dp[i][j] = dp[i - 1][j - 1] + 1  # diagonal: pair the two last elements
            else:
                dp[i][j] = max(dp[i - 1][j],     # up: drop a[i-1]
                               dp[i][j - 1])     # left: drop b[j-1]
    return dp

dp = lcs_table("abcde", "ace")
dp[3]                                  # -> [0, 1, 2, 2]
dp[4]                                  # -> [0, 1, 2, 2]
dp[-1][-1]                             # -> 3
lcs_table("abc", "def")[-1][-1]        # -> 0
lcs_table([1, 4, 2], [1, 2, 4])[-1][-1] # -> 2
```

Invariant: when the loop reaches (i, j), the cells up, left and diagonal are final. They hold the answers for smaller prefix pairs. Time and space are $O(mn)$. The table has $m + 1$ rows and $n + 1$ columns. The extra border row and column remove all index checks.

Top-down form. A memoized search fills the same states on demand. Here `dfs(i, j)` answers for the suffixes `a[i:]` and `b[j:]`, and the answer is `dfs(0, 0)`. This form is easier to write when the moves are irregular, for example with pattern symbols.

```
from functools import cache

def lcs_memo(a, b):
    @cache
    def dfs(i, j):                     # LCS length of the suffixes a[i:] and b[j:]
        if i == len(a) or j == len(b):
            return 0
        if a[i] == b[j]:
            return 1 + dfs(i + 1, j + 1)
        return max(dfs(i + 1, j), dfs(i, j + 1))
    return dfs(0, 0)

lcs_memo("abcde", "ace")               # -> 3
lcs_memo("", "abc")                    # -> 0
```

NOTE

`functools.cache` [Python 3.9+] is the short form of `@lru_cache(maxsize=None)`. Use the long form on an older Python. The recursion depth can reach $m + n$. The default limit is 1000, so two strings of length 500 already fail. When $m + n$ can reach about 1000, use the table or call `sys.setrecursionlimit`. In CPython, the memoized form is also often 2 to 3 times slower than the table.

60.3.1 Read the table

Each problem in the family is one row of this table. Write the meaning of a cell first. The border, the moves and the answer follow from that sentence.

Problem	`dp[i][j]` means	Border	Moves	Combine
LCS	LCS length of `a[:i]` and `b[:j]`	all 0	equal: diagonal + 1; else up, left	`max`
Common substring	length of the longest common suffix of `a[:i]` and `b[:j]`	all 0	equal: diagonal + 1; else 0. The answer is the max over all cells	`max`
Edit distance	edits that turn `a[:i]` into `b[:j]`	`dp[i][0] = i`, `dp[0][j] = j`	equal: diagonal; else 1 + (up, left, diagonal)	`min`
Distinct subsequences	ways to form `t[:j]` from `s[:i]`	`dp[i][0] = 1`	up; equal: also diagonal	sum
Interleaving	`s1[:i]` and `s2[:j]` form `s3[:i + j]`	`dp[0][0]` is true	up if `s1[i - 1]` equals `s3[i + j - 1]`; left if `s2[j - 1]` does	`or`
Regular expression (suffix form)	`s[i:]` matches `p[j:]`	`dp[m][n]` is true	token `x*`: skip it, or eat one character and stay; else eat one character in each	`or`
Palindromic subsequence (interval form)	longest palindromic subsequence of `s[i:j + 1]`	`dp[i][i] = 1`	equal ends: inside `dp[i + 1][j - 1] + 2`; else drop the left end or the right end	`max`

In the prefix form, the answer is `dp[m][n]`. In the suffix form, the answer is `dp[0][0]`. In the interval form on one string, the answer is `dp[0][n - 1]`.

60.4 Variations

60.4.1 Maximize the matches: longest common subsequence

Longest Common Subsequence is the core template. Memory hook: *last chars match: go diagonal plus one. Else take the best of up and left.* On a match, take the diagonal, not up or left. Up plus 1 or left plus 1 uses one character two times. The same table answers related questions. The minimum number of deletions that makes two strings equal is $m + n - 2 \cdot \text{LCS}$ (Delete Operation for Two Strings). Non-crossing links between equal numbers of two arrays are an LCS on integers (Uncrossed Lines).

A *substring* is contiguous. For the longest common substring, a cell holds the longest common suffix of the two prefixes. A mismatch resets the cell to 0, and the answer is the largest cell anywhere. Memory hook: *substring breaks on a mismatch: reset to 0 and keep the best cell.*

```python
def longest_common_substring(a, b):
    m, n = len(a), len(b)
    dp = [[0] * (n + 1) for _ in range(m + 1)]   # longest common suffix of a[:i], b[:j]
    best = 0
    for i in range(1, m + 1):
        for j in range(1, n + 1):
            if a[i - 1] == b[j - 1]:
                dp[i][j] = dp[i - 1][j - 1] + 1
                best = max(best, dp[i][j])
            # a mismatch leaves the cell at 0
    return best
```

```
longest_common_substring("abcde", "abfde")   # -> 2
longest_common_substring("abc", "xyz")       # -> 0
```

The LCS of `"abcde"` and `"abfde"` is 4 (`"abde"`), but the longest common substring is only 2.

60.4.2 Minimize a cost: edit distance

Edit Distance asks for the minimum number of inserts, deletes and replaces that turn `a` into `b`. Look at the last characters `a[i - 1]` and `b[j - 1]`. An optimal edit script pairs them (a free match or a replace), deletes `a[i - 1]`, or inserts `b[j - 1]`. Each case leaves a smaller prefix pair. Memory hook: *look at the last chars: same is free diagonal, else 1 plus min of up, left, diagonal.* Up deletes `a[i - 1]`. Left inserts `b[j - 1]`. Diagonal replaces `a[i - 1]`. The border is not 0: an empty `a` needs `j` inserts, and an empty `b` needs `i` deletes.

```
def edit_distance(a, b):
    m, n = len(a), len(b)
    dp = [[0] * (n + 1) for _ in range(m + 1)]   # edits that turn a[:i] into b[:j]
    for i in range(m + 1):
        dp[i][0] = i                             # delete all i characters
    for j in range(n + 1):
        dp[0][j] = j                             # insert all j characters
    for i in range(1, m + 1):
        for j in range(1, n + 1):
            if a[i - 1] == b[j - 1]:
                dp[i][j] = dp[i - 1][j - 1]      # same last character: free
            else:
                dp[i][j] = 1 + min(dp[i - 1][j],      # delete a[i-1]
                                   dp[i][j - 1],      # insert b[j-1]
                                   dp[i - 1][j - 1])  # replace a[i-1]
    return dp[m][n]

edit_distance("horse", "ros")           # -> 3
edit_distance("intention", "execution") # -> 5
edit_distance("", "abc")                # -> 3
edit_distance("abc", "abc")             # -> 0
```

With other costs per operation, the shape stays the same. Put each cost on its move, and use the costs in the border too. The optimal solution keeps only two rows. See the rolling rows variation below.

60.4.3 Count the ways: distinct subsequences

Distinct Subsequences counts the index sets in `s` that spell `t`. Split the ways to form `t[:j]` from `s[:i]` by the fate of the last character `s[i - 1]`: skip it, or use it as `t[j - 1]`. The two groups do not overlap, and together they cover every way. So add them. Memory hook: *ways equal skip plus use:* `dp[i][j] = dp[i-1][j]`, *plus* `dp[i-1][j-1]` *on a match.*

```
def num_distinct(s, t):
    m, n = len(s), len(t)
    dp = [[0] * (n + 1) for _ in range(m + 1)]   # ways to form t[:j] from s[:i]
    for i in range(m + 1):
        dp[i][0] = 1                             # empty t: choose nothing, one way
    for i in range(1, m + 1):
        for j in range(1, n + 1):
            dp[i][j] = dp[i - 1][j]              # skip s[i-1]
            if s[i - 1] == t[j - 1]:
                dp[i][j] += dp[i - 1][j - 1]     # use s[i-1] as t[j-1]
    return dp[m][n]

num_distinct("rabbbit", "rabbit")       # -> 3
num_distinct("babgbag", "bag")          # -> 5
num_distinct("ba", "a")                 # -> 1
num_distinct("a", "aa")                 # -> 0
```

Set `dp[i][0] = 1` for every `i`, not only for `i = 0`. Without it, a character after index 0 cannot start a match: `("ba", "a")` gives 0. Python integers do not overflow. Counts for short prefixes of `t` can be much larger than the final answer. If a problem asks for a modulo, apply it at each addition. The optimal solution keeps one row and updates `j` from right to left (see rolling rows).

60.4.4 Yes or no with a derived index: interleaving string

Interleaving String asks if `s3` uses each character of `s1` and `s2` one time and keeps the order of each string. Build `s3` from left to right. The next position in `s3` is `k = i + j`, so `k` adds nothing to the state. The state is only (i, j). When the next characters of `s1` and `s2` both match `s3[k]`, either choice can be the right one. So greedy fails, and the search must try both. Memory hook: *take from `s1` or `s2` if it matches `s3[i + j]`; cache `(i, j)`.*

```
from functools import cache

def is_interleave(s1, s2, s3):
    if len(s1) + len(s2) != len(s3):
        return False

    @cache
    def can(i, j):                      # can s1[i:] and s2[j:] form s3[i + j:]?
        if i == len(s1) and j == len(s2):
            return True
        k = i + j                       # next character of s3 to match
        if i < len(s1) and s1[i] == s3[k] and can(i + 1, j):
            return True
        if j < len(s2) and s2[j] == s3[k] and can(i, j + 1):
            return True
        return False

    return can(0, 0)

def greedy(s1, s2, s3):                 # WRONG: prefers s1 when both match
    i = j = 0
    for ch in s3:
        if i < len(s1) and s1[i] == ch:
            i += 1
        elif j < len(s2) and s2[j] == ch:
            j += 1
        else:
            return False
    return i == len(s1) and j == len(s2)

is_interleave("aabcc", "dbbca", "aadbbcbcac")   # -> True
is_interleave("aabcc", "dbbca", "aadbbbaccc")   # -> False
is_interleave("ab", "ac", "acab")               # -> True
greedy("ab", "ac", "acab")                      # -> False
is_interleave("", "", "")                       # -> True
```

Check the lengths first. Without the check, a short `s3` raises `IndexError`, and a long `s3` can return true with extra characters left over. General rule: when one index follows from the others, drop it from the state. For k sources, the state is the k-tuple of prefix lengths. The optimal solution is a rolling one-row table with $O(n)$ memory (see rolling rows).

60.4.5 Pattern matching: regular expression

Regular Expression Matching supports `.` (any one character) and `*` (zero or more of the element before it). The pattern must match the entire string. The trick: `*` is a postfix modifier. Look at `p[j + 1]` before you use `p[j]`, and treat `x*` as one token. A token `x*` has only two moves. Skip it (zero copies, `j + 2`). Or, if `x` matches `s[i]`, eat that character and stay on the same token (`i + 1`, same `j`). Staying on the token covers every count of copies without a loop over counts. Memory hook: *peek for a star: skip it, or eat one and stay.*

```python
from functools import cache

def is_match(s, p):
    @cache
    def dfs(i, j):                      # does s[i:] match p[j:]?
        if j == len(p):
            return i == len(s)          # both must end together
        first = i < len(s) and p[j] in (s[i], ".")
        if j + 1 < len(p) and p[j + 1] == "*":
            # skip "x*", or eat one character and stay on "x*"
            return dfs(i, j + 2) or (first and dfs(i + 1, j))
        return first and dfs(i + 1, j + 1)
    return dfs(0, 0)

is_match("aa", "a")                     # -> False
is_match("aa", "a*")                    # -> True
is_match("ab", ".*")                    # -> True
is_match("aab", "c*a*b")                # -> True
is_match("aaa", "a*a")                  # -> True
is_match("mississippi", "mis*is*p*.")   # -> False
```

Each pattern token becomes a small set of moves on (i, j). Use this table to add a new symbol. Here "eat" means: `x` matches `s[i]`, and `i` moves by 1.

Token	Moves from (i, j)
`x` or `.`	eat, and move to the next token: $(i+1, j+1)$
`x*` (zero or more)	skip the token: $(i, j+2)$; or eat and stay: $(i+1, j)$
`x+` (one or more)	eat and stay: $(i+1, j)$; or eat and leave: $(i+1, j+2)$. There is no skip move
`x?` (zero or one)	skip the token: $(i, j+2)$; or eat and leave: $(i+1, j+2)$

The "eat and stay" move is the same idea as unlimited reuse of an item in the unbounded knapsack. The optimal solution fills the same states bottom-up. Each state reads only cells with a larger `i` or `j`, so `i` goes from `m` down to 0 and `j` goes from `n - 1` down to 0. Use it when the inputs are long enough to reach the recursion limit.

```python
def is_match_table(s, p):
    m, n = len(s), len(p)
    dp = [[False] * (n + 1) for _ in range(m + 1)]   # s[i:] matches p[j:]
    dp[m][n] = True                                  # empty string, empty pattern
    for i in range(m, -1, -1):                       # i = m too: "a*b*" matches ""
        for j in range(n - 1, -1, -1):
            first = i < m and p[j] in (s[i], ".")
            if j + 1 < n and p[j + 1] == "*":
                dp[i][j] = dp[i][j + 2] or (first and dp[i + 1][j])
            else:
                dp[i][j] = first and dp[i + 1][j + 1]
    return dp[0][0]

is_match_table("aab", "c*a*b")          # -> True
is_match_table("ab", ".*c")             # -> False
```

60.4.6 Rolling rows: keep O(n) memory

Row `i` reads only row `i - 1` and row `i`. So keep two rows, or one row, instead of the full table. The rule for the direction comes from the neighbors that a cell reads. Memory hook: *up and diagonal are old, left is new. Reads up and diagonal: scan right to left. Reads up and left: scan left to right. Reads all three: keep two rows.*

A cell reads	Keep	Examples
up, left, diagonal	two rows `prev` and `cur`; or one row plus the old diagonal value in a variable	LCS, Edit Distance
up and diagonal	one row; `j` from right to left, so `dp[j - 1]` still holds the old row	Distinct Subsequences, 0/1 knapsack
up and left	one row; `j` from left to right. Before the write, `dp[j]` is up and `dp[j - 1]` is left	Interleaving String

```
def edit_distance_2rows(a, b):
    n = len(b)
    prev = list(range(n + 1))           # row 0: j inserts
    for i in range(1, len(a) + 1):
        cur = [i] + [0] * n             # a NEW list for each row; cur[0] = i deletes
        for j in range(1, n + 1):
            if a[i - 1] == b[j - 1]:
                cur[j] = prev[j - 1]
            else:
                cur[j] = 1 + min(prev[j], cur[j - 1], prev[j - 1])
        prev = cur
    return prev[n]

def num_distinct_1row(s, t):
    n = len(t)
    dp = [1] + [0] * n                  # dp[j] = ways to form t[:j]
    for ch in s:
        for j in range(n, 0, -1):       # right to left: dp[j - 1] is still old
            if ch == t[j - 1]:
                dp[j] += dp[j - 1]
    return dp[n]

def num_distinct_wrong(s, t):           # WRONG: left to right
    n = len(t)
    dp = [1] + [0] * n
    for ch in s:
        for j in range(1, n + 1):
            if ch == t[j - 1]:
                dp[j] += dp[j - 1]      # reads a value of the NEW row
    return dp[n]

edit_distance_2rows("horse", "ros")     # -> 3
num_distinct_1row("babgbag", "bag")     # -> 5
num_distinct_1row("a", "aa")            # -> 0
num_distinct_wrong("a", "aa")           # -> 1
```

In the wrong version, one character of `s` matches two characters of `t`.

```
def lcs_1row(a, b):
    dp = [0] * (len(b) + 1)
    for x in a:
        diag = 0                        # old dp[j - 1]: the upper-left cell
        for j in range(1, len(b) + 1):
            up = dp[j]                  # save the old value before the write
            dp[j] = diag + 1 if x == b[j - 1] else max(up, dp[j - 1])
            diag = up
    return dp[-1]

def is_interleave_1row(s1, s2, s3):
    m, n = len(s1), len(s2)
    if m + n != len(s3):
        return False
    dp = [False] * (n + 1)              # dp[j]: s1[:i] and s2[:j] form s3[:i + j]
```

```
    for i in range(m + 1):
        for j in range(n + 1):
            if i == 0 and j == 0:
                dp[j] = True
                continue
            top = i > 0 and dp[j] and s1[i - 1] == s3[i + j - 1]
            left = j > 0 and dp[j - 1] and s2[j - 1] == s3[i + j - 1]
            dp[j] = top or left
    return dp[n]

lcs_1row("abcde", "ace")                            # -> 3
is_interleave_1row("aabcc", "dbbca", "aadbbcbcac")  # -> True
is_interleave_1row("ab", "ac", "acab")              # -> True
is_interleave_1row("a", "", "b")                    # -> False
```

When the two inputs can swap roles (LCS, Edit Distance, Interleaving String), make the shorter one the column sequence. Then the memory is $O(\min(m, n))$.

60.4.7 Rebuild the answer: walk back through the table

A rolling table gives only the value. To get the sequence itself, keep the full table. Start at (m, n). At each cell, find the move that produced its value, and step to that neighbor. Collect an element on each match move, and reverse the list at the end. The same walk recovers an edit script or an alignment. Memory hook: *walk back from the corner along the move that made each value.*

```
def lcs_string(a, b):
    m, n = len(a), len(b)
    dp = [[0] * (n + 1) for _ in range(m + 1)]
    for i in range(1, m + 1):
        for j in range(1, n + 1):
            if a[i - 1] == b[j - 1]:
                dp[i][j] = dp[i - 1][j - 1] + 1
            else:
                dp[i][j] = max(dp[i - 1][j], dp[i][j - 1])
    out = []
    i, j = m, n
    while i > 0 and j > 0:
        if a[i - 1] == b[j - 1]:        # this cell came from the diagonal
            out.append(a[i - 1])
            i, j = i - 1, j - 1
        elif dp[i - 1][j] >= dp[i][j - 1]:
            i -= 1                      # the cell came from up
        else:
            j -= 1                      # the cell came from the left
    return "".join(reversed(out))

lcs_string("abcde", "ace")              # -> 'ace'
lcs_string("abcbdab", "bdcaba")         # -> 'bcba'
lcs_string("abc", "xyz")                # -> ''
```

60.4.8 Palindromic substrings: expand around each center

Longest Palindromic Substring returns the longest palindromic window. Memory hook: *every palindrome grows from a center: try each character and each gap, expand while the ends match, keep the longest.* Palindromic Substrings counts the palindromic windows by position, so `"aaa"` gives 6, not 3. Memory hook: *every palindrome has a middle: expand from each middle, odd and even, and count each match.*

Both problems use one helper. It moves `left` and `right` outward while the two ends match. The loop stops one step too far on each side. So the palindrome is `s[left + 1:right]`, and its length is `right - left - 1`. Even-length palindromes have no middle character. Call the helper on the gap `(c, c + 1)` too.

```python
def longest_palindrome(s):
    def expand(left, right):
        while left >= 0 and right < len(s) and s[left] == s[right]:
            left -= 1
            right += 1
        return s[left + 1:right]        # the loop stops one step too far
    best = ""
    for c in range(len(s)):
        best = max(best, expand(c, c), expand(c, c + 1), key=len)
    return best

def count_palindromes(s):
    n = len(s)
    def expand(left, right):
        count = 0
        while left >= 0 and right < n and s[left] == s[right]:
            count += 1                  # s[left:right + 1] is one more palindrome
            left -= 1
            right += 1
        return count
    return sum(expand(i, i) + expand(i, i + 1) for i in range(n))

longest_palindrome("babad")             # -> 'bab'
longest_palindrome("cbbd")              # -> 'bb'
longest_palindrome("a")                 # -> 'a'
count_palindromes("abc")                # -> 3
count_palindromes("aaa")                # -> 6
```

The optimal versions use the same algorithm. For the longest window, store only the start and the length, and slice one time at the end. For the count, one loop over the $2n - 1$ centers replaces the two calls. Center `c` gives `left = c // 2` and `right = left + c % 2`. Even `c` is a character and odd `c` is a gap.

```python
def count_palindromes_one_loop(s):
    n = len(s)
    count = 0
    for c in range(2 * n - 1):
        left = c // 2
        right = left + c % 2            # even c: a character; odd c: a gap
        while left >= 0 and right < n and s[left] == s[right]:
            count += 1
            left -= 1
            right += 1
    return count

count_palindromes_one_loop("aaa")       # -> 6
count_palindromes_one_loop("abba")      # -> 6
```

General rule: when a structure has two shapes (one middle item or two middle items), cover both shapes. Seed each shape explicitly, as the two calls do. Or encode both shapes in one index range, as the $2n - 1$ centers do. Then one loop body handles both cases. Or transform the input so that only one shape remains. For n of 10^5 or more, $O(n^2)$ is too slow. Then use Manacher's algorithm, which runs in $O(n)$. It first puts a separator between all characters and at both ends, so every palindrome has a middle character.

60.4.9 Palindromes by DP: the palindrome table

The same recurrence fills a table. `pal[i][j]` is true when `s[i:j + 1]` is a palindrome: the ends are equal, and the inside is a palindrome or has fewer than two characters. A cell reads `pal[i + 1][j - 1]`, so fill `i` from right to left, or fill by increasing length. The table costs $O(n^2)$ memory. Build it when many range questions follow, for example Palindrome Partitioning, which tries each cut and needs each test in $O(1)$. The number of true cells is the answer of Palindromic Substrings. Memory hook: *equal ends and a palindrome inside; fill `i` from right to left.*

```
def palindrome_table(s):
    n = len(s)
    pal = [[False] * n for _ in range(n)]
    for i in range(n - 1, -1, -1):      # right to left: row i + 1 is ready
        for j in range(i, n):
            pal[i][j] = s[i] == s[j] and (j - i < 2 or pal[i + 1][j - 1])
    return pal

pal = palindrome_table("aab")
[(i, j) for i in range(3) for j in range(i, 3) if pal[i][j]]
# -> [(0, 0), (0, 1), (1, 1), (2, 2)]
sum(map(sum, palindrome_table("aaa")))  # -> 6
```

When a palindrome can skip characters (a palindromic *subsequence*), a window has more than one move, as in the LCS. Then the walk along one center does not work. Use an interval table: `dp[i][j]` is the answer for `s[i:j + 1]`. Equal ends go inside, to `dp[i + 1][j - 1]`. Else drop the left end or the right end. Fill `i` from right to left, as in the palindrome table. The answer is `dp[0][n - 1]`. The fewest inserts (or deletes) that make `s` a palindrome is `n` minus the longest palindromic subsequence. That length is also the LCS of `s` and `s[::-1]`, so the core template solves it too (Minimum Insertion Steps to Make a String Palindrome, LeetCode 1312). Memory hook: *equal ends are free; else copy one end to the other side and pay 1.*

```
def min_insertions(s):                  # fewest inserts that make s a palindrome
    n = len(s)
    dp = [[0] * n for _ in range(n)]    # dp[i][j] = inserts for s[i:j + 1]
    for i in range(n - 2, -1, -1):      # right to left: row i + 1 is ready
        for j in range(i + 1, n):
            if s[i] == s[j]:
                dp[i][j] = dp[i + 1][j - 1]                     # equal ends: free
            else:
                dp[i][j] = 1 + min(dp[i + 1][j], dp[i][j - 1])  # copy one end, pay 1
    return dp[0][n - 1] if n else 0

min_insertions("mbadm")                 # -> 2
min_insertions("leetcode")              # -> 5
min_insertions("zzazz")                 # -> 0
min_insertions("")                      # -> 0
```

60.5 Adapt it to a new problem

Ask these questions in order.

1. Which sequences does the process read in order? Give each one an index. If one index follows from the others (`k = i + j`), drop it from the state.
2. Is the answer about a subsequence (skips allowed) or a substring (contiguous)? A substring of two sequences uses the "common suffix" cell. A palindromic substring uses centers. A palindromic subsequence uses an interval table.
3. Write the meaning of `dp[i][j]` in one sentence. Choose prefixes (`a[:i]`) for a table, or suffixes (`a[i:]`) for a memoized search.
4. List what can happen to the last element of each prefix (the first element of each suffix): match both, skip one, skip the other, or a special move such as "eat and stay". Each move is one neighbor cell. For a pattern, turn each token into its moves with the token table.
5. Select the combine step from the question: `max`, `min`, sum, or `or`. For a count, check that the cases do not overlap.
6. Fill the border. Ask: what is the answer for two empty prefixes, and for one empty prefix?
7. Select the order. Each cell must come after the cells that it reads. The prefix form goes up in `i` and `j`. The suffix form goes down.
8. Find the answer cell: `dp[m][n]`, `dp[0][0]`, or the best cell anywhere.
9. Check the size. In Python, about 10^6 cells take about 0.3 s. If memory is tight, roll the rows. If the problem asks for the sequence itself, keep the full table and walk back.

10. Test on two empty inputs, on one empty input, on equal inputs, and on a case where greedy fails.

60.6 Pitfalls

CAUTION

Off-by-one between the table and the strings. The table has size $(m+1) \times (n+1)$. Cell `dp[i][j]` compares `a[i - 1]` with `b[j - 1]`, not `a[i]` with `b[j]`. Build each row as a new list: `[[0] * (n + 1)] * (m + 1)` repeats one row.

CAUTION

A wrong border. Edit distance needs `dp[i][0] = i` and `dp[0][j] = j`, not 0. Distinct subsequences needs `dp[i][0] = 1` for every `i`. In interleaving, only `dp[0][0]` is true without a character match.

CAUTION

A wrong match move. LCS: diagonal plus 1, not up plus 1. Edit distance: a match is free, so do not add 1. Distinct subsequences: on a match, add both cases (skip and use), not only the diagonal.

CAUTION

Greedy instead of a search. When two choices both match the next character, a greedy choice can be wrong. Interleaving `("ab", "ac", "acab")` and the pattern `"a*a"` on `"aaa"` are small counterexamples.

CAUTION

Regular expression details. Test for `*` at `p[j + 1]`, not at `p[j]`. Remember that `x*` can match zero characters, so compute the row for an empty rest of the string. A full match needs both indexes at the end together. Each copy of `.` in `.*` matches any character, so `.*` matches any string.

CAUTION

Rolling rows. A wrong direction reads a value of the new row. `prev = cur` with one shared list makes both names point to the same row, so build a new `cur` for each row. In edit distance, set `cur[0] = i` at the start of each row.

CAUTION

Palindrome expansion. Do not forget the even centers: `"cbbd"` then gives `"c"`, not `"bb"`. Test `left >= 0` and `right < n` before you compare characters. Python accepts `s[-1]`, so a missing left bound compares the wrong character and gives no error. Count palindromes by position, not by distinct text.

CAUTION

Recursion and speed. A memoized search on two strings of length 1000 can go about 2000 levels deep. Use the bottom-up table, or raise the recursion limit. Keep the inner loop simple: no function calls and no slicing per cell. A plain recursion without a cache is exponential. In the pattern case, a string of 18 `a` with nine `a*` tokens and a final `b` makes about 11.6 million calls.

60.7 Problems in this family

Problem	What changes	Memory hook
Longest Common Subsequence	The core table: match goes diagonal + 1, mismatch takes `max` of up and left.	Last chars match: go diagonal plus one. Else take the best of up and left.
Edit Distance	Minimize a cost. The border is `i` and `j`. A mismatch costs 1 plus the `min` of three moves.	Look at the last chars: same is free diagonal, else 1 plus min of up, left, diagonal.
Distinct Subsequences	Count the ways: always skip, and also use on a match. The border column is 1.	Ways equal skip plus use: `dp[i][j] = dp[i-1][j]`, plus `dp[i-1][j-1]` on a match.
Interleaving String	Yes or no. The third index is `i + j`. Greedy fails, so search both sources.	Take from `s1` or `s2` if it matches `s3[i + j]`; cache `(i, j)`.
Regular Expression Matching	Pattern tokens become moves. `x*` is skip or "eat and stay". Suffix form.	Peek for a star: skip it, or eat one and stay.
Longest Palindromic Substring	One string. Expand the $2n - 1$ centers and keep the longest window.	Every palindrome grows from a center: try each character and each gap, expand while the ends match, keep the longest.
Palindromic Substrings	Same centers. Add 1 for each matching pair instead of tracking the max.	Every palindrome has a middle: expand from each middle, odd and even, and count each match.

The transfer drills for this technique are in Section 69.18 (page 424). The answers are in Section A.17 (page 458).

61 Dynamic programming on grids, intervals, games and states

61.1 The idea

A brute force for these problems lists a set of histories that grows exponentially. The histories are the paths through a grid, the orders of removal, the lines of play in a game, or the sequences of buy and sell actions. Many different histories reach the same situation, and the brute force solves the rest of the problem again each time. Dynamic programming gives each situation a name, the *state*. It solves each state one time and reuses the answer. The work drops from the number of histories to the number of states times the number of choices per state.

In 1-D DP the state is a prefix index. In this section the state has other shapes.

Shape	State	The choice that splits a state	Fill order
Grid path	a cell (r, c)	the last move into the cell	row by row
Path on an implicit DAG	a cell or a node	the first step out of the node	by a strict key, or a memoized DFS
Interval	an open range (l, r)	the element that acts last (or the first cut)	by range length
Two-player game	the remaining range, seen by the player to move	the move of the player to move	by range length
State machine	(step, status)	the action at this step	step by step

A worked example. Count the paths from the top-left corner to the bottom-right corner of a 3×3 grid. Each move goes one cell down or one cell right. A brute force lists the paths one by one. The DP asks a smaller question for each cell: how many paths end here? The last move into a cell comes from above or from the left. These two groups of paths do not overlap. So the count of a cell is the count above plus the count to the left. Each cell in row 0 and in column 0 has exactly one path, a straight line. Row 1 becomes $1, 2, 3$. Row 2 becomes $1, 3, 6$. The answer is 6, and each cell costs one addition. On a 100×100 grid the brute force lists about $2 \cdot 10^{58}$ paths. The DP does 10^4 additions.

Every DP in this section comes from the same four decisions:

1. **State.** What is the minimum information about the past that the rest of the problem needs?
2. **Split.** Which single choice breaks a state into smaller states that do not interact?
3. **Base.** Which states are so small that their answer is known?
4. **Order.** In which order is every smaller state ready before a larger state reads it?

61.2 Recognize it

- The question asks for a count, a minimum, a maximum or a winner. It does not ask for the list of paths or orders. The output is one number or one boolean for the full input.
- Moves on a grid go only forward, for example down and right. Each cell depends only on cells before it.
- A rule must hold at every step of a path, for example a running total that stays above 0. The rule depends on the steps that come later, so fill the table backward from the target.
- Moves go in all directions, but each move strictly increases a key (a cell value, a length). The strict key removes all cycles, so the move graph is a DAG.
- Elements disappear or merge one at a time, and the score of a step depends on the neighbors at that moment. Or a cut splits a piece in two. The statement often says “in any order” or “in an order that you choose”. Think of a DP over ranges with a split point.
- n is a few hundred. Then $O(n^3)$ over all ranges with a split point fits.
- Two players alternate, both play optimally, and each move takes from an end of a row. The rest of the row is always one contiguous range (i, j). $n \leq 20$ allows a memoized recursion. $n \leq 1000$ needs a bottom-up table of $O(n^2)$ ranges.
- Rules connect the actions over time: a cooldown, a fee per sale, at most one item held, at most k uses. The past matters only through a small status.
- The recursion can go n levels deep, and n is 1000 or more. Plan a bottom-up loop from the start.

61.3 Core template

Write the recurrence first. Then choose one of two forms. The top-down form is a recursive function with `@cache`. The cache finds a valid order by itself. The bottom-up form fills a table in an order where every state that it reads is already final. Both forms do the same work. The grid count is the smallest complete example.

```
from functools import cache

def count_paths(m, n):                      # bottom-up table
    ways = [[1] * n for _ in range(m)]      # row 0 and column 0: one straight path
    for r in range(1, m):                   # row by row: above and left are ready
        for c in range(1, n):
            ways[r][c] = ways[r - 1][c] + ways[r][c - 1]
    return ways[m - 1][n - 1]

def count_paths_memo(m, n):                 # top-down: the same recurrence
    @cache
    def ways(r, c):
        if r == 0 or c == 0:
            return 1
        return ways(r - 1, c) + ways(r, c - 1)
    return ways(m - 1, n - 1)

count_paths(3, 3), count_paths_memo(3, 3)   # -> (6, 6)
count_paths(3, 7), count_paths(3, 2)        # -> (28, 3)
count_paths(1, 1)                           # -> 1
```

Invariant. When the loop reaches a state, every state that it reads holds its final value. The fill order makes this true: row by row for a grid, decreasing key for a DAG, increasing length for a range, and step by step for a state machine.

The same skeleton holds for every shape in this section:

```
dp = a table with one slot per state       # 1. state: dp[s] = answer for subproblem s
dp[smallest states] = known values         # 2. base (often sentinels)
for s in an order where the parts of s come first:          # 3. order
    dp[s] = combine(value of choice x for each choice x)   # 4. transition
answer = dp[full problem]
```

The combine step depends on the question.

Question	Combine
number of ways	sum over the choices. The choices must split the paths into groups that do not overlap.
cheapest or best	min or max over the choices, plus the cost or score of the step
longest path	1 plus the max over the next nodes
score in a two-player game	max over the moves of (gain now minus the value of the next state)
win or lose, no score	a state wins if some move reaches a state where the opponent loses

NOTE

Use the top-down form when the order is not clear, or when most states are never reached. Use the bottom-up form when the recursion can go deeper than about 1000 calls (the default limit of CPython). Also use it when speed matters: in CPython the top-down form of the same DP is often 2 to 3 times slower.

61.4 Variations

61.4.1 Grid paths: count, cost and walls

Unique Paths counts the paths of a robot that moves down or right on an $m \times n$ grid ($m, n \leq 100$). The core template is the intuitive solution. Memory hook: *fill the grid with ones, then each cell equals up plus left.*

Two changes give most other grid path problems. A cost per cell changes the sum into a minimum plus the cell cost (Minimum Path Sum). A wall gets the count 0. Then row 0 and column 0 are not all ones, so start with zeros, seed only the start cell, and guard the indexes (Unique Paths II).

```
def min_path_sum(grid):
    rows, cols = len(grid), len(grid[0])
    INF = float("inf")
    best = [[INF] * cols for _ in range(rows)]
    for r in range(rows):
        for c in range(cols):
            if r == 0 and c == 0:
                best[r][c] = grid[0][0]
                continue
            up = best[r - 1][c] if r > 0 else INF
            left = best[r][c - 1] if c > 0 else INF
            best[r][c] = min(up, left) + grid[r][c]
    return best[rows - 1][cols - 1]

def paths_with_walls(grid):                 # 1 marks a wall
    rows, cols = len(grid), len(grid[0])
    ways = [[0] * cols for _ in range(rows)]
    for r in range(rows):
        for c in range(cols):
            if grid[r][c] == 1:
                continue                    # a wall keeps 0
            if r == 0 and c == 0:
                ways[r][c] = 1
                continue
            up = ways[r - 1][c] if r > 0 else 0
            left = ways[r][c - 1] if c > 0 else 0
            ways[r][c] = up + left
```

```
    return ways[rows - 1][cols - 1]

min_path_sum([[1, 3, 1], [1, 5, 1], [4, 2, 1]])        # -> 7
paths_with_walls([[0, 0, 0], [0, 1, 0], [0, 0, 0]])    # -> 2
paths_with_walls([[0, 1], [1, 0]])                     # -> 0
```

Each row reads only the row above, so one row is enough. Update `row[j] += row[j - 1]` from left to right. The old `row[j]` is the cell above, and the new `row[j - 1]` is the cell to the left. This cuts the memory from $O(mn)$ to $O(n)$. With no walls and no costs, the count also has a closed form. A path is a sequence of $m-1$ downs and $n-1$ rights, so the answer is $\binom{m+n-2}{m-1}$.

```
import math

def count_paths_row(m, n):
    row = [1] * n
    for _ in range(1, m):
        for j in range(1, n):
            row[j] += row[j - 1]            # old row[j] is above, new row[j - 1] is left
    return row[-1]

count_paths_row(3, 7), math.comb(3 + 7 - 2, 3 - 1)    # -> (28, 28)
count_paths_row(100, 100) == math.comb(198, 99)      # -> True
```

61.4.2 Longest path on an implicit DAG: sorted order or memoized DFS

Longest Increasing Path in a Matrix: from a cell, move up, down, left or right to a strictly larger value. Return the number of cells on the longest such path ($m, n \leq 200$). In a general graph the longest simple path is NP-hard, and no fast algorithm for it is known. The brute force tries every path from every cell, and the number of paths grows exponentially. The strict increase removes the difficulty. A path can never come back to a cell, so the move graph is a DAG. The best path from a cell then does not depend on how the path reached the cell. So keep one number per cell, and use no visited set.

The intuitive solution visits the cells from the largest value to the smallest. When it reaches a cell, every strictly larger neighbor already holds its final answer. In `[[9, 9, 4], [6, 6, 8], [2, 1, 1]]` the 9s get 1, the 6s get 2, the 2 gets 3, and the 1 next to the 2 gets 4. That is the path 1, 2, 6, 9. Memory hook: *biggest first: each cell is 1 plus its best bigger neighbor.*

```
def longest_increasing_path(matrix):
    rows, cols = len(matrix), len(matrix[0])
    best = [[1] * cols for _ in range(rows)]    # one cell alone is a path of length 1
    cells = sorted(((matrix[r][c], r, c) for r in range(rows) for c in range(cols)),
                   reverse=True)                # largest value first
    for value, r, c in cells:
        for nr, nc in ((r + 1, c), (r - 1, c), (r, c + 1), (r, c - 1)):
            if 0 <= nr < rows and 0 <= nc < cols and matrix[nr][nc] > value:
                best[r][c] = max(best[r][c], 1 + best[nr][nc])
    return max(max(row) for row in best)

longest_increasing_path([[9, 9, 4], [6, 6, 8], [2, 1, 1]])   # -> 4
longest_increasing_path([[3, 4, 5], [3, 2, 6], [2, 2, 1]])   # -> 4
longest_increasing_path([[1]])                               # -> 1
```

The sort costs $O(mn \log(mn))$ time. A memoized DFS takes $O(mn)$ time and needs no sort. It is the general template for a DAG with no simple fill order: the recursion finds the order. But a snake-shaped grid gives one path through all mn cells, so the recursion can go 40000 calls deep. Raise the recursion limit, or use the sorted order. Kahn's topological sort also gives an order with no recursion.

```
import sys
from functools import cache

def longest_increasing_path_memo(matrix):
    rows, cols = len(matrix), len(matrix[0])
```

```
    sys.setrecursionlimit(max(sys.getrecursionlimit(), rows * cols + 1000))
    @cache
    def longest_from(r, c):                 # longest path that starts at (r, c)
        best = 0
        for nr, nc in ((r + 1, c), (r - 1, c), (r, c + 1), (r, c - 1)):
            if 0 <= nr < rows and 0 <= nc < cols and matrix[nr][nc] > matrix[r][c]:
                best = max(best, longest_from(nr, nc))
        return best + 1
    return max(longest_from(r, c) for r in range(rows) for c in range(cols))

longest_increasing_path_memo([[9, 9, 4], [6, 6, 8], [2, 1, 1]])   # -> 4
```

The same frame solves other DAGs with a strict key: longest increasing subsequence (an edge goes to a later, larger element) and word chains (an edge goes to a longer word). To count paths instead, replace the max with a sum.

61.4.3 Interval DP: choose the element that acts last

Burst Balloons: burst all balloons in a row. Bursting balloon i earns the product of its value and the values of its two neighbors that are still present. A missing neighbor counts as 1. Maximize the total ($n \leq 300$).

A split on the balloon that bursts *first* does not work. Its two neighbors become adjacent, so the left part and the right part still interact. Reverse the time. Choose the balloon k that bursts *last* in the open range (l, r). Until the end, k stays alive and acts as a wall, so the balloons in (l, k) and in (k, r) never touch. When k bursts, its only neighbors are the walls l and r, so it earns a known amount. Pad the row with a 1 at each end, so that every range has two walls. Memory hook: *pick the balloon that pops last. It sees only the two walls.*

```
def max_coins(nums):
    vals = [1] + nums + [1]                 # walls with the value 1 at both ends
    n = len(vals)
    dp = [[0] * n for _ in range(n)]        # dp[l][r]: best for balloons strictly between l and r
    for length in range(2, n):              # short ranges first
        for l in range(n - length):
            r = l + length
            best = 0
            for k in range(l + 1, r):       # k bursts LAST, so it sees only the walls
                coins = dp[l][k] + vals[l] * vals[k] * vals[r] + dp[k][r]
                if coins > best:
                    best = coins
            dp[l][r] = best
    return dp[0][n - 1]

max_coins([3, 1, 5, 8]), max_coins([1, 5])     # -> (167, 10)
max_coins([7])                                 # -> 7
```

In `[3, 1, 5, 8]` one best plan bursts 8 last, between the walls 1 and 1, for $1 \cdot 8 \cdot 1 = 8$ coins. The range $(3, 1, 5)$ to its left is then a smaller problem with the walls 1 and 8. There are $O(n^2)$ ranges and $O(n)$ split points, so the time is $O(n^3)$. At $n = 300$ this is about 4.5 million inner steps, about 1 second in CPython. A top-down version with `@cache` is 2 to 3 times slower in CPython and risks a timeout. In the hot loop, a plain `if` is faster than `max()` over a generator.

The general rule: the split is the action that leaves two sides that cannot affect each other. A removal or a merge joins two neighbors, so the *last* action in a range is the clean split (matrix chain multiplication, polygon triangulation). A cut separates two pieces, so the *first* cut in a range is the clean split. Add sentinels so that the ends of the row follow the same formula as the inside. The reverse-time idea also works for deletions outside interval DP. Replay the deletions backward as insertions, for example with union-find when the edges of a graph are removed one by one.

61.4.4 Two-player games: store the score difference

Predict the Winner: two players take turns. Each turn takes one number from either end of the array and adds it to the score of the player. Both play optimally. Return true if player 1 wins or ties ($n \leq 20$).

The brute force plays out every line of play, about 2^n of them. But after any sequence of moves, the rest of the

array is one range `nums[i..j]` (both ends included). So only $O(n^2)$ different positions exist. Do not store two scores and a turn flag. The scores so far make too many states, and the cache stops helping. Store one number, the *gap*: the best (mover minus opponent) score difference that the player to move (the mover) can force on the range. If the mover takes `nums[i]`, the opponent becomes the mover on $(i+1, j)$ and gets their own best gap. So the mover gets `nums[i] - gap(i + 1, j)`. One subtraction replaces a separate min layer for the opponent. This form has the name negamax. Memory hook: *my pick minus your best gap, max of both ends. A tie wins.*

```
from functools import cache

def predict_the_winner(nums):
    @cache
    def best_gap(i, j):                     # best (mover - opponent) on nums[i..j]
        if i > j:
            return 0
        take_left = nums[i] - best_gap(i + 1, j)
        take_right = nums[j] - best_gap(i, j - 1)
        return max(take_left, take_right)
    return best_gap(0, len(nums) - 1) >= 0  # a tie is a win for player 1

predict_the_winner([1, 5, 2])         # -> False
predict_the_winner([1, 5, 233, 7])    # -> True
predict_the_winner([0])               # -> True
```

In `[1, 5, 233, 7]` the greedy pick 7 gives 233 to player 2. The DP takes 1 first and then gets 233.

Score of the first player, not the winner: the same game on cards with positive values and an even count ($n \le 1000$). Return the final score of player 1, not the winner. The intuitive solution stores the actual score. The total of a range is fixed, so the mover gets the range total minus what the opponent gets on the range that the mover leaves. The mover leaves the range where the opponent gets less. The recursion would go n levels deep, so fill a table by range length, with prefix sums for range totals. Memory hook: *my score is the range total minus the opponent's best on what I leave.*

```
def first_player_score(cards):
    n = len(cards)
    prefix = [0]
    for card in cards:
        prefix.append(prefix[-1] + card)
    best = [[0] * n for _ in range(n)]      # best[i][j]: most the mover collects from cards[i..j]
    for i in range(n):
        best[i][i] = cards[i]               # one card left: the mover takes it
    for length in range(2, n + 1):          # shorter ranges first
        for i in range(n - length + 1):
            j = i + length - 1
            range_sum = prefix[j + 1] - prefix[i]
            best[i][j] = range_sum - min(best[i + 1][j], best[i][j - 1])
    return best[0][n - 1]

first_player_score([2, 10, 3, 1])         # -> 11
first_player_score([5, 1, 7, 9, 2, 3])    # -> 15
```

In `[2, 10, 3, 1]` the greedy pick 2 leaves 10 at the left end, and player 1 ends with 5. The DP takes 1, then gets 10, for 11.

The optimal form keeps the gap in one array of length n and needs no prefix sums. For each right end j, it moves i down from $j-1$. Then `gap[i + 1]` already holds the range that ends at j, and `gap[i]` still holds the range that ends at $j-1$. Convert the gap back to a score with $p_1 + p_2 =$ total and $p_1 - p_2 =$ gap, so $p_1 = (\text{total} + \text{gap})/2$. Use this identity whenever two values have a known sum and a known difference.

```
def first_player_score_1d(cards):
    n = len(cards)
    gap = [0] * n                           # gap[i]: best gap on cards[i..j] for the current j
    for j in range(n):
        gap[j] = cards[j]
```

```
        for i in range(j - 1, -1, -1):      # i goes DOWN, so gap[i + 1] is already for this j
            gap[i] = max(cards[i] - gap[i + 1], cards[j] - gap[i])
    return (sum(cards) + gap[0]) // 2       # the sum is 2 * p1, so the division is exact

first_player_score_1d([2, 10, 3, 1]), first_player_score_1d([5, 1, 7, 9, 2, 3])   # -> (11, 15)
```

NOTE

With an even number of items taken from the ends, player 1 can take all items at even indexes or all items at odd indexes. So player 1 never loses (Stone Game). This argument gives only the winner, not the score. For `[5, 1, 7, 9, 2, 3]` the better parity sum is 14, but the optimal score is 15.

Other game shapes use the same negamax rule with a different state. If each move takes a prefix, the state is one start index (plus any extra parameter of the rules). If a move gives the same player another turn, add the next value instead of subtracting it. The difference is enough when one player maximizes it and the other player minimizes it. This is always true when the total of the two scores is fixed. It can also be true when the total changes with the order of the moves. **Stone Game VII** (LeetCode 1690): a removal from either end scores the sum of the stones that remain. Player 1 maximizes the difference and player 2 minimizes it. Only the gain of a move changes: it is the sum of the range that stays. Memory hook: *same gap rule; the gain is the sum of what stays.*

```
def stone_game_vii(stones):
    n = len(stones)
    prefix = [0]
    for x in stones:
        prefix.append(prefix[-1] + x)
    gap = [0] * n                           # gap[i]: best gap on stones[i..j] for the current j
    for j in range(n):
        gap[j] = 0                          # one stone: its removal scores 0
        for i in range(j - 1, -1, -1):
            take_left = (prefix[j + 1] - prefix[i + 1]) - gap[i + 1]    # stones[i + 1..j] stay
            take_right = (prefix[j] - prefix[i]) - gap[i]               # stones[i..j - 1] stay
            gap[i] = max(take_left, take_right)
    return gap[0]

stone_game_vii([5, 3, 1, 4, 2]), stone_game_vii([7, 90, 5, 1, 100, 10, 10, 2])   # -> (6, 122)
```

Store a pair of scores only when each player maximizes their own score and the total is not fixed.

61.4.5 State machines: a small status for each step

Best Time to Buy and Sell Stock with Cooldown: buy and sell one share as often as you like, but hold at most one share. After a sale you cannot buy on the next day. Maximize the profit ($n \leq 5000$).

A brute force tries every sequence of buy, sell and wait actions, so the work grows exponentially with n. But the future depends only on the day and on one status: you own a share, or you do not. So two numbers per day are enough. Walk the days backward and store the best profit from day i to the end for each status. In this backward table, the cooldown needs no third status: a sale on day i jumps to day $i + 2$. It only reads the answer that is already stored for day $i + 2$. In `[1, 2, 3, 0, 2]` the best plan is buy, sell, cooldown, buy, sell, for a profit of $1 + 2 = 3$. Memory hook: *walk the days backward. Free waits or buys, holding waits or sells, and a sale skips tomorrow.*

```
def max_profit_cooldown(prices):
    n = len(prices)
    free = [0] * (n + 2)                    # best profit from day i on, with no share
    holding = [0] * (n + 2)                 # best profit from day i on, with one share
    for i in range(n - 1, -1, -1):
        p = prices[i]
        free[i] = max(free[i + 1], holding[i + 1] - p)      # wait or buy
        holding[i] = max(holding[i + 1], free[i + 2] + p)   # wait or sell; a sale skips tomorrow
    return free[0]                          # day 0 starts with no share

max_profit_cooldown([1, 2, 3, 0, 2])   # -> 3
```

```
max_profit_cooldown([1])              # -> 0
max_profit_cooldown([5, 4, 3])        # -> 0
```

The optimal form runs forward with three variables and $O(1)$ memory. In this forward form, the cooldown is a third status, `sold`, that cannot go directly to `hold`. `hold` owns a share, `sold` sold today, and `rest` owns nothing and did not sell today. The legal arrows are: hold to hold (wait), rest to hold (buy), hold to sold (sell), sold to rest and rest to rest (wait). A buy reads `rest` and never `sold`. That one arrow is the cooldown.

```
def max_profit_states(prices):
    hold, sold, rest = float("-inf"), 0, 0  # holding before day 0 is impossible
    for p in prices:
        hold, sold, rest = max(hold, rest - p), hold + p, max(rest, sold)
    return max(sold, rest)                  # never end while holding

max_profit_states([1, 2, 3, 0, 2]), max_profit_states([7, 1, 5, 3, 6, 4])   # -> (3, 5)
```

The general method: list the statuses that the rules need, draw the legal arrows, and write one max per status per step.

- A fee: subtract it on each sale. No new status.
- At most k transactions: add the number of transactions to the state.
- A delay of k steps after an event: add k waiting statuses, or jump to step $i + k + 1$.
- A choice that blocks the next step (House Robber), or a color that the next house cannot repeat (Paint House): the status is the last choice.

61.5 Adapt it to a new problem

1. Say the brute force in one sentence. Name what it lists: paths, orders, lines of play or action sequences.
2. Ask what the rest of the problem needs to know about the past. That is the state. If the answer depends on the full path (a visited set), look for a strict key that removes the cycles.
3. Find the split. Grid: the last move into the cell. DAG: the first step out of the node. Range: the action that leaves two independent sides (the last removal or merge, the first cut). Game: the move of the player to move. State machine: the action at this step.
4. Choose the combine: sum, min, max, gain minus the next value, or "some move reaches a losing state".
5. Find the order. Forward moves only: row by row. A strict key: sort by the key. A range: by length. Time: by step. If no order is clear, memoize.
6. Fill from the end when the condition of a step depends on what comes after it. Then the value of a state means "the best from here to the end".
7. Add sentinels so that the boundary states use the same formula: walls with value 1, extra zero cells after the last day, or the positions 0 and the full length.
8. Check the size: states times choices. $n \leq 20$: an exponential search can pass. n up to a few hundred: $O(n^3)$. n up to a few thousand: $O(n^2)$. n up to 10^5: $O(n)$ with a few statuses.
9. Check the depth. If the recursion can go deeper than about 1000 calls, write the bottom-up form.
10. Reduce memory last, and only if the limits need it (one row, three variables, one array).

61.6 Pitfalls

CAUTION

Wrong fill order or wrong split in an interval DP. `dp[l][r]` reads `dp[k][r]` with $k > l$. If `l` goes up, that value is not computed yet. Fill by length, or move `l` down and `r` up. Also, a split on the element that acts first uses neighbors that are not fixed, so the two sides interact.

```
def coins(nums, l_order):
    vals = [1] + nums + [1]
    n = len(vals)
    dp = [[0] * n for _ in range(n)]
    for l in l_order(n):
        for r in range(l + 2, n):
            dp[l][r] = max(dp[l][k] + vals[l] * vals[k] * vals[r] + dp[k][r]
                           for k in range(l + 1, r))
```

```
    return dp[0][n - 1]

coins([3, 1, 5, 8], lambda n: range(n))                 # -> 63
coins([3, 1, 5, 8], lambda n: range(n - 1, -1, -1))     # -> 167
```

CAUTION

Update all state variables at the same time. If `hold`, `sold` and `rest` change one by one, later lines read new values of the same day. In the example below, `rest` then includes the sale of the same day. So the buy on the next day reads a sale of the day before, and the cooldown disappears. The result is the profit with no cooldown. Use one tuple assignment, or temporary variables.

```
def profit_one_by_one(prices):              # wrong
    hold, sold, rest = float("-inf"), 0, 0
    for p in prices:
        hold = max(hold, rest - p)
        sold = hold + p
        rest = max(rest, sold)
    return max(sold, rest)

def profit_together(prices):                # right
    hold, sold, rest = float("-inf"), 0, 0
    for p in prices:
        hold, sold, rest = max(hold, rest - p), hold + p, max(rest, sold)
    return max(sold, rest)

profit_one_by_one([1, 2, 3, 0, 2]), profit_together([1, 2, 3, 0, 2])   # -> (4, 3)
```

CAUTION

A non-strict key makes cycles. With `>=` instead of `>`, two equal neighbors point to each other. A memoized DFS then calls itself with no end, because the cache gets a value only after a call returns. Python stops it with a `RecursionError`. A strict comparison is the reason that the cache by cell is correct.

```
from functools import cache

def longest_non_strict(matrix):             # wrong: >= allows equal steps
    rows, cols = len(matrix), len(matrix[0])
    @cache
    def f(r, c):
        best = 0
        for nr, nc in ((r + 1, c), (r - 1, c), (r, c + 1), (r, c - 1)):
            if 0 <= nr < rows and 0 <= nc < cols and matrix[nr][nc] >= matrix[r][c]:
                best = max(best, f(nr, nc))
        return best + 1
    return f(0, 0)

longest_non_strict([[1, 1]])                # -> raises RecursionError
```

CAUTION

Recursion depth. A memoized DFS goes as deep as the longest chain of states. A row of 5000 days, 1000 cards or a 40000-cell snake path is deeper than the default limit of 1000. Write the bottom-up form, or raise the limit with `sys.setrecursionlimit`. A higher limit does not make the C stack larger, so very deep recursion can still crash the interpreter.

```
from functools import cache

@cache
def suffix_best(i):                         # depth grows with i
    return 0 if i == 0 else 1 + suffix_best(i - 1)
```

```
suffix_best(5000)                          # -> raises RecursionError
suffix_best.cache_clear()
for i in range(0, 5001, 500):              # warm the cache in small steps
    suffix_best(i)
suffix_best(5000)                          # -> 5000
```

CAUTION

Games: the tie rule and the memo marker. When a tie counts as a win, test `gap >= 0`, not `gap > 0`. A difference of 0 or a negative difference is a valid result. Do not use 0 or -1 as the "not computed" mark in a memo table. Use `None` or `@cache`.

CAUTION

Base values and the final status. In the forward stock form, `hold` starts at minus infinity, not at 0. A start at 0 pretends that you own a free share. The answer comes from a status with no share: `max(sold, rest)`.

CAUTION

Shared rows. `[[0] * n] * n` repeats one inner list, so all rows change together. Build each table with `[[0] * n for _ in range(n)]`.

CAUTION

Greedy rules fail here. Bursting the smallest balloon first gives 78 on `[3, 1, 5, 8]`. The answer is 167. Taking the larger end card gives player 1 a score of 5 on `[2, 10, 3, 1]`. The answer is 11. Test a greedy idea against the DP or a brute force on small inputs.

61.7 Problems in this family

Problem	What changes	Memory hook
Unique Paths	Count paths with down and right moves. The base template: a sum of two predecessors, row by row.	Fill the grid with ones, then each cell equals up plus left.
Longest Increasing Path in a Matrix	Four directions. A strict increase makes a DAG. Order the cells by value; 1 plus a max.	Biggest first: each cell is 1 plus its best bigger neighbor.
Burst Balloons	State is an open range between two walls. The split is the balloon that bursts last. $O(n^3)$.	Pick the balloon that pops last. It sees only the two walls.
Best Time to Buy and Sell Stock with Cooldown	State is (day, holding or not). Fill backward. A sale jumps to day $i + 2$.	Walk the days backward. Free waits or buys, holding waits or sells, and a sale skips tomorrow.
Predict the Winner	Two-player game on a range. Store the difference for the player to move. Answer: `gap >= 0`.	My pick minus your best gap, max of both ends. A tie wins.

The transfer drills for this technique are in Section 69.19 (page 425). The answers are in Section A.18 (page 459).

62 Greedy

62.1 The idea

A greedy algorithm makes one local choice at each step and never undoes it. Brute force tries every combination of choices, and that is exponential. Dynamic programming keeps the best result for every state. A greedy scan keeps one choice, or a small summary of all choices, and reads the input one time. The scan costs $O(n)$. A sort before the scan adds $O(n \log n)$.

NOTE

A variant can take a choice back. It keeps its past choices in a heap. When a budget breaks, it undoes the weakest choice and pays for that step in a different way. Each choice goes into the heap and comes out at most one time, so the cost stays $O(n \log n)$ (Furthest Building You Can Reach). See the variation "Take back the weakest choice" below.

A small example shows how a set of choices collapses into one number. In Jump Game, `nums[i]` is the longest jump from index i, and every shorter jump is also allowed. Take `nums = [2, 3, 1, 1, 4]`. From index 0, a search tries index 1 and index 2. But a jump that can land on index 2 can also stop on index 1. So the reachable indices always form a prefix `0..farthest`, and one number describes them. Index 0 sets `farthest` to 2. Index 1 sets it to $1 + 3 = 4$, which is the last index. The answer is true. For `[3, 2, 1, 0, 4]`, `farthest` stops at 3. Index 4 lies past it, so the answer is false.

A local choice can look good and still be wrong. So each greedy needs a reason. The reason is usually one of the six in the table below. The exchange argument is the most general reason. Take any optimal answer. Change its choices into the greedy choices, one at a time, and show that the answer never becomes worse. For example, to keep the most meetings that do not overlap, the greedy keeps the meeting that ends first. An optimal answer starts with some meeting that ends at the same time or later. Swap that meeting for the greedy one. The rest of the answer still fits, and the count does not change (Non-overlapping Intervals). Task Scheduler and the wall-crossing walk below use the same argument.

Reason	Question to ask	Examples
Forced choice	Does an extreme element (the smallest, the first) have only one possible role?	Hand of Straights
Exchange argument	Can you change an optimal answer into the greedy answer, one choice at a time, with no loss?	Task Scheduler, a walk that maximizes wall crossings, Furthest Building You Can Reach
Skip ahead	Does one failure rule out a whole range of candidates?	Gas Station, Maximum Subarray
Contiguous states	Is the set of reachable states always an interval? Then keep its ends.	Jump Game, Jump Game II, Partition Labels, Valid Parenthesis String
One-way operation	Can a value that passes the target ever come back? If not, drop that element.	Merge Triplets to Form Target Triplet
Two lower bounds	Do the rules point to both neighbors? Meet each side in its own pass, then take the max per item.	Candy

If you cannot name the reason, test the greedy against a brute force on all small inputs. The core template shows how.

62.2 Recognize it

- The task asks for one number (a maximum, a minimum, a count) or for yes or no. It does not ask for all solutions.
- Typical phrases point to a variation below: "can you reach" (one frontier), "the fewest jumps" (BFS windows), "the most parts" (cut at the farthest end), "at least n slots apart" (most frequent first), "after any number of operations" (filter, then combine), "more than its neighbors" (one pass per side).
- n is 10^4 to 10^5 or more. A DP over pairs of indices, or over each possible move, is too slow.
- Each step has a choice that cannot hurt: extend or restart, the farthest reach, the smallest element, the most frequent item.
- A move allows any step up to a limit, or a wildcard changes a counter by -1, 0 or $+1$. Then the reachable states have no gaps, and two numbers describe them.
- One failure at position j rules out every candidate between the current start and j.
- An operation moves values in one direction only: max, min, bitwise OR, bitwise AND. An element that passes the target is useless.
- The input order does not matter. Sort it or count it, then start from an extreme element.

- Two kinds of budget pay for the costs, and the scarce kind must cover the largest costs. Give each cost the scarce kind first, then take back the smallest one from a heap.
- An action can repeat at a fixed cost, and the budget is huge. Pay a one-time cost to unlock the action, then repeat it with an integer division.
- Counter-signal: a small input where the local choice blocks a better choice later. The classic case is the largest coin first, which fails for coins $\{1, 3, 4\}$ and amount 6. Use DP there.

62.3 Core template

Most greedy scans have the same shape. Keep a small state that summarizes all choices so far. At each element, test for a failure first. Then apply the local choice. Then update the answer.

```
state = start_value               # a few numbers that summarize all choices so far
for x in items:                   # sorted or counted first when the order is free
    if failed(state, x):          # a gap, a negative balance, a missing card
        ...                       # return early, or reset the candidate
    state = choose(state, x)      # the local choice; it is never undone
    best = better(best, state)
```

The frontier scan of Jump Game is the clearest example of this shape.

```
def can_reach_end(nums):
    farthest = 0                          # right end of the reachable prefix
    for i, jump in enumerate(nums):
        if i > farthest:                  # gap: no reachable index jumps to i
            return False
        farthest = max(farthest, i + jump)
    return True

can_reach_end([2, 3, 1, 1, 4])            # -> True
can_reach_end([3, 2, 1, 0, 4])            # -> False
can_reach_end([2, 0, 0])                  # -> True
can_reach_end([0])                        # -> True
```

Invariant: before index i, every index in `0..farthest` is reachable, and no jump from an index before i goes past `farthest`. If i is past `farthest`, no reachable index jumps to i, so index i and all later indices are cut off.

62.3.1 Check a greedy against a brute force

A greedy rule is easy to invent and easy to get wrong. Write the brute force that tries every choice. Then compare the two solutions on every small input. Generate the inputs from the smallest size up. The first difference is then the smallest counterexample, and you can trace it by hand.

```
from itertools import product

def can_reach_end(nums):                  # the greedy under test
    farthest = 0
    for i, jump in enumerate(nums):
        if i > farthest:
            return False
        farthest = max(farthest, i + jump)
    return True

def brute_reach(nums):                    # try every jump with a DFS
    seen, stack = {0}, [0]
    while stack:
        i = stack.pop()
        for j in range(i + 1, min(len(nums), i + nums[i] + 1)):
            if j not in seen:
                seen.add(j)
                stack.append(j)
    return len(nums) - 1 in seen
```

```
def first_counterexample(fast, slow, max_len=5, max_val=3):
    for n in range(1, max_len + 1):       # the smallest inputs first
        for t in product(range(max_val + 1), repeat=n):
            if fast(list(t)) != slow(list(t)):
                return list(t)
    return None

def zero_blocks(nums):                    # a plausible rule: "a zero blocks the path"
    return 0 not in nums[:-1]

first_counterexample(can_reach_end, brute_reach)     # -> None
first_counterexample(zero_blocks, brute_reach)       # -> [2, 0, 0]
```

Keep the value range small. Small ranges make zeros, ties and duplicates, and these break most wrong rules. When the inputs are too many to list, use random small inputs from `random.Random(seed)`. A clean run on all small inputs is strong evidence for a greedy. It is not a proof, but it catches most wrong rules.

62.4 Variations

62.4.1 Extend or restart: drop a negative run

The state is a running sum. The local choice is: extend the current run, or start a new run at this element. The best subarray that ends at index i either extends the best subarray that ends at $i-1$, or it starts at i. The extension helps only when the previous sum is positive. So clamp the previous sum at 0, then add the current element (Maximum Subarray). The reason is skip ahead: a run with a negative sum never helps a run that extends it to the right.

Memory hook: *A negative running sum only hurts, so drop it and restart.*

```
def max_subarray(nums):
    best = nums[0]                # the subarray is non-empty: never start at 0
    cur = 0                       # best sum of a subarray that ends at the previous index
    for x in nums:
        cur = max(cur, 0) + x     # extend a positive run, or restart at x
        best = max(best, cur)
    return best

max_subarray([-2, 1, -3, 4, -1, 2, 1, -5, 4])    # -> 6
max_subarray([-3, -1, -2])                       # -> -1
max_subarray([4, -1, 2, 1])                      # -> 6

def max_subarray_span(nums):      # also return (start, end) of one best subarray
    best, span = nums[0], (0, 0)
    cur = start = 0
    for i, x in enumerate(nums):
        if cur < 0:               # a negative run: restart here
            cur, start = 0, i
        cur += x
        if cur > best:
            best, span = cur, (start, i)
    return best, span

max_subarray_span([-2, 1, -3, 4, -1, 2, 1, -5, 4])   # -> (6, (3, 6))
```

Restart when the running sum is negative, not when the current element is negative. In `[4, -1, 2, 1]` the -1 belongs to the best subarray. When one state is not enough, keep a small fixed set of states for each index. Examples are the largest and the smallest product (Maximum Product Subarray), or "no deletion used" and "one deletion used" (Maximum Subarray Sum with One Deletion).

62.4.2 Reachability: keep one frontier

Each position allows any move up to a maximum, and the question is only yes or no (Jump Game). The reachable positions are always a prefix, so keep its right end, `farthest`. The core template is the full solution. Test for a gap before you stretch `farthest`. A backward scan also works: move a `goal` left to each index `i` with `i + nums[i] >= goal`, and test `goal == 0` at the end.

Memory hook: *Walk forward; if you pass farthest you are stuck, else stretch farthest.*

The same collapse works for other DPs. Suppose a cell is true when any cell in a window ahead of it is true. Check whether only the nearest or the farthest true cell matters. If it does, the DP needs $O(1)$ state.

62.4.3 BFS by levels on a line

Now the question is the minimum number of moves (Jump Game II). All moves cost the same, so this is a BFS. Each index reaches a contiguous range of next indices. So each BFS layer is also a contiguous window `[left, right]`, and two numbers replace the queue. The next window starts at `right + 1`. It ends at the farthest `i + nums[i]` over the current window. Each new window costs one more jump.

Memory hook: *BFS in windows: the next window runs from right + 1 to the farthest reach of this window.*

```
from itertools import product

def min_jumps(nums):
    jumps = 0
    left = right = 0                # window: the indices whose fewest jumps is jumps
    while right < len(nums) - 1:
        farthest = max(i + nums[i] for i in range(left, right + 1))
        if farthest <= right:       # no progress: the last index is unreachable
            return -1
        left, right = right + 1, farthest
        jumps += 1
    return jumps

min_jumps([2, 3, 1, 1, 4])          # -> 2
min_jumps([2, 3, 0, 1, 4])          # -> 2
min_jumps([0])                      # -> 0
min_jumps([1, 0, 2])                # -> -1

def min_jumps_flat(nums):           # needs a reachable last index
    jumps = cur_end = farthest = 0
    for i in range(len(nums) - 1): # stop before the last index
        farthest = max(farthest, i + nums[i])
        if i == cur_end:            # the current layer ends here
            jumps += 1
            cur_end = farthest
    return jumps

small = [list(t) for n in range(1, 6) for t in product(range(4), repeat=n)]
all(min_jumps_flat(a) == min_jumps(a) for a in small if min_jumps(a) >= 0)   # -> True
```

The flat form is the usual optimal code. It has the same $O(n)$ time. It closes a layer when `i` equals `cur_end`. Stop it at index $n - 2$. If it also visits index $n - 1$, it counts one extra jump when `cur_end` is exactly the last index. Update `farthest` before you test `cur_end`. To cover a line with the fewest intervals (Video Stitching), first store the farthest right end for each left end. Then run the same window scan, and return -1 on a gap.

62.4.4 Reset the start on failure

A process must keep a balance at zero or more: fuel, money, energy, or stack depth. Turn each step into one net change. On a circle, the question is which start works (Gas Station). First check the total. If the total gas is less than the total cost, no start works. Then drive one time from left to right. When the tank goes below zero at station j, the start moves to $j + 1$. The reason is skip ahead. The car reached every station between the old start and j with zero or more gas. So a start at one of these stations drops a non-negative amount of gas, and it also

fails at j. The surviving start sits just after the lowest prefix sum, and the total check covers the wrap.

Memory hook: *Not enough total gas means -1; out of gas means start after here.*

```python
def can_complete_circuit(gas, cost):
    if sum(gas) < sum(cost):       # not enough gas in total: no start works
        return -1
    start = tank = 0
    for i in range(len(gas)):
        tank += gas[i] - cost[i]
        if tank < 0:               # no start in start..i reaches i + 1
            start, tank = i + 1, 0
    return start

can_complete_circuit([1, 2, 3, 4, 5], [3, 4, 5, 1, 2])   # -> 3
can_complete_circuit([2, 3, 4], [3, 4, 3])               # -> -1
can_complete_circuit([2, 0], [0, 2])                     # -> 0
```

Maximum Subarray uses the same skip: a negative run is dropped, and the scan restarts after it.

62.4.5 Forced choice at the smallest element

All items must go into groups of a rigid shape, for example `group_size` consecutive values (Hand of Straights). Only the multiset matters, so count the values. The smallest card that is left has no smaller neighbor. It cannot be the middle or the end of a straight. So it must start a straight. The choice is forced, and a forced choice never needs backtracking. Walk the sorted hand. Skip a card whose count is already 0, because an earlier straight used it.

Memory hook: *Smallest card left starts a straight. Pull its next cards one by one.*

```python
from collections import Counter

def is_n_straight_hand(hand, group_size):
    if len(hand) % group_size:
        return False
    count = Counter(hand)
    for card in sorted(hand):
        if count[card] == 0:       # an earlier straight used this card
            continue
        for value in range(card, card + group_size):   # card starts a straight
            if count[value] == 0:
                return False
            count[value] -= 1
    return True

is_n_straight_hand([1, 2, 3, 6, 2, 3, 4, 7, 8], 3)   # -> True
is_n_straight_hand([1, 2, 3, 4, 5], 4)               # -> False
is_n_straight_hand([3, 3, 2, 2, 1, 1], 3)            # -> True
```

The optimal code walks only the distinct values and subtracts all copies at one time. It tests `count[v] < need`. The time is the same $O(n \log n)$, so use it only when the hand has many copies. Values go up to 10^9, so use a hash map and not an array indexed by value. The same forced choice pairs `x` with `2 * x` when you process the values from the smallest (Find Original Array From Doubled Array).

62.4.6 Filter, then combine everything

The operation combines elements with max, and you may apply it any number of times (Merge Triplets to Form Target Triplet). Max is associative, commutative and idempotent. So the order and the repeats do not matter. Every reachable result is the coordinate-wise max of some subset. Max never lowers a value. A triplet that passes the target in one coordinate can never be used, so drop it. Every other triplet is harmless. Combine all of them, and compare the result with the target.

Memory hook: *Drop any triplet that overshoots, max the rest, compare with target.*

```python
def merge_triplets(triplets, target):
    x, y, z = target
    best = [0, 0, 0]
    for a, b, c in triplets:
        if a <= x and b <= y and c <= z:     # an overshoot never comes back down
            best = [max(best[0], a), max(best[1], b), max(best[2], c)]
    return best == list(target)

merge_triplets([[2, 5, 3], [1, 8, 4], [1, 7, 5]], [2, 7, 5])   # -> True
merge_triplets([[3, 4, 5], [4, 5, 6]], [3, 2, 5])              # -> False
merge_triplets([[2, 2, 3], [1, 1, 2]], [2, 2, 2])              # -> False

def can_or_to(nums, target):       # the same idea for bitwise OR
    if target == 0:                # the empty selection does not count
        return 0 in nums
    acc = 0
    for v in nums:
        if v | target == target:   # v sets no bit outside target
            acc |= v
    return acc == target

can_or_to([1, 2, 8], 3)            # -> True
can_or_to([1, 6], 3)               # -> False
can_or_to([7, 1], 0)               # -> False
```

The accumulator starts at the identity value, so it equals the result of an empty selection. If the target can be that value, test it by itself. In Merge Triplets every target value is at least 1. Other one-way operations use the same filter. For AND, keep `v` with `v & target == target`. For min, keep the elements that are at least the target in every coordinate. For gcd, keep the multiples of the target.

62.4.7 Cut at the farthest required end

Split a sequence into the most contiguous parts so that each value stays inside one part (Partition Labels). Each letter forces a span from its first to its last occurrence. Store the last index of each letter. Scan from the left, and push the end of the current part out to the largest last index seen. When the scan index reaches that end, no letter on the left appears later. Cut there.

Memory hook: *Each letter drags the cut to its last spot; cut when you catch up.*

```python
def partition_labels(s):
    last = {c: i for i, c in enumerate(s)}   # a later index overwrites: the last one wins
    sizes = []
    start = end = 0
    for i, c in enumerate(s):
        end = max(end, last[c])    # this letter drags the end of the part
        if i == end:               # caught up: a legal cut
            sizes.append(end - start + 1)
            start = i + 1
    return sizes

partition_labels("ababcbacadefegdehijhklij")   # -> [9, 7, 8]
partition_labels("eccbbbbdec")                 # -> [10]
partition_labels("abc")                        # -> [1, 1, 1]
```

This is the running maximum of Jump Game again. The scan meets the start of each span in order, so the ends are enough. A running maximum of the ends replaces a sort and a merge of intervals. The same cut rule works whenever each element forces its part to reach some index, not only its last occurrence.

62.4.8 Rules toward both neighbors: one pass per side

Some rules compare an item with its left neighbor and with its right neighbor. For example, each child gets at least one candy. A child with a higher rating than a neighbor gets more candy than that neighbor (Candy). One left-to-right pass sees only the left neighbor, so it breaks the rules toward the right. One pass from the left meets every left rule with the smallest values. One pass from the right does the same for the right rules. Each pass gives a lower bound that every valid answer must meet. So no valid answer is smaller than the max of the two bounds. The max is also valid. When the rating goes up from $i-1$ to i, the right pass gives $i-1$ only 1. So the max keeps the left rule. The right rules hold for the same reason.

Memory hook: *Left pass for the left rules, right pass for the right rules, take the max per child.*

```
def candy(ratings):
    n = len(ratings)
    left = [1] * n                  # meets every rule toward the left neighbor
    for i in range(1, n):
        if ratings[i] > ratings[i - 1]:
            left[i] = left[i - 1] + 1
    right = [1] * n                 # meets every rule toward the right neighbor
    for i in range(n - 2, -1, -1):
        if ratings[i] > ratings[i + 1]:
            right[i] = right[i + 1] + 1
    return sum(max(a, b) for a, b in zip(left, right))

def candy_left_only(ratings):       # wrong: it looks only at the left neighbor
    c = [1] * len(ratings)
    for i in range(1, len(ratings)):
        if ratings[i] > ratings[i - 1]:
            c[i] = c[i - 1] + 1
    return sum(c)

candy([1, 0, 2])                    # -> 5
candy([1, 2, 2])                    # -> 4
candy([1, 3, 4, 5, 2])              # -> 11
candy_left_only([1, 0]), candy([1, 0])   # -> (2, 3)
```

Equal neighbors have no rule, so `[1, 2, 2]` gives `[1, 2, 1]`. The same shape, a left pass, a right pass and a combine per item, also gives the product of all other elements (Product of Array Except Self) and the water above each bar (Trapping Rain Water).

62.4.9 Track a range of possible states

A scan keeps a counter, and some steps are uncertain. In Valid Parenthesis String, each `*` acts as `(`, as `)`, or as nothing. Brute force tries 3^k assignments. But the set of reachable open counts is always a contiguous range `[lo, hi]`, because each step changes the counter by a run of consecutive amounts. So keep only the two ends. If `hi` goes below zero, no assignment survives. If `lo` goes below zero, clamp it to zero, because a negative count is never legal. At the end, zero must be in the range, so test `lo == 0`.

Memory hook: *Range of open counts: fail if the most goes below zero, clamp the fewest at zero, and end with fewest == 0.*

```
def check_valid_string(s):
    lo = hi = 0                     # fewest and most unmatched "(" after this prefix
    for c in s:
        if c == "(":
            lo, hi = lo + 1, hi + 1
        elif c == ")":
            lo, hi = lo - 1, hi - 1
        else:                       # "*" closes one, opens one, or does nothing
            lo, hi = lo - 1, hi + 1
        if hi < 0:                  # too many ")" even if every "*" opens
            return False
        lo = max(lo, 0)             # an open count is never negative
```

```
    return lo == 0                # zero open is in the range

check_valid_string("(*))")        # -> True
check_valid_string("(*)(")        # -> False
check_valid_string(")*")          # -> False
check_valid_string("*(")          # -> False
```

The range works when each step maps an interval of states to an interval, and the legal states form an interval. If a wildcard must be `(` or `)` and cannot be empty, all reachable counts have the same parity. Then raise `lo` by 2, not to 0, when it goes below zero. To count the valid assignments, the range is not enough. Use a DP over (index, open count).

62.4.10 Most frequent first, one window at a time

Equal items need a gap of at least n slots, and the question is the shortest schedule (Task Scheduler). Only the counts matter. The most frequent task is the bottleneck: if you delay it, its copies pile up at the end with idle slots between them. This is an exchange argument. Fill each window of $n+1$ slots with the most frequent different tasks, from a max-heap of counts. A full window costs $n+1$. The last window costs only the tasks that ran in it.

Memory hook: *Fill each window of n + 1 with the most frequent different tasks. Only the last window has no idle tail.*

```
import heapq
from collections import Counter

def least_interval(tasks, n):
    heap = [-k for k in Counter(tasks).values()]   # max-heap of counts
    heapq.heapify(heap)
    time = 0
    while heap:
        ran = []                                   # one window of n + 1 slots
        while heap and len(ran) < n + 1:
            ran.append(heapq.heappop(heap) + 1)    # run one copy
        for k in ran:
            if k:                                  # copies remain
                heapq.heappush(heap, k)
        time += n + 1 if heap else len(ran)
    return time

def least_interval_formula(tasks, n):
    counts = list(Counter(tasks).values())
    m = max(counts)
    ties = counts.count(m)                         # kinds in the last window
    return max(len(tasks), (m - 1) * (n + 1) + ties)

least_interval(list("AAABBB"), 2)                  # -> 8
least_interval(list("AAABBB"), 3)                  # -> 10
least_interval(list("ACABDB"), 1)                  # -> 6
least_interval(list("AAABBBCCCDDE"), 2)            # -> 12
least_interval_formula(list("AAABBBCCCDDE"), 2)    # -> 12
```

The closed form is the fastest to type: `max(N, (m - 1) * (n + 1) + ties)`. Here `N` is the number of tasks, `m` is the top count, and `ties` is the number of kinds with count `m`. Do not forget the `max` with the task count: when many kinds fill every gap, there is no idle slot. An event simulation with a FIFO queue of cooling tasks is the other optimal form. For a different cooldown per task, the simulation code still runs: the queue becomes a min-heap keyed by the ready time. But the most-frequent-first rule is then not always optimal, so check it against a brute force. For example, take three B tasks with cooldown 0 and two A tasks with cooldown 5. The greedy runs B first, so the second A cannot run before slot 8. A schedule that runs A first needs only 7 slots.

62.4.11 Take back the weakest choice

Two kinds of budget pay for a sequence of costs, and one kind is scarce. For example, each climb to a taller building costs bricks equal to its height, or one ladder (Furthest Building You Can Reach). A prefix of climbs is possible when the bricks pay for all climbs except the largest ones, one ladder for each. But the scan does not know the later climbs. So give each climb a ladder first, and keep these climbs in a min-heap. When the ladders run out, take back the smallest ladder climb and pay for it with bricks. The heap then always holds the largest climbs so far. Each climb goes in and comes out at most one time.

Memory hook: *Ladder first; when the ladders run out, the smallest ladder climb goes back to bricks.*

```
import heapq

def furthest_building(heights, bricks, ladders):
    climbs = []                          # min-heap: the climbs that use a ladder now
    for i in range(len(heights) - 1):
        d = heights[i + 1] - heights[i]
        if d <= 0:
            continue                     # a step down is free
        heapq.heappush(climbs, d)        # first give this climb a ladder
        if len(climbs) > ladders:        # one ladder too few: take one back
            bricks -= heapq.heappop(climbs)   # the smallest climb uses bricks
            if bricks < 0:
                return i
    return len(heights) - 1

furthest_building([4, 2, 7, 6, 9, 14, 12], 5, 1)          # -> 4
furthest_building([4, 12, 2, 7, 3, 18, 20, 3, 19], 10, 2) # -> 7
furthest_building([14, 3, 19, 3], 17, 0)                  # -> 3
```

The heap holds at most `ladders + 1` climbs, so the time is $O(n \log k)$ for k ladders. The same take-back keeps a total under a limit. Take the courses in order of deadline. When the time passes the deadline, drop the longest course taken so far (Course Schedule III).

62.4.12 Unlock, then farm: case analysis with a brute-force check

Some problems let you repeat one action at a fixed cost after a one-time cost to reach it. For example, take a walk that starts at position 0 of a number line. Walls sit at half-integer positions. A step costs 1, or the wall thickness `t` when it crosses a wall, and each crossing is one hit. Walls stay, so you can cross one wall again and again. Return the most hits with at most `energy`.

Case analysis gives the plan. Take any walk, and let `w` be the cheapest wall that it crosses. Let j be the number of walls between 0 and `w`, and let `base` be the cost to stand next to `w`. The inner walls give j hits inside `base`. Every other hit costs at least the thickness of `w`. This is an exchange argument: move every extra hit onto `w`, and the cost does not grow. So the walk has at most `j + (energy - base) // t` hits, and the walk "go straight to `w`, then bounce" reaches that bound. Try each wall as the bounce wall. Sort each side by distance, keep a running cost, and score each wall in $O(1)$.

Memory hook: *Walk out each side, and at every wall cash in all leftover energy as bounces.*

```
import random
from functools import lru_cache

def max_wall_hits(walls, thickness, energy):
    # plain steps from 0 to the near side of each wall, one sorted list per side
    right = sorted((int(w - 0.5), t) for w, t in zip(walls, thickness) if w > 0)
    left = sorted((int(-w - 0.5), t) for w, t in zip(walls, thickness) if w < 0)
    best = 0
    for side in (right, left):
        spent = pos = 0                  # cost to stand here; distance walked
        for j, (dist, t) in enumerate(side):
            spent += dist - pos          # plain steps up to this wall
            if spent + t > energy:       # cannot cross it once; farther walls cost more
```

```
                break
            best = max(best, j + (energy - spent) // t)   # j inner hits, then bounce
            spent += t                  # cross once and go on
            pos = dist + 1
    return best

def brute_hits(walls, thickness, energy):   # try every walk; fine for a tiny energy
    cost = {int(w - 0.5): t for w, t in zip(walls, thickness)}   # key k: wall between k and k + 1
    @lru_cache(maxsize=None)
    def most(pos, left):
        best = 0
        for gap, nxt in ((pos, pos + 1), (pos - 1, pos - 1)):
            c = cost.get(gap, 1)
            if c <= left:
                best = max(best, (gap in cost) + most(nxt, left - c))
        return best
    return most(0, energy)

max_wall_hits([-1.5, 0.5, 1.5, 5.5], [2, 4, 8, 3], 3)   # -> 1
max_wall_hits([0.5], [2], 7)                             # -> 3
max_wall_hits([2.5], [1], 2)                             # -> 0
rng = random.Random(7)
ok = True
for _ in range(300):
    ks = rng.sample(range(-5, 5), rng.randint(0, 4))
    walls = [k + 0.5 for k in ks]
    thick = [rng.randint(1, 4) for _ in ks]
    e = rng.randint(0, 12)
    ok = ok and max_wall_hits(walls, thick, e) == brute_hits(walls, thick, e)
ok                                                       # -> True
```

Use this method for any greedy that comes from a case analysis. State the canonical shape of an optimal answer, prove a bound, and confirm the code with a brute force on tiny inputs. The energy can be 10^{12}, so never simulate the bounces. Use an integer division, and keep all cost math in integers. Convert the half-integer positions to integer step counts one time, before any cost math.

62.5 Adapt it to a new problem

Ask these questions in this order.

1. What is the output: one number, yes or no, or a construction? Does the input order matter? If it does not, sort the input or count it first.
2. What does the brute force choose at each step, and what state does it carry?
3. Can the state collapse into a summary? Look for a prefix (one frontier), an interval (two ends), a running sum that you can drop, or a frequency map.
4. Which reason makes the local choice safe: forced choice, exchange argument, skip ahead, contiguous states, a one-way operation, or two lower bounds? If no reason fits, suspect DP. If a later item can show that an earlier choice was weak, keep the choices in a heap and take one back.
5. In one step of the scan, what is the order? Usually: test for failure, apply the choice, update the answer. Write the invariant in one sentence.
6. Which global checks come first? Examples are a total (Gas Station), a divisibility (Hand of Straights), and the impossible case (-1).
7. Test the greedy against a brute force on all small inputs. Include zeros, ties, negative values and $n = 1$.
8. If a counterexample appears, make the summary a DP state, and add the information that the greedy ignored.

62.6 Pitfalls

CAUTION

The wrong key for the local choice. Wrong: in Jump Game II, always jump as far as possible, or pick the next index with the largest `nums[i]`. Right: pick the next stop whose own reach `i + nums[i]` is the farthest. The window scan does this for you.

```
def jump_longest_each_time(nums):        # wrong: always take the longest jump
    i = jumps = 0
    while i < len(nums) - 1:
        if nums[i] == 0:
            return -1                    # stuck
        i, jumps = min(len(nums) - 1, i + nums[i]), jumps + 1
    return jumps

def min_jumps(nums):                     # windows: compare reach positions
    jumps = left = right = 0
    while right < len(nums) - 1:
        farthest = max(i + nums[i] for i in range(left, right + 1))
        if farthest <= right:
            return -1
        left, right, jumps = right + 1, farthest, jumps + 1
    return jumps

jump_longest_each_time([2, 2, 0, 0]), min_jumps([2, 2, 0, 0])   # -> (-1, 2)
```

CAUTION

Update and test in the wrong order. Wrong: in Jump Game, stretch `farthest` before you test `i > farthest`. An unreachable index with a positive jump then extends the frontier. Right: test for the gap first. In Partition Labels and Jump Game II the order is the other way. Update the end with the current element first. Then test whether the scan reached it.

```
def can_reach_update_first(nums):        # wrong order
    farthest = 0
    for i, jump in enumerate(nums):
        farthest = max(farthest, i + jump)
        if i > farthest:
            return False
    return True

can_reach_update_first([0, 2, 0])        # -> True
```

The correct answer for `[0, 2, 0]` is false: index 0 cannot move.

CAUTION

A strict test, and a missing global check. Wrong: in Gas Station, reset the start when the tank is 0, or skip the total check. Right: reset only when the tank is below 0. Arriving with exactly zero gas is fine. Check the total first, because the scan alone never returns -1.

```
def gas_start(gas, cost, reset_at_zero=False, check_total=True):
    if check_total and sum(gas) < sum(cost):
        return -1
    start = tank = 0
    for i in range(len(gas)):
        tank += gas[i] - cost[i]
        if tank < 0 or (reset_at_zero and tank == 0):
            start, tank = i + 1, 0
    return start

gas_start([2, 0], [0, 2])                          # -> 0
gas_start([2, 0], [0, 2], reset_at_zero=True)      # -> 2
```

```
gas_start([2, 3, 4], [3, 4, 3])                    # -> -1
gas_start([2, 3, 4], [3, 4, 3], check_total=False)  # -> 2
```

CAUTION

A range of states without the clamp or without the early failure. Wrong: in Valid Parenthesis String, let `lo` go below zero, or test `hi` only at the end. Right: return false as soon as `hi < 0`, then clamp `lo` at zero after each character.

```
STEP = {"(": (1, 1), ")": (-1, -1), "*": (-1, 1)}    # change of (lo, hi)

def valid_star(s, clamp=True, early_fail=True):
    lo = hi = 0
    for c in s:
        lo, hi = lo + STEP[c][0], hi + STEP[c][1]
        if early_fail and hi < 0:
            return False
        if clamp:
            lo = max(lo, 0)
    return lo == 0

valid_star("(*)("), valid_star("(*)(", clamp=False)          # -> (False, True)
valid_star(")*"), valid_star(")*", early_fail=False)         # -> (False, True)
```

CAUTION

The wrong start value for the best answer. Wrong: start `best = 0` in Maximum Subarray. An all-negative array then returns 0. Right: the subarray is non-empty, so start `best` at `nums[0]` or at `float("-inf")`.

CAUTION

A tuple compared with a list. Wrong: build the result as a tuple and compare it with the list `target`. The result is always `False`. Right: compare two lists, or convert both sides to tuples.

```
(2, 7, 5) == [2, 7, 5]                  # -> False
list((2, 7, 5)) == [2, 7, 5]            # -> True
```

CAUTION

A simulation of a huge budget. Wrong: repeat one action at a time when the budget is 10^{12}, or use `/` on large integers. Right: compute the repeat count with `//`. A float loses precision above 2^{53}.

```
energy, t = 10**17 + 1, 3
energy // t                             # -> 33333333333333333
int(energy / t)                         # -> 33333333333333332
```

CAUTION

A greedy rule with no reason. Wrong: trust a rule because it passes the given examples. Right: name the reason from the table in the idea subsection. If you cannot, run the brute-force check. A rule such as "a zero blocks the path" fails at `[2, 0, 0]`.

62.7 Problems in this family

Problem	What changes	Memory hook
Maximum Subarray (53)	The state is a running sum. Clamp the previous sum at 0 before you add the element.	A negative running sum only hurts, so drop it and restart.
Jump Game (55)	The reachable set is a prefix. Keep one frontier and fail on a gap.	Walk forward; if you pass farthest you are stuck, else stretch farthest.

Problem	What changes	Memory hook
Jump Game II (45)	The fewest moves. BFS layers are windows `[left, right]`.	BFS in windows: the next window runs from right + 1 to the farthest reach of this window.
Gas Station (134)	A circle. Check the total, then move the start past each failure.	Not enough total gas means -1; out of gas means start after here.
Hand of Straights (846)	A partition of a multiset. The smallest card has a forced role.	Smallest card left starts a straight. Pull its next cards one by one.
Merge Triplets to Form Target Triplet (1899)	A one-way operation (max). Drop the overshoots and combine the rest.	Drop any triplet that overshoots, max the rest, compare with target.
Partition Labels (763)	Each letter forces a span. Cut when the scan reaches the farthest required end.	Each letter drags the cut to its last spot; cut when you catch up.
Valid Parenthesis String (678)	Wildcards. Keep the range `[lo, hi]` of open counts.	Range of open counts: fail if the most goes below zero, clamp the fewest at zero, and end with fewest == 0.
Task Scheduler (621)	A cooldown. Most frequent first, in windows of $n + 1$ slots.	Fill each window of n + 1 with the most frequent different tasks. Only the last window has no idle tail.

The transfer drills for this technique are in Section 69.20 (page 425). The answers are in Section A.19 (page 461).

63 Intervals

63.1 The idea

An interval is a piece of a line: a meeting, a booking, a range of values, a balloon. Most interval questions are about pairs: do two intervals overlap, and which ones touch? With n intervals there are about $n^2/2$ pairs. At $n = 10^4$ that is $5 \cdot 10^7$ checks, which is too slow in Python.

Sort the intervals by one endpoint. Then each new interval can only touch a small state: the last merged group, the last kept interval, or the set of intervals that are still open. One pass over the sorted list then replaces all the pair checks. The total cost is $O(n \log n)$ for the sort.

A worked example: merge `[[1, 3], [8, 10], [2, 6]]`. Sort by start: `[1, 3]`, `[2, 6]`, `[8, 10]`. Open a group with `[1, 3]`. The next interval starts at 2, and 2 is not after the group end 3. So extend the group to `[1, 6]`. The next interval starts at 8, after the group end 6. Every later interval starts at 8 or later, so nothing can reach back to `[1, 6]` any more. Close it and open `[8, 10]`. The result is `[[1, 6], [8, 10]]`. The sweep compares each interval with one group only.

There are two main sort keys, and each one has a reason. Sort by start when you must handle every interval in time order: merge, conflict check, room count. Sort by end when you choose a subset: the interval that ends first leaves the most room for the rest. The question decides the sort key and the state of the sweep:

Question	Sort by	State during the sweep	Example
Union, merge, covered length, free gaps	start	the last merged group	Merge Intervals
Does any pair conflict?	start	the previous end	Meeting Rooms
Keep the most, remove the fewest, stab with the fewest points	end	the end of the last kept interval	Non-overlapping Intervals
Peak number at one time, minimum resources	events `(time, change)`	a running count	Meeting Rooms II
Best active interval for each query point	start, and the queries	a heap of active intervals	Minimum Interval to Include Each Query
Add one interval to a sorted, disjoint list	no sort	three buckets: left, middle, right	Insert Interval
Intersect two sorted, disjoint lists	no sort	two pointers	(Interval List Intersections)

Before any code, decide what happens at a shared endpoint. For closed intervals `[s, e]`, both ends belong to the interval, and two intervals that touch share a point. For half-open intervals `[s, e)`, the end does not belong, and touching intervals share no point. Two closed intervals overlap when the larger start is at or before the smaller end. Two half-open intervals overlap when the larger start is strictly before the smaller end.

```
def overlap_closed(a, b):           # [s, e]: both ends belong to the interval
    return max(a[0], b[0]) <= min(a[1], b[1])
def overlap_half_open(a, b):        # [s, e): the end does not belong
    return max(a[0], b[0]) < min(a[1], b[1])
overlap_closed([1, 2], [2, 3]), overlap_half_open([1, 2], [2, 3])   # -> (True, False)
overlap_closed([1, 2], [3, 4]), overlap_closed([1, 9], [3, 4])      # -> (False, True)
max(1, 3), min(2, 4)                # the intersection [3, 2] is empty
# -> (3, 2)
```

The intersection of two intervals is `[max of the starts, min of the ends]`. It is empty when that start is after that end.

63.2 Recognize it

- The input is a list of `[start, end]` pairs: meetings, bookings, trips, ranges, segments, time slots.
- The statement uses the words overlap, merge, conflict, attend all, rooms, cover, free time, or intersect.
- The answer does not depend on the input order. So you are free to sort.
- n is 10^4 to 10^5. A check of all pairs is too slow, so the budget is one sort and one pass.
- The question asks for a minimum number of removals so that the rest do not conflict. Rewrite it as the maximum number that you can keep.
- The question asks how many things run at the same time, or for the minimum number of resources. The answer is the peak number of open intervals.
- Many point queries ask about one fixed set of intervals, and all queries are given at the start. Sort the queries too, and answer them in sorted order (offline).
- The statement defines what happens at a shared endpoint. This tells you to choose `<` or `<=` with care.
- The intervals are hidden. Each item spans from a first position to a last position: the first and last index of a character, a subarray, a trip from one stop to another.

63.3 Core template

Sort by start. Compare each interval with the last group of the output only. Extend the group, or open a new group.

```
def merge(intervals):                       # closed intervals: touching ones merge
    out = []
    for s, e in sorted(intervals):          # sort by start; sorted() keeps the input
        if out and s <= out[-1][1]:         # shares a point with the open group
            out[-1][1] = max(out[-1][1], e) # max: a nested interval must not shrink it
```

```
        else:
            out.append([s, e])              # a new list, not the input row
    return out
merge([[1, 3], [8, 10], [2, 6]])           # -> [[1, 6], [8, 10]]
merge([[1, 4], [4, 5]])                    # -> [[1, 5]]
merge([[1, 10], [2, 3], [4, 5]])           # -> [[1, 10]]
merge([[1, 2], [3, 4]])                    # -> [[1, 2], [3, 4]]
merge([[5, 5], [5, 5]]), merge([])         # -> ([[5, 5]], [])
```

Invariant: after k intervals in sorted order, `out` holds the union of these k intervals as disjoint groups, sorted by start. Every earlier group ends before `out[-1]` starts. Every later interval starts at or after the start of `out[-1]`. So only `out[-1]` can overlap a later interval. For half-open intervals, where touching ones stay apart, change the test to `s < out[-1][1]`.

63.4 Variations

63.4.1 Merge, covered length and free gaps

Merge Intervals is the core template. Memory hook: *Sort, then extend the last one or append a new one.* The merged list answers more questions in one more step. The total covered length is the sum of the group lengths. The free gaps are the spaces between two neighbor groups (free time in a calendar, Employee Free Time).

```
def merge(intervals):
    out = []
    for s, e in sorted(intervals):
        if out and s <= out[-1][1]:
            out[-1][1] = max(out[-1][1], e)
        else:
            out.append([s, e])
    return out
def covered_length(intervals):             # length e - s (count integer points: e - s + 1)
    return sum(e - s for s, e in merge(intervals))
def free_gaps(intervals):                  # from the end of one group to the next start
    m = merge(intervals)
    return [[m[k][1], m[k + 1][0]] for k in range(len(m) - 1)]
covered_length([[1, 3], [2, 6], [8, 10]])   # -> 7
free_gaps([[1, 3], [2, 6], [8, 10], [12, 13]])   # -> [[6, 8], [10, 12]]
```

NOTE

Hidden intervals also merge. In Partition Labels, each character spans from its first to its last index. The merged groups are the parts.

63.4.2 Insert into a sorted, disjoint list

The list is already sorted and has no overlaps. One new interval arrives. Do not sort again. Each old interval has exactly one relation to the new interval: it ends before the new start, it starts after the new end, or it shares a point. Put each one in a bucket. The overlapping ones melt into the new interval, which grows with `min` and `max`. The buckets are already in order, so the answer is the left bucket, the merged interval, and the right bucket. Memory hook (Insert Interval): *Left stays left, right stays right, the middle melts into one.*

```
def insert(intervals, new):     # intervals: sorted by start, disjoint, closed
    start, end = new
    left, right = [], []
    for s, e in intervals:
        if e < start:           # ends before the new interval starts
            left.append([s, e])
        elif s > end:           # starts after the new interval ends
            right.append([s, e])
        else:                   # shares a point: melt it into the new interval
            start, end = min(start, s), max(end, e)
    return left + [[start, end]] + right
```

```
insert([[1, 3], [6, 9]], [2, 5])                              # -> [[1, 5], [6, 9]]
insert([[1, 2], [3, 5], [6, 7], [8, 10], [12, 16]], [4, 8])   # -> [[1, 2], [3, 10], [12, 16]]
insert([], [4, 8]), insert([[1, 2]], [2, 3])                  # -> ([[4, 8]], [[1, 3]])
insert([[1, 2], [5, 6]], [3, 4])                              # -> [[1, 2], [3, 4], [5, 6]]
```

This is $O(n)$. The common optimal form is the same idea with one index and two `while` loops: copy while `e < start`, absorb while `s <= end`, append the merged interval, then copy the rest with one slice. The same classify-and-concatenate pattern removes a range from a set of disjoint intervals.

NOTE

For many inserts into a stored list (Range Module, My Calendar), find the middle block with binary search. The intervals are disjoint, so the ends are sorted too. `bisect_left` on the ends gives the first interval with `e >= start`. `bisect_right` on the starts gives the first interval with `s > end`.

63.4.3 Conflict check: can one person attend all

Sort by start. If any two intervals overlap, then some neighbor pair overlaps. The reason: if each interval ends at or before the start of the next one, it also ends at or before every later start. So compare each start with the previous end only. Memory hook (Meeting Rooms): *Sort by start. No meeting may start before the previous one ends.*

```
def can_attend(intervals):              # a meeting that ends at t and one that starts at t do not
    conflict
    iv = sorted(intervals)
    for i in range(1, len(iv)):
        if iv[i][0] < iv[i - 1][1]:     # strict: touching meetings are fine
            return False
    return True
can_attend([[0, 30], [5, 10], [15, 20]])   # -> False
can_attend([[7, 10], [2, 4]])              # -> True
can_attend([[1, 5], [5, 10]]), can_attend([])   # -> (True, True)
```

The comparison with the previous interval is enough here, because the first conflict returns at once. In a merge, compare with the end of the open group instead. A long earlier interval can cover several later ones.

63.4.4 Keep the most: sort by end

The question "remove the fewest so that the rest do not overlap" is the same as "keep the most". Keep the interval that ends first. It leaves the most room on the line for all later intervals. An exchange argument proves it: any best selection can swap its first interval for the one that ends first, with no new conflict. Then repeat on the intervals that start at or after that end. Memory hook (Non-overlapping Intervals): *Sort by end, keep what fits, remove the rest.*

```
def min_removals(intervals):            # touching intervals do not overlap
    kept, prev_end = 0, float("-inf")
    for s, e in sorted(intervals, key=lambda iv: iv[1]):   # earliest end first
        if s >= prev_end:               # it fits after the last kept interval
            kept += 1
            prev_end = e
    return len(intervals) - kept
min_removals([[1, 2], [2, 3], [3, 4], [1, 3]])     # -> 1
min_removals([[1, 2], [1, 2], [1, 2]])             # -> 2
min_removals([[1, 100], [2, 3], [4, 5], [6, 7]])   # -> 1
```

The same loop solves related questions with one changed test:

- Return `kept` itself for the longest chain of non-overlapping intervals. If the chain needs a strict order (`b < c` for `[a, b]` then `[c, d]`), use `s > prev_end` (Maximum Length of Pair Chain).
- Stab every interval with the fewest points: place a point at the end of each interval that the last point misses. For closed intervals, a point on a shared end hits both. So place a new point only when `s > prev_end` (Minimum Number of Arrows to Burst Balloons).

- If the intervals have different values (profit, weight), the greedy fails. Use a DP over the intervals sorted by end, with `bisect_right` on the ends to find the last compatible interval (Maximum Profit in Job Scheduling).

63.4.5 Peak overlap: rooms and resources

The minimum number of rooms equals the largest number of meetings in progress at one moment. Fewer rooms fail at that moment. That many rooms are enough: give each meeting, in start order, any free room. The count changes only at a start or an end. So turn each meeting into two events, sort them, and keep a running count. Memory hook (Meeting Rooms II): *Start is +1, end is -1, sort the events, and keep the peak.*

```
def rooms(intervals):                    # half-open [s, e): an end at t frees a room for a start at t
    events = []
    for s, e in intervals:
        events.append((s, 1))            # a meeting starts: one more room busy
        events.append((e, -1))           # a meeting ends: one room frees
    events.sort()                        # at equal times, (t, -1) sorts before (t, 1)
    busy = most = 0
    for _, change in events:
        busy += change
        most = max(most, busy)
    return most
rooms([[0, 30], [5, 10], [15, 20]])      # -> 2
rooms([[1, 5], [5, 10]])                 # -> 1
rooms([[1, 10], [2, 3], [3, 4]])         # -> 2
```

The event sweep is the general tool: an event can also carry a weight, such as passengers or load. When the coordinates are small, a difference array with a prefix sum replaces the sort (see the section on prefix sums).

Two other forms give the same count with the same $O(n \log n)$ cost. Treat them as asides. The heap form sorts the meetings by start and keeps a min-heap with the end time of each room. Each meeting takes the room that frees first if that room is free. Otherwise it opens a new room. Use the heap when you must know which room frees first, for example to give out room numbers (Meeting Rooms III). The two-list form sorts all starts and all ends in two separate lists, and walks them with two pointers.

```
import heapq
def rooms_heap(intervals):
    ends = []                            # min-heap: the end time of the last meeting in each room
    for s, e in sorted(intervals):
        if ends and ends[0] <= s:        # the room that frees first is free now: reuse it
            heapq.heappop(ends)
        heapq.heappush(ends, e)
    return len(ends)                     # one heap entry for each room
def rooms_two_lists(intervals):
    starts = sorted(s for s, _ in intervals)
    ends = sorted(e for _, e in intervals)
    count = j = 0
    for s in starts:
        if s >= ends[j]:                 # some meeting ended: reuse its room
            j += 1
        else:
            count += 1                   # no room is free: open a new one
    return count
cases = [[[0, 30], [5, 10], [15, 20]], [[7, 10], [2, 4]], [[1, 5], [5, 10]], [[1, 10], [2, 3], [3, 4]]]
[rooms_heap(c) for c in cases]           # -> [2, 1, 1, 2]
[rooms_two_lists(c) for c in cases]      # -> [2, 1, 1, 2]
```

63.4.6 Best active interval for each query: offline sweep with a heap

Many point queries ask about one fixed set of intervals. A scan of all intervals for each query costs $O(nm)$. With $n = m = 10^5$, that is 10^{10} checks. If all queries are known at the start, answer them in sorted order, and sweep a point from left to right. An interval becomes active when the sweep passes its start. It is dead forever when the sweep passes its end. Keep the active intervals in a min-heap, keyed by the value that you optimize. Memory hook

(Minimum Interval to Include Each Query): *Sweep sorted queries, push started intervals by size, pop ended tops, read the top.*

```python
import heapq
def min_interval(intervals, queries):   # closed [l, r]; the size is r - l + 1
    intervals = sorted(intervals)
    order = sorted(range(len(queries)), key=lambda j: queries[j])   # indexes, not values
    ans = [-1] * len(queries)
    heap, i = [], 0                     # heap of (size, right); i is never reset
    for j in order:
        q = queries[j]
        while i < len(intervals) and intervals[i][0] <= q:   # started: push it
            left, right = intervals[i]
            heapq.heappush(heap, (right - left + 1, right))
            i += 1
        while heap and heap[0][1] < q:  # the top ended before q: drop it for good
            heapq.heappop(heap)
        if heap:
            ans[j] = heap[0][0]         # write at the original index
    return ans
min_interval([[1, 4], [2, 4], [3, 6], [4, 4]], [2, 3, 4, 5])      # -> [3, 3, 1, 4]
min_interval([[2, 3], [2, 5], [1, 8], [20, 25]], [2, 19, 5, 22])  # -> [2, -1, 4, 6]
```

The trick is lazy deletion. An interval that ended stays in the heap until it reaches the top. This is correct because the code reads only the top, and an ended interval never becomes valid again when the queries grow. Use this rule for any heap whose items expire in the order of the sweep. The same pointer-into-heap frame schedules work as time moves (Single-Threaded CPU, IPO).

NOTE

To count the intervals that contain each point, no heap is necessary. The count is the number of starts at or before q minus the number of ends before q. Two binary searches give both numbers.

```python
from bisect import bisect_left, bisect_right
def count_containing(intervals, queries):   # closed intervals
    starts = sorted(s for s, _ in intervals)
    ends = sorted(e for _, e in intervals)
    return [bisect_right(starts, q) - bisect_left(ends, q) for q in queries]
count_containing([[1, 6], [3, 7], [9, 12], [4, 13]], [2, 3, 7, 11])   # -> [1, 2, 2, 2]
```

63.4.7 Endpoint rules: closed and half-open

Each problem fixes one rule for a shared endpoint. Read the rule from the statement or from an example. Then let it decide every comparison in the code. Memory hook: *Closed ends touch, half-open ends do not. Decide once, then derive every* `<` *or* `<=`*.*

Problem	Touching intervals	Test in the code
Merge Intervals	share a point: they merge	extend when `s <= out[-1][1]`
Insert Interval	share a point: they merge	left when `e < start`, right when `s > end`
Non-overlapping Intervals	do not overlap	keep when `s >= prev_end`
Meeting Rooms	do not conflict	conflict when `s < prev_end`
Meeting Rooms II	the room is free (half-open)	`(t, -1)` sorts before `(t, 1)`; reuse when `ends[0] <= s`
Minimum Interval to Include Each Query	a query on an end is inside (closed)	drop the top when `right < q`

To switch a sweep from half-open to closed intervals, process starts before ends at equal times. On integer coordinates, you can also keep ends first and free the room at `e + 1`. This works because `[s, e]` holds the same integers as `[s, e + 1)`.

```
def peak(intervals, closed):
    events = []
    for s, e in intervals:
        events.append((s, 1))
        events.append((e + 1 if closed else e, -1))   # closed: the end time is still busy
    events.sort()
    busy = most = 0
    for _, change in events:
        busy += change
        most = max(most, busy)
    return most
peak([[1, 5], [5, 10]], closed=False), peak([[1, 5], [5, 10]], closed=True)   # -> (1, 2)
```

63.5 Adapt it to a new problem

Ask these questions in this order.

1. What is one interval? Some problems hide them: a trip from stop a to stop b, the first and last index of a value, a subarray with a given sum.
2. Do touching intervals overlap? Write the overlap test before the loop. Check it on the example that has a shared endpoint.
3. Is the input already sorted and disjoint? Then do not sort again. For one new interval or one removed range, use three buckets. For two such lists, use two pointers.
4. Must every interval be processed, or do you choose a subset? Process every interval (merge, conflicts, peak): sort by start, or sort events. Choose the most, or stab with the fewest points: sort by end.
5. What is the smallest state that a later interval needs? The last group, the last kept end, a running count, or a heap of active intervals. Only this state may go into the loop.
6. Are there ties in the sort order? Decide which one goes first: an end or a start at the same time, a longer or a shorter interval at the same start.
7. Are there queries? If all of them are known at the start, sort their indexes and sweep once. If they arrive one at a time, keep a sorted list and use `bisect`.
8. Do the intervals carry different values? Then a greedy on the ends fails. Use a DP with binary search.
9. Is the coordinate range small? A difference array or a marked line can replace the sort.

Test with touching intervals, nested intervals, duplicates, point intervals `[x, x]`, one interval, an empty list, and unsorted input. For a merge, a slow oracle marks every covered point on a doubled line. Doubling keeps a gap between `[1, 2]` and `[3, 4]`: the cells are 2 to 4 and 6 to 8, and cell 5 stays empty.

```
import random
def merge(intervals):
    out = []
    for s, e in sorted(intervals):
        if out and s <= out[-1][1]:
            out[-1][1] = max(out[-1][1], e)
        else:
            out.append([s, e])
    return out
def merge_brute(intervals):             # closed integer intervals; a test oracle only
    cells = sorted({x for s, e in intervals for x in range(2 * s, 2 * e + 1)})
    runs = []
    for x in cells:
        if runs and x == runs[-1][1] + 1:
            runs[-1][1] = x
        else:
            runs.append([x, x])
    return [[a // 2, b // 2] for a, b in runs]
merge_brute([[1, 2], [3, 4]]), merge_brute([[1, 2], [2, 4]])   # -> ([[1, 2], [3, 4]], [[1, 4]])
random.seed(7)
def random_case():
    return [sorted([random.randint(0, 9), random.randint(0, 9)]) for _ in range(random.randint(1, 5))]
```

```
all(merge(c) == merge_brute(c) for c in (random_case() for _ in range(500)))   # -> True
```

63.6 Pitfalls

CAUTION

The wrong strictness at a shared endpoint. Wrong: copy `<` or `<=` from another interval problem. Right: read the rule for touching intervals in the statement, and use the table of endpoint rules. Merge Intervals merges `[1, 4]` and `[4, 5]`. Meeting Rooms lets `[1, 5]` and `[5, 10]` both happen.

CAUTION

Compare with the open group, and take the max of the ends. Wrong: compare each interval with the previous input interval, or set the new end without `max`. A long interval can cover several later intervals, and a nested interval can shrink the group. Right: compare with `out[-1]`, and extend with `max(out[-1][1], e)`.

```
def merge_wrong(intervals):
    iv = sorted(intervals)
    out = [list(iv[0])]
    for k in range(1, len(iv)):
        if iv[k][0] <= iv[k - 1][1]:        # the previous input interval, not the group
            out[-1][1] = iv[k][1]           # no max
        else:
            out.append(list(iv[k]))
    return out
def merge(intervals):
    out = []
    for s, e in sorted(intervals):
        if out and s <= out[-1][1]:
            out[-1][1] = max(out[-1][1], e)
        else:
            out.append([s, e])
    return out
merge_wrong([[1, 10], [2, 3], [4, 5]])      # -> [[1, 3], [4, 5]]
merge([[1, 10], [2, 3], [4, 5]])            # -> [[1, 10]]
```

CAUTION

Rows of the input in the output. Wrong: append the input row itself to the output, and then change its end. The caller's list changes too. Also, `intervals.sort()` changes the order of the caller's list. Right: append a new list `[s, e]`. Use `sorted(intervals)` when the caller needs the input unchanged.

```
def merge_alias(intervals):
    out = []
    for row in sorted(intervals):
        if out and row[0] <= out[-1][1]:
            out[-1][1] = max(out[-1][1], row[1])
        else:
            out.append(row)                 # the input row itself
    return out
data = [[1, 3], [2, 6]]
merge_alias(data)                           # -> [[1, 6]]
data                                        # -> [[1, 6], [2, 6]]
```

CAUTION

Sort by start for a selection problem. Wrong: sort by start and keep the current interval when the next one overlaps it. A long early interval then blocks many short ones. Right: sort by end and keep what fits. A sort by start works only if, on each overlap, you keep the interval with the smaller end: `prev_end = min(prev_end, e)`. Also count removals, not overlapping pairs. One long interval can overlap many short ones, but it needs one removal.

```
def min_removals_wrong(intervals):
    removed, prev_end = 0, float("-inf")
    for s, e in sorted(intervals):          # by start
        if s < prev_end:
            removed += 1                    # always drops the new interval
        else:
            prev_end = e
    return removed
def min_removals(intervals):
    kept, prev_end = 0, float("-inf")
    for s, e in sorted(intervals, key=lambda iv: iv[1]):
        if s >= prev_end:
            kept += 1
            prev_end = e
    return len(intervals) - kept
min_removals_wrong([[1, 100], [2, 3], [4, 5]])   # -> 2
min_removals([[1, 100], [2, 3], [4, 5]])         # -> 1
```

CAUTION

The tie order of events, and the size of the heap. Wrong: sort events so that a start comes before an end at the same time, for half-open meetings. Wrong: pop every free room from the heap, and then return the final heap size. Right: in Python, `(t, -1)` sorts before `(t, 1)`, so ends come first with no extra code. Pop at most one room for each meeting. If you pop all free rooms, track the largest heap size.

```
import heapq
def rooms_starts_first(intervals):
    events = sorted([(s, 1) for s, e in intervals] + [(e, -1) for s, e in intervals],
                    key=lambda ev: (ev[0], -ev[1]))   # +1 before -1 at equal times
    busy = most = 0
    for _, change in events:
        busy += change
        most = max(most, busy)
    return most
def rooms_pop_all(intervals):
    ends = []
    for s, e in sorted(intervals):
        while ends and ends[0] <= s:        # frees every finished room
            heapq.heappop(ends)
        heapq.heappush(ends, e)
    return len(ends)                        # the size now can be below the peak
rooms_starts_first([[1, 5], [5, 10]])       # -> 2
rooms_pop_all([[1, 2], [1, 2], [5, 6]])     # -> 1
```

The correct answers are 1 and 2. In the two-list form (all starts sorted, all ends sorted), do not pair `starts[i]` with `ends[i]`. The sort breaks the pairs on purpose. Only the counts matter.

CAUTION

Offline queries. Wrong: sort the query values and return the answers in sorted order, or reset the interval pointer for each query. Right: sort a list of query indexes, write each answer at its original index, and keep one pointer for all queries. A reset pointer makes the cost $O(nm)$. Also check the size formula `r - l + 1` for closed integer intervals, and the `-1` for a query that no interval contains.

CAUTION

Work after the loop. Wrong: keep the open group in two variables and append it only when the next group starts. The last group then never gets into the output. Right: keep the open group as `out[-1]`, as in the template. In Insert Interval, append the merged interval exactly once, also when it goes at the very end.

63.7 Problems in this family

Problem	What changes	Memory hook
Merge Intervals	Unsorted input; touching intervals merge. Sort by start and extend the open group.	Sort, then extend the last one or append a new one.
Insert Interval	The input is sorted and disjoint. No sort: three buckets in one $O(n)$ pass.	Left stays left, right stays right, the middle melts into one.
Non-overlapping Intervals	Choose a subset: minimum removals equals n minus the most kept. Sort by end.	Sort by end, keep what fits, remove the rest.
Meeting Rooms	A yes or no conflict check. Sort by start and compare neighbors; touching is fine.	Sort by start. No meeting may start before the previous one ends.
Meeting Rooms II	The peak number of open intervals. Event sweep with ends before starts at a tie; or a heap of end times.	Start is +1, end is -1, sort the events, and keep the peak.
Minimum Interval to Include Each Query	Offline point queries. Sort the queries, push started intervals into a heap by size, drop ended tops lazily.	Sweep sorted queries, push started intervals by size, pop ended tops, read the top.

The transfer drills for this technique are in Section 69.21 (page 426). The answers are in Section A.20 (page 463).

64 Matrices and simulation

64.1 The idea

Some problems do not hide a clever algorithm. The statement gives exact rules, and the code must follow them. This family has two kinds of problems. In a grid transform or a grid traversal, each cell goes to a position that depends only on its coordinates: rotate, flip, spiral order. In a simulation, a state changes step by step under fixed rules: a cellular automaton, a robot program, rounds of a process. Speed is rarely the risk. The risk is a wrong index, a wrong order of writes, or a missed rule.

A direct simulation is often the full answer. The technique gives five ways to make it correct, and sometimes faster:

- **Turn moves and groups into numbers.** Store the four directions as a list of `(dr, dc)` pairs. A turn is then `d + 1` or `d - 1`, and one line of code moves in any direction. Compute the row, column or box of a cell with arithmetic on `(r, c)`. This removes the four-way `if` chains, where index bugs often hide.
- **Map one cell first.** Write where one cell goes. Then build that map from simple moves: transpose, reverse each row, reverse the row order.
- **Keep reads and writes apart.** Reads see the old state. Writes make the new state. Use a second grid, two passes, or a spare bit in the same cell.
- **Shrink the work.** When all items move the same way, track one shared offset, not many items. When a process repeats a state, skip the repeated cycles.
- **Reuse work between candidates.** When you try every single edit, the part before the edit is the same for all candidates. Compute it once, and do not replay it for each candidate.

Worked example (map one cell). Rotate the grid `[[1,2,3],[4,5,6],[7,8,9]]` by 90 degrees clockwise. The value 1 at $(0, 0)$ must go to the top-right corner $(0, 2)$. The value 7 at $(2, 0)$ must go to the top-left corner $(0, 0)$. So the map is $(i, j) \to (j, n - 1 - i)$. A transpose sends (i, j) to (j, i). A reverse of each row then sends (j, i) to $(j, n - 1 - i)$. The two moves together give the map. Check the value 7: the transpose moves it to $(0, 2)$, and the row reverse moves it to $(0, 0)$. The result is `[[7,4,1],[8,5,2],[9,6,3]]`. No second grid is necessary, because both moves are pair swaps.

Worked example (reads and writes apart). In Game of Life, a dead cell with exactly three live neighbors becomes live. Scan the grid row by row and write each new value at once. A cell in row 1 then counts its neighbors in row 0, but row 0 already holds the next generation. The count mixes two generations, and the result is wrong. The fix is a rule for the whole family: every read must see the old state.

Worked example (shrink the work). Every leaf of the garden `[[1,2,3],[4,5,6]]` moves with the gusts `"RL"`, and a leaf that leaves the grid is lost. The brute force moves every leaf for every gust. That is $G \cdot R \cdot C$ work, which is too slow for 10^5 gusts on a 300×300 grid. Instead, track one column offset for all leaves: 0, then 1 after R, then 0 after L. The net offset is 0, but the maximum offset is 1. So column 2 left the grid at the first gust. The survivors are columns 0 and 1, and the answer is $1 + 2 + 4 + 5 = 12$. The work drops to $O(G + R \cdot C)$.

64.2 Recognize it

- The task asks for a fixed visiting order over a grid: spiral, ring by ring, diagonal, zigzag.
- The statement says rotate, flip, mirror, reflect or transpose.
- The statement says "in place", "simultaneously", "all at once" or "use only the original values". Or the function returns nothing and must change its input. Reads and writes must stay apart.
- Each new cell value depends on a fixed neighborhood: 4 neighbors, 8 neighbors, or a 3×3 window.
- The statement gives an exact process: a scan order, a priority between choices, rounds, flags that reset. A small n with long rules means: simulate the rules literally.
- A robot, a cursor or a pointer follows commands such as F, B, L, R or U, D, L, R. The state is a position, plus a facing when some commands turn.
- Every item gets the same move at the same time. One shared offset describes all items. The number of steps times the grid size is far above 10^7, so a move of every item per step is too slow.
- The number of steps is huge (for example 10^9), but the number of different states is small. The process must repeat a cycle.
- The statement says "exactly one element is wrong" or "change exactly one position". Split each candidate into a prefix, the edit and a suffix. With n candidates and an $O(n)$ replay, the baseline is $O(n^2)$. It fits $n \leq 1000$.
- Each item belongs to several groups of a grid: a row, a column, a box, a diagonal. Compute the group ids from the coordinates.

64.3 Core template

Use one set of helpers for every grid problem. Rows grow downward, so "up" is `dr = -1`. Keep the four directions in clockwise order. Then a right turn is `d + 1` and a left turn is `d - 1`, both modulo 4.

```
DIRS = [(-1, 0), (0, 1), (1, 0), (0, -1)]   # up, right, down, left: clockwise
d = 0                                        # facing up
(d + 1) % 4, (d - 1) % 4, (d + 2) % 4        # -> (1, 3, 2)
DIRS8 = [(dr, dc) for dr in (-1, 0, 1) for dc in (-1, 0, 1) if (dr, dc) != (0, 0)]
len(DIRS8)                                   # -> 8

grid = [[1, 2, 3],
        [4, 5, 6]]
R, C = len(grid), len(grid[0])

def neighbors(r, c, offsets=DIRS):
    for dr, dc in offsets:
        nr, nc = r + dr, c + dc
        if 0 <= nr < R and 0 <= nc < C:      # check BEFORE you index
            yield nr, nc

sorted(neighbors(0, 0))                      # -> [(0, 1), (1, 0)]
sorted(grid[r][c] for r, c in neighbors(0, 1, DIRS8))   # -> [1, 3, 4, 5, 6]
grid[-1][0]                                  # -> 4
```

The last line shows the main trap. A negative index does not raise an error. It silently reads the last row.

The second template is one synchronous step. All cells change "at the same time". Read only the old grid and write only into a new grid.

```
def step(grid, rule):
    R, C = len(grid), len(grid[0])
    offsets = [(dr, dc) for dr in (-1, 0, 1) for dc in (-1, 0, 1) if (dr, dc) != (0, 0)]
    new = [[0] * C for _ in range(R)]              # one separate list per row
    for r in range(R):
```

```
        for c in range(C):
            around = [grid[r + dr][c + dc] for dr, dc in offsets
                      if 0 <= r + dr < R and 0 <= c + dc < C]
            new[r][c] = rule(grid[r][c], around)    # reads the old grid only
    return new

def life(cell, around):
    live = sum(around)
    return 1 if live == 3 or (live == 2 and cell) else 0

g = [[0, 1, 0],
     [0, 1, 0],
     [0, 1, 0]]
step(g, life)              # -> [[0, 0, 0], [1, 1, 1], [0, 0, 0]]
step(step(g, life), life)  # -> [[0, 1, 0], [0, 1, 0], [0, 1, 0]]
g                          # -> [[0, 1, 0], [0, 1, 0], [0, 1, 0]]
```

Invariant. During the step, `grid` never changes and `new` is never read. So every cell sees the same old generation, in any scan order. Change only `rule` and `offsets` to get another automaton or another stencil.

64.4 Variations

64.4.1 Walk with a direction and a turn rule

The path is a sequence of straight segments. Keep the position `(r, c)` and a direction index `d`. At each step, look at the next cell. If it is outside the grid or already visited, turn and look again. This is the intuitive form of Spiral Matrix. Memory hook: *walk straight, turn right when blocked.*

```
def spiral_order(matrix):
    rows, cols = len(matrix), len(matrix[0])
    dirs = [(0, 1), (1, 0), (0, -1), (-1, 0)]       # right, down, left, up
    visited = set()
    out = []
    r = c = d = 0
    for _ in range(rows * cols):                    # exactly one step per cell
        out.append(matrix[r][c])
        visited.add((r, c))
        nr, nc = r + dirs[d][0], c + dirs[d][1]
        if not (0 <= nr < rows and 0 <= nc < cols) or (nr, nc) in visited:
            d = (d + 1) % 4                         # blocked: turn right
            nr, nc = r + dirs[d][0], c + dirs[d][1]
        r, c = nr, nc
    return out

spiral_order([[1, 2, 3, 4], [5, 6, 7, 8], [9, 10, 11, 12]])
# -> [1, 2, 3, 4, 8, 12, 11, 10, 9, 5, 6, 7]
spiral_order([[1], [2], [3]])       # -> [1, 2, 3]
spiral_order([[1, 2, 3]])           # -> [1, 2, 3]
```

The same state `(r, c, d)` drives every robot problem with a facing. A command F adds `DIRS[d]`, a command B subtracts it, and L and R change only `d`. For an outward spiral, use segment lengths 1, 1, 2, 2, 3, 3 and record only the cells inside the grid. (Spiral Matrix; a grid robot program where exactly one command must change.)

64.4.2 Peel layers with shrinking bounds

The unvisited cells of a spiral always form one rectangle. Keep its four bounds `top`, `bottom`, `left`, `right`, all inclusive. Each straight pass reads one full edge of the rectangle. Then that bound moves inward by one. This is the optimal form of Spiral Matrix: $O(1)$ extra space and no visited set. Use it when constant extra space is required, or when the work goes ring by ring. Memory hook: *read an edge, move that wall in; check again before the third and the fourth pass.*

```
def spiral_bounds(matrix):
    top, bottom = 0, len(matrix) - 1
    left, right = 0, len(matrix[0]) - 1
    out = []
    while top <= bottom and left <= right:
        for c in range(left, right + 1):            # top row, left to right
            out.append(matrix[top][c])
        top += 1
        for r in range(top, bottom + 1):            # right column, downward
            out.append(matrix[r][right])
        right -= 1
        if top <= bottom:                           # a row is still left
            for c in range(right, left - 1, -1):    # bottom row, right to left
                out.append(matrix[bottom][c])
            bottom -= 1
        if left <= right:                           # a column is still left
            for r in range(bottom, top - 1, -1):    # left column, upward
                out.append(matrix[r][left])
            left += 1
    return out

spiral_bounds([[1, 2, 3, 4], [5, 6, 7, 8], [9, 10, 11, 12]])
# -> [1, 2, 3, 4, 8, 12, 11, 10, 9, 5, 6, 7]
spiral_bounds([[1, 2], [3, 4], [5, 6]])   # -> [1, 2, 4, 6, 5, 3]
spiral_bounds([[1, 2, 3]])                # -> [1, 2, 3]
```

Invariant. At the top of the loop, the unvisited cells are exactly rows `top..bottom` and columns `left..right`. The first two passes can empty the rectangle in the middle of one iteration. So check the bounds again before the third and the fourth pass. General rule: when one loop iteration does several steps and each step shrinks the state, test the loop condition again before each later step. The same bounds give ring-by-ring work. A grid has `(min(R, C) + 1) // 2` rings. Ring `k` has `top = left = k`, `bottom = R - 1 - k` and `right = C - 1 - k`. (Spiral Matrix; the 4-cycle form of Rotate Image; fill a grid in spiral order.)

64.4.3 Rotate and flip: compose reflections

Write the map for one cell, then build it from reflections. Each reflection is a set of pair swaps, so it works in place on a square grid. A rotation is two reflections. This is Rotate Image. Memory hook: *flip on the diagonal, then flip each row.*

```
def rotate(matrix):                     # in place, 90 degrees clockwise, square only
    n = len(matrix)
    for i in range(n):
        for j in range(i + 1, n):       # upper triangle: each pair swaps once
            matrix[i][j], matrix[j][i] = matrix[j][i], matrix[i][j]
    for row in matrix:
        row.reverse()

m = [[1, 2, 3], [4, 5, 6], [7, 8, 9]]
alias = m
rotate(m)
alias                                   # -> [[7, 4, 1], [8, 5, 2], [9, 6, 3]]
```

When a new grid is allowed, or the grid is not square, build the result with slices and `zip`.

```
g = [[1, 2, 3],
     [4, 5, 6]]
[list(r) for r in zip(*g[::-1])]        # -> [[4, 1], [5, 2], [6, 3]]
[list(r) for r in zip(*g)][::-1]        # -> [[3, 6], [2, 5], [1, 4]]
[row[::-1] for row in g[::-1]]          # -> [[6, 5, 4], [3, 2, 1]]
[row[::-1] for row in g]                # -> [[3, 2, 1], [6, 5, 4]]
g[::-1]                                 # -> [[4, 5, 6], [1, 2, 3]]
```

```
[list(c) for c in zip(*g)]              # -> [[1, 4], [2, 5], [3, 6]]
sq = [[1, 2], [3, 4]]
sq[:] = [list(r) for r in zip(*sq[::-1])]   # slice assignment keeps the same object
sq                                      # -> [[3, 1], [4, 2]]
```

Transform	Cell map $(n \times n)$	In place (square grid)	New grid (any shape)
Clockwise 90	$(i, j) \to (j, n-1-i)$	transpose, then reverse each row	`zip(*g[::-1])`
Counterclockwise 90	$(i, j) \to (n-1-j, i)$	transpose, then reverse the row order	`list(zip(*g))[::-1]`
180	$(i, j) \to (n-1-i, n-1-j)$	reverse the row order and each row	`[r[::-1] for r in g[::-1]]`
Left-right mirror	$(i, j) \to (i, n-1-j)$	reverse each row	`[r[::-1] for r in g]`
Up-down mirror	$(i, j) \to (n-1-i, j)$	reverse the row order	`g[::-1]`
Transpose	$(i, j) \to (j, i)$	swap only for $j > i$	`zip(*g)`

In 1D, a rotation by k is three reversals. This is the same idea: reflections composed into a rotation. When a map is not a reflection, split it into cycles and move each cycle with one temporary value. Rotate Image can also move each group of four matching cells of a ring as one 4-cycle. (Rotate Image; the invert step of an image task that either mirrors or blurs a grid.)

64.4.4 Stencils: a fixed neighborhood into a new grid

Each output cell is a function of a small window of the input. Loop over the cells, loop over a constant list of offsets, and skip the offsets outside the grid. Count the neighbors that you keep. Border cells then need no special code. This is the blur step of an image task that either mirrors or blurs a grid. Memory hook: *invert reverses each row; blur walks 8 offsets, skips out-of-bounds cells, and writes sum // count into a new grid.*

```
def transform_image(image, operation):
    if operation == "invert":                       # left-right mirror, a new list per row
        return [row[::-1] for row in image]
    rows, cols = len(image), len(image[0])
    offsets = [(dr, dc) for dr in (-1, 0, 1) for dc in (-1, 0, 1) if (dr, dc) != (0, 0)]
    result = [[0] * cols for _ in range(rows)]
    for r in range(rows):
        for c in range(cols):
            total = count = 0
            for dr, dc in offsets:
                nr, nc = r + dr, c + dc
                if 0 <= nr < rows and 0 <= nc < cols:
                    total += image[nr][nc]
                    count += 1
            result[r][c] = total // count if count else image[r][c]
    return result

img = [[1, 2, 3], [4, 5, 6]]
transform_image(img, "invert")          # -> [[3, 2, 1], [6, 5, 4]]
transform_image(img, "blur")            # -> [[3, 3, 4], [2, 3, 3]]
transform_image([[200]], "blur")        # -> [[200]]
img                                     # -> [[1, 2, 3], [4, 5, 6]]
```

Divide by `count`, not by 8. In a grid with at least two rows and two columns, a corner has 3 neighbors and an edge cell has 5. In a single row, a cell has only 1 or 2 neighbors. To include the center, add the offset `(0, 0)`. For a large $k \times k$ window, use 2D prefix sums so that the cost does not depend on k. (An image task that either mirrors or blurs a grid; image smoothing; neighbor counts in Game of Life.)

64.4.5 Mark first, then apply; borrow a row and a column

A write at one cell can change what a later read sees. A new 0 looks like an original 0 and spreads. Split the work into two passes. Pass 1 only reads and records. Pass 2 only writes. This is Set Matrix Zeroes. Memory hook: *mark zero rows and zero columns first, then clear.*

```
def set_zeroes(matrix):
    m, n = len(matrix), len(matrix[0])
    zero_rows, zero_cols = set(), set()
    for i in range(m):                      # pass 1: read only
        for j in range(n):
            if matrix[i][j] == 0:
                zero_rows.add(i)
                zero_cols.add(j)
    for i in range(m):                      # pass 2: write only
        for j in range(n):
            if i in zero_rows or j in zero_cols:
                matrix[i][j] = 0

g = [[1, 1, 1], [1, 0, 1], [1, 1, 1]]
set_zeroes(g)
g                                           # -> [[1, 0, 1], [0, 0, 0], [1, 0, 1]]
```

The two sets cost $O(m+n)$ extra space. For $O(1)$ extra space, store the column marks in row 0 and the row marks in column 0. This is safe: a marker cell gets the value 0 only when it must become 0 anyway. Save the original state of row 0 and column 0 in two booleans first, and clear them last.

```
def set_zeroes_o1(matrix):
    m, n = len(matrix), len(matrix[0])
    row0 = 0 in matrix[0]                               # save the borrowed row
    col0 = any(matrix[i][0] == 0 for i in range(m))     # save the borrowed column
    for i in range(1, m):
        for j in range(1, n):
            if matrix[i][j] == 0:
                matrix[i][0] = 0                        # mark row i
                matrix[0][j] = 0                        # mark column j
    for i in range(1, m):                               # inner block first
        for j in range(1, n):
            if matrix[i][0] == 0 or matrix[0][j] == 0:
                matrix[i][j] = 0
    if row0:                                            # borders last
        for j in range(n):
            matrix[0][j] = 0
    if col0:
        for i in range(m):
            matrix[i][0] = 0

g = [[0, 1, 2, 0], [3, 4, 5, 2], [1, 3, 1, 5]]
set_zeroes_o1(g)
g                           # -> [[0, 0, 0, 0], [0, 4, 5, 0], [0, 3, 1, 0]]
h = [[1, 2], [0, 4]]
set_zeroes_o1(h)
h                           # -> [[0, 2], [0, 0]]
```

The general idea: if a region of the input gets a known final value, use it as scratch storage before that. Other places to borrow are the sign of a positive number and values outside the legal range. (Set Matrix Zeroes; mark seen values by negation in an array.)

64.4.6 Synchronous update in place: pack the old and the new state

All cells must change at the same time, and no copy is allowed. Store the new state next to the old state in the same cell. Every read decodes the old state. A final pass promotes the new state. This is Game of Life. Memory

hook: *bit 0 is now, bit 1 is next: read `& 1`, write `|= 2`, then shift `>>= 1`.*

```
def game_of_life(board):
    m, n = len(board), len(board[0])
    for r in range(m):
        for c in range(n):
            alive = board[r][c] & 1                 # bit 0 = state now
            live = 0
            for dr in (-1, 0, 1):
                for dc in (-1, 0, 1):
                    if dr == 0 and dc == 0:
                        continue
                    nr, nc = r + dr, c + dc
                    if 0 <= nr < m and 0 <= nc < n:
                        live += board[nr][nc] & 1   # read bit 0 only
            if live == 3 or (live == 2 and alive):
                board[r][c] |= 2                    # bit 1 = state next
    for r in range(m):
        for c in range(n):
            board[r][c] >>= 1                       # promote next to now

b = [[0, 1, 0], [0, 0, 1], [1, 1, 1], [0, 0, 0]]
game_of_life(b)
b                       # -> [[0, 0, 0], [1, 0, 1], [0, 1, 1], [0, 1, 0]]
b2 = [[1, 1], [1, 0]]
game_of_life(b2)
b2                      # -> [[1, 1], [1, 1]]
```

The general form: if all values lie in 0 to $k-1$, a slot can hold `old + k * new`. Read the old value with `v % k` and the new value with `v // k`.

```
a = [2, 0, 1]                   # build a[a[i]] in place; all values are below n
n = len(a)
for i in range(n):
    a[i] += (a[a[i]] % n) * n   # old value stays in a[x] % n
for i in range(n):
    a[i] //= n
a                               # -> [1, 2, 0]
```

When the board is infinite or sparse, store only the live cells in a set and count neighbor hits in a `Counter`. (Game of Life; build an array from a permutation in place.)

64.4.7 Group ids from coordinates

Each cell belongs to several groups at the same time. Give each group its own seen-set. Map a cell to its group ids with arithmetic on `(r, c)`. This is Valid Sudoku. Memory hook: *each digit checks in at its row, its column and its box (r // 3, c // 3); a second check-in at any of the three fails.*

```
from collections import defaultdict

def is_valid_sudoku(board):
    rows = defaultdict(set)
    cols = defaultdict(set)
    boxes = defaultdict(set)            # (r // 3, c // 3) -> digits in that box
    for r in range(9):
        for c in range(9):
            digit = board[r][c]
            if digit == ".":
                continue
            box = (r // 3, c // 3)
            if digit in rows[r] or digit in cols[c] or digit in boxes[box]:
                return False
            rows[r].add(digit)
```

```
            cols[c].add(digit)
            boxes[box].add(digit)
    return True

board = [["."] * 9 for _ in range(9)]
board[0][0] = "5"
board[4][4] = "5"
is_valid_sudoku(board)          # -> True
board[2][1] = "5"               # same box as (0, 0)
is_valid_sudoku(board)          # -> False
r, c = 4, 7
(r // 3, c // 3), (r // 3) * 3 + c // 3     # -> ((1, 2), 5)
r - c, r + c                                # -> (-3, 11)
```

The last two lines show other group ids. The flat box id `(r // 3) * 3 + c // 3` indexes a list of 9 sets. On one diagonal `r - c` is constant, and on one anti-diagonal `r + c` is constant. For $h \times w$ blocks in a grid of width W, the flat id is `(r // h) * (W // w) + c // w` when w divides W. (Valid Sudoku; queens on diagonals.)

64.4.8 Rule-based processes: delete during the scan

The statement gives a scan order, a priority between choices, and rounds that repeat until nothing changes. Simulate the rules literally. Put the effort into the index bookkeeping, not into speed. Store each per-round flag inside the item, so that it moves with the item when a list delete shifts indexes. This is the shape of a line of organisms where each one, in turn, eats a smaller neighbor. Memory hook: *walk the line and delete what gets eaten; do not step after eating left.*

```
def largest_organism(families, sizes):
    line = [{"family": f, "size": s, "ate": False} for f, s in zip(families, sizes)]

    def can_eat(eater, prey):
        return not prey["ate"] and prey["size"] < eater["size"]

    changed = True
    while changed:                          # one pass = one round; stop at a fixed point
        changed = False
        for org in line:
            org["ate"] = False
        i = 0
        while i < len(line):
            me = line[i]
            if i > 0 and can_eat(me, line[i - 1]):              # rule 1: eat left
                me["size"] += line[i - 1]["size"]
                me["ate"] = changed = True
                del line[i - 1]             # me slides to i - 1; line[i] is the next organism
            elif i + 1 < len(line) and can_eat(me, line[i + 1]):  # rule 2: eat right
                me["size"] += line[i + 1]["size"]
                me["ate"] = changed = True
                del line[i + 1]
                i += 1
            else:                                                 # rule 3: nothing
                i += 1
    best = max(line, key=lambda org: org["size"])   # max keeps the leftmost tie
    return f"{best['family']} {best['size']}"

largest_organism(["A", "B", "C"], [3, 1, 2])    # -> 'A 6'
largest_organism(["A", "B"], [5, 5])            # -> 'A 5'
largest_organism(["X", "Y", "Z"], [1, 2, 3])    # -> 'Y 3'
```

Three rules keep such a loop correct. After a delete on the left, do not advance the index. After a delete on the right, skip the deleted item. Bound the number of rounds with a count that strictly decreases. Here each productive round removes an item. The optimal form keeps the survivors of each round on a stack: the stack top is the left

neighbor, so eating left is a pop. This is the same stack as in asteroid collision. For $n \leq 20$, both forms are fast. (A line of organisms that eat smaller neighbors in rounds.)

64.4.9 Many items, one move: track a shared offset

When every item gets the same move, all items share one displacement. Do not move the items. Track the offset per axis. When crossing a border removes an item for good, the whole path matters, not only the end. An item survives exactly when its start plus every prefix offset stays inside. That depends only on the running minimum and maximum of the offset, so the survivors form one rectangle. This is the shape of a garden grid where each gust moves all leaves by one cell. Memory hook: *move the frame, not the leaves: track the offset's min and max, then sum the surviving box.*

```
def remaining_leaves(garden, gusts):
    rows, cols = len(garden), len(garden[0])
    step = {"U": (-1, 0), "D": (1, 0), "L": (0, -1), "R": (0, 1)}
    dr = dc = 0
    min_dr = max_dr = min_dc = max_dc = 0       # the empty prefix counts
    for g in gusts:
        sr, sc = step[g]
        dr += sr
        dc += sc
        min_dr, max_dr = min(min_dr, dr), max(max_dr, dr)
        min_dc, max_dc = min(min_dc, dc), max(max_dc, dc)
    top, bottom = -min_dr, rows - 1 - max_dr
    left, right = -min_dc, cols - 1 - max_dc
    if top > bottom or left > right:            # guard: a negative slice end counts from the end
        return 0
    return sum(sum(row[left:right + 1]) for row in garden[top:bottom + 1])

garden = [[1, 2, 3], [4, 5, 6]]
remaining_leaves(garden, "R")         # -> 12
remaining_leaves(garden, "RL")        # -> 12
remaining_leaves(garden, "")          # -> 21
remaining_leaves([[1, 2, 3]], "LLRRR")  # -> 0
```

The cost is $O(G + R \cdot C)$ for G gusts, not $O(G \cdot R \cdot C)$. With wrap-around edges, nothing is lost, and the final grid is a cyclic shift. (Leaves on a grid that all move with each gust; a robot or a cursor that must stay inside a board.)

64.4.10 Very many steps: skip the cycle

A deterministic step on a finite state space must repeat a state. Store the first time of each state in a dict. At the first repeat, the period is known. Skip all full cycles and simulate only the rest. The state must be hashable: use a tuple or a string, not a list. Memory hook: *remember when each state first appeared; at a repeat, skip whole periods.*

```
def run_steps(state, step, k):
    first_seen = {}
    t = 0
    while t < k:
        if state in first_seen:
            period = t - first_seen[state]
            for _ in range((k - t) % period):   # the rest after the full cycles
                state = step(state)
            return state
        first_seen[state] = t
        state = step(state)
        t += 1
    return state

def brute(state, step, k):
    for _ in range(k):
        state = step(state)
```

```
    return state

f = lambda x: (x * x + 3) % 17
all(run_steps(s, f, k) == brute(s, f, k) for s in range(17) for k in range(60))  # -> True
run_steps((1, 2, 3), lambda t: t[1:] + t[:1], 10**9 + 1)   # -> (3, 1, 2)
```

The loop does at most about two steps per different state, not k steps. The first states can lie before the cycle. So measure the period from the first time of the repeated state, not from step 0. (Automata that run for 10^9 days; Game of Life for many generations.)

64.4.11 Exactly one wrong step: try every single edit with prefix states

Every candidate is the same as the original before the edit. So run the original forward once and keep the state before each index. For each index and each replacement, start from that stored state and ask one question: can the rest still succeed from here?

```
state = start                          # prefix state before index i
for i, old in enumerate(seq):
    for new in choices:                # try every edit at index i
        s = apply(state, new)
        if new != old and valid(s) and finishes(s, i + 1):   # suffix question
            return True
    state = apply(state, old)          # the original prefix, one step further
    if not valid(state):
        return False                   # an edit later cannot fix this step
return False
```

The baseline answers `finishes` with a replay of the suffix. That is $O(n^2)$ in total, and it fits $n \leq 1000$. When every step can be undone, there is a faster answer. Undo the program backward from the goal. Each goal state has at most one predecessor at each index, so the set of states that can still finish never grows. Each candidate then costs one set lookup. This is the shape of a grid robot program where exactly one command must change. Memory hook: *walk forward from S, undo backward from T, and the one edit must join them.*

```
DIRS = ((-1, 0), (0, 1), (1, 0), (0, -1))   # up, right, down, left
UNDO = {"F": "B", "B": "F", "L": "R", "R": "L"}

def move(state, cmd):                   # no wall check here
    r, c, d = state
    if cmd == "L":
        return r, c, (d - 1) % 4
    if cmd == "R":
        return r, c, (d + 1) % 4
    dr, dc = DIRS[d]
    if cmd == "B":
        dr, dc = -dr, -dc
    return r + dr, c + dc, d

def find(grid, ch):
    for r, row in enumerate(grid):
        if ch in row:
            return r, row.index(ch)

def repair_by_replay(grid, program):    # baseline: prefix state + suffix replay
    rows, cols = len(grid), len(grid[0])
    is_open = lambda s: 0 <= s[0] < rows and 0 <= s[1] < cols and grid[s[0]][s[1]] != "#"

    def finishes(state, start):
        for cmd in program[start:]:
            state = move(state, cmd)
            if not is_open(state):
                return False
        return state[:2] == find(grid, "T")
```

```
    state = (*find(grid, "S"), 0)
    for i, old in enumerate(program):
        for new in "FBLR":
            s = move(state, new)
            if new != old and is_open(s) and finishes(s, i + 1):
                return True
        state = move(state, old)
        if not is_open(state):
            return False
    return False

def can_repair(grid, program):          # forward from S, backward from T
    rows, cols = len(grid), len(grid[0])
    is_open = lambda s: 0 <= s[0] < rows and 0 <= s[1] < cols and grid[s[0]][s[1]] != "#"
    n = len(program)
    good = [set() for _ in range(n + 1)]    # good[j]: states before program[j] that finish on T
    for d in range(4):                      # the final facing is free
        state = (*find(grid, "T"), d)
        for j in range(n, 0, -1):
            good[j].add(state)
            state = move(state, UNDO[program[j - 1]])
            if not is_open(state):          # a blocked chain never resumes
                break
    state = (*find(grid, "S"), 0)
    for i, old in enumerate(program):
        for new in "FBLR":
            if new != old and move(state, new) in good[i + 1]:
                return True
        state = move(state, old)
        if not is_open(state):
            return False
    return False

cases = [(["..T", ".#.", "S.."], "FFRFL"), (["T..", "...", "..S"], "LFFRBF"), (["S#T"], "RF")]
[can_repair(g, p) for g, p in cases]                                    # -> [True, True, False]
all(can_repair(g, p) == repair_by_replay(g, p) for g, p in cases)       # -> True
can_repair(["ST"], "RF")                # -> False
```

The last case already succeeds, but the edit must change the command, and no other command works. If a step cannot be undone (a blocked move keeps the robot in place, a clamp, a max), the backward set can grow. Then keep the baseline. The same split solves "delete one element to make an array sorted" and "change one character to make a string valid". (A grid robot program where exactly one command must change.)

64.5 Adapt it to a new problem

Ask these questions in order.

1. **What is the state?** Write it down: a grid, a position and a facing, a line of items, a round counter.
2. **What is one step?** Write the rule for one cell or one step. For a transform, write the map of one cell. Trace it by hand on a non-square example.
3. **Does it fit?** Multiply the number of steps by the work per step. Up to about 10^7 simple operations, simulate literally. Above that, look for a shared offset, a cycle, or a closed form.
4. **Can a write corrupt a later read?** If yes, keep them apart: a new grid, two passes, or an encoding of old and new in one cell.
5. **Is in-place required?** Factor the map into pair swaps or cycles. Borrow input cells whose final value is known for the bookkeeping.
6. **Does the scan delete or insert items?** Decide how the index moves after each case. Or build the survivors in a new list (a stack) and avoid index shifts.
7. **Is the answer about one edit?** Store the prefix states forward. Answer the suffix question with a replay, or

with sets built backward from the goal when steps can be undone.

8. **Which shapes break it?** Test 1×1, $1 \times n$, $n \times 1$, and a tall and a wide rectangle.
9. **Is a faster version correct?** Keep the literal simulator. Compare the two versions on small inputs before you trust the faster one.

64.6 Pitfalls

CAUTION

Negative indexes wrap. `grid[r - 1][c]` with `r = 0` reads the last row and raises no error. Check `0 <= nr < R and 0 <= nc < C` before every index. Do not rely on `IndexError`. The same trap hits slices: `row[left:right + 1]` with `right = -2` is `row[left:-1]`, which can return real cells.

CAUTION

Writes during the scan. A value written early is read later as if it were old. Zeros spread over the whole matrix, or a neighbor count mixes two generations. Use a second grid, a read pass and a write pass, or an encoding that keeps the old value readable.

CAUTION

The transpose loop starts at i + 1. If `j` runs over all columns, each pair swaps two times and the matrix does not change.

CAUTION

The spiral reads a row or a column two times. A wide or tall grid leaves a single row or column at the end. Check `top <= bottom` before the bottom pass and `left <= right` before the left pass. Square test grids hide this bug.

CAUTION

The name is rebound, not the object. Inside a function, `matrix = new_grid` changes only the local name. The caller still sees the old grid. Change the rows, or use `matrix[:] = new_grid`. Slice assignment still builds a full new grid first, so it is not $O(1)$ extra space.

CAUTION

Rows made with *. `[[0] * C] * R` repeats one row object. A write to one row changes all rows. Use `[[0] * C for _ in range(R)]`.

CAUTION

Directions and turns. Rows grow downward, so "up" is `(-1, 0)`. With the clockwise order up, right, down, left, a right turn is `d + 1` and a left turn is `d - 1`. A swapped sign mirrors every path.

CAUTION

End state or whole path. "Ends on T" tests only the final state. "Falls off at any time" tests every prefix. A net offset of 0 does not mean that nothing left the grid.

CAUTION

Index after a delete. After `del line[i - 1]`, the current item moves to `i - 1` and the next item is at `i`. Do not delete from a list inside `for x in line`. A delete at or before the current position shifts the next item into the current index, and the loop skips it.

CAUTION

Wrong box id. The flat box id is `(r // 3) * 3 + c // 3`. The sum `r // 3 + c // 3` gives the same id to different boxes, for example to box `(0, 1)` and box `(1, 0)`. A pair key `(r // 3, c // 3)` avoids this bug.

CAUTION

Divisors and rounding at the border. A border cell has fewer neighbors. Divide by the count that you kept. Use `//` for a floor. `round()` rounds a half to the even neighbor.

64.7 Problems in this family

Problem	What changes	Memory hook
Rotate Image	Square grid, in place. Clockwise rotation as transpose plus a reverse of each row.	Flip on the diagonal, then flip each row.
Spiral Matrix	Fixed visiting order. Direction walk with a turn rule; optimal form uses four shrinking bounds.	Walk straight, turn right when blocked.
Set Matrix Zeroes	Writes would spread. Read pass, then write pass; $O(1)$ form borrows row 0 and column 0.	Mark zero rows and zero columns first, then clear.
Game of Life	Synchronous 8-neighbor update in place. Bit 0 holds now, bit 1 holds next.	Bit 0 is now, bit 1 is next: read `& 1`, write `|= 2`, then shift `>>= 1`.
Valid Sudoku	Each cell is in three groups. One seen-set per row, column and box.	Each digit checks in at its row, its column and its box (r // 3, c // 3). A second check-in at any of the three fails.

The transfer drills for this technique are in Section 69.22 (page 426). The answers are in Section A.21 (page 465).

65 Math and number manipulation

Problems in this family look like puzzles about numbers. Most of them use a small set of mechanical tools. The tools take a number apart digit by digit. They add or multiply digit sequences with a carry. They halve an exponent, guard a fixed range, sieve the primes up to a bound, and replace a loop with a closed formula. Two more tools count shapes through a fixed point and send a gradient back through an index map. Part I (Numbers) gives the Python rules for `//`, `%`, `divmod` and `pow`. This section gives the techniques that use them.

65.1 The idea

A number is a sequence of digits in a base. Two operations move one digit at a time:

- Pop the lowest digit: `x, d = divmod(x, 10)`. Now `x` has one digit less.
- Push a digit at the low end: `res = res * 10 + d`. Now `res` has one digit more.

A loop that pops from `x` and pushes onto `res` reverses the digits. For $x = 123$, the loop pops 3, then 2, then 1. The result grows from 3 to 32 to 321.

```
x = 123
x, d = divmod(x, 10)
(x, d)                  # -> (12, 3)
res = 0
for d in [3, 2, 1]:
    res = res * 10 + d
res                     # -> 321
```

When a number is too long for a machine integer, keep it as a list of digits. Then do the arithmetic as on paper. Start at the least significant digit, and move a carry to the left. For $1299 + 1$, the last 9 becomes 0, and a carry of 1 moves left. The next 9 also becomes 0. The 2 takes the carry and becomes 3. The result is 1300.

The other tools in this section replace slow repetition with structure:

Tool	Brute force	What the tool saves
Cycle detection	Simulate until the process stops. It can loop forever.	Stop at the first repeated state.
Overflow guard	Compute a wider value, then compare it with the limit.	Invert the operation and compare before it runs.
Repeated squaring	n multiplications.	At most about $2\log_2 n$ multiplications.
Closed formula	Store a set, or loop over all values.	One formula and $O(1)$ extra space.
Sieve	Test each number by trial division: $O(n\sqrt{n})$.	Each prime crosses out its multiples: $O(n \log\log n)$.
Anchor and forced corners	Try every triple of points.	One loop over the free corner, and hash lookups for the rest.
Backward through an index map	Build the $n \times n$ Jacobian.	One scatter through the argsort.

65.2 Recognize it

- The statement takes a number apart: reverse it, sum its digits, or test a property of its digits.
- A number arrives as a string, an array or a linked list of digits. It can be larger than a 64-bit integer can hold (about $9.2 \cdot 10^{18}$, so 19 or more digits can overflow).
- The statement forbids a big-integer library or a direct conversion to an integer. Lengths of a few hundred digits allow an $O(mn)$ loop over all digit pairs.
- The statement names a fixed range, such as $[-2^{31}, 2^{31} - 1]$, and says that no wider type exists. An input of at most 10 digits makes speed easy. The hard parts are the sign, a zero and the range limits.
- A rule changes a value again and again. The question is whether the process ends or reaches a target.
- A count or an exponent is huge (10^9 or more), and the repeated step is associative: a multiplication, a matrix product, or a composition of functions. The statement often asks for the answer modulo $10^9 + 7$.
- The full expected collection is known, for example all values $0, 1, \ldots, n$, and a few values are missing or extra.
- The task counts the primes up to a bound such as 10^6, or it factorizes many numbers up to a bound. One value up to about 10^{12} needs only trial division up to its square root.
- The task counts shapes that contain a given point. Duplicate points count as separate choices.
- The task gives an upstream gradient $v = \partial L/\partial y$ and asks for $\partial L/\partial x$ through sort, max, median or a division by a statistic. A length of 10^5 rules out any $n \times n$ object, such as the Jacobian.

65.3 Core template

Two templates do most of the work. The first template takes a number apart and builds a number from digits.

```
def digits_low_first(x, base=10):        # x >= 0
    out = []
    while x:
        x, d = divmod(x, base)           # pop the lowest digit
        out.append(d)
    return out or [0]

def number_from_digits(ds, base=10):     # ds: most significant digit first
    res = 0
    for d in ds:
        res = res * base + d             # push a digit at the low end
    return res

digits_low_first(1203)                   # -> [3, 0, 2, 1]
digits_low_first(0)                      # -> [0]
digits_low_first(6, 2)                   # -> [0, 1, 1]
number_from_digits([1, 2, 0, 3])         # -> 1203
number_from_digits(digits_low_first(1200))   # -> 21
sum(d * d for d in digits_low_first(19)) # -> 82
```

Invariant of the pop loop: the original value equals $x \cdot 10^k$ plus the value of the k digits already popped. The loop stops when `x` is 0. A push of the digits in pop order gives the reversed number.

The second template adds two digit sequences with a carry. Store the digits least significant first, so that index k is the place 10^k.

```
def add_digit_lists(a, b, base=10):      # a, b: least significant digit first
    out, carry, i = [], 0, 0
    while i < len(a) or i < len(b) or carry:
        total = carry
        if i < len(a):
            total += a[i]                # a missing digit counts as 0
        if i < len(b):
            total += b[i]
        carry, digit = divmod(total, base)
        out.append(digit)
        i += 1
    return out

add_digit_lists([9, 9], [1])             # -> [0, 0, 1]
add_digit_lists([2, 4, 3], [5, 6, 4])    # -> [7, 0, 8]
add_digit_lists([1, 1], [1], 2)          # -> [0, 0, 1]
def low_first(x):
    return [int(c) for c in reversed(str(x))]
def value(ds):
    return sum(d * 10**k for k, d in enumerate(ds))
all(value(add_digit_lists(low_first(p), low_first(q))) == p + q
    for p in range(150) for q in range(150))   # -> True
```

Invariant: after k positions, `out` holds the k low digits of the sum. The carry is 0 or 1, because the largest total is $9+9+1=19$. The loop runs while a list has digits or the carry is not 0. So a final carry becomes a new top digit. If the digits come most significant first, walk from the right end, or reverse the input and the output.

65.4 Variations

65.4.1 Iterated digit maps: detect the cycle

What changes: a rule replaces a value with a new value again and again. Happy Number replaces n with the sum of the squares of its digits and asks if the walk reaches 1. A loop "until 1" never stops for an unhappy number.

The key fact: a fixed rule on a finite set of states must repeat a state. After the first repeat, the walk loops forever. The walk is a tail followed by a cycle (a "rho" shape). Here the set is finite because the rule shrinks large values. A d-digit number maps to at most $81d$. For $d \geq 4$, this is less than the number itself. For a 32-bit input, every value is at most 243 after two steps.

The intuitive solution stores every value in a seen set. It stops at the target or at a repeat. Memory hook (Happy Number): *walk until you hit 1 or a repeat. A seen set catches the repeat.*

```
def digit_square_sum(x):
    total = 0
    while x:
        x, d = divmod(x, 10)
        total += d * d
    return total

def is_happy(n):
    seen = set()
    while n != 1 and n not in seen:
        seen.add(n)
        n = digit_square_sum(n)
    return n == 1

is_happy(19), is_happy(2), is_happy(1)   # -> (True, False, True)
digit_square_sum(2**31 - 1)              # -> 260
```

NOTE

The optimal form uses Floyd fast and slow pointers with $O(1)$ memory (see the Two pointers section). The target 1 maps to itself, so it is a cycle of length 1. The number is happy when the fast pointer reaches 1. The same view solves Linked List Cycle and Find the Duplicate Number, where `next(i)` is `nums[i]`.

The rho shape also answers "the state after k steps" for a huge k. Record the step where each state first appears. The first repeat gives the tail length μ and the cycle length λ. Then step $k \geq \mu$ is the same state as step $\mu + (k - \mu) \bmod \lambda$. This template works for any hashable state, such as a tuple of cells in a simulation.

```
def digit_square_sum(x):
    return sum(int(c) ** 2 for c in str(x))

def rho(x, step):
    first = {}                      # state -> step where it first appears
    order = []                      # order[i] is the state after i steps
    while x not in first:
        first[x] = len(order)
        order.append(x)
        x = step(x)
    mu = first[x]                   # tail length
    return order, mu, len(order) - mu  # last value: cycle length

def state_at(x, step, k):
    order, mu, lam = rho(x, step)
    if k < len(order):
        return order[k]
    return order[mu + (k - mu) % lam]

order, mu, lam = rho(2, digit_square_sum)
order[:4], mu, lam                  # -> ([2, 4, 16, 37], 1, 8)
state_at(2, digit_square_sum, 10**18)   # -> 20
def brute(x, k):
    for _ in range(k):
        x = digit_square_sum(x)
    return x
all(state_at(2, digit_square_sum, k) == brute(2, k) for k in range(40))   # -> True
```

65.4.2 Guard a fixed range before the operation

What changes: the result must stay inside a fixed range, and no wider type exists. Reverse Integer reverses the digits of a 32-bit integer and returns 0 if the result leaves the range. Python integers never overflow, so the check must be explicit. The check must also run before the operation, because a fixed-width language cannot store the wider value.

Invert the operation to get the check. For integers and $b > 0$, $a \cdot b + c > L$ is the same as $a > \lfloor (L - c)/b \rfloor$. For $0 \leq c \leq L$, the right side never leaves the range. Put the sign aside, work on the magnitude, and apply the sign at the end. Memory hook (Reverse Integer): *put the sign aside. Pop a digit, solve* `res * 10 + digit <= INT_MAX` *for* `res`*, and push only if it holds.*

```
INT_MAX = 2**31 - 1

def reverse(x):
    sign = -1 if x < 0 else 1
    x = abs(x)
    res = 0
    while x:
        x, digit = divmod(x, 10)
        if res > (INT_MAX - digit) // 10:    # res * 10 + digit > INT_MAX
            return 0
        res = res * 10 + digit
    return sign * res
```

```
reverse(123), reverse(-120), reverse(0)      # -> (321, -21, 0)
reverse(1534236469)                          # -> 0
reverse(-2**31), reverse(2**31 - 1)          # -> (0, 0)
all((a * 10 + d > INT_MAX) == (a > (INT_MAX - d) // 10)
    for a in range(214748360, 214748370) for d in range(10))   # -> True
```

NOTE

Many references write the guard in two parts: `res > INT_MAX // 10`, or `res == INT_MAX // 10` and `digit > 7`. Both forms are equivalent. One positive bound is enough for both signs here. No valid 32-bit input reverses to a magnitude of exactly 2^{31}. In C or Java, keep `x` signed, because the magnitude of -2^{31} does not fit a signed 32-bit integer.

The same guard protects a manual parse of a number from its digits, with no wider type. The pop and push loop also works in base 2 (Reverse Bits), with `2` in place of `10`. There the loop runs exactly 32 steps, and the result always fits, so no guard is necessary.

65.4.3 Carry through a digit array or a digit list

What changes: the digits live in an array, a string or a linked list, and the direction of the digits differs. The carry loop of the core template stays the same.

Plus One adds 1 to a digit array that is most significant first. Walk from the right. A 9 becomes 0 and passes the carry on. The first digit below 9 takes the carry, and the loop returns at once, because no digit to its left changes. If every digit was 9, the result needs a new leading 1. Memory hook (Plus One): *from the right, each 9 becomes 0 until a digit can take the +1. If all digits are 9, put a 1 in front.*

```
def plus_one(digits):
    for i in range(len(digits) - 1, -1, -1):
        if digits[i] < 9:
            digits[i] += 1          # this digit takes the carry
            return digits           # nothing to the left changes
        digits[i] = 0               # 9 + 1 = 10: write 0, carry 1 to the left
    return [1] + digits             # all digits were 9

plus_one([1, 2, 9, 9])              # -> [1, 3, 0, 0]
plus_one([9, 9])                    # -> [1, 0, 0]
plus_one([0])                       # -> [1]
```

Add Two Numbers adds two numbers stored in linked lists, least significant digit first. This order matches column addition, so one forward pass is enough. Read 0 for a list that has ended. Build the output with a dummy head and a tail pointer. Memory hook (Add Two Numbers): *add digits plus carry while anything is left. Write mod 10, carry div 10.*

```
class ListNode:
    def __init__(self, val=0, next=None):
        self.val, self.next = val, next

def build(vals):
    dummy = tail = ListNode()
    for v in vals:
        tail.next = ListNode(v)
        tail = tail.next
    return dummy.next

def to_list(node):
    out = []
    while node:
        out.append(node.val)
        node = node.next
    return out
```

```python
def add_two_numbers(l1, l2):
    dummy = ListNode()
    tail = dummy
    carry = 0
    while l1 or l2 or carry:
        total = carry
        if l1:
            total += l1.val
            l1 = l1.next
        if l2:
            total += l2.val
            l2 = l2.next
        carry, digit = divmod(total, 10)
        tail.next = ListNode(digit)
        tail = tail.next
    return dummy.next

to_list(add_two_numbers(build([2, 4, 3]), build([5, 6, 4])))   # -> [7, 0, 8]
to_list(add_two_numbers(build([9, 9]), build([1])))            # -> [0, 0, 1]
to_list(add_two_numbers(build([0]), build([0])))               # -> [0]
```

In base 2, the same loop solves Add Binary. With two decimal strings, it solves Add Strings. When the linked lists are most significant first (Add Two Numbers II), reverse them first, or push the digits on two stacks. For a subtraction, use a borrow in place of the carry, and strip the leading zeros of the result.

65.4.4 Multiply digit sequences

What changes: the operation is a multiplication. Every digit of one number multiplies every digit of the other number. Multiply Strings multiplies two decimal strings of up to 200 digits without a conversion to an integer.

The key fact is about place values. The digit at place 10^i times the digit at place 10^j lands at place 10^{i+j}. Reverse both digit lists, so that index k is place 10^k. Add every product into slot $i+j$ of an array with $m+n$ slots, because the product has at most $m+n$ digits. Then run one carry pass from the lowest slot, as in the addition template. Memory hook (Multiply Strings): *reverse so the index is the power, drop* `a[i] * b[j]` *into slot* `i + j`*, then carry once.*

```python
def multiply(num1, num2):
    a = [ord(c) - ord("0") for c in reversed(num1)]   # index k is place 10**k
    b = [ord(c) - ord("0") for c in reversed(num2)]
    slots = [0] * (len(a) + len(b))
    for i, x in enumerate(a):
        for j, y in enumerate(b):
            slots[i + j] += x * y            # no carry yet
    carry = 0
    for k in range(len(slots)):              # one carry pass from the lowest place
        carry, slots[k] = divmod(slots[k] + carry, 10)
    while len(slots) > 1 and slots[-1] == 0:
        slots.pop()                          # keep at least one digit for "0"
    return "".join(str(d) for d in reversed(slots))

multiply("123", "456")                       # -> '56088'
multiply("0", "52")                          # -> '0'
multiply("99", "99")                         # -> '9801'
all(multiply(str(p), str(q)) == str(p * q)
    for p in range(0, 1000, 7) for q in range(0, 1000, 13))   # -> True
```

NOTE

The optimal form keeps the left-to-right order. It adds `num1[i] * num2[j]` into slot `i + j + 1` and moves the carry inside the inner loop. The time is the same, $O(mn)$. Index pairs that combine by their sum form a convolution. The same slot idea multiplies two polynomials.

65.4.5 Fast exponentiation by repeated squaring

What changes: an associative step repeats n times, and n is huge. Pow(x, n) computes x^n for $|n|$ up to 2^{31}. A loop of n multiplications takes minutes.

The key fact: $x^n = (x^{n/2})^2$ for an even n, and one more factor x for an odd n. Each step halves n, so about 32 steps are enough. For example, $x^{13} = (x^6)^2 \cdot x$, $x^6 = (x^3)^2$, and $x^3 = (x^1)^2 \cdot x$. That is 5 multiplications in place of 12. A negative n becomes a positive one: $x^n = (1/x)^{-n}$. Fix the sign first, because `-1 // 2` is `-1` and never reaches 0. Memory hook (Pow(x, n)): *square the half, add one x if odd.*

```
def my_pow(x, n):
    if n < 0:
        return my_pow(1 / x, -n)
    if n == 0:
        return 1.0
    half = my_pow(x, n // 2)          # compute the half one time only
    if n % 2 == 0:
        return half * half
    return half * half * x

my_pow(2.0, 10), my_pow(2.0, -2), my_pow(5.0, 0)   # -> (1024.0, 0.25, 1.0)
t = 1.00001 ** 123456
abs(my_pow(1.00001, 123456) - t) < 1e-9 * t        # -> True
my_pow(1.0, -2**31)                                # -> 1.0
```

The optimal form is the iterative bit loop of Part I (Numbers). It uses $O(1)$ space in place of about 34 stack frames. Invariant of the bit loop: `result` times base^n always equals the answer. The loop works for any associative operation with an identity element. Use it for a modular power, or for a matrix power that jumps a linear recurrence n steps ahead.

```
def power(base, n, mul, one):            # n >= 0; mul must be associative
    result = one
    while n:
        if n & 1:
            result = mul(result, base)   # this bit of n is set
        base = mul(base, base)           # base^1, base^2, base^4, ...
        n //= 2
    return result

MOD = 10**9 + 7
power(3, 200, lambda a, b: a * b % MOD, 1) == pow(3, 200, MOD)   # -> True

def mat_mul(A, B):
    return [[sum(A[i][k] * B[k][j] for k in range(2)) for j in range(2)]
            for i in range(2)]

def fib(n):                              # [[1, 1], [1, 0]]^n holds F(n) in the corner
    return power([[1, 1], [1, 0]], n, mat_mul, [[1, 0], [0, 1]])[0][1]

[fib(n) for n in range(10)]              # -> [0, 1, 1, 2, 3, 5, 8, 13, 21, 34]
fib(90)                                  # -> 2880067194370816120
```

In Python, `pow(a, b, m)` does the modular power. Write the loop when the statement forbids the built-in or when the operation is not a number multiplication, as in a matrix power. Super Pow gives the exponent as a digit array. Push one digit at a time: $a^{10e+d} = (a^e)^{10} \cdot a^d$. Binary lifting (Kth Ancestor of a Tree Node) uses the same doubling on the parent function.

65.4.6 Closed formulas: the Gauss sum and its relatives

What changes: the full expected collection is known before the input is read. Missing Number gives n distinct values from $0, 1, \ldots, n$ and asks for the one value that is not there.

One unknown needs one equation. The sum of $0, 1, \ldots, n$ is $n(n+1)/2$ (the Gauss sum). The expected sum minus

the actual sum is the missing value. This uses $O(1)$ extra space and no set. Memory hook (Missing Number): *expected sum minus actual sum.*

```
def missing_number(nums):
    n = len(nums)
    expected = n * (n + 1) // 2        # sum of 0, 1, ..., n
    return expected - sum(nums)

missing_number([3, 0, 1])              # -> 2
missing_number([0, 1])                 # -> 2
missing_number([1])                    # -> 0
```

NOTE

The optimal form XORs all indexes, all values and n. Each present value appears two times and cancels. XOR cannot overflow in a fixed-width language. See the Bit manipulation section.

Two unknowns need two equations. Suppose that the values $1, \ldots, n$ have one value twice and one value missing. The sum gives dup $-$ miss. The sum of squares gives $\text{dup}^2 - \text{miss}^2$. Divide the second difference by the first to get dup $+$ miss. Other closed formulas replace a counting loop: the number of pairs, an arithmetic series, and the multiples of k in a range.

```
def dup_and_missing(nums):             # values 1..n: one value twice, one value missing
    n = len(nums)
    d1 = sum(nums) - n * (n + 1) // 2                                  # dup - miss
    d2 = sum(x * x for x in nums) - n * (n + 1) * (2 * n + 1) // 6     # dup^2 - miss^2
    s = d2 // d1                                                       # dup + miss
    return (s + d1) // 2, (s - d1) // 2

dup_and_missing([1, 2, 2, 4])          # -> (2, 3)
dup_and_missing([3, 1, 3])             # -> (3, 2)
n = 6
n * (n - 1) // 2                       # -> 15
first, last, count = 3, 21, 7          # 3, 6, ..., 21
(first + last) * count // 2            # -> 84
def multiples(lo, hi, k):              # multiples of k in [lo, hi]
    return hi // k - (lo - 1) // k
multiples(10, 30, 7), multiples(7, 7, 7)   # -> (3, 1)
```

65.4.7 Primes: sieve and smallest prime factor

What changes: the task needs all primes up to a bound n, or the prime factors of many numbers. Count Primes asks for the number of primes below $n \le 5 \cdot 10^6$. A trial-division test of each number costs $O(n\sqrt{n})$, which is too slow.

The sieve of Eratosthenes does not test each number. Each prime p crosses out its own multiples. Two facts limit the work. Every composite $m \le n$ has a prime factor $p \le \sqrt{n}$, so the outer loop stops at $\lfloor \sqrt{n} \rfloor$. A multiple kp with $k < p$ has a prime factor smaller than p, so it is already crossed out. Start at p^2. The total time is $O(n \log \log n)$, and the memory is $O(n)$. Memory hook (sieve): *each prime crosses out its multiples from p*p on. Stop when p*p is above n.*

```
import math

def sieve(n):                          # is_prime[i] for 0 <= i <= n
    is_prime = [i > 1 for i in range(n + 1)]
    for p in range(2, math.isqrt(n) + 1):
        if is_prime[p]:                # cross out p*p, p*p + p, ..., up to n
            is_prime[p * p::p] = [False] * len(range(p * p, n + 1, p))
    return is_prime

[i for i, ok in enumerate(sieve(30)) if ok]   # -> [2, 3, 5, 7, 11, 13, 17, 19, 23, 29]
sum(sieve(100)), sum(sieve(10**5))     # -> (25, 9592)
sum(sieve(0)), sum(sieve(1)), sum(sieve(2))   # -> (0, 0, 1)
```

```
def is_prime_slow(x):
    return x > 1 and all(x % d for d in range(2, math.isqrt(x) + 1))
sieve(500) == [is_prime_slow(i) for i in range(501)]   # -> True
```

The slice assignment does the same work as a loop over `range(p * p, n + 1, p)`. It runs as one operation in C, so it is many times faster. Use it for n near 10^6 or more.

To factorize many numbers, store the smallest prime factor (spf) of each number. The build is the same sieve, but the first prime that reaches m writes itself into `spf[m]`. Then divide x by `spf[x]` until x is 1. Each division at least halves x, so one factorization costs $O(\log x)$. For one large x, such as 10^{12}, a table is too large. Use trial division by every d with $d^2 \le x$. That costs $O(\sqrt{x})$. If the rest is above 1 after the loop, the rest is a prime. Memory hook (factorize): *the table keeps the first prime that reaches each number. Divide by it until 1.*

```
import math

def smallest_factor(n):                  # spf[i]: smallest prime factor of i, for i >= 2
    spf = list(range(n + 1))
    for p in range(2, math.isqrt(n) + 1):
        if spf[p] == p:                  # p is a prime
            for m in range(p * p, n + 1, p):
                if spf[m] == m:          # no smaller prime reached m before
                    spf[m] = p
    return spf

def factorize(x, spf):                   # O(log x) divisions
    out = []
    while x > 1:
        out.append(spf[x])
        x //= spf[x]
    return out

def trial_division(x):                   # one large x: O(sqrt(x)) steps
    out, d = [], 2
    while d * d <= x:
        while x % d == 0:
            out.append(d)
            x //= d
        d += 1
    if x > 1:
        out.append(x)                    # the rest is a prime
    return out

spf = smallest_factor(1000)
factorize(360, spf)                      # -> [2, 2, 2, 3, 3, 5]
factorize(997, spf)                      # -> [997]
trial_division(6 * (10**9 + 7))          # -> [2, 3, 1000000007]
all(factorize(x, spf) == trial_division(x) for x in range(2, 1001))   # -> True
```

For a common divisor or a common multiple, use `math.gcd` and `math.lcm` from Part I. The number of divisors follows from the factorization: multiply $(e + 1)$ over the exponents e.

65.4.8 Count shapes through an anchor point

What changes: the task counts shapes that contain a given point, over a stream of stored points. Detect Squares stores points with duplicates and counts the axis-aligned squares with positive area that have the query as one corner.

Ask how many corners fix the whole shape. The query is one corner. The diagonal corner (x, y) then fixes the other two corners: (x, q_y) and (q_x, y). The diagonal is valid only if $|x - q_x| = |y - q_y| \neq 0$. Store a counter of point copies, so each lookup is $O(1)$. Each corner can be any of its copies, so a square adds the product of the three counts. An add costs $O(1)$. A query costs $O(d)$, for d distinct stored points. Memory hook (Detect Squares): *pick the diagonal, the other two corners are forced, multiply the counts.*

```python
from collections import Counter

class DetectSquares:
    def __init__(self):
        self.points = Counter()                 # (x, y) -> number of copies

    def add(self, point):
        self.points[tuple(point)] += 1

    def count(self, point):
        qx, qy = point
        total = 0
        for (x, y), copies in self.points.items():
            if x == qx or abs(x - qx) != abs(y - qy):
                continue                        # not the diagonal of a real square
            total += copies * self.points.get((x, qy), 0) * self.points.get((qx, y), 0)
        return total

ds = DetectSquares()
for p in [[3, 10], [11, 2], [3, 2]]:
    ds.add(p)
ds.count([11, 10])                              # -> 1
ds.count([14, 8])                               # -> 0
ds.add([11, 2])
ds.count([11, 10])                              # -> 2
```

The general rule: find the fewest elements that fix the shape. Enumerate only those elements. Look up the forced rest in a hash map, and multiply the counts when duplicates are separate choices. Two Sum uses the same thinking. For squares at any angle, enumerate a second point to get the side (d_x, d_y). Turn it by 90 degrees to $(-d_y, d_x)$.

65.4.9 Chain rule through an index map

What changes: the task asks for a backward pass. It gives x and $v = \partial L/\partial y$ and asks for $\partial L/\partial x$. A typical example is a backward pass for $y = \text{sort}(x)/\text{median}(x)$ with an odd n and distinct values.

Do not build the Jacobian. Split the function into primitive steps, and apply one backward rule per step. Sort and median only move or pick elements. With distinct values, they act as fixed index maps. So the gradient goes back along the same indexes: a scatter through the argsort. The division by the median adds a second path. For $y = s/m$, the gradient of m is $-(v \cdot y)/m$. Only the median element receives this term, and it adds to the direct term of that element. Memory hook (gradient through a sort divided by the median): *scatter v over m through the argsort, then charge minus dot over m to the median.*

```python
def sort_median_backward(x, v):
    n = len(x)
    order = sorted(range(n), key=x.__getitem__)   # s[k] = x[order[k]]
    mid = order[n // 2]                           # index in x of the median
    m = x[mid]
    grad = [0.0] * n
    dot = 0.0                                     # v . y
    for k, i in enumerate(order):
        grad[i] = v[k] / m                        # path 1: back through the permutation
        dot += v[k] * x[i] / m
    grad[mid] -= dot / m                          # path 2: through the denominator
    return grad

sort_median_backward([3.0, 2.0, 1.0], [1.0, 1.0, 1.0])   # -> [0.5, -1.0, 0.5]

def f(x):                                         # forward pass, for the check
    s = sorted(x)
    return [t / s[len(x) // 2] for t in s]
```

```python
def numeric_grad(x, v, h=1e-6):                  # central differences of v . f(x)
    g = []
    for i in range(len(x)):
        up, down = x[:], x[:]
        up[i] += h
        down[i] -= h
        g.append((sum(a * b for a, b in zip(v, f(up)))
                  - sum(a * b for a, b in zip(v, f(down)))) / (2 * h))
    return g

x, v = [1.0, 4.0, 2.0, 8.0, 3.0], [0.5, -1.0, 2.0, 0.0, 1.0]
all(abs(a - b) < 1e-6 for a, b in zip(sort_median_backward(x, v), numeric_grad(x, v)))   # -> True
```

These rules cover most hand-written backward passes:

Forward step	Backward rule
$y_k = x_{\text{idx}[k]}$ (sort, gather, top-k, max-pool)	Add v_k to the gradient of $x_{\text{idx}[k]}$. Other entries get 0.
$y = u/c$, with a scalar c	The gradient of u is v/c. The gradient of c is $-(v \cdot y)/c$.
$c = x_j$ (max, median with an odd n)	Only x_j receives the gradient of c.
$c = \sum_i x_i$ or $c = \text{mean}(x)$	Every x_i receives the gradient of c, divided by n for the mean.
One value feeds two paths	Add the two contributions. Do not overwrite.

To test a backward pass, compare it with central differences on a small input, as in the example above. The argsort-then-scatter pattern also solves problems that compute in sorted order but answer in the original order.

65.5 Adapt it to a new problem

Ask these questions in order:

1. Does the value fit a machine integer, and does the statement allow a conversion? If not, keep the digits and do the arithmetic as on paper.
2. Which end holds the least significant digit? Iterate from that end, or reverse first.
3. How long can the result be? Preallocate: $\max(m, n) + 1$ digits for a sum, $m + n$ digits for a product.
4. Does the statement fix a range with no wider type? Invert each operation that can grow the value, and test before the operation runs.
5. Does a rule change a state again and again, on a finite set of states? A repeat must come. Use a seen set, Floyd pointers, or the cycle length to jump ahead.
6. Is a count or an exponent huge, with an associative step? Use repeated squaring. Take the modulo after each multiplication.
7. Is the full expected collection known? Write one equation for each unknown: a sum, a sum of squares, or an XOR.
8. Does the task need the primes or the factors of many numbers up to a bound? Build a sieve or a smallest-prime-factor table one time. For one large value, use trial division.
9. Does the task count shapes or tuples through a fixed point? Find the fewest elements that fix the shape. Enumerate them, look up the rest, and multiply the counts.
10. Is it a backward pass? Split the function into primitive steps. Scatter through each index map, add the contributions, and check with central differences.
11. Test the edge values: 0, 1, a negative value, the range limits, all 9s, and a single element.

65.6 Pitfalls

CAUTION

Negative numbers in a digit loop. Python `//` rounds toward negative infinity, and `%` takes the sign of the divisor. So `divmod(-123, 10)` pops the digit 7, not 3. Put the sign aside, work on `abs(x)`, and apply the sign at the end.

```
divmod(-123, 10)                # -> (-13, 7)
divmod(abs(-123), 10)           # -> (12, 3)
-1 // 2, -1 >> 1                # -> (-1, -1)
```

The last line shows why a halving loop on a negative exponent never stops. Fix the sign before the loop.

CAUTION

The final carry. A loop that stops when both inputs end drops a final carry. Then $[9] + [1]$ gives $[0]$, not $[0, 1]$. Keep the carry in the loop condition, or add it after the loop.

CAUTION

A zero result. A product with a zero input has only zero slots. A strip of all leading zeros then returns an empty string. Keep at least one digit, or return `"0"` at once when an input is zero.

CAUTION

Python integers do not overflow. A range check after the loop gives the right answer in Python, but it breaks a "no 64-bit type" rule. In Java the value wraps silently, and in C a signed overflow is undefined behavior. Run the guard before each push.

CAUTION

Floats for exact work. Do not compare floats with `==`. Use a tolerance or `math.isclose`. Do not use floats for big integers: `n ** 0.5` and `/` lose digits above about 2^{53}. Use `math.isqrt` and `//`.

```
import math
0.1 + 0.2 == 0.3                    # -> False
math.isclose(0.1 + 0.2, 0.3)        # -> True
int((10**30 - 1) ** 0.5)            # -> 1000000000000000
math.isqrt(10**30 - 1)              # -> 999999999999999
(10**16 + 1) / 1 == 10**16 + 1      # -> False
```

CAUTION

Two recursive calls for the two halves. `my_pow(x, n // 2) * my_pow(x, n // 2)` does $O(n)$ work in total. Compute the half one time, store it, and square it. Also add the extra factor of x for an odd n.

CAUTION

A cycle check on the start value only. The walk can enter a cycle that does not contain the start value. For example, 2 enters the cycle at 4 and never returns to 2. Store every value in the seen set, not only the first one. With Floyd pointers, start the fast pointer one step ahead, and return whether the final value is 1, not whether the pointers met.

CAUTION

Sieve bounds. 0 and 1 are not primes. Set them to `False` before the loop. The outer loop must include $\lfloor \sqrt{n} \rfloor$. Without the `+ 1`, a square of a prime such as 25 stays marked as a prime. Count Primes counts the primes strictly below n, so sieve up to $n - 1$ or exclude n.

```
import math
def sieve_bad(n):
    is_prime = [i > 1 for i in range(n + 1)]
    for p in range(2, math.isqrt(n)):           # the + 1 is missing
        if is_prime[p]:
            for m in range(p * p, n + 1, p):
                is_prime[m] = False
    return is_prime
sieve_bad(25)[25], sieve_bad(49)[49]            # -> (True, True)
```

CAUTION

Shape counts. Skip a diagonal with $x = q_x$, because the square has zero area. Require $|x - q_x| = |y - q_y|$, or the code also counts rectangles. Store counts, not a set, because duplicate points are separate choices. Do not read a `defaultdict` with `d[key]` while a loop runs over it. The read inserts the key.

```
from collections import defaultdict
d = defaultdict(int, {(0, 0): 1})
def scan():
    for key in d:
        d[(5, 5)]                       # a read of a missing key inserts it
scan()                                  # -> raises RuntimeError
```

CAUTION

Sorted order and original order in a backward pass. `v` is in sorted order, but the gradient must be in the original order of `x`. `order[k]` maps a sorted position to an original index. Do not confuse it with the rank array. The median element receives two terms, so add the second term. Do not overwrite the first term.

65.7 Problems in this family

Problem	What changes	Memory hook
Happy Number	An iterated digit map. Detect a repeat with a seen set; Floyd pointers for $O(1)$ space.	Walk until you hit 1 or a repeat. A seen set catches the repeat.
Plus One	A carry from the right end of a digit array, with an early exit.	From the right, each 9 becomes 0 until a digit can take the +1. If all digits are 9, put a 1 in front.
Pow(x, n)	Repeated squaring. A negative n becomes base $1/x$.	Square the half, add one x if odd.
Multiply Strings	Digit products into slot $i + j$ of $m + n$ slots, then one carry pass.	Reverse so the index is the power, drop `a[i] * b[j]` into slot `i + j`, then carry once.
Detect Squares	The query is an anchor. The diagonal corner forces the other two corners. Multiply the counts.	Pick the diagonal, the other two corners are forced, multiply the counts.
Reverse Integer	Digit pop and push, with a range guard before each push.	Put the sign aside. Pop a digit, solve `res * 10 + digit <= INT_MAX` for `res`, and push only if it holds.
Add Two Numbers	The carry loop on two linked lists, least significant digit first, with a dummy head.	Add digits plus carry while anything is left. Write mod 10, carry div 10.
Missing Number	The Gauss sum of $0, \ldots, n$ minus the actual sum. XOR also works.	Expected sum minus actual sum.
Count Primes (LeetCode 204)	A sieve up to n. Each prime crosses out its multiples from p^2.	Each prime crosses out its multiples from p*p on. Stop when p*p is above n.

The transfer drills for this technique are in Section 69.23 (page 426). The answers are in Section A.22 (page 467).

66 Bit manipulation

66.1 The idea

An integer is a row of bits. AND, OR, XOR and NOT work on all positions at the same time. For these operators, each position, or column, is independent of the others. A shift moves every bit to another column. Addition and subtraction link the columns, through the carry or the borrow. So many bit problems become one small question about one column, asked 32 times.

Three moves solve most problems in this family:

- **Read and shift.** `x & 1` reads the last bit. `x >> 1` removes it. A loop of these two steps visits every bit from low to high. It is the same loop as `divmod(x, 10)` on decimal digits.
- **Cancel with XOR.** `x ^ x` is 0, and `x ^ 0` is `x`. The order of the operands does not matter. So the XOR of many values keeps only the values that occur an odd number of times.
- **Reuse a smaller number.** For `x > 0`, `x >> 1` and `x & (x - 1)` are smaller than `x`. Each one is `x` with one known bit removed. A table over $0..n$ can then fill each entry from an earlier entry.

Worked example. Find the value without a pair in `[4, 1, 2, 1, 2]`. Write the values in binary: 100, 001, 010, 001, 010. XOR puts a 1 in a column when the column has an odd number of ones. Column 0 has two ones (from the two 1s). Column 1 has two ones (from the two 2s). Column 2 has one 1 (from the 4). So the XOR of all values is 100, which is 4. A hash map finds the same value, but it stores $O(n)$ values. The XOR stores one integer.

What it saves. The bit view replaces a hash map with one integer ($O(1)$ space). It replaces a per-number loop with a table that reuses earlier answers ($O(n)$ instead of $O(n \log n)$). It replaces a string conversion with direct operations. It also rebuilds an arithmetic operator when the statement bans it.

NOTE

A Python `int` has no fixed width. A negative value acts as if it has infinitely many leading 1 bits. Many problems assume a 32-bit C `int`. Emulate it with the mask `0xFFFFFFFF`, and convert the sign only at the end.

66.2 Recognize it

- The statement talks about bits: binary form, set bits, Hamming weight, XOR, powers of two.
- Every value occurs an even number of times, except one or two values. The statement asks for $O(1)$ extra space, so a hash map is not allowed.
- The input is a known range ($0..n$ or $1..n$) with one value missing or one value extra.
- The output has one value for each integer $0..n$, and the value depends on the binary form of the integer.
- The statement gives a fixed width, such as "a 32-bit unsigned integer". Leading zeros then count.
- The output rearranges the bits or the digits of one number: reverse, rotate, complement, or a palindrome test.
- The statement bans an operator (`+`, `-`, `*`, `/`). Build it from bit operations.
- The statement says "signed 32-bit", "overflow" or "return 0 if the result is out of range".
- The values are large (up to 10^9), and the question is about pairs or about OR, AND, XOR. Count per bit position (30 or 32 columns), not per pair.

66.3 Core template

66.3.1 The toolbox of identities

Expression	Effect
`x & 1`	The last bit. It is 1 when `x` is odd.
`x >> 1`	Remove the last bit. The same as `x // 2`: in Python both round down, also for a negative `x`.
`(x >> k) & 1`	Read bit `k`.
`x | (1 << k)`	Set bit `k`.
`x & ~(1 << k)`	Clear bit `k`.
`x ^ (1 << k)`	Toggle bit `k`.
`x & (x - 1)`	Clear the lowest set bit. It is 0 only when `x` is 0 or a power of two.
`x & -x`	Keep only the lowest set bit.
`(1 << k) - 1`	A mask of `k` ones.
`x ^ x`, `x ^ 0`	0 and `x`: a value cancels itself, and 0 is the identity.
`x.bit_length()`	The number of bits without leading zeros.
`x.bit_count()` [Python 3.10+]	The number of set bits. Older versions: `bin(x).count("1")`.

```
x = 0b101100                  # 44
x & 1                         # -> 0
x >> 1                        # -> 0b10110
(x >> 2) & 1                  # -> 1
x | (1 << 0)                  # -> 0b101101
x & ~(1 << 2)                 # -> 0b101000
x ^ (1 << 5)                  # -> 0b001100
x & (x - 1)                   # -> 0b101000
x & -x                        # -> 0b000100
(1 << 4) - 1                  # -> 0b1111
x ^ x, x ^ 0                  # -> (0, 44)
x.bit_length(), x.bit_count()   # -> (6, 3)
bin(x), format(x, "08b"), int("101100", 2)   # -> ('0b101100', '00101100', 44)
```

66.3.2 Template 1: walk the bits

Read the last bit, use it, then shift it away. Stop when the number is 0. When the width is fixed, loop exactly `width` times instead, because leading zeros count.

```
def popcount(x):                      # x >= 0
    count = 0
    while x > 0:
        count += x & 1                # read the last bit
        x >>= 1                       # shift it away
    return count

def reverse_bits(x, width=32):        # fixed width: exactly width steps
    res = 0
    for _ in range(width):
        res = (res << 1) | (x & 1)    # push the last bit of x onto res
        x >>= 1                       # peel it off x
    return res

popcount(11)                          # -> 3
reverse_bits(0b0011, 4)               # -> 0b1100
reverse_bits(1) == 1 << 31            # -> True
```

Invariant: after k steps, the work so far covers exactly the lowest k bits of the input, and `x` holds the other bits, shifted down by k.

66.3.3 Template 2: fold with XOR

```python
from functools import reduce
from operator import xor

def odd_ones(values):
    acc = 0                         # 0 is the identity of XOR
    for v in values:
        acc ^= v                    # a value seen two times cancels
    return acc

odd_ones([4, 1, 2, 1, 2])           # -> 4
odd_ones([7, -3, 7])                # -> -3
reduce(xor, [4, 1, 2, 1, 2], 0)     # -> 4
```

Invariant: `acc` is the XOR of the values that occurred an odd number of times so far. Negative values need no mask here. The XOR of two signed 32-bit values is again a signed 32-bit value.

66.3.4 Template 3: emulate a 32-bit integer

Use this template when the problem assumes a C `int` and the values can be negative.

```python
MASK = 0xFFFFFFFF                   # 32 ones
def to_u32(x):
    return x & MASK                 # the 32-bit pattern of x, as a value >= 0
def to_i32(u):
    return u if u <= 0x7FFFFFFF else u - (1 << 32)   # bit 31 set means negative

to_u32(-1) == MASK                  # -> True
to_u32(-5)                          # -> 0xFFFFFFFB
to_i32(0xFFFFFFFB)                  # -> -5
to_i32(to_u32(-2**31))              # -> -2147483648
to_i32((0x7FFFFFFF + 1) & MASK)     # -> -2147483648
```

Rule: mask after every step that can create bits above bit 31. Convert to a signed value only once, at the end.

66.4 Variations

66.4.1 Cancel pairs with XOR

Every value occurs two times except one value. The space must be $O(1)$, so a hash map is not allowed. Use Template 2 directly. Memory hook (Single Number): *pairs cancel under XOR, so XOR everything.* The same fold finds one extra item between two collections: fold both collections together.

```python
from functools import reduce
from operator import xor

def single_number(nums):
    result = 0
    for num in nums:
        result ^= num               # a pair cancels: x ^ x == 0, and x ^ 0 == x
    return result

single_number([2, 2, 1])            # -> 1
single_number([4, 1, 2, 1, 2])      # -> 4
single_number([-30000])             # -> -30000
s, t = "abcd", "dbcae"              # t is s, shuffled, with one extra letter
chr(reduce(xor, map(ord, s + t), 0)) # -> 'e'
```

- Two values without a pair: the fold gives `a ^ b`. Any set bit of `a ^ b` is a bit where `a` and `b` differ. Split all values into two groups by that bit, and fold each group.
- Values that repeat three times: plain XOR fails. Use per-bit counts (see below) modulo 3.
- One value repeated many times (Find the Duplicate Number): XOR fails. Use cycle detection.

(Single Number, Missing Number, Find the Difference.)

66.4.2 Count the set bits of one number

The question is about the bits of one number: how many are 1. Use Template 1: add `n & 1`, then shift. Memory hook (Number of 1 Bits): *read the last bit, then shift it away.* The optimal form uses `n & (n - 1)`. Subtracting 1 turns the lowest 1 bit into 0 and every 0 below it into 1. The AND then clears exactly the lowest set bit, so the loop runs once per set bit, not once per position. The same fact gives a family of tools. For `x > 0`, `(x & (x - 1)) == 0` tests for a power of two. `x & -x` keeps only the lowest set bit, and a Fenwick tree uses it to move between nodes. `sub = (sub - 1) & mask` lists all submasks of `mask` (Part I, "Subsets as bitmasks"). In each case, a subtraction of 1 changes only the lowest set bit and the zeros below it. For 32-bit input both loops are fast. For many calls, use a 256-entry table of byte counts, or `bit_count()`.

```
def hamming_weight(n):                # intuitive: one step per bit position
    count = 0
    while n > 0:
        count += n & 1
        n >>= 1
    return count

def hamming_weight_fast(n):           # optimal: one step per set bit
    count = 0
    while n:
        n &= n - 1                    # clear the lowest set bit
        count += 1
    return count

hamming_weight(11), hamming_weight_fast(11)       # -> (3, 3)
hamming_weight(2147483645)                        # -> 30
bin(0b101000 & (0b101000 - 1))                    # -> '0b100000'
def is_power_of_two(n):
    return n > 0 and (n & (n - 1)) == 0
[k for k in range(1, 20) if is_power_of_two(k)]   # -> [1, 2, 4, 8, 16]
hamming_weight_fast(-5 & 0xFFFFFFFF)              # -> 31
```

CAUTION

On a negative `n`, both loops fail. The shift loop above tests `n > 0`, so it stops at once and returns 0. A loop that tests `while n` never ends: `n >> 1` stops at `-1`, and `n & (n - 1)` never reaches 0. If the input can be a signed 32-bit value, mask it first: `n &= 0xFFFFFFFF`.

(Number of 1 Bits, Hamming Distance as the count of `x ^ y`, Power of Two.)

66.4.3 Bit counts for every number: DP over bits

The output has one answer for each `i` in $0..n$. A separate loop for each number costs $O(n \log n)$. Instead, reuse an answer that the table already holds. `i >> 1` is `i` without its last bit, so it is smaller than `i`. `i & 1` is the bit that the shift removes. So `ans[i] = ans[i >> 1] + (i & 1)`. Memory hook (Counting Bits): *drop the last bit, look it up, add the bit back.*

```
def count_bits(n):
    ans = [0] * (n + 1)               # ans[0] = 0 is the base case
    for i in range(1, n + 1):         # increasing i: i >> 1 is already filled
        ans[i] = ans[i >> 1] + (i & 1)
    return ans

count_bits(5)                         # -> [0, 1, 1, 2, 1, 2]
count_bits(0)                         # -> [0]
n = 8
alt = [0] * (n + 1)
for i in range(1, n + 1):
    alt[i] = alt[i & (i - 1)] + 1     # remove the lowest set bit: exactly one 1 fewer
alt == count_bits(8)                  # -> True
```

The general rule: when you need $f(i)$ for every i, find an $O(1)$ step that maps i to a smaller j with a known change in f. In base 2 the steps are `i >> 1`, `i & (i - 1)`, and "remove the top bit". In base 10 the steps are `i // 10` and `i % 10`. Bitmask DP uses the same step: `dp[mask]` comes from `dp[mask & (mask - 1)]`. (Counting Bits, Sort Integers by The Number of 1 Bits.)

66.4.4 Fixed width: reverse the bits

The statement fixes the width (32 bits), and the output moves bit `i` to bit `31 - i`. Use Template 1 with exactly 32 steps: shift `res` left, OR in the last bit of `n`, then shift `n` right. The first bit that you read ends at the top. Memory hook (Reverse Bits): *peel the low bit off n, push it onto res, 32 times.*

```
def reverse_bits(n):
    res = 0
    for _ in range(32):              # fixed width: exactly 32 steps
        res = (res << 1) | (n & 1)   # push the lowest bit of n onto res
        n >>= 1                      # peel that bit off n
    return res

def reverse_bits_wrong(n):           # stops when n is 0: the leading zeros are lost
    res = 0
    while n > 0:
        res = (res << 1) | (n & 1)
        n >>= 1
    return res

reverse_bits(43261596)               # -> 964176192
reverse_bits(2) == 1 << 30           # -> True
reverse_bits_wrong(2)                # -> 1
int(format(43261596, "032b")[::-1], 2)   # -> 964176192
```

For many calls on small fixed-width values, precompute the answer for every 8-bit chunk once. Then each call combines four table entries. This idea answers most "called many times" follow-ups.

```
REV8 = [int(format(b, "08b")[::-1], 2) for b in range(256)]
def reverse_bits_table(n):
    res = 0
    for _ in range(4):               # the bytes from low to high
        res = (res << 8) | REV8[n & 0xFF]
        n >>= 8
    return res
reverse_bits_table(43261596)         # -> 964176192
```

A second fast form swaps blocks: first the 16-bit halves, then the bytes, then the 4-bit, 2-bit and 1-bit blocks. This is divide and conquer on bit blocks, with 5 steps for 32 bits. The same masks count the set bits (SWAR popcount), with an addition of the two blocks in place of the swap.

```
def reverse_bits_swap(n):
    n = ((n >> 16) | (n << 16)) & 0xFFFFFFFF                # swap the 16-bit halves
    n = ((n & 0xFF00FF00) >> 8) | ((n & 0x00FF00FF) << 8)   # swap the bytes
    n = ((n & 0xF0F0F0F0) >> 4) | ((n & 0x0F0F0F0F) << 4)   # swap the 4-bit blocks
    n = ((n & 0xCCCCCCCC) >> 2) | ((n & 0x33333333) << 2)   # swap the 2-bit blocks
    n = ((n & 0xAAAAAAAA) >> 1) | ((n & 0x55555555) << 1)   # swap the single bits
    return n

reverse_bits_swap(43261596)          # -> 964176192
reverse_bits_swap(1) == 1 << 31      # -> True
```

(Reverse Bits, Number Complement, Binary Number with Alternating Bits.)

66.4.5 Find the gap: expected minus actual

You know the full collection before you read the input: all values $0..n$. One value is missing. Compute an aggregate of the expected collection and subtract the aggregate of the actual one. One unknown needs one equation. Memory hook (Missing Number): *expected sum minus actual sum.* The XOR form is the usual optimal answer, and it has the same cost. Seed the accumulator with `n`, then XOR each index and each value. A present value meets its index and cancels. XOR cannot overflow, so it is the safe choice in a fixed-width language.

```
def missing_number(nums):
    n = len(nums)
    expected = n * (n + 1) // 2      # sum of 0, 1, ..., n
    return expected - sum(nums)

def missing_number_xor(nums):
    res = len(nums)                  # the indexes cover 0..n-1, so seed with n
    for i, x in enumerate(nums):
        res ^= i ^ x                 # a present value cancels its index
    return res

missing_number([3, 0, 1]), missing_number([0, 1]), missing_number([1])   # -> (2, 2, 0)
missing_number_xor([9, 6, 4, 2, 3, 5, 7, 0, 1])                          # -> 8
```

Two unknowns need two equations: the sum and the sum of squares, or the XOR of the two values and then a split by one bit. Many unknowns need index marking or a set. (Missing Number, Find the Difference.)

66.4.6 One column at a time: per-bit counts

The answer depends on each bit position separately. Count the ones in each column over all numbers. This takes $O(32n)$ and needs no pairs. The XOR fold is a special case: bit `k` of the single value is the count of column `k` modulo 2.

```
def column_counts(nums, width=32):
    # counts[k] is how many numbers have bit k set
    return [sum((x >> k) & 1 for x in nums) for k in range(width)]

counts = column_counts([4, 1, 2, 1, 2], width=4)
counts                                          # -> [2, 2, 1, 0]
sum((c % 2) << k for k, c in enumerate(counts))    # -> 4
```

Change the rule per column to fit the problem: modulo k when the other values repeat k times, or a count of pairs when the question is about pairs. Memory hook (per-bit counting): *count the ones in each column, then apply the rule to each column.* For a negative value, `(x >> k) & 1` still gives bit `k` of the 32-bit pattern. Rebuild the result as a 32-bit pattern, then convert it with `to_i32`. (Minimum Flips to Make a OR b Equal to c, Largest Combination With Bitwise AND Greater Than Zero.)

66.4.7 Add without plus: XOR sum and AND carry

The statement bans `+` and `-`. Add column by column, like a circuit. The sum of two bits without the carry is their XOR. A carry appears only where both bits are 1, and it moves one column left: `(a & b) << 1`. So $a + b = (a \oplus b) + ((a \,\&\, b) \ll 1)$. Repeat this step until the carry is 0.

Worked example: $5 + 3$. In binary, `101 ^ 011` is 110 (6), and the carry is `001` shifted left, 010 (2). Next, `110 ^ 010` is 100 (4), and the carry is 100 (4). Next, `100 ^ 100` is 0, and the carry is 1000 (8). Next, the sum is 8, and the carry is 0. The result is 8.

In Python, the int has no top bit. When one operand is negative and the true sum is 0 or more, the carry moves left forever. Some mixed-sign inputs still stop, such as -3 and 1. So a few tests can hide this bug. So mask both parts to 32 bits (Template 3). The carry then falls off bit 31, and the loop ends after at most 32 rounds. Convert bit 31 to a sign at the end without a minus: `~(a ^ MASK)`. Memory hook (Sum of Two Integers): *XOR is the sum, AND shifted left is the carry. Repeat until no carry, mask to 32 bits, and fix the sign with* `~(a ^ MASK)`.

```
MASK = 0xFFFFFFFF                    # keep 32 bits, like a C int
def get_sum(a, b):
    while b != 0:
        # sum without carry, and the carry moved one column left; both use the old a
        a, b = (a ^ b) & MASK, ((a & b) << 1) & MASK
    # bit 31 set means negative: flip the low 32 bits, then flip all bits
    return a if a <= 0x7FFFFFFF else ~(a ^ MASK)

get_sum(5, 3), get_sum(-1, 1), get_sum(-3, 5)     # -> (8, 0, 2)
get_sum(-1000, -1000)                             # -> -2000
all(get_sum(a, b) == a + b for a in range(-40, 41) for b in range(-40, 41))   # -> True

def unmasked_rounds(a, b, cap=200):  # the same loop without the mask, stopped at cap
    rounds = 0
    while b != 0 and rounds < cap:
        a, b = a ^ b, (a & b) << 1
        rounds += 1
    return rounds
unmasked_rounds(-3, 1), unmasked_rounds(-1, 1)    # -> (2, 200)
```

The same column-and-carry loop adds binary strings and digit lists. Multiplication without `*` is shift and add. Division without `/` is shift and subtract. (Sum of Two Integers, Add Binary.)

66.4.8 Signs and 32-bit overflow

The statement says that the result must fit a signed 32-bit int, and that no wider type exists. Two rules handle most of these problems. First, put the sign aside and work on the magnitude. Then Python rules for `//` and `%` on negative values do not matter. Second, test for overflow before the step that can overflow, not after it. Solve the step for the old value: `res * 10 + digit <= INT_MAX` becomes `res <= (INT_MAX - digit) // 10`. The digit loop is the base-10 copy of Template 1: pop with `divmod(x, 10)`, push with `res * 10 + digit`. Memory hook (Reverse Integer): *put the sign aside. Pop a digit, solve* `res * 10 + digit <= INT_MAX` *for* `res`*, and push only if it holds.*

```
INT_MAX = 2**31 - 1
def reverse_integer(x):
    sign = -1 if x < 0 else 1        # put the sign aside
    x = abs(x)
    res = 0
    while x:
        x, digit = divmod(x, 10)     # pop the last digit
        if res > (INT_MAX - digit) // 10:   # res * 10 + digit would pass INT_MAX
            return 0
        res = res * 10 + digit       # push the digit
    return sign * res

reverse_integer(123), reverse_integer(-123), reverse_integer(120)   # -> (321, -321, 21)
reverse_integer(1534236469), reverse_integer(-2**31)               # -> (0, 0)
reverse_integer(-2147483412)                                        # -> -2143847412
-123 // 10, -123 % 10               # -> (-13, 7)
```

The usual optimal answer uses the classic two-part guard: `res > INT_MAX // 10`, or `res == INT_MAX // 10` and `digit > 7`. It is equivalent for `res >= 0` and digits 0 to 9. Most C and Java solutions use it. One positive bound is enough for both signs here. No 32-bit input reverses to a magnitude of exactly 2^{31}. In a fixed-width language, keep `x` signed, because `abs(INT_MIN)` does not fit in 32 bits. (Reverse Integer, Palindrome Number.)

66.5 Adapt it to a new problem

Ask these questions in order.

1. Write three or four small inputs in binary, one value in each row. Look down the columns, and look for a carry or a cancel.

2. Does each bit position behave on its own? Then solve one column and repeat for each of the 32 columns. Count the ones per column. Do not compare pairs.
3. Do equal items cancel? Pairs: fold with XOR. Groups of k: column counts modulo k. Two values without a pair: fold, then split by one set bit of the fold.
4. Do you know the full expected collection? Subtract the actual aggregate from the expected aggregate. One unknown needs one equation (sum or XOR). Two unknowns need two equations.
5. Do you need an answer for every number up to n? Fill a table from `i >> 1` or from `i & (i - 1)`.
6. Is the width fixed? Loop exactly `width` times. Build masks with `(1 << w) - 1`. For many calls, use a table over 8-bit chunks.
7. Can a value be negative, or must the result fit in 32 bits? Mask every step with `0xFFFFFFFF` and convert bit 31 at the end. Or put the sign aside and work on magnitudes. Guard each step that can overflow before you do it.
8. Is an operator banned? Copy the circuit or the long-hand method: XOR and AND for addition, shift and add for multiplication, shift and subtract for division.
9. Is the set of items small ($n \leq 20$)? Store a subset as a mask: bit `i` is item `i`. `range(1 << n)` then lists all subsets.
10. Check the bit solution against a brute force on all small values, negative values included.

66.6 Pitfalls

CAUTION

Operator precedence. In Python, `+` and `-` bind tighter than the shifts, and the shifts bind tighter than `&`, `^` and `|`. So `ans[i >> 1] + i & 1` means `(ans[i >> 1] + i) & 1`, and `1 << n - 1` means `1 << (n - 1)`. In Python, comparisons bind looser than all bit operators. In C and Java, `==` binds tighter than `&`, `^` and `|`. Write the parentheses every time.

```
ans, i = [0, 1, 1, 2], 3
ans[i >> 1] + i & 1, ans[i >> 1] + (i & 1)        # -> (0, 2)
1 << 3 - 1                                        # -> 4
2 ^ 3, 2 ** 3                                     # -> (1, 8)
```

CAUTION

`^` is XOR, not power. Use `**` for a power.

CAUTION

Negative values have infinite leading ones. `-1 >> k` is always `-1`, so a "shift until 0" loop never ends. `~x` is `-x - 1`, not a 32-bit flip: use `x ^ 0xFFFFFFFF`. `bin(-5)` is `'-0b101'`, not a two's complement pattern: use `bin(x & 0xFFFFFFFF)`.

```
-1 >> 10                          # -> -1
~5                                # -> -6
5 ^ 0xFFFFFFFF                    # -> 4294967290
bin(-5), bin(-5 & 0xFF)           # -> ('-0b101', '0b11111011')
```

CAUTION

Mask both parts of the addition. If only the carry has the mask, `a` can stay a negative Python int, and the final sign test is wrong. Do not use `a - 2**32` to fix the sign when `-` is banned.

CAUTION

Fixed width. `bin(x)` has a `'0b'` prefix and no leading zeros. Use `format(x, "032b")`. A "while x > 0" loop also drops the leading zeros. Loop exactly 32 times. Shift `res` before the OR, not after it. Otherwise the result is two times too large.

CAUTION

Seeds and identities. Start an XOR fold at 0. In the XOR form of the missing value, start at `n`: the indexes cover only $0..n-1$. Without the seed, `[0, 1]` gives 0 instead of 2.

CAUTION

Floor division on negative values. `-123 // 10` is `-13`, and `-123 % 10` is `7`. C and Java truncate toward zero. Put the sign aside, or use `int(a / b)` and `math.fmod` for small values.

NOTE

`int.bit_count()` needs Python 3.10 or later. Some judges run older versions, and some problems ban built-in bit counts. Know the shift loop and the `n & (n - 1)` loop.

66.7 Problems in this family

Problem	What changes	Memory hook
Single Number	All values occur two times except one; $O(1)$ space. XOR fold.	Pairs cancel under XOR, so XOR everything.
Number of 1 Bits	Count the set bits of one number. Shift loop; `n & (n - 1)` is the optimal form.	Read the last bit, then shift it away.
Counting Bits	A count for every `i` in $0..n$ in $O(n)$. DP from `i >> 1`.	Drop the last bit, look it up, add the bit back.
Reverse Bits	Fixed width of 32 bits; leading zeros count. Peel and push exactly 32 times.	Peel the low bit off n, push it onto res, 32 times.
Missing Number	The full range $0..n$ is known. Expected sum minus actual sum; XOR seeded with `n` is the optimal form.	Expected sum minus actual sum.
Sum of Two Integers	`+` and `-` are banned; negative values. XOR sum, AND carry, 32-bit mask, sign fix.	XOR is the sum, AND shifted left is the carry. Repeat until no carry, mask to 32 bits, and fix the sign with `~(a ^ MASK)`.
Reverse Integer	Base-10 digits, a sign, and no 64-bit values. Guard before each push.	Put the sign aside. Pop a digit, solve `res * 10 + digit <= INT_MAX` for `res`, and push only if it holds.

The transfer drills for this technique are in Section 69.24 (page 427). The answers are in Section A.23 (page 469).

67 Design problems: combining data structures

67.1 The idea

A design problem asks for a class. The class has a constructor and a few methods. The judge calls the methods many times, in any order, and checks each return value. Each method has a cost target. The statement gives it ("$O(1)$ average"), or the call count implies it. A brute-force design keeps one plain list and scans it on every call. That design is correct, but each call costs $O(n)$, and 10^5 calls cost 10^{10} steps.

Start from the operations, not from a structure. Write each method and the access that it needs: by key, by order, by rank, by prefix, or the extreme value. Then, for each access, pick the structure that makes it cheap. When one structure cannot give every access, keep two structures that hold the same items. The second structure removes the scan: it answers in one step the access that the first structure can only answer by a scan. Each change then updates both structures, so they always agree.

Worked example in words (LRU cache). The cache must find a value by key, mark a key as the newest on each use, and drop the oldest key when it is full. A dict finds a key in $O(1)$, but it cannot tell which key is the oldest without a scan. A list keeps an order, but it moves an item out of the middle in $O(n)$. A doubly linked list moves any node in $O(1)$, but only when you already hold that node. So keep a dict from key to node, and keep the nodes in a linked list in order of use. The dict finds the node. The list moves the node to the newest end. Eviction removes the node at the oldest end. For example, take capacity 2. Put key 1, then put key 2. The order is 1, 2. A get of key 1 moves it to the newest end, so the order is 2, 1. A put of key 3 makes the cache too full, so it evicts key 2 from the oldest end. Each step is $O(1)$, and no step scans the keys. The Python `OrderedDict` is exactly this pair, written in C.

67.1.1 Choose a structure from the access

Access that a method needs	Structure	Cost per call
Find, add or delete by key; membership	`dict`, `set`	$O(1)$ average
Count equal items, duplicates allowed	`Counter` (a dict of counts)	$O(1)$
An aggregate (min, max, sum) of a stack; undo the last change	list of pairs (value, aggregate so far)	$O(1)$
Order of use; move any key to one end; drop the other end	`OrderedDict`, or dict + doubly linked list	$O(1)$
Add and remove at both ends (FIFO, LIFO)	`deque`	$O(1)$
Smallest or largest item; removal only at the top	heap (`heapq`)	$O(\log n)$ add, $O(1)$ peek
Smallest or largest item; items change or leave anywhere	a dict as the truth, plus a heap (or two) with lazy deletion: on a read, pop each top that disagrees with the dict	$O(\log n)$ amortized
Counts that change by ± 1; a key with the smallest and with the largest count	doubly linked list of count buckets (sets of keys) in count order, plus a dict from key to bucket; a change moves the key to the next bucket or to the previous bucket	$O(1)$
The k-th largest of all items so far	min-heap that keeps k items	$O(\log k)$
The median or another rank that moves with n	two heaps: max-heap of the low half, min-heap of the high half	$O(\log n)$
The first k items of many sorted sources	heap with one cursor per source	$O(\log F)$ per item
Latest item at or before x; items arrive in sorted order	list per key + `bisect`	$O(1)$ add, $O(\log n)$ query
Ordered queries; items arrive in any order	sorted list + `insort`, or a balanced tree	$O(n)$ or $O(\log n)$ add
Prefix or wildcard queries on strings	trie (dict children + end flag)	$O(L)$ per insert or prefix walk
Shapes or tuples through a query point	`Counter` of points; enumerate the free corner	$O(d)$ per query
Store or send a list or a tree as one string	length prefix, or tokens with a null marker	$O(n)$

Here F is the number of sources, L is the word length, and d is the number of distinct points.

NOTE

Extremes over counts that change. All O'one Data Structure (LeetCode 432) asks for `inc`, `dec`, a key with the largest count and a key with the smallest count, each in $O(1)$. A heap version passes in practice: a dict from key to count, two heaps of (count, key), and lazy deletion (see the section on heaps and priority queues). The bucket list meets the $O(1)$ target. Each bucket holds the keys that have one count, and the list keeps only the buckets that are not empty. A change of ± 1 moves the key to the next bucket or to the previous bucket. Create that bucket when it is missing, and unlink each bucket that becomes empty. The first bucket then holds the smallest count, and the last bucket holds the largest count.

67.2 Recognize it

- The statement says "Design" or "Implement the class", and it lists a constructor and methods.
- The example input is two parallel lists: the method names and the argument lists.
- Each call has a cost target: $O(1)$, $O(1)$ average, or $O(\log n)$. Or the call count (for example $2 \cdot 10^5$) makes a scan per call too slow.
- Writes and reads interleave. The class must answer after each change, so one sort at the end does not work.
- The same items need two kinds of access, for example by key and by order. This points to two structures that

index the same items.

- Typical words: cache, capacity, evict, recently used, stream, timestamp, "as of time t", feed, most recent, median, k-th largest, prefix, autocomplete, wildcard.
- Codec words: "one string", "send over the network", "rebuild the original". The decoder sees only the string, so the string must carry every boundary.
- Point words: "add a point", "duplicate points count", "the number of ways to form" a square or a rectangle with the query point. The answer is a count, not a list, so a `Counter` of points is enough.
- A small bound in the statement: at most 2 wildcards, coordinates up to 1000, or a few thousand calls. A small bound allows a direct enumeration per query, so check it before you build a complex structure.

67.3 Core template

Use the same five steps for every design problem.

1. Write a table of the operations: the method, what it reads, what it changes, and the target cost.
2. For each access, pick a structure from the table above. Prefer the standard library: `dict`, `Counter`, `defaultdict`, `OrderedDict`, `deque`, `heapq`, `bisect`.
3. If two structures hold the same items, write the invariant that ties them. Decide what each item stores to find itself in the other structure: a key in a node, or an index in a map.
4. Put each kind of change in one helper that updates every structure. The public methods call the helpers. Reads change nothing.
5. Replay the example calls. Then compare the class with a slow, obviously correct model on many random calls.

```
class Design:
    def __init__(self):
        self.by_key = {}          # access 1: find an item by key
        self.order = ...          # access 2: order, rank, prefix or extreme
        # invariant: by_key and order always hold the same items
    def _add(self, item): ...     # updates by_key AND order
    def _remove(self, item): ...  # updates by_key AND order
    def query(self, x): ...       # reads only; creates no entries
```

The block below applies the steps to the LRU cache. `replay` runs an example that is given as two lists: the method names and the argument lists. `agrees` runs random calls on the fast class and on a slow model. The model check finds bugs that the example misses. For example, a `put` on an existing key must count as a use, but the example never puts an existing key.

```
import random
from collections import OrderedDict

class LRUCache:
    def __init__(self, capacity):
        self.capacity = capacity
        self.cache = OrderedDict()          # oldest key first, newest key last
    def get(self, key):
        if key not in self.cache:
            return -1
        self.cache.move_to_end(key)         # a get is a use
        return self.cache[key]
    def put(self, key, value):
        self.cache[key] = value
        self.cache.move_to_end(key)         # a put is a use
        if len(self.cache) > self.capacity:
            self.cache.popitem(last=False)  # evict the oldest key

def replay(cls, methods, args):
    obj = cls(*args[0])                     # methods[0] is the constructor
    return [None] + [getattr(obj, m)(*a) for m, a in zip(methods[1:], args[1:])]

methods = ["LRUCache", "put", "put", "get", "put", "get", "put", "get", "get", "get"]
args = [[2], [1, 1], [2, 2], [1], [3, 3], [2], [4, 4], [1], [3], [4]]
```

```
replay(LRUCache, methods, args)
# -> [None, None, None, 1, None, -1, None, -1, 3, 4]

class SlowLRU:                              # obviously correct, O(n) per call
    def __init__(self, capacity):
        self.capacity, self.items = capacity, []     # (key, value), oldest first
    def get(self, key):
        for i, (k, v) in enumerate(self.items):
            if k == key:
                self.items.append(self.items.pop(i))
                return v
        return -1
    def put(self, key, value):
        self.items = [(k, v) for k, v in self.items if k != key] + [(key, value)]
        if len(self.items) > self.capacity:
            self.items.pop(0)

def agrees(fast_cls, slow_cls, init_args, make_call, rounds=300, calls=30, seed=1):
    rng = random.Random(seed)
    for _ in range(rounds):
        fast, slow = fast_cls(*init_args), slow_cls(*init_args)
        for _ in range(calls):
            name, args = make_call(rng)
            if getattr(fast, name)(*args) != getattr(slow, name)(*args):
                return False
    return True

def lru_call(rng):
    if rng.random() < 0.5:
        return "get", (rng.randint(0, 4),)
    return "put", (rng.randint(0, 4), rng.randint(0, 9))

all(agrees(LRUCache, SlowLRU, (cap,), lru_call) for cap in (1, 2, 3))   # -> True

class StalePutLRU(LRUCache):                # bug: a put on an existing key is not a use
    def put(self, key, value):
        self.cache[key] = value             # an existing key keeps its old place
        if len(self.cache) > self.capacity:
            self.cache.popitem(last=False)

replay(StalePutLRU, methods, args) == replay(LRUCache, methods, args)   # -> True
agrees(StalePutLRU, SlowLRU, (2,), lru_call)   # -> False
```

Invariant. The `OrderedDict` holds at most `capacity` keys in order of last use. The first key is the least recently used key, and the last key is the most recently used key.

67.4 Variations

67.4.1 Store the aggregate with each element

What changes. A stack must also return its minimum (or maximum, or sum) in $O(1)$. One variable for the minimum fails: when the minimum leaves, the variable cannot give the old minimum. The minimum has no inverse, so arithmetic cannot undo it. But a stack removes only the top, so each pop returns the stack to an earlier state exactly. So store, with each element, the answer for the stack up to that element. **Min Stack** uses one list of pairs (value, minimum so far). Memory hook: *Push the pair (value, min so far). Pop removes both.*

```
class MinStack:
    def __init__(self):
        self.stack = []                     # (value, min of this value and all below it)
    def push(self, val):
        low = min(val, self.stack[-1][1]) if self.stack else val
        self.stack.append((val, low))
```

```
    def pop(self):
        self.stack.pop()
    def top(self):
        return self.stack[-1][0]
    def getMin(self):
        return self.stack[-1][1]

s = MinStack()
for v in [5, 2, 7, 2]:
    s.push(v)
s.getMin()                                  # -> 2
s.pop(); s.pop(); s.top(), s.getMin()       # -> (2, 2)
s.pop(); s.top(), s.getMin()                # -> (5, 5)
```

The same idea works for any aggregate that grows by one element: max, sum, gcd, a count. An undo operation is then a pop. A queue with a minimum uses two such stacks: push to an in-stack, pop from an out-stack, and move all items when the out-stack is empty. For the maximum of a sliding window (FIFO), use a monotonic deque instead. A common alternative keeps two parallel lists. It has the same cost, but each pop must remove from both lists.

67.4.2 Find by key and keep an order of use

What changes. Lookup by key, a move of any key to the newest end, and eviction at the oldest end, all in $O(1)$. **LRU Cache** uses an `OrderedDict` (see the core template). Memory hook: *Ordered dict: move to end on use, pop the front on overflow.* Write the structure by hand when `OrderedDict` is not allowed, or when the task asks for the linked list itself. Keep a dict from key to node and a doubly linked list between two sentinel nodes. The sentinels remove all checks for an empty list and for the ends. Hook for this form: the map finds the node, the list keeps the order; remove, then insert at the right; evict `left.next`.

```
class Node:
    def __init__(self, key=0, val=0):
        self.key, self.val = key, val       # the key lets eviction clean the map
        self.prev = self.next = None

class LRUCache:
    def __init__(self, capacity):
        self.cap, self.map = capacity, {}   # map: key to node
        self.left, self.right = Node(), Node()      # sentinels: left.next is the LRU
        self.left.next, self.right.prev = self.right, self.left
    def _remove(self, node):
        node.prev.next, node.next.prev = node.next, node.prev
    def _insert(self, node):                # insert just before the right sentinel
        prev = self.right.prev
        prev.next, node.prev, node.next, self.right.prev = node, prev, self.right, node
    def get(self, key):
        if key not in self.map:
            return -1
        node = self.map[key]
        self._remove(node)
        self._insert(node)
        return node.val
    def put(self, key, value):
        if key in self.map:
            self._remove(self.map[key])     # unlink the old node: no ghost node stays
        node = self.map[key] = Node(key, value)
        self._insert(node)
        if len(self.map) > self.cap:
            lru = self.left.next
            self._remove(lru)
            del self.map[lru.key]
```

```
c = LRUCache(2)
c.put(1, 1); c.put(2, 2); c.get(1)          # -> 1
c.put(3, 3); c.get(2)                       # -> -1
c.put(4, 4); c.get(1), c.get(3), c.get(4)   # -> (-1, 3, 4)
```

NOTE

Each node stores its key. Eviction holds only the node, so the key in the node is the only way to delete the map entry. In general, when two structures index the same items, each item carries the data to find itself in the other structure. Do not replace `OrderedDict` with a plain dict and `next(iter(d))`: in CPython that eviction is not $O(1)$.

67.4.3 Find by key, then search an ordered timeline

What changes. The query is "the latest value at or before time t". Split it into two parts. The key is an exact match, so a hash map handles it. The timestamp is a range match, so it needs a sorted list and binary search. In **Time Based Key-Value Store**, the timestamps of `set` strictly increase. So an append keeps each list sorted for free, and `set` is $O(1)$. `get` is a floor query: `i = bisect_right(times, t)` is the count of timestamps $\leq t$, so the answer is at index $i - 1$. Memory hook: *Two lists per key: times and values.* ***`bisect_right`*** *counts times* $\leq t$*, so take the value one left.*

```
from bisect import bisect_right
from collections import defaultdict

class TimeMap:
    def __init__(self):
        self.times = defaultdict(list)      # key to timestamps, increasing
        self.values = defaultdict(list)     # key to values, same order as times
    def set(self, key, value, timestamp):
        self.times[key].append(timestamp)   # increasing timestamps: append keeps the order
        self.values[key].append(value)
    def get(self, key, timestamp):
        times = self.times.get(key, [])     # .get adds no entry for a missing key
        i = bisect_right(times, timestamp)  # i = count of times <= timestamp
        return self.values[key][i - 1] if i > 0 else ""

tm = TimeMap()
tm.set("foo", "bar", 1)
tm.get("foo", 1), tm.get("foo", 3)          # -> ('bar', 'bar')
tm.set("foo", "bar2", 4)
tm.get("foo", 4), tm.get("foo", 5), tm.get("foo", 0)   # -> ('bar2', 'bar2', '')
tm.get("nope", 9), "nope" in tm.times       # -> ('', False)
```

Always check whether the input order already sorts the data. If it does, a plain list replaces a balanced tree. If the timestamps can arrive in any order, use `bisect.insort` ($O(k)$ per insert) or a sorted container. When the task asks for the search by hand, write it on (timestamp, value) pairs. The cost is the same. Its loop is: while `lo <= hi`, if the time at `mid` is $\leq t$, save the value and set `lo = mid + 1`; else set `hi = mid - 1`.

67.4.4 Keep only the k best: one heap of size k

What changes. Items only arrive, and each query asks for the k-th largest item so far. The answer never decreases. So a value below the current answer can never be the answer again, and you can drop it. **Kth Largest Element in a Stream** keeps a min-heap of the k largest values. The root is the weakest kept value, which is the k-th largest. Memory hook: *Keep the top k in a min-heap. Every value goes through add: push, pop if over k, return the root.*

```
import heapq

class KthLargest:
    def __init__(self, k, nums):
        self.k, self.top_k = k, []          # min-heap of the k largest values so far
        for num in nums:
            self.add(num)                   # every value goes through the same gate
```

```
    def add(self, val):
        heapq.heappush(self.top_k, val)
        if len(self.top_k) > self.k:
            heapq.heappop(self.top_k)       # the weakest value can never be the answer again
        return self.top_k[0]

kl = KthLargest(3, [4, 5, 8, 2])
[kl.add(v) for v in [3, 5, 10, 9, 4]]       # -> [4, 5, 5, 8, 8]
KthLargest(1, []).add(-3)                   # -> -3
```

The heap has the opposite polarity of the request: a min-heap for "largest", a max-heap for "smallest". The root is the threshold that a new value must beat. An alternative constructor calls `heapify` on a copy of the list and then pops down to k values. The cost per `add` is the same. If items can also leave, a size-k heap is not enough, because a dropped value can become the answer again.

67.4.5 Two heaps that face each other at a moving rank

What changes. The query asks for the median, a rank that moves as n grows. The median depends only on the boundary between the low half and the high half. **Find Median from Data Stream** keeps the low half in a max-heap `small` (negated values) and the high half in a min-heap `large`. The two tops face each other at the boundary. Memory hook: *Push into small, pass its top to large, give one back if large gets bigger.*

```
import heapq, random

class MedianFinder:
    def __init__(self):
        self.small = []                     # max-heap of the low half (negated values)
        self.large = []                     # min-heap of the high half
    def addNum(self, num):
        heapq.heappush(self.small, -num)                          # enter through the low half
        heapq.heappush(self.large, -heapq.heappop(self.small))    # pass its top up
        if len(self.large) > len(self.small):
            heapq.heappush(self.small, -heapq.heappop(self.large))   # give one back
    def findMedian(self):
        if len(self.small) > len(self.large):
            return float(-self.small[0])
        return (-self.small[0] + self.large[0]) / 2

mf, out = MedianFinder(), []
for v in [5, 15, 1, 3]:
    mf.addNum(v)
    out.append(mf.findMedian())
out                                         # -> [5.0, 10.0, 5.0, 4.0]

rng, mf, seen, ok = random.Random(7), MedianFinder(), [], True
for _ in range(200):                        # compare with a sort after each insert
    v = rng.randint(-5, 5)
    mf.addNum(v)
    seen.append(v)
    s, n = sorted(seen), len(seen)
    ok = ok and mf.findMedian() == (s[n // 2] if n % 2 else (s[n // 2 - 1] + s[n // 2]) / 2)
ok                                          # -> True
```

Invariants. Every value in `small` is $\le$ every value in `large`. `len(small)` equals `len(large)` or `len(large) + 1`. The route through `small` needs no comparison with either top: the value that moves up is always the largest low value. For a percentile p, keep `small` at $\lceil pn \rceil$ values. If values can also leave (a sliding window), add lazy deletion or use a sorted container.

67.4.6 Merge many sorted sources with a heap of cursors

What changes. The read combines many lists, and it needs only the first few items of the merge. **Design Twitter** stamps each tweet with a global clock, because tweet IDs do not show the post order. Each user has an append-only list of (time, tweetId), so each list is sorted for free. Each user has a set of followees, so follow and unfollow are $O(1)$ and repeated calls are harmless. The feed seeds a heap with the newest tweet of each relevant user. It pops the newest tweet, then pushes the next older tweet of the same user, and stops at 10. The heap holds at most one cursor per user, and the loop stops after 10 pops, so the old tweets are never read. Memory hook: *Stamp each tweet with a global clock. Merge the timelines with a heap: one cursor per user, pop the newest, push that user's previous tweet, stop at ten.*

```
import heapq
from collections import defaultdict

class Twitter:
    def __init__(self):
        self.time = 0                       # global clock: one order across all users
        self.tweets = defaultdict(list)     # user to [(time, tweetId)], oldest first
        self.following = defaultdict(set)   # user to the set of followees
    def postTweet(self, userId, tweetId):
        self.tweets[userId].append((self.time, tweetId))
        self.time += 1
    def getNewsFeed(self, userId):
        heap = []
        for uid in self.following.get(userId, set()) | {userId}:   # temporary union: no side effect
            ts = self.tweets.get(uid)
            if ts:
                heap.append((-ts[-1][0], ts[-1][1], uid, len(ts) - 1))   # one cursor per user
        heapq.heapify(heap)
        feed = []
        while heap and len(feed) < 10:
            _, tid, uid, i = heapq.heappop(heap)
            feed.append(tid)
            if i > 0:                       # push the next older tweet of the same user
                t, prev_id = self.tweets[uid][i - 1]
                heapq.heappush(heap, (-t, prev_id, uid, i - 1))
        return feed
    def follow(self, followerId, followeeId):
        if followerId != followeeId:
            self.following[followerId].add(followeeId)
    def unfollow(self, followerId, followeeId):
        self.following[followerId].discard(followeeId)     # no error if absent

tw = Twitter()
tw.postTweet(1, 5); tw.getNewsFeed(1)       # -> [5]
tw.follow(1, 2); tw.postTweet(2, 6); tw.getNewsFeed(1)  # -> [6, 5]
tw.unfollow(1, 2); tw.getNewsFeed(1)        # -> [5]
for tid in range(100, 112):
    tw.postTweet(3, tid)
tw.getNewsFeed(3) == list(range(111, 101, -1))         # -> True
```

A feed costs $O(F + 10 \log F)$ for F sources. The cost depends on the sources and the output size, not on the total data. The standard library has the same lazy merge: `heapq.merge` with `reverse=True` merges lists that are each sorted from newest to oldest.

```
from heapq import merge
from itertools import islice

def feed(tweets, users, k=10):              # tweets[u]: (time, id) pairs, oldest first
    newest_first = (reversed(tweets[u]) for u in users if u in tweets)
    return [tid for _, tid in islice(merge(*newest_first, reverse=True), k)]
```

```
tweets = {1: [(0, 5), (3, 8)], 2: [(1, 6), (2, 7)]}
feed(tweets, [1, 2])                          # -> [8, 7, 6, 5]
feed(tweets, [1, 2], k=3), feed(tweets, [9])   # -> ([8, 7, 6], [])
```

67.4.7 Prefix tree classes

What changes. Many stored strings, and queries about prefixes or about letters at fixed positions. **Implement Trie (Prefix Tree)** stores each word as a path. Each node stands for one prefix. A node has a dict of children and an end flag. One `_walk` helper serves `search` and `startsWith`; only the final check differs. Memory hook: *One node per prefix: walk the letters, flag the word end.* A wildcard `.` that matches one letter turns the walk into a DFS: a letter follows one edge, a dot follows every edge.

```
class TrieNode:
    def __init__(self):
        self.children = {}                  # letter to child node
        self.is_end = False

class Trie:
    def __init__(self):
        self.root = TrieNode()
    def insert(self, word):
        node = self.root
        for ch in word:
            if ch not in node.children:
                node.children[ch] = TrieNode()
            node = node.children[ch]
        node.is_end = True
    def _walk(self, s):                     # the node of prefix s, or None; creates nothing
        node = self.root
        for ch in s:
            node = node.children.get(ch)
            if node is None:
                return None
        return node
    def search(self, word):
        node = self._walk(word)
        return node is not None and node.is_end
    def startsWith(self, prefix):
        return self._walk(prefix) is not None
    def match(self, pattern):               # "." matches exactly one letter
        def dfs(i, node):
            if i == len(pattern):
                return node.is_end
            if pattern[i] == ".":
                return any(dfs(i + 1, child) for child in node.children.values())
            child = node.children.get(pattern[i])
            return child is not None and dfs(i + 1, child)
        return dfs(0, self.root)

t = Trie()
t.insert("apple")
t.search("apple"), t.search("app"), t.startsWith("app")   # -> (True, False, True)
t.insert("app")
t.search("app"), t.startsWith("apple")      # -> (True, True)
t.match("a.p"), t.match("..ple")           # -> (True, True)
t.match(".pl"), t.match("ap.")             # -> (False, True)
```

Design Add and Search Words Data Structure allows at most two dots per query and words of length at most 25. The intuitive answer needs no trie. On add, store the word and each copy of it with one letter masked by a dot. A query with at most one dot is then one set lookup. A query with two dots fills its first dot with each letter and searches again. Memory hook: *Mask one letter on add, fill one dot on search.*

```
class WordDictionary:
    def __init__(self):
        self.patterns = set()               # each word, plus each copy with one letter masked
    def addWord(self, word):
        self.patterns.add(word)
        for i in range(len(word)):
            self.patterns.add(word[:i] + "." + word[i + 1:])
    def search(self, word):
        if word.count(".") <= 1:
            return word in self.patterns
        i = word.index(".")
        return any(self.search(word[:i] + c + word[i + 1:])
                   for c in "abcdefghijklmnopqrstuvwxyz")

wd = WordDictionary()
for w in ["bad", "dad", "mad"]:
    wd.addWord(w)
wd.search("pad"), wd.search("bad")          # -> (False, True)
wd.search(".ad"), wd.search("b..")          # -> (True, True)
wd.search("..."), wd.search("b."), wd.search("..d.")    # -> (True, False, False)
```

An add costs $O(L^2)$. A search with $k \geq 1$ dots costs 26^{k-1} lookups, so the pattern set suits only a small dot limit. The standard answer is the trie with the DFS (the `match` method above). Use it when the question asks for a trie, when you also need prefix queries, or when the dot limit is large.

67.4.8 Count shapes with a point counter

What changes. The class stores points, duplicates count, and a query counts the shapes that have the query point as a corner. Fix the query corner, then enumerate the one free choice that fixes the whole shape. In **Detect Squares**, the free choice is the diagonal corner (x, y) with $x \neq q_x$ and $|x - q_x| = |y - q_y|$. The other two corners are forced: (x, q_y) and (q_x, y). The number of ways is the product of the three counts. Memory hook: *Pick the diagonal, the other two corners are forced, multiply the counts.*

```
from collections import Counter

class DetectSquares:
    def __init__(self):
        self.points = Counter()             # (x, y) to the number of copies
    def add(self, point):
        self.points[tuple(point)] += 1      # a list is not hashable
    def count(self, point):
        qx, qy = point
        total = 0
        for (x, y), copies in self.points.items():
            if x == qx or abs(x - qx) != abs(y - qy):
                continue                    # not the diagonal corner of a square with area
            total += copies * self.points.get((x, qy), 0) * self.points.get((qx, y), 0)
        return total

ds = DetectSquares()
for p in [[3, 10], [11, 2], [3, 2]]:
    ds.add(p)
ds.count([11, 10]), ds.count([14, 8])       # -> (1, 0)
ds.add([11, 2])
ds.count([11, 10])                          # -> 2
```

`add` is $O(1)$ and `count` is $O(d)$ for d distinct points. The general move: find the fewest elements that fix the shape, enumerate them, and look up the rest in a hash map. Two Sum is the one-dimensional case. Count rectangles by enumerating diagonal pairs.

67.4.9 Codec classes: flatten and rebuild

What changes. Two methods: one flattens a structure to a string, and the other rebuilds it from the string only. The encoding must be self-delimiting: the decoder must know where each part ends. **Encode and Decode Strings** allows any character in the data, so no delimiter is safe. Write each string as its length, then `#`, then the string. The header holds only digits, so the first `#` at or after the read position ends the header. The payload is skipped by count, so it can contain `#`. Memory hook: *Write length, hash, string; read length, then jump.*

```
def encode(strs):
    return "".join(f"{len(s)}#{s}" for s in strs)   # each item: length, "#", payload

def decode(s):
    res, i = [], 0
    while i < len(s):
        j = s.index("#", i)                 # the first "#" from i ends the header
        length = int(s[i:j])
        start = j + 1
        res.append(s[start:start + length])
        i = start + length                  # jump over the payload
    return res

encode(["a#b", ""])                         # -> '3#a#b0#'
cases = [["neet", "code"], [""], [], ["", ""], ["#", "12#", "a" * 12]]
all(decode(encode(c)) == c for c in cases)  # -> True
"#".join(["a#b"]) == "#".join(["a", "b"])   # -> True
```

The last line shows why a bare delimiter fails: two different lists give the same string.

Serialize and Deserialize Binary Tree must keep the shape, not only the values. Write a null marker `N` for each empty child. The intuitive form follows the example list: level order with a queue. The queue also holds the `None` children. To decode, each real node in the queue takes the next two tokens as its children. Memory hook: *Write it level by level like the example list, with N for every null; each real node in the queue takes the next two tokens as its kids.*

```
from collections import deque

class TreeNode:
    def __init__(self, x):
        self.val, self.left, self.right = x, None, None

class Codec:
    def serialize(self, root):
        out, queue = [], deque([root])
        while queue:
            node = queue.popleft()
            if node is None:
                out.append("N")             # the null marker keeps the shape
                continue
            out.append(str(node.val))
            queue.append(node.left)         # the queue also holds None children
            queue.append(node.right)
        return ",".join(out)
    def deserialize(self, data):
        tokens = iter(data.split(","))
        def make(tok):
            return None if tok == "N" else TreeNode(int(tok))
        root = make(next(tokens))
        queue = deque([root])
        while queue:
            node = queue.popleft()
            if node is None:
                continue                    # a null has no children to read
```

```
            node.left = make(next(tokens))  # the next two tokens are its children
            node.right = make(next(tokens))
            queue.append(node.left)
            queue.append(node.right)
        return root

c = Codec()
s = "1,2,3,N,N,4,5,N,N,N,N"
c.serialize(c.deserialize(s)) == s          # -> True
c.serialize(None), c.deserialize("N") is None   # -> ('N', True)
root = TreeNode(1)
root.left = TreeNode(-20)
c.serialize(root)                           # -> '1,-20,N,N,N'
```

The standard optimal answer uses preorder DFS with the same null markers. It is shorter, and its grammar is "tree = N, or a value, a tree and a tree". The decoder is the same recursion as the encoder, and it reads tokens in the same order. A skewed tree of 10^4 nodes goes past the default recursion limit of 1000, so raise the limit or use the level-order form.

```
class TreeNode:
    def __init__(self, x):
        self.val, self.left, self.right = x, None, None

def serialize(root):
    out = []
    def dfs(node):
        if node is None:
            out.append("N")
            return
        out.append(str(node.val))
        dfs(node.left)
        dfs(node.right)
    dfs(root)
    return ",".join(out)

def deserialize(data):
    tokens = iter(data.split(","))
    def build():                            # grammar: tree = N | value tree tree
        tok = next(tokens)
        if tok == "N":
            return None
        node = TreeNode(int(tok))
        node.left = build()                 # left first: the same order as the writer
        node.right = build()
        return node
    return build()

s = "1,2,N,N,3,4,N,N,5,N,N"
serialize(deserialize(s)) == s              # -> True
serialize(None)                             # -> 'N'
```

Test every codec with a round trip on hard inputs: an empty list, empty strings, the marker inside the data, multi-digit lengths, negative values, and the empty tree.

67.5 Adapt it to a new problem

Ask these questions in this order.

1. What are the methods? For each one, what does it read, what does it change, and how often is it called?
2. What is the cost target? Use the stated bound, or divide about 10^7 simple steps by the number of calls.
3. What access does each method need? By key, by order of arrival or use, by rank (k-th, median), by extreme

(min, max), by prefix, or by a floor on time?
4. Where do removals happen? Never, only at the top (stack), only at the front (queue), at the extreme (heap), or anywhere (dict + linked list, or lazy deletion)?
5. Does the input order already sort the data? Increasing timestamps or a global counter make an append-only list sorted.
6. Does a query need only a small part of the data, such as the top k, the boundary of two halves, or one cursor per source? Keep only that part.
7. Does one structure give every access? If not, combine two. Write the invariant that ties them. Decide what each item stores to find itself in the other structure.
8. Does the class flatten data to a string? Then ask what tells the decoder where each part ends: a length prefix, or a null marker for each empty slot.
9. Route each change through one helper that updates every structure. Make each read free of side effects.
10. Test the edge cases: an empty structure, a missing key, duplicates, capacity 1, ties, the value 0, the first call. Then compare with a slow model on random calls (see the core template).

67.6 Pitfalls

CAUTION

The structures drift apart. Wrong: remove a node from the list but leave its key in the map, or create a new node for an existing key and leave the old node in the list. The old node is a ghost: its later eviction deletes the map entry of the live node. Right: put each change in one helper that updates every structure, and store in each item the key that finds it in the other structure.

CAUTION

A use that is not recorded. In an LRU cache, a successful `get` is a use, and a `put` on an existing key is a use. A `put` on an existing key updates the value and does not evict. A `get` on a missing key changes nothing. Insert first, then evict only when the size is larger than the capacity. This order is safe only because the new key is the newest key, so eviction never removes it. If another rule picks the key to evict, check that the new key cannot be that key.

CAUTION

A read that writes. Indexing a `defaultdict` with a missing key creates an entry. A later membership test then passes, the memory grows, and a loop over the same dict raises `RuntimeError`. Read with `.get(key, default)`. A `Counter` returns 0 for a missing key and does not insert it. Also, 0 and the empty string are legal values: test membership with `in`, not with truthiness.

```
from collections import Counter, defaultdict
d = defaultdict(list)
len(d["missing"]), "missing" in d           # -> (0, True)
d.get("other", []), "other" in d            # -> ([], False)
c = Counter()
c["x"], "x" in c                            # -> (0, False)
pts = defaultdict(int, {(0, 0): 1})
for p in pts:
    pts[(p[0] + 1, p[1])]                   # -> raises RuntimeError
cache = {"k": 0}
bool(cache.get("k")), "k" in cache          # -> (False, True)
```

CAUTION

State shared between objects. A container in the class body, or a mutable default argument, is one object for every instance. Create each container in `__init__`. A codec must keep its state local to each call, because the judge can use one object to encode and another object to decode.

```
class Bad:
    items = []                              # one list for every instance
    def add(self, x):
        self.items.append(x)
a, b = Bad(), Bad()
```

```
a.add(1)
b.items                                  # -> [1]
```

CAUTION

A hidden $O(n)$ step. `min(self.stack)` inside `getMin`, `list.pop(0)`, a sort after each insert, `s = s[j:]` in a decode loop, and `next(iter(d))` for eviction from a plain dict all look short. Each one costs $O(n)$ or more per call. Move an index instead of a slice, and use `deque` for a queue.

CAUTION

Heap details. `heapq` is a min-heap: negate the keys for a max-heap, on push and on pop. Tuples compare field by field. If the first fields can tie, put a unique counter before any field that cannot compare, such as a node. For the median of an even count, divide with `/`, not `//`: the median of 2 and 3 is 2.5.

```
import heapq
class Node:
    pass
h = []
heapq.heappush(h, (1, Node()))
heapq.heappush(h, (1, Node()))           # -> raises TypeError
h = []
heapq.heappush(h, (1, 0, Node()))
heapq.heappush(h, (1, 1, Node()))        # the counter breaks the tie
len(h), (2 + 3) / 2, (2 + 3) // 2        # -> (2, 2.5, 2)
```

CAUTION

Ambiguous encodings. A bare delimiter fails when the data can contain it. A separator between items cannot tell `[]` from `[""]`: give every item its own header. A tree without null markers loses its shape: `[1, 2]` and `[1, null, 2]` give the same values. Preorder plus inorder without markers fails when values repeat.

CAUTION

Names and signatures. The judge calls the methods by the exact names of the statement, often in camelCase: `getMin`, `startsWith`, `addNum`, `getNewsFeed`. A snake_case name fails.

67.7 Problems in this family

Problem	What changes	Memory hook
Min Stack	Stack with an $O(1)$ minimum; store the minimum so far with each element.	Push the pair (value, min so far). Pop removes both.
LRU Cache	Lookup by key plus an order of use; evict the oldest key. `OrderedDict`, or dict + doubly linked list.	Ordered dict: move to end on use, pop the front on overflow.
Time Based Key-Value Store	Lookup by key, then a floor search on timestamps that increase.	Two lists per key: times and values. `bisect_right` counts times $\leq t$, so take the value one left.
Design Twitter	Merge per-user timelines; the first 10 by a global clock; a heap of cursors.	Stamp each tweet with a global clock. Merge the timelines with a heap: one cursor per user, pop the newest, push that user's previous tweet, stop at ten.
Find Median from Data Stream	A rank that moves with n; two heaps meet at the boundary.	Push into small, pass its top to large, give one back if large gets bigger.
Implement Trie (Prefix Tree)	Prefix queries; one node per prefix with an end flag.	One node per prefix: walk the letters, flag the word end.
Design Add and Search Words Data Structure	Wildcard letters; a set of masked patterns (or a trie DFS).	Mask one letter on add, fill one dot on search.

Problem	What changes	Memory hook
Kth Largest Element in a Stream	A fixed rank k, inserts only; a min-heap of size k.	Keep the top k in a min-heap. Every value goes through add: push, pop if over k, return the root.
Detect Squares	Count shapes through a query point; a `Counter` of points.	Pick the diagonal, the other two corners are forced, multiply the counts.
Encode and Decode Strings	Flatten a list of any strings; a length prefix per item.	Write length, hash, string; read length, then jump.
Serialize and Deserialize Binary Tree	Flatten a tree; a null marker for each empty child.	Write it level by level like the example list, with N for every null; each real node in the queue takes the next two tokens as its kids.

The transfer drills for this technique are in Section 69.25 (page 427). The answers are in Section A.24 (page 471).

Part IV
Retention

68 Recall sheet

This sheet has one row for each section of Part III, in the same order. Memory hooks are in italics. Cover the three right columns, recall them, then uncover the row and check. Read the section again for each row that you miss. For the reverse drill, cover the left column and name the technique from the signals.

Technique	Signals	Core move	Invariant or memory hook
Hashing: counting, membership and canonical keys	A pair with an exact sum, and the output is indexes. Anagrams, or groups equal up to one detail. Consecutive runs in $O(n)$.	Replace the inner search loop with one lookup. Ask for the partner, then record. To group, use a canonical key.	The map holds only the items before j. *Ask the map for the partner, then add yourself.*
Prefix sums, running aggregates and offsets	Count ranges with an exact sum and negative values. "All except `i`" with no division. "Buy before sell". Many range updates, one read.	Write a range as `P[r + 1] - P[l]`. Sweep: seed, extend, ask, add. Keep a count map or a running minimum of the past.	`P[0] = 0` is the empty prefix. Ask before you add. *Two Sum on prefix sums: look up prefix minus k, then store prefix.*
Two pointers	Sorted or sortable input, and a pair or a triple with a target sum. A mirror check. A list cycle or middle. Remove in place.	Start at both ends. Each step discards an end that cannot be in a better answer. Fast and slow pointers on a list. Read and write pointers.	Every answer pair stays inside `[lo, hi]`. *Too small, move left up. Too big, move right down. Move the shorter wall in.*
Sliding window	One contiguous segment: the longest, the shortest or the count. A monotone rule: "at most k", "contains all of". A fixed length.	Grow the right end. Longest: shrink the left while invalid. Shortest: shrink while valid. Count when the rule survives shrinking: shrink while invalid, then add `right - left + 1`. Count with "contains all of" or "at least": shrink while valid, then add `left`. Exactly k: at most k minus at most $k-1$.	The state updates in $O(1)$, and `left` never moves back. *Window length minus top count must be at most k.*
Stacks and monotonic stacks	Openers and closers that nest. The nearest greater or smaller element. A minimum times a width.	Push pending indexes. Pop each index that the new element beats: that is its *next* answer. The top is the *previous* answer.	Each index is pushed one time and popped at most one time: $O(n)$. *Pop what can never be an answer again.*
Binary search	Sorted or rotated data. "The minimum x such that it is possible", with answers up to 10^9. The latest value at or before t.	Write `ok(x)`, and name why it flips one time. Find the first true on a half-open range. `bisect_left` gives the first `a[i] >= x`.	Below `lo`, all fail. `hi` passes or is the end. *If mid passes, keep it; if mid fails, skip it.*

Technique	Signals	Core move	Invariant or memory hook
Linked lists	A chain of nodes, "in place". The head can change. A position from the end, or the middle.	Save the next node, then rewire. Build each output with a dummy and a tail. Or copy the nodes into a list.	`prev` heads the reversed part, and `curr` heads the rest. *Save next, flip the arrow, step both, return prev.*
Trees	A node answer comes from its children or its ancestors. "Level", "side view". "Path between any two nodes". Order in a BST.	Bottom-up DFS returns a summary. Top-down DFS passes the path state. BFS goes by levels. A BST in-order walk is sorted.	Height past 1000: use an explicit stack. *Add the two heights for best, store one plus the bigger height.*
Tries	Many strings and prefix queries: "starts with", autocomplete. Inserts and queries interleave. A wildcard. Many words on one grid.	Each node is a dict of children plus an end flag. Walk down from the root. A dot follows every edge. With 1 or 2 dots, a set of masked patterns is simpler: *mask one letter on add, fill one dot on search.*	A missing child ends the branch at once. Each walk is $O(L)$. *One node per prefix: walk the letters, flag the word end.*
Heaps and priority queues	The kth largest, the top k or the k closest, not the full order. A running median. A k-way merge.	Keep the k best in a heap of the opposite polarity. Or pop the best, push what follows. Two heaps for a median. Negate for a max-heap.	*Top k in a min-heap: push, trim to k, return the root. Heap the fronts; pop the smallest, push its next.*
Backtracking	"Return all" or "generate all". Tiny bounds: $n \leq 10$ to 20. A partial answer can break a rule.	Choose, explore, un-choose. Pick a shape: take or skip, start index, pool, slots, grid path. Prune early. Duplicates: sort, skip equal siblings.	A call leaves the path as it found it. *Check the cell, mark it, try four neighbors, unmark it.*
Graph traversal: grids and implicit graphs	Islands or regions in a grid. The fewest moves at equal cost. A spread from many cells at once. A small move changes a state.	Flood fill to count. BFS for the fewest steps, with all sources at distance 0. Search backward from fixed targets.	BFS: mark on push. The first distance is the shortest. *Count each new land cell, then sink its whole island.*
Graph structure: ordering, cycles and connectivity	Prerequisites: "b before a". Return a valid order. Count the groups. Find the edge that closes a cycle.	Kahn: queue indegree-0 nodes. Pop one, and lower the indegree of its neighbors. Union-find: a shared root means a cycle.	A short order means a cycle. *Take what is ready, free its dependents, check that you took everything.*
Weighted and advanced graphs	Edge costs and a minimum total. The lowest level that allows a path. Connect all points. At most k stops.	Dijkstra: pop `(d, u)`, skip a final `u`, relax. Minimax: max, not plus. k stops: $k+1$ Bellman-Ford rounds.	*The first pop of a node is its arrival time.* MST on points (a complete graph): *keep a dict of outside points: pop the nearest, then let it lower the others.* Sparse edge list: *cheapest edge first; keep it only when it joins two components.*
Dynamic programming: foundations and 1-D	"How many ways", a best total, or "is it possible". A few choices at each step. Greedy fails.	Recurse on the last move. The arguments are the state. Cache it, or fill a table from the base case. Segments: a state that ends at `i`.	Combine with a sum, min, max or `any`. *Skip it and keep the last best, or rob it and add its money to the best two back.*
Dynamic programming: knapsack family	Items make an amount or fit a capacity. "Each item once" or "unlimited". Signs on items.	Index a table by the total. Items outside: combinations. Totals outside: ordered sequences. One copy: totals go down. Unlimited: totals go up. Signs: memo on (index, sum), or a subset sum.	*Coins outside, amounts going up inside:* `ways[a] += ways[a - coin]`.
Dynamic programming on strings and sequences	Two strings: longest common, fewest edits, ways to form, pattern match. Lengths up to about 10^3.	`dp[i][j]` answers `a[:i]` and `b[:j]`. Fill the border, then read up, left and diagonal. Palindromes: expand from each center.	*Last chars match: go diagonal plus one. Else take the best of up and left.*
Dynamic programming on grids, intervals, games and states	Forward-only grid moves. Removals "in any order". Two players take from the ends. A cooldown.	Split on the last move into a cell, the element that acts last, or the move of the player to move. Fill by row, by length or by step.	*Pick the balloon that pops last. It sees only the two walls. My pick minus your best gap, max of both ends.*
Greedy	One number or yes or no, and n up to 10^5. A choice that cannot hurt: the farthest reach. One failure rules out a range.	Keep a small summary. At each element, test for a failure, apply the choice, and update the answer. Test it with a brute force.	Before index i, every index in `0..farthest` is reachable. *Walk forward; if you pass farthest you are stuck, else stretch farthest.*
Intervals	`[start, end]` pairs: overlap, merge, conflict, rooms. The fewest removals. The peak overlap.	Sort by start; compare only with the last group. Sort by end to keep the most. Sweep $+1$ and -1 events. Decide if touching ends overlap.	*Sort, then extend the last one or append a new one. Sort by end, keep what fits, remove the rest.*

Technique	Signals	Core move	Invariant or memory hook
Matrices and simulation	Spiral, rotate or flip. "In place" or "simultaneously". A rule-based process, or a huge step count.	Map one cell first. Clockwise directions: a turn is $d \pm 1$ modulo 4. A synchronous step writes a new grid. Skip whole periods of a repeat.	With a copy, `grid` never changes during a step. In place, one cell holds the old and the new state. *Flip on the diagonal, then flip each row.*
Math and number manipulation	Reverse a number or sum its digits. Digits in a string or a list. A 32-bit range. A huge exponent. A rule that can loop.	Pop digits with `divmod(x, 10)`. Add from the low digit with a carry. Guard the range first. Square the half. A seen set stops a loop.	*Add digits plus carry while anything is left. Write mod 10, carry div 10.*
Bit manipulation	Every value occurs twice except one, with $O(1)$ space. A fixed 32-bit width. A banned operator.	Read `x & 1`, then shift. Fold with XOR so that pairs cancel. Mask to 32 bits after each step.	*Pairs cancel under XOR, so XOR everything. XOR is the sum, AND shifted left is the carry.*
Design problems: combining data structures	"Design" or "Implement the class", with a cost target per call. Two kinds of access to one set of items.	Write a table of the operations. Pick one structure per access. Combine two, and write the invariant that ties them.	*Ordered dict: move to end on use, pop the front on overflow.*

68.1 Ten questions for any problem

When no technique comes to mind, ask these questions in this order.

1. **What does the output ask for?** All solutions: backtracking. A count, a best value, yes or no: DP or greedy.
2. **What do the bounds allow?** Plan for about 10^7 Python steps. $n \leq 20$: exponential search. $n \leq 5000$: $O(n^2)$. $n \leq 10^5$: $O(n \log n)$. An answer range up to 10^9: binary search on it.
3. **Which work does the brute force repeat?** Name its inner loop. The technique removes that work.
4. **Does the input order matter?** If it does not, sort the input or count it first.
5. **What is the shape of the answer?** A segment: a window or prefix sums. A pair: a hash map or two pointers. A subsequence: DP. The smallest subsequence after removals: a monotonic stack. A path: a graph.
6. **Is the rule one expression?** Exact key: a lookup. Survives shrinking: a window. Survives growing: a shortest window, or a count that adds `left`. Flips one time: binary search.
7. **What must the rest know about the past?** That is the state. Keep it small: a frontier, a minimum, a count map.
8. **What is a node, and what is an edge?** Equal costs: BFS. Costs 0 or 1: a 0-1 BFS. Non-negative weights: Dijkstra. Negative weights: Bellman-Ford. "Before": a topological order. "Linked": components. "Two groups" with no conflict inside a group: a two-coloring with BFS.
9. **Which access does each step need?** By key, order, rank, extreme or prefix. Pick the structure for it.
10. **Does it survive the edge cases?** Test empty, one element, ties, zeros and negatives. Past depth 1000, use a loop. Compare with a brute force.

69 Transfer drills

Each drill is a problem outside NeetCode 150 and Blind 75. Solve it with the method in Section 35. First name the signal, the technique and the key step. Then write the code. The answers are in Appendix A. Start with the mixed drills in Section 69.1, because they test recognition. Each later set practices one technique. Its heading names the technique, and the line below the heading gives the sections of the technique and of its answers.

69.1 Mixed drills

These drills come from all techniques, in a random order. No heading names the technique. So for each drill, first name the signal and the technique. The answer key below the list gives the answer section and the item number.

1. **Matrix Block Sum (LeetCode 1314).** You get an $m \times n$ matrix with $m, n \leq 100$ and an integer `k`. Return a matrix where `answer[i][j]` is the sum of all `mat[r][c]` with `|r - i| <= k`, `|c - j| <= k`, and `(r, c)` inside the matrix.
2. **Find Peak Element (LeetCode 162).** An array of up to 1000 integers has no two equal neighbors. Treat the values outside the array as $-\infty$. A peak is a value greater than both of its neighbors. Return the index of any peak in $O(\log n)$ time. The array is not sorted.

3. **Design Browser History (LeetCode 1472).** A browser starts on a home page. `visit(url)` opens a new page after the current page and clears all forward history. `back(steps)` and `forward(steps)` move at most `steps` pages and return the URL of the current page. When fewer pages are available, the move stops at the first page or at the last page. There are at most 5000 calls, and `steps` is 1 to 100. Make each call $O(1)$ amortized, independent of `steps`.
4. **Divide Two Integers (LeetCode 29).** Divide two signed 32-bit integers without `*`, `/` and `%`. Truncate the quotient toward zero, so $-7/2$ gives -3. The divisor is not 0. Clamp the quotient to $[-2^{31}, 2^{31}-1]$: a quotient above $2^{31}-1$ gives $2^{31}-1$.
5. **Shortest Bridge (LeetCode 934).** An $n \times n$ binary grid, $2 \le n \le 100$, has exactly two islands of 4-connected land. You can change water cells to land. Return the fewest changes that join the two islands into one island.
6. **Prison Cells After N Days (LeetCode 957).** Eight cells in a row hold 0 or 1. Each day all cells change at the same time. An inner cell becomes 1 if its two neighbors are equal, else 0. The two end cells become 0. Return the state after n days, with n up to 10^9.
7. **Array of Doubled Pairs (LeetCode 954).** You get an array of even length. It can hold negative values and zeros. Return true if you can reorder it so that `arr[2 * i + 1] == 2 * arr[2 * i]` for every pair. $n \le 3 \cdot 10^4$ and $|a_i| \le 10^5$.
8. **Sudoku Solver (LeetCode 37).** You get a 9×9 board of characters. A digit cell holds `"1"` to `"9"`, and `"."` marks an empty cell. Fill every empty cell in place. Then each row, each column and each of the nine 3×3 boxes must hold each digit exactly one time. The input has exactly one solution.
9. **Dungeon Game (LeetCode 174).** A knight starts in the top-left cell of an $m \times n$ grid and moves only right or down to the bottom-right cell. Each cell adds or removes health (values from -1000 to 1000). The knight dies as soon as his health is 0 or less, also on the first and on the last cell. Return the minimum start health. $1 \le m, n \le 200$.
10. **Single-Threaded CPU (LeetCode 1834).** Task i is `[enqueueTime, processingTime]`. When the CPU is idle, it starts the available task with the shortest processing time. A tie goes to the smallest index. A started task runs to the end. If no task is available, the CPU waits until the next task arrives. Return the order of the task indexes. There are up to 10^5 tasks, and times go up to 10^9.
11. **Lowest Common Ancestor of a Binary Tree (LeetCode 236).** A binary tree with 2 to 10^5 nodes has unique values but no order rule. You get two different nodes p and q of the tree. Return their lowest common ancestor. A node is a descendant of itself.
12. **Minimum Remove to Make Valid Parentheses (LeetCode 1249).** A string has lowercase letters, `(` and `)`. Its length is up to 10^5. Remove the minimum number of parentheses so that the result is a valid parentheses string, and return any such result. Letters stay in their order.
13. **Shortest Common Supersequence (LeetCode 1092).** You get two strings `str1` and `str2` of lowercase letters. Each has length 1 to 1000. Return the shortest string that has both `str1` and `str2` as subsequences. If more than one answer exists, return any of them.
14. **Count Number of Nice Subarrays (LeetCode 1248).** A contiguous subarray is nice when it contains exactly `k` odd numbers. Return the number of nice subarrays of `nums`. $1 \le n \le 5 \cdot 10^4$, $1 \le$ `nums[i]` $\le 10^5$, $1 \le k \le n$.
15. **Remove Duplicates from Sorted Array II (LeetCode 80).** A sorted array has up to $3 \cdot 10^4$ elements. Change it in place so that each value appears at most two times, in the original order, and return the new length k. Use $O(1)$ extra memory. The judge reads only the first k elements.
16. **Sort List (LeetCode 148).** Sort a linked list in ascending order and return the head. The list has 0 to $5 \cdot 10^4$ nodes. The target is $O(n \log n)$ time. The follow-up asks for $O(1)$ extra memory.
17. **Delete and Earn (LeetCode 740).** You get an integer array `nums`. In one operation, pick an element `x`, delete it, and earn `x` points. Then delete every element equal to `x - 1` or `x + 1`. Return the maximum total of points. The limits are $1 \le n \le 2 \cdot 10^4$ and $1 \le$ `nums[i]` $\le 10^4$.
18. **Maximum Score of a Node Sequence (LeetCode 2242).** An undirected graph has $n \le 5 \cdot 10^4$ nodes with scores and up to $5 \cdot 10^4$ edges. A valid sequence has 4 different nodes, and each two consecutive nodes share an edge. Return the maximum sum of scores of a valid sequence, or -1 if none exists.
19. **Least Number of Unique Integers after K Removals (LeetCode 1481).** You get up to 10^5 integers with values up to 10^9, and an integer k. Remove exactly k elements. Return the smallest possible number of distinct values that remain.
20. **Number of Ways to Arrive at Destination (LeetCode 1976).** A connected undirected graph has up to 200 intersections. Each road takes a time from 1 to 10^9. Count the routes from intersection 0 to intersection $n-1$ that take the shortest possible time. Return the count modulo $10^9 + 7$.

21. **Maximum XOR With an Element From Array (LeetCode 1707).** You get an array `nums` and queries `[x, m]`. For each query, return the largest `x ^ v` over the values `v` of `nums` with `v <= m`, or `-1` if no value qualifies. There are up to 10^5 values and 10^5 queries. All values, `x` and `m` are integers from 0 to 10^9.
22. **Minimum Number of Work Sessions to Finish the Tasks (LeetCode 1986).** You get $n \leq 14$ tasks with durations from 1 to 10 hours, and a session length `sessionTime` from the longest task up to 15. A task must start and finish in the same session, and the tasks can go in any order. Return the fewest sessions that finish all the tasks.
23. **Backward pass of max normalization.** Define $y = x/\max(x)$ for a vector x of length up to 10^5, with distinct values and a positive maximum. You get x and the upstream gradient $v = \partial L/\partial y$. Return $\partial L/\partial x$ in the original order of x, in $O(n)$ time.
24. **Car Pooling (LeetCode 1094).** A car drives east only and has `capacity` empty seats. Each trip `[p, start, end]` picks up `p` passengers at kilometer `start` and drops them at kilometer `end`. At one kilometer, the passengers leave before new passengers get in. Up to 1000 trips, kilometers up to 1000. Return whether the car can do all trips.

Answer key. 1: A.2 item 5; 2: A.6 item 6; 3: A.24 item 3; 4: A.23 item 6; 5: A.12 item 6; 6: A.21 item 4; 7: A.19 item 3; 8: A.11 item 6; 9: A.18 item 1; 10: A.10 item 4; 11: A.8 item 6; 12: A.5 item 1; 13: A.17 item 3; 14: A.4 item 4; 15: A.3 item 4; 16: A.7 item 5; 17: A.15 item 1; 18: A.13 item 6; 19: A.1 item 4; 20: A.14 item 3; 21: A.9 item 6; 22: A.16 item 6; 23: A.22 item 6; 24: A.20 item 3.

69.2 Hashing: counting, membership and canonical keys

Technique: Section 44 (page 136). Answers: Section A.1 (page 428).

1. **4Sum II (LeetCode 454).** You get four integer arrays a, b, c and d, each of length $n \leq 200$. Count the tuples (i, j, k, l) with $a[i] + b[j] + c[k] + d[l] = 0$. The values go up to 2^{28} in absolute value. A check of all $n^4 \approx 1.6 \cdot 10^9$ tuples is too slow.
2. **Count Nice Pairs in an Array (LeetCode 1814).** You get an array `nums` of up to 10^5 integers, each from 0 to 10^9. The function `rev(x)` writes the digits of x in reverse order, so `rev(120) == 21`. A pair of indexes $i < j$ is nice when `nums[i] + rev(nums[j]) == nums[j] + rev(nums[i])`. Return the number of nice pairs modulo $10^9 + 7$.
3. **Group Shifted Strings (LeetCode 249).** A shift moves every letter of a word one step forward in the alphabet, and `z` becomes `a`. So `abc`, `bcd` and `xyz` belong to one shifting sequence. Group up to 200 lowercase words (each up to 50 letters) by shifting sequence. The order of the groups does not matter.
4. **Least Number of Unique Integers after K Removals (LeetCode 1481).** You get up to 10^5 integers with values up to 10^9, and an integer k. Remove exactly k elements. Return the smallest possible number of distinct values that remain.
5. **Longest Arithmetic Subsequence of Given Difference (LeetCode 1218).** You get an array `arr` of up to 10^5 integers and an integer `difference`. All values are from -10^4 to 10^4. Return the length of the longest subsequence in which each element minus the element before it equals `difference`. A subsequence keeps the order of the array. An $O(n^2)$ check of all pairs is too slow.
6. **Number of Boomerangs (LeetCode 447).** You get $n \leq 500$ distinct points with integer coordinates. A boomerang is an ordered tuple of points (i, j, k) in which the distance from i to j equals the distance from i to k. Count the boomerangs. An $O(n^3)$ check of all tuples is too slow.

69.3 Prefix sums, running aggregates and offsets

Technique: Section 45 (page 148). Answers: Section A.2 (page 430).

1. **Contiguous Array (LeetCode 525).** You get a binary array `nums` with up to 10^5 elements. Return the length of the longest contiguous subarray that has the same number of 0s and 1s. Return 0 if no such subarray exists.
2. **Shifting Letters II (LeetCode 2381).** You get a string `s` of lowercase letters and a list `shifts`. Each shift `[start, end, direction]` moves every letter in `s[start..end]` one step forward if `direction` is 1, or one step backward if it is 0. The alphabet wraps: `z` goes to `a`, and `a` goes back to `z`. Both lengths are up to $5 \cdot 10^4$. Return the final string.
3. **Maximum Value of an Ordered Triplet II (LeetCode 2874).** You get an array `nums` with $3 \leq n \leq 10^5$. Each value is an integer from 1 to 10^6. A triplet of indexes `i < j < k` has the value `(nums[i] - nums[j]) *`

`nums[k]`. Return the largest value of a triplet, or 0 if all triplets have a negative value.

4. **Maximum Absolute Sum of Any Subarray (LeetCode 1749).** You get an array `nums` of up to 10^5 integers. Each value is from -10^4 to 10^4. The absolute sum of a contiguous subarray is the absolute value of its sum. An empty subarray has the absolute sum 0. Return the largest absolute sum of a contiguous subarray.
5. **Matrix Block Sum (LeetCode 1314).** You get an $m \times n$ matrix with $m, n \leq 100$ and an integer `k`. Return a matrix where `answer[i][j]` is the sum of all `mat[r][c]` with `|r - i| <= k`, `|c - j| <= k`, and `(r, c)` inside the matrix.
6. **Find All Good Indices (LeetCode 2420).** You get an array of up to 10^5 integers and `k` with $1 \leq k \leq n/2$. Only an index `i` with `k <= i < n - k` can be good. It is good when the `k` elements just before `i` are non-increasing and the `k` elements just after `i` are non-decreasing. Return all good indices in increasing order.

69.4 Two pointers

Technique: Section 46 (page 157). Answers: Section A.3 (page 431).

1. **Valid Palindrome II (LeetCode 680).** A string `s` of lowercase letters has a length of up to 10^5. Return `True` if `s` can become a palindrome after you delete at most one character. A test of every possible deletion costs $O(n^2)$ and is too slow.
2. **Boats to Save People (LeetCode 881).** `people[i]` is the weight of person i, and no weight is more than `limit`. A boat carries at most two people, and their total weight is at most `limit`. Return the minimum number of boats that carry all $n \leq 5 \cdot 10^4$ people.
3. **Valid Triangle Number (LeetCode 611).** Count the index triples $i < j < k$ whose three values can be the sides of a triangle: the sum of any two sides is larger than the third side. $n \leq 1000$ and $0 \leq$ `nums[i]` ≤ 1000. Equal values at different indexes give different triples.
4. **Remove Duplicates from Sorted Array II (LeetCode 80).** A sorted array has up to $3 \cdot 10^4$ elements. Change it in place so that each value appears at most two times, in the original order, and return the new length k. Use $O(1)$ extra memory. The judge reads only the first k elements.
5. **Delete the Middle Node of a Linked List (LeetCode 2095).** A singly linked list has n nodes, with $1 \leq n \leq 10^5$. Delete the node at the 0-based index $\lfloor n/2 \rfloor$ and return the head. A list of one node becomes empty.
6. **Merge Sorted Array (LeetCode 88).** `nums1` has the length $m + n$. Its first m values are sorted, and its last n slots hold placeholder zeros. `nums2` holds n sorted values. Merge `nums2` into `nums1` in place, so that `nums1` is sorted. Use $O(m + n)$ time and no second array.

69.5 Sliding window

Technique: Section 47 (page 170). Answers: Section A.4 (page 433).

1. **Frequency of the Most Frequent Element (LeetCode 1838).** You get an integer array `nums` and an integer `k`. In one operation, you add 1 to one element. Return the largest possible count of one value after at most `k` operations. $1 \leq n \leq 10^5$, $1 \leq$ `nums[i]` $\leq 10^5$, $1 \leq k \leq 10^5$.
2. **Replace the Substring for Balanced String (LeetCode 1234).** A string `s` has only the letters `Q`, `W`, `E` and `R`. Its length n is a multiple of 4. The string is balanced when each letter appears exactly $n/4$ times. You may replace one substring with any string of the same length. Return the minimum length of that substring, or 0 if `s` is already balanced. $4 \leq n \leq 10^5$.
3. **Subarray Product Less Than K (LeetCode 713).** You get an array `nums` of positive integers and an integer `k`. Return the number of contiguous subarrays whose product is strictly less than `k`. $1 \leq n \leq 3 \cdot 10^4$, $1 \leq$ `nums[i]` ≤ 1000, $0 \leq k \leq 10^6$.
4. **Count Number of Nice Subarrays (LeetCode 1248).** A contiguous subarray is nice when it contains exactly `k` odd numbers. Return the number of nice subarrays of `nums`. $1 \leq n \leq 5 \cdot 10^4$, $1 \leq$ `nums[i]` $\leq 10^5$, $1 \leq k \leq n$.
5. **Maximum Points You Can Obtain from Cards (LeetCode 1423).** Cards lie in a row, and each card has a number of points. In each step, you take the card at the left end or the card at the right end of the row. You take exactly `k` cards. Return the largest possible total of points. $1 \leq k \leq n \leq 10^5$, $1 \leq$ points $\leq 10^4$.
6. **Longest Continuous Subarray With Absolute Diff Less Than or Equal to Limit (LeetCode 1438).** Return the length of the longest non-empty contiguous subarray in which the absolute difference of any two elements is at most `limit`. $1 \leq n \leq 10^5$, $1 \leq$ `nums[i]` $\leq 10^9$, $0 \leq$ `limit` $\leq 10^9$.

69.6 Stacks and monotonic stacks

Technique: Section 48 (page 181). Answers: Section A.5 (page 435).

1. **Minimum Remove to Make Valid Parentheses (LeetCode 1249).** A string has lowercase letters, `(` and `)`. Its length is up to 10^5. Remove the minimum number of parentheses so that the result is a valid parentheses string, and return any such result. Letters stay in their order.
2. **Decode String (LeetCode 394).** An encoded string uses the rule `k[text]`: repeat `text` exactly `k` times. Brackets can nest, as in `3[a2[c]]`, and `k` can have several digits (up to 300). The input is always valid, and the decoded length is at most 10^5. Return the decoded string.
3. **Basic Calculator II (LeetCode 227).** A string holds non-negative integers, the operators `+ - * /` and spaces, but no parentheses. Its length is up to $3 \cdot 10^5$. Evaluate it with the usual precedence, and truncate each division toward zero. Do not use `eval`.
4. **Asteroid Collision (LeetCode 735).** A row of up to 10^4 asteroids is given as non-zero integers. The absolute value is the size, and the sign is the direction (positive moves right, negative moves left). All asteroids move at the same speed. When two meet, the smaller one explodes, and equal sizes both explode. Return the asteroids that remain.
5. **Online Stock Span (LeetCode 901).** Design a class with `next(price)`. It returns the span of today's price. The span is the number of consecutive days that end today (today included) with a price less than or equal to today's price. There are up to 10^4 calls, and each call must be amortized $O(1)$.
6. **Sum of Subarray Minimums (LeetCode 907).** An array has up to $3 \cdot 10^4$ integers from 1 to $3 \cdot 10^4$, and values can repeat. Return the sum of `min(sub)` over every contiguous subarray `sub`, modulo $10^9 + 7$. An $O(n^2)$ scan is too slow.

69.7 Binary search

Technique: Section 49 (page 191). Answers: Section A.6 (page 438).

1. **Capacity To Ship Packages Within D Days (LeetCode 1011).** Up to $5 \cdot 10^4$ packages, each with a weight from 1 to 500, must ship in the given order. Each day, the ship loads the next packages in order, and the total load must not exceed its capacity. Return the smallest capacity that ships all packages within `days` days.
2. **Magnetic Force Between Two Balls (LeetCode 1552).** You get up to 10^5 distinct basket positions, each up to 10^9, and $m \geq 2$ balls. Put each ball in a different basket. Make the smallest distance between two balls as large as possible, and return that distance.
3. **Snapshot Array (LeetCode 1146).** Design an array of length up to $5 \cdot 10^4$. All values are 0 at the start. `set(index, val)` changes one value. `snap()` takes a snapshot and returns its id: 0, then 1, then 2. `get(index, snap_id)` returns the value at that index when that snapshot was taken. Up to $5 \cdot 10^4$ calls occur, so a full copy of the array at each snapshot is too slow.
4. **Kth Smallest Element in a Sorted Matrix (LeetCode 378).** Each row and each column of an $n \times n$ matrix ($n \leq 300$) is sorted in ascending order. The values are in $[-10^9, 10^9]$. Return the k-th smallest of all n^2 values, with duplicates counted. Use less than $O(n^2)$ extra memory.
5. **Russian Doll Envelopes (LeetCode 354).** You get up to 10^5 envelopes as pairs `[w, h]`, and each side is from 1 to 10^5. One envelope fits in another only when its width and its height are both strictly smaller. Return the largest number of envelopes that you can put one inside the other. You cannot rotate an envelope.
6. **Find Peak Element (LeetCode 162).** An array of up to 1000 integers has no two equal neighbors. Treat the values outside the array as $-\infty$. A peak is a value greater than both of its neighbors. Return the index of any peak in $O(\log n)$ time. The array is not sorted.

69.8 Linked lists

Technique: Section 50 (page 203). Answers: Section A.7 (page 440).

1. **Palindrome Linked List (LeetCode 234).** Return `True` if the values of a singly linked list read the same forward and backward. The list has 1 to 10^5 nodes. The follow-up asks for $O(n)$ time and $O(1)$ extra memory.
2. **Reverse Linked List II (LeetCode 92).** Reverse only the nodes from position `left` to position `right` (1-indexed, $1 \leq left \leq right \leq n$), and return the head. The list has 1 to 500 nodes. Change the links, not the values. The follow-up asks for one pass.
3. **Rotate List (LeetCode 61).** Rotate a list to the right by k places: the last k nodes move to the front in the same order. The list has 0 to 500 nodes, and k can be as large as $2 \cdot 10^9$.

4. **Partition List (LeetCode 86).** Given a list and a value x, put all nodes with a value less than x before all nodes with a value of x or more. Keep the original relative order inside each part. The list has 0 to 200 nodes. Rewire the given nodes.
5. **Sort List (LeetCode 148).** Sort a linked list in ascending order and return the head. The list has 0 to $5 \cdot 10^4$ nodes. The target is $O(n \log n)$ time. The follow-up asks for $O(1)$ extra memory.
6. **Linked List Cycle II (LeetCode 142).** Return the node where the cycle of a list begins, or `None` if there is no cycle. The list has 0 to 10^4 nodes. Do not change the list. The follow-up asks for $O(1)$ extra memory.

69.9 Trees

Technique: Section 51 (page 217). Answers: Section A.8 (page 442).

1. **Maximum Difference Between Node and Ancestor (LeetCode 1026).** A binary tree has 2 to 5000 nodes, with values from 0 to 10^5. Find the largest value of $|a.val - b.val|$, where node a is an ancestor of node b. The tree can be skewed (one child at each node), so a check of every ancestor-descendant pair is $O(n^2)$.
2. **Longest Univalue Path (LeetCode 687).** Return the number of edges on the longest path in a binary tree where all nodes have the same value. The path can start and end at any node. It does not need to pass through the root. The tree has up to 10^4 nodes and a depth of at most 1000.
3. **Maximum Width of Binary Tree (LeetCode 662).** The width of a level is the number of positions from its leftmost to its rightmost non-null node. The null positions between them count, as if the level were full. Return the largest width of all levels. The tree has up to 3000 nodes, and it can be skewed, 3000 levels deep.
4. **Minimum Absolute Difference in BST (LeetCode 530).** A binary search tree has 2 to 10^4 nodes, with values from 0 to 10^5. Return the smallest difference between the values of two different nodes. A check of all pairs is $O(n^2)$.
5. **Construct Binary Tree from Inorder and Postorder Traversal (LeetCode 106).** You get the in-order list and the postorder list of a binary tree with up to 3000 nodes. All values are unique. Rebuild the tree and return its root.
6. **Lowest Common Ancestor of a Binary Tree (LeetCode 236).** A binary tree with 2 to 10^5 nodes has unique values but no order rule. You get two different nodes p and q of the tree. Return their lowest common ancestor. A node is a descendant of itself.

69.10 Tries

Technique: Section 52 (page 233). Answers: Section A.9 (page 443).

1. **Map Sum Pairs (LeetCode 677).** Design a class with `insert(key, val)` and `sum(prefix)`. An insert of a key that exists replaces its old value. `sum(prefix)` returns the sum of the values of all keys that start with `prefix`. Design each call to cost O(length of its string argument), not O(number of keys).
2. **Implement Magic Dictionary (LeetCode 676).** `buildDict(words)` stores distinct lowercase words. The method `search(w)` returns `True` if a change of exactly one letter of `w` gives a stored word. A stored word equal to `w` does not count. There are up to 100 words of length up to 100, and up to 100 searches.
3. **Search Suggestions System (LeetCode 1268).** You get distinct product names and a search word. After each typed letter of the search word, return up to three products that start with the typed prefix, in lexicographic order. There are up to 1000 products with a total length up to $2 \cdot 10^4$, and the search word has up to 1000 letters.
4. **Concatenated Words (LeetCode 472).** You get up to 10^4 distinct lowercase words. Each word has up to 30 letters, and the total length is up to 10^5. Return every word that is a concatenation of at least two shorter words of the same list. A shorter word can appear more than one time in a concatenation, for example `catcat`.
5. **Stream of Characters (LeetCode 1032).** You get up to 2000 words of length up to 200. Then letters arrive one at a time, in up to $4 \cdot 10^4$ calls. After each letter, return `True` if some suffix of the stream so far is one of the words.
6. **Maximum XOR With an Element From Array (LeetCode 1707).** You get an array `nums` and queries `[x, m]`. For each query, return the largest `x ^ v` over the values `v` of `nums` with `v <= m`, or `-1` if no value qualifies. There are up to 10^5 values and 10^5 queries. All values, `x` and `m` are integers from 0 to 10^9.

69.11 Heaps and priority queues

Technique: Section 53 (page 244). Answers: Section A.10 (page 446).

1. **Top K Frequent Words (LeetCode 692).** You get a list of words and an integer k. Return the k most frequent words, sorted by frequency from high to low. Words with the same frequency come in lexicographic order. There are at most 500 words of at most 10 lowercase letters. The follow-up asks for $O(n \log k)$ time.
2. **Minimum Operations to Halve Array Sum (LeetCode 2208).** You get an array of positive integers. One operation picks any number and replaces it with exactly half of it. A later operation can pick the same number again. Return the minimum number of operations that reduce the sum of the array by at least half. There are up to 10^5 numbers, and each number is at most 10^7.
3. **Reorganize String (LeetCode 767).** Rearrange the characters of a string `s` so that no two adjacent characters are equal. Return any valid arrangement, or an empty string if no arrangement exists. `s` has up to 500 lowercase letters.
4. **Single-Threaded CPU (LeetCode 1834).** Task i is `[enqueueTime, processingTime]`. When the CPU is idle, it starts the available task with the shortest processing time. A tie goes to the smallest index. A started task runs to the end. If no task is available, the CPU waits until the next task arrives. Return the order of the task indexes. There are up to 10^5 tasks, and times go up to 10^9.
5. **Find K Pairs with Smallest Sums (LeetCode 373).** Two arrays `nums1` and `nums2` are sorted in non-decreasing order. Each has up to 10^5 items, and k is at most 10^4. Return the k pairs `(u, v)`, with `u` from `nums1` and `v` from `nums2`, that have the smallest sums. A list of all $n_1 \cdot n_2$ pairs is too large.
6. **Sliding Window Median (LeetCode 480).** You get an array `nums` and a window size k. Return the median of each window of size k as the window moves right by one position. For an even k, the median is the mean of the two middle values. n is up to 10^5, the values are 32-bit integers, and answers within 10^{-5} are accepted.

69.12 Backtracking

Technique: Section 54 (page 256). Answers: Section A.11 (page 448).

1. **Permutations II (LeetCode 47).** You get a list of 1 to 8 integers from -10 to 10, and values can repeat. Return all distinct orderings of the list. Two orderings are the same when they have the same value at each position.
2. **Combination Sum III (LeetCode 216).** Find all sets of exactly k distinct numbers from 1 to 9 with the sum n. Each number is used at most one time, and the order inside a set does not matter. The bounds are $2 \le k \le 9$ and $1 \le n \le 60$.
3. **Restore IP Addresses (LeetCode 93).** You get a string of 1 to 20 digits. Insert three dots to make a valid IPv4 address, and return all such addresses. Each of the four parts is a number from 0 to 255 without a leading zero: `"0"` is valid, `"01"` is not.
4. **Path with Maximum Gold (LeetCode 1219).** A grid of at most 15×15 cells holds gold amounts from 0 to 100, and at most 25 cells hold gold. Start at any gold cell and move up, down, left or right. Never enter a cell with 0 gold, and never visit a cell two times. Return the largest total gold of one walk.
5. **Find Minimum Time to Finish All Jobs (LeetCode 1723).** You get n job durations of up to 10^7 each and k workers, with $1 \le k \le n \le 12$. Each job goes to exactly one worker, and the time of a worker is the sum of its jobs. Return the smallest possible value of the largest worker time.
6. **Sudoku Solver (LeetCode 37).** You get a 9×9 board of characters. A digit cell holds `"1"` to `"9"`, and `"."` marks an empty cell. Fill every empty cell in place. Then each row, each column and each of the nine 3×3 boxes must hold each digit exactly one time. The input has exactly one solution.

69.13 Graph traversal: grids and implicit graphs

Technique: Section 55 (page 270). Answers: Section A.12 (page 449).

1. **Count Sub Islands (LeetCode 1905).** Two binary grids `grid1` and `grid2` have the same shape, up to 500×500. Islands connect 4-directionally. An island of `grid2` is a sub-island if each of its cells is land in `grid1`. Return the number of sub-islands in `grid2`.
2. **Number of Enclaves (LeetCode 1020).** A binary grid, up to 500×500, holds land (1) and sea (0). One move goes from a land cell to a 4-directionally adjacent land cell, or off the edge of the grid. Return the number of land cells from which no sequence of moves leaves the grid.
3. **As Far from Land as Possible (LeetCode 1162).** An $n \times n$ grid, $n \le 100$, holds water (0) and land (1). Find the water cell with the largest Manhattan distance to its nearest land cell, and return that distance. Return -1 if the grid has no land or no water.
4. **Detonate the Maximum Bombs (LeetCode 2101).** There are up to 100 bombs. Bomb i is a list $[x_i, y_i, r_i]$:

a center and a blast radius, with values up to 10^5. A bomb that explodes detonates every bomb whose center is at a distance of r_i or less, and the chain continues. You detonate exactly one bomb. Return the largest number of bombs that can explode.

5. **Sliding Puzzle (LeetCode 773).** A 2×3 board holds the tiles 1 to 5 and one empty square 0. One move swaps the 0 with a tile that shares an edge with it. Return the fewest moves to reach `[[1, 2, 3], [4, 5, 0]]`, or -1 if the board cannot reach it.
6. **Shortest Bridge (LeetCode 934).** An $n \times n$ binary grid, $2 \leq n \leq 100$, has exactly two islands of 4-connected land. You can change water cells to land. Return the fewest changes that join the two islands into one island.

69.14 Graph structure: ordering, cycles and connectivity

Technique: Section 56 (page 282). Answers: Section A.13 (page 451).

1. **Parallel Courses III (LeetCode 2050).** There are n courses and pairs `[prev, next]`: course `prev` must finish before course `next` starts. Course i takes `time[i]` months, and any number of courses can run at the same time. Return the minimum number of months to finish all courses. $n \leq 5 \cdot 10^4$, at most $5 \cdot 10^4$ pairs, and the pairs form no cycle.
2. **Find Eventual Safe States (LeetCode 802).** A directed graph with $n \leq 10^4$ nodes comes as adjacency lists with up to $4 \cdot 10^4$ edges. Self-loops are possible. A node is safe if every path from it ends at a node with no outgoing edge. Return all safe nodes in increasing order.
3. **Satisfiability of Equality Equations (LeetCode 990).** You get up to 500 strings of the form `"a==b"` or `"a!=b"`. Each lowercase letter is a variable. Return `True` if you can give each variable an integer so that every equation holds.
4. **Number of Operations to Make Network Connected (LeetCode 1319).** There are $n \leq 10^5$ computers and up to 10^5 cables, each between two computers. One operation removes a cable and uses it to join any two computers. Return the minimum number of operations to connect all computers, or -1 if it is impossible.
5. **Longest Cycle in a Graph (LeetCode 2360).** A directed graph has $n \leq 10^5$ nodes. Each node has at most one outgoing edge: `edges[i]` is its target, or -1. Return the length of the longest cycle, or -1 if there is no cycle.
6. **Maximum Score of a Node Sequence (LeetCode 2242).** An undirected graph has $n \leq 5 \cdot 10^4$ nodes with scores and up to $5 \cdot 10^4$ edges. A valid sequence has 4 different nodes, and each two consecutive nodes share an edge. Return the maximum sum of scores of a valid sequence, or -1 if none exists.

69.15 Weighted and advanced graphs

Technique: Section 57 (page 295). Answers: Section A.14 (page 453).

1. **Path With Minimum Effort (LeetCode 1631).** A grid of heights has up to 100×100 cells, and each height is from 1 to 10^6. You walk from the top-left cell to the bottom-right cell in 4 directions. The effort of a route is the largest absolute height difference between two consecutive cells on it. Return the minimum effort.
2. **Path with Maximum Probability (LeetCode 1514).** An undirected graph has up to 10^4 nodes and $2 \cdot 10^4$ edges. Each edge has a success probability in $[0, 1]$, and the probability of a path is the product of its edge probabilities. Return the maximum probability of a path from `start` to `end`, or 0 when no path exists.
3. **Number of Ways to Arrive at Destination (LeetCode 1976).** A connected undirected graph has up to 200 intersections. Each road takes a time from 1 to 10^9. Count the routes from intersection 0 to intersection $n - 1$ that take the shortest possible time. Return the count modulo $10^9 + 7$.
4. **Optimize Water Distribution in a Village (LeetCode 1168).** A village has up to 10^4 houses, labeled 1 to n. A well in house i costs `wells[i - 1]`. A pipe `[a, b, c]` joins houses a and b in both directions at cost c, and there are up to 10^4 pipes. A house has water when it has a well, or when pipes connect it to a house with a well. Return the minimum total cost to give water to every house.
5. **Minimum Cost to Reach Destination in Time (LeetCode 1928).** An undirected graph has up to 1000 cities and 1000 roads, and two cities can have more than one road. Each road takes 1 to 1000 minutes. Each city has a fee from 1 to 1000, which you pay for every city on the route, the first and the last included. Return the minimum total fee from city 0 to city $n - 1$ in at most `maxTime` minutes (≤ 1000), or -1.
6. **Valid Arrangement of Pairs (LeetCode 2097).** You get up to 10^5 distinct pairs `[start, end]` of integers up to 10^9, with `start != end`. Put all the pairs in a list so that the end of each pair equals the start of the next pair. A valid arrangement is known to exist. Return any one.

69.16 Dynamic programming: foundations and 1-D

Technique: Section 58 (page 305). Answers: Section A.15 (page 454).

1. **Delete and Earn (LeetCode 740).** You get an integer array `nums`. In one operation, pick an element `x`, delete it, and earn `x` points. Then delete every element equal to `x - 1` or `x + 1`. Return the maximum total of points. The limits are $1 \le n \le 2 \cdot 10^4$ and $1 \le$ `nums[i]` $\le 10^4$.
2. **Count Number of Texts (LeetCode 2266).** A phone keypad maps the digits 2 to 9 to letters. The keys 7 and 9 have four letters, and the other keys have three letters. To type the k-th letter of a key, press that key k times. You get the string `pressedKeys` of the pressed digits, with a length up to 10^5. Return the number of text messages that produce it, modulo $10^9 + 7$.
3. **Minimum Cost For Tickets (LeetCode 983).** You get the travel days of one year in increasing order, each from 1 to 365. A pass costs `costs[0]` for 1 day, `costs[1]` for 7 days, and `costs[2]` for 30 days. A pass covers that number of consecutive days from the day of purchase. Each cost is at most 1000. Return the minimum total cost that covers every travel day.
4. **Maximum Sum Circular Subarray (LeetCode 918).** You get a circular integer array `nums`, so the last element is next to the first element. Return the maximum sum of a non-empty subarray. A subarray can wrap past the end, but it uses each element at most one time. The limits are $n \le 3 \cdot 10^4$ and values from $-3 \cdot 10^4$ to $3 \cdot 10^4$.
5. **Maximum Length of Subarray With Positive Product (LeetCode 1567).** You get an integer array `nums` with $n \le 10^5$ and values from -10^9 to 10^9, zeros included. Return the maximum length of a contiguous subarray whose product is positive.
6. **Number of Longest Increasing Subsequence (LeetCode 673).** You get an integer array `nums` with $n \le 2000$ and values from -10^6 to 10^6. Return the number of strictly increasing subsequences that have the maximum length. Two subsequences are different when they use different indices.

69.17 Dynamic programming: knapsack family

Technique: Section 59 (page 315). Answers: Section A.16 (page 456).

1. **Last Stone Weight II (LeetCode 1049).** You get the weights of up to 30 stones, each from 1 to 100. In each turn, smash any two stones with weights $x \le y$. Equal stones both disappear; otherwise only one stone of weight $y - x$ stays. Return the smallest possible weight of the last stone, or 0 if no stone stays.
2. **Ones and Zeroes (LeetCode 474).** You get up to 600 binary strings (each of length at most 100) and two limits m and n (each at most 100). Return the size of the largest subset of the strings that contains at most m zeros and at most n ones in total.
3. **Number of Dice Rolls With Target Sum (LeetCode 1155).** You roll n dice, and each die has the faces 1 to k. Count the roll results (out of k^n, so the order of the dice matters) whose sum is exactly `target`. Return the count modulo $10^9 + 7$. Limits: $1 \le n, k \le 30$ and $1 \le \mathit{target} \le 1000$.
4. **Count Ways To Build Good Strings (LeetCode 2466).** Start with an empty string. In each step, append the character `0` exactly `zero` times, or the character `1` exactly `one` times. Count the different strings that you can build with a length from `low` to `high`, modulo $10^9 + 7$. Limits: $1 \le \mathit{zero}, \mathit{one} \le \mathit{low} \le \mathit{high} \le 10^5$.
5. **Extra Characters in a String (LeetCode 2707).** Break a string `s` into non-overlapping substrings that are dictionary words. Characters that are in no chosen word are extra. Return the fewest extra characters. The string and each word have at most 50 characters, and there are at most 50 distinct words.
6. **Minimum Number of Work Sessions to Finish the Tasks (LeetCode 1986).** You get $n \le 14$ tasks with durations from 1 to 10 hours, and a session length `sessionTime` from the longest task up to 15. A task must start and finish in the same session, and the tasks can go in any order. Return the fewest sessions that finish all the tasks.

69.18 Dynamic programming on strings and sequences

Technique: Section 60 (page 325). Answers: Section A.17 (page 458).

1. **Minimum ASCII Delete Sum for Two Strings (LeetCode 712).** You get two strings `s1` and `s2` of lowercase letters. Each has length 1 to 1000. Delete characters from either string until the two strings are equal. Return the lowest possible sum of the ASCII codes of the deleted characters.
2. **Number of Ways to Form a Target String Given a Dictionary (LeetCode 1639).** You get up to 1000 words, all of the same length $L \le 1000$, and a string `target` of length up to 1000. Build `target` from left to

right. Each character comes from column k of any word, and the columns that you use must strictly increase. Return the number of ways modulo $10^9 + 7$. A different word or a different column gives a different way.

3. **Shortest Common Supersequence (LeetCode 1092).** You get two strings `str1` and `str2` of lowercase letters. Each has length 1 to 1000. Return the shortest string that has both `str1` and `str2` as subsequences. If more than one answer exists, return any of them.
4. **Wildcard Matching (LeetCode 44).** You get a string `s` of lowercase letters and a pattern `p` of letters, `?` and `*`. Each has length 0 to 2000. The symbol `?` matches any one character. The symbol `*` matches any sequence of characters, also the empty sequence. Return true if `p` matches all of `s`.
5. **Longest Palindromic Subsequence (LeetCode 516).** You get a string `s` of lowercase letters with length 1 to 1000. Return the length of the longest subsequence of `s` that is a palindrome. The characters of the subsequence do not need to be contiguous.
6. **Max Dot Product of Two Subsequences (LeetCode 1458).** You get two integer arrays `nums1` and `nums2`. Each has length 1 to 500, with values from -1000 to 1000. Pick a non-empty subsequence from each array. The two subsequences must have the same length. Their dot product is the sum of the products of the values at the same position. Return the largest possible dot product.

69.19 Dynamic programming on grids, intervals, games and states

Technique: Section 61 (page 336). Answers: Section A.18 (page 459).

1. **Dungeon Game (LeetCode 174).** A knight starts in the top-left cell of an $m \times n$ grid and moves only right or down to the bottom-right cell. Each cell adds or removes health (values from -1000 to 1000). The knight dies as soon as his health is 0 or less, also on the first and on the last cell. Return the minimum start health. $1 \leq m, n \leq 200$.
2. **Number of Increasing Paths in a Grid (LeetCode 2328).** Count all strictly increasing paths in an $m \times n$ grid. A path moves up, down, left or right. It can start and end at any cell, and one cell alone is a path. Return the count modulo $10^9 + 7$. $m \cdot n \leq 10^5$, and the values are from 1 to 10^5.
3. **Minimum Cost to Cut a Stick (LeetCode 1547).** A stick has the length $n \leq 10^6$. You must cut it at each of the given positions (at most 100 distinct positions), in an order that you choose. A cut costs the length of the piece that it cuts. Return the minimum total cost.
4. **Stone Game III (LeetCode 1406).** Stones in a row have values from -1000 to 1000 ($n \leq 5 \cdot 10^4$). Alice and Bob alternate, and Alice starts. Each turn takes the first 1, 2 or 3 remaining stones and adds their values to the score of the player. Both play optimally. Return `"Alice"`, `"Bob"` or `"Tie"`.
5. **Minimum Swaps To Make Sequences Increasing (LeetCode 801).** Two arrays have the same length n ($2 \leq n \leq 10^5$). One operation swaps `nums1[i]` and `nums2[i]` at the same index i. Return the minimum number of operations that makes both arrays strictly increasing. The input guarantees that this is possible.
6. **Best Time to Buy and Sell Stock IV (LeetCode 188).** You get daily prices ($n \leq 1000$, prices from 0 to 1000) and an integer $k \leq 100$. You may complete at most k transactions, and you hold at most one share at a time. Return the maximum profit.

69.20 Greedy

Technique: Section 62 (page 345). Answers: Section A.19 (page 461).

1. **Two City Scheduling (LeetCode 1029).** You get $2n$ people. Person i costs `costs[i][0]` to fly to city A and `costs[i][1]` to fly to city B. Send exactly n people to each city. Return the smallest total cost. $2n \leq 100$, and each cost is from 1 to 1000.
2. **Minimum Number of Taps to Open to Water a Garden (LeetCode 1326).** A garden covers the interval $[0, n]$ of a line. The tap at each integer point i waters $[i - r_i, i + r_i]$. Return the fewest taps that water the whole garden, or -1 if no set of taps does. $n \leq 10^4$ and $0 \leq r_i \leq 100$.
3. **Array of Doubled Pairs (LeetCode 954).** You get an array of even length. It can hold negative values and zeros. Return true if you can reorder it so that `arr[2 * i + 1] == 2 * arr[2 * i]` for every pair. $n \leq 3 \cdot 10^4$ and $|a_i| \leq 10^5$.
4. **Max Chunks To Make Sorted (LeetCode 769).** The array is a permutation of $0, 1, \ldots, n-1$. Split it into contiguous chunks, sort each chunk, and join the chunks. The result must be the sorted array. Return the largest possible number of chunks. Find an $O(n)$ method.
5. **Check if a Parentheses String Can Be Valid (LeetCode 2116).** You get a string of `(` and `)`, and a binary string `locked` of the same length. You can change each character whose `locked` value is `0` into `(` or `)`.

You cannot delete a character. Return true if some set of changes makes a valid bracket sequence. $n \le 10^5$.

6. **Candy (LeetCode 135).** Children stand in a line, and each child has a rating. Give each child at least one candy. A child with a higher rating than a neighbor must get more candies than that neighbor. Return the smallest total number of candies. $n \le 2 \cdot 10^4$.

69.21 Intervals

Technique: Section 63 (page 358). Answers: Section A.20 (page 463).

1. **Remove Covered Intervals (LeetCode 1288).** You get up to 1000 distinct half-open intervals `[a, b)`. An interval `[a, b)` is covered by `[c, d)` when `c <= a` and `b <= d`. Remove every covered interval and return the number of intervals that remain. Aim for $O(n \log n)$.
2. **Interval List Intersections (LeetCode 986).** You get two lists of closed intervals. Each list is sorted by start, and its intervals are pairwise disjoint. Each list has up to 1000 intervals. Return the intersection of the two sets as a sorted list of closed intervals. A shared single point, such as `[5, 5]`, is a valid interval of the answer.
3. **Car Pooling (LeetCode 1094).** A car drives east only and has `capacity` empty seats. Each trip `[p, start, end]` picks up `p` passengers at kilometer `start` and drops them at kilometer `end`. At one kilometer, the passengers leave before new passengers get in. Up to 1000 trips, kilometers up to 1000. Return whether the car can do all trips.
4. **Remove Interval (LeetCode 1272).** A set of numbers is a sorted list of disjoint half-open intervals `[a, b)`. Remove every number in the half-open range `[lo, hi)`. Return the rest as a sorted list of disjoint intervals. Up to 10^4 intervals; the values can be negative. Aim for one $O(n)$ pass.
5. **Maximum Number of Events That Can Be Attended (LeetCode 1353).** Each event `[s, e]` can be attended on any one day `d` with `s <= d <= e`. You can attend at most one event per day. Return the largest number of events that you can attend. Up to 10^5 events, days up to 10^5, so $O(n \log n)$ is the target.
6. **Maximum Number of Non-Overlapping Subarrays With Sum Equals Target (LeetCode 1546).** You get an array `nums` of up to 10^5 integers, with negative values allowed, and an integer `target`. Return the largest number of non-empty subarrays that have the sum `target` and share no index. Aim for $O(n)$.

69.22 Matrices and simulation

Technique: Section 64 (page 367). Answers: Section A.21 (page 465).

1. **Walking Robot Simulation (LeetCode 874).** A robot starts at $(0, 0)$ on an infinite plane and faces north. Command -2 turns it left, -1 turns it right, and k in 1..9 moves it k unit steps forward. If the next unit cell is an obstacle, the robot stays on its current cell and goes to the next command. Return the largest squared distance from the origin at any time. There are up to 10^4 commands and 10^4 obstacles.
2. **Cyclically Rotating a Grid (LeetCode 1914).** An $m \times n$ grid has even m and n (2..50). It is a set of nested rings (layers). One rotation moves every element of every ring one place counterclockwise. Return the grid after k rotations, with k up to 10^9.
3. **Rotating the Box (LeetCode 1861).** An $m \times n$ box holds stones (`#`), fixed obstacles (`*`) and empty cells (`.`). Rotate the box 90 degrees clockwise. Then each stone falls down until it lands on an obstacle, another stone or the floor. Obstacles do not move. Return the $n \times m$ result. m and n are up to 500.
4. **Prison Cells After N Days (LeetCode 957).** Eight cells in a row hold 0 or 1. Each day all cells change at the same time. An inner cell becomes 1 if its two neighbors are equal, else 0. The two end cells become 0. Return the state after n days, with n up to 10^9.
5. **Find All Numbers Disappeared in an Array (LeetCode 448).** An array of n integers holds values in $1..n$. Some values repeat and some values are missing. Return all missing values in $O(n)$ time with no extra space except the output list.
6. **Remove One Element to Make the Array Strictly Increasing (LeetCode 1909).** Remove exactly one element of `nums` ($2 \le n \le 1000$). Return `True` if the rest is strictly increasing. An array that is already strictly increasing also gives `True`. The $O(n^2)$ replay fits, but find the $O(n)$ answer.

69.23 Math and number manipulation

Technique: Section 65 (page 379). Answers: Section A.22 (page 467).

1. **Add to Array-Form of Integer (LeetCode 989).** You get a digit array `num`, most significant digit first,

with up to 10^4 digits, and an integer k with $1 \le k \le 10^4$. Return the digit array of num $+ k$. Do not convert the whole array to an integer.

2. **String to Integer (atoi) (LeetCode 8).** Parse a string of up to 200 characters. Skip the leading spaces, read an optional sign, and then read digits until the first character that is not a digit. Clamp the result to $[-2^{31}, 2^{31} - 1]$, and return 0 if there are no digits. Assume that no 64-bit type exists, so the value must never leave the range while you build it.
3. **Smallest Integer Divisible by K (LeetCode 1015).** For $1 \le k \le 10^5$, find the length of the smallest positive integer that is divisible by k and has only the digit 1 (1, 11, 111, ...). Return -1 if no such integer exists. The integer itself can have up to 10^5 digits.
4. **Count Good Numbers (LeetCode 1922).** A digit string of length n is good when each digit at an even index is even (0, 2, 4, 6, 8) and each digit at an odd index is a prime (2, 3, 5, 7). Leading zeros are allowed. Return the number of good strings of length n modulo $10^9 + 7$, for $1 \le n \le 10^{15}$.
5. **Minimum Area Rectangle (LeetCode 939).** You get up to 500 distinct points with coordinates from 0 to $4 \cdot 10^4$. Return the minimum area of a rectangle with sides parallel to the axes and all four corners in the set. Return 0 if no such rectangle exists.
6. **Backward pass of max normalization.** Define $y = x / \max(x)$ for a vector x of length up to 10^5, with distinct values and a positive maximum. You get x and the upstream gradient $v = \partial L / \partial y$. Return $\partial L / \partial x$ in the original order of x, in $O(n)$ time.

69.24 Bit manipulation

Technique: Section 66 (page 392). Answers: Section A.23 (page 469).

1. **Single Number II (LeetCode 137).** In an array of 32-bit signed integers, every value occurs three times, except one value that occurs once. Return that value. The array has up to $3 \cdot 10^4$ values, and the values can be negative. Use $O(n)$ time and $O(1)$ extra space.
2. **Single Number III (LeetCode 260).** In an array of 32-bit signed integers, exactly two values occur once, and every other value occurs exactly two times. Return the two values in any order. Use $O(n)$ time and $O(1)$ extra space.
3. **Set Mismatch (LeetCode 645).** An array of length n must hold each value $1..n$ one time. One value was overwritten with a copy of another value. So one value occurs two times, and one value is missing. Return `[duplicate, missing]`. The length is $2 \le n \le 10^4$, and the values can be in any order.
4. **Total Hamming Distance (LeetCode 477).** The Hamming distance of two integers is the number of bit positions where they differ. Return the sum of the Hamming distances over all pairs in an array. The array has up to 10^4 values, each in $[0, 10^9]$. A loop over all pairs is too slow in Python.
5. **Concatenation of Consecutive Binary Numbers (LeetCode 1680).** Write each integer from 1 to n in binary, without leading zeros, and join the strings in order. Return the value of the joined binary string modulo $10^9 + 7$. The bound is $1 \le n \le 10^5$, so the joined string has about $1.6 \cdot 10^6$ bits. For $n = 3$, the string is 11011, and the answer is 27.
6. **Divide Two Integers (LeetCode 29).** Divide two signed 32-bit integers without `*`, `/` and `%`. Truncate the quotient toward zero, so $-7/2$ gives -3. The divisor is not 0. Clamp the quotient to $[-2^{31}, 2^{31} - 1]$: a quotient above $2^{31} - 1$ gives $2^{31} - 1$.

69.25 Design problems: combining data structures

Technique: Section 67 (page 400). Answers: Section A.24 (page 471).

1. **Insert Delete GetRandom O(1) (LeetCode 380).** Design a set of integers. `insert(val)` and `remove(val)` return `True` only when they change the set. `getRandom()` returns a stored value, and each stored value must have the same probability. Each method must run in $O(1)$ average time. There are at most $2 \cdot 10^5$ calls, and `getRandom` is called only on a non-empty set.
2. **Maximum Frequency Stack (LeetCode 895).** Design a stack with `push(val)` and `pop()`. `pop` removes and returns the most frequent value in the stack. On a tie, it removes the tied value that is closest to the top. Values are up to 10^9, there are at most $2 \cdot 10^4$ calls, and `pop` is called only on a non-empty stack.
3. **Design Browser History (LeetCode 1472).** A browser starts on a home page. `visit(url)` opens a new page after the current page and clears all forward history. `back(steps)` and `forward(steps)` move at most `steps` pages and return the URL of the current page. When fewer pages are available, the move stops at the first page or at the last page. There are at most 5000 calls, and `steps` is 1 to 100. Make each call $O(1)$ amortized,

independent of `steps`.

4. **LFU Cache (LeetCode 460).** Design a cache with `get(key)` and `put(key, value)`, each in $O(1)$ average time. When the cache is full, `put` of a new key evicts the key with the smallest use count. On a tie, it evicts the least recently used of those keys. A `get` of a stored key and every `put` count as a use. The capacity is 1 to 10^4, and there are at most $2 \cdot 10^5$ calls.
5. **Stock Price Fluctuation (LeetCode 2034).** Records `update(timestamp, price)` arrive out of order. A later record for the same timestamp corrects the earlier price. `current()` returns the price at the latest timestamp. `maximum()` and `minimum()` return the largest and the smallest current price over all timestamps. There are at most 10^5 calls, and the queries come only after at least one update.
6. **Serialize and Deserialize BST (LeetCode 449).** Encode a binary search tree to a string and decode the string back to the same tree. The string must be as compact as possible, and the decoder sees only the string. The tree has 0 to 10^4 nodes, with values from 0 to 10^4.

A Answers to the transfer drills

A.1 Hashing: counting, membership and canonical keys

Drills: Section 69.2 (page 418). Technique: Section 44 (page 136).

1. **4Sum II. Signal:** an exact sum over four independent choices, and $n^2 = 40{,}000$ is cheap. **Technique:** complement lookup on pair sums (meet in the middle). **Key step:** count every sum $a[i] + b[j]$ in a `Counter`. Then, for every sum $c[k] + d[l]$, add the count of its negation. Time and space are $O(n^2)$. **Hook:** Two Sum on pair sums.

```python
from collections import Counter

def four_sum_count(a, b, c, d):
    pair = Counter(x + y for x in a for y in b)
    return sum(pair[-(x + y)] for x in c for y in d)

four_sum_count([1, 2], [-2, -1], [-1, 2], [0, 2])   # -> 2
```

2. **Count Nice Pairs in an Array. Signal:** pairs $i < j$ with $n = 10^5$, and a relation in which each side mixes both indexes. **Technique:** rewrite the relation so that each side depends on one index only. Move the terms: a pair is nice when `nums[i] - rev(nums[i]) == nums[j] - rev(nums[j])`. An equality of two keys is a hash lookup. **Key step:** count the pairs with the ask-then-record scan on the key `x - rev(x)`. A value has at most 10 digits, so the time is $O(n)$. If the relation is an inequality, count the equal keys and subtract them from $n(n-1)/2$. **Hook:** put all terms of i on one side; equal keys make the pairs.

```python
from collections import Counter

def nice_pairs(nums):
    seen = Counter()
    total = 0
    for x in nums:
        key = x - int(str(x)[::-1])      # x - rev(x)
        total += seen[key]               # ask
        seen[key] += 1                   # record
    return total % (10**9 + 7)

nice_pairs([42, 11, 1, 97])              # -> 2
nice_pairs([13, 10, 35, 24, 76])         # -> 4
```

3. **Group Shifted Strings. Signal:** group items that are equal up to one detail, here the same shift of every letter. **Technique:** canonical key and a dict of lists. **Key step:** the key is the tuple of differences of neighbor letters, modulo 26. It also fixes the length. All one-letter words get the empty key, so they form one group. **Hook:** the gaps between neighbor letters, modulo 26, are the key.

```python
from collections import defaultdict

def group_shifted(words):
    groups = defaultdict(list)
    for w in words:
        key = tuple((ord(b) - ord(a)) % 26 for a, b in zip(w, w[1:]))
        groups[key].append(w)
    return list(groups.values())

group_shifted(["abc", "bcd", "acef", "xyz", "az", "ba", "a", "z"])
# -> [['abc', 'bcd', 'xyz'], ['acef'], ['az', 'ba'], ['a', 'z']]
```

4. **Least Number of Unique Integers after K Removals. Signal:** the answer depends on how often each value occurs, not on the values. **Technique:** count, then select by count. A greedy choice removes the rarest values first, because a value leaves the array only when all its copies go. **Key step:** walk the counts from small to large and remove whole values while k covers the count. A count is at most n, so buckets indexed by count give $O(n)$. A sort of the counts gives $O(n \log n)$. **Hook:** count, then delete the rarest values first.

```python
from collections import Counter

def least_unique(arr, k):
    left = len(set(arr))
    for f in sorted(Counter(arr).values()):   # rarest values first
        if k < f:
            break
        k -= f
        left -= 1
    return left

least_unique([5, 5, 4], 1)               # -> 1
least_unique([4, 3, 1, 1, 3, 3, 2], 3)   # -> 2
```

5. **Longest Arithmetic Subsequence of Given Difference. Signal:** a chain of neighbors in value space, where the neighbor of x is $x + d$. But the chain is a subsequence, so the order in the array matters. A set and a walk from each run start, as in Longest Consecutive Sequence, ignore the order. For `[3, 1, 2]` and $d = 1$, the walk finds 3 values, but the answer is 2. **Technique:** a dict from a value to the length of the longest chain that ends at that value. This is a 1-D DP with a hash map as the table. **Key step:** scan from left to right and set `best[x] = best.get(x - d, 0) + 1`. The dict holds only earlier elements, so each chain keeps the array order. The time is $O(n)$. **Hook:** each value extends the best chain that ends at $x - d$.

```python
def longest_subseq(arr, d):
    best = {}                            # longest chain that ends at each value
    for x in arr:
        best[x] = best.get(x - d, 0) + 1
    return max(best.values())

longest_subseq([1, 5, 7, 8, 5, 3, 4, 2, 1], -2)   # -> 4
longest_subseq([3, 1, 2], 1)             # -> 2
```

6. **Number of Boomerangs. Signal:** count shapes through a fixed point. The vertex i anchors the shape, as the query corner does in Detect Squares. **Technique:** fix the anchor, then group the other points by a key in a `Counter`. **Key step:** for each anchor, count the other points by squared distance. The squared distance is an exact integer key. A distance shared by c points gives $c(c-1)$ ordered pairs (j, k). Time is $O(n^2)$. The code also counts the anchor itself at distance 0. The points are distinct, so that count is 1 and adds 0. **Hook:** fix the vertex, count by distance, add $c(c-1)$.

```python
from collections import Counter

def boomerangs(points):
    total = 0
    for ax, ay in points:
        dist = Counter((x - ax) ** 2 + (y - ay) ** 2 for x, y in points)
```

```
        total += sum(c * (c - 1) for c in dist.values())
    return total

boomerangs([(0, 0), (1, 0), (2, 0)])     # -> 2
boomerangs([(1, 1)])                     # -> 0
```

A.2 Prefix sums, running aggregates and offsets

Drills: Section 69.3 (page 418). Technique: Section 45 (page 148).

1. **Signal:** the longest range with a balance of two kinds of items. **Technique:** prefix sums and a hash map of first indexes. **Key step:** map 0 to -1 and 1 to $+1$. A balanced range starts and ends at equal prefix values. Seed `first = {0: -1}`, keep only the first index of each value, and maximize `j - first[balance]`. $O(n)$. **Hook:** equal prefixes, first index wins.

```
def find_max_length(nums):
    first = {0: -1}
    balance = best = 0
    for j, x in enumerate(nums):
        balance += 1 if x == 1 else -1
        if balance in first:
            best = max(best, j - first[balance])
        else:
            first[balance] = j
    return best

find_max_length([0, 1]), find_max_length([0, 1, 0])   # -> (2, 2)
find_max_length([0, 0, 1, 0, 0, 0, 1, 1])             # -> 6
```

2. **Signal:** many updates that each add to a whole range, then one read of the final values. **Technique:** a difference array over the indexes. **Key step:** map the direction to $+1$ or -1. Do `diff[start] += v` and `diff[end + 1] -= v`. Run the prefix sum to get the net shift at each index. Apply it with `(ord(c) - 97 + shift) % 26`. Python gives a non-negative remainder, so backward shifts wrap correctly. $O(n + q)$. **Hook:** plus at the start, minus just past the end, then run the sum.

```
def shifting_letters(s, shifts):
    diff = [0] * (len(s) + 1)
    for start, end, direction in shifts:
        v = 1 if direction == 1 else -1
        diff[start] += v
        diff[end + 1] -= v
    out, shift = [], 0
    for i, c in enumerate(s):
        shift += diff[i]
        out.append(chr((ord(c) - 97 + shift) % 26 + 97))
    return "".join(out)

shifting_letters("abc", [[0, 1, 0], [1, 2, 1], [0, 2, 1]])   # -> 'ace'
shifting_letters("dztz", [[0, 0, 0], [1, 1, 1]])             # -> 'catz'
```

3. **Signal:** the best triplet with `i < j < k`. Each index adds one part of the score. **Technique:** a chain of running maximums. **Key step:** keep `best_left`, the largest `nums[i]`, and `best_diff`, the largest `nums[i] - nums[j]`. At each value `x`, first update the answer with `best_diff * x`, then `best_diff` with `best_left - x`, then `best_left` with `x`. This update order makes sure that `i < j < k`. All values are positive, so the largest difference gives the largest product. Seed all three with 0. A seed never gives a product above 0, and 0 is the lowest answer. $O(n)$ time, $O(1)$ space. A prefix maximum before `j` and a suffix maximum after `j` also work: fix the middle. **Hook:** best left, then best left minus middle, then times right.

```
def max_triplet_value(nums):
    best_left = best_diff = answer = 0
    for x in nums:
        answer = max(answer, best_diff * x)        # x is nums[k]
        best_diff = max(best_diff, best_left - x)  # x is nums[j]
        best_left = max(best_left, x)              # x is nums[i]
    return answer

max_triplet_value([12, 6, 1, 2, 7])    # -> 77
max_triplet_value([1, 10, 3, 4, 19])   # -> 133
max_triplet_value([1, 2, 3])           # -> 0
```

4. **Signal:** the best range sum, but the absolute value removes the order of the two prefixes. **Technique:** the extremes of the prefix sum. **Key step:** a range sum is `P[j] - P[i]`, so its absolute value is the distance between two prefix values. The order of `i` and `j` does not matter, so the global maximum and the global minimum of the prefixes give the answer. Include the empty prefix 0. The answer is `max(P) - min(P)`. $O(n)$ time, $O(1)$ space. Two Kadane scans, one for the largest sum and one for the smallest sum, also work. **Hook:** the absolute sum is the spread of the prefixes: highest minus lowest.

```
def max_absolute_sum(nums):
    prefix = high = low = 0              # the empty prefix counts
    for x in nums:
        prefix += x
        high, low = max(high, prefix), min(low, prefix)
    return high - low

max_absolute_sum([1, -3, 2, 3, -4])       # -> 5
max_absolute_sum([2, -5, 1, -4, 3, -2])   # -> 8
```

5. **Signal:** one rectangle sum for each cell of a static grid. **Technique:** a padded 2D prefix table. **Key step:** clamp the box: `r1 = max(0, i - k)`, `r2 = min(m - 1, i + k)`, and the same for the columns. Then use four lookups: the whole block, minus the top strip, minus the left strip, plus the corner. $O(mn)$. **Hook:** clamp the box, then whole minus strips plus corner.
6. **Signal:** a condition on the left side of `i` and a separate condition on the right side of `i`. The aggregate is a run length, so it has no inverse. **Technique:** a prefix pass and a suffix pass. **Key step:** `dec[i]` is the length of the non-increasing run that ends at `i` (left to right). `inc[i]` is the length of the non-decreasing run that starts at `i` (right to left). Index `i` is good when `dec[i - 1] >= k` and `inc[i + 1] >= k`. $O(n)$. **Hook:** left run before `i`, right run after `i`.

A.3 Two pointers

Drills: Section 69.4 (page 419). Technique: Section 46 (page 157).

1. **Signal:** a symmetry check from both ends, with one allowed fix. **Technique:** converging pointers. **Key step:** move inward while the two ends match. At the first mismatch (i, j), all outer pairs already match, so the deleted character is `s[i]` or `s[j]`. Test `s[i+1..j]` and `s[i..j-1]` with one more converging scan each. The total is $O(n)$. **Hook:** first mismatch: skip the left or the right, one time.

```
def is_pal_range(s, i, j):
    while i < j:
        if s[i] != s[j]:
            return False
        i += 1
        j -= 1
    return True

def valid_palindrome_ii(s):
    i, j = 0, len(s) - 1
    while i < j:
        if s[i] != s[j]:
            return is_pal_range(s, i + 1, j) or is_pal_range(s, i, j - 1)
```

```
        i += 1
        j -= 1
    return True

valid_palindrome_ii("aba"), valid_palindrome_ii("abca"), valid_palindrome_ii("abc")
# -> (True, True, False)
```

2. **Signal:** pairs under a sum limit and a minimum count. The order of the people does not matter, so sort. **Technique:** converging pointers with a greedy discard. **Key step:** the heaviest person who is left always needs a boat, so every step moves `hi`. The lightest person is the best partner for that person. If the lightest does not fit, nobody fits, and the heaviest rides alone. Move `lo` only when the pair fits. Loop while `lo <= hi`, because one person can be left in the middle. **Hook:** each boat takes the heaviest who is left; the lightest joins if it fits.

```
def num_boats(people, limit):
    a = sorted(people)
    lo, hi, boats = 0, len(a) - 1, 0
    while lo <= hi:
        if a[lo] + a[hi] <= limit:
            lo += 1                  # the lightest rides with the heaviest
        hi -= 1                      # the heaviest always takes this boat
        boats += 1
    return boats

num_boats([1, 2], 3), num_boats([3, 2, 2, 1], 3), num_boats([3, 5, 3, 4], 5)
# -> (1, 3, 4)
```

3. **Signal:** count triples with a sum condition; $n \leq 1000$ allows $O(n^2)$. **Technique:** sort, fix one element, and count pairs with converging pointers. **Key step:** fix the longest side `a[k]`. Then one test is enough: `a[lo] + a[hi] > a[k]` implies the two other triangle rules, because `a[k]` is the largest side. For `lo < hi < k`: if `a[lo] + a[hi] > a[k]`, every index from `lo` to `hi - 1` also works with `hi`. Add `hi - lo` and move `hi` down. Otherwise move `lo` up. A side of length 0 never counts. **Hook:** fix the longest side, count the pairs that beat it.

```
def triangle_number(nums):
    a = sorted(nums)
    count = 0
    for k in range(len(a) - 1, 1, -1):      # a[k] is the longest side
        lo, hi = 0, k - 1
        while lo < hi:
            if a[lo] + a[hi] > a[k]:
                count += hi - lo            # lo..hi-1 all work with hi
                hi -= 1
            else:
                lo += 1
    return count

triangle_number([2, 2, 3, 4]), triangle_number([4, 2, 3, 4]), triangle_number([0, 0, 1])
# -> (3, 4, 0)
```

4. **Signal:** an in-place filter of a sorted array with $O(1)$ extra memory. **Technique:** read and write pointers. **Invariant:** `a[:w]` is the answer so far, and it is sorted. So a value `x` already has two copies in the output exactly when `a[w - 2] == x`. Keep `x` when `w < 2` or `a[w - 2] != x`. Compare with the written part, not with the read part. The same rule with `w - m` keeps at most m copies. **Hook:** look two slots back in the written part.

```
def remove_duplicates_ii(a):
    w = 0
    for x in a:
        if w < 2 or a[w - 2] != x:
            a[w] = x
            w += 1
    return w
```

```
b = [0, 0, 1, 1, 1, 1, 2, 3, 3]
k = remove_duplicates_ii(b)
k, b[:k]                    # -> (7, [0, 0, 1, 1, 2, 3, 3])
```

5. **Signal:** the middle of a forward-only list, in one pass. **Technique:** fast and slow pointers, plus a dummy node. **Key step:** a deletion needs the node before the middle. Put `slow` on a dummy node in front of the head and `fast` on the head. While `fast and fast.next` exist, move `slow` one node and `fast` two nodes. Then `slow` is at index $\lfloor n/2 \rfloor - 1$, or on the dummy when $n = 1$. Set `slow.next = slow.next.next` and return `dummy.next`. **Hook:** slow starts one node behind, on a dummy.
6. **Signal:** two sorted inputs, one sorted output, in place, with the free space at the end of the first array. **Technique:** merge with one pointer per input, from the back. **Key step:** compare the largest values that are not placed yet, `nums1[i]` and `nums2[j]`. Write the larger value at `w`, and move that pointer and `w`. **Invariant:** `w == i + j + 1`, so `w > i` while `nums2` has values left. A write never destroys an unread value of `nums1`. Stop when `nums2` is empty: the rest of `nums1` is already in place. **Hook:** fill from the back; the larger of the two last values takes the last free slot.

```
def merge_in_place(nums1, m, nums2, n):
    i, j, w = m - 1, n - 1, m + n - 1
    while j >= 0:
        if i >= 0 and nums1[i] > nums2[j]:
            nums1[w] = nums1[i]
            i -= 1
        else:
            nums1[w] = nums2[j]
            j -= 1
        w -= 1

x = [1, 2, 3, 0, 0, 0]
merge_in_place(x, 3, [2, 5, 6], 3)
x                           # -> [1, 2, 2, 3, 5, 6]
y = [0]
merge_in_place(y, 0, [1], 1)
y                           # -> [1]
```

A.4 Sliding window

Drills: Section 69.5 (page 419). Technique: Section 47 (page 170).

1. **Signal:** "at most k operations" is a budget. After a sort, the best group is a contiguous block that you raise to its largest value. **Technique:** longest window under a budget, on the sorted array. **Key step:** the cost of the window is `nums[right] * length - window_sum`. The cost never drops when the window grows, so shrink while the cost is more than `k`. Keep the window sum as a running sum. The time is $O(n \log n)$ for the sort. **Hook:** sort, then raise the window to its right end.

```
def max_frequency(nums, k):
    nums = sorted(nums)
    left = total = best = 0
    for right, x in enumerate(nums):
        total += x
        while x * (right - left + 1) - total > k:   # cost to raise all to x
            total -= nums[left]
            left += 1
        best = max(best, right - left + 1)
    return best

max_frequency([1, 2, 4], 5), max_frequency([1, 4, 8, 13], 5)   # -> (3, 2)
```

2. **Signal:** the shortest substring, and a longer window is never worse, because it can replace more. **Technique:** shortest window, shrink while valid. **Key step:** the state is the letter counts *outside* the window. A letter that enters the window leaves the outside counts. The window is valid when every outside count is at most $n/4$. Record inside the shrink loop. Return 0 first if `s` is already balanced. Then the window never becomes empty.

Hook: count what stays outside; shrink while the outside is fine.

```
from collections import Counter

def balanced_string(s):
    quota = len(s) // 4
    outside = Counter(s)
    if all(outside[c] <= quota for c in "QWER"):
        return 0
    best, left = len(s), 0
    for right, ch in enumerate(s):
        outside[ch] -= 1                                  # ch moves into the window
        while all(outside[c] <= quota for c in "QWER"):  # valid: record, then shrink
            best = min(best, right - left + 1)
            outside[s[left]] += 1
            left += 1
    return best

[balanced_string(s) for s in ["QWER", "QQWE", "QQQW", "QQQQ"]]   # -> [0, 1, 2, 3]
```

3. **Signal:** count subarrays. All values are positive, so a shrink never makes the product larger, and the rule survives shrinking. **Technique:** counting window. **Key step:** multiply in `nums[right]`. While `left <= right` and the product is at least `k`, divide out `nums[left]`. Then add `right - left + 1`. For `k <= 1`, the guard keeps the window empty and every step adds 0. **Hook:** divide out from the left, add `right - left + 1`.

```
def num_subarray_product_less_than_k(nums, k):
    product = 1
    left = total = 0
    for right, x in enumerate(nums):
        product *= x
        while left <= right and product >= k:
            product //= nums[left]          # exact: nums[left] divides the product
            left += 1
        total += right - left + 1
    return total

num_subarray_product_less_than_k([10, 5, 2, 6], 100)   # -> 8
num_subarray_product_less_than_k([1, 2, 3], 0)         # -> 0
```

4. **Signal:** count subarrays with *exactly* k of something. "Exactly" does not survive shrinking, but "at most k odd numbers" does. **Technique:** counting window, two times. **Key step:** the answer is `at_most(k) - at_most(k - 1)`. The state is the number of odd values in the window. **Hook:** exactly k = at most k minus at most $k-1$.

```
def number_of_subarrays(nums, k):
    def at_most(k):
        odd = left = total = 0
        for right, x in enumerate(nums):
            odd += x % 2
            while odd > k:
                odd -= nums[left] % 2
                left += 1
            total += right - left + 1
        return total
    return at_most(k) - at_most(k - 1)

number_of_subarrays([1, 1, 2, 1, 1], 3)                  # -> 2
number_of_subarrays([2, 2, 2, 1, 2, 2, 1, 2, 2, 2], 2)   # -> 16
```

5. **Signal:** you take cards from both ends. The cards that stay always form one contiguous block of $n-k$ cards. **Technique:** fixed window on the complement. **Key step:** slide a window of size $m = n-k$ and keep its smallest sum. The answer is the total minus that sum. When $k = n$, the window is empty and the answer is the total. **Hook:** take k from the ends = leave $n-k$ in the middle.

```python
def max_score(cards, k):
    m = len(cards) - k              # the cards that stay: one window of length m
    window = sum(cards[:m])
    smallest = window
    for right in range(m, len(cards)):
        window += cards[right] - cards[right - m]
        smallest = min(smallest, window)
    return sum(cards) - smallest

max_score([1, 2, 3, 4, 5, 6, 1], 3), max_score([9, 7, 7, 9, 7, 7, 9], 7)   # -> (12, 55)
```

6. **Signal:** the longest subarray, and the rule `max - min <= limit` survives shrinking. The state needs the maximum and the minimum of a moving window. **Technique:** variable window with two monotonic deques. **Key step:** keep a deque of indices with decreasing values for the maximum, and one with increasing values for the minimum. While the two fronts differ by more than `limit`, move `left` and pop each front whose index is less than `left`. The time is $O(n)$. **Hook:** two deques: one for the max, one for the min.

```python
from collections import deque

def longest_subarray(nums, limit):
    maxq, minq = deque(), deque()   # indices; values go down / go up
    left = best = 0
    for right, x in enumerate(nums):
        while maxq and nums[maxq[-1]] <= x:
            maxq.pop()
        maxq.append(right)
        while minq and nums[minq[-1]] >= x:
            minq.pop()
        minq.append(right)
        while nums[maxq[0]] - nums[minq[0]] > limit:
            left += 1
            if maxq[0] < left:
                maxq.popleft()
            if minq[0] < left:
                minq.popleft()
        best = max(best, right - left + 1)
    return best

longest_subarray([8, 2, 4, 7], 4), longest_subarray([10, 1, 2, 4, 7, 2], 5)   # -> (2, 4)
```

A.5 Stacks and monotonic stacks

Drills: Section 69.6 (page 420). Technique: Section 48 (page 181).

1. **Minimum Remove to Make Valid Parentheses.** Signal: openers and closers must pair up, and positions matter. Technique: matching stack of *indices*. Push the index of each `(`. A `)` pops a partner, or it is marked for removal when the stack is empty. The indices left on the stack at the end are unmatched openers, so remove them too. Hook: *unmatched closers now, unmatched openers at the end.*

```python
def min_remove(s):
    stack, drop = [], set()          # open indices, indices to remove
    for i, c in enumerate(s):
        if c == "(":
            stack.append(i)
        elif c == ")":
            if stack:
                stack.pop()
            else:
                drop.add(i)          # a closer with no partner
    drop.update(stack)               # openers with no partner
    return "".join(c for i, c in enumerate(s) if i not in drop)
```

```
min_remove("lee(t(c)o)de)")   # -> 'lee(t(c)o)de'
min_remove("))((")            # -> ''
min_remove("a)b(c)d")         # -> 'ab(c)d'
```

2. **Decode String.** Signal: nested brackets, and the closer finishes the newest open unit. Technique: a stack that carries state. On `[`, push (text so far, repeat count) and start a fresh text. On `]`, pop the pair and set `text = before + text * k`. Build the count digit by digit. Hook: *push the context at the opener, repeat and glue at the closer.*

```
def decode(s):
    stack = []                        # (text before "[", repeat count)
    text, num = "", 0
    for c in s:
        if c.isdigit():
            num = num * 10 + int(c)
        elif c == "[":
            stack.append((text, num))
            text, num = "", 0
        elif c == "]":
            before, k = stack.pop()
            text = before + text * k
        else:
            text += c
    return text

decode("3[a2[c]]")        # -> 'accaccacc'
decode("2[abc]3[cd]ef")   # -> 'abcabccdcdcdef'
decode("10[a]")           # -> 'aaaaaaaaaa'
```

3. **Basic Calculator II.** Signal: infix expression with precedence and no parentheses. Technique: a stack of pending terms. Keep the operator that came before the current number. When the number ends, push `+num` or `-num`, or pop the top and push `top * num` or `int(top / num)`. The answer is the sum of the stack. Hook: *plus and minus wait on the stack, times and divide act at once.*

```
def calculate(s):
    stack, num, op = [], 0, "+"
    for i, c in enumerate(s):
        if c.isdigit():
            num = num * 10 + int(c)
        if c in "+-*/" or i == len(s) - 1:
            if op == "+":
                stack.append(num)
            elif op == "-":
                stack.append(-num)
            elif op == "*":
                stack.append(stack.pop() * num)
            else:
                stack.append(int(stack.pop() / num))   # toward zero
            num, op = 0, c
    return sum(stack)

calculate("3+2*2")        # -> 7
calculate(" 3/2 ")        # -> 1
calculate("14-3/2")       # -> 13
```

4. **Asteroid Collision.** Signal: objects on a line, and each new object meets only the newest survivor. Technique: a stack of survivors. A left-moving asteroid pops every smaller right-moving top. It stops when it meets a bigger or equal top, or when no right-moving top is left. Only a positive top followed by a negative new value collide. Hook: *a left mover smashes small right movers off the top until it meets a bigger one.*

```python
def asteroid_collision(asteroids):
    stack = []                          # survivors so far
    for a in asteroids:
        alive = True
        while alive and a < 0 and stack and stack[-1] > 0:
            if stack[-1] < -a:
                stack.pop()             # the top explodes
            elif stack[-1] == -a:
                stack.pop()             # both explode
                alive = False
            else:
                alive = False           # a explodes
        if alive:
            stack.append(a)
    return stack

asteroid_collision([5, 10, -5])     # -> [5, 10]
asteroid_collision([8, -8])         # -> []
asteroid_collision([-2, -1, 1, 2])  # -> [-2, -1, 1, 2]
asteroid_collision([1, -2, -2, -2]) # -> [-2, -2, -2]
```

5. **Online Stock Span.** Signal: previous greater element, but in a stream. Technique: a monotonic stack of (price, span) pairs with strictly decreasing prices. A new price pops every price that is less than or equal to it and absorbs their spans. Each price is pushed once and popped at most once, so each call is amortized $O(1)$. Hook: *pop the cheaper days and add up their spans.*

```python
class StockSpanner:
    def __init__(self):
        self.stack = []             # (price, span)

    def next(self, price):
        span = 1
        while self.stack and self.stack[-1][0] <= price:
            span += self.stack.pop()[1]
        self.stack.append((price, span))
        return span

sp = StockSpanner()
[sp.next(p) for p in [100, 80, 60, 70, 60, 75, 85]]   # -> [1, 1, 1, 2, 1, 4, 6]
```

6. **Sum of Subarray Minimums.** Signal: a sum over all subarrays of their minimum. Technique: fix the element that is the minimum, and find the nearest smaller element on each side with a monotonic stack. Element `i` is counted as the minimum of `(i - left) * (right - i)` subarrays. Use the previous strictly smaller element on the left and the next smaller-or-equal element on the right. Then a subarray with equal minimums is counted once. Hook: *each element counts the subarrays it rules: left reach times right reach.*

```python
def sum_subarray_mins(arr):
    n = len(arr)
    left, right = [-1] * n, [n] * n
    stack = []
    for i in range(n):
        while stack and arr[stack[-1]] >= arr[i]:
            right[stack.pop()] = i   # next smaller or equal
        left[i] = stack[-1] if stack else -1   # previous strictly smaller
        stack.append(i)
    total = sum(arr[i] * (i - left[i]) * (right[i] - i) for i in range(n))
    return total % (10**9 + 7)

sum_subarray_mins([3, 1, 2, 4])          # -> 17
sum_subarray_mins([11, 81, 94, 43, 3])   # -> 444
sum_subarray_mins([2, 2])                # -> 6
```

A.6 Binary search

Drills: Section 69.7 (page 420). Technique: Section 49 (page 191).

1. **Ship capacity.** Signal: "the smallest capacity such that it is possible", and more capacity never needs more days. Technique: binary search on the answer, first true. Key step: a greedy check counts the days. Start a new day when the next package does not fit. Bounds: `lo = max(weights)`, because the heaviest package must fit, and `hi = sum(weights)`, which needs one day. The cost is $O(n \log \sum w)$. Hook: *guess a capacity, count the days, find the first capacity that fits.*

```
def ship_within_days(weights, days):
    def days_needed(cap):
        d, load = 1, 0
        for w in weights:
            if load + w > cap:      # the package does not fit: start a new day
                d, load = d + 1, 0
            load += w
        return d
    lo, hi = max(weights), sum(weights)
    while lo < hi:
        mid = (lo + hi) // 2
        if days_needed(mid) <= days:
            hi = mid
        else:
            lo = mid + 1
    return lo

ship_within_days([1, 2, 3, 4, 5, 6, 7, 8, 9, 10], 5)   # -> 15
ship_within_days([3, 2, 2, 4, 1, 4], 3)                # -> 6
ship_within_days([1, 2, 3, 1, 1], 4)                   # -> 3
```

2. **Magnetic force.** Signal: "maximize the smallest distance". If a distance d is possible, every smaller distance is also possible. So the test is T ... T F ... F. Technique: binary search on the answer, last true. Key step: sort the positions. The check `can(d)` puts the first ball in the first basket, then each next ball in the first basket at least d after the previous ball. It passes when it places m balls. Bounds: `lo = 1` and `hi = (max(pos) - min(pos)) // (m - 1)`. Hook: *guess a gap, place the balls greedily from the left, keep the largest gap that fits.*

```
def max_distance(position, m):
    pos = sorted(position)
    def can(d):                         # greedy: each ball in the first basket far enough
        placed, last = 1, pos[0]
        for p in pos[1:]:
            if p - last >= d:
                placed, last = placed + 1, p
        return placed >= m
    lo, hi, best = 1, (pos[-1] - pos[0]) // (m - 1), 0
    while lo <= hi:
        mid = (lo + hi) // 2
        if can(mid):
            best, lo = mid, mid + 1     # mid fits: save it and try bigger
        else:
            hi = mid - 1
    return best

max_distance([1, 2, 3, 4, 7], 3)                  # -> 3
max_distance([5, 4, 3, 2, 1, 1000000000], 2)      # -> 999999999
```

3. **Snapshot array.** Signal: "the value at an index as of snapshot s" is a floor query on a sorted history. Technique: a history per index and a floor search, as in a time-based store. Key step: each index keeps a list of snapshot ids and a list of values. The ids only grow, so they stay sorted. `set` overwrites the last value if its id equals the current snapshot id. Else it appends. `get` computes `i = bisect_right(ids, snap_id)` and returns `values[i - 1]`, or 0 when `i == 0`. Hook: *history per index, floor search on the snapshot id.*

```
from bisect import bisect_right
class SnapshotArray:
    def __init__(self, length):
        self.ids = [[] for _ in range(length)]
        self.vals = [[] for _ in range(length)]
        self.snap_id = 0
    def set(self, index, val):
        ids, vals = self.ids[index], self.vals[index]
        if ids and ids[-1] == self.snap_id:
            vals[-1] = val            # a second set before the next snapshot
        else:
            ids.append(self.snap_id)
            vals.append(val)
    def snap(self):
        self.snap_id += 1
        return self.snap_id - 1
    def get(self, index, snap_id):
        i = bisect_right(self.ids[index], snap_id)   # count of ids <= snap_id
        return self.vals[index][i - 1] if i > 0 else 0

sa = SnapshotArray(3)
sa.set(0, 5)
sa.snap()                    # -> 0
sa.set(0, 6)
sa.set(0, 7)
sa.get(0, 0), sa.get(1, 0)  # -> (5, 0)
sa.snap(), sa.get(0, 1)     # -> (1, 7)
```

4. **Kth smallest in a sorted matrix.** Signal: a k-th smallest value of sorted data, but the data has no single sorted order to index. Technique: binary search on the answer value, first true. Key step: `count_le(v)` counts the values `<= v` with a staircase walk from the bottom-left corner in $O(n)$. The answer is the first v in `[matrix[0][0], matrix[-1][-1]]` with `count_le(v) >= k`. That first v is always a matrix value, because the count changes only at matrix values. The cost is $O(n \log(\text{range}))$, and the extra memory is $O(1)$. Hook: *guess a value, count the values at or below it, find the first value whose count reaches k.*

```
def kth_smallest(matrix, k):
    n = len(matrix)
    def count_le(v):                    # staircase from the bottom-left corner
        r, c, count = n - 1, 0, 0
        while r >= 0 and c < n:
            if matrix[r][c] <= v:
                count += r + 1          # this cell and every cell above it are <= v
                c += 1
            else:
                r -= 1
        return count
    lo, hi = matrix[0][0], matrix[-1][-1]
    while lo < hi:
        mid = (lo + hi) // 2
        if count_le(mid) >= k:
            hi = mid
        else:
            lo = mid + 1
    return lo

m = [[1, 5, 9], [10, 11, 13], [12, 13, 15]]
kth_smallest(m, 8), kth_smallest(m, 1), kth_smallest([[-5]], 1)    # -> (13, 1, -5)
```

5. **Nested envelopes.** Signal: the longest chain where two keys must both strictly increase, with n up to 10^5. The $O(n^2)$ chain DP is too slow. Technique: sort on one key, then a longest increasing subsequence with `bisect_left` on the other key. Key step: sort by width ascending, and by height descending for equal widths.

Then a strict LIS on the heights never takes two envelopes of the same width, because their heights go down. Hook: *sort width up and height down, then LIS on the heights.*

```
from bisect import bisect_left
def max_envelopes(envelopes):
    env = sorted(envelopes, key=lambda e: (e[0], -e[1]))   # equal widths: tallest first
    tails = []
    for _, h in env:
        pos = bisect_left(tails, h)
        if pos == len(tails):
            tails.append(h)
        else:
            tails[pos] = h
    return len(tails)

max_envelopes([[5, 4], [6, 4], [6, 7], [2, 3]])    # -> 3
max_envelopes([[1, 1], [1, 1], [1, 1]])            # -> 1
max_envelopes([[1, 2], [1, 3], [2, 4]])            # -> 2
```

6. **Peak element.** Signal: $O(\log n)$ on data that is not sorted, with a local rule at each index. Technique: a first-true loop where the kept half always holds an answer. Key step: if `nums[mid] > nums[mid + 1]`, a peak is at `mid` or to its left, so set `hi = mid`. Else a peak is to the right of `mid`, so set `lo = mid + 1`. The test is not monotone over the whole array, but the range `[lo, hi]` always holds a peak. Hook: *walk uphill: if the right neighbor is lower, a peak is at mid or to its left.*

```
def find_peak(nums):
    lo, hi = 0, len(nums) - 1
    while lo < hi:
        mid = (lo + hi) // 2
        if nums[mid] > nums[mid + 1]:
            hi = mid
        else:
            lo = mid + 1
    return lo

find_peak([1, 2, 3, 1]), find_peak([5]), find_peak([1, 2])   # -> (2, 0, 1)
find_peak([1, 2, 1, 3, 5, 6, 4]) in (1, 5)                   # -> True
```

A.7 Linked lists

Drills: Section 69.8 (page 420). Technique: Section 50 (page 203).

1. **Palindrome Linked List.** Signal: compare the two ends of a forward-only list with $O(1)$ memory. Technique: split at the middle and reverse the back half, as in the $O(1)$ form of Reorder List. Key step: put `slow` on the last node of the front half (`slow, fast = head, head.next`). Reverse `slow.next`, and compare the back half with the front half node by node. Loop on the back half, because it is never longer. Reverse it again to restore the input. Hook: *middle, reverse the back, compare.* With $O(n)$ memory, `vals == vals[::-1]` is enough.
2. **Reverse Linked List II.** Signal: change one contiguous range, and the head can change. Technique: block surgery with a dummy node and four names. Key step: walk `left - 1` steps from the dummy to `before`, then `right - left` steps from `first` to `last`. Save `after = last.next`, reverse with `prev = after`, and link `before.next = last`. Return `dummy.next`. This reads the range two times. For a strict single pass, reverse `right - left + 1` nodes with a counter and `prev = None`. Then `curr` is `after` and `prev` is `last`: set `first.next = curr` and `before.next = prev`. Hook: *before, first, last, after; prev starts at after.*
3. **Rotate List.** Signal: a cut point measured from the end, and k much larger than the length. Technique: count the length, then cut at a position from the end. Key step: return an empty list as it is, because k mod 0 raises `ZeroDivisionError`. Walk to the tail and count n. Set $k = k \bmod n$, and return the head unchanged if $k = 0$. The new tail is node $n - k - 1$ (0-indexed): its `next` is the new head. Set its link to `None`, and link the old tail to the old head. Hook: *count, k mod n, cut at n - k, close the old tail to the old head.*
4. **Partition List.** Signal: a stable split of nodes into two groups, then a join. Technique: build two output lists, each with its own dummy and tail. Key step: append each node to the small chain or the big chain. Then set

the link of the big tail to `None` and link the small tail to the first big node. Without the cut, the old link of the last big node can make a cycle. Hook: *two dummies, two tails, cut the big tail, glue.*

```
class ListNode:
    def __init__(self, val=0, next=None):
        self.val, self.next = val, next
def build(vals):
    head = None
    for v in reversed(vals):
        head = ListNode(v, head)
    return head
def to_list(head):
    out = []
    while head:
        out.append(head.val)
        head = head.next
    return out

def partition(head, x):
    small = s_tail = ListNode()
    big = b_tail = ListNode()
    while head:
        if head.val < x:
            s_tail.next = head
            s_tail = head
        else:
            b_tail.next = head
            b_tail = head
        head = head.next
    b_tail.next = None                # cut the old link
    s_tail.next = big.next            # glue
    return small.next

to_list(partition(build([1, 4, 3, 2, 5, 2]), 3))   # -> [1, 2, 2, 4, 3, 5]
to_list(partition(build([2, 1]), 2))               # -> [1, 2]
partition(None, 0) is None                         # -> True
```

5. **Sort List.** Signal: sort a linked list in $O(n \log n)$; a list has no random access, so merge sort fits. Technique: split at the middle (fast and slow pointers), sort each half, and merge two sorted lists with a dummy and a tail. Key step: use the left middle (`slow, fast = head, head.next`) and cut after it. With the right middle, a 2-node list does not split, and the recursion never ends. The recursion depth is only $\log n$. For $O(1)$ memory, merge bottom-up: sizes 1, 2, 4, and so on. Hook: *split at the middle, sort both halves, merge.*
6. **Linked List Cycle II.** Signal: one deterministic step per node, find where the loop starts, $O(1)$ memory. Technique: Floyd, both phases, as in Find the Duplicate Number but on real nodes. Key step: move `slow` one node and `fast` two nodes until they are the same node, or `fast` falls off the end. Then start a new pointer at the head and move it and `slow` one node at a time. They meet at the entrance. The reason: the step count of `slow` is a multiple of the cycle length. Let μ be the number of nodes before the cycle. Then μ more steps from the meeting point reach the entrance, and μ steps from the head reach it too. Hook: *meet, reset one to the head, walk together.*

```
class ListNode:
    def __init__(self, val=0, next=None):
        self.val, self.next = val, next

def detect_cycle(head):
    slow = fast = head
    while fast and fast.next:
        slow = slow.next
        fast = fast.next.next
        if slow is fast:              # phase 1 ends inside the cycle
            start = head
            while start is not slow: # phase 2: one step each
```

```
                start = start.next
                slow = slow.next
            return start
    return None

nodes = [ListNode(v) for v in [3, 2, 0, -4]]
for a, b in zip(nodes, nodes[1:]):
    a.next = b
detect_cycle(nodes[0]) is None       # -> True
nodes[-1].next = nodes[1]
detect_cycle(nodes[0]) is nodes[1]   # -> True
nodes[-1].next = nodes[0]
detect_cycle(nodes[0]) is nodes[0]   # -> True
```

A.8 Trees

Drills: Section 69.9 (page 421). Technique: Section 51 (page 217).

1. **Signal:** the rule pairs a node with its ancestors. **Technique:** top-down DFS with a path state. **Key step:** for node b, the best ancestor is the minimum or the maximum of the path above it. Carry `(node, path_min, path_max)` on a stack. Include the node in both values, then update the answer with `path_max - path_min`. **Hook:** carry the path min and max down.
2. **Signal:** "longest path", and it need not pass through the root. **Technique:** answer through the node, value up to the parent (as in the diameter). **Key step:** `arm(node)` returns the longest same-value chain down from the node, in edges. A child chain counts only when the child value equals the node value: `arm(child) + 1`, else 0. Record `left + right`; return `max(left, right)`. **Hook:** the diameter with a gate on equal values.
3. **Signal:** "level" and "width". **Technique:** BFS by levels with a position index for each node. **Key step:** number the nodes like a heap: the children of index i are $2i$ and $2i + 1$. The width of a level is `last - first + 1`. Subtract the first index of each level before you go down, or a deep skewed tree makes numbers with 3000 bits. **Hook:** heap numbers, last minus first plus one.

```
class TreeNode:
    def __init__(self, val=0, left=None, right=None):
        self.val, self.left, self.right = val, left, right
T = TreeNode

def width_of_tree(root):
    best, level = 0, [(root, 0)]
    while level:
        best = max(best, level[-1][1] - level[0][1] + 1)
        base = level[0][1]                      # renumber: keep the numbers small
        nxt = []
        for node, i in level:
            if node.left:
                nxt.append((node.left, 2 * (i - base)))
            if node.right:
                nxt.append((node.right, 2 * (i - base) + 1))
        level = nxt
    return best

width_of_tree(T(1, T(3, T(5), T(3)), T(2, None, T(9))))             # -> 4
width_of_tree(T(1, T(3, T(5, T(6))), T(2, None, T(9, T(7)))))       # -> 7
```

4. **Signal:** a BST and "the difference between any two values". **Technique:** the in-order stream is sorted. **Key step:** in sorted order, the closest pair is always two neighbors. Walk in-order with a stack, keep the previous value, and take the minimum of `val - prev`. Start `prev` at `None`, not at 0. **Hook:** sorted stream, compare neighbors.
5. **Signal:** two traversals and unique values. **Technique:** build a tree from traversals. **Key step:** postorder ends with the root, so read it from the end with one shared cursor. Reversed postorder lists the root, then the right subtree, then the left subtree. So build the right subtree first. Find the root in in-order with a dict, and recurse

on in-order bounds. **Hook:** postorder names the root from the end; build the right side first.

```
class TreeNode:
    def __init__(self, val=0, left=None, right=None):
        self.val, self.left, self.right = val, left, right

def build(inorder, postorder):
    pos = {v: i for i, v in enumerate(inorder)}
    roots = iter(reversed(postorder))           # root, right subtree, left subtree
    def make(lo, hi):
        if lo > hi:
            return None
        node = TreeNode(next(roots))
        mid = pos[node.val]
        node.right = make(mid + 1, hi)          # right first
        node.left = make(lo, mid - 1)
        return node
    return make(0, len(inorder) - 1)

def post(n):
    return [] if n is None else post(n.left) + post(n.right) + [n.val]
root = build([9, 3, 15, 20, 7], [9, 15, 7, 20, 3])
root.val, root.left.val, root.right.val, post(root)   # -> (3, 9, 20, [9, 15, 7, 20, 3])
```

6. **Signal:** two targets, but no order on the values. **Technique:** bottom-up DFS that reports what each subtree contains. **Key step:** `f(node)` returns p or q when the subtree holds one of them, the answer when it holds both, and `None` otherwise. Return `node` when it is p or q, or when both children return a node. Else return the non-empty side. The height can reach 10^5. For a deep tree, use parent pointers instead. Map each node to its parent with a stack. Collect p and all its ancestors in a set. Then walk up from q to the first node in the set. Put p itself in the set, because q can be in the subtree of p. **Hook:** each subtree reports what it found; found on both sides means here.

A.9 Tries

Drills: Section 69.10 (page 421). Technique: Section 52 (page 233).

1. **Map Sum Pairs.** Signal: a sum over all keys with a prefix, and keys that change. Technique: a trie with data on the nodes. Each node keeps the sum of the values of all keys below it. Key step: keep a dict of the current values. On insert, compute `delta = val - old` and add `delta` at each node on the path, the root included. `sum` walks the prefix and returns the node sum, or 0 at a missing child. Hook: *add the delta along the path.*

```
class MapSum:
    def __init__(self):
        self.root = {"sum": 0}
        self.vals = {}

    def insert(self, key, val):
        delta = val - self.vals.get(key, 0)
        self.vals[key] = val
        node = self.root
        node["sum"] += delta
        for ch in key:
            node = node.setdefault(ch, {"sum": 0})
            node["sum"] += delta

    def sum(self, prefix):
        node = self.root
        for ch in prefix:
            if ch not in node:
                return 0
            node = node[ch]
        return node["sum"]
```

```
m = MapSum()
m.insert("apple", 3)
m.sum("ap")                     # -> 3
m.insert("app", 2)
m.sum("ap")                     # -> 5
m.insert("apple", 5)            # replaces 3 with 5
m.sum("ap"), m.sum("b")         # -> (7, 0)
```

2. **Implement Magic Dictionary.** Signal: a match with exactly one different letter. This is a wildcard with a budget of one. Technique: a trie DFS with the state (index, node, used). Key step: follow the child with the same letter and keep `used`. If `used` is `False`, also follow every other child with `used = True`. At the end of the query, require the end flag and `used`. A set also works: map each masked pattern to the letters that were at the mask, and look for a letter other than the query letter. Hook: *a wildcard with a budget of one.*

```
class MagicDictionary:
    def __init__(self):
        self.root = {}

    def buildDict(self, words):
        for w in words:
            node = self.root
            for ch in w:
                node = node.setdefault(ch, {})
            node["$"] = True

    def search(self, word):
        def dfs(i, node, used):
            if i == len(word):
                return used and "$" in node
            ch = word[i]
            if ch in node and dfs(i + 1, node[ch], used):
                return True
            if not used:            # spend the one change here
                for key, child in node.items():
                    if key != "$" and key != ch and dfs(i + 1, child, True):
                        return True
            return False
        return dfs(0, self.root, False)

d = MagicDictionary()
d.buildDict(["hello", "leetcode"])
d.search("hello"), d.search("hhllo"), d.search("hell"), d.search("leetcoded")
# -> (False, True, False, False)
```

3. **Search Suggestions System.** Signal: suggestions for each prefix, in lexicographic order, from a list that does not change. Technique: sort the list and use `bisect`. Key step: for each prefix, find `i = bisect_left(products, prefix)`. Keep the words of `products[i:i + 3]` that start with the prefix. The trie form also works: insert the products in sorted order, and let each node keep the first three words that pass through it. Hook: *sort, bisect, take three.*

```
from bisect import bisect_left
def suggested(products, search_word):
    products = sorted(products)
    out, prefix = [], ""
    for ch in search_word:
        prefix += ch
        i = bisect_left(products, prefix)
        out.append([p for p in products[i:i + 3] if p.startswith(prefix)])
    return out

r = suggested(["mobile", "mouse", "moneypot", "monitor", "mousepad"], "mouse")
```

```
r[0], r[2]                      # -> (['mobile', 'moneypot', 'monitor'], ['mouse', 'mousepad'])
suggested(["havana"], "tatiana")[0]   # -> []
```

4. **Concatenated Words.** Signal: split each string into dictionary words, for many strings. Technique: a trie of the words, the walk from each start index, and a Word Break DP. Key step: sort the words by length. Run the DP on each word against the trie of the shorter words only. Then insert the word. A full split then has at least two parts, because no part can be the word itself. A set of words with the usual DP also passes, because the words are short. The trie walk stops at the first missing child. Hook: *shortest first; break each word with the words before it.*

```
def concatenated(words):
    trie, out = {}, []
    for w in sorted(words, key=len):    # only shorter words are in the trie
        n = len(w)
        ok = [True] + [False] * n       # ok[i]: w[:i] splits into words
        for i in range(n):
            if not ok[i]:
                continue
            node = trie
            for j in range(i, n):
                node = node.get(w[j])
                if node is None:
                    break
                if "$" in node:
                    ok[j + 1] = True
        if ok[n]:
            out.append(w)
        node = trie                     # insert w after its own test
        for ch in w:
            node = node.setdefault(ch, {})
        node["$"] = True
    return out

sorted(concatenated(["cat", "cats", "catsdogcats", "dog", "dogcatsdog",
                     "hippopotamuses", "rat", "ratcatdogcat"]))
# -> ['catsdogcats', 'dogcatsdog', 'ratcatdogcat']
concatenated(["a", "aa", "b"])          # -> ['aa']
```

5. **Stream of Characters.** Signal: a question about the suffixes of a growing stream. Technique: a trie of the reversed words. Key step: keep only the last $L_{\max}$ letters in a `deque`. On each query, walk the trie from the newest letter backward. Return `True` at the first end flag and `False` at a missing child. Each query costs $O(L_{\max})$. Hook: *reverse the words, read the stream backward.*

```
from collections import deque
class StreamChecker:
    def __init__(self, words):
        self.root = {}
        for w in words:
            node = self.root
            for ch in reversed(w):
                node = node.setdefault(ch, {})
            node["$"] = True
        self.recent = deque(maxlen=max(map(len, words)))

    def query(self, letter):
        self.recent.appendleft(letter)
        node = self.root
        for ch in self.recent:      # the newest letter first
            if ch not in node:
                return False
            node = node[ch]
            if "$" in node:
```

```
                return True
        return False

s = StreamChecker(["cd", "f", "kl"])
"".join("T" if s.query(ch) else "-" for ch in "abcdefghijkl")   # -> '---T-T-----T'
```

6. **Maximum XOR With an Element From Array.** Signal: the largest XOR with a query value, but only over the values up to a limit. Technique: a binary trie of 30 bits ($10^9 < 2^{30}$), and the queries in sorted order (offline). Key step: sort `nums`, and sort the query indexes by `m`. Before each query, insert every value `v <= m` that is not in the trie yet. Then walk from the highest bit and take the opposite bit when that child exists. If the trie is empty, the answer is `-1`. An online form also works: store the smallest value below each node, and enter only a child whose smallest value is at most `m`. Hook: *sort the limits, grow the trie, take the opposite bit.*

A.10 Heaps and priority queues

Drills: Section 69.11 (page 421). Technique: Section 53 (page 244).

1. **Top K Frequent Words.** Signal: the k most frequent items, a full tie rule, and a sorted output. Technique: count with `Counter`, then select the k smallest keys `(-count, word)`. A sort and a slice is the simplest. `heapq.nsmallest` gives $O(n \log k)$ and returns the items in order. A hand-made size-k heap needs the weakest kept word at the root: the lowest count, then the larger word. Words cannot be negated, so push `(count, Desc(word))` with a reversed `__lt__`. Hook: *key = (-count, word); keep the k smallest keys.*

```
import heapq
from collections import Counter

def top_k_words(words, k):
    cnt = Counter(words)
    return heapq.nsmallest(k, cnt, key=lambda w: (-cnt[w], w))

top_k_words(["i", "love", "leetcode", "i", "love", "coding"], 2)    # -> ['i', 'love']
words = "the day is sunny the the the sunny is is".split()
top_k_words(words, 4)                                               # -> ['the', 'is', 'sunny', 'day
    ']
top_k_words(["b", "a", "c"], 2)                                     # -> ['a', 'b']
```

2. **Minimum Operations to Halve Array Sum.** Signal: the same operation repeats on a free choice of item, and the largest item gives the largest gain. Technique: a max-heap simulation with negated values. Pop the largest value `x`, add `x / 2` to the reduction, push `x / 2` back, and count. Stop when the reduction is at least half of the original sum. An exchange argument shows that this greedy is optimal, and a brute force on small inputs agrees. The answer is at most n, because one halving of every number removes exactly half. Hook: *halve the biggest until half is gone.*

```
import heapq

def halve_ops(nums):
    heap = [-float(x) for x in nums]
    heapq.heapify(heap)
    target = sum(nums) / 2
    reduced, ops = 0.0, 0
    while reduced < target:
        x = -heapq.heappop(heap)
        reduced += x / 2
        heapq.heappush(heap, -(x / 2))
        ops += 1
    return ops

halve_ops([5, 19, 8, 1]), halve_ops([3, 8, 20]), halve_ops([1])    # -> (3, 3, 1)
```

3. **Reorganize String.** Signal: equal items must be apart (a cooldown of 1), and the output is an arrangement. Technique: a max-heap of letter counts with a hold-back, as in a cooldown schedule. Pop the most frequent letter and append it. Then push back the letter that you held in the previous step, and hold the current letter if

copies remain. If the heap is empty while a letter is held, no arrangement exists. A quick test: an arrangement exists exactly when the top count is at most `(len(s) + 1) // 2`. Hook: *most frequent first; hold the last letter back for one turn.*

```
import heapq
from collections import Counter

def reorganize(s):
    heap = [(-c, ch) for ch, c in Counter(s).items()]
    heapq.heapify(heap)
    out, held = [], None
    while heap:
        c, ch = heapq.heappop(heap)
        out.append(ch)
        if held:
            heapq.heappush(heap, held)          # the previous letter is allowed again
        held = (c + 1, ch) if c + 1 < 0 else None
    return "" if held else "".join(out)

reorganize("aab"), reorganize("aaab"), reorganize("aaabbc")   # -> ('aba', '', 'ababac')
```

4. **Single-Threaded CPU.** Signal: items become available over time, and each step takes the best available item. Technique: sort the task indexes by arrival time. A pointer releases the tasks that have arrived into a min-heap keyed by `(processing time, index)`. Pop the root, add its processing time to the clock, and repeat. If the heap is empty, jump the clock to the next arrival. Each task enters and leaves one time, so the cost is $O(n \log n)$. Hook: *release what has arrived, run the shortest, jump the clock when idle.*

```
import heapq

def get_order(tasks):
    order = sorted(range(len(tasks)), key=lambda i: tasks[i][0])
    heap, out, time, p = [], [], 0, 0
    while len(out) < len(tasks):
        if not heap and time < tasks[order[p]][0]:
            time = tasks[order[p]][0]           # idle: jump to the next arrival
        while p < len(order) and tasks[order[p]][0] <= time:
            heapq.heappush(heap, (tasks[order[p]][1], order[p]))
            p += 1
        proc, i = heapq.heappop(heap)
        time += proc
        out.append(i)
    return out

get_order([[1, 2], [2, 4], [3, 2], [4, 1]])             # -> [0, 2, 3, 1]
get_order([[7, 10], [7, 12], [7, 5], [7, 4], [7, 2]])   # -> [4, 3, 2, 0, 1]
```

5. **Find K Pairs with Smallest Sums.** Signal: the smallest items of a combined order, built from sorted inputs. Technique: a k-way merge on implicit streams. For each index i of `nums1`, the sums `nums1[i] + nums2[j]` for $j = 0, 1, \ldots$ form a sorted stream. Only the first k streams can matter. Seed the heap with `(nums1[i] + nums2[0], i, 0)` for each such i. Pop k times, and after each pop push the next item `(i, j + 1)` of the same stream. The cost is $O(k \log k)$. Hook: *one stream per row; pop the smallest, push its right neighbor.*

```
import heapq

def k_smallest_pairs(nums1, nums2, k):
    heap = [(nums1[i] + nums2[0], i, 0) for i in range(min(k, len(nums1)))]
    heapq.heapify(heap)
    out = []
    while heap and len(out) < k:
        _, i, j = heapq.heappop(heap)
        out.append([nums1[i], nums2[j]])
        if j + 1 < len(nums2):
            heapq.heappush(heap, (nums1[i] + nums2[j + 1], i, j + 1))
```

```
    return out

k_smallest_pairs([1, 7, 11], [2, 4, 6], 3)     # -> [[1, 2], [1, 4], [1, 6]]
k_smallest_pairs([1, 2], [3], 3)               # -> [[1, 3], [2, 3]]
```

6. **Sliding Window Median.** Signal: a rank that moves (the median), and items that leave the window. Technique: two heaps at the median, as in a running median, plus lazy deletion by index. Store `(value, index)` so that each entry is unique, and record which heap holds each index. Update that record each time a root moves to the other heap. Keep a live count for each heap. At each step, do this in order. First, when index $i - k$ leaves, decrease the live count of its heap. Do not search for it. Then route the new value through the lower heap, as in a running median. Then rebalance by the live counts, so that the lower heap has the same count as the upper heap, or one more. Before you read or move a root, pop it while its index is older than the window start. The cost is $O(n \log n)$. A simpler choice is a sorted window with `bisect.insort` and a delete by `bisect`; each step is $O(k)$, which is fast when k is small. Hook: *two heaps for the rank; old indexes leave only from the top.*

A.11 Backtracking

Drills: Section 69.12 (page 422). Technique: Section 54 (page 256).

1. **Permutations II.** Signal: all orderings, and the input repeats values. Technique: pool backtracking with one canonical form for equal values. Key step: count the values with a `Counter`. At each level, loop over the distinct values with a count above 0. Each level then places each value one time, so no ordering repeats. The sorted form also works: skip `nums[i]` when it equals `nums[i - 1]` and `nums[i - 1]` is not used. Hook: *each level places each distinct value once.*

```
from collections import Counter
def permute_unique(nums):
    count = Counter(nums)
    res, path = [], []
    def backtrack():
        if len(path) == len(nums):
            res.append(path[:])
            return
        for v in count:                        # each distinct value once per level
            if count[v] == 0:
                continue
            count[v] -= 1                      # choose
            path.append(v)
            backtrack()
            path.pop()                         # un-choose
            count[v] += 1
    backtrack()
    return res

permute_unique([1, 1, 2])                      # -> [[1, 1, 2], [1, 2, 1], [2, 1, 1]]
len(permute_unique([1, 1, 2, 2, 3, 3]))        # -> 90
```

2. **Combination Sum III.** Signal: all sets, each number used one time, a fixed size k. Technique: start index combinations under a sum. Recurse with `v + 1`, because a number cannot repeat. Key step: stop the branch when the path has k numbers, and record it only if the remainder is 0. The values go up, so `break` when `v` exceeds the remainder. Invariant: the path increases, and its sum plus the remainder is n. Hook: *start index, next number, stop at k.*
3. **Restore IP Addresses.** Signal: split a string into contiguous pieces, and each piece must pass a local test. Technique: partition backtracking over the end of the next piece, which has 1 to 3 digits. Key step: a piece is valid when it has no leading zero (except `"0"`) and its value is at most 255. Let `left` be the number of parts that are still open. Prune when the rest of the string has fewer than `left` digits or more than `3 * left` digits. Record when four parts use the full string. Hook: *cut a valid byte off the front, four times.*
4. **Path with Maximum Gold.** Signal: a walk in a grid that never repeats a cell, and the task asks for a best value. Technique: grid backtracking with marking, started from every gold cell. Key step: `dfs(r, c)` saves the

gold of the cell and sets the cell to 0 as the mark. It takes the best result of the gold neighbors, restores the cell, and returns the gold plus that best result. Do not stop at the first success: take the maximum over all branches. A dense 5×5 block of gold is the slow case, with about 3 million calls. One cheap cut helps there: stop the search when one walk collects all the gold on the grid. Hook: *mark, take the best neighbor, unmark.*

5. **Find Minimum Time to Finish All Jobs.** Signal: put every item into one of k interchangeable buckets, $n \leq 12$, and minimize the largest bucket. Technique: bucket placement with symmetry pruning and a bound from the best answer so far. Key step: sort the jobs from the largest down. For each job, try each worker, but only one worker for each distinct load. This also tries only one empty worker. Stop a branch when its current largest load is not below the best answer. Other forms: binary search on the time limit with a feasibility search, or bitmask DP over subsets of jobs. Hook: *big jobs first, one try per distinct load, cut at the best.*
6. **Sudoku Solver.** Signal: fill slots, and a digit must not repeat on its row, its column or its box. Technique: constraint placement with conflict keys. One level is one empty cell, and the options are the digits. Key step: keep one set of used digits for each row, each column and each box. The box key of `(r, c)` is `r // 3 * 3 + c // 3`. Skip a digit that one of the three sets already holds. One solution is enough, so return true at the first full board, and undo a placement only when its branch fails. To prune harder, fill the empty cell with the fewest legal digits first. Hook: *one level per empty cell; block the row, the column and the box.*

```python
def solve_sudoku(board):
    rows, cols, boxes = [[set() for _ in range(9)] for _ in range(3)]
    empty = []
    for r in range(9):
        for c in range(9):
            d = board[r][c]
            if d == ".":
                empty.append((r, c))
            else:
                rows[r].add(d)
                cols[c].add(d)
                boxes[r // 3 * 3 + c // 3].add(d)
    def fill(i):
        if i == len(empty):
            return True                         # every cell holds a digit
        r, c = empty[i]
        b = r // 3 * 3 + c // 3                 # box key
        for d in "123456789":
            if d in rows[r] or d in cols[c] or d in boxes[b]:
                continue                        # conflict: prune
            rows[r].add(d)                      # choose
            cols[c].add(d)
            boxes[b].add(d)
            board[r][c] = d
            if fill(i + 1):
                return True                     # one solution is enough
            rows[r].remove(d)                   # un-choose
            cols[c].remove(d)
            boxes[b].remove(d)
            board[r][c] = "."
        return False
    return fill(0)

puzzle = "53..7.... 6..195... .98....6. 8...6...3 4..8.3..1 7...2...6 .6....28. ...419..5 ....8..79"
board = [list(row) for row in puzzle.split()]
solve_sudoku(board)                             # -> True
["".join(row) for row in board[:3]]             # -> ['534678912', '672195348', '198342567']
["".join(row) for row in board[6:]]             # -> ['961537284', '287419635', '345286179']
```

A.12 Graph traversal: grids and implicit graphs

Drills: Section 69.13 (page 422). Technique: Section 55 (page 270).

1. **Signal:** a test of each island against a second grid with the same coordinates. **Technique:** flood fill each island of `grid2` and carry a flag. **Key step:** the flag becomes `False` when a popped cell is 0 in `grid1`. Finish the fill anyway, so that the rest of the island is marked. The rule is containment, so the border-ring check of an equality test is not necessary. Memory hook: *flag through the fill; finish before you decide.*
2. **Signal:** "cannot leave the grid" is an enclosed-region question. **Technique:** a border-first flood fill (a reverse search from the edge). **Key step:** push every border cell, sink each land cell that the flood reaches, then count the land cells that remain. One flood replaces one escape search per cell. Memory hook: *sink from the border, count what is left.*
3. **Signal:** the distance from each water cell to the nearest of many land cells. **Technique:** multi-source BFS with all land cells at distance 0. **Key step:** BFS pops cells in nondecreasing distance, so the distance of the last popped cell is the answer. With no obstacles, the BFS distance equals the Manhattan distance. Return -1 before the search if there is no land or no water. Memory hook: *all land in the queue; the last level is the answer.*
4. **Signal:** objects, a rule "bomb i reaches bomb j", and a chain reaction. The input is not a graph, so build one. **Technique:** a directed adjacency list, then one flood fill from each bomb. **Key step:** add the edge $i \to j$ when $(x_i - x_j)^2 + (y_i - y_j)^2 \le r_i^2$. Compare squares of integers, not square roots. The edge is directed: a large bomb can reach a small bomb that cannot reach it back. So union-find and one count per component give a wrong answer. The cost is $O(n^3)$, about 10^6 steps. Memory hook: *build the arrows, then flood from every bomb.*

```python
def maximum_detonation(bombs):
    n = len(bombs)
    adj = [[] for _ in range(n)]
    for i, (xi, yi, ri) in enumerate(bombs):
        for j, (xj, yj, _) in enumerate(bombs):
            if i != j and (xi - xj) ** 2 + (yi - yj) ** 2 <= ri * ri:
                adj[i].append(j)        # directed: i reaches j
    best = 0
    for start in range(n):
        seen = {start}
        stack = [start]
        while stack:
            for nxt in adj[stack.pop()]:
                if nxt not in seen:
                    seen.add(nxt)
                    stack.append(nxt)
        best = max(best, len(seen))
    return best

maximum_detonation([[1, 2, 3], [2, 3, 1], [3, 4, 2], [4, 5, 3], [5, 6, 4]])   # -> 5
maximum_detonation([[0, 0, 2], [2, 0, 1], [4, 0, 2]])   # -> 2
```

   In the last example, bombs 0 and 2 both reach bomb 1, but bomb 1 reaches no bomb. An undirected component holds all 3 bombs, but the true answer is 2.
5. **Signal:** the fewest moves between two configurations, and the state space is tiny ($6! = 720$ boards). **Technique:** BFS on an implicit state graph. **Key step:** the state is the board as a 6-character string. A fixed table lists the neighbor indexes of each position of the 0. Swap, and push each state that is not seen. Half of the boards cannot reach the goal, so BFS ends with -1 for them. Memory hook: *board as a string, swap the zero, BFS.*

```python
from collections import deque

ADJ = {0: (1, 3), 1: (0, 2, 4), 2: (1, 5), 3: (0, 4), 4: (1, 3, 5), 5: (2, 4)}

def sliding_puzzle(board):
    start = "".join(str(v) for row in board for v in row)
    seen = {start}
    queue = deque([(start, 0)])
    while queue:
        state, moves = queue.popleft()
        if state == "123450":
            return moves
        z = state.index("0")
```

```
        for k in ADJ[z]:
            s = list(state)
            s[z], s[k] = s[k], s[z]
            nxt = "".join(s)
            if nxt not in seen:
                seen.add(nxt)
                queue.append((nxt, moves + 1))
    return -1

sliding_puzzle([[1, 2, 3], [4, 0, 5]])  # -> 1
sliding_puzzle([[4, 1, 2], [5, 0, 3]])  # -> 5
sliding_puzzle([[1, 2, 3], [5, 4, 0]])  # -> -1
```

6. **Signal:** the shortest distance between two regions. **Technique:** two traversals in a chain: a flood fill, then a multi-source BFS. **Key step:** flood fill the first island that the scan finds, and mark its cells with 2. Then run a multi-source BFS from all of its cells at distance 0 across the water. When BFS pops a cell at distance d that touches a cell of the other island, return d. The path crosses d water cells, so d is the number of changes. Memory hook: *paint one island, then grow it one ring at a time until it touches the other.*

A.13 Graph structure: ordering, cycles and connectivity

Drills: Section 69.14 (page 423). Technique: Section 56 (page 282).

1. **Parallel Courses III.** Signal: "finish before" pairs, plus a value for each node that depends on all its predecessors. Technique: Kahn's algorithm with a DP update in topological order. Key step: when a course leaves the queue, its finish time is its start time plus its duration. Raise the start time of each successor to at least that finish time. A course leaves the queue only after all its predecessors, so its start time is final. The answer is the largest finish time. Hook: *Kahn order; finish = latest input plus own time.*

```
from collections import deque

def min_time(n, relations, time):
    graph = [[] for _ in range(n)]
    indegree = [0] * n
    for a, b in relations:              # 1-based labels
        graph[a - 1].append(b - 1)
        indegree[b - 1] += 1
    start = [0] * n                     # earliest start of each course
    queue = deque(v for v in range(n) if indegree[v] == 0)
    best = 0
    while queue:
        u = queue.popleft()
        finish = start[u] + time[u]
        best = max(best, finish)
        for v in graph[u]:
            start[v] = max(start[v], finish)
            indegree[v] -= 1
            if indegree[v] == 0:
                queue.append(v)
    return best

min_time(3, [[1, 3], [2, 3]], [3, 2, 5])                                  # -> 8
min_time(5, [[1, 5], [2, 5], [3, 5], [3, 4], [4, 5]], [1, 2, 3, 4, 5])    # -> 12
```

2. **Find Eventual Safe States.** Signal: "every path ends at a dead end" means "cannot reach a cycle". Technique: Kahn's algorithm on the reversed graph. Count the outgoing edges of each node. Start the queue with the nodes that have none. When a node leaves the queue, lower the count of each node that points to it, and push a node whose count reaches 0. The nodes that leave the queue are safe. A DFS with three colors also works: a node is safe if its DFS finishes black. Hook: *peel from the dead ends backward.*
3. **Satisfiability of Equality Equations.** Signal: "equal" is symmetric and transitive, so the variables fall into groups. Technique: union-find on the 26 letters. Key step: do all the unions for `==` first. Then test each `!=`: if

both sides have the same root, return `False`. A test before all unions are done can miss a later merge. Hook: *merge the equals, then test the not-equals.*

```
def equations_possible(equations):
    parent = {c: c for c in "abcdefghijklmnopqrstuvwxyz"}
    def find(x):
        while parent[x] != x:
            parent[x] = parent[parent[x]]
            x = parent[x]
        return x
    for e in equations:
        if e[1] == "=":
            parent[find(e[0])] = find(e[3])
    return all(find(e[0]) != find(e[3]) for e in equations if e[1] == "!")

equations_possible(["a==b", "b!=a"])            # -> False
equations_possible(["b!=c", "a==b", "c==a"])    # -> False
equations_possible(["a==b", "c!=d", "b==c"])    # -> True
equations_possible(["a!=a"])                    # -> False
```

4. **Number of Operations to Make Network Connected.** Signal: groups of computers and a count of moves. Technique: counting facts plus connected components. Key step: n nodes need at least $n-1$ edges, so fewer cables give -1. Otherwise count the components c with a flood fill or union-find. A spanning forest uses $n-c$ cables, so $m-(n-c) \geq c-1$ cables are spare (each one closes a cycle). Each move joins two components, so the answer is $c-1$. Hook: *enough cables? Then groups minus one.*
5. **Longest Cycle in a Graph.** Signal: at most one outgoing edge per node, so it is a functional graph. Technique: walk from each unvisited node and give each visited node a time stamp from one global clock. Key step: a walk stops at a dead end or at a node with a stamp. If that stamp is from the current walk (at least the start stamp of the walk), the cycle length is the clock minus that stamp. A stamp from an earlier walk means no new cycle. Each node gets one stamp, so the total work is $O(n)$. Hook: *stamp the walk; a stamp from this walk closes a cycle.*

```
def longest_cycle(edges):
    stamp = [0] * len(edges)           # 0 means not visited
    clock, best = 1, -1
    for s in range(len(edges)):
        if stamp[s]:
            continue
        walk_start, u = clock, s
        while u != -1 and not stamp[u]:
            stamp[u] = clock
            clock += 1
            u = edges[u]
        if u != -1 and stamp[u] >= walk_start:   # the walk met itself
            best = max(best, clock - stamp[u])
    return best

longest_cycle([3, 3, 4, 2, 3])         # -> 3
longest_cycle([2, -1, 3, 1])           # -> -1
longest_cycle([1, 0, 3, 4, 5, 2])      # -> 4
```

6. **Maximum Score of a Node Sequence.** Signal: a path of exactly 4 nodes in a large graph. Technique: fix the middle edge (a, b) and choose the two ends independently. Key step: here no labels force the nodes apart, so check $p_0 \neq b$, $p_3 \neq a$ and $p_0 \neq p_3$. At most two candidates are excluded on each side, so only the 3 best-scored neighbors of each node can matter. Precompute them, then try 3×3 pairs for each edge. The sorts cost $O(m \log m)$, and the loop over the edges costs $O(m)$. Hook: *fix the middle edge, keep the top three on each side.*

```
def maximum_score(scores, edges):
    nbrs = [[] for _ in scores]
    for a, b in edges:
        nbrs[a].append(b)
        nbrs[b].append(a)
    top = [sorted(nb, key=lambda v: -scores[v])[:3] for nb in nbrs]
    best = -1
    for a, b in edges:                  # the edge a - b is the middle p1 - p2
        for p in top[a]:
            for q in top[b]:
                if p != b and q != a and p != q:
                    best = max(best, scores[p] + scores[a] + scores[b] + scores[q])
    return best

maximum_score([5, 2, 9, 8, 4], [[0, 1], [1, 2], [2, 3], [0, 2], [1, 3], [2, 4]])   # -> 24
maximum_score([9, 20, 6, 4, 11, 12], [[0, 3], [5, 3], [2, 4], [1, 3]])             # -> -1
```

A.14 Weighted and advanced graphs

Drills: Section 69.15 (page 423). Technique: Section 57 (page 295).

1. **Signal:** the cost of a route is its largest step, and the task asks for the minimum. **Technique:** a bottleneck path on edge weights. Run Dijkstra on the grid with the key `max(e, abs(h[nr][nc] - h[r][c]))`, and return the key when the bottom-right cell pops. **Key step:** make a cell final when it pops. The cost depends on the edge, so a mark on push can give wrong answers. Binary search on the effort with one BFS for each check also works. **Hook:** max instead of plus, final on pop.
2. **Signal:** a product of factors in $[0, 1]$ never grows along a path, and the task asks for the maximum. **Technique:** Dijkstra with a max-heap. Push `(-p, node)`, and push `(-p * prob, v)` for each edge. Add each undirected edge in both directions. **Key step:** the first pop of `end` gives the answer. If `end` never pops, return 0. **Hook:** multiply, and pop the largest first.
3. **Signal:** a shortest path plus "how many". **Technique:** Dijkstra that also keeps `ways[v]`. A strictly shorter path sets `dist[v]`, copies `ways[u]` and pushes v. A path of equal length adds `ways[u]` to `ways[v]`. **Invariant:** every time is at least 1, so all the nodes that come before u on a shortest path pop before u. So `ways[u]` is final when u pops. **Hook:** a better path resets the count; an equal path adds to it.

```
import heapq
from collections import defaultdict

def count_paths(n, roads, mod=10**9 + 7):
    graph = defaultdict(list)
    for u, v, t in roads:
        graph[u].append((v, t))
        graph[v].append((u, t))
    dist, ways = [float("inf")] * n, [0] * n
    dist[0], ways[0] = 0, 1
    heap = [(0, 0)]
    while heap:
        d, u = heapq.heappop(heap)
        if d > dist[u]:
            continue                    # an old copy
        for v, t in graph[u]:
            if d + t < dist[v]:         # better: reset the count
                dist[v], ways[v] = d + t, ways[u]
                heapq.heappush(heap, (d + t, v))
            elif d + t == dist[v]:      # equal: add the count
                ways[v] = (ways[v] + ways[u]) % mod
    return ways[n - 1]

count_paths(7, [[0, 6, 7], [0, 1, 2], [1, 2, 3], [1, 3, 3], [6, 3, 3], [3, 5, 1],
                [6, 5, 1], [2, 5, 1], [0, 4, 5], [4, 6, 2]])   # -> 4
```

```
count_paths(2, [[1, 0, 10]])            # -> 1
```

4. **Signal:** connect all houses at the minimum total cost, but a house can also pay for its own source. **Technique:** an MST with a virtual node 0 for the water. A well in house i becomes the edge `(wells[i - 1], 0, i)`. Run Kruskal on the pipes and these n edges. **Key step:** the graph has $n + 1$ nodes, so stop after n kept edges. **Hook:** a well is a pipe to a virtual source.
5. **Signal:** the cheapest path under a limit on a second resource (time), not on the number of edges. **Technique:** Dijkstra on states `(fee, time, city)`, ordered by fee. Start with the fee of city 0. Push a road only when `time + t <= maxTime`, and return the fee when city $n - 1$ pops. **Key step:** skip a state when its city already popped with a time that is not larger. That earlier state had a fee that is not larger, so it is at least as good. A DP over time (minimum fee to stand at v at time t) also fits: $O(\text{maxTime} \cdot E)$. **Hook:** time is the resource: put it in the state, pop by fee.
6. **Signal:** use every pair one time, and the end of one pair is the start of the next. **Technique:** an Euler path. The values are nodes and the pairs are directed edges. Start at the node with out-degree minus in-degree equal to 1, or at any start value if all nodes are balanced. **Key step:** run Hierholzer with an explicit stack, because 10^5 edges is too deep for recursion. Reverse the post-order list of nodes, and output each two consecutive nodes as one pair. **Hook:** pairs are edges: start at the extra out-edge, write down when stuck, reverse.

A.15 Dynamic programming: foundations and 1-D

Drills: Section 69.16 (page 424). Technique: Section 58 (page 305).

1. **Delete and Earn.** *Signal:* a pick forbids its two neighbor values, and you want a maximum total. *Technique:* take or skip on the value line (House Robber). *Key step:* put all equal values in one bucket, `points[v] = v * count(v)`, because a pick of `v` can take every copy of `v`. Then values `v` and `v + 1` are neighbor houses. Run House Robber over `points[0..max]` in $O(n + \max)$. *Hook:* bucket by value, then rob the value line.

```
def delete_and_earn(nums):
    points = [0] * (max(nums) + 1)
    for x in nums:
        points[x] += x            # all copies of x together
    best = [0, 0]
    for p in points:              # House Robber on the value line
        best.append(max(best[-1], best[-2] + p))
    return best[-1]
delete_and_earn([3, 4, 2])        # -> 6
delete_and_earn([2, 2, 3, 3, 3, 4])   # -> 9
```

2. **Count Number of Texts.** *Signal:* count the ways to cut a string into valid pieces, and the answer is modulo $10^9 + 7$. *Technique:* count over the last move with validity guards (Decode Ways). *Key step:* `dp[i]` counts the messages for the prefix of length `i`, and `dp[0] = 1`. The last letter uses `k` presses, for `k` from 1 to 3 (to 4 for the keys 7 and 9). Add `dp[i - k]` only while the last `k` digits are all equal. Apply the modulo after each state. *Hook:* Climbing Stairs with guards, up to 3 or 4 steps inside a run of one digit.
3. **Minimum Cost For Tickets.** *Signal:* a minimum cost to cover points on a short line (365 days), with moves of a few fixed lengths. *Technique:* min over the last move (Min Cost Climbing Stairs) on the calendar days, not on the list of travel days. *Key step:* `dp[d]` is the minimum cost to cover the travel days up to day `d`, and `dp[0] = 0`. On a day without travel, `dp[d] = dp[d - 1]`. On a travel day, the last pass covers day `d`. Let it end on day `d`, because then it covers the most earlier days. So take the minimum of `dp[max(0, d - L)] + cost` over the pass lengths `L` of 1, 7 and 30. The cost is $O(365 \cdot 3)$. *Hook:* on a travel day, pay for the last pass and jump back its length.

```
def min_cost_tickets(days, costs):
    travel = set(days)
    dp = [0] * (days[-1] + 1)     # dp[d] = cheapest cover of the travel days up to day d
    for d in range(1, days[-1] + 1):
        if d not in travel:
            dp[d] = dp[d - 1]     # no travel on day d: no new pass
            continue
        dp[d] = min(dp[max(0, d - length)] + cost
                    for length, cost in zip((1, 7, 30), costs))
    return dp[-1]
```

```
min_cost_tickets([1, 4, 6, 7, 8, 20], [2, 7, 15])                      # -> 11
min_cost_tickets([1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 30, 31], [2, 7, 15])  # -> 17
min_cost_tickets(list(range(1, 31)), [2, 7, 15])                       # -> 15
```

4. **Maximum Sum Circular Subarray.** *Signal:* the best contiguous segment on a circle. *Technique:* Kadane plus a case split on the circle. *Key step:* a segment does not wrap, or it wraps. A wrapped segment is the whole array minus one segment that does not wrap. So run Kadane for the maximum and for the minimum in the same pass. The answer is `max(best_max, total - best_min)`. When all values are negative, `total - best_min` is the empty segment, so return `best_max`. *Hook:* max run, or total minus min run; all negative gives the max run.

```
def max_circular(nums):
    total, cur_max, cur_min = 0, 0, 0
    best_max = best_min = nums[0]
    for x in nums:
        total += x
        cur_max = max(cur_max, 0) + x
        cur_min = min(cur_min, 0) + x
        best_max = max(best_max, cur_max)
        best_min = min(best_min, cur_min)
    if best_max < 0:              # all values negative: the wrap case is empty
        return best_max
    return max(best_max, total - best_min)
max_circular([1, -2, 3, -2])      # -> 3
max_circular([5, -3, 5])          # -> 10
max_circular([-3, -2, -3])        # -> -2
```

5. **Maximum Length of Subarray With Positive Product.** *Signal:* a best contiguous segment, and a negative factor flips the sign. *Technique:* two states that end at index `i` (as in Maximum Product Subarray). *Key step:* `pos` is the longest positive product that ends here, and `neg` is the longest negative product that ends here. A positive value grows both (a `neg` of 0 stays 0). A negative value swaps them, then grows them. A zero resets both to 0. Do not multiply: the products are too large, and only the signs matter. *Hook:* carry the positive and the negative length; a negative swaps them, a zero resets them.

```
def max_positive_len(nums):
    pos = neg = best = 0          # longest positive / negative product that ends here
    for x in nums:
        if x > 0:
            pos, neg = pos + 1, (neg + 1 if neg else 0)
        elif x < 0:
            pos, neg = (neg + 1 if neg else 0), pos + 1
        else:
            pos = neg = 0
        best = max(best, pos)
    return best
max_positive_len([1, -2, -3, 4])      # -> 4
max_positive_len([0, 1, -2, -3, -4])  # -> 3
max_positive_len([-1, -2, -3, 0, 1])  # -> 2
```

6. **Number of Longest Increasing Subsequence.** *Signal:* count the best solutions, not only find the best value, with $n \leq 2000$. *Technique:* the chain template (state that ends at index `i`) with two values per index. *Key step:* for each end index `i`, keep a pair: `length[i]`, the longest chain that ends at `i`, and `count[i]`, the number of these chains. For each `j < i` with `a[j] < a[i]`, a longer `length[j] + 1` replaces the pair, and an equal length adds `count[j]`. The answer is the sum of `count[i]` over each `i` with the maximum length. This is $O(n^2)$. The `bisect_left` tails do not keep counts, so use the $O(n^2)$ form. *Hook:* carry (length, count) per end index; longer replaces, equal adds.

```
def count_lis(a):
    n = len(a)
    length = [1] * n              # longest chain that ends at index i
    count = [1] * n               # number of chains with that length
    for i in range(n):
```

```
        for j in range(i):
            if a[j] < a[i]:
                if length[j] + 1 > length[i]:
                    length[i], count[i] = length[j] + 1, count[j]    # longer: replace
                elif length[j] + 1 == length[i]:
                    count[i] += count[j]                             # equal: add
    best = max(length)
    return sum(count[i] for i in range(n) if length[i] == best)
count_lis([1, 3, 5, 4, 7])        # -> 2
count_lis([2, 2, 2, 2, 2])        # -> 5
count_lis([1, 2, 4, 3, 5, 4, 7, 2])   # -> 3
```

A.16 Dynamic programming: knapsack family

Drills: Section 69.17 (page 424). Technique: Section 59 (page 315).

1. **Signal:** each smash subtracts one stone from another, so every stone ends with a plus or a minus sign. The last weight is always $|P - N|$ for some split into two groups, and the split with the smallest difference can always be played out. So the answer is the smallest $|P - N|$. **Technique:** signs become a 0/1 subset sum (yes or no), here as a bitset. **Key step:** find the largest reachable sum $s \le total/2$ and return $total - 2s$. **Hook:** smash means signs; get as close to half as possible.

```
def last_stone_weight_ii(stones):
    total = sum(stones)
    bits = 1                                # bit s: some subset has the sum s
    for x in stones:
        bits |= bits << x
    best = max(s for s in range(total // 2 + 1) if bits >> s & 1)
    return total - 2 * best

last_stone_weight_ii([2, 7, 4, 1, 8, 1])      # -> 1
last_stone_weight_ii([31, 26, 33, 21, 40])    # -> 5
```

2. **Signal:** each string is used at most once, there are two budgets, and the question asks for the most items. **Technique:** 0/1 knapsack with two capacities and a max of the item count. **Key step:** `dp[i][j] = max(dp[i][j], dp[i - z][j - o] + 1)`, with both capacity loops going down. The cost is $O(L \cdot m \cdot n)$ for L strings. **Hook:** two budgets, both loops down, plus one per string.

```
def find_max_form(strs, m, n):
    dp = [[0] * (n + 1) for _ in range(m + 1)]   # dp[i][j]: most strings in i zeros, j ones
    for s in strs:
        z, o = s.count("0"), s.count("1")
        for i in range(m, z - 1, -1):            # down: each string once
            for j in range(n, o - 1, -1):
                dp[i][j] = max(dp[i][j], dp[i - z][j - o] + 1)
    return dp[m][n]

find_max_form(["10", "0001", "111001", "1", "0"], 5, 3)   # -> 4
find_max_form(["10", "0", "1"], 1, 1)                     # -> 2
```

3. **Signal:** exactly one face from each die, a count, and a small target. **Technique:** group knapsack: each die is a group, and the answer must take exactly one item from each group. **Key step:** build a new row for each die: `new[t] = sum(old[t - f] for f in 1..min(k, t))`. A new row stops one die from adding two faces. The cost is $O(n \cdot target \cdot k) \le 9 \cdot 10^5$; a prefix sum over the old row makes it $O(n \cdot target)$. **Hook:** one new row per die.

```
def num_rolls_to_target(n, k, target):
    MOD = 10**9 + 7
    ways = [1] + [0] * target                    # zero dice: only the sum 0
    for _ in range(n):
        new = [0] * (target + 1)                 # a new row: this die adds one face
        for t in range(1, target + 1):
```

```
            new[t] = sum(ways[t - f] for f in range(1, min(k, t) + 1)) % MOD
        ways = new
    return ways[target]

num_rolls_to_target(1, 6, 3)       # -> 1
num_rolls_to_target(2, 6, 7)       # -> 6
num_rolls_to_target(30, 30, 500)   # -> 222616187
```

4. **Signal:** count strings built by steps of fixed lengths, where the order of the blocks changes the string. **Technique:** unbounded count of ordered sequences, with the total (the length) in the outer loop, as in Combination Sum IV and Climbing Stairs. **Key step:** `ways[L] = ways[L - zero] + ways[L - one]`, then sum `ways[low..high]`. Two different step sequences always give two different strings: the first different step writes a `0` or a `1` at the same position. So the count is correct even when `zero == one`. **Hook:** the length is the total; climb stairs of sizes `zero` and `one`.

```
def count_good_strings(low, high, zero, one):
    MOD = 10**9 + 7
    ways = [1] + [0] * high                      # ways[L]: strings of length L
    for L in range(1, high + 1):
        if L >= zero:
            ways[L] += ways[L - zero]            # the last block is zeros
        if L >= one:
            ways[L] += ways[L - one]             # the last block is ones
        ways[L] %= MOD
    return sum(ways[low:high + 1]) % MOD

count_good_strings(3, 3, 1, 1)    # -> 8
count_good_strings(2, 3, 1, 2)    # -> 5
```

5. **Signal:** cut a string into dictionary words, but some characters may stay outside every word, and the question asks for a minimum. **Technique:** prefix segmentation (Word Break) with a min instead of an `or`. **Key step:** `dp[i]` is the fewest extra characters in `s[:i]`. Take `dp[i - 1] + 1` (the character `s[i - 1]` is extra), or `dp[j]` for every word `s[j:i]`. **Hook:** Word Break where a skipped character costs one.

```
def min_extra_char(s, dictionary):
    words = set(dictionary)
    dp = [0] * (len(s) + 1)                      # dp[i]: fewest extras in s[:i]
    for i in range(1, len(s) + 1):
        dp[i] = dp[i - 1] + 1                    # s[i - 1] is extra
        for j in range(i):
            if s[j:i] in words:                  # s[j:i] is the last word
                dp[i] = min(dp[i], dp[j])
    return dp[len(s)]

min_extra_char("leetscode", ["leet", "code", "leetcode"])   # -> 1
min_extra_char("sayhelloworld", ["hello", "world"])         # -> 3
```

6. **Signal:** $n \leq 14$, and the items go into an unknown number of bins with one capacity. **Technique:** bitmask DP over the set of finished tasks, as in Partition to K Equal Sum Subsets. **Key step:** for each mask, keep the smallest pair (sessions, fill of the open session). A task that fits gives (sessions, fill + t); otherwise it opens a session: (sessions + 1, t). A smaller pair is never worse later, so one pair per mask is enough. The cost is $O(n \cdot 2^n)$. **Hook:** mask of done tasks; keep (sessions, fill); open a new session only on overflow.

```
def min_sessions(tasks, session_time):
    n = len(tasks)
    best = [None] * (1 << n)
    best[0] = (1, 0)                             # one open session, 0 hours used
    for mask in range(1 << n):
        if best[mask] is None:
            continue
        sessions, fill = best[mask]
        for i in range(n):
            if mask >> i & 1:
```

```
                continue
            if fill + tasks[i] <= session_time:
                cand = (sessions, fill + tasks[i])
            else:
                cand = (sessions + 1, tasks[i])
            nxt = mask | 1 << i
            if best[nxt] is None or cand < best[nxt]:
                best[nxt] = cand
    return best[-1][0]

min_sessions([1, 2, 3], 3)          # -> 2
min_sessions([3, 1, 3, 1, 1], 8)    # -> 2
min_sessions([1, 2, 3, 4, 5], 15)   # -> 1
```

A.17 Dynamic programming on strings and sequences

Drills: Section 69.18 (page 424). Technique: Section 60 (page 325).

1. **Minimum ASCII Delete Sum.** Signal: two strings, "make them equal", a minimum cost. Technique: a two-sequence cost table, like edit distance with deletes only. `dp[i][j]` is the cheapest way to make `s1[:i]` and `s2[:j]` equal. The border is the code sum of the prefix that must go. Equal last characters take the diagonal for free. Else take the `min` of up plus `ord(s1[i - 1])` and left plus `ord(s2[j - 1])`. Hook: *edit distance with deletes only, each delete costs its code.*

```
def min_delete_sum(a, b):
    m, n = len(a), len(b)
    dp = [[0] * (n + 1) for _ in range(m + 1)]
    for i in range(1, m + 1):
        dp[i][0] = dp[i - 1][0] + ord(a[i - 1])     # delete all of a[:i]
    for j in range(1, n + 1):
        dp[0][j] = dp[0][j - 1] + ord(b[j - 1])     # delete all of b[:j]
    for i in range(1, m + 1):
        for j in range(1, n + 1):
            if a[i - 1] == b[j - 1]:
                dp[i][j] = dp[i - 1][j - 1]
            else:
                dp[i][j] = min(dp[i - 1][j] + ord(a[i - 1]),
                               dp[i][j - 1] + ord(b[j - 1]))
    return dp[m][n]

min_delete_sum("sea", "eat")            # -> 231
min_delete_sum("delete", "leet")        # -> 403
```

2. **Number of Ways to Form a Target String.** Signal: build a target in order from strictly increasing positions, and count the ways. Technique: Distinct Subsequences, where the long sequence is the list of columns. `dp[j]` is the number of ways to form `target[:j]` with the columns seen so far. For each column `k`, count `cnt[c]`, the number of words with letter `c` in that column. Then update `j` from right to left: `dp[j] += dp[j - 1] * cnt[target[j - 1]]`, modulo $10^9 + 7$. The right-to-left order uses each column at most one time. Hook: *skip plus use, and "use" has `cnt` choices.*

```
def num_ways(words, target, mod=10**9 + 7):
    t = len(target)
    dp = [1] + [0] * t                  # dp[j] = ways to form target[:j]
    for k in range(len(words[0])):
        cnt = {}
        for w in words:
            cnt[w[k]] = cnt.get(w[k], 0) + 1
        for j in range(t, 0, -1):       # right to left: column k used once
            dp[j] = (dp[j] + dp[j - 1] * cnt.get(target[j - 1], 0)) % mod
    return dp[t]
```

```
num_ways(["acca", "bbbb", "caca"], "aba")    # -> 6
num_ways(["abba", "baab"], "bab")            # -> 4
```

3. **Shortest Common Supersequence.** Signal: the shortest string that contains both inputs as subsequences. Its length is $m + n - \text{LCS}$, and the problem asks for the string itself. Technique: keep the full LCS table and walk back from (m, n). On equal last characters, write the character one time and go diagonal. Else go to the larger neighbor and write the character that the move drops: `str1[i - 1]` for up, `str2[j - 1]` for left. When one index reaches 0, keep stepping back on the other index and write each character. Reverse at the end. Hook: *walk back the LCS table; write shared characters one time and all others as you drop them.*
4. **Wildcard Matching.** Signal: a pattern with wildcard symbols must match the entire string. Technique: a pattern-matching table on (i, j), where `dp[i][j]` tells if `s[i:]` matches `p[j:]`. A `?`, or a letter equal to `s[i]`, eats one character: $(i + 1, j + 1)$. A `*` needs no peek because it stands alone: skip it, $(i, j + 1)$, or eat one character and stay, $(i + 1, j)$. The base `dp[m][n]` is true. On the row `i = m`, only stars can match. With $4 \cdot 10^6$ cells and a recursion depth up to 4000, fill the suffix table bottom-up with two rows. Hook: *a star skips, or eats one and stays.*
5. **Longest Palindromic Subsequence.** Signal: a palindrome, but the word *subsequence*, so characters can be skipped. The center walk fails because a window has more than one move. Technique: an interval table. `dp[i][j]` is the answer for `s[i:j + 1]`, and `dp[i][i] = 1`. Equal ends give `dp[i + 1][j - 1] + 2`. Else take the `max` of skipping the left end or the right end. Fill `i` from right to left. The LCS of `s` and `s[::-1]` gives the same number. Hook: *a palindromic subsequence is the LCS with its own reverse.*

```
def lps(s):
    n = len(s)
    dp = [[0] * n for _ in range(n)]
    for i in range(n - 1, -1, -1):      # row i + 1 is ready
        dp[i][i] = 1
        for j in range(i + 1, n):
            if s[i] == s[j]:
                dp[i][j] = dp[i + 1][j - 1] + 2
            else:
                dp[i][j] = max(dp[i + 1][j], dp[i][j - 1])
    return dp[0][n - 1]

lps("bbbab")                            # -> 4
lps("cbbd")                             # -> 2
```

6. **Max Dot Product of Two Subsequences.** Signal: two sequences, subsequences with skips, and a maximum. Technique: an LCS-shaped table on integers. `dp[i][j]` is the best dot product of non-empty subsequences of `nums1[:i]` and `nums2[:j]`. Pair the two last values: their product plus `max(0, dp[i - 1][j - 1])`, so a negative history is dropped. Or skip one last value: up or left. Take the `max` of the three moves. The border is $-\infty$, not 0, because the answer must use at least one pair, and all products can be negative. Hook: *LCS with a weight: pair the last two or skip one, and start from minus infinity.*

```
def max_dot(a, b):
    m, n = len(a), len(b)
    NEG = float("-inf")                 # empty prefix: no pair yet
    dp = [[NEG] * (n + 1) for _ in range(m + 1)]
    for i in range(1, m + 1):
        for j in range(1, n + 1):
            pair = a[i - 1] * b[j - 1] + max(0, dp[i - 1][j - 1])
            dp[i][j] = max(pair, dp[i - 1][j], dp[i][j - 1])
    return dp[m][n]

max_dot([2, 1, -2, 5], [3, 0, -6])      # -> 18
max_dot([3, -2], [2, -6, 7])            # -> 21
max_dot([-1, -1], [1, 1])               # -> -1
```

A.18 Dynamic programming on grids, intervals, games and states

Drills: Section 69.19 (page 425). Technique: Section 61 (page 336).

1. **Dungeon Game.** Signal: a grid with right and down moves, but the rule (health above 0 at every cell) depends on the cells that come later. A forward DP must keep two numbers (the current health and the lowest point), and one number per cell is not enough. Technique: grid path DP, filled backward from the target. `need[r][c]` is the health that you need on entering (r, c) to survive to the end: `max(1, min(need below, need right) - dungeon[r][c])`. Sentinels of infinity on the extra row and column, and 1 next to the target, remove the boundary tests. Hook: *walk back from the goal; never below 1.*

```
def calculate_minimum_hp(dungeon):
    m, n = len(dungeon), len(dungeon[0])
    INF = float("inf")
    need = [[INF] * (n + 1) for _ in range(m + 1)]    # need[r][c]: health to enter (r, c)
    need[m][n - 1] = need[m - 1][n] = 1               # sentinels next to the target
    for r in range(m - 1, -1, -1):
        for c in range(n - 1, -1, -1):
            need[r][c] = max(1, min(need[r + 1][c], need[r][c + 1]) - dungeon[r][c])
    return need[0][0]

calculate_minimum_hp([[-2, -3, 3], [-5, -10, 1], [10, 30, -5]])   # -> 7
calculate_minimum_hp([[0]])                                       # -> 1
```

2. **Number of Increasing Paths in a Grid.** Signal: paths in four directions, but each step strictly increases the value, so the move graph is a DAG. The question is a count, not a longest path. Technique: DAG DP with a sum instead of a max. `ways(cell) = 1 + sum of ways(larger neighbor)`. The answer is the sum over all cells, modulo $10^9 + 7$. Visit the cells from the largest value to the smallest. A memoized DFS can go 10^5 calls deep, so the sorted order is safer. Hook: *biggest first, sum instead of max.*
3. **Minimum Cost to Cut a Stick.** Signal: all actions happen in an order that you choose, and the cost of an action depends on the piece at that moment. At most about 100 positions allow $O(m^3)$. Technique: interval DP over the sorted cut positions with the sentinels 0 and n. A cut separates two pieces, so the *first* cut in a range is the clean split, and it costs the full range: `dp[l][r] = c[r] - c[l] + min(dp[l][k] + dp[k][r])` over the positions k strictly between l and r. Hook: *the first cut pays for the whole piece.*

```
def min_cost(n, cuts):
    c = [0] + sorted(cuts) + [n]            # sentinels: the two ends of the stick
    m = len(c)
    dp = [[0] * m for _ in range(m)]        # dp[l][r]: all cuts strictly between c[l] and c[r]
    for length in range(2, m):
        for l in range(m - length):
            r = l + length
            dp[l][r] = c[r] - c[l] + min(dp[l][k] + dp[k][r] for k in range(l + 1, r))
    return dp[0][m - 1]

min_cost(7, [1, 3, 4, 5]), min_cost(9, [5, 6, 1, 4, 2])   # -> (16, 22)
```

4. **Stone Game III.** Signal: two players, optimal play, a fixed total. Each move takes a prefix of the rest, so the state is one start index i. Technique: negamax on the score difference. `gap[i] = max(sum of the next k stones - gap[i + k])` for $k = 1, 2, 3$, and `gap[n] = 0`. A move is required and values can be negative, so start the max at minus infinity. $n = 5 \cdot 10^4$ is too deep for recursion, so fill i from $n - 1$ down to 0. Then test the sign of `gap[0]`. Hook: *my chunk minus your best gap on the rest.*

```
def stone_game_iii(values):
    n = len(values)
    gap = [0] * (n + 1)                     # gap[i]: best (mover - opponent) on values[i:]
    for i in range(n - 1, -1, -1):
        best, chunk = float("-inf"), 0
        for k in range(i, min(i + 3, n)):
            chunk += values[k]
            best = max(best, chunk - gap[k + 1])
        gap[i] = best
    return "Alice" if gap[0] > 0 else "Bob" if gap[0] < 0 else "Tie"

stone_game_iii([1, 2, 3, 7]), stone_game_iii([1, 2, 3, -9]), stone_game_iii([1, 2, 3, 6])
# -> ('Bob', 'Alice', 'Tie')
```

5. **Minimum Swaps To Make Sequences Increasing.** Signal: one yes-or-no choice per index, and a choice is legal only by the choice at the previous index. Technique: state machine with two statuses per index: `keep` (index i not swapped) and `swap` (index i swapped). If both arrays increase as they are, the status can stay the same. If they increase across (`a[i - 1] < b[i]` and `b[i - 1] < a[i]`), the status can flip. Both tests can be true, so take the min. Hook: *two statuses per index: kept or swapped.*

```
def min_swap(a, b):
    keep, swap = 0, 1                       # index 0 kept, or index 0 swapped
    for i in range(1, len(a)):
        new_keep = new_swap = float("inf")
        if a[i - 1] < a[i] and b[i - 1] < b[i]:     # same status as index i - 1
            new_keep, new_swap = keep, swap + 1
        if a[i - 1] < b[i] and b[i - 1] < a[i]:     # opposite status
            new_keep, new_swap = min(new_keep, swap), min(new_swap, keep + 1)
        keep, swap = new_keep, new_swap
    return min(keep, swap)

min_swap([1, 3, 5, 4], [1, 2, 3, 7]), min_swap([0, 3, 5, 8, 9], [2, 1, 4, 6, 9])   # -> (1, 1)
```

6. **Best Time to Buy and Sell Stock IV.** Signal: a known stock DP plus a cap on the number of uses. Technique: state machine with the transaction count in the state. `buy[j]` holds a share after j buys, and `sell[j]` holds no share after j complete transactions: `buy[j] = max(buy[j], sell[j - 1] - p)` and `sell[j] = max(sell[j], buy[j] + p)`. A buy and a sale on the same day earn 0, so the in-place update in one day is safe. The time is $O(nk) = 10^5$ steps. Hook: *add the count to the state.*

```
def max_profit_k(k, prices):
    buy = [float("-inf")] * (k + 1)         # buy[j]: own a share, j buys used
    sell = [0] * (k + 1)                    # sell[j]: no share, j transactions done
    for p in prices:
        for j in range(1, k + 1):
            buy[j] = max(buy[j], sell[j - 1] - p)
            sell[j] = max(sell[j], buy[j] + p)
    return max(sell)

max_profit_k(2, [2, 4, 1]), max_profit_k(2, [3, 2, 6, 5, 0, 3])   # -> (2, 7)
```

A.19 Greedy

Drills: Section 69.20 (page 425). Technique: Section 62 (page 345).

1. **Two City Scheduling.** Signal: split the items into two groups of fixed size, and each item has a cost in each group. Technique: exchange argument on the cost difference, then a sort. Key step: let `a` and `b` be the two costs of a person. First send all people to city B. A move of one person to city A then adds `a - b` to the total. Pick the n smallest changes: sort by `a - b` and send the first n people to city A. Exchange: if a person in B has a smaller `a - b` than a person in A, swap them. The total does not grow. Hook: *all to B, then buy the n cheapest moves to A.*

```
def two_city_cost(costs):
    costs = sorted(costs, key=lambda c: c[0] - c[1])  # cheapest move first
    n = len(costs) // 2
    return sum(a for a, _ in costs[:n]) + sum(b for _, b in costs[n:])

two_city_cost([[10, 20], [30, 200], [400, 50], [30, 20]])   # -> 110
two_city_cost([[5, 1], [9, 8], [2, 7], [4, 4]])              # -> 15
```

2. **Minimum Number of Taps to Open to Water a Garden.** Signal: cover a line with the fewest intervals. Technique: BFS by levels on a line, as in Jump Game II. Key step: for each left end `l` (clipped at 0), store the farthest right end (clipped at n) of a tap that starts there. Then run the window scan from position 0. Return -1 when a window does not grow. Hook: *the next window runs to the farthest reach of this window.*

```python
def min_taps(n, ranges):
    reach = list(range(n + 1))      # reach[l]: farthest right end of a tap that starts at l
    for i, r in enumerate(ranges):
        l = max(0, i - r)
        reach[l] = max(reach[l], min(n, i + r))
    taps = left = right = 0         # [0, right] is watered
    while right < n:
        farthest = max(reach[i] for i in range(left, right + 1))
        if farthest <= right:       # a gap that no tap covers
            return -1
        left, right, taps = right + 1, farthest, taps + 1
    return taps

min_taps(5, [3, 4, 1, 1, 0, 0])   # -> 1
min_taps(3, [0, 0, 0, 0])         # -> -1
```

3. **Array of Doubled Pairs.** Signal: pair all items by a rigid relation, and the order is free. Technique: forced choice at an extreme element, with a `Counter`. Key step: process the values by absolute value, smallest first. The value `x` with the smallest absolute value has no half left, so each copy must pair with `2 * x`. Zero pairs only with zero, so its count must be even. Hook: *the smallest absolute value left takes its double.*

```python
from collections import Counter

def can_reorder_doubled(arr):
    count = Counter(arr)
    for x in sorted(count, key=abs):
        if x == 0:
            if count[0] % 2:
                return False
            continue
        if count[2 * x] < count[x]:   # every copy of x needs a 2x
            return False
        count[2 * x] -= count[x]
    return True

can_reorder_doubled([4, -2, 2, -4])   # -> True
can_reorder_doubled([3, 1, 3, 6])     # -> False
can_reorder_doubled([0, 0, 0, 1])     # -> False
```

4. **Max Chunks To Make Sorted.** Signal: split into the most contiguous parts, with a rule for each element. Technique: cut at the farthest required end, as in Partition Labels. Key step: the value `v` must move to index `v`, so its chunk must reach index `v`. Keep `end = max(end, arr[i])`, and count a chunk when `i == end`. Hook: *each value drags the cut to its home index.*

```python
def max_chunks(arr):
    chunks = end = 0
    for i, v in enumerate(arr):
        end = max(end, v)           # v must reach index v
        if i == end:                # caught up: a legal cut
            chunks += 1
    return chunks

max_chunks([4, 3, 2, 1, 0])       # -> 1
max_chunks([1, 0, 2, 3, 4])       # -> 4
```

5. **Check if a Parentheses String Can Be Valid.** Signal: brackets with uncertain characters, and the answer is yes or no. Technique: track a range `[lo, hi]` of open counts. Key step: an unlocked character does `lo - 1` and `hi + 1`. It has no "empty" option, so all reachable counts have the parity of the prefix length. Fail when `hi < 0`. When `lo < 0`, add 2 to `lo`, not set it to 0. At the end, test `lo == 0`. An odd length then fails on its own. Hook: *a range of open counts that moves in steps of 2.*

```
def can_be_valid(s, locked):
    lo = hi = 0                     # fewest and most open "(", same parity
    for c, lock in zip(s, locked):
        if lock == "0":
            lo, hi = lo - 1, hi + 1
        elif c == "(":
            lo, hi = lo + 1, hi + 1
        else:
            lo, hi = lo - 1, hi - 1
        if hi < 0:
            return False
        if lo < 0:
            lo += 2                 # keep the parity
    return lo == 0

can_be_valid("))()))", "010100")  # -> True
can_be_valid("()()", "0000")      # -> True
can_be_valid(")", "0")            # -> False
```

6. **Candy.** Signal: each element has a rule against its left neighbor and a rule against its right neighbor. Technique: two passes, one for each direction. Key step: the left pass sets `left[i] = left[i - 1] + 1` when the rating goes up from the left, else 1. The right pass does the same from the right. Each child takes `max(left[i], right[i])`. This value satisfies both rules. Each pass value is also a lower bound, so no smaller total exists. Equal neighbors have no rule, so a child next to an equal rating can drop back to 1. Hook: *one pass for each direction, then take the max.*

```
def candy(ratings):
    n = len(ratings)
    left = [1] * n                  # rule against the left neighbor
    for i in range(1, n):
        if ratings[i] > ratings[i - 1]:
            left[i] = left[i - 1] + 1
    right = [1] * n                 # rule against the right neighbor
    for i in range(n - 2, -1, -1):
        if ratings[i] > ratings[i + 1]:
            right[i] = right[i + 1] + 1
    return sum(max(a, b) for a, b in zip(left, right))

candy([1, 0, 2])                  # -> 5
candy([1, 2, 2])                  # -> 4
candy([1, 3, 4, 5, 2])            # -> 11
```

A.20 Intervals

Drills: Section 69.21 (page 426). Technique: Section 63 (page 358).

1. **Remove Covered Intervals.** Signal: a pairwise relation ("covered by") on intervals. A check of all pairs passes at $n = 1000$, but one sort and one pass give $O(n \log n)$. Technique: sort by start, then sweep with one state, the largest end so far. Key step: sort by start up and by end down at equal starts, so a cover comes before what it covers. An interval survives only when its end is larger than the largest end so far. Hook: *start up, end down; a new largest end survives.*

```
def remove_covered(intervals):
    left, max_end = 0, float("-inf")
    for s, e in sorted(intervals, key=lambda iv: (iv[0], -iv[1])):
        if e > max_end:               # no earlier interval covers it
            left += 1
            max_end = e
    return left
remove_covered([[1, 4], [3, 6], [2, 8]]), remove_covered([[1, 4], [1, 6]])   # -> (2, 1)
```

Without the tie rule, `[1, 4]` comes before `[1, 6]` and survives by mistake.

2. **Interval List Intersections.** Signal: two sorted, disjoint lists. Do not merge them and do not sort. Technique: two pointers, one on each list. Key step: the intersection is `[max of the starts, min of the ends]`; keep it when that start is at or before that end. Then move the pointer of the interval that ends first. That interval cannot meet any later interval of the other list. Hook: *max of the starts to min of the ends; drop the one that ends first.*

```
def intersect(A, B):
    i = j = 0
    out = []
    while i < len(A) and j < len(B):
        lo, hi = max(A[i][0], B[j][0]), min(A[i][1], B[j][1])
        if lo <= hi:                # closed: a single shared point counts
            out.append([lo, hi])
        if A[i][1] < B[j][1]:
            i += 1
        else:
            j += 1
    return out
intersect([[0, 2], [5, 10], [13, 23], [24, 25]], [[1, 5], [8, 12], [15, 24], [25, 26]])
# -> [[1, 2], [5, 5], [8, 10], [15, 23], [24, 24], [25, 25]]
```

3. **Car Pooling.** Signal: "how many at the same time" against a limit; each interval carries a weight. Technique: the event sweep of Meeting Rooms II with weights. Key step: make events `(start, +p)` and `(end, -p)` and sort them. At one kilometer, the negative change sorts first, so the passengers leave before others get in (half-open trips). Return `False` as soon as the running load is above `capacity`. With kilometers up to 1000, a difference array also works. Hook: *start is +load, end is -load; the peak must fit.*

```
def car_pooling(trips, capacity):
    events = []
    for p, start, end in trips:
        events.append((start, p))
        events.append((end, -p))        # at one kilometer, -p sorts before +p
    load = 0
    for _, change in sorted(events):
        load += change
        if load > capacity:
            return False
    return True
car_pooling([[2, 1, 5], [3, 3, 7]], 4), car_pooling([[2, 1, 5], [3, 5, 7]], 3)   # -> (False, True)
```

4. **Remove Interval.** Signal: a sorted, disjoint list and one range that changes it, as in Insert Interval. Technique: three buckets in one pass, no sort. Key step: an interval with `b <= lo` or `a >= hi` shares no point with the range (half-open), so keep it whole. For any other interval, keep only its parts outside the range: `[a, lo)` if `a < lo`, and `[hi, b)` if `b > hi`. Hook: *left stays, right stays, the middle keeps only its edges.*

```
def remove_interval(intervals, lo, hi):   # all intervals half-open
    out = []
    for a, b in intervals:
        if b <= lo or a >= hi:            # no shared point: keep it whole
            out.append([a, b])
        else:
            if a < lo:
                out.append([a, lo])       # the part on the left of the range
            if b > hi:
                out.append([hi, b])       # the part on the right of the range
    return out
remove_interval([[0, 2], [3, 4], [5, 7]], 1, 6)   # -> [[0, 1], [6, 7]]
remove_interval([[0, 5]], 2, 3)                   # -> [[0, 2], [3, 5]]
remove_interval([[-5, -4], [-3, -2]], -4, -3)     # -> [[-5, -4], [-3, -2]]
```

5. **Maximum Number of Events That Can Be Attended.** Signal: items become available at a start and expire after an end; one item per time step. Technique: sweep the days with a pointer into the events sorted by

start, and a min-heap of the end days of started events. Key step: on each day, push the events that started, pop the tops that already ended (lazy deletion), then attend the event with the earliest end. When the heap is empty, jump the day to the next start. An exchange argument shows that the earliest deadline is never a worse choice. Hook: *each day: push what started, drop what ended, attend the earliest end.*

```
import heapq
def max_events(events):
    events = sorted(events)
    heap, i, n, day, count = [], 0, len(events), 0, 0
    while i < n or heap:
        if not heap:
            day = max(day, events[i][0])    # skip the empty days
        while i < n and events[i][0] <= day:
            heapq.heappush(heap, events[i][1])
            i += 1
        while heap and heap[0] < day:       # ended before today
            heapq.heappop(heap)
        if heap:
            heapq.heappop(heap)             # attend the earliest end today
            count += 1
            day += 1
    return count
max_events([[1, 2], [2, 3], [3, 4], [1, 2]]), max_events([[1, 1], [1, 1], [1, 2]])   # -> (4, 2)
```

6. **Maximum Number of Non-Overlapping Subarrays With Sum Equals Target.** Signal: "the most non-overlapping" pieces. Each subarray with the target sum is a hidden interval. Technique: keep the most by the earliest end, as in Non-overlapping Intervals, with prefix sums to find the ends. Key step: scan left to right with a running prefix sum and a set of the prefix sums seen since the last cut. When `prefix - target` is in the set, a valid subarray ends here, and it has the earliest possible end. Count it and restart the set with `seen = {prefix}`, so later subarrays start after this index. Negative values break a sliding window, so do not use one. Hook: *cut at the first end you find, then restart the prefix set.*

```
def max_non_overlapping(nums, target):
    seen, prefix, count = {0}, 0, 0
    for x in nums:
        prefix += x
        if prefix - target in seen:   # a subarray with the target sum ends here
            count += 1
            seen = {prefix}           # cut: the next subarray starts after this index
        else:
            seen.add(prefix)
    return count
max_non_overlapping([1, 1, 1, 1, 1], 2), max_non_overlapping([-1, 3, 5, 1, 4, 2, -9], 6)   # -> (2,
    2)
```

A.21 Matrices and simulation

Drills: Section 69.22 (page 426). Technique: Section 64 (page 367).

1. **Walking Robot Simulation.** Signal: a robot with a facing and turn commands. Technique: direction walk with the state `(x, y, d)`. Keep the directions in the clockwise order north, east, south, west. Then a right turn is `d + 1` and a left turn is `d - 1`, both modulo 4. Put the obstacles in a set of tuples, move one unit at a time, and check the next cell before each unit. Update the maximum after every unit step, because the question is about the whole path, not the end. Hook: *state is position plus facing; check before you step.*

```
def robot_sim(commands, obstacles):
    DIRS = [(0, 1), (1, 0), (0, -1), (-1, 0)]       # north, east, south, west
    blocked = set(map(tuple, obstacles))
    x = y = d = best = 0
    for cmd in commands:
        if cmd == -1:
            d = (d + 1) % 4
```

```
        elif cmd == -2:
            d = (d - 1) % 4
        else:
            dx, dy = DIRS[d]
            for _ in range(cmd):
                if (x + dx, y + dy) in blocked:
                    break
                x, y = x + dx, y + dy
                best = max(best, x * x + y * y)
    return best

robot_sim([4, -1, 3], [])                  # -> 25
robot_sim([4, -1, 4, -2, 4], [[2, 4]])     # -> 65
```

2. **Cyclically Rotating a Grid.** Signal: "layers" and a ring-by-ring move. Technique: shrinking bounds, one ring at a time. For the ring number `layer`, list its cells clockwise from the top-left corner. Its bounds are `layer`, `m - 1 - layer` and `n - 1 - layer`. Read the values in that order, rotate the list by `s = k % len(ring)` places, and write them back to the same cells. One counterclockwise step means each cell takes the value of the next cell in clockwise order: `ring[s:] + ring[:s]`. Hook: *peel the ring into a list, rotate the list, put it back.*

```
def rotate_grid(grid, k):
    m, n = len(grid), len(grid[0])
    for layer in range(min(m, n) // 2):
        top, left, bottom, right = layer, layer, m - 1 - layer, n - 1 - layer
        cells = ([(top, c) for c in range(left, right)]
                 + [(r, right) for r in range(top, bottom)]
                 + [(bottom, c) for c in range(right, left, -1)]
                 + [(r, left) for r in range(bottom, top, -1)])
        ring = [grid[r][c] for r, c in cells]
        s = k % len(ring)
        for (r, c), v in zip(cells, ring[s:] + ring[:s]):
            grid[r][c] = v
    return grid

rotate_grid([[40, 10], [30, 20]], 1)       # -> [[10, 20], [40, 30]]
rotate_grid([[1, 2, 3, 4], [5, 6, 7, 8], [9, 10, 11, 12], [13, 14, 15, 16]], 2)
# -> [[3, 4, 8, 12], [2, 11, 10, 16], [1, 7, 6, 15], [5, 9, 13, 14]]
```

3. **Rotating the Box.** Signal: a rotation plus a gravity rule. Technique: map one cell, then simulate in the easier frame. A clockwise turn sends (i, j) to $(j, m-1-i)$, so "down" after the turn is "right" before it. Let the stones fall right in each original row first. Scan the row from the right end and keep a write index `empty` for the next free landing cell. An obstacle at `j` resets `empty` to `j - 1`. Then rotate with `zip(*box[::-1])`. The cost is $O(mn)$. Hook: *fall right first, then turn.*

```
def rotate_the_box(box):
    box = [row[:] for row in box]
    for row in box:
        empty = len(row) - 1                # the next free landing cell
        for j in range(len(row) - 1, -1, -1):
            if row[j] == "*":
                empty = j - 1
            elif row[j] == "#":             # a stone
                row[j], row[empty] = ".", row[j]
                empty -= 1
    return [list(r) for r in zip(*box[::-1])]

rotate_the_box([["#", ".", "#"]])          # -> [['.'], ['#'], ['#']]
rotate_the_box([["#", ".", "*", "."], ["#", "#", "*", "."]])
# -> [['#', '.'], ['#', '#'], ['*', '*'], ['.', '.']]
```

4. **Prison Cells After N Days.** Signal: all cells change "at the same time", and n is huge while there are only 2^8 states. Technique: a synchronous step into a new tuple, plus a cycle skip. Build each new day from the old

day only. Store the first day of each state in a dict. At the first repeat, the period is known. Simulate only `(n - t) % period` more days. The first state can lie before the cycle, because its end cells can be 1. So do not assume that the cycle starts at day 0. Hook: *new row from the old row; at a repeat, skip the full cycles.*

```
def prison_after(cells, n):
    step = lambda s: tuple(0 if i in (0, 7) else int(s[i - 1] == s[i + 1]) for i in range(8))
    state, first_seen, t = tuple(cells), {}, 0
    while t < n:
        if state in first_seen:
            for _ in range((n - t) % (t - first_seen[state])):
                state = step(state)
            return list(state)
        first_seen[state] = t
        state = step(state)
        t += 1
    return list(state)

prison_after([0, 1, 0, 1, 1, 0, 0, 1], 7)           # -> [0, 0, 1, 1, 0, 0, 0, 0]
prison_after([1, 0, 0, 1, 0, 0, 1, 0], 10**9)       # -> [0, 0, 1, 1, 1, 1, 1, 0]
```

5. **Find All Numbers Disappeared in an Array.** Signal: values in $1..n$, one flag per value, and no extra space. Technique: an in-place marker. Borrow the sign of `nums[x - 1]` as the flag "value x is present". For each `x`, use `abs(x)`, because an earlier mark can already have made it negative. Make `nums[abs(x) - 1]` negative if it is positive. At the end, each index `i` with a positive value gives the missing value `i + 1`. Hook: *the sign is the seen-flag; read with abs.*

```
def disappeared(nums):
    for x in nums:
        i = abs(x) - 1
        if nums[i] > 0:
            nums[i] = -nums[i]               # mark: value i + 1 is present
    return [i + 1 for i, v in enumerate(nums) if v > 0]

disappeared([4, 3, 2, 7, 8, 2, 3, 1])        # -> [5, 6]
disappeared([1, 1])                          # -> [2]
```

6. **Remove One Element to Make the Array Strictly Increasing.** Signal: "exactly one" edit of a sequence. Technique: try every single edit with prefix states. Compute `pre[i]` ("`nums[:i]` is strictly increasing") forward and `suf[i]` ("`nums[i:]` is strictly increasing") backward. A removal of index `i` works when `pre[i]` and `suf[i + 1]` hold and the two parts join. They join when `i` is an end, or when `nums[i - 1] < nums[i + 1]`. This is $O(n)$. The baseline removes each index and rechecks in $O(n^2)$, which also fits. Hook: *good prefix, good suffix, and the edit must join them.*

```
def can_be_increasing(nums):
    n = len(nums)
    pre = [True] * (n + 1)                   # pre[i]: nums[:i] is strictly increasing
    for i in range(2, n + 1):
        pre[i] = pre[i - 1] and nums[i - 2] < nums[i - 1]
    suf = [True] * (n + 1)                   # suf[i]: nums[i:] is strictly increasing
    for i in range(n - 2, -1, -1):
        suf[i] = suf[i + 1] and nums[i] < nums[i + 1]
    return any(pre[i] and suf[i + 1] and (i in (0, n - 1) or nums[i - 1] < nums[i + 1])
               for i in range(n))

can_be_increasing([1, 2, 10, 5, 7])          # -> True
can_be_increasing([2, 3, 1, 2])              # -> False
can_be_increasing([1, 1, 1])                 # -> False
can_be_increasing([1, 2, 3])                 # -> True
```

A.22 Math and number manipulation

Drills: Section 69.23 (page 426). Technique: Section 65 (page 379).

1. **Add to Array-Form of Integer.** A digit array plus a small integer → a carry loop from the right end. Use k itself as the carry. At each position from the right, add the digit to k, write k mod 10, and set k to $\lfloor k/10 \rfloor$. Continue while digits remain or k is not 0, then reverse the output. Hook: *k is the carry.*

```python
def add_to_array_form(num, k):
    out, i = [], len(num) - 1
    while i >= 0 or k:
        if i >= 0:
            k += num[i]
            i -= 1
        k, digit = divmod(k, 10)
        out.append(digit)
    return out[::-1]
add_to_array_form([1, 2, 0, 0], 34)   # -> [1, 2, 3, 4]
add_to_array_form([9, 9], 1)          # -> [1, 0, 0]
add_to_array_form([0], 10000)         # -> [1, 0, 0, 0, 0]
```

2. **String to Integer (atoi).** A number built digit by digit inside a fixed range, with no wider type → the digit push with a guard before each push. Keep the sign apart and build the magnitude. Before `res = res * 10 + digit`, test `res > (INT_MAX - digit) // 10`. If the test is true, return `INT_MAX` for a plus sign and `INT_MIN` for a minus sign. One bound is enough: a magnitude of exactly 2^{31} with a minus sign clamps to `INT_MIN`, which is the correct value. Test for a digit with `"0" <= c <= "9"`. `str.isdigit` also accepts other Unicode digits, such as a superscript 2. Hook: *sign aside, guard each push, clamp.*

```python
INT_MAX, INT_MIN = 2**31 - 1, -2**31
def my_atoi(s):
    i, n = 0, len(s)
    while i < n and s[i] == " ":
        i += 1
    sign = 1
    if i < n and s[i] in "+-":
        sign = -1 if s[i] == "-" else 1
        i += 1
    res = 0
    while i < n and "0" <= s[i] <= "9":          # ASCII digits only
        digit = ord(s[i]) - ord("0")
        if res > (INT_MAX - digit) // 10:        # the push would pass INT_MAX
            return INT_MAX if sign == 1 else INT_MIN
        res = res * 10 + digit
        i += 1
    return sign * res
my_atoi("   -42"), my_atoi("4193 with words"), my_atoi("words 1")   # -> (-42, 4193, 0)
my_atoi("-91283472332"), my_atoi("2147483648")                     # -> (-2147483648, 2147483647)
my_atoi("-2147483648"), my_atoi("+-1")                             # -> (-2147483648, 0)
"\u00b2".isdigit()      # -> True
```

3. **Smallest Integer Divisible by K.** The number is too long, and only its remainder matters → a walk on a finite set of states. Push a digit 1 into the remainder: $r \leftarrow (10r + 1) \bmod k$. Stop when $r = 0$. There are only k remainders, so after k steps without a 0, a remainder repeated and the walk cycles forever. Hook: *push a 1 into the remainder; k states, so k steps at most.*

```python
def smallest_repunit_div_by_k(k):
    r = 0
    for length in range(1, k + 1):
        r = (r * 10 + 1) % k
        if r == 0:
            return length
    return -1
smallest_repunit_div_by_k(1), smallest_repunit_div_by_k(3)   # -> (1, 3)
smallest_repunit_div_by_k(7), smallest_repunit_div_by_k(2)   # -> (6, -1)
```

4. **Count Good Numbers.** A huge n, and a count that is a product of independent choices → modular fast power. There are $\lceil n/2 \rceil$ even indexes with 5 choices and $\lfloor n/2 \rfloor$ odd indexes with 4 choices. The answer is

$5^{\lceil n/2 \rceil} \cdot 4^{\lfloor n/2 \rfloor} \bmod (10^9 + 7)$, by repeated squaring or `pow(a, b, m)`. Hook: *count the choices per slot, then square and multiply.*

```
MOD = 10**9 + 7
def count_good_numbers(n):
    return pow(5, (n + 1) // 2, MOD) * pow(4, n // 2, MOD) % MOD
count_good_numbers(1), count_good_numbers(4), count_good_numbers(50)   # -> (5, 400, 564908303)
```

5. **Minimum Area Rectangle.** An axis-aligned shape from a point set → enumerate the free part, and look up the forced corners. Two opposite corners (x_1, y_1) and (x_2, y_2) with $x_1 \neq x_2$ and $y_1 \neq y_2$ fix the rectangle. Check (x_1, y_2) and (x_2, y_1) in a set, and keep the smallest area. There are about 125 000 pairs. Hook: *pick a diagonal, the other two corners are forced.*

```
def min_area_rect(points):
    seen = set(map(tuple, points))
    best = float("inf")
    for i, (x1, y1) in enumerate(points):
        for x2, y2 in points[i + 1:]:
            if x1 != x2 and y1 != y2 and (x1, y2) in seen and (x2, y1) in seen:
                best = min(best, abs(x1 - x2) * abs(y1 - y2))
    return 0 if best == float("inf") else best
min_area_rect([[1, 1], [1, 3], [3, 1], [3, 3], [2, 2]])           # -> 4
min_area_rect([[1, 1], [1, 3], [3, 1], [3, 3], [4, 1], [4, 3]])   # -> 2
min_area_rect([[0, 0], [1, 1]])                                   # -> 0
```

6. **Backward pass of max normalization.** A vector divided by a statistic of itself, with v given → the chain rule through an index map. The direct path gives v_i/M to every element. The path through $M = \max(x)$ gives $-(v \cdot y)/M$, and only the argmax receives it, because the max picks one element. Add this term to the direct term of the argmax. Hook: *v over M everywhere, minus dot over M at the argmax.*

```
def max_norm_backward(x, v):
    j = max(range(len(x)), key=x.__getitem__)       # the argmax
    M = x[j]
    grad = [vk / M for vk in v]                     # direct path
    dot = sum(vk * xk / M for vk, xk in zip(v, x))  # v . y
    grad[j] -= dot / M                              # path through the max
    return grad
max_norm_backward([1.0, 4.0, 2.0], [1.0, 1.0, 1.0])   # -> [0.25, -0.1875, 0.25]
```

A.23 Bit manipulation

Drills: Section 69.24 (page 427). Technique: Section 66 (page 392).

1. **Single Number II.** Signal: groups of three and $O(1)$ space. Plain XOR fails, because three copies do not cancel. Technique: per-bit counts (one column at a time) and the 32-bit sign fix. Key step: for each bit `k` from 0 to 31, count the values with bit `k` set. The triples add a multiple of 3, so bit `k` of the answer is `count % 3`. For a negative value, `(x >> k) & 1` still reads its 32-bit pattern. Convert bit 31 to a sign at the end. The one-pass optimal form keeps two masks, `ones` and `twos`, as a counter modulo 3. Hook: *count each column, keep it modulo 3, fix the sign.*

```
def single_number_ii(nums):
    u = 0
    for k in range(32):
        count = sum((x >> k) & 1 for x in nums)   # ones in column k
        if count % 3:                             # the triples add a multiple of 3
            u |= 1 << k
    return u if u <= 0x7FFFFFFF else u - (1 << 32)   # bit 31 set means negative

single_number_ii([2, 2, 3, 2])                    # -> 3
single_number_ii([0, 1, 0, 1, 0, 1, 99])          # -> 99
single_number_ii([5, -7, 5, 5])                   # -> -7
```

2. **Single Number III.** Signal: pairs cancel, but two values have no pair. Technique: XOR fold, then a split.

Key step: the fold gives `a ^ b`, which is not 0 because `a != b`. `low = both & -both` keeps one bit where `a` and `b` differ. Split all values by `x & low` and fold each group. Both copies of a pair go to the same group, so they still cancel. Hook: *XOR all, split by a differing bit, XOR each half.*

```
def single_number_iii(nums):
    both = 0
    for x in nums:
        both ^= x                   # a ^ b: the pairs cancel
    low = both & -both              # one bit where a and b differ
    a = b = 0
    for x in nums:
        if x & low:
            a ^= x                  # both copies of a pair land in one group
        else:
            b ^= x
    return sorted([a, b])

single_number_iii([1, 2, 1, 3, 2, 5])   # -> [3, 5]
single_number_iii([-1, 0])              # -> [-1, 0]
```

3. **Set Mismatch.** Signal: a known range $1..n$ with one value extra and one value missing. Technique: expected minus actual, with two unknowns. Key step: a set removes the copy. The duplicate is `sum(nums) - sum(set(nums))`, and the missing value is the expected sum minus `sum(set(nums))`. For $O(1)$ extra space, use two equations: the sum difference gives $dup - miss$, and the sum-of-squares difference gives $dup^2 - miss^2$. Divide them to get $dup + miss$. Hook: *expected minus actual; the set removes the copy.*

```
def find_error_nums(nums):
    n = len(nums)
    distinct = sum(set(nums))           # each value one time: the copy is gone
    return [sum(nums) - distinct, n * (n + 1) // 2 - distinct]

find_error_nums([1, 2, 2, 4])           # -> [2, 3]
find_error_nums([2, 2])                 # -> [2, 1]
```

4. **Total Hamming Distance.** Signal: a question about all pairs, and each bit position counts on its own. Technique: per-bit counts, not pairs. Key step: in column `k`, let `c` values have a 1. Each pair of one value with a 1 and one value with a 0 differs there, so the column adds `c * (n - c)`. Sum over 30 columns, because $10^9 < 2^{30}$. The cost is $O(30n)$. Hook: *per column, ones times zeros.*

```
def total_hamming_distance(nums):
    n, total = len(nums), 0
    for k in range(30):                     # 10^9 < 2^30
        c = sum((x >> k) & 1 for x in nums)  # ones in column k
        total += c * (n - c)                # each one-zero pair differs here
    return total

total_hamming_distance([4, 14, 2])          # -> 6
total_hamming_distance([4, 14, 4])          # -> 4
```

5. **Concatenation of Consecutive Binary Numbers.** Signal: the bits of each number go onto the low end of one result, and the work repeats for every `i` from 1 to n. Technique: the push step of Template 1 (shift left, then OR), with a fixed width per number, and an answer that reuses the previous step. Key step: shift `res` left by the bit length of `i`, then OR in `i`. The bit length grows by 1 exactly when `i` is a power of two, so test `i & (i - 1) == 0`. Take the modulo after each push, so `res` stays small. The cost is $O(n)$. Hook: *grow the width at each power of two, shift by the width, OR in i.*

```
MOD = 10**9 + 7
def concatenated_binary(n):
    res, width = 0, 0
    for i in range(1, n + 1):
        if i & (i - 1) == 0:                # i is a power of two: one more bit
            width += 1
        res = ((res << width) | i) % MOD    # push all bits of i onto res
```

```
    return res

concatenated_binary(1), concatenated_binary(3), concatenated_binary(12)   # -> (1, 27, 505379714)
```

6. **Divide Two Integers.** Signal: banned operators, a sign, and a 32-bit range. Technique: put the sign aside, then do long division in base 2 (shift and subtract). Key step: for `s` from 31 down to 0, if the divisor shifted left by `s` still fits in the rest of the dividend, subtract it and set bit `s` of the quotient. Each quotient bit is decided one time, so the loop has 32 steps. Apply the sign, then clamp. Only $-2^{31}/-1$ overflows. In C or Java, the magnitude 2^{31} does not fit, so work with negative values there. Hook: *sign aside, subtract the biggest shifted divisor, clamp at the end.*

```
INT_MAX, INT_MIN = 2**31 - 1, -2**31
def divide(dividend, divisor):
    negative = (dividend < 0) != (divisor < 0)   # put the sign aside
    a, b = abs(dividend), abs(divisor)
    q = 0
    for s in range(31, -1, -1):                  # quotient bits from high to low
        if (b << s) <= a:                        # the divisor times 2^s still fits
            a -= b << s
            q |= 1 << s
    q = -q if negative else q
    return max(INT_MIN, min(INT_MAX, q))         # only INT_MIN / -1 overflows

divide(10, 3), divide(7, -3), divide(-7, 2)      # -> (3, -2, -3)
divide(INT_MIN, -1), divide(INT_MIN, 1)          # -> (2147483647, -2147483648)
```

A.24 Design problems: combining data structures

Drills: Section 69.25 (page 427). Technique: Section 67 (page 400).

1. **Signal:** $O(1)$ insert, $O(1)$ delete of any value, and a uniform random pick. **Technique:** combine a list (a random index in $O(1)$) with a dict from each value to its index in the list. **Key step:** to remove a value, move the last value into its slot, update the index of the moved value, then pop the end. The list never has holes, so `random.choice` is uniform. Invariant: `vals[pos[v]] == v` for every stored value `v`. **Hook:** swap with the last, then pop.

```
import random

class RandomizedSet:
    def __init__(self):
        self.vals, self.pos = [], {}        # pos: value to its index in vals
    def insert(self, val):
        if val in self.pos:
            return False
        self.pos[val] = len(self.vals)
        self.vals.append(val)
        return True
    def remove(self, val):
        if val not in self.pos:
            return False
        i, last = self.pos[val], self.vals[-1]
        self.vals[i], self.pos[last] = last, i      # fill the hole with the last value
        self.vals.pop()
        del self.pos[val]
        return True
    def getRandom(self):
        return random.choice(self.vals)

rs = RandomizedSet()
rs.insert(1), rs.remove(2), rs.insert(2), rs.insert(2)   # -> (True, False, True, False)
rs.remove(1), rs.vals, rs.pos               # -> (True, [2], {2: 0})
```

2. **Signal:** the value to pop depends on its frequency, and ties go to the most recent push. **Technique:** a `Counter` of each value, plus one stack per count: `group[f]` holds the values that reached count `f`, in push order. Track `maxfreq`. **Key step:** a value with count 3 sits in groups 1, 2 and 3, so nothing moves between groups. `pop` takes the top of `group[maxfreq]`, lowers its count, and lowers `maxfreq` when that group is empty. **Hook:** one stack per count; pop from the highest count.

```
from collections import Counter, defaultdict

class FreqStack:
    def __init__(self):
        self.freq, self.group, self.maxfreq = Counter(), defaultdict(list), 0
    def push(self, val):
        self.freq[val] += 1
        f = self.freq[val]
        self.maxfreq = max(self.maxfreq, f)
        self.group[f].append(val)
    def pop(self):
        val = self.group[self.maxfreq].pop()
        self.freq[val] -= 1
        if not self.group[self.maxfreq]:
            self.maxfreq -= 1
        return val

fs = FreqStack()
for v in [5, 7, 5, 7, 4, 5]:
    fs.push(v)
[fs.pop() for _ in range(4)]                # -> [5, 7, 5, 4]
```

3. **Signal:** a history with a cursor that moves back and forward, and a new visit cuts off the forward part. **Technique:** one list of pages, a cursor index `cur`, and a logical end `end` (the index of the last valid page). **Key step:** `visit` moves `cur` to `cur + 1`, writes the URL there (append only at the physical end), and sets `end = cur`. The pages after `end` stay in the list, but they are dead, and the next visits overwrite them. `back` sets `cur = max(0, cur - steps)`, and `forward` sets `cur = min(end, cur + steps)`. Invariant: `pages[:end + 1]` is the history, and `0 <= cur <= end`. Two stacks (back and forward) also work, but each move then costs $O(\text{steps})$. **Hook:** one list, a cursor and a logical end; overwrite instead of delete.

```
class BrowserHistory:
    def __init__(self, homepage):
        self.pages, self.cur, self.end = [homepage], 0, 0
    def visit(self, url):
        self.cur += 1
        if self.cur == len(self.pages):
            self.pages.append(url)
        else:
            self.pages[self.cur] = url      # overwrite a dead page
        self.end = self.cur                 # forward history is gone
    def back(self, steps):
        self.cur = max(0, self.cur - steps)
        return self.pages[self.cur]
    def forward(self, steps):
        self.cur = min(self.end, self.cur + steps)
        return self.pages[self.cur]

bh = BrowserHistory("home")
for url in ["a", "b", "c"]:
    bh.visit(url)
bh.back(2), bh.forward(1)                   # -> ('a', 'b')
bh.back(1)
bh.visit("d")
bh.forward(5), bh.back(7), bh.forward(9)    # -> ('d', 'home', 'd')
# 'c' is a dead page after end
bh.pages, bh.end                            # -> (['home', 'a', 'd', 'c'], 2)
```

4. **Signal:** an LRU cache with one more rule: evict by use count first, and by recency only on a tie. **Technique:** a dict from key to value, a dict from key to count, a dict from each count to an `OrderedDict` of keys (oldest use first), and the smallest count `min_count`. **Key step:** a use moves the key from group `f` to the end of group `f + 1`. If group `f` becomes empty and `f == min_count`, set `min_count = f + 1`. For a new key in a full cache, evict first with `groups[min_count].popitem(last=False)`. Then add the new key with count 1 and set `min_count = 1`. Do not insert first: the new key can then be the only key with count 1, and it evicts itself. Put on an existing key updates the value and counts as a use, with no eviction. **Hook:** one LRU list per use count, plus the smallest count.
5. **Signal:** max and min over a set whose entries change in place. **Technique:** a dict from timestamp to price is the truth. Add a max-heap and a min-heap of (price, timestamp) with lazy deletion, and keep the latest timestamp. **Key step:** `update` pushes a new pair to both heaps and leaves the old pair there. A query pops each top whose price differs from `price[timestamp]`, because that pair is stale, then reads the top. Each pair is popped at most once, so the cost is $O(\log n)$ amortized. **Hook:** the dict is the truth; drop stale tops on read.
6. **Signal:** a tree codec where the BST order already gives the shape. **Technique:** write the preorder values only, with no null markers. Decode with value bounds. **Key step:** `build(low, high)` takes the next value only if `low < value < high`, else it returns `None`. The left call uses (`low`, value), and the right call uses (value, `high`). Each value is read once, so decoding is $O(n)$. The strict bounds assume distinct values, as in a standard BST. A skewed tree of 10^4 nodes needs a higher recursion limit or a stack-based build. **Hook:** preorder values; the bounds tell where each value goes.

B Problem map: NeetCode 150 and Blind 75

Each row gives the problem, the Part III sections that teach its technique, and a memory hook. The section names are the short names of the signal-to-technique table (Section 38). The number after each name is the section number. In the PDF, each section name is also a link to that section. DP stands for dynamic programming. Cover the right column, read the problem name, and recall the hook. Then check.

B.1 Arrays & Hashing

Problem	Sections	Memory hook
Contains Duplicate (217)	Hashing (44)	Ask the set if you saw it, then add it.
Valid Anagram (242)	Hashing (44)	Count both, compare counts.
Two Sum (1)	Hashing (44)	Ask the map for the partner, then add yourself.
Group Anagrams (49)	Hashing (44)	Sort each word, group by the sorted word.
Top K Frequent Elements (347)	Hashing (44)	Bucket values by count, read buckets top down, take the first k.
Encode and Decode Strings (271)	Hashing (44); Design problems (67)	Write length, hash, string; read length, then jump.
Product of Array Except Self (238)	Prefix sums (45)	Answer = left product times right product. In each pass, use the running product, then multiply in nums[i].
Valid Sudoku (36)	Hashing (44); Matrices and simulation (64)	Each digit checks in at its row, its column and its box (r // 3, c // 3). A second check-in at any of the three fails.
Longest Consecutive Sequence (128)	Hashing (44); Graph structure (56)	Only start counting where x-1 is missing.

B.2 Two Pointers

Problem	Sections	Memory hook
Valid Palindrome (125)	Two pointers (46)	Clean it, flip it, compare.
Two Sum II - Input Array Is Sorted (167)	Two pointers (46)	Sorted pair sum: start at both ends. Too small, move left up. Too big, move right down.
3Sum (15)	Two pointers (46)	Sort, fix one, squeeze two pointers, skip repeats.

Problem	Sections	Memory hook
Container With Most Water (11)	Two pointers (46)	Start widest, keep the best area, and move the shorter wall in.
Trapping Rain Water (42)	Prefix sums (45); Two pointers (46); Stacks and monotonic stacks (48)	Water at i is the lower wall minus the bar: min(left max, right max) - height.

B.3 Sliding Window

Problem	Sections	Memory hook
Best Time to Buy and Sell Stock (121)	Prefix sums (45); Sliding window (47)	Remember the cheapest day, sell today.
Longest Substring Without Repeating Characters (3)	Sliding window (47)	Drop from the left until the new character is unique, then add it on the right.
Longest Repeating Character Replacement (424)	Sliding window (47)	Window length minus top count must be at most k.
Permutation in String (567)	Sliding window (47)	Slide a window of len(s1) over s2 and compare its letter counts to s1's counts.
Minimum Window Substring (76)	Sliding window (47)	Grow right until have equals required, then shrink left while it holds.
Sliding Window Maximum (239)	Sliding window (47)	Kick out the old from the front, kick out the weak from the back, and the front is the max.

B.4 Stack

Problem	Sections	Memory hook
Valid Parentheses (20)	Stacks and monotonic stacks (48)	Push openers. Each closer must pop its own opener off the top. End with an empty stack.
Min Stack (155)	Stacks and monotonic stacks (48); Design problems (67)	Push the pair (value, min so far). Pop removes both.
Evaluate Reverse Polish Notation (150)	Stacks and monotonic stacks (48)	Push numbers. An operator pops b, then a, and pushes a op b. Divide with int(a / b).
Generate Parentheses (22)	Stacks and monotonic stacks (48); Backtracking (54)	Open while opens remain. Close only what is open. Save at length 2n.
Daily Temperatures (739)	Stacks and monotonic stacks (48)	Today pops every colder waiting day and writes the gap.
Car Fleet (853)	Stacks and monotonic stacks (48)	Front car first: a car that arrives later than the fleet ahead starts a new fleet.
Largest Rectangle in Histogram (84)	Stacks and monotonic stacks (48)	Each bar is the shortest: stretch it to the nearest shorter bar on both sides.

B.5 Binary Search

Problem	Sections	Memory hook
Binary Search (704)	Binary search (49)	Closed range, check the middle, drop the half that cannot hold target.
Search a 2D Matrix (74)	Binary search (49)	The grid is one sorted list cut into rows, so binary search the flat index and divmod by cols.
Koko Eating Bananas (875)	Binary search (49)	Guess a speed, count the hours, and binary search for the first speed that fits.

Problem	Sections	Memory hook
Find Minimum in Rotated Sorted Array (153)	Binary search (49)	Low run means at most the last value; find the first one.
Search in Rotated Sorted Array (33)	Binary search (49)	Find the drop, then binary search the side that can hold target.
Time Based Key-Value Store (981)	Binary search (49); Design problems (67)	Two lists per key: times and values. bisect_right counts times <= t, so take the value one left.
Median of Two Sorted Arrays (4)	Binary search (49)	Cut the shorter array so the left half is full: too many from A, go left; too few, go right; just right, read the borders.

B.6 Linked List

Problem	Sections	Memory hook
Reverse Linked List (206)	Linked lists (50)	Save next, flip the arrow, step both, return prev.
Merge Two Sorted Lists (21)	Two pointers (46); Linked lists (50)	Dummy and tail. Take the smaller head, move the tail. Glue the rest.
Reorder List (143)	Two pointers (46); Linked lists (50)	Put the nodes in an array, take front then back, chain them, end with None.
Remove Nth Node From End of List (19)	Two pointers (46); Linked lists (50)	Line up the nodes behind a dummy, then nodes[-n - 1] skips its next.
Copy List with Random Pointer (138)	Linked lists (50)	Make every copy first, then link copies through the old-to-new map.
Add Two Numbers (2)	Linked lists (50); Math and number manipulation (65)	Add digits plus carry while anything is left; write mod 10, carry div 10.
Linked List Cycle (141)	Two pointers (46); Linked lists (50)	Hare runs two, tortoise runs one. If they meet, it loops. If the hare falls off the end, it does not.
Find the Duplicate Number (287)	Two pointers (46); Linked lists (50)	Index points to value: tortoise and hare from 0, then reset one to 0, walk together, and meet at the duplicate.
LRU Cache (146)	Linked lists (50); Design problems (67)	Ordered dict: move to end on use, pop the front on overflow.
Merge k Sorted Lists (23)	Linked lists (50); Heaps and priority queues (53)	Heap the fronts; pop the smallest, push its next.
Reverse Nodes in k-Group (25)	Linked lists (50)	Before, first, last, after: probe k, reverse with prev = after, link before to last, step to first.

B.7 Trees

Problem	Sections	Memory hook
Invert Binary Tree (226)	Trees (51)	Swap the two children, then invert each child.
Maximum Depth of Binary Tree (104)	Trees (51)	Depth is 1 plus the deeper child; for a very deep tree, count the BFS levels instead.
Diameter of Binary Tree (543)	Trees (51)	At each node, record left height + right height as a candidate for best, and pass 1 + the bigger height up. Deep tree: list parents first, then walk backward.
Balanced Binary Tree (110)	Trees (51)	List parents first, walk backwards, compare child heights.
Same Tree (100)	Trees (51)	Both null is true, one null or different values is false, else check left pair and right pair.
Subtree of Another Tree (572)	Trees (51)	Try every node as a start, and ask Same Tree at each one. Both loops pop, skip the empty case, check, and push the children.
Lowest Common Ancestor of a Binary Search Tree (235)	Trees (51)	Both smaller go left, both bigger go right, else this node is the split.
Binary Tree Level Order Traversal (102)	Trees (51)	Take the whole level, save its values, swap in its children.

Problem	Sections	Memory hook
Binary Tree Right Side View (199)	Trees (51)	Level-order BFS; the last node of each level is the one you see.
Count Good Nodes in Binary Tree (1448)	Trees (51)	Carry the path max down; count a node when it is at least the max.
Validate Binary Search Tree (98)	Trees (51)	Every node lives in a window: going left lowers the ceiling, going right raises the floor.
Kth Smallest Element in a BST (230)	Trees (51)	Push the left spine; pop, count, push the right child's left spine.
Construct Binary Tree from Preorder and Inorder Traversal (105)	Trees (51)	Preorder names the root. Inorder splits at the root. Slice both lists and push each side as a job.
Binary Tree Maximum Path Sum (124)	Trees (51)	List parents first, then walk back. At each node, cut negative sides to 0, record left plus me plus right, and store me plus the better side.
Serialize and Deserialize Binary Tree (297)	Trees (51); Design problems (67)	Write it level by level like the example list, with N for every null; each real node in the queue takes the next two tokens as its kids.

B.8 Tries

Problem	Sections	Memory hook
Implement Trie (Prefix Tree) (208)	Tries (52); Design problems (67)	One node per prefix: walk the letters, flag the word end.
Design Add and Search Words Data Structure (211)	Tries (52); Design problems (67)	Mask one letter on add, fill one dot on search.
Word Search II (212)	Tries (52); Backtracking (54)	Word Search I, but walk a trie instead of one word, and cut empty leaves on the way back.

B.9 Heap / Priority Queue

Problem	Sections	Memory hook
Kth Largest Element in a Stream (703)	Heaps and priority queues (53); Design problems (67)	Keep the top k in a min-heap. Every value goes through add: push, pop if over k, return the root.
Last Stone Weight (1046)	Heaps and priority queues (53)	Negate into a heap, smash the top two, push back only a nonzero difference.
K Closest Points to Origin (973)	Heaps and priority queues (53)	Sort by x*x + y*y, keep the first k.
Kth Largest Element in an Array (215)	Heaps and priority queues (53)	Top k in a min-heap: push, trim to k, return the root.
Task Scheduler (621)	Heaps and priority queues (53); Greedy (62)	Fill each window of n + 1 with the most frequent different tasks. Only the last window has no idle tail.
Design Twitter (355)	Heaps and priority queues (53); Design problems (67)	Stamp each tweet with a global clock. Merge the timelines with a heap: one cursor per user, pop the newest, push that user's previous tweet, stop at ten.
Find Median from Data Stream (295)	Heaps and priority queues (53); Design problems (67)	Push into small, pass its top to large, give one back if large gets bigger.

B.10 Backtracking

Problem	Sections	Memory hook
Subsets (78)	Backtracking (54)	Each number doubles the list. Keep the old subsets and add each one plus the number.
Combination Sum (39)	Backtracking (54)	Sort first. In dfs, save a copy at zero, then loop j from start: break if too big, pick, recurse with the same j, pop.
Permutations (46)	Backtracking (54)	Pick one from the pool. Recurse on the rest. An empty pool means a finished permutation.
Subsets II (90)	Backtracking (54)	Count each value. Every old subset takes 0 to count copies of it.
Combination Sum II (40)	Backtracking (54)	Sort, then each level tries each value once.
Word Search (79)	Backtracking (54)	Check the cell, mark it, try four neighbors, unmark it. Speed-up: fill can-finish cells from the back first.
Palindrome Partitioning (131)	Backtracking (54)	Cut a palindrome off the front, recurse on the rest.
Letter Combinations of a Phone Number (17)	Backtracking (54)	Start with one empty string, and grow every prefix by every letter of the next digit.
N-Queens (51)	Backtracking (54)	One queen per row. Block the column, r minus c and r plus c.

B.11 Graphs

Problem	Sections	Memory hook
Number of Islands (200)	Graph traversal (55)	Count each new land cell, then sink its whole island.
Clone Graph (133)	Graph traversal (55)	Copy a tree, but check the map first and register the copy before you recurse.
Max Area of Island (695)	Graph traversal (55)	Scan for land. Sink on push, count on pop, keep the max.
Pacific Atlantic Water Flow (417)	Graph traversal (55)	Water flows down, so flood uphill from each ocean and keep the cells both floods reach.
Surrounded Regions (130)	Graph traversal (55)	Flood from the border, O becomes T; then O to X, T to O.
Rotting Oranges (994)	Graph traversal (55)	All rotten in the queue, count the fresh, one level is one minute, stop when fresh is zero.
Walls and Gates (286)	Graph traversal (55)	Push all gates first, then BFS and fill only INF rooms.
Course Schedule (207)	Graph structure (56)	Take what is ready, free its dependents, check that you took everything.
Course Schedule II (210)	Graph structure (56)	Count unmet prerequisites. Take any course at zero. A short order means a cycle.
Redundant Connection (684)	Graph structure (56)	Same root already? That edge closes the cycle.
Number of Connected Components in an Undirected Graph (323)	Graph structure (56)	Count how many times you must start a new flood.
Graph Valid Tree (261)	Graph structure (56)	A tree has n - 1 edges and reaches everyone.
Word Ladder (127)	Graph traversal (55)	BFS of (word, length) pairs: change one letter, keep the word only if it is still in the unvisited set.

B.12 Advanced Graphs

Problem	Sections	Memory hook
Reconstruct Itinerary (332)	Weighted and advanced graphs (57)	Take the smallest ticket, write the airport down when stuck, then reverse.
Min Cost to Connect All Points (1584)	Weighted and advanced graphs (57)	Keep a dict of outside points: pop the nearest, then let it lower the others.
Network Delay Time (743)	Weighted and advanced graphs (57)	The first pop of a node is its arrival time.
Swim in Rising Water (778)	Weighted and advanced graphs (57)	Dijkstra, but the path cost is the highest cell, so use max instead of plus.
Alien Dictionary (269)	Graph structure (56)	Compare neighbors: a longer word before its prefix fails, else the first difference gives one edge. Kahn gives the order. A short order means a cycle.

Problem	Sections	Memory hook
Cheapest Flights Within K Stops (787)	Weighted and advanced graphs (57)	k + 1 rounds: relax every flight from last round's copy.

B.13 1-D Dynamic Programming

Problem	Sections	Memory hook
Climbing Stairs (70)	DP: foundations and 1-D (58)	The last move is a 1-step or a 2-step, so add the ways for the two steps below.
Min Cost Climbing Stairs (746)	DP: foundations and 1-D (58)	To stand on step i, come from one back or two back, and pay the step you leave.
House Robber (198)	DP: foundations and 1-D (58)	Skip it and keep the last best, or rob it and add its money to the best two back.
House Robber II (213)	DP: foundations and 1-D (58)	Drop the first or drop the last, rob each row, take the max.
Longest Palindromic Substring (5)	DP on strings and sequences (60)	Every palindrome grows from a center: try each character and each gap, expand while the ends match, keep the longest.
Palindromic Substrings (647)	DP on strings and sequences (60)	Every palindrome has a middle: expand from each middle, odd and even, and count each match.
Decode Ways (91)	DP: foundations and 1-D (58)	Climbing Stairs with guards: one step if the digit is not 0, two steps if the pair is 10 to 26.
Coin Change (322)	DP: knapsack family (59)	Pick the last coin: dp[a] = 1 + min of dp[a - coin], with dp[0] = 0.
Maximum Product Subarray (152)	DP: foundations and 1-D (58)	Kadane, but carry the biggest and the smallest, because a negative swaps them.
Word Break (139)	DP: foundations and 1-D (58); DP: knapsack family (59)	Cut off a first word, ask if the rest can split, and cache by start index.
Longest Increasing Subsequence (300)	Binary search (49); DP: foundations and 1-D (58)	Smallest tail per length: bisect_left, then replace or append.
Partition Equal Subset Sum (416)	DP: knapsack family (59)	Odd total means no. Otherwise grow the set of reachable sums and ask if half the total is in it.

B.14 2-D Dynamic Programming

Problem	Sections	Memory hook
Unique Paths (62)	DP on grids, intervals, games and states (61)	Fill the grid with ones, then each cell equals up plus left.
Longest Common Subsequence (1143)	DP on strings and sequences (60)	Last chars match: go diagonal plus one. Else take the best of up and left.
Best Time to Buy and Sell Stock with Cooldown (309)	DP on grids, intervals, games and states (61)	Walk the days backward. Free waits or buys, holding waits or sells, and a sale skips tomorrow.
Coin Change II (518)	DP: knapsack family (59)	Coins outside, amounts going up inside: ways[a] += ways[a - coin].
Target Sum (494)	DP: knapsack family (59)	Plus or minus each number, cache on (index, running sum).
Interleaving String (97)	DP on strings and sequences (60)	Take from s1 or s2 if it matches s3[i + j]; cache (i, j).
Longest Increasing Path in a Matrix (329)	DP on grids, intervals, games and states (61)	Biggest first: each cell is 1 plus its best bigger neighbor.
Distinct Subsequences (115)	DP on strings and sequences (60)	Ways equal skip plus use: dp[i][j] = dp[i-1][j], plus dp[i-1][j-1] on a match.
Edit Distance (72)	DP on strings and sequences (60)	Look at the last chars: same is free diagonal, else 1 plus min of up, left, diagonal.

Problem	Sections	Memory hook
Burst Balloons (312)	DP on grids, intervals, games and states (61)	Pick the balloon that pops last. It sees only the two walls.
Regular Expression Matching (10)	DP on strings and sequences (60)	Peek for a star: skip it, or eat one and stay.

B.15 Greedy

Problem	Sections	Memory hook
Maximum Subarray (53)	DP: foundations and 1-D (58); Greedy (62)	A negative running sum only hurts, so drop it and restart.
Jump Game (55)	Greedy (62)	Walk forward; if you pass farthest you are stuck, else stretch farthest.
Jump Game II (45)	Greedy (62)	BFS in windows: the next window runs from right + 1 to the farthest reach of this window.
Gas Station (134)	Greedy (62)	Not enough total gas means -1; out of gas means start after here.
Hand of Straights (846)	Greedy (62)	Smallest card left starts a straight. Pull its next cards one by one.
Merge Triplets to Form Target Triplet (1899)	Greedy (62)	Drop any triplet that overshoots, max the rest, compare with target.
Partition Labels (763)	Greedy (62)	Each letter drags the cut to its last spot; cut when you catch up.
Valid Parenthesis String (678)	Greedy (62)	Range of open counts: fail if the most goes below zero, clamp the fewest at zero, and end with fewest == 0.

B.16 Intervals

Problem	Sections	Memory hook
Insert Interval (57)	Intervals (63)	Left stays left, right stays right, the middle melts into one.
Merge Intervals (56)	Intervals (63)	Sort, then extend the last one or append a new one.
Non-overlapping Intervals (435)	Intervals (63)	Sort by end, keep what fits, remove the rest.
Meeting Rooms (252)	Intervals (63)	Sort by start. No meeting may start before the previous one ends.
Meeting Rooms II (253)	Heaps and priority queues (53); Intervals (63)	Start is +1, end is -1, sort the events, and keep the peak.
Minimum Interval to Include Each Query (1851)	Heaps and priority queues (53); Intervals (63)	Sweep sorted queries, push started intervals by size, pop ended tops, read the top.

B.17 Math & Geometry

Problem	Sections	Memory hook
Rotate Image (48)	Matrices and simulation (64)	Flip on the diagonal, then flip each row.
Spiral Matrix (54)	Matrices and simulation (64)	Walk straight, turn right when blocked.
Set Matrix Zeroes (73)	Matrices and simulation (64)	Mark zero rows and zero columns first, then clear.
Happy Number (202)	Math and number manipulation (65)	Walk until you hit 1 or a repeat. A seen set catches the repeat.

Problem	Sections	Memory hook
Plus One (66)	Math and number manipulation (65)	From the right, each 9 becomes 0 until a digit can take the +1. If all digits are 9, put a 1 in front.
Pow(x, n) (50)	Math and number manipulation (65)	Square the half, add one x if odd.
Multiply Strings (43)	Math and number manipulation (65)	Reverse so index is the power, drop a[i] * b[j] into slot i + j, then carry once.
Detect Squares (2013)	Hashing (44); Math and number manipulation (65); Design problems (67)	Pick the diagonal, the other two corners are forced, multiply the counts.

B.18 Bit Manipulation

Problem	Sections	Memory hook
Single Number (136)	Bit manipulation (66)	Pairs cancel under XOR, so XOR everything.
Number of 1 Bits (191)	Bit manipulation (66)	Read the last bit, then shift it away.
Counting Bits (338)	Bit manipulation (66)	Drop the last bit, look it up, add the bit back.
Reverse Bits (190)	Bit manipulation (66)	Peel the low bit off n, push it onto res, 32 times.
Missing Number (268)	Math and number manipulation (65); Bit manipulation (66)	Expected sum minus actual sum.
Sum of Two Integers (371)	Bit manipulation (66)	XOR is the sum, AND shifted left is the carry. Repeat until no carry, mask to 32 bits, and fix the sign with ~(a ^ MASK).
Reverse Integer (7)	Math and number manipulation (65); Bit manipulation (66)	Put the sign aside. Pop a digit, solve res * 10 + digit <= INT_MAX for res, and push only if it holds.

B.19 Blind 75 extra

Problem	Sections	Memory hook
Combination Sum IV (377)	DP: knapsack family (59)	Ways to make a total: try every last number, add the ways for what is left.